Designing Data-Intensive Applications

Martin Kleppmann: beginner-friendly study notes

14 parts · every chapter explained in plain English, with redrawn figures

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes written from the book. Figure numbers refer to the book. Not for redistribution.

Contents

1. **Preface — How to Read This Book**
   - What this book is about
   - What kind of book it is (and is not)
   - Who should read it
   - Scope: what is covered and what is deliberately left out
   - Map of the book
   - How to use these notes
2. **Chapter 1 — Reliable, Scalable, and Maintainable Applications**
   - Overview
   - Key terms
   - Thinking About Data Systems
   - Reliability
   - Scalability
   - Maintainability
   - Summary
   - Interview angle
   - Self-check
3. **Chapter 2 — Data Models and Query Languages**
   - Overview
   - Key terms
   - Relational Model Versus Document Model
   - Query Languages for Data
   - Graph-Like Data Models
   - Summary
   - Interview angle
   - Self-check
4. **Chapter 3 — Storage and Retrieval**
   - Overview
   - Key terms
   - Data Structures That Power Your Database
   - Transaction Processing or Analytics?
   - Column-Oriented Storage
   - Summary
   - Chapter summary
   - Interview angle
   - Self-check
5. **Chapter 4 — Encoding and Evolution**
   - Overview
   - Key terms
   - Formats for Encoding Data
   - Modes of Dataflow
   - Summary
   - Looking ahead
   - Chapter summary
   - Interview angle
   - Self-check
6. **Chapter 5 — Replication**
   - Overview
   - Key terms
   - Leaders and Followers
   - Problems with Replication Lag
   - Multi-Leader Replication
   - Leaderless Replication
   - Summary
   - Interview angle
   - Self-check
7. **Chapter 6 — Partitioning**
   - Overview
   - Key terms
   - Partitioning and Replication
   - Partitioning of Key-Value Data
   - Partitioning and Secondary Indexes
   - Rebalancing Partitions
   - Request Routing
   - Summary
   - Interview angle
   - Self-check
8. **Chapter 7 — Transactions**
   - Overview
   - Key terms
   - The Slippery Concept of a Transaction
   - Weak Isolation Levels
   - Serializability
   - Chapter summary
   - Interview angle
   - Self-check
9. **Chapter 8 — The Trouble with Distributed Systems**
   - Overview
   - Key terms
   - Faults and Partial Failures
   - Unreliable Networks
   - Unreliable Clocks
   - Knowledge, Truth, and Lies

    - Chapter summary
    - Interview angle
    - Self-check
10. **Chapter 9 — Consistency and Consensus**
    - Overview
    - Key terms
    - Consistency Guarantees
    - Linearizability
    - Ordering Guarantees
    - Distributed Transactions and Consensus
    - Summary
    - Chapter summary
    - Interview angle
    - Self-check
11. **Chapter 10 — Batch Processing**
    - Overview
    - Key terms
    - Batch Processing with Unix Tools
    - MapReduce and Distributed Filesystems
    - Beyond MapReduce
    - Chapter summary
    - Interview angle
    - Self-check
12. **Chapter 11 — Stream Processing**
    - Overview
    - Key terms
    - Transmitting Event Streams
    - Databases and Streams
    - Processing Streams
    - Chapter summary
    - Interview angle
    - Self-check
13. **Chapter 12 — The Future of Data Systems**
    - Overview
    - Key terms
    - Data Integration
    - Unbundling Databases
    - Aiming for Correctness
    - Doing the Right Thing
    - Chapter summary
    - Interview angle
    - Self-check
14. **Glossary**

Preface — How to Read This Book

Book: Preface, pp. xiii–xix

What this book is about

Anyone who has spent time around backend engineering has heard a stream of buzzwords shouted with exclamation marks: NoSQL, Big Data, web-scale, sharding, ACID, eventual consistency, CAP theorem, MapReduce, cloud services, real-time. The preface opens by admitting that this noise is real, and then promises something better: a way to see the lasting ideas that sit underneath all of these fashionable labels.

The author points to several forces that pushed databases and distributed systems to change so much over the preceding decade:

- **Huge internet companies** (Google, Yahoo!, Amazon, Facebook, LinkedIn, Microsoft, Twitter) had so much data and traffic that they had to invent new tools just to cope.
- **Businesses want agility**: short development cycles, cheap experiments, and data models that can bend as the market changes.
- **Open source won**: free and open source software is now often chosen over commercial or home-grown products.
- **Hardware shifted**: single CPU cores are barely getting faster, but machines have many cores and networks keep speeding up, so parallelism will only grow.'
- **Distribution became cheap**: because of infrastructure as a service (IaaS), for example Amazon Web Services, even a tiny team can run software spread over many machines and several geographic regions.
- **Downtime is less acceptable**: users now expect services to be highly available, so long outages (planned or not) are increasingly unacceptable.

> **Definition: data-intensive vs compute-intensive.** An application is *data-intensive* when its hardest problem is the data itself: how much of it there is, how complicated it is, or how fast it changes. It is *compute-intensive* when the bottleneck is raw CPU work. This whole book is about the first kind.

Data-intensive applications are rarely built on one tool. Beyond databases (including the newer "NoSQL" ones), they lean on message queues, caches, search indexes, and batch and stream processing frameworks, usually several of them at once.

The book's central promise is this: tools change quickly, but the **principles and trade-offs** behind them change slowly. If you understand those principles, you can look at any new product and work out where it fits, how to use it well, and where it will hurt you.

What kind of book it is (and is not)

- It is **not a tutorial** for one product and **not a dry theory textbook**.
- It studies **real, successful data systems** that run in production every day under demanding requirements for scale, performance and reliability.
- It opens up their **internals and key algorithms** and asks *why* they are built the way they are, not only *how* they work.

After reading it you should be able to pick the right technology for a job and combine several tools into a sound architecture. You will **not** be ready to write your own storage engine from scratch (the

author notes this is seldom needed), but you will have a solid mental model of what your systems do under the hood, which helps with design decisions and debugging.

Who should read it

The target reader builds software that has a server or backend which stores or processes data and talks over the internet, such as web apps, mobile apps, or connected sensors. That includes software engineers, architects, and technical managers who still enjoy code, and it is most useful to anyone who chooses tools and designs architectures. Even if you cannot choose your tools, knowing their strengths and weak spots helps.

Assumed background:

You should know	Nice to have	Does not matter
Building web apps or network services; relational databases and SQL	Non-relational databases and other data tools; a general feel for TCP and HTTP	Your programming language or framework

The book is especially valuable if you want to:

- make a system scale (for example, to millions of web or mobile users);
- keep an application highly available (little downtime) and robust in operation;
- keep a system maintainable over years as it grows and requirements shift;
- simply satisfy your curiosity about what happens inside large websites and online services.

> **"You're not Google or Amazon, so forget scale and use a relational database."** The author agrees there is truth here: engineering for scale you do not need is wasted effort, a kind of premature optimization, and can lock you into a rigid design. But choosing the right tool still matters, every technology has strengths and weaknesses, and relational databases, although important, are not the last word on handling data.

Scope: what is covered and what is deliberately left out

Covered: the architecture of data systems and how they are woven into data-intensive applications; the fundamental principles and trade-offs; and the different design choices made by different products.

Deliberately left out:

- Step-by-step installation or usage guides for specific packages or APIs (their own documentation already does that).
- Deployment, operations, security, and management. The author considers these too important to squeeze in as footnotes; they deserve dedicated books.
- The term "Big Data" as an organising idea. It is judged too vague for serious engineering talk. Instead the book uses sharper contrasts such as **single-node vs distributed** systems and **online/interactive vs offline/batch** processing.

Bias toward open source. The book favours free and open source software (FOSS) because you can read, change and run the code to see exactly how it works, and open platforms reduce vendor lock-in. Proprietary systems (closed source, software as a service, or in-house systems known only from papers) are still discussed where relevant.

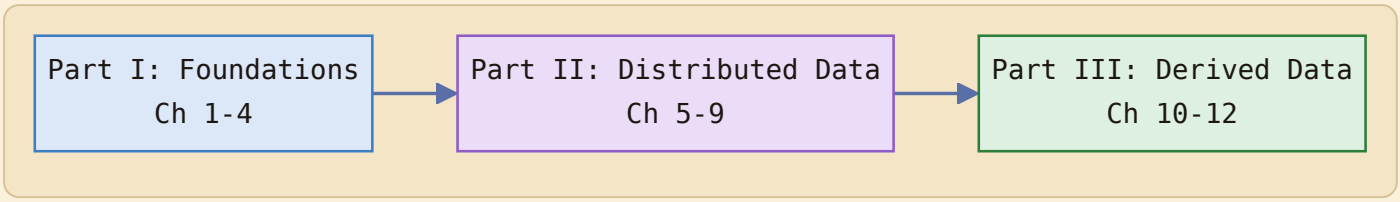
References. Almost everything in the book has appeared somewhere before, in conference talks, research papers, blog posts, source code, bug trackers, mailing lists, and the folklore engineers

pass around. The book gathers the most important of these ideas and cites the originals; each chapter ends with a reference list (the book has 800+ references), and most of it is freely available online. Up-to-date links are kept at `github.com/ept/ddia-references`; for papers, searching the title in Google Scholar usually finds an open-access PDF.

Acknowledgments (in brief). The author describes the book as a systematic gathering of many other people's ideas from both research and industry, and argues that engineers can learn a great deal from older work instead of chasing only what is new. It took almost four years to write, and the maps at the start of each chapter were illustrated by Shabbir Diwan and Edie Freedman. (The preface also has publisher contact details and an O'Reilly Safari notice, which are not relevant to the content.)

Map of the book

The book has three Parts. Part I builds single-machine foundations, Part II spreads data over many machines, and Part III connects many systems together through derived data.



Part I — Foundations of Data Systems

Ideas that apply to every data system, whether it runs on one machine or a cluster.

Ch	Title	One-line summary
1	Reliable, Scalable, and Maintainable Applications	What these three goals really mean, how to describe load and performance, and how to reason about them.
2	Data Models and Query Languages	Relational vs document vs graph models, and their query languages: declarative ones (SQL, Cypher, SPARQL, Datalog) versus MapReduce, which sits between declarative and imperative.
3	Storage and Retrieval	How storage engines lay data out on disk: hash indexes, LSM-trees, B-trees, and column-oriented storage for analytics.
4	Encoding and Evolution	Formats for turning data into bytes (JSON, Thrift, Protocol Buffers, Avro) and how schemas change safely over time.

Part II — Distributed Data

Moving from one machine to many, which scalability often requires, and the unusual problems that follow.

Ch	Title	One-line summary
5	Replication	Keeping copies of data on several machines: leaders and followers, multi-leader, leaderless quorums, replication lag.
6	Partitioning	Splitting a large dataset into shards: by key range or hash, secondary indexes, rebalancing, request routing.
7	Transactions	What ACID means, weak isolation levels and their anomalies, and ways to get serializability (serial execution, 2PL, SSI).
8	The Trouble with Distributed Systems	Partial failures, unreliable networks, unreliable clocks, and how to reason about truth when nodes can be wrong.
9	Consistency and Consensus	Linearizability, ordering guarantees, two-phase commit, and fault-tolerant consensus and coordination services.

Part III — Derived Data

Systems in which one dataset is computed from another. This is common in heterogeneous architectures where no single database does everything well, so an application combines databases, caches, indexes and so on.

Ch	Title	One-line summary
10	Batch Processing	From Unix pipelines to MapReduce and beyond: joins, workflows, and dataflow engines over large fixed datasets.
11	Stream Processing	Event streams, message brokers and partitioned logs, change data capture, event sourcing, and processing unbounded data.
12	The Future of Data Systems	Putting it all together: data integration, "unbundling" databases, correctness end to end, and doing the right thing ethically.

The book finishes with a **Glossary** of key terms (p. 553).

How to use these notes

1. Read Chapter 1 first; its vocabulary (faults, load parameters, percentiles, maintainability) is used everywhere later.
2. Treat Part I as prerequisites for Part II, and Part II as prerequisites for Part III. Chapters inside a Part build on each other too.
3. Each chapter README follows the same layout: overview, key terms, the book's sections in order, a summary, an interview angle, and a self-check. Try the self-check before rereading.
4. When a new tool appears in the wild, ask the book's favourite questions: what model does it use, what guarantees does it give, and what does it give up to get them?

Chapter 1 — Reliable, Scalable, and Maintainable Applications

Book: Chapter 1, pp. 3–26

Overview

Part I in brief (Foundations of Data Systems). Chapters 1 to 4 cover ideas that hold for every data system, whether it lives on one machine or across a cluster:

1. **Chapter 1** (this one) sets up vocabulary and a way of thinking: what *reliability*, *scalability* and *maintainability* actually mean, and how to aim for them.
2. **Chapter 2** compares data models and query languages, which is the most visible difference between databases from a developer's seat.
3. **Chapter 3** opens up storage engines and how they arrange data on disk; different engines suit different workloads, and the choice can dominate performance.
4. **Chapter 4** looks at encoding (serialization) formats and how well they cope when requirements change and schemas must evolve.

Part II then moves on to problems specific to distributed data.

What question does Chapter 1 answer? "When we say a data system should be *good*, what exactly do we mean, and how do we talk about it precisely?" The chapter picks three goals and gives each one concrete, measurable meaning:

- **Reliability**: keep working correctly even when things break.
- **Scalability**: have sensible options when load grows.
- **Maintainability**: let many people keep operating and changing the system productively for years.

Why it matters. Most modern apps are *data-intensive*: their pain comes from the volume, complexity, or rate of change of data, not from CPU power. They are assembled from standard parts:

Need	Building block
Keep data so it can be found later	Database
Keep the answer to a costly computation so later reads are fast	Cache
Let users search by keyword or filter	Search index
Hand a message to another process to handle asynchronously	Stream processing
Periodically crunch a big pile of accumulated data	Batch processing

These building blocks are so successful that we barely notice them. But there are many databases, many caching styles, many ways to build search indexes, and so on, each suited to different needs. Picking and combining them well is the real skill.

Everyday analogy. Think of running a restaurant. *Reliability* is still serving good meals when an oven breaks or a cook drops a tray. *Scalability* is having a plan when you go from 50 to 500 diners a night. *Maintainability* is a kitchen organised so a new cook can find things, the manager can see what is happening, and the menu can change without rebuilding the kitchen.

Key terms

Term	Plain-English meaning
Data-intensive application	An app whose main difficulty is data (amount, complexity, change speed), not CPU.
Compute-intensive application	An app whose bottleneck is CPU cycles.
Data system	Umbrella name for databases, queues, caches, search indexes, etc., which increasingly overlap.
Reliability	The system keeps doing the right thing at the needed performance despite faults.
Fault	One component deviating from its specification (a disk dies, a process crashes).
Failure	The system as a whole stops giving users the required service.
Fault-tolerant / resilient	Able to anticipate and cope with certain kinds of faults.
Chaos Monkey	Netflix tool that deliberately kills processes to exercise fault-handling.
MTTF (mean time to failure)	Average time before a component fails; for hard disks about 10–50 years.
RAID	Arrangement of several disks so one can fail without data loss.
Rolling upgrade	Patching or upgrading one node at a time so the whole system never goes down.
Systematic error	A fault correlated across many nodes, such as a shared software bug.
Cascading failure	A small fault in one part triggers faults in others, spreading outward.
Telemetry	Detailed monitoring data (metrics, error rates) about a running system.
Scalability	The ability to cope with increased load; discussed as "what are our options if load grows like X?"
Load parameters	A few numbers that describe current load (requests/sec, read/write ratio, active users, cache hit rate...).
Fan-out	How many further requests (or writes) one incoming request triggers.
Throughput	How many records are processed per second, or how long a job takes on a dataset of some size.
Response time	What the client sees: time from sending a request to getting the reply.
Latency	Time a request spends waiting to be handled (latent). Not the same as response time.
Service time	The time actually spent processing the request.
Percentile (p50, p95, p99, p999)	The value below which that percentage of measurements fall.
Median	The 50th percentile: half of requests are faster, half slower.
Tail latency	The high percentiles of response time (the slow outliers).
SLO / SLA	Service level objective / agreement: stated targets or contracts for performance and availability.
Head-of-line blocking	A few slow requests at the front of a queue delay the fast ones behind them.

Term	Plain-English meaning
Tail latency amplification	When one user request needs many backend calls, the chance that at least one is slow grows.
Scaling up (vertical)	Moving to a bigger, more powerful machine.
Scaling out (horizontal)	Spreading load across many smaller machines.
Shared-nothing architecture	Another name for scaling out: machines do not share memory or disk.
Elastic system	Automatically adds resources when it detects rising load.
Stateless / stateful service	Keeps no durable data of its own / owns data that must be preserved.
Maintainability	How easily people can keep the system running and change it over time.
Operability	How easy it is for operations teams to keep things running smoothly.
Simplicity	How easy it is for new engineers to understand the system.
Evolvability	How easy it is to adapt the system to new requirements (also extensibility, modifiability, plasticity).
Big ball of mud	A software project drowning in complexity.
Accidental complexity	Complexity caused by the implementation rather than by the problem users need solved.
Abstraction	A clean interface that hides lots of implementation detail and can be reused.
Functional vs nonfunctional requirements	What the system must do vs general properties it must have (security, reliability, scalability...).

Thinking About Data Systems

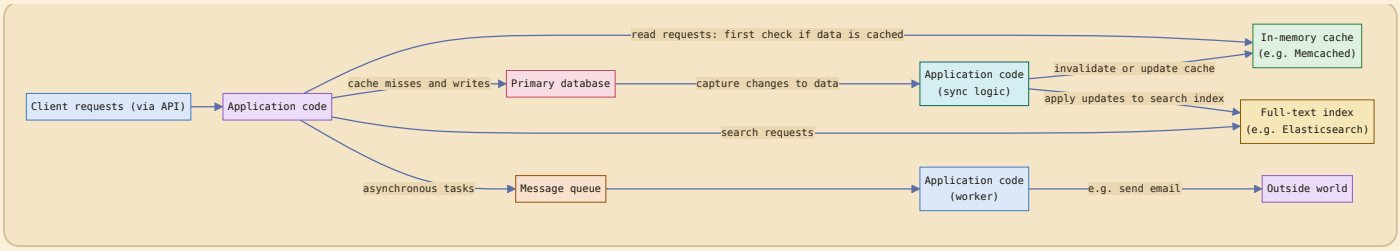
What it is. The book lumps databases, queues, caches and similar tools together as **data systems**. At first this seems odd: a database and a message queue both hold data for a while, but they are accessed very differently, so they perform differently and are built differently.

Why group them anyway? Two reasons:

1. **The categories are blurring.** Newer tools target a mix of use cases. Redis is a datastore that people also use as a message queue; Apache Kafka is a message queue with durability guarantees similar to a database.
2. **One tool is rarely enough.** Many applications have such wide-ranging needs that the work is split into tasks, each handled well by a specific tool, and *your application code* stitches the tools together.

Example. Suppose your app has its own caching layer (for example Memcached) and a separate full-text search server (Elasticsearch or Solr) next to the main database. Usually it is the application's job to keep the cache and the search index in step with the database.

Figure 1-1 (redrawn): one possible architecture combining several data components behind an application.



Clients only see the service's API (application programming interface). Behind it, you have effectively built a **new, special-purpose data system** out of general-purpose parts, and it may promise things like "the cache is correctly invalidated or updated on every write so clients see consistent results." So you are now a **data system designer**, not just an app developer.

Questions a designer must face: How do we keep data correct and complete when parts break? How do we give steady performance when some parts are degraded? How do we handle growing load? What should the API look like?

Many situational factors also shape a design: team skills, legacy dependencies, delivery deadline, the organisation's appetite for risk, regulations. The book focuses on three concerns that matter for nearly all software:

Concern	Short definition
Reliability	Keeps working correctly (right function, acceptable performance) despite adversity: hardware faults, software faults, even human mistakes.
Scalability	As data volume, traffic, or complexity grows, there are reasonable ways to cope.
Maintainability	Over time many people (engineering and operations) can work on it productively, both keeping it running and adapting it.

Common confusion: these words get thrown around loosely. The rest of the chapter gives each one a precise meaning.

Reliability

What it is. Informally, users expect software to:

- do what they expected it to do;
- tolerate users making mistakes or using it in odd ways;
- perform well enough for the use case at the expected load and data volume;
- block unauthorised access and abuse.

If all of that is "working correctly", reliability roughly means **still behaving correctly while things are going wrong**.

Faults vs failures.

	Fault	Failure
Scope	One component	The whole system
Meaning	A part deviates from its spec	Users no longer get the required service
Goal	Can never be driven to zero probability	Prevent faults from turning into failures

Things that can go wrong are **faults**; systems that anticipate and cope with them are **fault-tolerant** or **resilient**. "Fault-tolerant" slightly oversells: no system can survive every conceivable fault.

(The book's joke: if a black hole swallowed Earth, surviving that would require hosting in space, which no budget will approve.) So we always talk about tolerating *specific kinds* of faults. The book covers many techniques for building reliable systems out of unreliable parts.

Deliberately causing faults. It can make sense to *increase* the fault rate on purpose, for example by randomly killing processes without warning. Many serious bugs come from poor error handling, so constantly exercising the recovery machinery makes you more confident it will work when a real fault happens. The **Netflix Chaos Monkey** is the classic example.

Prevention vs cure. Usually tolerating faults beats preventing them, but some faults have no cure. **Security** is the main case: once an attacker has stolen sensitive data, you cannot undo it. The book mostly deals with faults that *can* be cured.

Hardware Faults

What they are. Disks crash, RAM goes bad, the power grid blacks out, someone unplugs the wrong network cable. In a large datacenter these happen constantly.

The numbers. Hard disks have an MTTF of roughly **10 to 50 years**. With **10,000 disks** in a cluster, you should expect around **one disk to die every day**.

First response: hardware redundancy.

1. Put disks in a **RAID** configuration.
2. Give servers **dual power supplies** and **hot-swappable CPUs**.
3. Give datacenters **batteries and diesel generators** for backup power.
4. When one part dies, the redundant part takes over while the broken one is replaced.

It cannot fully stop hardware trouble from causing failures, yet it is a well-understood technique and often keeps a single machine up without interruption for years.

Why that stopped being enough.

- Until recently, redundant components made total loss of a single machine rare, and restoring a backup onto a new machine was usually quick enough. Only a few apps truly needed multiple machines for high availability.
- Now data volumes and compute demands mean more apps use **many machines**, which raises the total rate of hardware faults proportionally.
- On some cloud platforms such as **AWS**, virtual machine instances can vanish without warning, because the platforms favour **flexibility and elasticity** over single-machine reliability.

The shift. Systems increasingly use **software fault tolerance** to survive losing a *whole machine*, instead of or in addition to hardware redundancy. Bonus: operations get easier. A single-server system needs planned downtime to reboot (for example for OS security patches), whereas a system that tolerates node loss can be patched **node by node** with no overall downtime: a **rolling upgrade** (see Chapter 4).

Software Errors

How they differ from hardware faults. Hardware faults are usually treated as **random and independent**: one disk dying does not make another die (apart from weak correlations such as rack temperature). Many components failing together is unlikely.

Systematic errors are different: they are harder to predict and are **correlated across nodes**, so they cause far more system failures than independent hardware faults. Examples:

- A bug that crashes *every* instance of an app server when it sees a particular bad input. Real case: the **leap second on June 30, 2012**, which made many applications hang at the same moment because of a **Linux kernel** bug.
- A runaway process that exhausts something shared, such as memory, CPU time, network bandwidth, or disk space.

- A dependency that slows down, stops responding, or starts returning corrupted data.
- **Cascading failures**: a minor fault in one part causes a fault in a second part, which in turn causes further faults.

Why they hide. Such bugs can sit quietly for ages until some unusual combination of circumstances sets them off. The trigger exposes an assumption the software made about its environment that was *usually* true and then, one day, was not.

What helps (no silver bullet):

1. Think carefully about assumptions and interactions in the system.
2. Test thoroughly.
3. Isolate processes.
4. Let processes crash and restart.
5. Measure, monitor and analyse behaviour in production.
6. If the system promises some guarantee, have it **check itself continuously** and alert on discrepancies. Example: a message queue can check that messages in equals messages out.

Human Errors

The problem. Humans design, build and operate systems, and even well-meaning humans are unreliable. One study of large internet services found **operator configuration errors were the leading cause of outages**, while hardware faults (servers or network) were involved in just **10–25%** of outages.

How the best systems cope (they combine several approaches):

Approach	What it means in practice
Design to minimise error opportunities	Good abstractions, APIs and admin interfaces make the right thing easy and the wrong thing hard. But if they are too restrictive, people work around them, so balance is needed.
Decouple mistake-prone places from failure-causing places	Provide full-featured **non-production sandbox** environments with real data where people can experiment without affecting real users.
Test at every level	Unit tests, whole-system integration tests, manual tests. Automated tests are especially good at covering rare corner cases.
Make recovery quick and easy	Fast rollback of configuration changes, **gradual rollouts** of new code so a bug hits only a few users, and tools to **recompute data** if an old computation was wrong.
Detailed, clear monitoring	Performance metrics and error rates, called **telemetry** in other engineering fields (a rocket in flight depends on it). Monitoring gives early warnings, checks assumptions, and helps diagnosis.
Good management and training	Important but outside the book's scope.

How Important Is Reliability?

Reliability is not only for nuclear plants or air traffic control. Ordinary apps must be reliable too:

- Bugs in business software cost productivity, and wrong reported figures can bring **legal risk**.
- Outages of **ecommerce** sites cost revenue and reputation.
- Even "noncritical" apps carry responsibility. Picture a parent keeping every photo and video of their children in your photo app. How would they feel if the database were corrupted, and would they even know how to restore a backup?

Sometimes we knowingly trade reliability away, to lower **development cost** (a prototype for an unproven market) or **operational cost** (a service with razor-thin margins). The book's point: do it consciously, knowing where you are cutting corners.

Scalability

What it is. A system that works today may not work tomorrow. A common reason is **increased load**: perhaps concurrent users grow from 10,000 to 100,000, or from 1 million to 10 million, or data volumes grow a lot.

Scalability is the word for how well a system can handle growing load.

> **Common confusion:** scalability is *not* a yes/no label. Saying "X scales" or "Y doesn't scale" means nothing. The useful questions are:

- "If the system grows *in this particular way*, what options do we have?"
- "How can we add computing resources to absorb the extra load?"

Describing Load

What it is. Before asking "what if load doubles?" you must describe today's load with a few numbers, the **load parameters**. Which ones depend on your architecture: requests per second to a web server, read/write ratio in a database, simultaneously active users in a chat room, cache hit rate, or something else. Sometimes the average matters; sometimes a few extreme cases are the real bottleneck.

Worked example: Twitter (data from November 2012). Two main operations:

Operation	What it does	Load
Post tweet	Publish a message to your followers	**4.6k** requests/sec on average, over **12k** requests/sec at peak
Home timeline	View tweets from people you follow	**300k** requests/sec

Handling 12,000 writes per second alone would be fairly easy. The real challenge is **fan-out**: each user follows many people and is followed by many people.

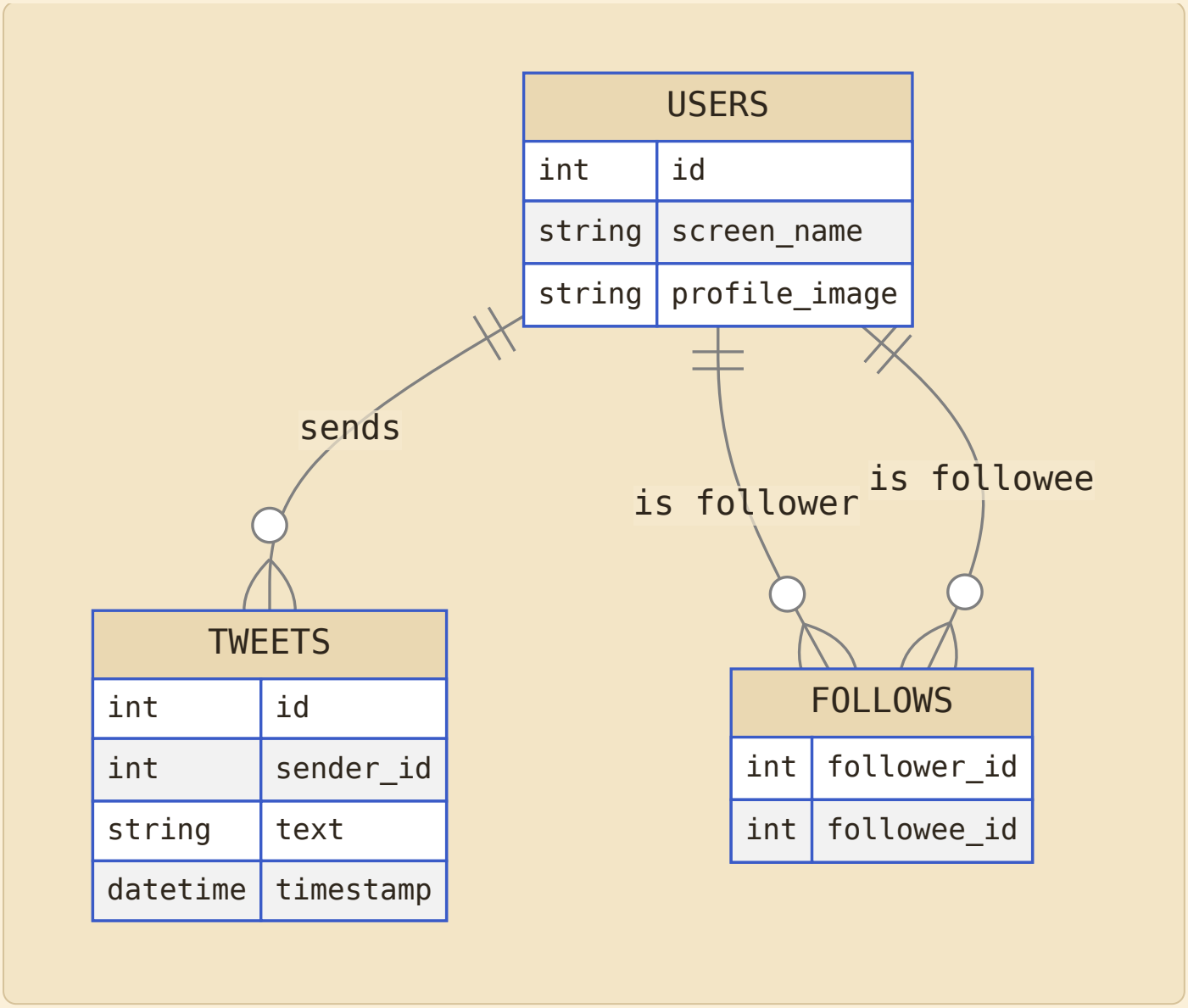
> **Aside: where "fan-out" comes from.** It is borrowed from electronics, where it counts how many logic gate inputs hang off one gate's output (which must supply enough current for all of them). In transaction processing it means how many requests to other services one incoming request causes.

Two ways to implement it:

Approach 1 — compute on read. Posting a tweet just inserts it into one global collection of tweets. When someone opens their home timeline:

1. Look up everyone they follow.
2. Fetch all tweets from each of those users.
3. Merge them, sorted by time.

Figure 1-2 (redrawn): a simple relational schema for a Twitter-style home timeline.



In SQL, approach 1 is one join-heavy query. Here is an equivalent idea for a different app, a recipe site where you "subscribe" to chefs and see their latest recipes:

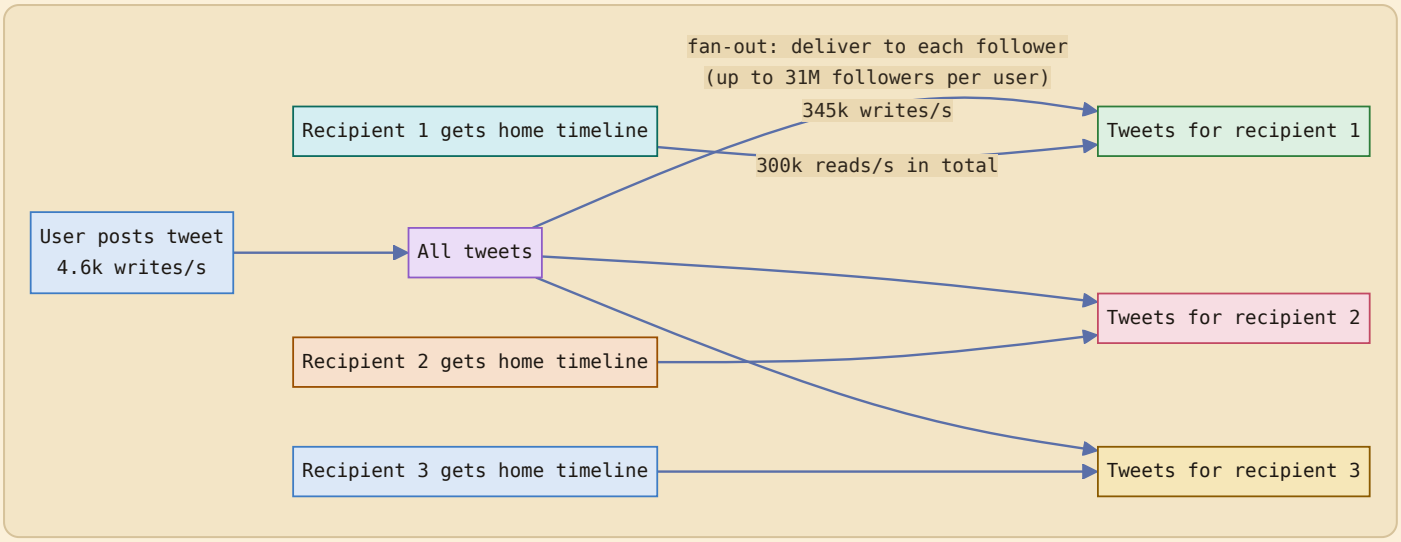
```
SELECT recipes.*, chefs.display_name      -- the recipe plus who wrote it
FROM recipes
  JOIN chefs         ON recipes.author_id = chefs.id          -- attach author details
  JOIN subscriptions ON subscriptions.chef_id = chefs.id      -- keep chefs someone subscribes to
WHERE subscriptions.reader_id = :me                           -- ...and that someone is me
ORDER BY recipes.published_at DESC;                           -- newest first
```

- Line 1 picks the columns to show.
- Lines 3–4 link each recipe to its author and each author to their subscribers.
- Line 5 keeps only rows where the subscriber is the current user.
- Line 6 sorts by time, just like merging timelines. All the work happens at **read time**.

Approach 2 — precompute on write. Keep a cache per user holding their home timeline, like a mailbox. When someone posts:

1. Look up all of that user's followers.
2. Insert the new tweet into each follower's home timeline cache.
3. Reading a timeline is now cheap, because the result was built in advance.

Figure 1-3 (redrawn): Twitter's pipeline for delivering tweets to followers, with November 2012 load parameters.



Which is better? Twitter started with approach 1, but the system could not keep up with timeline reads, so it switched to approach 2. It works better because tweets are published at a rate almost **two orders of magnitude lower** than timelines are read, so it is worth shifting effort to the write path to make the read path cheap.

The downside of approach 2. Posting becomes expensive. An average tweet reaches about **75 followers**, turning **4.6k tweets/sec into 345k cache writes/sec**. But the average hides huge variation: some users have **over 30 million followers**, so one tweet may trigger over 30 million writes. Twitter aims to deliver tweets within **five seconds**, which makes this hard.

Lesson. For Twitter, the **distribution of followers per user** (perhaps weighted by how often each tweets) is a key load parameter because it drives fan-out load. Your app will differ, but you can reason about its load the same way.

The final twist: a hybrid. With approach 2 working robustly, Twitter moved to a mix. Most users' tweets are still fanned out on write, but a small number of **celebrities** with enormous follower counts are exempt. Their tweets are fetched separately at read time and merged in, like approach 1. This gives consistently good performance. (The book returns to this in Chapter 12.)

	Approach 1: fan-out on read	**Approach 2: fan-out on write**	**Hybrid**
Cost of posting	Cheap (one insert)	Expensive (one write per follower)	Cheap for celebrities, per-follower for others
Cost of reading	Expensive (join + merge)	Cheap (read one cache)	Cheap cache read + merge in celebrity tweets
Pain point	Read rate is ~100x write rate	Users with 30M+ followers	More moving parts

Describing Performance

Once load is described, you can ask what happens when it grows, in two ways:

1. **Fixed resources:** raise a load parameter but keep CPU, memory and bandwidth the same. How does performance change?
2. **Fixed performance:** raise a load parameter. How much must you add in resources to keep performance the same?

Both need performance numbers.

Throughput vs response time.

- In **batch** systems (Hadoop is the book's example), the usual concern is **throughput**: records processed per second, or total time for a job on a dataset of a given size.
- In **online** systems, **response time** usually matters most: the time from when a client sends a request until the reply arrives.

> **Aside: batch running time.** Ideally a batch job takes dataset size divided by throughput. In practice it takes longer because of **skew** (data not evenly spread among workers) and waiting for the slowest task to finish.

> **Latency vs response time.** People often treat them as synonyms, but they differ. **Response time** is what the client experiences: the actual processing time (**service time**) plus network delays and queueing delays. **Latency** is only the time a request spends *waiting* to be handled, when it is latent.

Response time is a distribution, not a number. Sending the same request repeatedly still gives slightly different times, and a real system with varied requests varies a lot. Some slow requests are genuinely more expensive (more data). But even identical requests vary because of random extra delays: switching the CPU to a background process (context switch), a dropped packet that TCP must retransmit, a pause for garbage collection, a page fault that forces a disk read, even mechanical vibration in the server rack, and plenty of other causes.

Figure 1-4 (described): a bar chart of response times for 100 sample requests. Most bars are short, a handful of outliers are much taller, and the chart marks the mean, the median, and the 95th and 99th percentiles, showing how a single average hides the spread while the high percentiles reveal how bad the outliers are.

Why the mean is misleading. "Average" usually means the arithmetic mean (add n values, divide by n). It does not tell you how many users actually waited that long, so it is a poor measure of the *typical* experience.

Use percentiles instead.

1. Sort all response times from fastest to slowest.
2. The **median** (50th percentile, **p50**) is the halfway point. If p50 = 200 ms, half of requests finish faster than 200 ms and half take longer. It tells you how long users *typically* wait.
3. Higher percentiles show how bad the outliers are: **p95, p99, p999** (99.9th). If p95 = 1.5 s, then 95 of 100 requests take under 1.5 s and 5 of 100 take 1.5 s or more.

> **Common confusion:** the median is about a *single request*. If a user makes several requests (over a session, or because one page loads several resources), the chance that *at least one* is slower than the median is well above 50%.

Why tail latencies matter. High percentiles, or **tail latencies**, directly shape user experience.

- **Amazon** sets internal service response-time requirements at the **99.9th percentile**, even though that concerns only 1 in 1,000 requests. The slowest requests often belong to customers with the most data, because they have bought the most, making them the **most valuable customers**.
- Amazon observed that **100 ms** more response time cuts sales by **1%**; others report a **1-second** slowdown lowers a customer satisfaction metric by **16%**.
- But optimising the **99.99th percentile** (1 in 10,000) was judged too costly for too little gain at Amazon. Very high percentiles are dominated by random events you cannot control, and returns diminish.

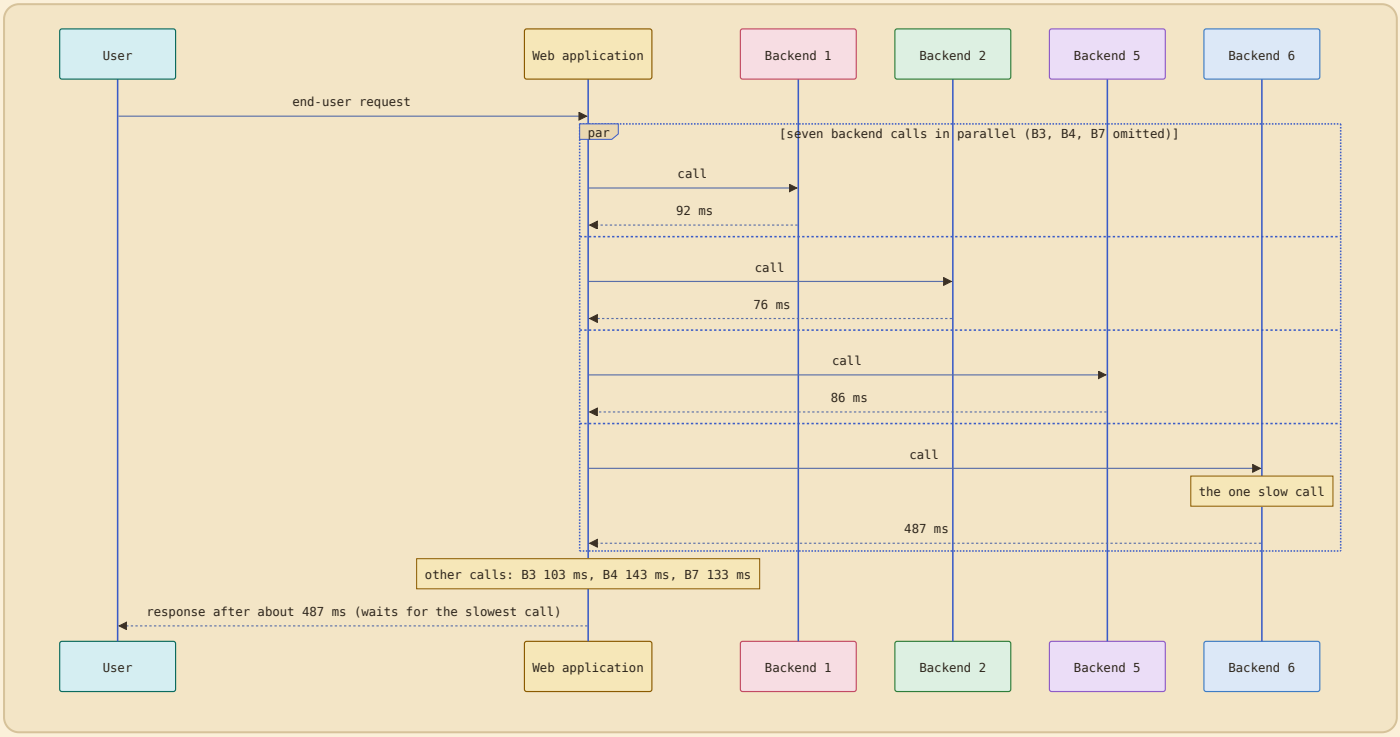
SLOs and SLAs. Percentiles are commonly written into **SLOs (service level objectives)** and **SLAs (service level agreements)**, the contracts stating how fast and how available a service is expected to be. Example SLA: the service counts as "up" if its median response time is below **200 ms** and its p99 below **1 s** (slower is as bad as down), and it must meet that bar for no less than **99.9%** of the time. Clients know what to expect and can claim refunds if it is missed.

Queueing delays and head-of-line blocking. A server can only work on a few things at once (limited by, for example, CPU cores). A few slow requests can hold up everything queued behind them: **head-of-line blocking**. The later requests may be quick to process, yet the client sees a slow response because of waiting. Therefore **measure response time on the client side**.

Load-testing pitfall. When generating artificial load, the test client must keep sending requests **regardless of response time**. If it waits for each reply before sending the next, queues stay artificially short and the results are skewed.

> **Percentiles in practice (sidebar).**
>
> - **Tail latency amplification:** when one end-user request calls several backends (even in parallel), it must wait for the *slowest* call. One slow call makes the whole request slow. So even if only a small fraction of backend calls are slow, a larger fraction of user requests end up slow.
> - **Computing percentiles on dashboards:** e.g. keep a rolling window of the last 10 minutes of response times and, every minute, compute the median and other percentiles and plot them.
> - The naive way keeps every response time in the window and sorts the list every minute. Cheaper probabilistic algorithms estimate percentiles closely while using very little CPU and memory: **forward decay**, **t-digest**, **HdrHistogram**.
> - **Never average percentiles** (for example to lower time resolution or merge several machines). It is mathematically meaningless. To combine response-time data correctly, you **add up the histograms**.

Figure 1-5 (redrawn): a web application fans one end-user request out to seven backends (92, 76, 103, 143, 86, 487, 133 ms); the single 487 ms call sets the time for the whole request.



A quick illustration of amplification (our own numbers, not the book's): if each backend call has a 1% chance of being slow and a page makes 100 independent calls, the chance that at least one is slow

is 1 − 0.99^100 ≈ 0.634, about 63%.

Approaches for Coping with Load

Core idea. An architecture suited to one level of load is unlikely to handle **10 times** that load. A fast-growing service will probably need to rethink its architecture at **every order of magnitude** of growth, or even more often.

Scaling up vs scaling out.

	Scaling up (vertical)	Scaling out (horizontal, shared-nothing)
How	Move to a more powerful machine	Spread load over many smaller machines
Pros	Simpler; everything on one machine	Can go beyond any single machine's limits
Cons	High-end machines get very expensive	More complexity, especially for stateful data

In practice good architectures mix both pragmatically. For example, a few fairly powerful machines can be simpler and cheaper than lots of small virtual machines.

Elastic vs manual scaling.

- **Elastic** systems add resources automatically when load rises. Useful when load is highly unpredictable.
- **Manually scaled** systems rely on a human to analyse capacity and add machines. Simpler, with fewer operational surprises (see "Rebalancing Partitions", Chapter 6).

Stateless vs stateful. Spreading **stateless** services over many machines is fairly easy. Taking a **stateful** data system from one node to many adds a lot of complexity. So conventional wisdom until recently was: **keep your database on one node (scale up)** until cost or high-availability needs force you to distribute it. As distributed tools and abstractions improve, this may change, and distributed data systems might become the default even for modest data or traffic. The rest of the book examines many distributed data systems for scalability, ease of use and maintainability.

No magic scaling sauce. Large-scale architectures are usually very specific to the application; there is no generic, one-size-fits-all scalable design. The problem might be read volume, write volume, data volume, data complexity, response time needs, access patterns, or (usually) a mix of all of these and more.

Example of why. A system for **100,000 requests/sec of 1 kB each** looks very different from one for **3 requests/minute of 2 GB each**, even though both move the same amount of data per second.

Assumptions matter. A well-scaling architecture is built on assumptions about which operations are common and which are rare (the load parameters). If those assumptions are wrong, scaling work is wasted at best and harmful at worst. In an **early-stage startup or unproven product**, fast iteration on features usually matters more than scaling for imagined future load.

Still, even application-specific scalable designs tend to be assembled from **general-purpose building blocks in recurring patterns**, and those blocks and patterns are what the book teaches.

Maintainability

Why it matters. Most of the cost of software comes **after** initial development: fixing bugs, keeping systems running, investigating failures, moving to new platforms, adapting to new use cases, paying back technical debt, and adding features.

Many engineers dislike maintaining **legacy systems** (fixing others' mistakes, outdated platforms, systems stretched to do things they were not designed for). Every legacy system is painful in its own way, so general advice is hard. What we *can* do is design so that maintenance hurts less, and avoid creating legacy software ourselves.

Three design principles:

Principle	Goal
Operability	Make it easy for operations to keep the system running smoothly.
Simplicity	Remove as much complexity as you can so newcomers can understand the system. (This is about the system's internals, not a simple user interface.)
Evolvability	Make it easy to change the system later for unanticipated needs. Also called extensibility, modifiability, or plasticity.

Just like reliability and scalability, these have no quick recipe; the aim is to keep the three principles in mind while designing.

Operability: Making Life Easy for Operations

One often-quoted view: a strong operations team can frequently compensate for weak or incomplete software, whereas even excellent software will not run reliably if operations are poor. Some operations work can and should be automated, yet humans must still set up that automation and check it works.

What a good operations team typically does:

- Monitors system health and quickly restores service when it goes bad.
- Tracks down causes of problems such as failures or degraded performance.
- Keeps platforms and software current, security patches included.
- Understands how systems affect each other, so a risky change is caught before damage.
- Anticipates future problems (for example **capacity planning**) and solves them early.
- Establishes good practices and tools for deployment and configuration management.
- Performs complex maintenance, such as migrating an application between platforms.
- Keeps the system secure as configuration changes.
- Defines processes that make operations predictable and production stable.
- Preserves organisational knowledge about the system as people come and go.

What data systems can do to make routine tasks easy (freeing ops for high-value work):

- Give **visibility** into runtime behaviour and internals via good monitoring.
- Support **automation** and integration with standard tools.
- **Avoid depending on individual machines**, so any one machine can be switched off for maintenance while the rest keeps serving.
- Provide good documentation and an **operational model** that is easy to grasp (doing X leads to Y).
- Offer **good defaults**, but let admins override them.
- **Self-heal** where sensible, but give admins manual control of system state when needed.
- Behave **predictably**, with few surprises.

Simplicity: Managing Complexity

The problem. Small projects can have beautifully simple code, but big projects tend to become complex and hard to understand. Complexity slows everyone down and raises maintenance cost. A project stuck in complexity is called a **big ball of mud**.

Symptoms of complexity: a state space that explodes, modules coupled too tightly, dependencies in a tangle, names and terms used inconsistently, performance hacks, special cases to work around problems elsewhere, and more.

Why it hurts. Budgets and schedules overrun. Changes are riskier: when a system is hard to reason about, hidden assumptions, unintended consequences and surprising interactions slip by. Reducing complexity greatly improves maintainability, so **simplicity should be a key goal**.

Accidental vs essential complexity. Simpler does not have to mean fewer features. It can mean removing **accidental complexity**, which Moseley and Marks define as complexity that is not inherent in the problem as users see it but arises only from the implementation.

Abstraction is the best tool. A good abstraction hides lots of detail behind a clean, easy façade and can be reused by many applications. Reuse is more efficient than rebuilding similar things, and improvements to the shared component benefit every user of it.

Examples:

- **High-level programming languages** hide machine code, CPU registers and syscalls. The machine code is still there; you just do not deal with it directly.
- **SQL** shields you from intricate data structures on disk and in memory, from other clients' concurrent requests, and from inconsistencies left by crashes.

The catch. Good abstractions are hard to find. In distributed systems there are many good algorithms, but it is much less clear how to package them into abstractions that keep complexity manageable. The book keeps looking for abstractions that turn parts of large systems into well-defined, reusable components.

Evolvability: Making Change Easy

Requirements almost never stay fixed. You learn new facts, new use cases appear, business priorities shift, users ask for features, new platforms replace old ones, laws and regulations change, and growth forces architectural changes.

- **Agile** working patterns give an organisational framework for change, and the Agile community contributed technical practices like **test-driven development (TDD)** and **refactoring**.
- Most of those discussions are small-scale (a couple of source files in one app). This book looks for agility at the level of a **whole data system**, possibly many applications and services. Example question: how would you "refactor" Twitter's timeline architecture from approach 1 to approach 2?
- How easily a data system can be changed depends heavily on its **simplicity and abstractions**: simple, understandable systems are easier to modify.

Because this system-level agility is so important, the book gives it its own name: **evolvability**.

Summary

- An application must meet **functional requirements** (what it does: store, retrieve, search, process data) and **nonfunctional requirements** (general properties like security, reliability, compliance, scalability, compatibility, maintainability). This chapter focused on three nonfunctional ones.
- **Reliability** = working correctly even when faults happen. Faults come from **hardware** (usually random and uncorrelated), **software** (bugs are usually systematic and hard to handle), and **humans** (who will make mistakes). Fault-tolerance techniques can hide some kinds of faults from users.
- **Scalability** = having strategies to keep performance good as load grows. To discuss it, describe **load** quantitatively (Twitter's home timeline and fan-out) and **performance** quantitatively (response time percentiles). A scalable system lets you add capacity to stay reliable under high load.
- **Maintainability** is about making life better for the engineers and operators who work on the system. Good abstractions cut complexity and make the system simpler to modify and adapt to new use cases; good operability means clear visibility into health and effective ways to manage it.
- There is no easy fix for any of the three, but certain patterns and techniques recur across many kinds of applications; the coming chapters study them. Part III returns to multi-component systems like Figure 1-1.

Interview angle

1. "What is the difference between a fault and a failure, and how do you build reliable systems?" *30-second answer:* A fault is one component misbehaving; a failure is the whole system no longer serving users. You cannot eliminate faults, so you design fault tolerance so faults do not become failures: redundancy (RAID, dual power, multiple machines), software tolerance of whole-node loss, process isolation and restarts, monitoring, and quick rollback. Deliberately injecting faults (Chaos Monkey) keeps the recovery paths tested.

- *Follow-up: Why are software bugs worse than hardware faults?* Hardware faults are mostly independent; a bug is systematic and hits every node that sees the same input or condition at once (e.g. the 2012 leap-second Linux bug), so it can take down the whole fleet.
- *Follow-up: What is the biggest cause of outages?* In the study cited, operator configuration errors; hardware played a role in only 10–25%.

2. "Design a Twitter home timeline. Fan-out on write or on read?" *30-second answer:* Reads (300k/s) outnumber posts (4.6k/s avg) by about 100x, so precompute timelines at write time into per-user caches. But celebrities with tens of millions of followers make write fan-out explode, so use a hybrid: fan out for normal users, and fetch celebrity tweets at read time and merge.

- *Follow-up: What is the key load parameter?* The distribution of followers per user (weighted by tweet frequency), because it determines fan-out cost.
- *Follow-up: How many cache writes per second?* About 75 followers on average, so roughly 345k writes/s.

3. "Why use percentiles instead of the average for latency?" *30-second answer:* The mean hides how many users actually suffer; percentiles show the distribution. p50 is the typical experience, p99/p999 show the tail. Tails matter because the slowest requests often belong to the most valuable users and because fan-out to many backends amplifies tail latency.

- *Follow-up: Can I average p99s from 10 servers?* No, that is meaningless; merge the histograms instead (or use t-digest/HdrHistogram).
- *Follow-up: Where should latency be measured?* On the client side, because queueing and head-of-line blocking add delay the server does not see.
- *Follow-up: Latency vs response time?* Response time = service time + network + queueing delays, as seen by the client; latency is only the waiting time.

4. "Scale up or scale out?" *30-second answer:* Scaling up is simpler but high-end hardware gets very expensive; scaling out (shared-nothing) goes further but adds complexity, especially for stateful data. Stateless services are easy to scale out; the classic advice is to keep the database on one node until cost or availability forces distribution. Real systems mix both, and expect to redesign at each order-of-magnitude of growth.

- *Follow-up: Elastic or manual scaling?* Elastic helps with unpredictable load; manual is simpler with fewer surprises.

5. "What makes a system maintainable?" *30-second answer:* Operability (easy to run: monitoring, automation, no single-machine dependencies, predictable behaviour), simplicity (remove accidental complexity with good abstractions), and evolvability (easy to adapt as requirements change).

- *Follow-up: What is accidental complexity?* Complexity that comes from the implementation, not from the problem users need solved.

Self-check

1. **What makes an application "data-intensive"?** Its main challenge is the data (volume, complexity, or rate of change) rather than CPU power.

2. **Give two reasons the book groups databases, queues and caches under "data systems".** Tools now overlap (Redis as a queue, Kafka with database-like durability), and apps combine several tools with application code, effectively creating a new composite data system.
3. **A cluster has 10,000 disks with MTTF of 10–50 years. Roughly how many fail per day?** About one per day on average.
4. **Why did the industry shift from hardware redundancy toward software fault tolerance?** More machines mean more total hardware faults, cloud VMs can disappear without warning, and tolerating whole-node loss enables rolling upgrades without downtime.
5. **Your p50 is 100 ms and p99 is 2 s. A page makes 20 backend calls in parallel. Is the typical page load closer to 100 ms or 2 s? Why?** Neither number describes it well. The page waits for the slowest of its 20 calls, so it is almost never as fast as the per-call median (the chance that all 20 calls beat p50 is 0.5^20, about one in a million). And with a 1% chance per call of being p99-slow, about 18% of pages (1 − 0.99^20 ≈ 0.18) wait 2 s or more, compared with 1% of single calls. That jump from 1% to 18% is tail latency amplification. (Our own illustrative numbers, assuming independent calls; the book states the effect without arithmetic.)
6. **Why did Twitter switch from approach 1 to approach 2, and why then move to a hybrid?** Timeline reads vastly outnumbered posts, so precomputing on write made reads cheap. But users with 30M+ followers made write fan-out huge, so celebrities' tweets are fetched at read time instead.
7. **A load-test client waits for each response before sending the next request. What goes wrong?** Queues stay artificially short, so measured response times look better than reality.
8. **Two systems each handle the same bytes per second: one with 100,000 × 1 kB requests/sec, one with 3 × 2 GB requests/minute. Should they share an architecture?** No. Load parameters differ completely, so the architectures would look very different; there is no generic scalable design.
9. **Name three things a data system can do to improve operability.** Any three of: good monitoring/visibility, automation support, no dependency on single machines, clear documentation and operational model, sensible overridable defaults, self-healing with manual override, predictable behaviour.
10. **When is it acceptable to sacrifice reliability?** When deliberately trading it for lower development cost (prototype for an unproven market) or operational cost (very thin margins), as long as you know you are cutting corners.

Chapter 2 — Data Models and Query Languages

Book: Chapter 2, pp. 27–68

Overview

What question does this chapter answer? "In what *shape* should I store my data, and how do I ask questions of it?" It compares the three big general-purpose **data models** (relational, document, graph) and the **query languages** that go with them (SQL, relational algebra, MapReduce, the MongoDB aggregation pipeline, Cypher, SPARQL and Datalog), plus CSS/XSL as a side analogy.

Why it matters. The data model is arguably the most important decision in a piece of software: it shapes not only how the code is written but how you *think* about the problem. Every model assumes certain usage. Some operations are easy and fast in it, others are awkward, slow, or impossible. Choosing a model that fits your data's relationships saves years of pain.

Layers of data models. Software is built by stacking one model on another, and each layer hides the one below behind a cleaner model (an **abstraction**):

1. **Application developer:** models the real world (people, organisations, goods, actions, money flows, sensors...) as objects/data structures plus APIs that manipulate them. Often app-specific.
2. **General-purpose data model:** to store those structures you express them as JSON or XML documents, relational tables, or a graph. *This chapter lives here.*
3. **Database engineers:** decided how to represent JSON/XML/tables/graphs as bytes in memory, on disk, or on the network, in ways that allow querying, searching and processing. *That is Chapter 3.*
4. **Hardware engineers:** represent bytes as electrical currents, light pulses, magnetic fields, and more.

Bigger systems may have more layers (APIs on top of APIs), but the idea holds: each layer hides the complexity beneath, which lets different groups (say, the database vendor's engineers and the app developers using that database) work together effectively. Mastering even one model is a lot of work (think of how many books exist on relational modeling), so choose carefully.

Everyday analogy. Think of storing information about your family. A **document** is like a folder per person with everything about them inside. A **relational** store is like a set of spreadsheets (people, addresses, jobs) linked by ID numbers. A **graph** is like a big family-tree wall chart where anything can be connected to anything with a labelled string.

Key terms

Term	Plain-English meaning
Data model	The general shape used to represent data (tables, documents, graphs).
Relational model	Data organised as relations (tables), each an unordered collection of tuples (rows). Proposed by Edgar Codd in 1970.
RDBMS	Relational database management system.
Transaction processing	Day-to-day business operations: sales, banking transactions, airline reservations, warehouse stock-keeping.
Batch processing (here)	Periodic bulk jobs: customer invoicing, payroll, reporting.
Network model / hierarchical model	Pre-relational database models of the 1970s (CODASYL, IMS).
NoSQL	Loose label for nonrelational databases; later reinterpreted as "Not Only SQL".
Polyglot persistence	Using relational and various nonrelational stores side by side.
Impedance mismatch	The awkward translation between application objects and tables/rows/columns.
ORM (object-relational mapping)	Frameworks like ActiveRecord and Hibernate that reduce translation boilerplate.
One-to-many relationship	One item owns many sub-items (a user has many jobs); forms a tree.
Many-to-one relationship	Many items point to one shared item (many people live in one region).
Many-to-many relationship	Items on both sides relate to many on the other (people and the companies they worked at).
Normalization	Storing each human-meaningful fact once and referring to it by ID; avoids duplication.
Denormalization	Deliberately duplicating data (e.g. to avoid joins), at the cost of keeping copies in sync.
Join	Combining records from different tables/collections that reference each other.
Foreign key / document reference	An ID pointing at another record, in relational / document databases respectively.
Locality	Storing related data together so it can be read in one go.
Access path	In the network model, the chain of links you follow from a root record to reach a record.
Query optimizer	Database component that automatically decides how to execute a query (order, indexes, join methods).
Shredding	Splitting a document-like structure into several tables.
Schema-on-write	Explicit schema enforced by the database when data is written (like static type checking).
Schema-on-read	Implicit structure interpreted by the code that reads the data (like dynamic type checking).
"Schemaless"	Misleading name for schema-on-read.
BSON	MongoDB's binary encoding of JSON-like documents.
Imperative language	Tells the computer *which operations to perform in which order*.

Term	Plain-English meaning
Declarative language	States *what result you want* (conditions, transformations), not how to compute it.
Relational algebra	The mathematical operators behind SQL, e.g. σ (selection).
MapReduce	Programming model for bulk processing across many machines using map and reduce functions.
Pure function	Uses only its inputs, makes no extra database queries, and has no side effects.
Aggregation pipeline	MongoDB's declarative, JSON-syntax query language (added in 2.2).
Graph, vertex, edge	A graph is vertices (nodes/entities) connected by edges (relationships/arcs).
Property graph	Graph model where vertices and edges have IDs, labels (on edges) and key-value properties.
Tail / head vertex	Where an edge starts / where it ends.
Cypher	Neo4j's declarative language for querying property graphs.
Recursive common table expression	SQL:1999 `WITH RECURSIVE` feature for variable-length traversals.
Triple-store	Stores facts as (subject, predicate, object) triples.
Turtle / N3	Human-readable text formats for triples (Turtle is a subset of Notation3).
RDF	Resource Description Framework, the semantic web's triple data model.
Semantic web	Idea that websites should publish machine-readable data that can be combined into a "web of data".
SPARQL	Query language for RDF triple-stores (pronounced "sparkle").
Datalog	Old logic query language (studied in the 1980s), a subset of Prolog; builds queries from rules.
Rule (Datalog)	Defines a new derived predicate from existing facts and other rules.

Relational Model Versus Document Model

What it is. The best-known data model is the **relational model** behind SQL, proposed by **Edgar Codd in 1970**. Data sits in **relations** (SQL calls them *tables*), and every relation is a collection of **tuples** (SQL: *rows*) with **no inherent order**.

A short history.

- At first it was only a theoretical proposal and many doubted it could be implemented efficiently. By the **mid-1980s**, RDBMSes and SQL were the default choice for storing and querying data with some regular structure. That dominance lasted about **25–30 years**, an eternity in computing.
- Its roots are in **business data processing** on **mainframes in the 1960s and '70s**: mostly **transaction processing** (recording sales or banking transactions, booking airline seats, tracking warehouse stock) and **batch processing** (invoicing customers, payroll, reporting).
- Other databases of that era made developers think hard about how data was represented internally. The relational model's goal was to **hide that detail behind a cleaner interface**.
- Challengers came and went: the **network model** and **hierarchical model** (1970s and early 1980s), **object databases** (late 1980s and early 1990s), **XML databases** (early 2000s, only niche adoption). Each was hyped; none lasted.

- Meanwhile relational databases generalised far beyond business data processing. Much of today's web (online publishing, discussion, social networking, ecommerce, games, SaaS productivity apps) still runs on them.

The Birth of NoSQL

In the **2010s**, **NoSQL** became the latest attempt to dethrone the relational model.

- The name refers to no particular technology. It began as a catchy **Twitter hashtag for a 2009 meetup** about open source, distributed, nonrelational databases. It caught on in the startup world and beyond, and was later reinterpreted as **"Not Only SQL"**.

Why people adopted NoSQL:

1. Need for **more scalability** than relational databases easily provide, for instance huge datasets or a very high rate of writes.
2. A broad preference for **free and open source** products over commercial databases.
3. **Specialised query operations** that the relational model does not support well.
4. Frustration with **restrictive relational schemas** and a wish for a more dynamic, expressive data model.

Different applications have different needs, so relational databases will likely keep being used **alongside** a wide range of nonrelational stores. This mix is called **polyglot persistence**.

The Object-Relational Mismatch

The problem. Most application code is written in object-oriented languages. If the data lives in relational tables, you need an awkward translation layer between objects in code and tables/rows/columns in the database. This disconnect is called the **impedance mismatch**.

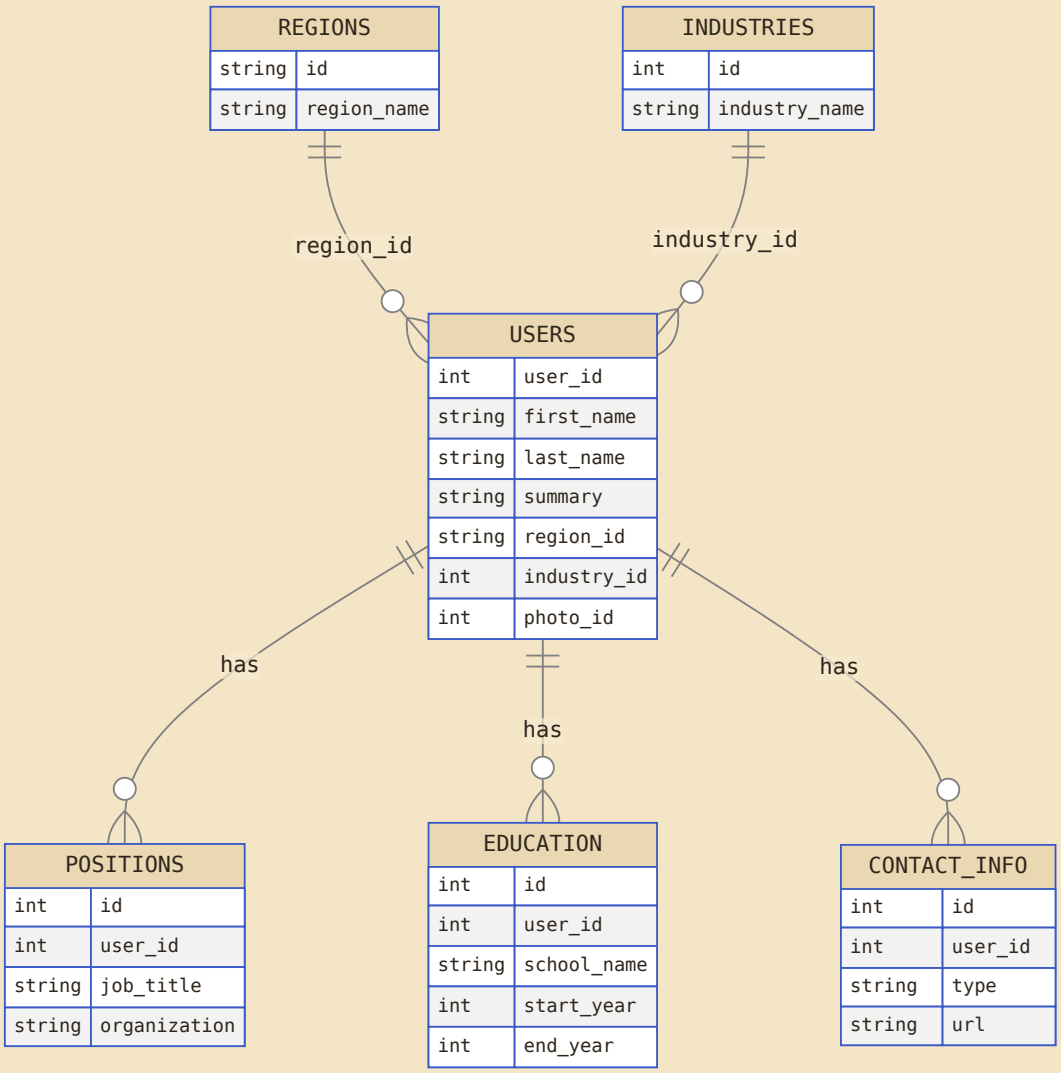
> **Where "impedance mismatch" comes from.** It is an electronics term. Every circuit has an impedance (resistance to alternating current) at its inputs and outputs. Power transfers best when the output impedance of one circuit matches the input impedance of the next; a mismatch causes signal reflections and other problems.

ORMs such as **ActiveRecord** and **Hibernate** cut down the boilerplate for this translation, but they cannot fully hide the differences between the two models.

The résumé example. The book models a résumé, specifically a **LinkedIn profile** (of Bill Gates), in a relational schema.

- The whole profile is identified by a unique `user_id`.
- Fields that appear once per user, like `first_name` and `last_name`, are columns on the `users` table.
- But people have had several jobs (**positions**), a varying number of **education** entries, and any amount of **contact information**. These are **one-to-many** relationships from the user.

Figure 2-1 (redrawn): a LinkedIn-style profile represented with a relational schema.



(In the book's figure the `users` table holds a `photo_id`; the JSON version in Example 2-1 instead carries a `photo_url` string.)

Three ways to store one-to-many data in SQL:

Option	How	Notes
1. Separate tables (classic normalized)	`positions`, `education`, `contact_info` tables, each with a foreign key to `users` (as in Figure 2-1)	The common approach before SQL:1999.
2. Structured / XML / JSON datatypes	Store multi-valued data inside one row, with querying and indexing inside those documents	Oracle, IBM DB2, MS SQL Server and PostgreSQL support this, each to a different extent. Several (including IBM DB2, MySQL, PostgreSQL) also offer a JSON datatype.
3. Encode as JSON/XML text	Put the jobs, education and contacts in a text column and let the application interpret it	You usually **cannot query** inside that column with the database.

The document alternative. A résumé is mostly a **self-contained document**, so a JSON representation fits well. JSON is also much simpler than XML. Document-oriented databases **MongoDB**, **RethinkDB**, **CouchDB** and **Espresso** support this model. Here is our own profile in the same style (fictional person):

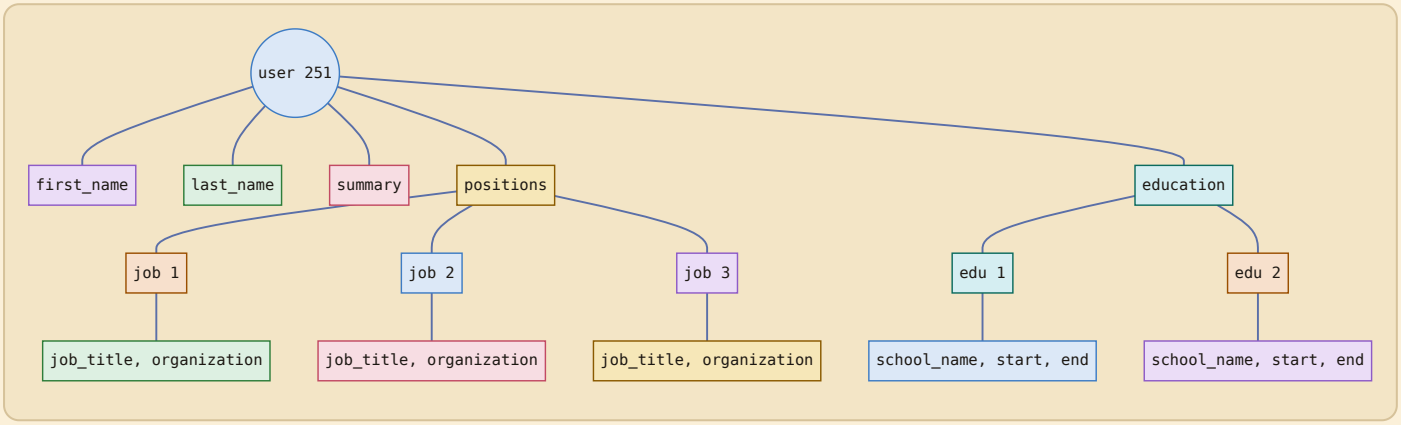
```json
{
  "user_id": 7042,
  "first_name": "Asha",
  "last_name": "Rao",
  "summary": "Marine ecologist and science writer.",
  "region_id": "in:ka",
  "industry_id": 57,
  "positions": [
    {"job_title": "Research Lead", "organization": "Coral Watch Trust"},
    {"job_title": "Field Scientist", "organization": "Ocean Survey Institute"}
  ],
  "education": [
    {"school_name": "University of Mysore", "start": 2006, "end": 2010},
    {"school_name": "Bishop Cotton School", "start": null, "end": null}
  ],
  "contact_info": {
    "blog": "https://example.org/asha",
    "mastodon": "@asha@example.social"
  }
}
```

- `user_id`, `first_name`, `last_name`, `summary`: one-per-user fields, like columns.
- `region_id`, `industry_id`: stored as **IDs**, not text (explained in the next section).
- `positions` and `education`: **arrays** nested inside the document, replacing whole separate tables.
- `contact_info`: a nested **object**.

Pros of the JSON version:

- Some developers find it **reduces the impedance mismatch** between code and storage. (JSON has its own problems as an encoding format, covered in Chapter 4.)
- The **lack of a schema** is often called an advantage (see "Schema flexibility in the document model" below).
- **Better locality:** in the relational version, fetching a profile needs several queries (one per table by `user_id`) or a messy multi-way join. In JSON all the relevant data sits together, so a **single query** suffices.
- The one-to-many relationships form a **tree**, and JSON makes that tree **explicit**.

Figure 2-2 (redrawn): the book's user 251 as a tree. Scalar fields hang directly off the user; positions and education are one-to-many branches whose entries each carry their own fields. (The book's tree leaves out contact_info; our JSON above would add it as a third branch.)



Many-to-One and Many-to-Many Relationships

Why IDs instead of text? In the book's JSON, `region_id` and `industry_id` are IDs, not strings like `"Greater Seattle Area"` and `"Philanthropy"`. Free-text input could just be stored as text, but **standardised lists** (drop-downs or autocomplete) bring benefits:

- **Consistent** style and spelling across profiles.
- **No ambiguity**, for example when several cities share a name.
- **Easy updates:** the name lives in one place, so a change (say a city renamed after political events) happens everywhere at once.
- **Localisation:** when the site is translated, the standard lists can be translated, so region and industry show in the viewer's language.
- **Better search:** if someone looks for philanthropists located in Washington state, this profile can show up, because the standard region list can encode that Seattle lies in Washington, a fact the text "Greater Seattle Area" does not reveal.

ID vs text is really about duplication.

- With an ID, the **human-meaningful** information (like the word *Philanthropy*) is stored **once**, and everything else refers to it by an ID that only has meaning inside the database.
- With text, that human-meaningful information is **copied into every record**.
- Because an ID means nothing to humans, it **never needs to change**, even if what it identifies changes. Anything meaningful to humans might change someday, and if it is duplicated, every copy must be updated. That costs **write overhead** and risks **inconsistency** (some copies updated, others not).
- Removing such duplication is the core idea of **normalization**.

> **Normal forms aside.** Relational theory defines several normal forms, but the differences between them matter little in practice. Rule of thumb: if you duplicate values that could live in one place, the schema is not normalized.

> The book notes that DBAs and developers love arguing about normalization vs denormalization and defers judgement; Part III returns to caching, denormalization and derived data systematically.

The catch for documents. Normalizing needs **many-to-one** relationships (many people live in one region, many work in one industry), and these do not fit the document model nicely.

- In relational databases, referring to other tables' rows by ID is normal because **joins are easy**.
- In document databases, one-to-many trees need no joins, and **join support is often weak**. At the time of writing: **RethinkDB supports joins, MongoDB does not, and CouchDB supports them only in predeclared views.**
- Without database joins you **emulate them in application code** with multiple queries. (Here the region and industry lists are small and change rarely, so the app could just keep them in memory. Still, the join work has moved from the database to your code.)

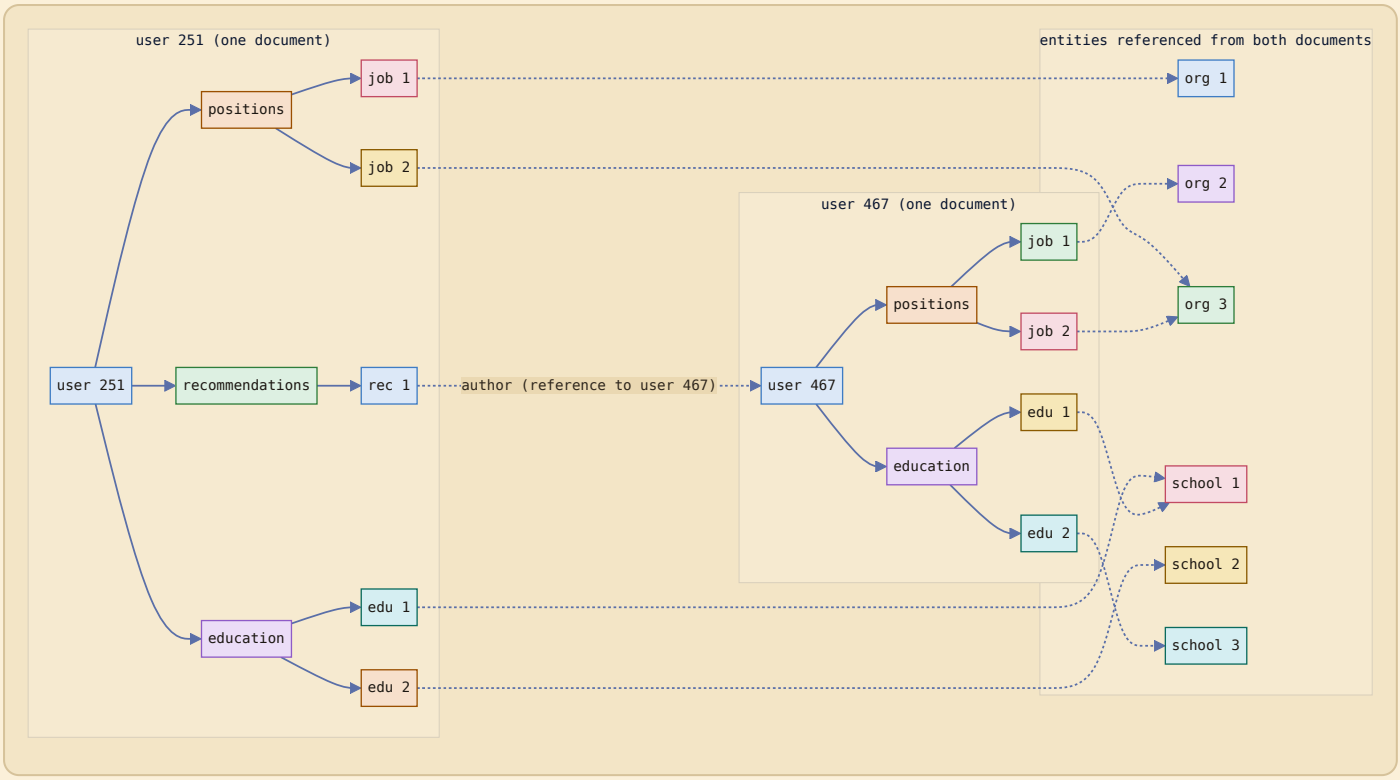
Data grows more connected over time. Even if version 1 of an app fits a join-free document model, new features tend to add connections. Two extensions to the résumé:

- **Organizations and schools as entities.** `organization` and `school_name` are currently just strings. They could become references to entities, each with its own web page (logo, news feed, etc.), and each résumé could link to them and show their logos.

Figure 2-3 (described): a LinkedIn screenshot where the company name on a profile is a link to a company entity page (with logo), not just plain text.

- **Recommendations.** Suppose users can write recommendations for each other. A recommendation appears on the recommended user's résumé along with the **name and photo of the author**. If the author changes their photo, every recommendation they wrote must show the new photo, so the recommendation must **reference the author's profile**.

Figure 2-4 (redrawn): two résumés (user 251 and user 467) extended with many-to-many links. Everything inside one dashed box could be one document; the arrows leaving a box are references to organizations, schools or other users, which need joins when queried.



Are Document Databases Repeating History?

Many-to-many relationships and joins are routine in relational databases, but document databases and NoSQL **reopened the debate** about how to represent them. That debate is far older than NoSQL.

IMS and the hierarchical model.

- In the **1970s**, the leading database for business data processing was IBM's **Information Management System (IMS)**. It was originally developed to track stock for the **Apollo space program**, went on commercial release in **1968**, and is still maintained and used today on **OS/390** on IBM mainframes.
- IMS used the **hierarchical model**: all data is a **tree of records nested inside records**, strikingly similar to JSON documents (like Figure 2-2).
- Like document databases, IMS handled **one-to-many** well, made **many-to-many** hard, and **did not support joins**. Developers had to choose between **duplicating (denormalizing)** data or **manually resolving references** between records. These 1960s–70s problems look very much like what developers face with document databases today.

Two main solutions were proposed: the **relational model** (which turned into SQL and conquered the industry) and the **network model** (which had many fans early on but faded into obscurity). The "great debate" between them lasted much of the 1970s.

The network model

- A committee named the **Conference on Data Systems Languages (CODASYL)** standardised it, several vendors implemented it, which is why people also call it the **CODASYL model**.
- It **generalised** the hierarchical model: in a tree each record has exactly **one parent**; in the network model a record could have **multiple parents**. Example: a single "Greater Seattle Area" region record that every user in that area links to. That made it possible to model many-to-one and many-to-many links.
- Links between records resembled **pointers** in a programming language (although kept on disk), rather than **foreign keys**.

- To get to a record you had no choice but to start at a **root record** and walk along chains of links; that route is the **access path**.

How a CODASYL query worked:

1. Start from a root record.
2. Move a **cursor** through the database, iterating over lists of records and following links (in the simplest case like walking a linked list until you find the record you want).
3. With many-to-many relationships, several different paths could lead to the same record, and the programmer had to track them all mentally. If a record had several parents (incoming pointers), the application code had to track all those relationships. CODASYL committee members themselves likened this to **navigating an n-dimensional data space**.

Trade-off. Hand-chosen access paths used the very limited 1970s hardware efficiently (e.g. **tape drives**, whose seeks are extremely slow). But query and update code became **complicated and inflexible**. With both hierarchical and network models, if you lacked a path to the data you needed, you were stuck. Changing access paths meant rewriting lots of hand-written query code, so changing the data model was hard.

The relational model

What it did differently: put all data **in the open**. A relation is just a collection of tuples. No labyrinth of nested structures, no access paths to follow.

- Read any or all rows of a table, **selecting those matching an arbitrary condition**.
- Fetch a specific row by designating certain columns as a **key** and looking it up.
- **Add a row to any table** without caring about foreign-key links pointing in or out.

> **Foreign key constraints aside.** Constraints can restrict modifications, but the relational model does not require them. Even when constraints exist, joining on foreign keys is performed **when you query**; CODASYL in effect did the join **when you inserted**.

The query optimizer. Relational systems have a **query optimizer** that automatically picks the execution order of a query's parts and the indexes to use. Those choices *are* the access path, but they are made by the optimizer, not the developer, so we rarely think about them.

- Want to query data in a new way? **Declare a new index**; queries automatically use the most suitable indexes, and you do not change the queries. That made adding features much easier.
- Query optimizers are complicated and have consumed many years of R&D. The key insight: **build the optimizer once, and every application benefits**. Without one, hand-coding access paths for one query is easier than writing a general optimizer, yet over time the **general-purpose approach comes out ahead**.

Comparison to document databases

- Document databases went **back to the hierarchical model in one respect**: they store nested one-to-many records (positions, education, contact_info) **inside the parent** record instead of separate tables.
- For **many-to-one and many-to-many**, the two kinds of database are **basically alike**: both point at the related item using a unique ID, a **foreign key** (relational) or **document reference** (document model). That ID gets resolved **when reading**, via a join or extra queries.
- So far, document databases have **not** followed CODASYL's path.

Relational Versus Document Databases Today

Many differences exist (fault tolerance, Chapter 5; concurrency, Chapter 7), but this chapter compares only the **data models**.

	Document model	Relational model
Main arguments for	Schema flexibility; better performance from locality; closer to app data structures for some apps	Stronger support for joins and for many-to-one / many-to-many links

Which data model leads to simpler application code?

- If your data is **document-like** (a tree of one-to-many relationships, usually loaded all at once), use a **document model**. Relational **shredding** (splitting such a structure across several tables like positions, education, contact_info) can produce clumsy schemas and needlessly complex code.
- **Document limitations:** you cannot refer directly to a nested item; you say something like "the second item in user 251's positions list" (much like a hierarchical access path). Usually fine if documents are **not too deeply nested**.
- **Weak joins** may or may not matter. An **analytics app** recording which events happened when may never need many-to-many relationships.
- If your app **does** need many-to-many relationships, documents get less attractive. You can **denormalize** to reduce joins, but then the app must keep the copies consistent. You can **emulate joins** with several requests, but that moves complexity into the app and is usually **slower** than a join done by specialised code inside the database. The result can be significantly more complex code and worse performance.
- **Bottom line:** it depends on the relationships between data items. For **highly interconnected** data, documents are awkward, relational is acceptable, and **graph models are most natural**.

Schema flexibility in the document model

- Most document databases do **not enforce any schema** on document contents, and neither does the JSON support in relational databases. XML support in relational databases usually offers **optional** schema validation.
- No schema means you can add arbitrary keys and values, and readers get **no guarantee** about which fields exist.

> **Common confusion: "schemaless".** It is misleading. The reading code usually assumes some structure, so there is an **implicit schema** that the database does not enforce. Better terms:

	Schema-on-read	Schema-on-write
Structure	Implicit; interpreted when data is read	Explicit; the database checks all written data conforms
Typical of	Document databases	Traditional relational databases
Programming analogy	**Dynamic (runtime) type checking**	**Static (compile-time) type checking**

As with static vs dynamic typing, there is a big debate and **no universally right answer**.

Where the difference shows: changing the data format. The book's scenario: user names are kept as one full-name field, and you decide to split them into first name and last name. Our own variant: splitting a `location` string like `"Pune, India"` into a separate `city` field.

Document database (schema-on-read): start writing new documents with the new field and handle old ones in code.

```
if (member && member.location && !member.city) {
    // Records created before the 2021 redesign lack a "city" field
    member.city = member.location.split(",")[0].trim();
}
```

- Line 1: only act if this is an old-style record (has `location`, lacks `city`).
- Line 3: derive `city` on the fly when reading.

Relational database (schema-on-write): run a **migration**.

```
ALTER TABLE members ADD COLUMN city text;                                    -- new column, NULL by default
UPDATE members SET city = split_part(location, ',', 1);                      -- PostgreSQL
UPDATE members SET city = substring_index(location, ',', 1);                 -- MySQL
```

- `ALTER TABLE` adds the column.
- `UPDATE` fills it for every existing row (the function name differs by database).

Are schema changes slow? Their reputation for slowness and downtime is **not entirely deserved**:

- Most relational databases run `ALTER TABLE` in **a few milliseconds**.
- **MySQL is the notable exception**: `ALTER TABLE` makes it **copy the entire table**, so altering a big table can cost minutes or even hours of downtime (various tools work around this).
- Running `UPDATE` on a large table is **slow on any database**, since every row is rewritten. If that is unacceptable, leave the new column as its default `NULL` and populate it when rows are read, as a document database would.

When schema-on-read is better: when items in a collection do **not all share a structure** (heterogeneous data), for example because:

- you have **lots of object types** and giving each its own table is impractical; or
- the structure is dictated by **external systems** you do not control, which may change at any time.

Then a schema may hurt more than help. But when all records are expected to share a structure, a schema is a useful way to **document and enforce** it. (Schemas and schema evolution are covered in Chapter 4.)

Data locality for queries

- A document is normally saved as **a single contiguous string** in JSON, XML, or a binary form of them such as **MongoDB's BSON**.
- If the app often needs the **whole document** (e.g. to render a web page), this **storage locality** is a performance win. In a multi-table layout like Figure 2-1, you need several index lookups, possibly more disk seeks and more time.

Limitations:

- The locality benefit exists only when you read **big portions** of the document together. The database usually loads the **entire document** even for a small piece, which is wasteful for big documents.
- **Updates** usually rewrite the whole document; only changes that **do not alter the encoded size** are easy to do in place.
- So the usual advice: keep documents **small-ish** and avoid writes that make them **larger**. These limits significantly narrow where document databases are useful.

Locality is not unique to documents:

System	Locality feature
Google's **Spanner**	Relational model where the schema can declare that a table's rows are **interleaved (nested)** inside a parent table
Oracle	**Multi-table index cluster tables**
The **Bigtable** model (as used by **Cassandra** and **HBase**)	**Column families** (the column-family data model), which serve to control locality

(Chapter 3 says more about locality.)

Convergence of document and relational databases

- Since the **mid-2000s**, most relational databases **except MySQL** have offered **XML support**, including local modifications of XML documents and indexing/querying inside them, giving apps a model very close to a document database.
- **JSON support** at a similar level: **PostgreSQL since 9.3**, **MySQL since 5.7**, **IBM DB2 since 10.5**. Given JSON's popularity in web APIs, others will likely follow.
- From the document side: **RethinkDB**'s query language can do joins much like a relational database, and certain **MongoDB drivers** resolve database references for you automatically. That is a **client-side join**, likely slower than an in-database join because of extra network round-trips and less optimisation.

> **Codd's nonsimple domains.** Codd's original 1970 description actually allowed something very like JSON documents inside a relational schema. He called them **nonsimple domains**: a value in a row need not be a primitive like a number or string, but could itself be a nested relation (table), allowing arbitrarily nested trees, much like the XML/JSON support SQL gained over 30 years later.

Takeaway. The two families are **converging**, which is a good thing: the models complement each other. A **hybrid** that handles document-like data and also supports relational queries is a good direction for future databases.

Query Languages for Data

Imperative vs declarative. When the relational model arrived, it brought a new way to query: **SQL is declarative**, whereas **IMS and CODASYL** were queried with **imperative code**.

Our own example. Given a list of fruits, return only the citrus ones. Imperative JavaScript:

```
function getCitrus() {
    var result = [];
    for (var i = 0; i < fruits.length; i++) {        // walk the list in order
        if (fruits[i].group === "Citrus") {         // test each item
            result.push(fruits[i]);                 // collect matches
        }
    }
    return result;
}
```

In **relational algebra** you would write the same thing as a selection:

```
citrus = σ group = "Citrus" (fruits)
```

where **σ (sigma)** is the **selection operator**, returning only items matching the condition. SQL was defined to follow relational algebra quite closely:

```
SELECT * FROM fruits WHERE "group" = 'Citrus';   -- "group" is quoted because it is a SQL keyword
```

	Imperative	Declarative
You specify	Which operations, in which order (loop, test, update variables)	Only the pattern of the result: conditions, and how to transform (sort, group, aggregate)
Who picks the algorithm	You	The database's **query optimizer** (indexes, join methods, execution order)

Why declarative wins:

1. **Concise and easier** to work with than an imperative API.
2. **Hides engine internals**, so the database can improve performance **without changing queries**. Example: the imperative loop visits fruits in some order. Suppose the database decides to free up disk space by shuffling records around behind the scenes, which changes that order; it cannot tell whether some imperative code depends on it. SQL promises **no particular order**, so reordering is safe. Being **more limited** gives the database more room for automatic optimisation.
3. **Parallel execution.** CPUs now get faster by adding **cores**, not by much higher clock speeds. Imperative code fixes an order of instructions and is very hard to parallelise across cores and machines. Declarative queries describe only the shape of the answer, leaving the database free to execute them **in parallel** where that helps.

Declarative Queries on the Web

The advantage is not limited to databases. The book compares approaches in a **web browser**, using a website about ocean animals where the user is on the "Sharks" page. Our own version: a site about planets, user viewing "Mars".

```
<ul>
    <li class="selected">          <!-- the current page's menu item -->
        <p>Mars</p>                <!-- title of the selected page -->
        <ul>
            <li>Olympus Mons</li>
            <li>Valles Marineris</li>
        </ul>
    </li>
    <li>
        <p>Venus</p>
        <ul>
            <li>Maxwell Montes</li>
        </ul>
    </li>
</ul>
```

Goal: give the selected page's title a blue background.

Declarative with CSS:

```
li.selected > p {           /* any <p> whose direct parent is <li class="selected"> */
    background-color: blue;
}
```

The **selector** `li.selected > p` states the pattern; `<p>Mars</p>` matches, `<p>Venus</p>` does not (its `<li>` lacks the class).

Declarative with XSL: the **XPath** expression `li[@class='selected']/p` means the same as that CSS selector.

```
<xsl:template match="li[@class='selected']/p">   <!-- same pattern in XPath -->
    <fo:block background-color="blue">
        <xsl:apply-templates/>                   <!-- keep processing children -->
    </fo:block>
</xsl:template>
```

Imperative with the JavaScript DOM API:

```
var items = document.getElementsByTagName("li");
for (var i = 0; i < items.length; i++) {
    if (items[i].className === "selected") {                    // find selected <li>
        var kids = items[i].childNodes;
        for (var j = 0; j < kids.length; j++) {
            var k = kids[j];
            if (k.nodeType === Node.ELEMENT_NODE && k.tagName === "P") {
                k.setAttribute("style", "background-color: blue"); // colour it
            }
        }
    }
}
```

Problems with the imperative version (beyond being longer and harder to read):

- If the `selected` class is **removed** (the user clicks another page), the blue stays, even if the code runs again, until the page reloads. With CSS the browser **notices automatically** when the rule stops applying and removes the colour.
- To use a newer, possibly faster API (like `document.getElementsByClassName("selected")` or `document.evaluate()`), you must **rewrite the code**. Browser vendors, by contrast, can speed up CSS and XPath without breaking compatibility.

Conclusion. In browsers, declarative CSS beats imperative style manipulation; likewise in databases, declarative SQL turned out much better than imperative query APIs.

> **Historical note.** Both IMS and CODASYL offered only imperative query APIs; programs were typically written in **COBOL** and stepped through records one by one.

MapReduce Querying

What it is. **MapReduce**, made popular by **Google**, is a programming model for bulk processing of big datasets spread over many machines. A few NoSQL stores, such as **MongoDB** and **CouchDB**, offer a **restricted version** of it for running **read-only queries over many documents**. (MapReduce in general is covered in Chapter 10.)

Where it sits. Neither fully declarative nor fully imperative, but **in between**: the query logic is written as **snippets of code** that the framework calls repeatedly. It is based on the **map** (also called *collect*) and **reduce** (also called *fold* or *inject*) functions from functional programming.

The book's example: a marine biologist records each ocean animal sighting and wants a report of **how many sharks were seen per month**. Our own parallel: a birdwatcher counting **herons per month**.

In PostgreSQL (declarative):

```
SELECT date_trunc('month', seen_at) AS sighting_month,    -- round timestamp down to month start
       sum(bird_count)              AS total_birds        -- add up birds per month
FROM sightings
WHERE family = 'Herons'                                   -- only herons
GROUP BY sighting_month;                                  -- one row per month
```

- `date_trunc('month', timestamp)` finds the calendar month containing the timestamp and returns a timestamp for that month's start (rounds down).
- Steps: **filter** to herons, **group** by month, **sum** the counts.

In MongoDB's MapReduce:

```
db.sightings.mapReduce(
    function map() {                                    // called once per matching document
        var y = this.seenAt.getFullYear();
        var m = this.seenAt.getMonth() + 1;             // JS months start at 0
        emit(y + "-" + m, this.birdCount);              // key = "year-month", value = count
    },
    function reduce(key, values) {                      // called once per distinct key
        return Array.sum(values);                       // total birds that month
    },
    {
        query: { family: "Herons" },                    // declarative filter (MongoDB extension)
        out: "monthlyHeronReport"                       // collection to write results into
    }
);
```

How it runs, step by step:

1. The `query` filter selects matching documents declaratively (a MongoDB-specific extension to MapReduce).
2. `map` is called **once per matching document**, with `this` bound to that document.
3. `map` **emits** a key (a year-month string such as `"2013-12"` or `"2014-1"`) together with a value (the animal count of that observation).
4. Emitted pairs are **grouped by key**; `reduce` is called **once per key**.
5. `reduce` adds up the counts for that month.
6. Output is written to the named collection (in the book, `monthlySharkReport`).

Worked example from the book: two shark observations, on **25 Dec 1995** (3 animals, *Carcharodon carcharias*) and **12 Dec 1995** (4 animals, *Carcharias taurus*). `map` runs twice, giving `emit("1995-12", 3)` and then `emit("1995-12", 4)`; afterwards the call `reduce("1995-12", [3, 4])` yields **7**.

Restrictions: pure functions. `map` and `reduce` may only use their inputs, may **not** run additional database queries, and must have **no side effects**. That freedom lets the database execute them **on any machine, in any sequence, and retry them after a failure**. They are still powerful: they can parse strings, call libraries, calculate, and more.

Clearing up confusions:

- MapReduce is a fairly **low-level** model for distributed execution on a cluster. Higher-level languages like SQL **can** be implemented as pipelines of MapReduce jobs (Chapter 10), but many distributed SQL implementations do **not** use MapReduce.
- Nothing restricts SQL to a single machine, and MapReduce has **no monopoly** on distributed query execution.
- Running JavaScript inside a query is great for advanced queries, but not unique to MapReduce: some **SQL databases also accept JavaScript extensions** (user functions).

Usability problem and the fix. You must write **two carefully coordinated** functions, often harder than one query, and a declarative language gives the optimizer more room. So **MongoDB 2.2** added a declarative **aggregation pipeline**:

```
db.sightings.aggregate([
    { $match: { family: "Herons" } },                    // filter, like WHERE
    { $group: {                                          // group, like GROUP BY
        _id: {
            year:  { $year:  "$seenAt" },
            month: { $month: "$seenAt" }
        },
        totalBirds: { $sum: "$birdCount" }               // aggregate, like sum()
    } }
]);
```

Its expressiveness is similar to a **subset of SQL**, but with **JSON-based syntax** instead of SQL's English-sentence style (arguably a matter of taste). The moral: a NoSQL system may end up **accidentally reinventing SQL**, in disguise.

Graph-Like Data Models

When to use graphs. Many-to-many relationships are the key differentiator between models. Mostly one-to-many (trees) or no relationships: **documents** fit. Some simple many-to-many: **relational** copes. **Very common and complex** connections: model data as a **graph**.

What a graph is. Two kinds of objects: **vertices** (also *nodes* or *entities*) and **edges** (also *relationships* or *arcs*). Typical examples:

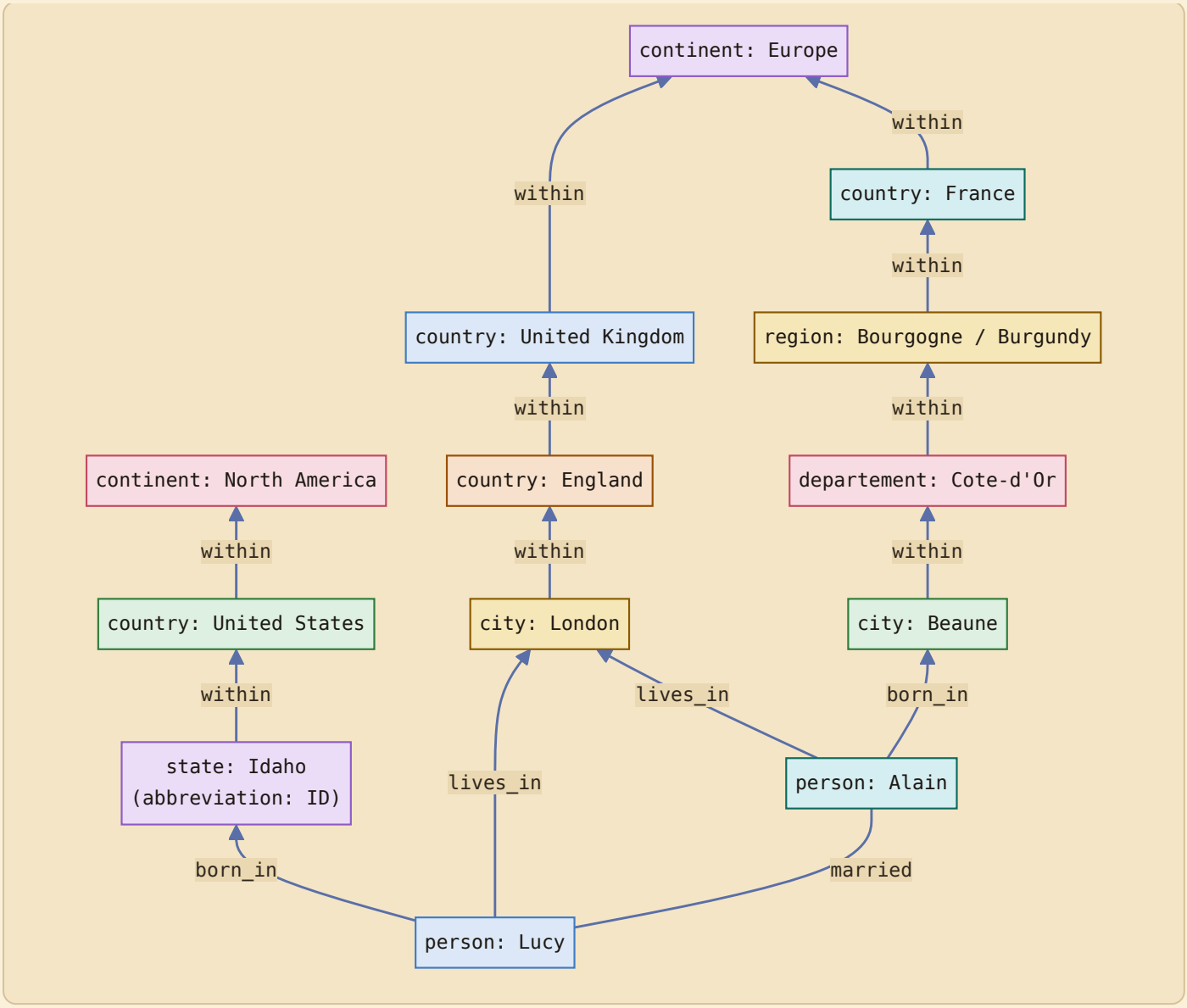
Graph	Vertices	Edges
Social graphs	People	Who knows whom
The web graph	Web pages	HTML links
Road or rail networks	Junctions	Roads or railway lines

Well-known algorithms run on these: **car navigation** finds shortest paths in road networks; **PageRank** on the web graph estimates page popularity and hence search ranking.

Heterogeneous graphs. Vertices need not all be the same kind. Graphs also give a consistent way to store **completely different types of objects** in one datastore. **Facebook** keeps a single graph whose vertices include **people, locations, events, checkins, and user comments**, and whose edges record **who is friends with whom, where each checkin took place, who commented on what post, who went to which event**, and more.

The book's running example (Figure 2-5). It might belong to a social network or a genealogy database: **Lucy, originally from Idaho**, and **Alain, originally from Beaune in France**, who are **married and living in London**.

Figure 2-5 (redrawn): graph-structured data. Boxes are vertices (each with a type and a name), arrows are edges. Note that France's hierarchy (city, département, région, country) differs from the US one (state, country).



Ways to structure and query graphs covered:

	Models	Implementations
Property graph	vertices + edges with properties	**Neo4j, Titan, InfiniteGraph**
Triple-store	(subject, predicate, object)	**Datomic, AllegroGraph**, others

Declarative graph query languages covered: **Cypher, SPARQL, Datalog**. Also exist: **imperative** graph query languages such as **Gremlin**, and graph processing frameworks like **Pregel** (Chapter 10).

Property Graphs

Structure.

Each vertex has	Each edge has
A unique identifier	A unique identifier
A set of outgoing edges	The **tail vertex** (where the edge starts)
A set of incoming edges	The **head vertex** (where the edge ends)
A collection of properties (key-value pairs)	A **label** describing the relationship
	A collection of properties (key-value pairs)

As relational tables. A graph store can be imagined as a pair of relational tables: a vertex table and an edge table. The book's Example 2-2 uses PostgreSQL's `json` type for properties. Our own sketch of the same idea (different names, same structure):

```
CREATE TABLE nodes (
    node_id  integer PRIMARY KEY,                    -- unique vertex ID
    attrs    json                                    -- key-value properties
);

CREATE TABLE links (
    link_id  integer PRIMARY KEY,                    -- unique edge ID
    from_node integer REFERENCES nodes (node_id),    -- tail vertex
    to_node   integer REFERENCES nodes (node_id),    -- head vertex
    kind     text,                                   -- the edge label
    attrs    json                                    -- edge properties
);

CREATE INDEX links_from ON links (from_node);        -- fast outgoing-edge lookups
CREATE INDEX links_to   ON links (to_node);          -- fast incoming-edge lookups
```

To get a vertex's **outgoing** edges, query by tail vertex; for **incoming** edges, query by head vertex.

Important properties of this model:

1. **Any vertex can connect to any other.** No schema restricts which kinds of things may be associated.
2. From any vertex you can efficiently find **both incoming and outgoing** edges, so you can **traverse** the graph forwards and backwards. (Hence indexes on both columns.)
3. Using **different labels** for different relationships lets one graph hold many kinds of information while staying a clean model.

Why this flexibility matters (Figure 2-5 again). Things that are hard in a traditional relational schema become easy:

- **Different regional structures per country**: France has *départements* and *régions*, the US has *counties* and *states*.
- **Quirks of history**, such as a **country inside another country** (England inside the United Kingdom), setting aside the subtleties of sovereign states versus nations.
- **Varying granularity**: Lucy's current residence is a **city**, but her birthplace is given only as a **state**.

Evolvability. You could extend the graph with more facts. The book's example: **food allergies**. Add a vertex per allergen, an edge from a person to each allergen they react to, and link allergens to vertices showing which foods contain which substances. Then a query can work out which foods are **safe for each person**. Graphs extend easily as features are added.

The Cypher Query Language

What it is. **Cypher** is a **declarative** language for querying property graphs, originally made for the **Neo4j** graph database. Its name comes from a character in the film *The Matrix* and has nothing to do with cryptographic ciphers.

Inserting data. The book's Example 2-3 inserts the left-hand part of Figure 2-5 (North America, United States, Idaho, Lucy, with WITHIN and BORN_IN edges). Each vertex gets a **symbolic name** that other parts of the query use, and arrows create edges: `(Idaho) -[:WITHIN]-> (USA)` makes an edge labelled WITHIN with Idaho as the **tail** and USA as the **head**. Our own example with different data:

```
CREATE
  (Asia:Location    {name:'Asia',      type:'continent'}),   // vertex with label + properties
  (India:Location   {name:'India',     type:'country'}),
  (Kerala:Location  {name:'Kerala',    type:'state'}),
  (Priya:Person     {name:'Priya'}),
  (Kerala) -[:WITHIN]-> (India) -[:WITHIN]-> (Asia),         // chain of WITHIN edges
  (Priya)  -[:BORN_IN]-> (Kerala)                            // Priya's birthplace
```

Querying. The book's question: **find the names of everyone who emigrated from the United States to Europe**, i.e. vertices with a `BORN_IN` edge to a place within the US **and** a `LIVES_IN` edge to a place within Europe, returning each one's `name`. Our own version: people **born in India who now live in Canada**.

```
MATCH
  (p) -[:BORN_IN]->  () -[:WITHIN*0..]-> (:Location {name:'India'}),
  (p) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (:Location {name:'Canada'})
RETURN p.name
```

How to read it:

1. `(p) -[:BORN_IN]-> ()` matches any two vertices joined by a BORN_IN edge; the tail is bound to variable `p`, the head is left unnamed.
2. `-[:WITHIN*0..]->` means **traverse WITHIN edges any number of times, including zero** (like `*` in a regular expression), until reaching a Location whose name is India.
3. The second line requires the **same** `p` to also have a LIVES_IN edge leading, via zero or more WITHIN edges, to Canada.
4. Return the `name` property of each such `p`.

Execution is the optimizer's job. Possible strategies:

- **Forward:** scan all people, check each one's birthplace and residence, keep matches.
- **Backward:** use an index on `name` to find the two Location vertices (in the book, US and Europe), follow all **incoming** WITHIN edges to collect every location inside each, then find people reachable through incoming BORN_IN or LIVES_IN edges.

As usual with declarative languages, you do not choose; the optimizer picks the strategy it predicts is most efficient.

Graph Queries in SQL

Can SQL query graph data stored in tables? Yes, **with some difficulty**.

- In typical relational queries you **know in advance which joins** you need.
- In graph queries you may traverse a **variable number of edges** to reach the target, so the **number of joins is not fixed**.
- In the example, a LIVES_IN edge could lead to any kind of place: street, city, district, region, state and so on. Cities sit WITHIN regions, regions WITHIN states, states WITHIN countries. The place you want might be directly at the end of the edge or several levels up the hierarchy.

The SQL tool: since **SQL:1999**, variable-length traversal can be written with **recursive common table expressions** (`WITH RECURSIVE`), which **PostgreSQL, Oracle, IBM DB2 and SQL Server** support. It works but is **very clumsy** compared with Cypher. Our own version of the India→Canada query over the `nodes`/`links` tables above:

```
WITH RECURSIVE
  in_india(node_id) AS (                                  -- all places inside India
      SELECT node_id FROM nodes WHERE attrs->>'name' = 'India'
    UNION
      SELECT links.from_node FROM links
        JOIN in_india ON links.to_node = in_india.node_id
        WHERE links.kind = 'within'
  ),
  in_canada(node_id) AS (                                 -- all places inside Canada
      SELECT node_id FROM nodes WHERE attrs->>'name' = 'Canada'
    UNION
      SELECT links.from_node FROM links
        JOIN in_canada ON links.to_node = in_canada.node_id
        WHERE links.kind = 'within'
  ),
  born_in_india(node_id) AS (                             -- people born somewhere in India
    SELECT links.from_node FROM links
      JOIN in_india ON links.to_node = in_india.node_id
      WHERE links.kind = 'born_in'
  ),
  lives_in_canada(node_id) AS (                           -- people living somewhere in Canada
    SELECT links.from_node FROM links
      JOIN in_canada ON links.to_node = in_canada.node_id
      WHERE links.kind = 'lives_in'
  )
SELECT nodes.attrs->>'name'
FROM nodes
JOIN born_in_india   ON nodes.node_id = born_in_india.node_id     -- intersect the
JOIN lives_in_canada ON nodes.node_id = lives_in_canada.node_id;  -- two sets
```

What it does, step by step:

1. Find the vertex named India; it seeds the set `in_india`.
2. Repeatedly follow **incoming** `within` edges from vertices already in the set, adding their tails, until no new ones appear.
3. Do the same from Canada to build `in_canada`.
4. From every vertex in `in_india`, walk incoming `born_in` edges to collect everyone born there.
5. From every vertex in `in_canada`, follow incoming `lives_in` edges to find residents.
6. **Join** (intersect) the two sets of people.

Lesson. The book notes the same query takes **4 lines** in one language and **29 lines** in another. That shows different data models are built for different use cases; pick one that suits your application.

Triple-Stores and SPARQL

`What it is.` The **triple-store** model says **nearly the same thing** as the property graph, just in different words. Still worth learning for its tools and languages.

Triples. All information is stored as three-part statements: **(subject, predicate, object)**. In the book's (*Jim, likes, bananas*), *Jim* plays subject, *likes* plays predicate (the verb), and *bananas* plays object.

- The **subject** is like a **vertex**.
- The **object** is either:
  1. **A primitive value** (string, number). Then predicate + object act like a **property key and value** on the subject. Book example: *(lucy, age, 33)* is like vertex `lucy` with properties `{"age":33}`.
  2. **Another vertex.** Then the predicate is an **edge**: subject = tail, object = head. Book example: *(lucy, marriedTo, alain)*, where `marriedTo` is the edge label.

Turtle format. The book writes Figure 2-5's data as triples in **Turtle**, a subset of **Notation3 (N3)**. Our own data:

```
@prefix : <urn:example:>.
_:priya   a       :Person.          # "a" means "is of type"
_:priya   :name   "Priya".          # property: object is a literal
_:priya   :bornIn _:kerala.         # edge: object is another vertex
_:kerala  a       :Location.
_:kerala  :name   "Kerala".
_:kerala  :type   "state".
_:kerala  :within _:india.
_:india   a       :Location.
_:india   :name   "India".
_:india   :type   "country".
_:india   :within _:asia.
_:asia    a       :Location.
_:asia    :name   "Asia".
_:asia    :type   "continent".
```

- Vertices are written `_:someName`. The name means nothing outside the file; it only tells us which triples refer to the same vertex.
- When the predicate is an **edge**, the object is a vertex (`_:kerala :within _:india`). When it is a **property**, the object is a literal string value (`_:india :name "India"`).

Repeating the subject is tedious, so **semicolons** let you say several things about one subject:

```
@prefix : <urn:example:>.
_:priya  a :Person;   :name "Priya";  :bornIn _:kerala.
_:kerala a :Location; :name "Kerala"; :type "state";     :within _:india.
_:india  a :Location; :name "India";  :type "country";   :within _:asia.
_:asia   a :Location; :name "Asia";   :type "continent".
```

The semantic web

- The triple-store model is **completely independent** of the semantic web. **Datomic** is a triple-store that claims no link to it. But many people associate the two, so the book covers it briefly.
- **The idea:** websites already publish text and pictures for humans; why not also publish **machine-readable data** for computers? The **Resource Description Framework (RDF)** was meant to let sites publish data in a consistent format so data from different sites could be combined automatically into a **"web of data"**, an internet-wide "database of everything".
- **Reality:** it was **overhyped in the early 2000s**, has shown no sign of being realised in practice, and made many people cynical. It was also weighed down by endless acronyms, over-complicated standards proposals, and hubris.
- **But** the project produced useful work: **triples make a fine internal data model** for an application, even one with no interest in publishing RDF.

> **Datomic aside.** Technically Datomic stores **5-tuples**, not triples; the two extra fields are **metadata for versioning**.

The RDF data model

- Turtle is a **human-readable** RDF format. RDF can also be written in **XML**, which is much more verbose. Turtle/N3 is far more readable, and tools such as **Apache Jena** can translate between RDF formats automatically when needed.

Our Kerala data in RDF/XML (compare how much longer it is):

```
<rdf:RDF xmlns="urn:example:" xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
  <Location rdf:nodeID="kerala">
    <name>Kerala</name>
    <type>state</type>
    <within>
      <Location rdf:nodeID="india">
        <name>India</name>
        <type>country</type>
      </Location>
    </within>
  </Location>
  <Person rdf:nodeID="priya">
    <name>Priya</name>
    <bornIn rdf:nodeID="kerala"/>
  </Person>
</rdf:RDF>
```

RDF quirks from being built for internet-wide exchange:

- Subjects, predicates and objects are often **URIs**. A predicate might be `<http://my-company.com/namespace#within>` rather than just `WITHIN`.
- **Why:** so you can merge your data with someone else's. If they mean something different by *within* or *lives_in*, there is no clash, because theirs would be, say, `<http://other.org/foo#within>`.
- The namespace URL **need not resolve** to anything; to RDF it is just a namespace. To avoid confusion with real `http://` URLs, the book's examples use non-resolvable URIs like `urn:example:within`. You write the prefix a **single time** at the top and then stop thinking about it.

The SPARQL query language

- **SPARQL** is the query language for RDF triple-stores. Its expansion is **SPARQL Protocol and RDF Query Language** (the book calls it an acronym), and you say it like **"sparkle"**.
- It **predates Cypher**; Cypher's pattern matching was **borrowed from SPARQL**, so they look alike.
- The US-to-Europe query is **even more concise** in SPARQL than in Cypher. Our India-to-Canada version:

```
PREFIX : <urn:example:>

SELECT ?who WHERE {
  ?p :name ?who.                                # bind the person's name
  ?p :bornIn  / :within* / :name "India".       # born somewhere inside India
  ?p :livesIn / :within* / :name "Canada".      # living somewhere inside Canada
}
```

- Variables start with `?`.
- `/` chains predicates into a path; `:within*` means follow `within` zero or more times.

Side by side:

```
(p) -[:BORN_IN]-> () -[:WITHIN*0..]-> (place)    # Cypher
?p :bornIn / :within* ?place.                    # SPARQL

(ind {name:'India'})                             # Cypher: vertex with a name property
?ind :name "India".                              # SPARQL: same thing
```

Because RDF uses predicates for **both properties and edges**, the same syntax matches properties too. SPARQL is a good query language, useful internally even if the semantic web never happens.

Graph Databases Compared to the Network Model (sidebar). Are graph databases just CODASYL again? **No**, in several important ways:

CODASYL (network model)	Graph databases
A schema dictated which record type could nest within which	No such restriction: any vertex can link to any other, so apps adapt more easily
The only way to reach a record was to traverse an access path	Refer to any vertex directly by its **unique ID**, or use an **index** to find vertices by value
A record's children were an **ordered set**; the database maintained the order (affecting storage layout) and inserts had to care about positions	Vertices and edges are **unordered**; you sort only when querying
All queries imperative, hard to write, easily broken by schema changes	Imperative traversal is possible, but most support high-level **declarative** languages such as Cypher or SPARQL

The Foundation: Datalog

What it is. **Datalog** is much **older** than SPARQL or Cypher, studied extensively by academics in the **1980s**. Less known among engineers, but important because it is the **foundation** later query languages build on.

Where it is used: it is the query language of **Datomic**, and **Cascalog** implements Datalog for querying big datasets on **Hadoop**. (Datomic and Cascalog use Clojure S-expression syntax; the book uses **Prolog** syntax, which is easier to read and functionally the same.)

Data model. Like triples, generalised a bit: instead of *(subject, predicate, object)* you write **predicate(subject, object)**. Our own facts:

```
name(asia, 'Asia').
type(asia, continent).
name(india, 'India').
type(india, country).
within(india, asia).
name(kerala, 'Kerala').
type(kerala, state).
within(kerala, india).
name(priya, 'Priya').
born_in(priya, kerala).
```

Queries are built from rules. Datalog is a **subset of Prolog**. Our version of the emigration query (Priya born in India, suppose she lives in Toronto within Canada):

```
inside(Place, Name) :- name(Place, Name).                  /* Rule 1 */
inside(Place, Name) :- within(Place, Via),                 /* Rule 2 */
                       inside(Via, Name).

moved(Name, From, To) :- name(Person, Name),               /* Rule 3 */
                         born_in(Person, BirthPlace),
                         inside(BirthPlace, From),
                         lives_in(Person, HomePlace),
                         inside(HomePlace, To).

?- moved(Who, 'India', 'Canada').
/* Who = 'Priya'. */
```

(The book's rules are named `within_recursive` and `migrated`, and the query asks for `'United States'` to `'Europe'`, answering `Who = 'Lucy'`.)

How rules work:

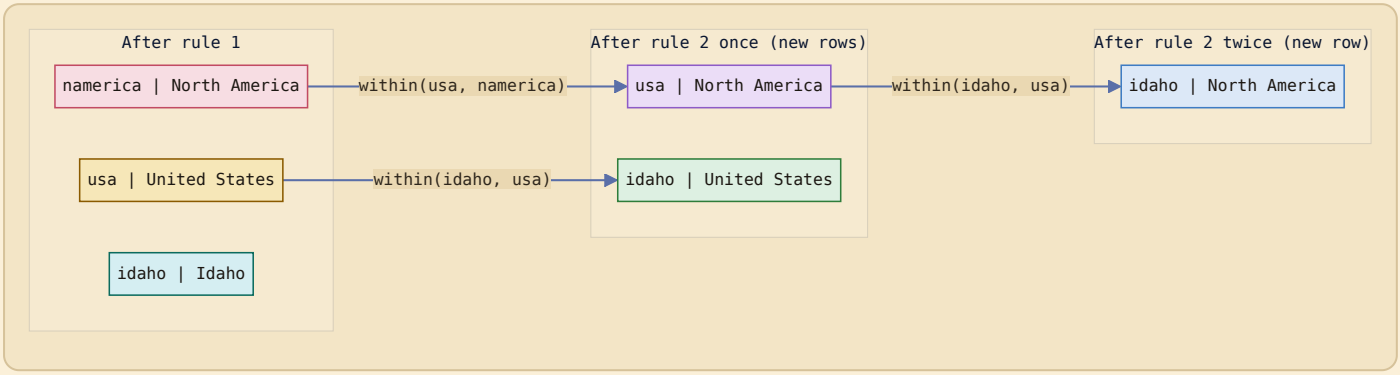
- Cypher and SPARQL jump straight to `SELECT`; Datalog proceeds **a small step at a time**. Rules define **new predicates** (here `inside` and `moved`) that are **not stored** but **derived** from data or other rules.
- Rules can call other rules, or themselves **recursively**, like functions. Complex queries are built piece by piece.
- Words starting with an **uppercase letter are variables**. Predicates match like in Cypher/SPARQL: `name(Place, Name)` matches `name(asia, 'Asia')` with `Place = asia`, `Name = 'Asia'`.
- A rule **applies** if the system finds matches for **all** predicates on the right of `:-`. When it applies, it is as if the left side (with variables substituted) were **added to the database**.

One possible evaluation (our data, mirroring the book's three steps):

1. `name(asia, 'Asia')` exists, so **rule 1** applies and generates `inside(asia, 'Asia')`.
2. `within(india, asia)` exists and step 1 produced `inside(asia, 'Asia')`, so **rule 2** generates `inside(india, 'Asia')`.
3. `within(kerala, india)` exists and step 2 produced `inside(india, 'Asia')`, so **rule 2** generates `inside(kerala, 'Asia')`.

Repeating rules 1 and 2 lets the recursive predicate list every location inside Asia (or any named place).

Figure 2-6 (redrawn): the within_recursive table (Location, Name) as the rules of Example 2-11 are applied. Rule 1 seeds each place with its own name; each pass of rule 2 uses one within fact to copy a name one level down. After two passes of rule 2, (idaho, North America) appears.



Rule 3 then finds people born in some place `BornIn` and living in some place `LivingIn`. Querying with `BornIn = 'United States'`, `LivingIn = 'Europe'` and the person left as variable `Who` asks the system which values `Who` can take, giving the same answer as the Cypher and SPARQL queries.

Trade-off. Datalog needs a different way of thinking, but it is very powerful because **rules can be combined and reused across queries**. It is less handy for simple one-off queries but copes better with **complex data**.

Summary

- **Data models** are a huge subject; this chapter surveyed a broad range to help you find the one that fits.
- **History:** data began as **one big tree** (the **hierarchical model**), which handled **many-to-many** relationships badly, so the **relational model** was invented. Later, some applications did not fit relational well either, and **NoSQL** stores split in two main directions:

1. **Document databases**: data comes in **self-contained documents**, and relationships between documents are **rare**.
2. **Graph databases**: the opposite, where **anything may be related to everything**.

- All three models (document, relational, graph) are widely used, each good in its domain. Any one can be **emulated** using another (graphs stored in relational tables, say), but that tends to be **awkward**. Hence different systems for different purposes rather than one universal solution.
- Document and graph databases usually **do not enforce a schema**, easing adaptation to changing requirements. But the application still assumes some structure; the question is whether the schema is **explicit (on write)** or **implicit (on read)**.
- Every model brings its own query language or framework; the chapter showed **SQL, MapReduce, the MongoDB aggregation pipeline, Cypher, SPARQL and Datalog**. **CSS and XSL/XPath** are not database languages but show interesting parallels.
- **Models not covered**, briefly:
  - **Genome data:** researchers need **sequence-similarity searches**, matching one very long string (a DNA molecule) against a large database of similar but not identical strings. None of the databases here handle this, so specialised software like **GenBank** exists.
  - **Particle physics:** Big Data–style analysis for decades; the **Large Hadron Collider (LHC)**, for instance, now deals with **hundreds of petabytes**, requiring custom solutions to keep hardware costs under control.
  - **Full-text search:** arguably a data model often used beside databases. Information retrieval is a big specialist topic; search indexes appear in Chapter 3 and Part III.
- Next chapter: the trade-offs involved in **implementing** these data models.

Interview angle

1. "Relational or document database — how do you choose?" *30-second answer:* Look at the relationships in your data. If it is a self-contained tree of one-to-many data usually loaded whole (a profile, an order with line items), a document model gives locality and schema flexibility and avoids shredding. If you have many-to-one or many-to-many relationships, relational joins are better; emulating joins in app code is slower and more complex. For highly interconnected data, consider a graph database. Modern systems blur the line (JSON in PostgreSQL/MySQL, joins in RethinkDB).

- *Follow-up: Is a document DB "schemaless"?* Not really: it is schema-on-read. The code still assumes structure; the database just does not enforce it. Schema-on-write is like static typing, schema-on-read like dynamic typing.
- *Follow-up: When is document locality a disadvantage?* When you only need a small part of a large document (the whole thing is loaded), or when updates grow the document (it usually has to be rewritten). Keep documents small.

2. "What is normalization and why store IDs instead of text?" *30-second answer:* Normalization means storing each human-meaningful fact once and referencing it by ID. IDs never need to change, so renames are one update, you avoid inconsistent copies, and you gain consistent spelling, disambiguation, localisation and better search. The cost is needing many-to-one relationships and joins, which document databases handle poorly.

- *Follow-up: When denormalize?* To avoid joins for read performance, accepting the extra write work and the risk of inconsistency.

3. "Why is SQL declarative and why does that matter?" *30-second answer:* You describe the result, not the algorithm; the query optimizer chooses indexes, join methods and order. That makes queries shorter, lets the database improve performance or reorganise storage without breaking queries (no promised order), and makes parallel execution much easier as CPUs gain cores rather than speed.

- *Follow-up: Where does MapReduce fit?* In between: you write map and reduce code snippets (pure functions) that the framework runs. MongoDB added the declarative aggregation pipeline in 2.2 because coordinating two functions is harder and gives the optimizer less room.

4. "When would you use a graph database, and how does it differ from the old network model?" *30-second answer:* When many-to-many relationships are common and complex, and queries traverse a variable number of hops (social networks, the web graph, road networks, heterogeneous data like Facebook's single graph). Unlike CODASYL, graph databases have no schema restricting links, allow direct access by ID or index, have no ordered children, and offer declarative languages like Cypher and SPARQL.

- *Follow-up: Can I do this in SQL?* Yes, with recursive CTEs (`WITH RECURSIVE`, SQL:1999), but it is clumsy: the book's example is 29 lines of SQL versus 4 in Cypher.
- *Follow-up: Property graph vs triple-store?* Mostly equivalent; triples express both properties (object is a value) and edges (object is a vertex) as (subject, predicate, object).

5. "Why did the relational model beat the network model?" *30-second answer:* CODASYL required programmers to navigate access paths through pointers by hand, making code complex and hard to change. The relational model exposed data as plain tables and let a query optimizer choose access paths automatically, so new indexes and queries could be added without rewriting code. Build the optimizer once and every app benefits.

Self-check

1. **What are the four layers of data models the chapter describes, and which one does this chapter focus on?** Application objects/data structures; general-purpose model (JSON/XML, tables, graph); bytes in memory/disk/network; electrical/optical/magnetic hardware. The chapter focuses on the second.
2. **Name the four driving forces behind NoSQL.** Greater scalability (large datasets, high write throughput); preference for free/open source; specialised queries poorly supported by relational; frustration with restrictive schemas and a wish for a more dynamic model.
3. **Why does a document database struggle once résumés gain organisation pages and recommendations?** The new features introduce many-to-one and many-to-many links (references to organisations, schools, other users). Document databases often have weak join support, so you must denormalize or emulate joins in application code.
4. **How is IMS similar to modern document databases?** IMS's hierarchical model stored data as trees of nested records like JSON, handled one-to-many well, made many-to-many hard, and lacked joins, forcing duplication or manual reference resolution.
5. **What was an "access path" in CODASYL, and what replaced it in relational databases?** A chain of pointer links followed from a root record to reach a record, chosen by the programmer. In relational databases the query optimizer chooses execution paths automatically.
6. **Scenario: you need to split a full-name field into a new `first_name` column across 500 million rows in MySQL with no downtime. What does the chapter warn about, and what workaround does it suggest?** MySQL copies the entire table on `ALTER TABLE` (possible hours of downtime; external tools exist), and a mass `UPDATE` is slow on any database. You can add the column with a NULL default and fill values at read time, schema-on-read style.
7. **Why can a database reorder records safely under SQL but not under imperative query code?** SQL does not promise any order, so the database can move records; imperative code might depend on iteration order, and the database cannot tell.
8. **In the book's MapReduce example, what do `map` and `reduce` output for the two December 1995 shark sightings?** Two emits for key `"1995-12"` with values 3 and 4; the reduce call receives `[3, 4]` for that key and returns 7.

9. **Why must map and reduce be pure functions?** So the database can run them anywhere, in any order, and rerun them after failures.
10. **Give three ways graph databases differ from the CODASYL network model.** Any three of: no schema restricting which vertex connects to which; direct access by unique ID or index instead of only via access paths; unordered vertices/edges instead of ordered child sets; declarative query languages instead of only imperative queries.

Chapter 3 — Storage and Retrieval

Book: Chapter 3, pp. 69–110

Overview

Any database has two jobs at its core: accept data and keep it safe, then hand that same data back when someone asks for it later. Chapter 2 looked at this from the application developer's side (which data model and query language you use). This chapter flips the viewpoint and looks *inside* the database: how does a **storage engine** (the part of a database that actually lays bytes out on disk or in memory and finds them again) do its job?

Why should you care if you will never write a storage engine? Because you still have to *pick* one, and you have to *tune* it. Choosing well and tuning well both require a rough mental model of what the engine does under the hood. In particular, engines built for **transactional** workloads (many small reads/writes from users) behave very differently from engines built for **analytics** (a few huge scans by analysts).

The chapter has three big parts:

1. **Indexes for OLTP-style engines** — two families: *log-structured* engines (append-only files, e.g. LSM-trees) and *page-oriented* engines (update-in-place, e.g. B-trees), plus other kinds of indexes and in-memory databases.
2. **OLTP vs. analytics** — why companies build separate *data warehouses*, and how warehouse schemas (star/snowflake) look.
3. **Column-oriented storage** — how analytic engines store data by column, compress it, sort it, write to it, and pre-compute aggregates.

Everyday analogy. Think of a notebook where you jot down every change to your phone contacts, always on the next empty line (a *log*). Writing is instant, but finding someone's latest number means reading the whole notebook backwards. Adding an alphabetical tab index at the back (an *index*) makes lookup fast, but now every new entry also means updating the tabs. A library that wants to answer "how many books per genre per decade?" would rather keep one list per attribute (all genres in one list, all years in another) — that is column storage.

Key terms

Term	Plain-English meaning
Storage engine	The component of a database responsible for storing data on disk/memory and retrieving it.
Log	An append-only sequence of records (not necessarily human-readable text).
Index	An extra structure, derived from the main data, that helps find records quickly. Speeds reads, slows writes.
Key-value store	A database that maps a key to a value, like a dictionary/hash map.
Hash index	An in-memory hash map from each key to the byte offset of its value in a data file.
Byte offset	The position (in bytes) inside a file where a record starts.
Segment	One chunk of the log; a file that is closed once it reaches a certain size.
Compaction	Rewriting a log segment keeping only the latest value for each key.
Merging	Combining several segments into one new segment.
Tombstone	A special record meaning "this key has been deleted".
Checksum	A small value computed from data, used to detect corruption (e.g., half-written records).
SSTable (Sorted String Table)	A segment file whose key-value pairs are sorted by key, each key appearing once.
Memtable	An in-memory balanced tree (e.g., red-black tree) holding recent writes, sorted by key.
LSM-tree (Log-Structured Merge-Tree)	A storage design that keeps a memtable plus a cascade of SSTables merged in the background.
Bloom filter	A compact, memory-cheap data structure that can say "this key is definitely not here".
Size-tiered / leveled compaction	Two strategies for choosing when and which SSTables to merge.
B-tree	The most common index: a balanced tree of fixed-size pages (usually 4 KB) updated in place.
Page	A fixed-size block on disk that a B-tree reads/writes as a unit.
Leaf page	Bottom-level B-tree page containing the actual keys (and values or pointers to them).
Branching factor	How many child-page references one B-tree page holds (typically several hundred).
Write-ahead log (WAL) / redo log	An append-only file where each B-tree change is recorded before the tree pages are modified, for crash recovery.
Latch	A lightweight lock protecting an in-memory/on-disk data structure from concurrent threads.
Copy-on-write	Instead of overwriting a page, write a modified copy elsewhere and update parents to point at it.

Term	Plain-English meaning
B+ tree	B-tree variant where interior pages store abbreviated keys (only enough to separate ranges).
Write amplification	One logical write to the database causing multiple physical writes to disk over time.
Fragmentation	Unused gaps of space inside files/pages.
Primary key	A key that uniquely identifies one row/document/vertex.
Secondary index	An index on a non-unique field, used to find all records with a given value.
Heap file	Place where rows are stored in no particular order; indexes point into it.
Forwarding pointer	A note left at a record's old heap location pointing to its new location.
Clustered index	An index that stores the whole row inside the index itself.
Covering index / index with included columns	Stores some columns in the index so certain queries can be answered from the index alone.
Concatenated index	Multi-column index formed by joining several columns into one key, in a fixed order.
Multi-dimensional index	Index that can filter on several columns at once (e.g., latitude *and* longitude).
R-tree	A specialised spatial index for multi-dimensional range queries.
Space-filling curve	A way of mapping 2D (or more) points to one number so a normal B-tree can be used.
Edit distance	How many single-letter insertions, deletions or substitutions separate two words.
Levenshtein automaton	A state machine that efficiently finds words within a given edit distance.
In-memory database	A database that serves all reads from RAM (may still log to disk for durability).
Anti-caching	Evicting least-recently-used records from an in-memory database to disk and reloading them on access.
NVM (non-volatile memory)	Memory that keeps its contents without power; may change engine design in future.
Transaction	A group of reads and writes treated as one logical unit (need not be ACID here).
OLTP (online transaction processing)	Interactive workload: many small key-based reads/writes from users.
OLAP (online analytic processing)	Analytic workload: few queries that scan huge numbers of rows to compute aggregates.
Business intelligence	Reports and analyses that help management make decisions.
Data warehouse	A separate, read-only-copy database built for analysts.
ETL (Extract–Transform–Load)	The process of copying data from OLTP systems into the warehouse.
Star schema / dimensional modeling	Warehouse layout: a central fact table surrounded by dimension tables.
Fact table	Big central table where each row is one event (e.g., a sale).

Term	Plain-English meaning
Dimension table	Table describing the who/what/where/when/how/why of events (product, store, date…).
Snowflake schema	Star schema whose dimensions are further split into sub-dimensions.
Row-oriented storage	All values of one row stored next to each other.
Column-oriented storage	All values of one column stored together, each column in its own file.
Bitmap encoding	For each distinct value in a column, a bitmap with one bit per row (1 = row has that value).
Run-length encoding	Compressing repeated bits/values by storing "value, count" runs.
Vectorized processing	Running tight loops over chunks of compressed column data that fit in CPU cache.
SIMD	Single-instruction-multiple-data CPU instructions that process many values at once.
Sort key	Column(s) by which a column store orders its rows.
Materialized view	A stored (on-disk) copy of a query's results, kept up to date when data changes.
Virtual view	A saved query definition; expanded on the fly each time it is used.
Data cube / OLAP cube	A grid of precomputed aggregates grouped by several dimensions.

Data Structures That Power Your Database

The world's simplest database. The book opens with a tiny key-value store built from just two **Bash** shell functions (`db_set` and `db_get`), storing values such as JSON documents about cities (London, San Francisco). Here is an equivalent sketch of the same idea with different names:

```
#!/bin/bash
put () {                        # usage: put <key> <value>
  echo "$1,$2" >> store.log     # append "key,value" as a new line at the end of the file
}
fetch () {                      # usage: fetch <key>
  grep "^$1," store.log |       # find every line whose key matches
    sed -e "s/^$1,//" |         # strip the "key," prefix, leaving only the value
    tail -n 1                   # keep only the LAST match = the newest value
}
```

- `put` never edits old lines; it only adds new ones at the end. If a key is updated three times, all three versions stay in the file.
- `fetch` has to take the **last** matching line, because the newest write wins.

For example, `put 7 '{"city":"Oslo"}'` then `put 7 '{"city":"Bergen"}'` leaves two lines for key 7; `fetch 7` returns the Bergen version.

Why this is half good, half terrible:

- **Writes are fast.** Appending to the end of a file is about the cheapest disk operation there is. Many real databases use exactly this internally: a **log**.
- **Reads are slow.** `fetch` scans the whole file every time. Cost is **O(n)**: double the records, double the lookup time.

> **What "log" means in this book:** not the text messages an app prints for debugging, but any *append-only sequence of records*. It may be binary and meant only for programs to read.

Real databases must also deal with concurrency, stopping the log from growing forever, and crashes that leave half-written records — but the basic principle stays the same.

Indexes. To find a key without scanning everything, we keep extra metadata on the side that acts like a signpost: an **index**. Important properties:

1. An index is **derived** from the primary data. Adding or dropping one never changes the data, only query speed.
2. Every index must be updated on every write, so **indexes slow down writes**. Nothing beats a plain append for write speed.
3. Hence the core **trade-off**: good indexes speed up reads, but each one costs write performance. That is why databases don't index everything automatically; the developer or DBA picks indexes based on the application's actual query patterns.

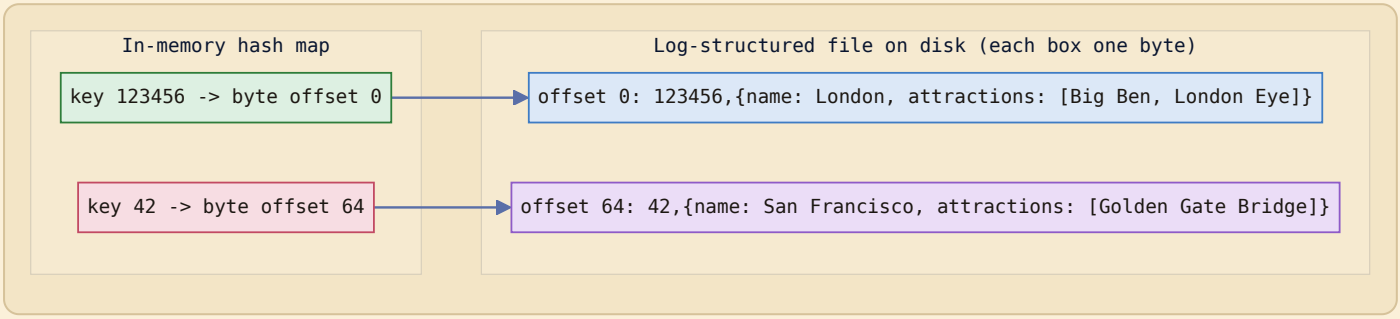
Hash Indexes

What it is. The simplest index for key-value data: keep an **in-memory hash map** (the same dictionary type found in most programming languages) where each key maps to the **byte offset** in the data file where its latest value begins.

How it works:

1. **Write:** append `key,value` to the end of the data file, then set `hashmap[key] = offset_of_that_record`. (Works for both new keys and updates.)
2. **Read:** look up the key in the hash map to get the offset, seek straight to that position in the file, and read the value.

Figure 3-1 (redrawn): an append-only CSV-like log of key-value records, with an in-memory hash map pointing at the byte offset where each key's record starts



Real system: this is essentially **Bitcask**, the default storage engine of **Riak**. It gives high-performance reads and writes **as long as all keys fit in RAM**. Values can be larger than memory, since reading one costs only a single disk seek (and zero disk I/O if that part of the file is already in the operating system's filesystem cache).

Good fit: workloads with few distinct keys but many updates per key. Book example: key = URL of a cat video, value = play count, incremented every time someone presses play.

Problem: the log grows forever. Solution:

1. Split the log into **segments**: when a segment file reaches a certain size, close it and start writing to a new one.
2. Run **compaction** on closed segments: throw away older duplicates, keeping only the most recent value per key.

3. Because compaction usually shrinks segments a lot, **merge** several segments together while compacting.

Figure 3-2 (redrawn): compacting one segment of a cat-video play-count log keeps only the newest value per key

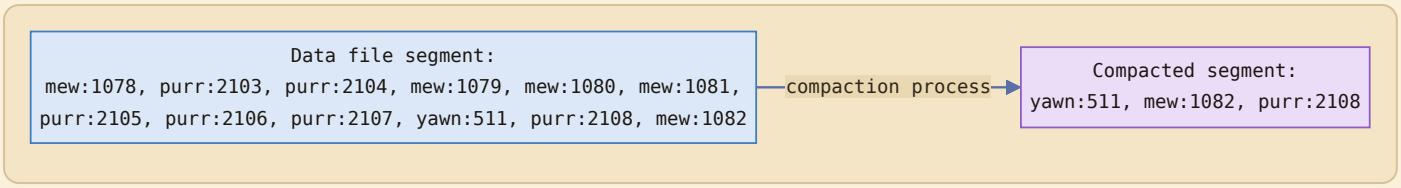
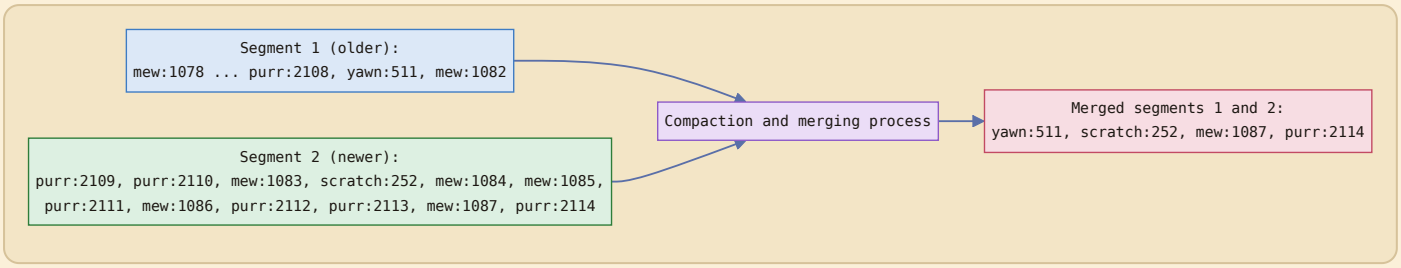


Figure 3-3 (redrawn): compaction and merging of two segments done together, producing one new segment



Key details of merging:

- Segments are **never modified** after being written; the merged result goes into a **new file**.
- Merging runs in a **background thread**. Meanwhile, reads and writes continue against the old segments.
- When merging finishes, reads switch to the new segment and the old files are simply deleted.
- **Each segment has its own hash map.** A read checks the newest segment's map first, then the next-newest, and so on. Merging keeps the number of segments (and so maps to check) small.

Practical issues in a real implementation:

Issue	How it's handled
File format	Use a binary format (length of the string in bytes, then raw bytes) instead of CSV — faster, no escaping needed.
Deleting records	Append a **tombstone** record. During merging, the tombstone tells the process to drop all earlier values for that key.
Crash recovery	In-memory hash maps vanish on restart. You *could* rebuild each by reading the whole segment, but that's slow for big files. Bitcask speeds this up by persisting a **snapshot of every segment's hash map** to disk, which loads back much faster.
Partially written records	A crash can leave a half-appended record. Bitcask includes **checksums** so corrupt tails are detected and ignored.
Concurrency control	Writes are strictly sequential, so typically there is **one writer thread**. Segments are immutable once written, so **many threads can read** them at the same time.

Why append-only instead of overwriting in place? It looks wasteful at first, but:

- Appends and merges are **sequential writes**, which beat random writes by a wide margin on spinning magnetic disks and, to a lesser degree, on SSDs as well.
- **Concurrency and crash recovery are simpler** with immutable files — you can never end up with a value that is half old, half new because a crash hit mid-overwrite.
- **Merging avoids fragmentation** of data files over time.

Limitations of hash indexes:

- **The hash table must fit in memory.** On-disk hash maps perform poorly: lots of random I/O, costly to grow, and fiddly collision handling.
- **Range queries are inefficient.** You can't cheaply scan all keys between `kitty00000` and `kitty99999`; each key must be looked up individually.

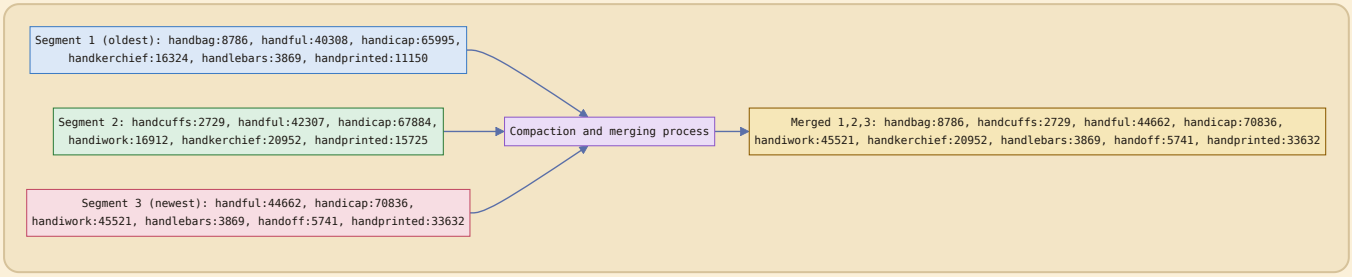
SSTables and LSM-Trees

The change. In the hash-index design, records inside a segment are in write order. Now require that each segment is **sorted by key**, and that each key appears **only once** per merged segment. This format is called a **Sorted String Table (SSTable)**.

Three big advantages over hash-indexed log segments:

1. **Merging is simple and efficient, even for files bigger than memory.** It works like the merge step of *mergesort*: walk through all input files in parallel, compare their current first keys, emit whichever key sorts lowest to the output, and repeat. The output is also sorted.
   - If the same key appears in several inputs, take the value from the **newest segment** and drop the older ones (each segment covers a time period, so one segment's values are all newer than an older adjacent segment's, assuming we always merge adjacent segments).

   Figure 3-4 (redrawn): merging three SSTable segments mergesort-style; for a key found in several segments, the value from the newest segment wins

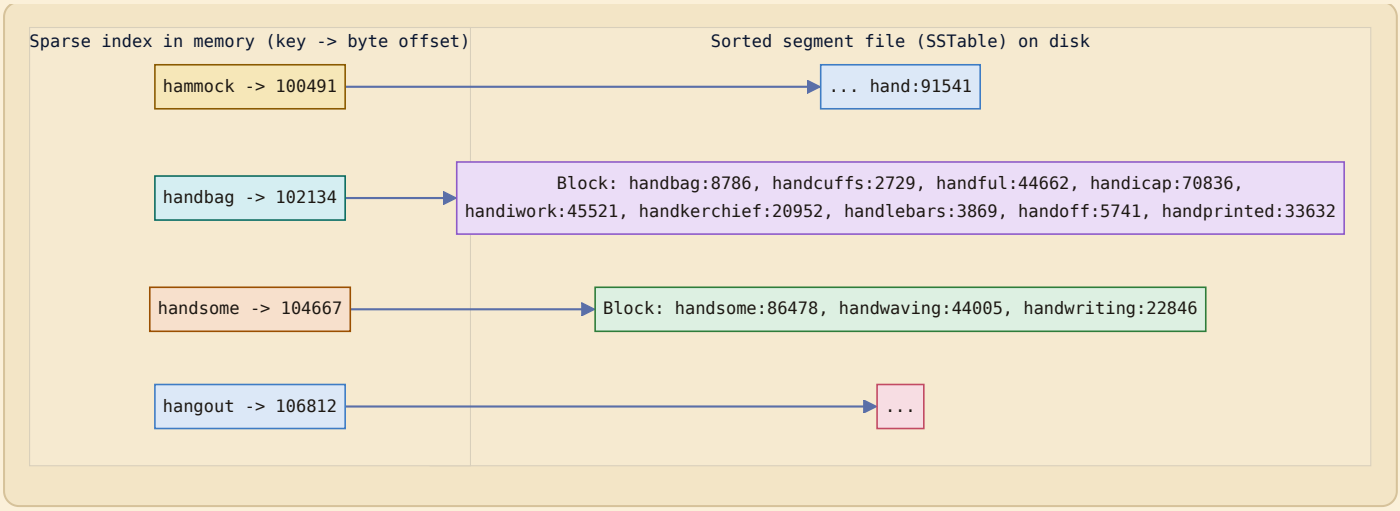


2. **You no longer need every key in memory.** Because keys are sorted, a **sparse** in-memory index is enough. Example from the book: to find `handiwork`, you may only know the offsets of `handbag` and `handsome`; since `handiwork` must fall between them, jump to `handbag` and scan forward. One index entry per **few kilobytes** of file is enough, because scanning a few KB is very fast.

   > **Footnote:** with fixed-size keys and values, a binary search over the file would work and skip the in-memory index entirely. In practice they vary in length, so you can't tell where records start without an index.

3. **Blocks can be compressed.** Since a read scans several records anyway, bundle neighbouring records into a block and compress that block before it goes to disk. Each sparse index entry points to the start of a compressed block. This saves disk space *and* I/O bandwidth.

Figure 3-5 (redrawn): an SSTable with a sparse in-memory index; each entry points to the start of a compressible block of the sorted file



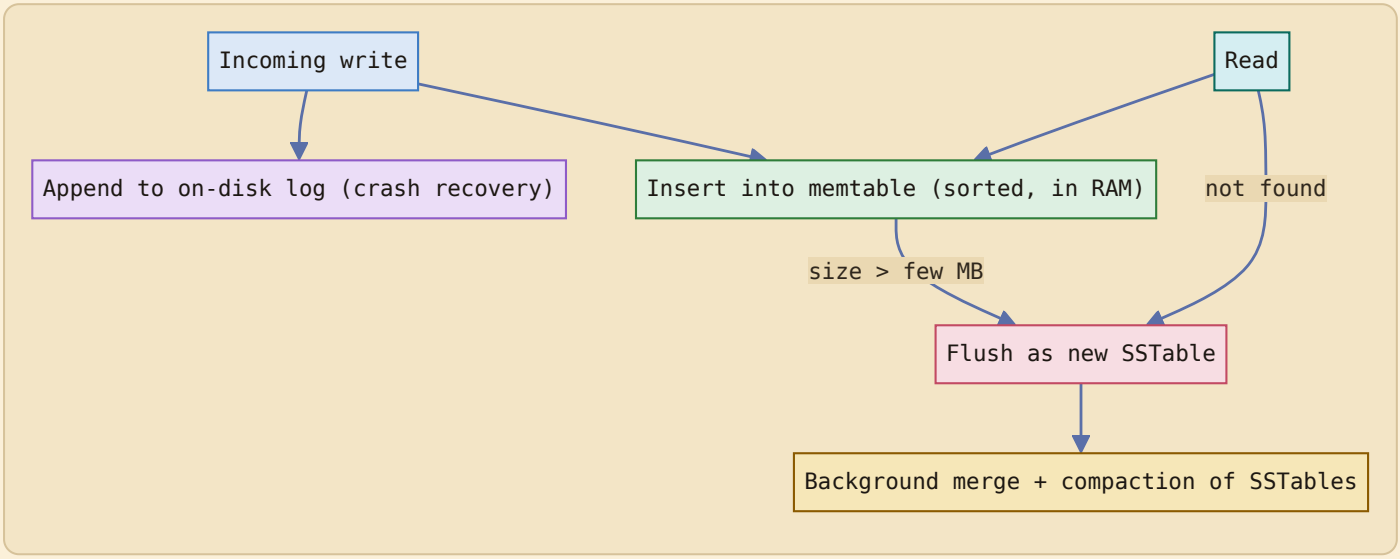
Constructing and maintaining SSTables

Incoming writes arrive in random key order, so how do we get sorted files? Keeping a sorted structure **on disk** is possible (that's the B-tree), but keeping one **in memory** is easy using a balanced tree like a **red-black tree** or **AVL tree** — insert in any order, read back sorted.

How the engine works:

1. **Write:** insert into an in-memory balanced tree called the **memtable**.
2. **Flush:** when the memtable exceeds a threshold (typically a **few megabytes**), write it to disk as a new SSTable — easy, since the tree is already sorted. That file becomes the newest segment. New writes go to a fresh memtable while the flush happens.
3. **Read:** check the memtable, then the newest on-disk segment, then the next-older one, and so on.
4. **Background:** periodically merge and compact segments, dropping overwritten and deleted values.

The crash problem. If the database crashes, the memtable (not yet on disk) is lost. Fix: keep a separate **unsorted append-only log** on disk; every write is appended there immediately. Its only job is rebuilding the memtable after a crash. Once a memtable has been flushed to an SSTable, its log can be thrown away.



Making an LSM-tree out of SSTables

- This algorithm is essentially what **LevelDB** and **RocksDB** use — key-value engine libraries meant to be embedded in other applications. LevelDB can replace Bitcask inside **Riak**.
- Similar engines power **Cassandra** and **HBase**, both inspired by Google's **Bigtable** paper, which coined the terms *SSTable* and *memtable*.

- The structure was first described by **Patrick O'Neil et al.** as the **Log-Structured Merge-Tree (LSM-Tree)**, building on earlier work on log-structured filesystems. Engines based on merging and compacting sorted files are called **LSM storage engines**.
- **Lucene** (the full-text indexing engine behind **Elasticsearch** and **Solr**) uses a similar approach for its **term dictionary**. A full-text index maps a word (**term**) to the list of IDs of documents containing it (the **postings list**). Lucene keeps these term → postings mappings in SSTable-like sorted files merged in the background.

Performance optimizations

- **Lookups for missing keys are slow:** you'd have to check the memtable and every segment back to the oldest, possibly reading disk each time. Engines add **Bloom filters** — memory-efficient structures that approximate a set's contents and can say for certain that a key is *absent*, avoiding pointless disk reads.
- **Compaction strategies:**

Strategy	How it works	Used by
Size-tiered	Newer, smaller SSTables are successively merged into older, larger ones.	HBase; Cassandra (supports both)
Leveled	The key range is split into smaller SSTables, and older data moves into separate "levels"; compaction is more incremental and uses less disk space.	LevelDB (hence the name), RocksDB; Cassandra

Why LSM-trees work well: the core idea (a cascade of SSTables that background merging keeps tidy) is both simple and powerful; it keeps working when the data is far bigger than memory, supports **range queries** (data is sorted), and achieves **very high write throughput** because disk writes are sequential.

B-Trees

What it is. The **most widely used** index structure. Introduced in **1970** and described as "ubiquitous" less than 10 years later, B-trees are still the standard index in almost all relational databases and many non-relational ones.

Similarity to SSTables: keys are kept sorted, so key lookups and range queries are efficient.
Difference: a completely different design philosophy.

	Log-structured (LSM)	B-tree
Unit of storage	Variable-size segments (several MB+)	Fixed-size **pages** (traditionally **4 KB**, sometimes bigger)
Write style	Always write a whole segment sequentially	Read/write one page at a time, overwriting in place
Hardware match	—	Close to how disks are physically organised (fixed-size blocks)

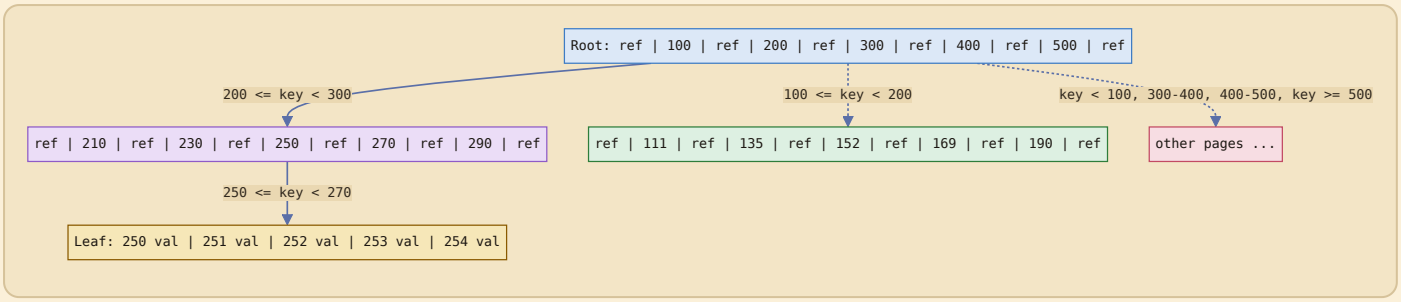
How it works:

1. Each page has an address, so a page can refer to another page — like a pointer, but on disk. Pages form a **tree**.
2. One page is the **root**; every lookup starts there.
3. A page contains a handful of keys interleaved with child-page references. Every child is in charge of a **continuous range of keys**; the keys between references mark the boundaries.

4. Follow the reference whose range contains your key, repeatedly, until you reach a **leaf page**, which holds the keys and either the values inline or references to where values live.

Book example: looking up key **251** — at the root, follow the reference between boundaries **200** and **300**; that child splits 200–300 into smaller ranges; keep descending to the leaf.

Figure 3-6 (redrawn): looking up user_id 251 by descending from the root through range boundaries to a leaf page

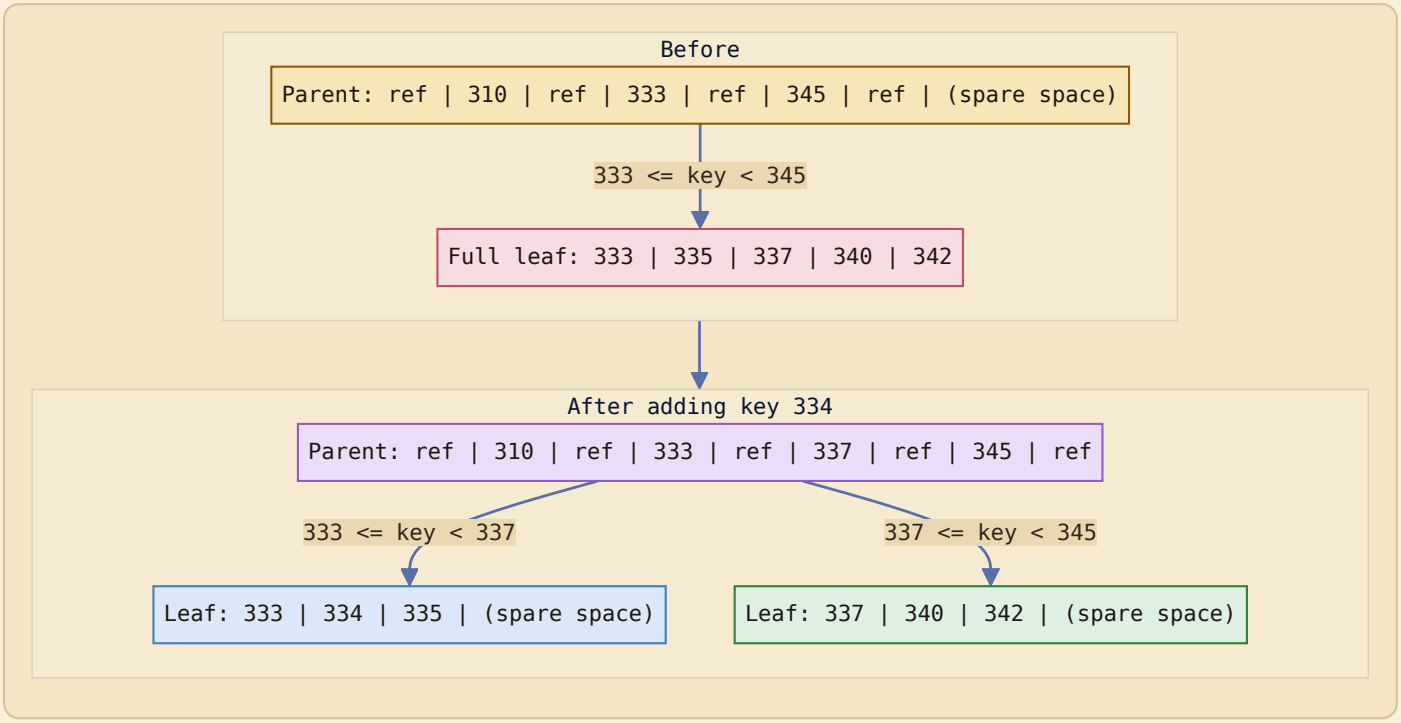


- **Branching factor** = number of child references per page. It's six in the book's figure; in practice **several hundred**, depending on how much space references and boundaries take.

Updating and inserting:

- **Update an existing key:** find its leaf, change the value in the page, write the page back. References to that page stay valid.
- **Insert a new key:** find the page whose range covers it and add it. If the page is **full**, **split** it into two half-full pages and update the **parent** to reflect the new division of ranges.

Figure 3-7 (redrawn): adding key 334 to a full leaf splits it into two pages with spare space; the parent gains the new boundary key 337



- This keeps the tree **balanced**: depth is always **O(log n)** for n keys. Most databases fit in a tree **three or four levels deep**.
- **Numbers to remember:** with **4 KB pages** and a **branching factor of 500**, just **four levels** are enough for up to **256 TB** of data.

> **Footnote:** inserting is intuitive; deleting a key while keeping the tree balanced is more involved.

Making B-trees reliable

- The fundamental B-tree write **overwrites a page on disk in place**, assuming its location doesn't change (so all references remain valid). This is the opposite of LSM-trees, which only append and delete whole files.
- On a spinning disk, an overwrite means moving the head, waiting for the platter to rotate to the right spot, and rewriting the sector. SSDs are more complicated: an SSD has to erase and rewrite a fairly large block of the storage chip in one go.
- **Multi-page operations are dangerous.** A page split writes two child pages *and* the parent. If the database crashes halfway, the index is corrupted — e.g., an **orphan page** with no parent.
- **Fix — the write-ahead log (WAL, a.k.a. redo log):** an append-only file where *every* B-tree change is recorded **before** being applied to the tree pages. After a crash, the WAL is replayed to bring the tree back to a consistent state.
- **Concurrency is harder too:** with in-place updates, threads could see the tree mid-change. B-trees protect their structures with **latches** (lightweight locks). Log-structured engines have it easier: they merge in the background without disturbing queries and swap old segments for new ones atomically.

B-tree optimizations

- **Copy-on-write (append-only B-trees)** instead of overwrite + WAL (e.g., **LMDB**): write the modified page to a new location and create new versions of parent pages pointing to it. Also helpful for concurrency control (see snapshot isolation, Chapter 7).
- **Abbreviated keys:** interior pages only need enough of a key to separate ranges. Packing in more keys increases the branching factor → fewer levels. (This variant is sometimes called a **B+ tree**, though the optimisation is so common it's rarely singled out.)
- **Sequential leaf layout:** pages can live anywhere on disk, so a big range scan might need a seek per page. Many implementations try to place leaf pages in key order on disk, but that's hard to maintain as the tree grows. LSM-trees find this easier, since merging rewrites large chunks in one go.
- **Sibling pointers:** each leaf links to its left and right neighbours, so an ordered scan doesn't need to climb back up to parents.
- **Fractal trees:** B-tree variants that borrow log-structured ideas to reduce disk seeks (nothing to do with fractals).

Comparing B-Trees and LSM-Trees

Rule of thumb: LSM-trees are usually **faster for writes**; B-trees are generally believed to **read faster**. LSM reads are slower because they may check several structures and SSTables at different compaction stages. **But:** benchmarks are often inconclusive and depend heavily on workload — test with *your* workload.

Advantages of LSM-trees

- **Write amplification.** In a B-tree, every piece of data hits disk **at least twice** — first in the WAL, then in the tree page itself (plus more on splits). It also writes a **whole page** even if only a few bytes changed; some engines even write the same page **twice** to avoid a torn page on power loss. LSM-trees also rewrite data repeatedly through compaction. The general effect — one database write causing multiple disk writes over the data's lifetime — is **write amplification**. It matters especially on **SSDs**, which wear out after a limited number of overwrites per block.

- **Higher write throughput.** In write-heavy apps, disk bandwidth can be the bottleneck, so every extra byte written costs throughput. LSM-trees often sustain more writes because they *sometimes* have lower write amplification (depends on configuration and workload) and because they write compact SSTables **sequentially** instead of overwriting scattered pages. This matters most on magnetic disks.
- **Better compression, smaller files.** B-trees leave unused space from **fragmentation** (after splits, or when a row doesn't fit a page). LSM-trees aren't page-oriented and periodically rewrite SSTables, so they have lower storage overhead — **especially with leveled compaction**.
- **On SSDs:** firmware often uses a log-structured algorithm internally to turn random writes into sequential ones, so the engine's write pattern matters less. Still, lower write amplification and less fragmentation help: more compact data means more requests fit in the available I/O bandwidth.

Downsides of LSM-trees

- **Compaction can interfere with live traffic.** Disks have finite resources, so a request may wait behind an expensive compaction. Average throughput and response time barely suffer, but **high-percentile (tail) latencies** can be quite high. **B-trees are more predictable.**
- **Compaction can fall behind at high write rates.** Disk bandwidth is shared between initial writes (logging, flushing memtables) and background compaction. On an empty database, all bandwidth goes to new writes, but as data grows, compaction needs more. If compaction can't keep up, **unmerged segments pile up** until the disk fills, and reads slow down as they must check more files. SSTable engines usually **don't throttle** incoming writes in this case, so you need **explicit monitoring**.
- In a B-tree, **every key lives in exactly one spot** of the index, while an LSM engine may hold copies in several segments. This makes B-trees attractive for **strong transactional semantics**: many relational databases implement isolation with **locks on key ranges**, which can be attached directly to the B-tree (more in Chapter 7).

Aspect	B-tree	LSM-tree
Typical strength	Reads, predictable latency	Writes, throughput
Write amplification	WAL + full page writes (sometimes twice)	Repeated compaction (sometimes lower overall, depending on configuration and workload)
Disk pattern	Random page overwrites	Sequential segment writes
Space usage	Fragmentation leaves empty space	More compact, better compression
Background work	No background compaction	Compaction may hurt tail latency / fall behind
Key location	Exactly one place (good for range locks)	Possibly many segments
Maturity	Very mature, ingrained in databases	Increasingly popular in new datastores

Conclusion: B-trees are deeply embedded in database architecture and consistently good, so they won't disappear. Log-structured indexes are growing in popularity in new systems. There's no quick rule — test empirically.

Other Indexing Structures

So far: **key-value indexes**, which behave like a **primary key** index — a key that uniquely identifies one row (relational), document (document DB) or vertex (graph DB). Other records refer to it by that key/ID, and the index resolves the reference.

Secondary indexes are very common. In relational databases you create several per table with `CREATE INDEX`; they're often essential for efficient joins (e.g., an index on `user_id` columns to find all

rows for one user).

```
-- A secondary index on a non-unique column
CREATE INDEX idx_orders_customer ON orders (customer_id);
-- Now this query can use the index instead of scanning every order:
SELECT * FROM orders WHERE customer_id = 812;
```

- Line 2 builds an index keyed on `customer_id`; many orders can share the same customer.
- Line 4 finds all orders for customer 812 via the index.

The only real difference from a primary index: **keys are not unique**. Two fixes:

1. Make each index value a **list of matching row IDs** (like a postings list), or
2. Make each key unique by **appending the row ID** to it.

With either trick, secondary indexes can be built on **B-trees or on log-structured indexes** alike.

Storing values within the index

An index entry's key is what you search by; the value is either **the row itself** or a **reference** to where the row lives.

- **Heap file:** the place where rows are stored in no particular order (may be append-only, or track deleted rows to reuse space). Common because with several secondary indexes, the data lives in **one place** and each index just points to it.
  - Updating a value **without changing the key** is efficient if the new value isn't bigger: overwrite it in place.
  - If it **grows** and must move, either update **all indexes** to the new location, or leave a **forwarding pointer** at the old spot.
- **Clustered index:** when the extra hop from index to heap hurts read performance too much, store the **whole row inside the index**.
  - **MySQL InnoDB:** the primary key is always a clustered index; secondary indexes refer to the **primary key**, not a heap location.
  - **SQL Server:** you may define **one** clustered index per table.
- **Covering index / index with included columns:** a middle ground — store **some** columns in the index, so certain queries can be answered from the index alone (the index **covers** the query).

```
-- SQL Server-style covering index
CREATE INDEX idx_emp_dept ON employees (dept_id) INCLUDE (salary);
SELECT salary FROM employees WHERE dept_id = 4;  -- answered from the index alone
```

- `dept_id` is the search key; `salary` is copied into the index, so the query never touches the main table.

Trade-off: clustered and covering indexes duplicate data → faster reads, but more storage, extra write overhead, and more effort for the database to keep transactional guarantees (apps must not see inconsistencies caused by the duplication).

Option	What the index stores	Read cost	Write/storage cost
Nonclustered (heap)	Pointer to row	Extra hop	Lowest
Covering	Some columns	No hop for covered queries	Medium
Clustered	Entire row	No hop	Highest

Multi-column indexes

Single-key indexes can't handle queries on several columns at once.

- **Concatenated index** (the most common kind): joins several columns into one key in a defined order. Like a paper **phone book** indexed by (last name, first name): you can find everyone with a given last name, or a given last+first combination, but it's **useless for finding by first name alone**.
- **Multi-dimensional indexes:** a more general approach, important for **geospatial** data. A restaurant-map site must find restaurants inside the visible rectangle — a **2D range query**:

```
SELECT * FROM cafes
WHERE lat BETWEEN 48.850 AND 48.862      -- north-south band
  AND lon BETWEEN 2.330  AND 2.355;      -- east-west band
```

- A normal B-tree or LSM index could give you everything in the latitude band (any longitude) *or* everything in the longitude band (any latitude) — not both efficiently at once.

Solutions:

- Map 2D points to one number with a **space-filling curve**, then use a regular B-tree.
- More commonly, use a specialised spatial index such as an **R-tree**. **PostGIS** builds geospatial R-tree indexes on PostgreSQL's **Generalized Search Tree (GiST)** facility.

Not just maps: e-commerce could use a 3D index on **(red, green, blue)** to search by colour range; a weather database could index **(date, temperature)** to find all readings in **2013** between **25 and 30 ℃** without scanning all of 2013 and filtering (or vice versa). **HyperDex** uses this technique.

Full-text search and fuzzy indexes

Previous indexes need **exact** keys or ranges. They can't find **similar** keys, such as misspellings — that's **fuzzy** querying.

- Full-text engines expand searches to **synonyms**, ignore **grammatical variations**, find words **near each other**, and support other linguistic features.
- **Lucene** can find words within a given **edit distance** (distance 1 = one letter added, removed or replaced) to cope with typos.
- Lucene's term dictionary is SSTable-like and needs a small in-memory index. In **LevelDB** that index is a sparse set of keys; in **Lucene** it's a **finite state automaton** over the characters of the keys, similar to a **trie**. This automaton can be turned into a **Levenshtein automaton**, which efficiently finds words within a given edit distance.
- Other fuzzy techniques lean toward document classification and machine learning (see information-retrieval textbooks).

Keeping everything in memory

Everything so far works around disk limitations. Disks are awkward (data must be laid out carefully for performance), but we put up with them because they're **durable** (survive power loss) and **cheaper per GB** than RAM.

As RAM gets cheaper, many datasets fit in memory — possibly spread over several machines. Hence **in-memory databases**:

- **Cache-only** stores like **Memcached** accept losing data on restart.
- **Durable** in-memory databases use battery-backed RAM, a **log of changes** on disk, **periodic snapshots** on disk, or **replication** to other machines. On restart they reload state from disk or a replica. They are still "in-memory" because disk is used only as an append-only log for durability — **all reads come from RAM**. Disk files also make backups, inspection and analysis by external tools easy.

Named systems:

- **VoltDB, MemSQL, Oracle TimesTen** — in-memory, relational; vendors claim large speedups by dropping on-disk data-structure overhead.
- **RAMCloud** — an open-source, durable in-memory key-value store; it organises data log-structured style, both in RAM and on disk.
- **Redis, Couchbase** — weak durability, writing to disk **asynchronously**.

> **Common confusion:** in-memory databases are *not* fast mainly because they skip disk reads. A disk-based engine with enough RAM may never read from disk either, because the OS caches recently used blocks. The real win is avoiding the cost of **encoding in-memory structures into a disk-friendly format**.

Richer data models: in-memory systems can offer structures that are hard to build on disk — e.g., **Redis** exposes priority queues and sets, and its implementation stays comparatively simple.

Anti-caching: research suggests in-memory architectures can handle data larger than RAM by evicting **least recently used** records to disk and reloading them when accessed. It resembles OS virtual memory/swap, but the database is smarter: it works on **individual records** rather than whole memory pages. Indexes must still fit in memory (like Bitcask).

Future: wider adoption of **non-volatile memory (NVM)** may require further changes to storage engine design; it's an active research area.

Transaction Processing or Analytics?

Early on, a database write usually matched a **commercial transaction**: a sale, a supplier order, a salary payment. The word **transaction** stuck even as databases expanded to non-financial data, now meaning a group of reads and writes forming one logical unit.

> **Note:** a transaction here doesn't need full ACID guarantees (atomicity, consistency, isolation, durability — Chapter 7). *Transaction processing* simply means clients can make **low-latency** reads and writes, in contrast to **batch processing** jobs that run only periodically, e.g. once a day (Chapter 10).

OLTP. Whether the data is blog comments, game actions or address-book contacts, the pattern resembles business transactions: look up a few records by key using an index; insert or update records based on user input. Because it's interactive, it's called **online transaction processing (OLTP)**.

OLAP. Databases also got used for **analytics**, a very different pattern: scan a **huge number of records**, read only **a few columns** each, and compute **aggregates** (count, sum, average) instead of returning raw rows. Examples for a sales table:

- Total revenue per store in January?
- How did banana sales during our most recent promotion compare with normal?
- Which baby-food brand is most often bought with brand X diapers?

Analysts write such queries to produce reports that feed **business intelligence**. This pattern is called **online analytic processing (OLAP)**.

> **Footnote:** the "online" in OLAP is vague; it probably means analysts run exploratory queries

> interactively, not just predefined reports.

Table 3-1 (paraphrased): OLTP vs OLAP characteristics

Property	OLTP	OLAP
Main read pattern	Each query fetches a few records by key	Aggregates computed across very many records
Main write pattern	Low-latency, random-access writes driven by user input	Event stream or bulk import (ETL)
Primarily used by	Customers/end users, through a web app	Internal analysts supporting decisions
What data represents	Latest state (current point in time)	History of events over time
Dataset size	GB up to TB	TB up to PB

At first the same databases served both. SQL handles both kinds of queries well. But in the **late 1980s and early 1990s**, companies started running analytics on a **separate** database: the **data warehouse**.

Data Warehousing

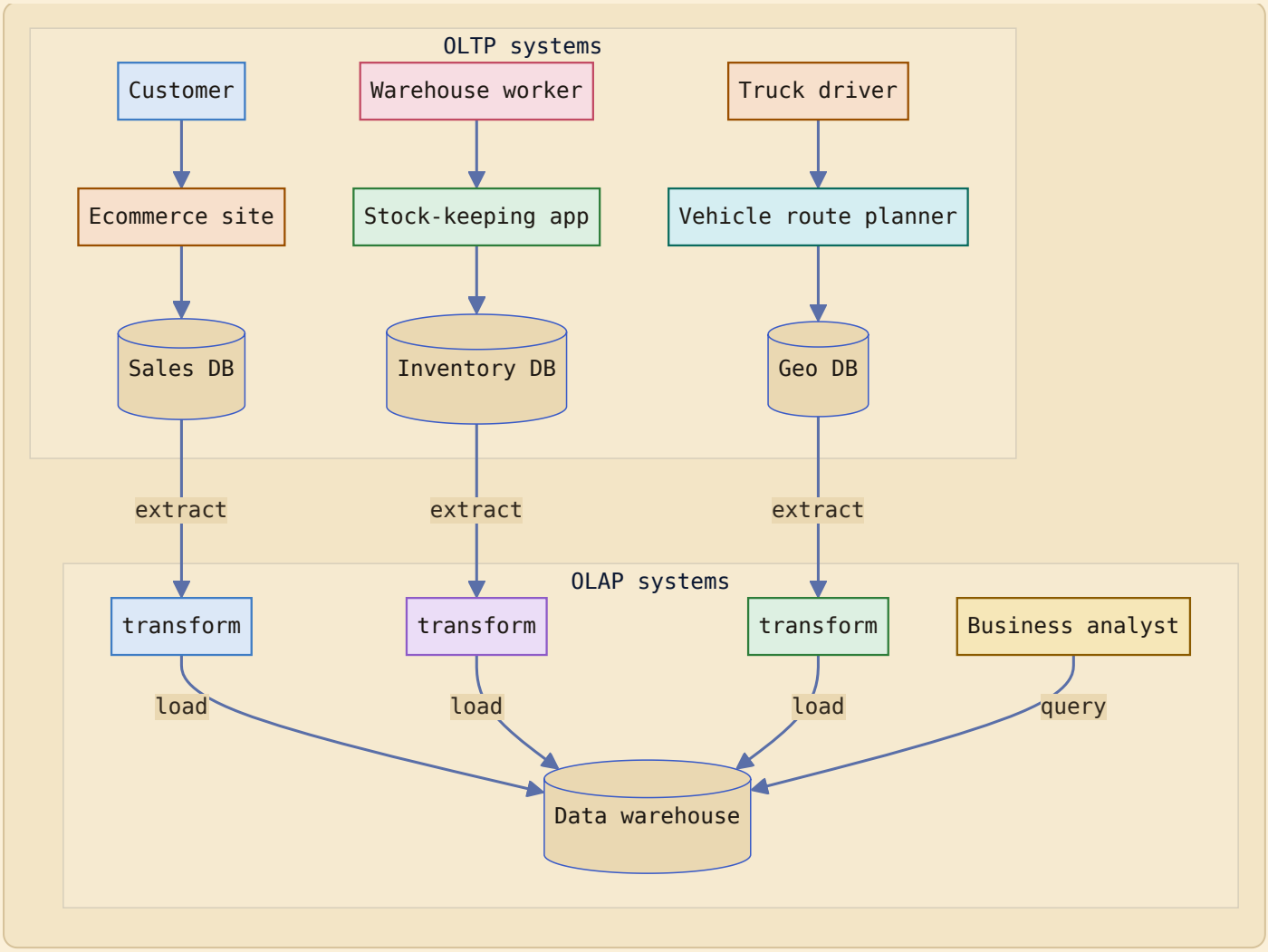
A large company may run **dozens** of OLTP systems: the customer website, point-of-sale (checkout) in physical stores, warehouse inventory, vehicle route planning, supplier management, HR, and more. Each is complex, has its own team, and runs mostly on its own.

These systems must be **highly available** and **low-latency** because the business depends on them. DBAs therefore guard them and don't want analysts running expensive **ad hoc** queries that scan large parts of the data and slow down live transactions.

A **data warehouse** is a separate database analysts can query freely without hurting OLTP. It holds a **read-only copy** of data from all the OLTP systems. Data gets there via **Extract–Transform–Load (ETL)**:

1. **Extract** from OLTP databases (periodic dump or continuous stream of updates).
2. **Transform** into an analysis-friendly schema and clean it up.
3. **Load** into the warehouse.

Figure 3-8 (redrawn): simplified ETL. Each OLTP system (with its own users and database) is extracted and transformed separately, then loaded into one data warehouse that business analysts query



- Warehouses exist in almost all **large** enterprises but are rare in **small** companies, which have fewer OLTP systems and little enough data to query with a normal SQL database or even a spreadsheet.
- **Big advantage:** a warehouse can be optimised for analytic access. The indexes from the first half of the chapter suit OLTP but are **poor at analytic queries**.

The divergence between OLTP databases and data warehouses

- Warehouses are usually **relational**, since SQL suits analytics. Many graphical tools generate SQL, visualise results, and support **drill-down** and **slicing and dicing**.
- On the surface a warehouse and an OLTP database look alike (both speak SQL), but their **internals differ greatly** because they're optimised for different query patterns. Many vendors now focus on one or the other.
- **Microsoft SQL Server** and **SAP HANA** support both in one product, but increasingly as two separate storage/query engines behind a shared SQL interface.
- **Commercial warehouses:** Teradata, Vertica, SAP HANA, ParAccel (expensive licences). **Amazon RedShift** is ParAccel offered as a hosted service.
- **Open-source SQL-on-Hadoop** (younger, aiming to compete): Spark SQL, Apache Hive, Apache Drill, Apache Tajo, Facebook Presto and Cloudera Impala. Some build on ideas from Google's **Dremel**.

Stars and Snowflakes: Schemas for Analytics

OLTP uses many different data models (Chapter 2), but analytics is much more uniform. Most warehouses use a **star schema** (a.k.a. **dimensional modeling**).

Fact table (at the centre, e.g. `fact_sales` for a grocery chain): each row is an **event** at a particular time — here, one customer buying one product. For web analytics, a row might be a page

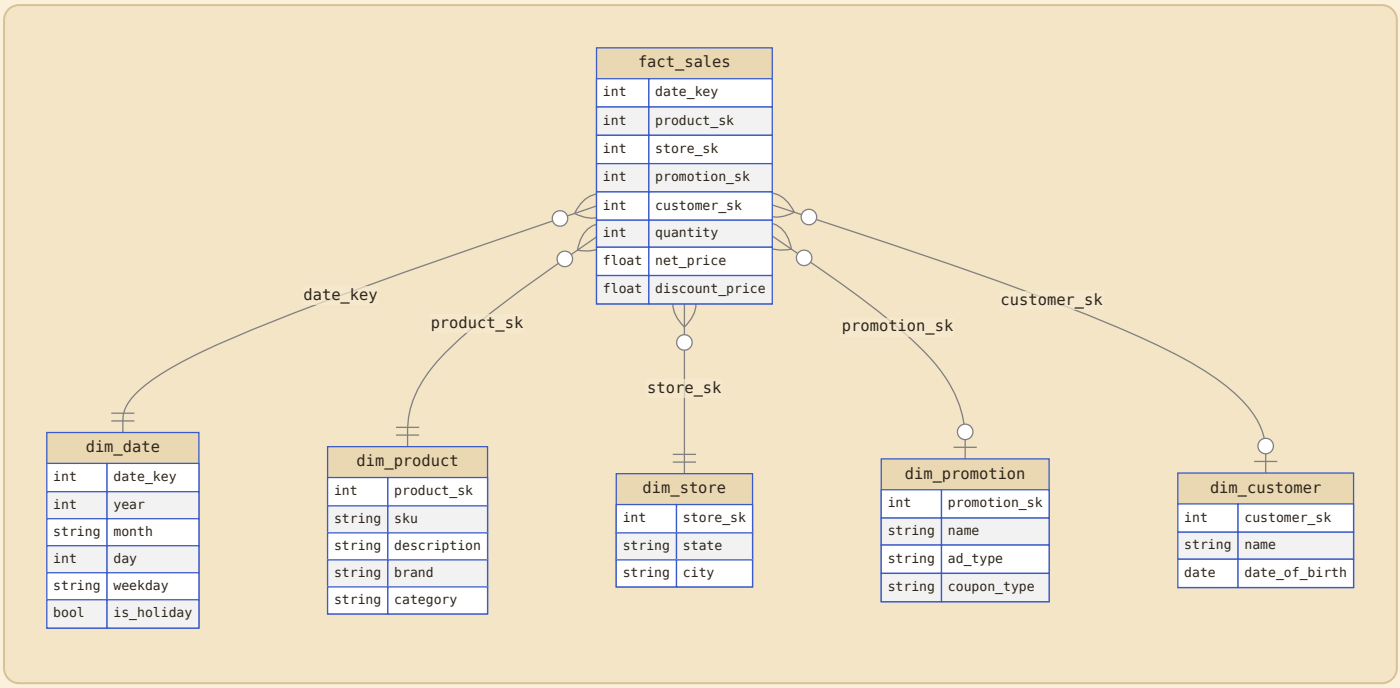
view or click.

- Recording individual events maximises flexibility later, but fact tables become **enormous**. Companies like **Apple, Walmart, eBay** may hold **tens of petabytes** of history, mostly in fact tables.
- Some columns are **attributes** (sale price, supplier cost → profit margin).
- Others are **foreign keys** to **dimension tables**, which describe an event's **who, what, where, when, how and why**.

Dimension tables: e.g., `dim_product` has one row per product type with SKU (stock-keeping unit), description, brand, category, fat content, package size… Each `fact_sales` row points to the product sold. (If a customer buys several products at once, each is a separate fact row.)

- Even **date and time** get a dimension table, so extra info like **public holidays** can be attached and queries can compare holiday vs non-holiday sales.

Figure 3-9 (redrawn): star schema for a grocery retailer. fact_sales sits in the centre; its foreign keys point at five dimension tables (sample rows in the book: product 31 = Fish food by Aquatech, store 3 = Palo Alto CA, promotion 19 = Aquarium deal, customer 191 = Bob, date 140102 = Thu 2 Jan 2014)



- **Why "star":** drawn out, the fact table sits in the middle with dimension tables around it, like rays of a star.
- **Snowflake schema:** dimensions are split further into sub-dimensions (e.g., separate `brand` and `category` tables referenced by `dim_product`). More **normalised**, but star schemas are usually preferred because they're **simpler for analysts**.
- **Tables are wide:** fact tables often exceed **100 columns**, sometimes several hundred. Dimension tables can be wide too — e.g., `dim_store` might record services offered, whether there's an in-store bakery, floor area, opening date, last remodel date, distance to the nearest highway.

Column-Oriented Storage

With **trillions of rows** and **petabytes** in fact tables, storage and querying get hard. Dimension tables are much smaller (**millions** of rows), so the focus is on **facts**.

Key observation: although fact tables have 100+ columns, a typical warehouse query touches only **4 or 5** at a time (`SELECT *` is rare in analytics).

The book's Example 3-1 asks whether people prefer buying fresh fruit or candy depending on the weekday, touching only three `fact_sales` columns (`date_key`, `product_sk`, `quantity`) while ignoring the rest, even though it covers every fruit or candy purchase in calendar year 2013. A similar query of our own:

```sql
SELECT d.month, s.region, SUM(f.net_price) AS revenue   -- only a few columns needed
FROM fact_sales f
  JOIN dim_date  d ON f.date_key = d.date_key            -- attach date details
  JOIN dim_store s ON f.store_sk = s.store_sk            -- attach store details
WHERE d.year = 2015 AND s.region IN ('North', 'West')    -- filter
GROUP BY d.month, s.region;                              -- one total per month per region
```

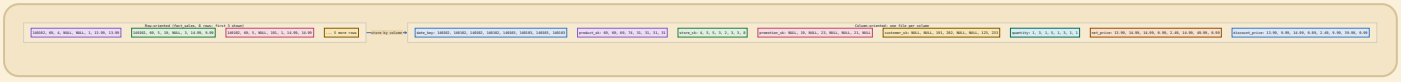
- Although `fact_sales` might have 100+ columns, this reads just `date_key`, `store_sk` and `net_price` from it — yet across a huge number of rows.

Why row storage struggles: OLTP databases store data **row-oriented** — all values of a row are adjacent (document databases similarly store each document contiguously; see the CSV log in Figure 3-1). Even with indexes on `date_key` or `product_sk`, a row store must load whole rows (100+ attributes each) from disk, parse them, and filter — slow.

The column-oriented idea: store all values of each **column** together, each column in its **own file**. A query reads and parses only the columns it needs.

> **Aside:** column storage is easiest to picture in the relational model, but it works for non-relational data too. **Parquet** is a columnar format that supports a document model, based on Google's **Dremel**.

Figure 3-10 (redrawn): the book's eight fact_sales rows stored as one file per column; position k in every file belongs to row k



Crucial rule: every column file must list rows in the **same order**. To rebuild row 23, take the 23rd entry from each column file.

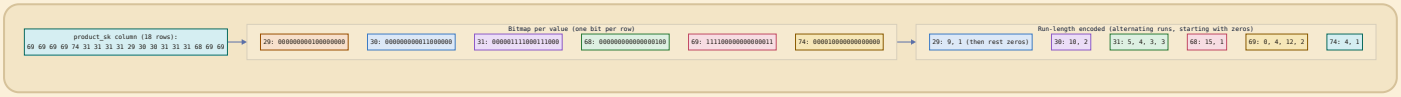
Column Compression

Reading fewer columns already cuts disk I/O; **compression** cuts it further, and columns compress well because their values are often repetitive.

Bitmap encoding (especially effective in warehouses):

1. A column usually holds few **distinct values** relative to the number of rows (a retailer may have **billions** of sales but only **100,000** products).
2. For a column with **n distinct values**, create **n bitmaps** — one per value — with **one bit per row**: 1 if the row has that value, 0 otherwise.
3. If n is **very small** (e.g., a country column with about **200** values), store bitmaps as-is.
4. If n is larger, most bitmaps are mostly zeros (**sparse**), so also apply **run-length encoding**: store lengths of alternating runs of 0s and 1s.

Figure 3-11 (redrawn): an 18-row product_sk column turned into one bitmap per distinct value, then run-length encoded



Bitmaps make common filters cheap:

- `WHERE product_sk IN (30, 68, 69)` → fetch the bitmaps for those three products and **OR** them together bit by bit.
- A filter on product 31 *and* store 3 (`product_sk = 31 AND store_sk = 3`) → fetch the two bitmaps and **AND** them together bit by bit. This works because all columns keep rows in the same order, so bit k refers to the same row in every bitmap.

There are many other compression schemes for other kinds of data.

> **Column-oriented storage vs column families (common confusion):** **Cassandra** and **HBase** have *column families*, inherited from **Bigtable**. Calling them column-oriented is very misleading: within a column family they store all columns of a row together with a row key and don't use column compression. The Bigtable model is mostly **row-oriented**.

Memory bandwidth and vectorized processing

For scans over millions of rows, disk-to-memory bandwidth is a big bottleneck, but not the only one. Analytic database developers also care about:

- how fast data moves from **RAM into the CPU cache**,
- avoiding **branch mispredictions** and **bubbles** in the CPU pipeline,
- using **SIMD** (single-instruction-multi-data) instructions.

Column layouts help here too: the query engine can grab a piece of compressed column data small enough for the CPU's **L1 cache** and loop over it tightly with **no function calls** — far faster than per-record code full of calls and branches. Compression lets **more rows fit** in L1. Operators like the bitwise AND/OR above can run **directly on compressed chunks**. This is **vectorized processing**.

Sort Order in Column Storage

Rows can be stored in **insertion order** (then inserting = appending to every column file). But we can instead impose a **sort order**, like SSTables, and use it as an index.

- **You can't sort each column independently** — you'd lose which values belong to the same row. Sort **whole rows**, even though storage is by column.
- The DBA picks **sort keys** based on common queries. If queries often target date ranges (e.g., last month), make `date_key` the **first** sort key so the engine scans only that month's rows.
- A **second** sort key orders rows that tie on the first. E.g., `date_key` then `product_sk` groups all sales of the same product on the same day together — useful for grouping/filtering by product within a date range.

Sorting also boosts compression: if the first sort column has few distinct values, sorting creates **long runs** of identical values. Simple run-length encoding can shrink it to a **few kilobytes** even with **billions** of rows.

- The first sort key benefits most; the second and third keys are less orderly; later columns look essentially random and compress less. Still, sorting the first few columns is a net win.

Several different sort orders

Introduced in **C-Store** and adopted by the commercial warehouse **Vertica**: different queries prefer different sort orders, and you must **replicate** data to several machines anyway to survive failures — so store each replica **sorted differently**, and have each query use the copy that suits it best.

	Multiple sort orders (column store)	Multiple secondary indexes (row store)
Where the full data lives	Each sorted copy contains the column values themselves	Each row in one place (heap file or clustered index)
What the extra structures hold	No pointers — just columns of values	Pointers to matching rows

Writing to Column-Oriented Storage

Column storage, compression and sorting make **reads** fast — ideal since warehouse load is mostly large read-only queries — but make **writes harder**.

- **Update-in-place (B-tree style) is impossible** with compressed columns.
- Placing a new row **midway through** a sorted table would probably mean **rewriting every column file**, because rows are identified by position and all columns must be updated consistently.

Solution: LSM-trees again.

1. Every write lands first in an **in-memory store**, where it is sorted and staged for disk. (Row- or column-oriented in memory — it doesn't matter.)
2. When enough writes accumulate, **merge** them with the on-disk column files and write **new files in bulk**. This is essentially what **Vertica** does.
3. **Queries** read both the on-disk column data and the recent in-memory writes and combine them. The **query optimizer hides** this, so analysts see inserts, updates and deletes reflected immediately.

Aggregation: Data Cubes and Materialized Views

Not every warehouse is a column store (row-oriented and other architectures are also used), but columnar storage can be **much faster for ad hoc analytics** and is quickly gaining popularity.

Materialized aggregates. Warehouse queries often use `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`. If many queries use the same aggregates, recomputing from raw data every time is wasteful — so **cache** the popular ones.

Materialized view vs virtual view:

	Virtual view	Materialized view
What it is	A named query shortcut	The query's results really stored **on disk**
On read	SQL engine expands it into the underlying query on the fly	Read precomputed results directly
On data change	Nothing to do	Must be **updated** (it's a denormalised copy); database can do it automatically
Cost	Slow reads for heavy queries	More expensive **writes**

Because of the write cost, materialized views are rare in **OLTP** but more sensible in **read-heavy warehouses** (whether they help depends on the case).

```
-- PostgreSQL-style example
CREATE MATERIALIZED VIEW daily_store_revenue AS
  SELECT date_key, store_sk, SUM(net_price) AS revenue
  FROM fact_sales
  GROUP BY date_key, store_sk;          -- precompute totals per day per store
REFRESH MATERIALIZED VIEW daily_store_revenue;  -- recompute after new sales arrive
```

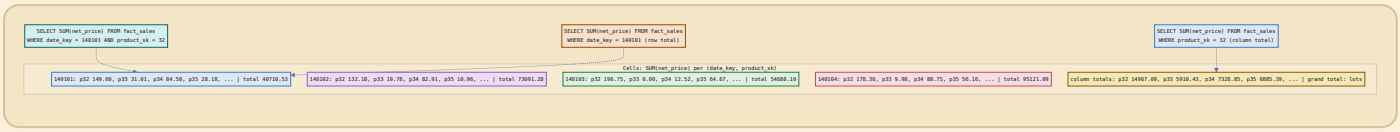
- The first statement stores the grouped totals on disk; later queries read them instead of scanning `fact_sales`.

- `REFRESH` brings the stored copy up to date after the underlying data changes.

Data cube / OLAP cube: a special materialized view — a **grid of precomputed aggregates, laid out by several dimensions**.

1. Suppose facts reference only two dimensions, **date** and **product**. Make a 2D table: dates on one axis, products on the other.
2. Every cell holds an aggregate such as `SUM`, applied to one attribute such as `net_price`, over all facts with that date–product pair.
3. Aggregate each row or column again to get totals reduced by one dimension (sales per product regardless of date, or per date regardless of product).

Figure 3-12 (redrawn): two dimensions of a data cube (date_key by product_sk). Each cell is SUM(net_price) for that pair; summing along a row gives the per-date total, along a column the per-product total



- With more dimensions (Figure 3-9 has **five**: date, product, store, promotion, customer), it becomes a **hypercube**, harder to picture but the same idea: each cell holds sales for one combination, summarisable along any dimension.
- **Advantage:** some queries become near-instant because they're **precomputed** — e.g., total sales per store yesterday = read the totals along one dimension, no scanning millions of rows.
- **Disadvantage: less flexible** than raw data. E.g., you can't compute what share of sales came from items priced above **$100**, because price isn't a dimension.
- Therefore most warehouses **retain raw data wherever they can**, treating cubes merely as a **speed-up for particular queries**.

Summary

(Book section) The chapter explored what a database does when you store data and when you query it later.

- **Two big categories** of storage engines: optimised for **OLTP** vs optimised for **analytics (OLAP)**.
  - **OLTP:** user-facing, huge request volume, each query touches few records found by key via an index. **Disk seek time** is often the bottleneck.
  - **Analytics/warehouses:** used by business analysts, far fewer queries, but each scans millions of records quickly. Here the usual limit is **disk bandwidth**, not seek time; **column-oriented storage** is the growing answer.
- **Two OLTP schools:**
  - **Log-structured:** only append to files and delete obsolete ones; never modify a written file. Members include Bitcask, SSTables, LSM-trees, LevelDB, HBase, Cassandra, Lucene and more.
  - **Update-in-place:** disk as fixed-size overwritable pages. **B-trees** are the main example, used in all major relational databases and many non-relational ones.
- Log-structured engines are relatively new; their key trick is turning **random writes into sequential writes**, enabling higher write throughput on HDDs and SSDs.
- The chapter also toured more complex indexes (secondary, clustered/covering, multi-dimensional, fuzzy) and **in-memory** databases.
- **Warehouses:** when queries scan many rows sequentially, indexes matter less; what matters is **very compact encoding** to minimise data read — hence columnar storage, compression, sorting.

- **Takeaway:** knowing engine internals helps you choose the right tool and reason about what a tuning parameter change will do, and helps you understand your database's documentation.

Chapter summary

- A **log** (append-only file) gives cheap writes; an **index** gives fast reads but slows writes — choose indexes deliberately.
- **Hash indexes** (Bitcask): in-memory key → offset map; great for many updates to few keys; keys must fit in RAM; no range queries. Segments + compaction + merging reclaim space; tombstones delete; checksums catch partial writes.
- **SSTables/LSM-trees** (LevelDB, RocksDB, Cassandra, HBase, Lucene): memtable → sorted SSTables → background merging (size-tiered or leveled). Sparse index, compressed blocks, Bloom filters for missing keys, WAL-like log for crash safety. High write throughput, range queries supported.
- **B-trees**: fixed 4 KB pages, in-place updates, splits keep the tree balanced (3–4 levels; 4 levels × 500 branching × 4 KB = 256 TB). WAL and latches for reliability; copy-on-write (LMDB), abbreviated keys (B+ tree), sibling pointers, fractal trees as optimisations.
- **LSM vs B-tree**: LSM usually better for writes/space; B-tree better for reads, predictable latency, and key-range locking. Always benchmark your own workload.
- **Other indexes**: secondary (non-unique), heap file vs clustered (InnoDB, SQL Server) vs covering; concatenated vs multi-dimensional (R-trees, PostGIS, HyperDex); fuzzy (Lucene automata, edit distance). **In-memory DBs** win by avoiding encoding overhead, not disk reads.
- **OLTP vs OLAP**: different access patterns → separate **data warehouses** loaded via **ETL**, typically in **star** (or snowflake) schemas.
- **Column stores**: read only needed columns; compress (bitmaps + RLE); vectorised processing; sort rows by chosen keys (multiple sort orders in C-Store/Vertica); write via LSM-style in-memory buffer; **materialized views/data cubes** precompute aggregates at the cost of flexibility.

Interview angle

1. B-tree vs LSM-tree — which would you pick? *30-second answer:* B-trees update fixed-size pages in place with a WAL; every key lives in one place, reads are fast and latency is predictable — the default in relational databases. LSM-trees buffer writes in a memtable, flush sorted SSTables sequentially, and compact in the background — higher write throughput, better compression, but reads may check several files and compaction can hurt tail latency. Write-heavy (logs, metrics, time-series) → LSM; read-heavy/transactional → B-tree. Benchmark your workload.

- *Follow-up: What is write amplification?* One logical write causing several physical disk writes over time — WAL + page writes in B-trees, repeated compaction in LSM. It limits throughput and wears out SSDs.
- *Follow-up: How does an LSM avoid checking every SSTable for a missing key?* Bloom filters per SSTable say "definitely not here", skipping disk reads.
- *Follow-up: What happens if compaction can't keep up?* Unmerged segments pile up, reads slow down, and disk can fill; engines usually don't throttle writes, so monitor it.

2. How does an LSM-tree handle a write and survive a crash? *30-second answer:* The write is appended to an on-disk log and inserted into the in-memory sorted memtable. When the memtable reaches a few MB, it's flushed as an immutable SSTable. On crash, the log is replayed to rebuild the memtable; after a flush, that log is discarded.

- *Follow-up: How are deletes done?* By writing a tombstone; compaction later drops older values for that key.
- *Follow-up: Size-tiered vs leveled?* Size-tiered merges smaller/newer tables into bigger/older ones (HBase); leveled splits key ranges into small tables across levels — more incremental, less disk

space (LevelDB, RocksDB). Cassandra supports both.

3. How does a B-tree stay balanced and crash-safe? *30-second answer:* When an insert overflows a page, it is split into two roughly half-empty pages and a new boundary is added to the parent, so depth stays O(log n) — usually 3–4 levels. Every change is written to a write-ahead log first, so a crash mid-split can be repaired; latches protect concurrent access.

- *Follow-up: Alternative to a WAL?* Copy-on-write pages (LMDB): write modified pages elsewhere and create new parent versions.

4. OLTP vs OLAP, and why a separate data warehouse? *30-second answer:* OLTP = many small key-based reads/writes from users, latest state, GB–TB. OLAP = few large scans computing aggregates for analysts, event history, TB–PB. Running analyst queries on OLTP databases hurts customer-facing latency, and OLTP indexes are poor for scans, so data is copied via ETL into a warehouse optimised for analytics (often columnar, star schema).

- *Follow-up: What is a star schema?* A central fact table of events with foreign keys to dimension tables (date, product, store…). Snowflake further normalises dimensions.

5. Why are column stores fast for analytics? *30-second answer:* Queries touch 4–5 of 100+ columns, so storing each column separately means reading only what's needed. Columns compress very well (bitmaps, run-length encoding), especially the sort key, and compressed chunks fit CPU cache for vectorised processing.

- *Follow-up: How do you write to a column store?* Buffer writes in memory (LSM-style) and periodically merge into new column files in bulk; queries combine both (Vertica).
- *Follow-up: Are Cassandra/HBase column stores?* No — their column families are still row-oriented within each family and don't use column compression.

6. Clustered vs covering vs heap-file indexes? *30-second answer:* Heap-file indexes store pointers to rows kept elsewhere — no duplication but an extra hop. Clustered indexes store the full row in the index (InnoDB primary key; one per table in SQL Server). Covering indexes store some columns so certain queries need only the index. More duplication = faster reads, more storage and write cost.

Self-check

1. **Why is appending to a log fast, and why is reading from an unindexed log slow?** Appending is a sequential write at the file's end — the simplest possible write. Reading without an index means scanning the whole file, O(n).
2. **What is the main limitation of Bitcask-style hash indexes, and what workload suits them?** All keys must fit in RAM and range queries are inefficient. They suit many updates to a modest number of keys (e.g., play counts per video URL).
3. **Why can an SSTable use a sparse index while a hash-indexed log cannot?** Because the keys are sorted: knowing the offsets of neighbouring keys tells you the range to scan. In an unsorted log, a key could be anywhere.
4. **Trade-off: your service ingests millions of sensor readings per second and rarely updates old data. B-tree or LSM-tree? Why?** LSM-tree: sequential writes, higher write throughput, better compression. Watch compaction and tail latency.
5. **What happens in a B-tree when you insert a key into a full page, and why is that risky without a WAL?** The page splits into two and the parent is updated — several page writes. A crash in between could leave an orphan page/corrupt index; the WAL lets you restore consistency.
6. **How big can a 4-level B-tree with 4 KB pages and branching factor 500 get?** Up to 256 TB.
7. **Scenario: a map app needs "all shops inside this rectangle". Why won't a concatenated (lat, lon) B-tree index work well, and what should you use?** A concatenated index can narrow by latitude

range but then must scan all longitudes in that band. Use a spatial index such as an R-tree (e.g., PostGIS on GiST) or a space-filling curve.

8. **What is the real reason in-memory databases are faster?** They avoid the overhead of encoding in-memory data structures into disk-friendly form — not simply avoiding disk reads, since the OS cache already does that for disk-based engines with enough RAM.
9. **In a column store, why can't each column be sorted independently? What benefit does sorting give?** Rows are reconstructed by position; independent sorting would break that alignment. Sorting whole rows helps range filters on the sort key and creates long runs that compress extremely well.
10. **What's the downside of relying on a data cube instead of raw data?** It's inflexible: only questions along the chosen dimensions can be answered (e.g., can't compute the share of sales for items over $100 if price isn't a dimension). Keep raw data and use cubes as a speed-up.

Chapter 4 — Encoding and Evolution

Book: Chapter 4, pp. 111–144

Overview

Software never stands still. New features ship, requirements become clearer, the business shifts — and nearly every such change also changes the **data** the application keeps: a new field appears, a new record type is needed, or old data must be shown differently. Chapter 1 called the ability to cope with this **evolvability**.

The data models from Chapter 2 cope with change in different ways:

- **Relational databases** assume all data follows **one schema** at any moment. You can change it with **schema migrations** (`ALTER TABLE` statements), but only one version is in force at a time.
- **Schema-on-read ("schemaless") databases** impose no schema at all, so old-format and new-format records can sit side by side.

When the data format changes, the code that reads and writes it usually has to change too. In a big system, though, code **cannot change everywhere at the same instant**:

- **Server-side:** you typically do a **rolling upgrade** (also called a **staged rollout**) — deploy the new version to a few nodes, check it behaves, then continue node by node. This avoids downtime and so encourages frequent releases.
- **Client-side:** you depend on users, who may not install an update for a long time.

So **old and new code, and old and new data formats, will coexist**. To keep working, the system needs compatibility **in both directions**:

Kind	Meaning	Difficulty
Backward compatibility	*Newer* code can read data written by *older* code.	Usually easy — the new code's author knows the old format and can handle it explicitly (even by keeping the old reading code).
Forward compatibility	*Older* code can read data written by *newer* code.	Harder — old code must be able to *ignore* additions it has never heard of.

The chapter answers two questions:

1. **How do encoding formats** (JSON, XML, Protocol Buffers, Thrift, Avro) **handle schema changes** so that old and new code/data can coexist?
2. **How do these formats get used when data flows** between processes — through databases, through services (REST and RPC), and through asynchronous messages (message brokers and actors)?

Everyday analogy. Imagine a paper form your office has used for years. A new version adds a "mobile phone" box. Staff trained on the new form must still understand old forms (backward compatibility). Staff still using old instructions must be able to process the new forms by simply skipping the box they don't recognise — and, if they re-file the form, not tear that box off (forward compatibility + preserving unknown fields).

Key terms

Term	Plain-English meaning
Evolvability	How easily a system can be changed over time.
Schema migration	Changing a relational schema, e.g. with `ALTER TABLE`.
Schema-on-read	No schema enforced on write; readers interpret the structure (so-called "schemaless").
Rolling upgrade / staged rollout	Rolling out a new version gradually, a handful of nodes per step.
Backward compatibility	New code can read old data.
Forward compatibility	Data produced by newer code is still readable by older code.
Encoding (serialization, marshalling)	Converting in-memory data into a self-contained byte sequence.
Decoding (parsing, deserialization, unmarshalling)	Converting a byte sequence back into in-memory data structures.
Language-specific format	Encoding built into one programming language (Java Serializable, Ruby Marshal, Python pickle).
Textual format	Human-readable encodings such as JSON, XML, CSV.
Binary string	A sequence of raw bytes with no character encoding.
Base64	A way to write binary data as text; makes it about 33% larger.
Binary encoding	A compact, non-human-readable byte format.
MessagePack	One binary encoding of JSON.
Schema	A formal description of the structure and types of data.
IDL (Interface Definition Language)	The language used to write a schema (e.g., Thrift IDL, Avro IDL).
Code generation	A tool turns a schema into classes in your programming language.
Field tag	A number identifying a field in Thrift/Protocol Buffers instead of its name.
Variable-length integer (varint)	An integer encoding that uses fewer bytes for small numbers.
Schema evolution	Changing a schema over time while keeping compatibility.
Writer's schema	The schema the writing application used to encode the data (Avro).
Reader's schema	The schema the reading application expects the data in (Avro).
Union type	Avro type allowing a value to be one of several types, e.g. `null` or `long`.
Object container file	Avro file format that stores the writer's schema once, followed by many records.
Self-describing file	A file that carries the metadata needed to interpret it.
ASN.1 / DER	Old (1984) schema language and its binary encoding, still used for X.509 certificates.
Dataflow	The path by which encoded data goes from one process to another.

Term	Plain-English meaning
Data outlives code	Old data stays in its original encoding long after the code that wrote it is gone.
Migrating data	Rewriting stored data into a new schema.
Archival storage	Snapshots/dumps of a database, e.g. for backups or warehouses.
Client / server / service	Clients send requests to servers; the API a server exposes is a service.
Ajax	JavaScript in the browser making HTTP requests (XMLHttpRequest) for data.
SOA / microservices	Building an application from many smaller services that call each other.
Web service	A service accessed over HTTP.
Middleware	Software supporting service-to-service calls inside one organisation.
REST	Design philosophy built on HTTP principles (URLs for resources, HTTP features).
RESTful	An API following REST principles.
SOAP	XML-based protocol for network API requests, with the WS-* standards.
WSDL	XML-based language describing a SOAP API; enables code generation.
OpenAPI (Swagger)	A format for describing RESTful APIs and generating documentation.
RPC (remote procedure call)	Making a network request look like calling a local function.
Location transparency	The idea of hiding whether something is local or remote.
Idempotence	Doing an action twice has the same effect as doing it once — needed for safe retries.
Futures / promises	Objects representing a result that will arrive later (or fail).
Service discovery	Finding which IP address and port a service is running on.
API versioning	How a client indicates which version of an API it wants.
Message broker (message queue, message-oriented middleware)	An intermediary that temporarily stores messages between sender and receiver.
Queue / topic	Named destinations in a message broker.
Producer / consumer (subscriber)	Sender / receiver of messages.
Actor model	Concurrency model where independent actors hold private state and exchange async messages.
Distributed actor framework	Actor model spread over many nodes (Akka, Orleans, Erlang OTP).

Formats for Encoding Data

Programs hold data in **at least two representations**:

1. **In memory:** objects, structs, lists, arrays, hash tables, trees… These are tuned for fast CPU access and manipulation, usually with **pointers**.
2. **On disk / on the network:** a **self-contained sequence of bytes** (e.g., a JSON document). A pointer means nothing to another process, so this form looks very different from the in-memory one.

> **Footnote:** exceptions exist, e.g. some memory-mapped files, or operating directly on compressed data (as with column compression in Chapter 3).

So we need translation in both directions:

- **Encoding** (also **serialization** or **marshalling**): in-memory → bytes.
- **Decoding** (also **parsing**, **deserialization**, **unmarshalling**): bytes → in-memory.

> **Terminology clash:** "serialization" also has a totally different meaning for transactions (Chapter 7). To avoid confusion the book says **encoding**, even though serialization is probably the more common word.
>
> **Footnote:** encoding is unrelated to **encryption**, which the book doesn't cover.

There are countless libraries and formats; here's the tour.

Language-Specific Formats

What they are. Many languages have built-in ways to turn objects into bytes: **Java** has `java.io.Serializable`, **Ruby** has `Marshal`, **Python** has `pickle`. There are third-party ones too, e.g. **Kryo** for Java.

Why people like them: you can save and restore in-memory objects with almost no extra code.

Why they're a bad idea (deep problems):

1. **Tied to one language.** Reading the data from another language is very hard, so you lock yourself into your current language for a long time and make integration with other organisations (using other languages) difficult.
2. **Security risk.** To rebuild objects of the original types, the decoder must be able to **instantiate arbitrary classes**. If an attacker can feed your app arbitrary bytes to decode, they can create arbitrary classes — which often lets them do terrible things like **remote code execution**.
3. **Versioning is an afterthought.** Built for quick-and-easy encoding, these libraries often ignore forward and backward compatibility.
4. **Efficiency is an afterthought.** Both CPU cost and encoded size may be poor; **Java's built-in serialization** is infamous for being slow and producing bloated bytes.

Verdict: use your language's built-in encoding only for **very short-lived** purposes.

```
import pickle
blob = pickle.dumps({"sku": "A-17", "qty": 3})  # encode a dict into Python-only bytes
item = pickle.loads(blob)                       # decode; never do this with untrusted input!
```

- Line 2 produces bytes only Python's `pickle` understands.
- Line 3 rebuilds the object; with attacker-controlled bytes, `loads` can run arbitrary code — the security problem above.

JSON, XML, and Binary Variants

Standardised, cross-language formats: JSON and **XML** are the obvious choices — widely known, widely supported, and nearly as widely disliked. XML is often called too verbose and needlessly complicated. JSON is popular mainly because web browsers support it natively (it's a subset of

JavaScript) and it's simpler than XML. **CSV** is another language-independent format, though less powerful.

All three are **textual** and so somewhat human-readable. Beyond arguments about syntax, they have **subtle problems**:

1. **Number ambiguity.**
    - In **XML and CSV**, a number and a string of digits look identical (unless you consult an external schema).
    - **JSON** separates strings from numbers, but doesn't separate **integers from floating-point numbers** and specifies **no precision**.
    - This bites with large numbers: integers above **2^53** can't be represented exactly in an **IEEE 754 double**, so languages that parse JSON numbers as floats (like **JavaScript**) get them wrong. Real example: **Twitter** uses **64-bit** numbers as tweet IDs. To stop JavaScript clients from mangling them, its API includes every tweet ID **twice**: as a JSON number and also as a **decimal string**.
2. **No binary strings.** JSON and XML handle Unicode text well but not raw bytes. The workaround is to **Base64**-encode binary data as text and use the schema to say "this is Base64". It works, but it's hacky and grows the data by **33%**.
3. **Optional, complicated schemas.** Both XML and JSON have optional schema languages; they're powerful and therefore complicated to learn and implement. XML schemas are fairly widespread; many JSON tools skip schemas entirely. Because interpreting numbers and binary strings depends on the schema, apps without one must **hardcode** the encoding/decoding logic.
4. **CSV has no schema at all.** The app decides what each row and column means; if a change adds a row or column, you handle it by hand. CSV is also **vague** — what if a value contains a comma or newline? Its escaping rules have been formally specified, but not every parser implements them correctly.

Still good enough. Despite the flaws, JSON, XML and CSV work for many uses, especially as **data interchange formats** between organisations. There, as long as everyone agrees on the format, prettiness and efficiency hardly matter — getting organisations to agree on *anything* is the hard part.

Binary encoding

Inside **one organisation**, there's less pressure to pick a lowest-common-denominator format; you can choose something more compact or faster to parse. For small datasets the gain is negligible, but at **terabyte** scale the choice matters a lot.

JSON is less verbose than XML, yet both use much more space than binary formats. This led to many **binary encodings**:

- **For JSON:** MessagePack, BSON, BJSON, UBJSON, BISON, Smile, and others.
- **For XML:** e.g. Fast Infoset and WBXML.

These found niche adoption, but none is as widespread as textual JSON/XML. Some add datatypes (e.g., integers vs floats, binary strings) but otherwise keep the JSON/XML data model. Crucially, **since they have no schema, they must include every field name** in the encoded data.

The running example (Example 4-1) — a person record the book encodes in several formats. Its fields are `userName` = "Martin", `favoriteNumber` = 1337, and `interests` = ["daydreaming", "hacking"]. A similar JSON record of our own:

```
{
  "bookTitle": "Dune",
  "pageCount": 412,
  "genres": ["sci-fi", "classic"]
}
```

- `bookTitle` is a string, `pageCount` an integer, `genres` a list of strings. Any schemaless binary JSON format must still store the three field-name strings inside the bytes.

MessagePack walkthrough (encoding the book's Example 4-1):

1. First byte **0x83**: the high nibble 0x80 means "object", the low nibble 0x03 means it has **three fields**. (Objects with more than 15 fields don't fit in four bits; they get a different type marker and the count is stored in two or four bytes.)
2. Second byte **0xa8**: high nibble 0xa0 means "string", low nibble 0x08 gives its length of **eight bytes**.
3. The next eight bytes: the field name `userName` in ASCII. The length was already given, so no end marker or escaping is needed.
4. The next seven bytes: the six-letter value `Martin`, with the prefix **0xa6**; and so on.

Result: 66 bytes, versus **81 bytes** for textual JSON with whitespace removed — only slightly smaller. All binary JSON variants are similar. It's unclear whether such a small saving (and perhaps faster parsing) is worth losing human-readability. The following formats get the same record down to **32 bytes**.

Figure 4-1 (redrawn): Example 4-1 encoded with MessagePack — every field name travels inside the bytes



Total: 66 bytes.

Thrift and Protocol Buffers

What they are. Apache Thrift (originally from **Facebook**) and **Protocol Buffers / protobuf** (originally from **Google**) are binary encoding libraries built on the same principle. Both were open-sourced in **2007–08**.

Both require a schema. You write it in an **interface definition language (IDL)**. Thrift-style, for our own book record:

```
struct Book {
  1: required string       bookTitle,   // tag 1, must be present
  2: optional i64          pageCount,   // tag 2, 64-bit integer, may be absent
  3: optional list<string> genres       // tag 3, list of strings
}
```

- Each field has a **tag number** (1, 2, 3), a `required`/`optional` marker, a type, and a name.

The Protocol Buffers version looks almost the same:

```
message Book {
  required string book_title = 1;   // field tag 1
  optional int64  page_count = 2;   // field tag 2
  repeated string genres     = 3;   // "repeated" = may appear many times (a list)
}
```

Each comes with a **code generation tool** that turns the schema into classes in many languages; your application calls that generated code to encode/decode records.

Thrift has two binary formats (confusingly): **BinaryProtocol** and **CompactProtocol**.

> **Footnote:** actually three — **DenseProtocol** too, but only the C++ implementation supports it, so it

> isn't cross-language. There are also two JSON-based encodings. "What fun!" as the book jokes.

Thrift BinaryProtocol — Example 4-1 takes **59 bytes**:

- Like MessagePack, each field has a **type annotation** (string, integer, list…) and where needed a **length** (string length, list item count). Strings ("Martin", "daydreaming", "hacking") are ASCII (really UTF-8).
- **Big difference: no field names.** Instead each field carries a **field tag** — the numbers 1, 2, 3 from the schema. Tags act as compact **aliases** for fields.

Figure 4-2 (redrawn): Example 4-1 in Thrift BinaryProtocol — type byte, 2-byte tag, then length/value

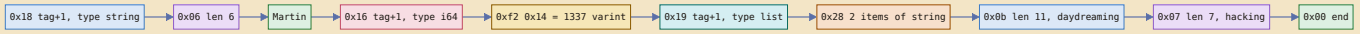


Total: 59 bytes.

Thrift CompactProtocol — same meaning, only **34 bytes**:

- Packs the **field type and tag number into one byte**.
- Uses **variable-length integers**: 1337 takes **two bytes** instead of eight. The top bit of each byte says whether more bytes follow. So numbers from **–64 to 63** fit in one byte, **–8192 to 8191** in two bytes, and so on; bigger numbers use more bytes.

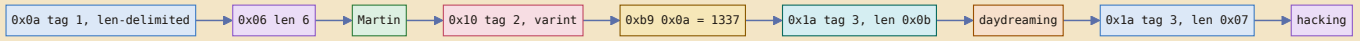
Figure 4-3 (redrawn): Example 4-1 in Thrift CompactProtocol — type+tag share a byte, integers are varints



Total: 34 bytes.

Protocol Buffers — has **one** binary format; bit packing differs slightly but is very similar to CompactProtocol. **33 bytes**.

Figure 4-4 (redrawn): Example 4-1 in Protocol Buffers — a repeated field simply appears once per element



Total: 33 bytes.

Byte-size scoreboard for Example 4-1:

Encoding	Bytes
Textual JSON (whitespace removed)	81
MessagePack	66
Thrift BinaryProtocol	59
Thrift CompactProtocol	34
Protocol Buffers	33
Avro	32

> **Common confusion — required vs optional:** the marker doesn't change the encoding at all; nothing in the bytes says whether a field was required. All `required` does is switch on a

> **check at runtime**, which raises an error when the field is missing — useful for spotting bugs.

Field tags and schema evolution

Schema evolution = schemas inevitably change over time. How do Thrift and protobuf keep backward and forward compatibility?

An encoded record is just its encoded fields concatenated. Every field carries its **tag number** plus a **datatype** annotation, and fields with no value are just **left out**. Hence the tags are what give the bytes their meaning:

- **Renaming a field is fine** — encoded data never contains names.
- **Changing a field's tag is not** — it would make all existing encoded data invalid.

Adding a field:

1. Give it a **new tag number**.
2. **Forward compatibility:** old code reading new data sees an unknown tag and **ignores** it. The datatype annotation tells the parser **how many bytes to skip**.
3. **Backward compatibility:** since each tag still means the same thing, newer code has no trouble with older data. **But** the new field **cannot be** `required` — new code reading old data would fail the check, since old code never wrote it. So every field added after the initial deployment must be **optional or have a default**.

Removing a field is the mirror image:

- You may only remove an **optional** field (a required field can never be removed).
- You may **never reuse** that tag number, because old data containing it may still exist and new code must ignore it.

Datatypes and schema evolution

Changing a field's type may be possible (check the docs), but risks **losing precision or truncation**. Example: 32-bit → 64-bit integer:

- New code reading old data: fine — the parser fills missing bits with zeros.
- Old code reading new data: it still stores the value in a **32-bit** variable, so a 64-bit value that doesn't fit gets **truncated**.

Protocol Buffers `repeated`**:** protobuf has **no list/array type**; it has a `repeated` marker (a third option beside `required` and `optional`). As Figure 4-4 shows, a repeated field is just the **same tag appearing multiple times**. Handy side effect: a field that was **optional (single value)** may later become **repeated (many values)**:

- Newer code, given old records, gets a list holding **zero or one** item (depending on whether the field was set).
- Older code, given new records, just keeps the **final** item of the list.

Thrift has a real **list** type, parameterised by element type. It doesn't allow that single→multi evolution, but it supports **nested lists**.

Avro

What it is. **Apache Avro** is another binary format, interestingly different from protobuf and Thrift. It began in **2009** as a **Hadoop** subproject because Thrift didn't fit Hadoop's use cases well.

Two schema languages: Avro IDL (for humans to edit) and a **JSON-based** one (easier for machines). Our own example in Avro IDL:

```
record Book {
  string               bookTitle;            // plain string, no tag number
  union { null, long } pageCount = null;     // may be null or a long; default null
  array<string>        genres;               // list of strings
}
```

The equivalent JSON schema:

```
{
  "type": "record",
  "name": "Book",
  "fields": [
    {"name": "bookTitle", "type": "string"},
    {"name": "pageCount", "type": ["null", "long"], "default": null},
    {"name": "genres",    "type": {"type": "array", "items": "string"}}
  ]
}
```

- `"type": ["null", "long"]` is a union; `"default": null` is used when a reader expects this field but the writer didn't provide it.

No tag numbers. Encoding Example 4-1 takes only **32 bytes** — the most compact of all.

How the bytes look: there's **nothing identifying fields or their types** — just values concatenated. A string is a length prefix plus UTF-8 bytes, but nothing says it's a string; it could be an integer or anything else. Integers use variable-length encoding (same as Thrift CompactProtocol).

Figure 4-5 (redrawn): Example 4-1 in Avro — only values, in schema order, with no tags or type markers (lengths and counts are zigzag varints, so 0x0c means 6)



Total: 32 bytes.

Consequence: to parse, you walk the fields **in schema order**, using the schema to know each type. So data decodes correctly **only if the reader uses exactly the same schema as the writer** — any mismatch means garbage. So how does Avro allow evolution?

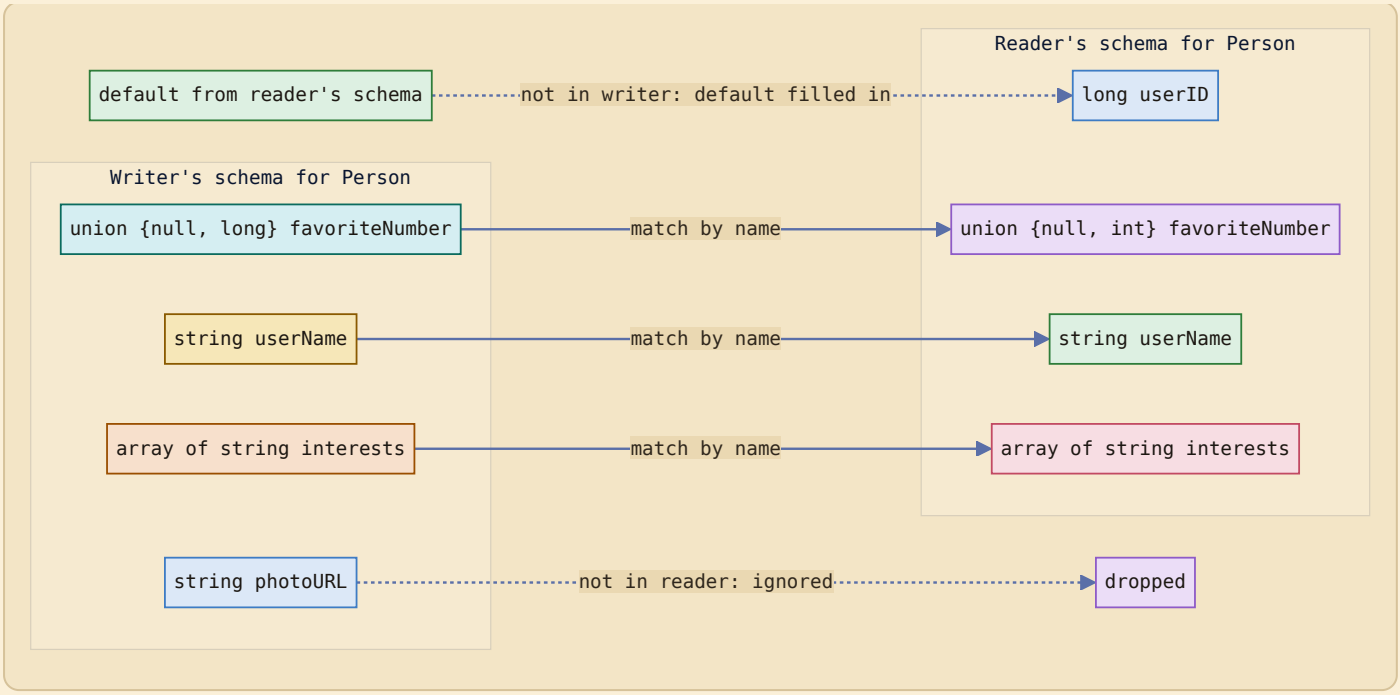
The writer's schema and the reader's schema

- **Writer's schema:** when an app encodes data (for a file, database, network…), it uses whatever schema version it knows — possibly compiled into it.
- **Reader's schema:** when an app decodes data, it expects a certain schema — the one its code relies on, perhaps generated at build time.

Key idea: the two **need not be identical, only compatible**. At decode time, the Avro library puts the writer's and reader's schemas **side by side** and translates data from the former into the latter (**schema resolution**, precisely defined in the Avro specification):

1. **Field order may differ** — fields are matched **by name**.
2. A field in the **writer's** schema but not the **reader's** → **ignored**.
3. A field the **reader** expects but the **writer's** schema lacks → filled with the **default** declared in the reader's schema.

Figure 4-6 (redrawn): an Avro reader resolves the writer's and reader's Person schemas by field name. Order differs between the two; photoURL exists only in the writer's schema, userID only in the reader's



Schema evolution rules

In Avro:

- **Forward compatibility** = new schema as **writer**, old schema as **reader**.
- **Backward compatibility** = new schema as **reader**, old schema as **writer**.

Rule: a field may be **added or removed only if it has a default value**. (In the book's schema, `favoriteNumber` defaults to `null`.)

- **Add a field with a default:** a new-schema reader reading old records fills in the default.
- **Add a field without a default:** new readers can't read old writers' data → **breaks backward compatibility**.
- **Remove a field without a default:** old readers can't read new writers' data → **breaks forward compatibility**.

Null handling via unions. In some languages anything can be null by default; **not in Avro**. To allow null you must use a **union**, e.g. `union { null, long, string } field;` means the field is a number, a string, or null. **`null` can only be a default if it's one of the union's branches.** More verbose, but being explicit about nullability prevents bugs.

> **Footnote:** strictly, the default must match the type of the union's **first branch** — an Avro-specific limitation, not a general property of union types.

Therefore Avro has **no `optional`/`required` markers** like protobuf/Thrift; it uses **union types and defaults** instead.

Other changes:

- **Changing a field's type:** allowed if Avro can convert between the types.
- **Renaming a field:** possible but tricky — the reader's schema can list **aliases**, matching an old writer's field names. So renaming is **backward compatible but not forward compatible**.
- **Adding a branch to a union:** likewise **backward compatible but not forward compatible**.

But what is the writer's schema?

How does a reader know which writer's schema encoded a piece of data? Attaching the full schema to every record would waste far more space than the binary encoding saves. It depends on the context:

Context	How the reader gets the writer's schema
Large file with lots of records (common with Hadoop; see Chapter 10)	Millions of records share one schema, so the writer puts the schema **once at the start of the file**. Avro defines **object container files** for this.
Database with individually written records	Records written at different times may use different schemas. Simplest fix: prefix every record with a **version number** and keep a **list of schema versions in the database**. The reader extracts the version, fetches that writer's schema, and decodes the rest. **Espresso** works this way.
Sending records over a network connection	Two processes on a **bidirectional** connection **negotiate the schema version at connection setup** and use it for the whole connection. The **Avro RPC** protocol does this.

A **database of schema versions** is useful anyway: it's documentation and lets you **check compatibility** of schema changes. The version can be a simple **incrementing integer** or a **hash of the schema**.

Dynamically generated schemas

Why does it matter that Avro schemas have **no tag numbers**? Because Avro is friendlier to **dynamically generated schemas**.

Example: dumping a relational database to a binary file (avoiding the textual-format problems of JSON, CSV, SQL):

1. Generate an Avro schema (JSON form) from the relational schema — **one record schema per table**, **one field per column**, with **column name = field name**.
2. Encode the table contents with it and dump everything into an Avro **object container file**.
3. When the database schema changes (say one column added, one removed), just **regenerate** the Avro schema and export with it. The export needn't care about the change.
4. Readers of new files see changed fields, but since fields are matched **by name**, the new writer's schema still resolves against an old reader's schema.

With Thrift or protobuf, field tags would probably be assigned **by hand**: every database schema change would require an administrator to update the column-name → tag mapping. Automating it is possible, but the generator must be very careful **never to reuse old tags**. Dynamic schema generation just **wasn't a design goal** of Thrift/protobuf; it **was** for Avro.

Code generation and dynamically typed languages

- **Thrift and protobuf rely on code generation.** That suits **statically typed** languages (**Java, C++, C#**): efficient in-memory structures for decoded data, plus type checking and IDE autocompletion.
- In **dynamically typed** languages (**JavaScript, Ruby, Python**), there's no compile-time checker to satisfy, so generated code adds little. Code generation is often frowned upon there because they otherwise avoid a compile step. When the schema itself is **generated dynamically** (say, from a database table), a code-generation step just gets in the way of reading the data.
- **Avro** offers **optional** code generation for static languages but works fine without it. An **object container file** embeds the writer's schema, so you can open it with the Avro library and browse it like a JSON file — it's **self-describing**.
- This is especially handy with dynamically typed data processing languages like **Apache Pig**: open Avro files, analyse them, and write derived datasets back as Avro, without thinking about schemas.

The Merits of Schemas

Protobuf, Thrift and Avro all use schemas for a binary format. Their schema languages are **far simpler** than **XML Schema** or **JSON Schema**, both of which offer elaborate validation rules (e.g., "this

string must match a regular expression", "this integer must be between 0 and 100"). Because they're easier to implement and use, libraries for them now exist in quite a broad set of languages.

Not a new idea:

- **ASN.1**, a schema definition language first standardised in **1984**, was used to define many network protocols. Its binary encoding **DER** still encodes **SSL certificates (X.509)**. ASN.1 supports schema evolution with **tag numbers**, like protobuf/Thrift — but it's very complex and badly documented, so it's unlikely to be a wise pick for a new application.
- Many data systems use their own **proprietary binary encodings**. E.g., most relational databases have a **network protocol** for queries and responses, specific to that database; the vendor supplies a **driver** (e.g., via **ODBC** or **JDBC** APIs) that decodes responses into in-memory structures.

Nice properties of schema-based binary encodings:

1. **Much more compact** than "binary JSON" variants, because field names can be left out.
2. The schema is **valuable documentation** — and since decoding requires it, it's guaranteed **up to date** (unlike hand-written docs that drift).
3. A **database of schemas** lets you check forward/backward compatibility **before deploying** anything.
4. For statically typed languages, **generated code** enables compile-time type checking.

Bottom line: schema evolution offers flexibility comparable to schemaless (schema-on-read) JSON databases, **while adding** stronger guarantees about your data and better tooling.

Format family	Examples	Size	Human-readable	Schema	Compatibility story
Language-specific	Java Serializable, Ruby Marshal, Python pickle, Kryo	Often bloated	No	Implicit in classes	Often none; insecure
Textual	JSON, XML, CSV	Large	Yes	Optional (XML/JSON), none (CSV)	Depends on usage; number/binary ambiguity
Binary JSON/XML	MessagePack, BSON, WBXML…	Slightly smaller	No	None	Like JSON; field names still stored
Schema-driven binary	Thrift, Protocol Buffers	Very compact (tags)	No	Required, with tags	Clear rules via tag numbers
Schema-driven binary	Avro	Most compact (no tags)	No	Writer's + reader's schema	Clear rules via name matching + defaults

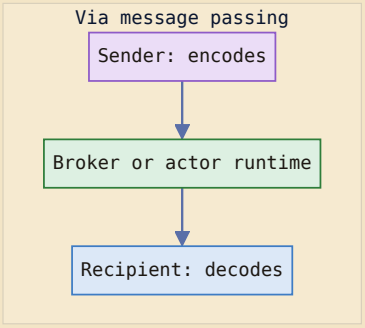
---

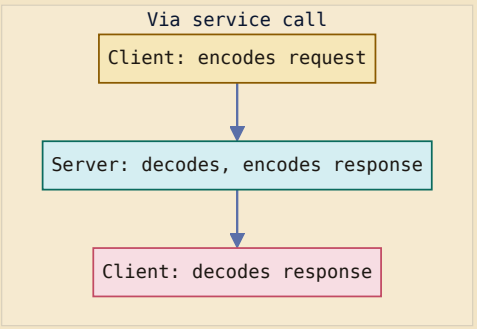
Modes of Dataflow

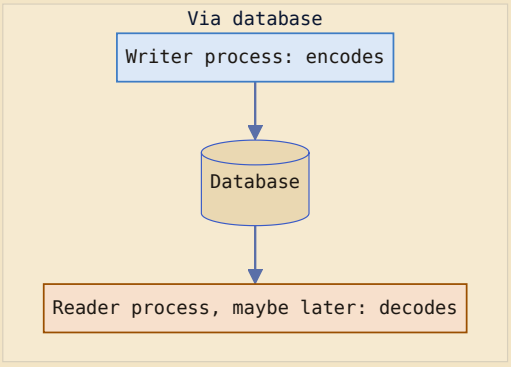
Whenever you send data to a process you don't share memory with — over the network or into a file — you must encode it as bytes. Compatibility is a relationship between **the process that encodes** and **the process that decodes**. It matters for evolvability: it lets you **upgrade parts of the system independently** instead of all at once.

But *who* encodes and *who* decodes? The rest of the chapter looks at the most common ways data flows between processes:

- **Through databases**
- **Through service calls** (REST and RPC)
- **Through asynchronous message passing**







Dataflow Through Databases

Who encodes/decodes: whoever **writes** to the database does the encoding; whoever **reads** does the decoding.

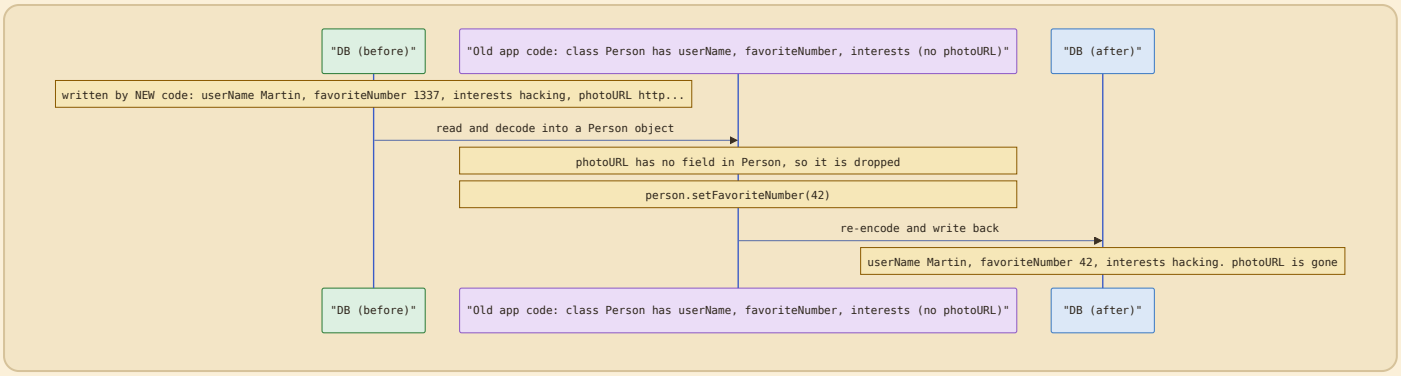
- **Single process:** the reader might just be a **later version** of the same process — writing to the database is like **leaving a note for your future self**. That makes **backward compatibility** obviously necessary, or your future self can't read what you wrote.
- **Many processes:** usually several apps/services, or several instances of one service (for scale or fault tolerance), access the database at once. During a **rolling upgrade**, some run new code and some old. So a value may be **written by new code and later read by old code → forward compatibility** is often needed too.

The extra snag — preserving unknown fields:

1. A new field is introduced in the schema, and **new code** stores a value in it.
2. Then **old code**, which has never heard of that field, loads the record, **modifies** something, and saves it.
3. What you normally want: the old code **leaves the new field untouched**, despite not understanding it.

The encoding formats above support preserving unknown fields, but sometimes you must handle it **at the application level**: e.g., if you decode into **model objects** and re-encode them, the unknown field can be **lost** in that translation. Not hard to solve — just be aware of it.

Figure 4-7 (redrawn): an older app version reads a record written by a newer version, updates one field through its model object, and writes it back. The newer field (`photoURL`) is silently lost.



Different values written at different times

- A database lets **any value be updated at any time**, so one database may contain values written **five milliseconds ago** alongside values written **five years ago**.
- A new server-side app version can **fully replace** the old one within **minutes**. Database contents don't change that way: the **five-year-old data stays in its original encoding** unless explicitly rewritten. This is summed up as **"data outlives code."**
- **Migrating** (rewriting) data into a new schema is possible but **expensive** on large datasets, so most databases avoid it.
- **Most relational databases** allow simple changes, like **adding a column with a null default**, **without rewriting** existing data; when reading an old row, missing columns are filled with nulls.

> **Footnote:** except **MySQL**, which often rewrites the whole table even when not strictly needed (Chapter 2).

- **LinkedIn's Espresso** document database stores data in **Avro**, so it can use Avro's schema evolution rules.
- **Net effect:** schema evolution lets the whole database **appear** to use one schema, even though storage contains records encoded with many historical schema versions.

Archival storage

- You may **snapshot** the database periodically, e.g. for **backups** or loading into a **data warehouse** (Chapter 3).
- The dump is typically encoded with the **latest schema**, even if the source mixed many schema versions — since you're copying all the data anyway, you may as well encode it consistently.
- The dump is written once and then **immutable**, so **Avro object container files** fit well. It's also a chance to write the data in a **column-oriented** layout that suits analytics, such as **Parquet**.
- More on archival data in Chapter 10.

Dataflow Through Services: REST and RPC

Clients and servers. The most common arrangement for network communication: **servers** expose an API over the network; **clients** connect and send requests. The API a server exposes is called a **service**.

- **The web works this way:** browsers (clients) issue **GET** requests to fetch HTML, CSS, JavaScript and images, and **POST** requests to send data to the server. The API is a set of **standard** protocols and formats (HTTP, URLs, SSL/TLS, HTML…). Because browsers, servers and site authors largely agree on these standards, any browser can in theory access any website.
- **Other clients:** native mobile/desktop apps also make network requests; client-side JavaScript in a browser can use **XMLHttpRequest** to act as an HTTP client — a technique called **Ajax**. Here the

response is usually not HTML for humans but data (e.g., **JSON**) for the client code. HTTP is the transport, but the API on top is **application-specific**, and client and server must agree on it.

- **A server can itself be a client** of another service (e.g., a web app server is a client of its database). Decomposing a large application into smaller services by functionality, each calling others as needed, has long been known as **service-oriented architecture (SOA)**; a refined, renamed version of the idea is now called **microservices**.

Services vs databases. Both let clients submit and query data. But databases accept **arbitrary queries** (Chapter 2's query languages), while services expose an **application-specific API** where inputs and outputs are fixed by the service's business logic. This gives **encapsulation**: services can finely restrict what clients may do.

Why compatibility matters here: a key goal of SOA/microservices is to make applications easier to change by making services **independently deployable and evolvable** — each service owned by **one team** that can release often **without coordinating** with others. So expect **old and new versions of clients and servers running simultaneously**, and the data encoding must be compatible across API versions — exactly this chapter's theme.

Web services

When a service uses **HTTP** as its protocol, it's a **web service** — a slight misnomer, since it's used well beyond the web:

1. **Client app on a user's device** (native mobile app, or JavaScript web app using Ajax) calling a service over HTTP, typically across the **public internet**.
2. **Service to service within one organisation**, often in the **same datacenter**, as part of SOA/microservices. (Software supporting this is sometimes called **middleware**.)
3. **Service to service across organisations**, usually over the internet — data exchange between companies' backends. Examples are **public APIs** of online services, like **credit card processing**, or **OAuth** for letting one service access user data held by another.

Two popular approaches: REST and SOAP — almost diametrically opposed philosophies, and the subject of heated debate.

> **Footnote:** even inside each camp there are arguments; **HATEOAS** (short for hypermedia as the engine of application state), for instance, is a frequent source of debate.

REST (Representational State Transfer):

- **Not a protocol** but a **design philosophy** built on HTTP's principles.
- Emphasises **simple data formats**, **URLs to identify resources**, and **HTTP's built-in features** for caching control, authentication and negotiating content types.
- Gaining popularity over SOAP, at least for **cross-organisational** integration, and often associated with **microservices**. An API following REST is **RESTful**.

SOAP:

- An **XML-based protocol** for network API requests. Usually used over HTTP, but aims to be **independent of HTTP** and avoids most HTTP features.
- Comes with a sprawling, complex family of related standards — the **web service framework, WS-*** — adding various features.
- APIs are described in **WSDL (Web Services Description Language)**, XML-based. WSDL enables **code generation** so clients can call the remote service via local classes and method calls (encoded to XML and back by the framework). Useful for static languages, less so for dynamic ones.
- WSDL isn't designed to be human-readable and SOAP messages are often too complex to build by hand, so SOAP users rely heavily on **tools, code generation and IDEs**. Languages not supported by SOAP vendors have a hard time integrating.

- Although nominally standardised, **interoperability between vendors' implementations** often causes problems.
- As a result, SOAP survives in plenty of **large enterprises** but most smaller companies have **moved away** from it.

> **Footnote:** despite similar acronyms, **SOAP is not required for SOA**. SOAP is one particular technology, while SOA is a broad way of structuring systems.

RESTful APIs favour simpler approaches with less code generation and automated tooling. A definition format like **OpenAPI** (also known as **Swagger**) can describe RESTful APIs and produce documentation.

	REST	SOAP
Nature	Design philosophy on top of HTTP	XML-based protocol, HTTP-independent
Data format	Simple (commonly JSON)	XML
Uses HTTP features	Yes (URLs, caching, auth, content negotiation)	Mostly avoids them
Description	OpenAPI / Swagger (optional)	WSDL
Tooling dependence	Low	High (code generation, IDEs)
Where popular	Public/cross-org APIs, microservices	Large enterprises

The problems with remote procedure calls (RPCs)

Web services are just the latest in a long line of technologies for API calls over a network — many heavily hyped but seriously flawed:

- Java's **Remote Method Invocation (RMI)** and **Enterprise JavaBeans (EJB**, part of Java Enterprise Edition): **Java-only**.
- **DCOM (Distributed Component Object Model)**: limited to **Microsoft** platforms.
- **CORBA (Common Object Request Broker Architecture)**: **excessively complex**, and provides **no backward or forward compatibility**.

All are based on the **remote procedure call (RPC)**, around since the **1970s**. RPC tries to make a remote network call **look like a local function call** in the same process — an abstraction called **location transparency**.

Why that's fundamentally flawed — a network request differs from a local call:

Local function call	Network request
Predictable: succeeds or fails based only on parameters you control.	**Unpredictable**: request/response may be lost, remote machine slow or down — none of it under your control. Such failures happen often, so you must expect them (e.g., by **retrying**).
Returns a result, throws an exception, or never returns (infinite loop/crash).	Can also **time out** and return nothing — then you **don't know** whether the request got through (see Chapter 8).
Calling twice is something you choose to do.	**Retrying** may repeat an action whose response was lost, unless the protocol has **deduplication (idempotence)** (see Chapter 11).
Takes about the same time each call.	**Much slower** and **wildly variable** latency: under a millisecond at good times, **many seconds** when the network is congested or the service overloaded.
Can pass **pointers/references** to local objects efficiently.	Every parameter must be **encoded into bytes** — fine for numbers/strings, problematic for large objects.
Same language on both sides.	Client and service may be in **different languages**, so the framework must translate datatypes — can be ugly (recall JavaScript's trouble with numbers above **2^53**).

Conclusion: it's pointless to disguise a remote service as a local object, because the two are fundamentally unlike each other. Part of REST's appeal is that it **doesn't hide** that it's a network protocol (though people still build RPC libraries on top of REST).

Current directions for RPC

RPC isn't going away. Frameworks exist on top of every encoding in this chapter:

- **Thrift** and **Avro** include RPC support.
- **gRPC** is RPC built on **Protocol Buffers**.
- **Finagle** uses **Thrift**.
- **Rest.li** uses **JSON over HTTP**.

This new generation is more honest that remote ≠ local:

- **Finagle** and **Rest.li** wrap asynchronous, possibly-failing actions in **futures (promises)**. Futures also make it easier to call **several services in parallel** and combine the results.
- **gRPC** supports **streams**: one call may involve a **stream** of many requests and many responses spread over time, rather than exactly one of each.
- Some frameworks provide **service discovery** — letting a client find the IP address and port of a service (more in "Request Routing", Chapter 6).

RPC vs REST trade-off:

- Custom RPC protocols with **binary encoding** can **outperform** generic JSON-over-REST.
- But **RESTful APIs** have big advantages: easy to **experiment and debug** (just use a browser or **curl**, no code generation or installs), supported by **all mainstream languages and platforms**, and a **huge ecosystem** of supporting tools — servers, caches, proxies, load balancers, firewalls, plus monitoring, testing and debugging tools.
- Hence **REST dominates public APIs**, while **RPC frameworks** focus on requests **between services of the same organisation**, usually in the **same datacenter**.

```
# Poking a RESTful API directly from the command line, no generated client needed
curl -H "Accept: application/json" https://api.example.com/v2/books/42
```

- `-H "Accept: application/json"` asks for JSON; `/v2/` shows one common way of choosing an API version (in the URL).

Data encoding and evolution for RPC

For evolvability, RPC clients and servers must be changeable and deployable **independently**. Services allow a **simplifying assumption** compared with databases: **servers are updated first, clients second**. So you need:

- **Backward compatibility on requests** (new server reads old client's requests),
- **Forward compatibility on responses** (old client reads new server's responses).

An RPC scheme's compatibility is **inherited from its encoding**:

- **Thrift, gRPC (Protocol Buffers), Avro RPC** evolve under their encoding's rules.
- **SOAP:** requests/responses are defined with **XML schemas**; they can evolve but there are subtle pitfalls.
- **RESTful APIs:** responses are typically **JSON with no formally specified schema**; requests use JSON or **URI-encoded / form-encoded** parameters. Changes generally seen as safe: **new optional request parameters** and **extra fields in response objects**.

Harder across organisations: the provider often can't control or force clients to upgrade, so compatibility may need to last a **very long time, perhaps forever**. If a breaking change is needed, providers often run **multiple API versions side by side**.

API versioning — no agreement on how clients should indicate a version:

- For REST, commonly: put the **version in the URL**, or put it in the **HTTP `Accept` header**.
- For services using **API keys** to identify clients: the server can **remember which API version each client asked for**, and let clients change it through a **separate admin interface**.

Message-Passing Dataflow

So far: **REST/RPC** (one process sends a request and expects a quick response) and **databases** (one writes, another reads maybe much later). **Asynchronous message-passing** sits **between** them:

- **Like RPC:** a client's request (**message**) reaches another process with **low latency**.
- **Like a database:** the message doesn't go over a direct connection but through an intermediary — a **message broker** (other names: **message queue**, **message-oriented middleware**) that **holds on to it for a while**.

Advantages of a broker over direct RPC:

1. Serves as a **buffer** when the recipient is down or overwhelmed → more reliable system.
2. Can **redeliver** messages to a crashed process → prevents message loss.
3. Sender needn't know the recipient's **IP address and port** — very useful in the cloud where **virtual machines come and go**.
4. One message can go to **several recipients**.
5. **Logically decouples** sender and recipient: the sender just publishes and doesn't care who consumes.

Difference from RPC: communication is usually **one-way** — the sender doesn't normally expect a reply. A reply is possible but usually via a **separate channel**. This makes it **asynchronous**: the sender fires off the message and moves on without waiting to see it delivered.

Message brokers

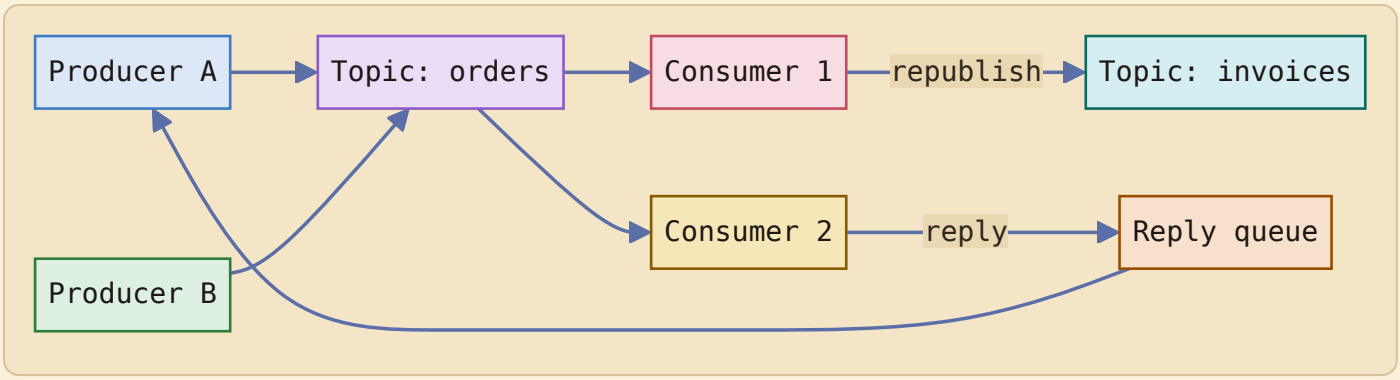
- **Past:** dominated by commercial enterprise software from **TIBCO**, **IBM WebSphere** and **webMethods**.
- **Now:** open-source **RabbitMQ**, **ActiveMQ**, **HornetQ**, **NATS** and **Apache Kafka** are popular (compared in detail in Chapter 11).

How they're used (general pattern; details vary):

1. A producer publishes a message to a **queue** or **topic** identified by name.
2. The broker ensures it's delivered to one or more **consumers** or **subscribers** of that queue/topic.

3. One topic may have **multiple producers as well as multiple consumers**.

Message broker dataflow (illustrative, not a numbered book figure)



- A topic is **one-way** only. But a consumer can **publish to another topic** (chaining, see Chapter 11) or to a **reply queue** read by the original sender (giving request/response similar to RPC).
- Brokers usually **don't enforce a data model**: a message is just **bytes plus some metadata**, so any encoding works. If it's **backward and forward compatible**, publishers and consumers can change independently and deploy **in any order**.
- When a consumer **re-publishes** messages onto a further topic, take care to **preserve unknown fields** — the same issue as with databases (Figure 4-7).

Distributed actor frameworks

The actor model is a concurrency programming model **within a single process**. Instead of dealing with threads directly (with race conditions, locking and deadlock), logic lives in **actors**:

- Each actor usually represents **one client or entity**, may hold **local state** (not shared with other actors), and talks to others by sending and receiving **asynchronous messages**.
- **Message delivery is not guaranteed** — in some error cases, messages are lost.
- Each actor handles **one message at a time**, so it needn't worry about threads, and the framework can schedule actors independently.

Distributed actor frameworks use this model to scale across **multiple nodes**. Messaging works identically whether the two actors share a node or not; across nodes, the framework quietly **encodes the message, ships it over the network, and decodes it** at the far end.

Location transparency fits actors better than RPC: actors are **already designed around possible message loss**, even inside one process. Network latency is higher, but local vs remote is less fundamentally mismatched.

A distributed actor framework effectively **combines a message broker with the actor model**. For **rolling upgrades** you still need forward and backward compatibility, since messages may go from a new-version node to an old one and vice versa.

How three popular frameworks encode messages:

Framework	Default encoding	Rolling upgrades
Akka	**Java's built-in serialization** — no forward/backward compatibility	Possible if you swap in e.g. **Protocol Buffers**
Orleans	A **custom** encoding without rolling-upgrade support	To upgrade: set up a **new cluster**, move traffic over, shut down the old one. Custom serialization plug-ins can be used, as with Akka.
Erlang OTP	Record schemas are surprisingly hard to change, despite many high-availability features	Possible but must be **planned carefully**. A newer experimental **maps** datatype (JSON-like; arrived in **Erlang R17, 2014**) could ease this in future.

Summary

(Book section) The chapter looked at how data structures become bytes on the network or on disk, and how those encoding choices affect not only **efficiency** but, more importantly, **application architecture** and **deployment options**.

- Many services need **rolling upgrades**: the new version spreads gradually across nodes instead of hitting all of them simultaneously. This allows releases **without downtime** (encouraging frequent small releases over rare big ones) and makes deployments **less risky** (bad releases can be caught and rolled back before affecting many users). Both help **evolvability**.
- Because different nodes may run different code versions, all data flowing through the system must be encoded with **backward compatibility** (new code reads old data) and **forward compatibility** (old code reads new data).

Encoding formats and their compatibility:

- **Language-specific encodings:** limited to one language, often without forward/backward compatibility.
- **Textual formats (JSON, XML, CSV):** widespread; compatibility depends on how you use them. Optional schema languages can help or hinder. Vague about datatypes — take care with **numbers** and **binary strings**.
- **Binary schema-driven formats (Thrift, Protocol Buffers, Avro):** compact, efficient, with **clearly defined** compatibility semantics; schemas help documentation and code generation in static languages. Downside: data must be **decoded before a human can read it**.

Modes of dataflow:

- **Databases:** writer encodes, reader decodes.
- **RPC and REST APIs:** client encodes a request, server decodes it and encodes a response, client decodes the response.
- **Asynchronous message passing** (brokers or actors): sender encodes, recipient decodes.

Conclusion: a little care makes both compatibility directions — and therefore rolling upgrades — entirely realistic.

Looking ahead

That closes Part I, which dealt with data on a single machine. Part II, *Distributed Data*, opens with a short introduction on why data gets spread across several machines (scalability, fault tolerance, latency) and on shared-memory, shared-disk and shared-nothing architectures. That introduction is summarised at the start of the Chapter 5 notes, which then cover replication.

Chapter summary

- Old and new **code and data coexist** (rolling upgrades, slow client updates), so you need **backward** (new reads old) and **forward** (old reads new) compatibility.
- **Language-specific formats** (Java Serializable, Ruby Marshal, Python pickle, Kryo): convenient but language-locked, insecure, poor versioning and efficiency.
- **JSON/XML/CSV**: universal and readable, but ambiguous numbers (2^53 problem; Twitter's string IDs), no binary strings (Base64 +33%), optional/complex or missing schemas. Binary JSON (MessagePack: 66 vs 81 bytes) saves little.

- **Thrift/Protocol Buffers**: schema + **field tags**; Thrift BinaryProtocol 59 bytes, CompactProtocol 34, protobuf 33. Rename fields freely; never change or reuse tags; new fields must be optional/defaulted; only optional fields can be removed.
- **Avro**: no tags, 32 bytes; **writer's vs reader's schema** resolved by field name; add/remove only fields with defaults; nulls via unions; writer's schema found via container file header, per-record version number, or connection negotiation. Great for **dynamically generated schemas** and dynamic languages.
- **Schemas** give compactness, up-to-date documentation, compatibility checks and code generation (ASN.1 did it first, in 1984).
- **Dataflow via databases**: data outlives code; preserve unknown fields; add-null-column without rewrite; archive dumps with the latest schema (Avro/Parquet).
- **Dataflow via services**: REST (HTTP-based philosophy) vs SOAP (XML protocol, WSDL); RPC's location transparency is flawed; modern RPC (gRPC, Finagle, Rest.li, Thrift, Avro) is explicit about the network. Assume servers upgrade first: backward-compatible requests, forward-compatible responses. Versioning via URL, Accept header, or per-API-key settings.
- **Dataflow via messages**: brokers buffer, redeliver, decouple, fan out; one-way and async. Actor frameworks (Akka, Orleans, Erlang OTP) still need compatible encodings for rolling upgrades.

Interview angle

1. What are backward and forward compatibility, and why do you need both? *30-second answer:* Backward = new code reads old data; forward = old code reads new data. During rolling upgrades and with un-updated clients, old and new versions run at the same time and read each other's data, so both are needed. Backward is easy (you know the old format); forward requires old code to ignore unknown additions.

- *Follow-up: Which one does a database need?* Both: a future version reads old data (backward), and during a rolling upgrade old instances read rows written by new ones (forward). Also preserve unknown fields on read-modify-write.
- *Follow-up: Which one do services need?* Assuming servers upgrade first: backward compatibility for requests, forward compatibility for responses.

2. How do Protocol Buffers / Thrift handle schema evolution? *30-second answer:* Each field has a numeric tag; encoded data stores tag + type + value, never names. Old code skips unknown tags (using the type to know the length) → forward compatible. New code reads old data since tags keep their meaning → backward compatible, as long as new fields are optional or have defaults. You can rename fields, but never change or reuse tags, and only optional fields may be removed.

- *Follow-up: Can you change int32 to int64?* New code reads old fine; old code reading new values may truncate them.
- *Follow-up: optional → repeated in protobuf?* Allowed: new code sees 0/1-element lists, old code sees the last element.

3. How is Avro different? *30-second answer:* No tag numbers and no type markers in the data — just values in schema order (32 bytes for the example). Decoding needs the exact writer's schema, which is resolved against the reader's schema by field name: unknown fields ignored, missing fields defaulted. Only fields with defaults may be added/removed. The writer's schema is found from a file header, a per-record version number with a schema registry, or negotiation on connection. That makes Avro great for schemas generated automatically, e.g. from database tables.

- *Follow-up: How is null handled?* Via union types like `union { null, long }`; null can be the default only if it's a branch (the first one, strictly).

4. Why is RPC's "make a remote call look local" idea flawed? *30-second answer:* Network calls can be lost, time out with unknown outcome, get duplicated on retry (need idempotence), have highly

variable latency, require encoding all parameters, and cross language type systems. Hiding that leads to fragile systems. Modern frameworks use futures, streams, and service discovery, and REST is explicit that it's a network protocol.

- *Follow-up: REST or RPC?* REST for public APIs (debuggable with curl, universal tooling); binary RPC (gRPC, Thrift) for internal service-to-service calls where performance matters.

5. When would you use a message broker instead of direct RPC? *30-second answer:* When you want buffering against slow or offline consumers, automatic redelivery after crashes, no need to know recipients' addresses, fan-out to many consumers, and decoupled producers/consumers. The trade-off is one-way, asynchronous communication; replies need a separate reply queue.

- *Follow-up: What's the encoding pitfall?* Consumers that republish messages must preserve fields they don't understand.

6. Why avoid Java Serializable / pickle for stored data? *30-second answer:* Language lock-in, remote-code-execution risk from decoding untrusted bytes, weak versioning, and poor efficiency (Java serialization is notoriously bloated). Only use them for very short-lived data.

Self-check

1. **Define backward and forward compatibility. Which is usually harder, and why?** Backward: new code reads old data. Forward: old code reads new data. Forward is harder because old code must ignore additions it doesn't know about.
2. **Why can't JavaScript reliably read Twitter tweet IDs sent as JSON numbers, and how did Twitter work around it?** Tweet IDs are 64-bit, but JavaScript parses numbers as IEEE 754 doubles, exact only up to 2^53. Twitter returns each ID both as a number and as a decimal string.
3. **Rank these by encoded size for Example 4-1: Protobuf, JSON, Avro, MessagePack, Thrift Compact, Thrift Binary.** Avro 32 < Protobuf 33 < Thrift Compact 34 < Thrift Binary 59 < MessagePack 66 < JSON 81.
4. **In Protocol Buffers, can you rename a field? Change its tag? Add a required field?** Rename: yes (names aren't encoded). Change tag: no (existing data becomes invalid). Add required: no (new code would fail on old data); new fields must be optional or have defaults.
5. **Avro data has no field tags or types. How can a reader possibly decode it after the schema changes?** The reader has both the writer's schema (from the file header, a version number lookup, or connection negotiation) and its own reader's schema; the Avro library matches fields by name, ignores extra fields and fills defaults for missing ones.
6. **Scenario: you add a field without a default to an Avro schema. What breaks?** Backward compatibility: new readers can't read records written with the old schema, because there's no value to fill in.
7. **Scenario: an old service version reads a user record, changes the email, and writes it back — and a newly added field disappears. Why, and how do you prevent it?** The old code decoded into a model object that had no slot for the new field, and re-encoding dropped it. Preserve unknown fields (the formats support it) and avoid lossy model translation.
8. **Why is it reasonable to need only backward-compatible requests and forward-compatible responses for RPC?** Because you can assume servers are upgraded before clients: new servers must accept old requests, and old clients must accept new responses.
9. **Give three advantages of a message broker over direct RPC and one disadvantage.** Buffers when the recipient is down/overloaded; redelivers after crashes; sender needn't know recipient's address; fan-out; decoupling. Disadvantage: usually one-way/asynchronous — no direct reply.
10. **Why does location transparency work better for actors than for RPC?** The actor model already assumes messages can be lost even within one process, so going over the network doesn't break its

assumptions — unlike RPC, which pretends a remote call is as reliable as a local one.

Chapter 5 — Replication

Book: Chapter 5, pp. 151–198 (plus the Part II introduction that precedes it)

Overview

First, the Part II introduction: why spread data over many machines?

Part I of the book looked at data that lives on one computer. Part II asks what changes once **several machines** take part in storing and serving the data. The book gives three motivations:

Reason	What it buys you
Scalability	When the amount of data, or the number of reads or writes, outgrows one machine, the work can be shared among several.
Fault tolerance / high availability	If one machine (or several, or the network, or a whole datacenter) dies, a redundant machine can take over so the application keeps working.
Latency	With servers on several continents, each user talks to a nearby datacenter instead of waiting for packets to cross the planet.

Scaling to Higher Load

There are three broad hardware architectures for handling more load:

Architecture	How it works	Strengths	Weaknesses
Shared-memory (vertical scaling, "scaling up")	Buy one bigger machine: many CPUs, RAM chips and disks joined under one operating system; a fast interconnect lets any CPU reach any memory or disk. It behaves like one computer.	Simple to program: it is still "one machine".	Cost rises faster than linearly (a machine twice as big costs much more than twice as much), and bottlenecks mean twice the hardware does not give twice the throughput. Some fault tolerance through hot-swappable parts (disks, memory, even CPUs replaced while running), but it sits in a single location.
Shared-disk	Several machines, each with its own CPUs and RAM, all reading and writing one shared array of disks over a fast network (NAS = Network Attached Storage, or SAN = Storage Area Network).	Used for some data-warehouse workloads.	Contention for the disks and the cost of locking limit how far it scales.
Shared-nothing (horizontal scaling, "scaling out")	Every machine (or VM) that runs the database software is called a **node**, and it relies solely on its own CPUs, RAM and disks. Nodes coordinate purely in software over an ordinary network.	No special hardware, so you pick whatever gives the best price/performance. Data can be spread across regions, cutting latency and even surviving the loss of a whole datacenter. With cloud VMs, even small companies can run multi-region setups.	More complexity for the application; can restrict which data models you can use.

> **NUMA aside.** Even inside one big shared-memory machine, some memory banks are physically nearer to certain CPUs (nonuniform memory access, NUMA). To use such a machine well you must split the work so each CPU mostly touches nearby memory, so a form of partitioning is needed even on "one machine".

The rest of Part II concentrates on **shared-nothing** systems. Not because they are always best, but because they are the ones where the application developer has to be most careful: the database cannot hide all the constraints of distribution from you. The book also warns that a simple single-threaded program can sometimes beat a cluster of more than 100 CPU cores, so distribution is not free performance.

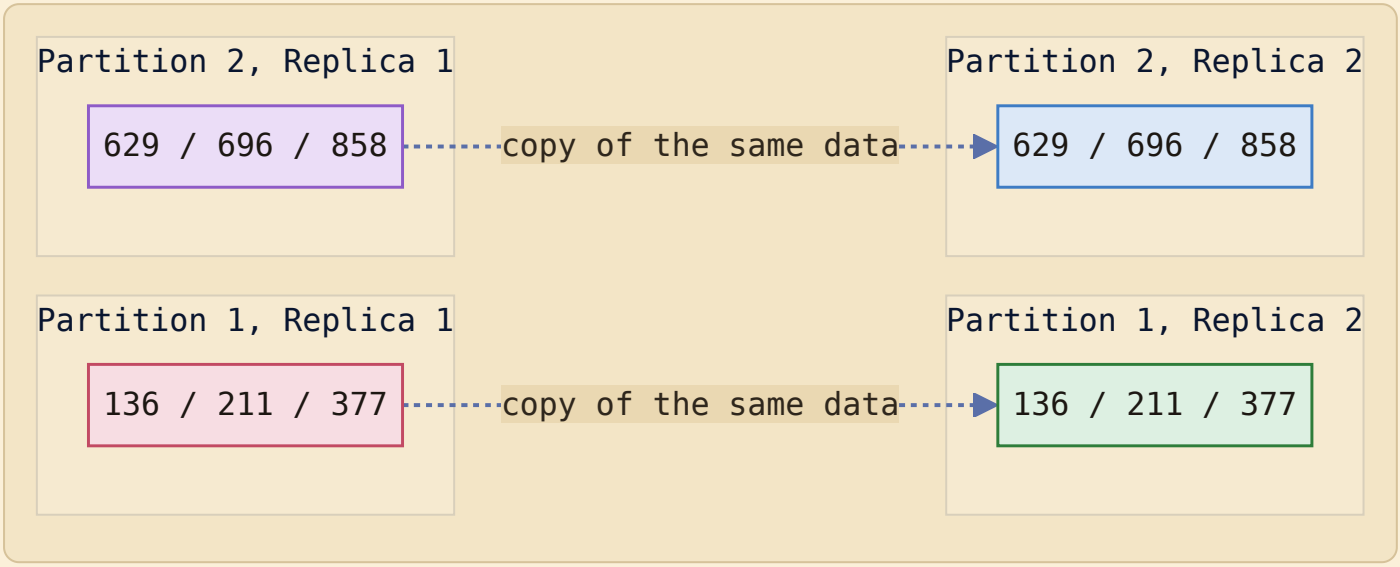
Replication Versus Partitioning

There are two standard ways to lay data out across nodes:

- **Replication**: the *same* data is copied onto several nodes, possibly in different places. This gives redundancy (if some nodes are down, others still serve the data) and can also improve performance. This is Chapter 5.
- **Partitioning** (also called **sharding**): a large database is cut into smaller pieces, called **partitions**, and different partitions go on different nodes. This is Chapter 6.

The two are independent mechanisms but are usually combined.

Figure II-1 (redrawn): a database split into two partitions, each partition kept as two replicas holding the same records (the book shows records keyed 136, 211, 377 in partition 1 and 629, 696, 858 in partition 2).



After these two chapters, Chapter 7 covers transactions, and Chapters 8 and 9 cover the fundamental limits of distributed systems. Part III then shows how to combine several data stores into a larger system.

What this chapter answers

Replication means keeping copies of the same data on several networked machines. The chapter asks: *when data keeps changing, how do we keep all those copies in step, and what goes wrong when they are not?*

Reasons to replicate:

1. Keep data physically near users (lower **latency**).
2. Keep working when some parts fail (higher **availability**).
3. Let more machines answer read queries (higher **read throughput**).

For this chapter we assume every machine can hold the entire dataset (Chapter 6 lifts that assumption). If data never changed, replication would be trivial: copy it once. All the hard parts come from **changes**. The chapter covers the three algorithms that almost every distributed database uses:

1. **Single-leader** replication
2. **Multi-leader** replication
3. **Leaderless** replication

It also covers synchronous vs asynchronous replication, handling failed replicas, and the confusing idea of **eventual consistency**. The theory dates back to the 1970s because the basic limits of networks have not changed; what is newer is that ordinary developers now use distributed databases, which has caused a lot of misunderstanding.

Everyday analogy. A company keeps its price list in several branch offices. One head office edits the list and mails updates (single leader). Or every branch may edit and they post changes to each other (multi-leader). Or a customer phones three branches at once and trusts the answer that most branches give (leaderless). Each scheme fails in different, instructive ways.

Key terms

Term	Plain-English meaning
Replica	Any node holding its own full copy of the data.
Leader (master, primary)	The one replica that accepts writes and sends them to the others.
Follower (read replica, slave, secondary, hot standby)	A replica that copies changes from the leader and only serves reads.
Leader-based replication (active/passive, master–slave)	The scheme where all writes go to one leader and followers copy them.
Replication log / change stream	The ordered list of data changes the leader sends to followers.
Synchronous replication	The leader waits for a follower to confirm a write before reporting success.
Asynchronous replication	The leader reports success without waiting for followers.
Semi-synchronous	One follower is synchronous, the rest asynchronous.
Chain replication	A synchronous-replication variant used in, e.g., Microsoft Azure Storage.
Snapshot	A consistent copy of the whole database as of one moment.
Log sequence number / binlog coordinates	PostgreSQL's / MySQL's name for a precise position in the replication log.
Caught up	A follower has applied all changes up to the leader's current position.
Catch-up recovery	A restarted follower asking the leader for changes it missed.
Failover	Promoting a follower to be the new leader when the old leader dies.
Controller node	A previously elected node that can appoint a new leader.
Split brain	Two nodes both believing they are leader.
Fencing / STONITH	Forcibly shutting down one node when two leaders are detected ("Shoot The Other Node In The Head").
Statement-based replication	Sending the SQL statements themselves to followers.
Nondeterministic function	A function like `NOW()` or `RAND()` that may give a different result each time.
Write-ahead log (WAL) shipping	Sending the storage engine's low-level append-only log to followers.
Logical (row-based) log	A replication log describing row-level changes, independent of storage format.
Change data capture	Streaming a database's changes to outside systems (Chapter 11).
Trigger	Custom code the database runs automatically when data changes.
Trigger-based replication	Using triggers to record changes that an external process then replicates.
Read-scaling architecture	Many followers added to share read traffic.

Term	Plain-English meaning
Replication lag	Delay between a write on the leader and its appearance on a follower.
Eventual consistency	If writes stop, replicas will *eventually* agree; no bound on how long.
Read-after-write (read-your-writes) consistency	A user always sees their own submitted updates.
Cross-device read-after-write	The same guarantee across a user's different devices.
Monotonic reads	A user never sees data go "back in time" across successive reads.
Consistent prefix reads	Writes that happened in some order are seen in that same order.
Multi-leader (master–master, active/active)	More than one node accepts writes; each leader is also a follower of the others.
Write conflict	Two leaders accept different changes to the same data concurrently.
Conflict avoidance	Routing all writes for a record through one leader so conflicts cannot happen.
Convergent conflict resolution	All replicas end up with the same final value.
Last write wins (LWW)	Keep the write with the highest timestamp, discard the rest.
CRDT	Conflict-free replicated datatype; a data structure that merges concurrent edits automatically.
Mergeable persistent data structure	Tracks history like Git and uses three-way merges.
Operational transformation	Conflict resolution algorithm used in Google Docs and Etherpad.
Replication topology	The paths along which writes travel between leaders (circular, star, all-to-all).
Leaderless (Dynamo-style) replication	Any replica accepts writes; client or coordinator sends to many replicas.
Coordinator node	A node that forwards a client's request to replicas, without imposing an order.
Read repair	A client that sees a stale replica during a read writes the newer value back to it.
Anti-entropy	A background process that finds and fixes differences between replicas.
Quorum (n, w, r)	n replicas; a write needs w acks, a read queries r nodes; w + r > n gives overlap.
Strict quorum	A quorum using only a value's designated n nodes.
Sloppy quorum	During failures, writes may land on nodes outside the designated n.
Hinted handoff	Returning those temporarily held writes to their proper "home" nodes.
Concurrent operations	Two operations where neither knows about the other.
Happens-before	A happens before B if B knows about, depends on, or builds on A.
Causally dependent	B depends on A (so A must be applied first).

Term	Plain-English meaning
Siblings	Riak's name for concurrent values that are kept side by side.
Tombstone	A marker left in place of a deleted item so the deletion survives merges.
Version vector	One version counter per replica, used to tell overwrites from concurrent writes.
Dotted version vector	A version-vector variant used in Riak 2.0.
Causal context	Riak's string encoding of a version vector.

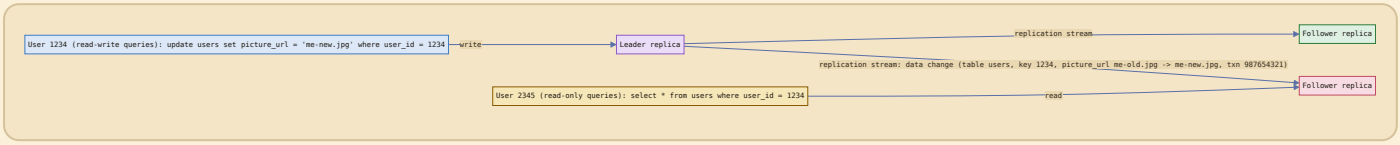
Leaders and Followers

Every write must reach every replica, otherwise the copies drift apart. The most common approach is **leader-based replication**:

1. One replica is chosen as **leader**. Clients must send all writes to it; it first writes the change to its own storage.
2. The other replicas are **followers**. Whenever the leader stores a change, it also sends it to every follower through a **replication log**. Each follower applies the changes **in the same order** the leader processed them.
3. Clients may read from the leader or any follower, but only the leader accepts writes; followers look read-only to clients.

> **"Hot" vs "warm" standby.** People define these differently. In PostgreSQL a hot standby serves client reads, while a warm standby applies the leader's changes but answers no queries. The book treats the difference as unimportant.

Figure 5-1 (redrawn): leader-based replication. User 1234 changes their profile picture by writing to the leader; the leader streams the row change to both followers; user 2345 reads user 1234's profile from a follower.



Who uses it:

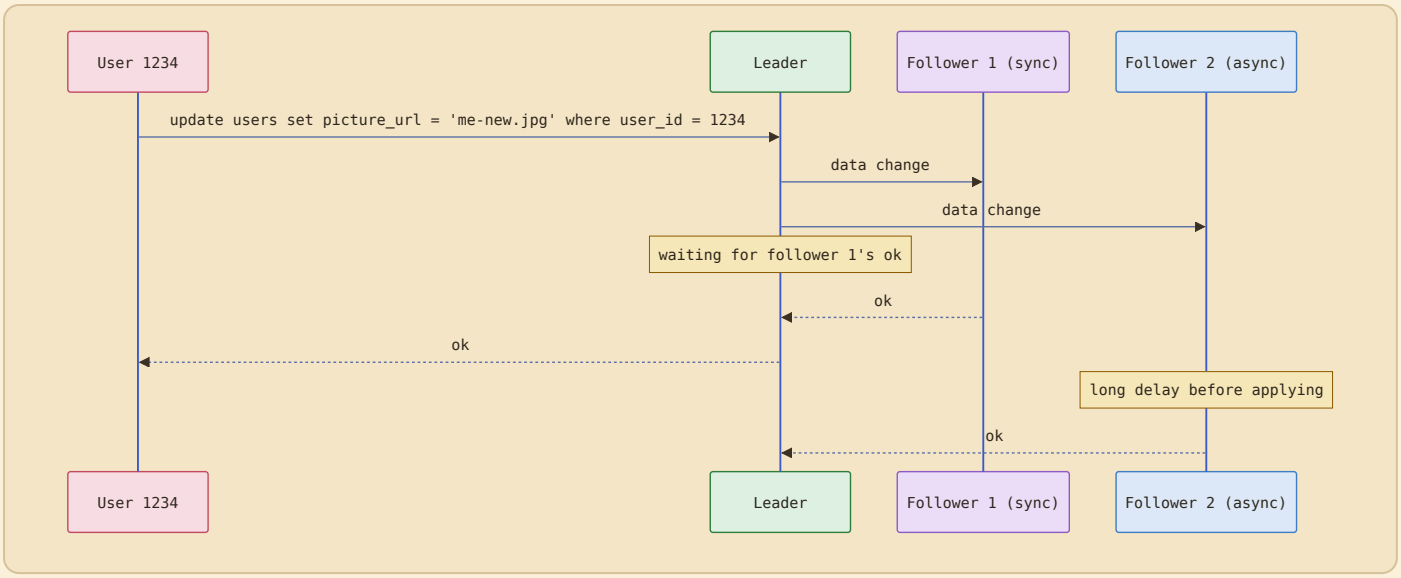
- Relational: PostgreSQL (since 9.0), MySQL, Oracle Data Guard, SQL Server AlwaysOn Availability Groups.
- Non-relational: MongoDB, RethinkDB, Espresso.
- Beyond databases: message brokers Kafka and RabbitMQ highly available queues. DRBD (a replicated block device) and some network filesystems follow a similar pattern.

Synchronous Versus Asynchronous Replication

What it is. When the leader forwards a change, does it wait for the follower's confirmation before telling the client "done"? If yes, that follower is **synchronous**; if no, it is **asynchronous**. Relational databases often let you configure this; other systems hard-code one choice.

Example: a user changes their profile picture. The client sends the update to the leader; the leader forwards it to followers; at some point the leader tells the client it succeeded.

Figure 5-2 (redrawn): follower 1 is synchronous (leader waits for its ok before answering the user), follower 2 is asynchronous (leader answers without waiting; follower 2 applies the change and acknowledges much later).



Normally followers apply changes within a second, but **there is no upper bound**. A follower can be minutes behind if it is recovering from a crash, the system is near maximum capacity, or the network is misbehaving.

	Synchronous follower	**Asynchronous follower**
Up-to-date copy guaranteed?	Yes: if the leader dies, the data is safe on this follower	No: unreplicated writes can be lost
Effect if the follower is down/slow	Leader must **block all writes** until it returns	Leader carries on regardless

Because one stuck synchronous node would halt everything, making *all* followers synchronous is impractical. The usual compromise is **semi-synchronous**: exactly one follower is synchronous and the rest asynchronous; if the synchronous one becomes slow or unavailable, one of the asynchronous followers is switched to synchronous mode. This guarantees the data is current on at least two nodes (leader plus one follower).

Very often, leader-based replication is **fully asynchronous**. Then a write the client was told succeeded can still vanish if the leader fails for good before sending it on, meaning durability is **not guaranteed**. The upside: the leader keeps accepting writes even if every follower lags. This weaker durability is still widely accepted, especially with many followers or followers spread across regions.

> **Research on replication.** Losing data when an async leader fails is serious, so researchers keep looking for methods that avoid loss yet stay fast and available. **Chain replication**, a synchronous variant, has been deployed successfully in a handful of systems, Microsoft Azure Storage being one. Replication consistency is closely tied to **consensus** (getting nodes to agree on a value), covered in Chapter 9.

Setting Up New Followers

You add followers to get more replicas or to replace dead ones. Just copying the data files does not work: writes keep arriving, so a plain file copy would capture different parts of the database at different moments, which could be nonsense. Locking the database during the copy would work but kills availability. Instead, the usual no-downtime procedure is:

1. **Snapshot** the leader's database consistently at one point in time, ideally without locking the whole database. Most databases can do this because backups need it; sometimes a third-party tool is needed (e.g. *innobackupex* for MySQL, part of Percona XtraBackup).
2. **Transfer** that snapshot onto the new follower's machine.
3. The new follower **asks the leader for every change made after the snapshot**. This only works if the snapshot is tied to an exact position in the replication log: PostgreSQL calls this the **log sequence number**, MySQL the **binlog coordinates**.
4. Once the follower has worked through that backlog it has **caught up**, and from then on it just follows the live stream of changes.

How automated this is depends heavily on the database: some do it all for you, others need an admin to run an obscure multi-step procedure.

Handling Node Outages

Nodes go down from faults and, just as often, planned maintenance (for example rebooting to apply a kernel security patch). Being able to restart individual nodes without downtime is a big operational win. The goal: keep the whole system running and minimise the impact of any single node being down.

Follower failure: Catch-up recovery

Each follower keeps on its local disk a log of changes received from the leader. After a crash/restart, or a temporary network break, the follower:

1. Looks in its log for the last transaction it processed.
2. Asks the leader for all changes after that point.
3. Applies them, is caught up, and resumes receiving the stream.

Leader failure: Failover

Harder. **Failover** means: promote a follower to leader, point clients' writes at it, and have the other followers consume from it. It can be manual (an admin is alerted and acts) or automatic. Automatic failover usually goes:

1. **Decide that the leader has failed.** Crashes, power cuts, network trouble... there is no foolproof detector, so systems use a **timeout**: nodes exchange messages constantly, and a node silent for, say, **30 seconds** is presumed dead. (Planned maintenance is different; you know the leader is going down.)
2. **Pick a new leader**, either by an **election** (a majority of remaining replicas choose) or by a previously elected **controller node** appointing one. The best choice is usually whichever replica holds the freshest copy of the old leader's changes, to lose as little as possible. Getting all nodes to agree is a **consensus** problem (Chapter 9).
3. **Reconfigure the system** so clients send writes to the new leader (see "Request Routing" in Chapter 6). If the old leader returns and still thinks it is leader, the system must force it to become a follower.

What can go wrong:

- **Lost writes.** With async replication, the new leader may be missing some of the old leader's last writes. If the old leader rejoins, what to do with them? The new leader may have accepted conflicting writes meanwhile. The usual answer is to **throw the old leader's unreplicated writes away**, which can break the durability promise made to clients.
- **Danger with outside systems.** Discarding writes is worse when other storage depends on the database. In an incident at **GitHub**, a stale MySQL follower became leader. Primary keys came from an autoincrementing counter, and the new leader's counter was behind, so it **reused primary keys** the old leader had already handed out. Those keys were also used in **Redis**, so MySQL and Redis disagreed and some private data was shown to the wrong users.

- **Split brain.** In some faults (Chapter 8) two nodes both believe they are leader. If both accept writes with no conflict-resolution process, data can be lost or corrupted. Some systems shut one down when two leaders are detected (**fencing**, or **STONITH**), but a poorly designed version can end up shutting down *both* (the book cites a GitHub outage postmortem for this).
- **Choosing the timeout.** Long timeout: slow recovery after a real failure. Short timeout: needless failovers, e.g. a load spike pushes response time past the limit, or a network glitch delays packets. A needless failover on a system already struggling with load or network trouble usually makes things **worse**.

There are no easy fixes, which is why some operations teams do failover **manually** even when the software could automate it. Node crashes, flaky networks, and the tension between replica consistency, durability, availability and latency are fundamental distributed-systems problems, revisited in Chapters 8 and 9.

Implementation of Replication Logs

What exactly does the leader send? Four methods are used in practice.

Statement-based replication

How it works. The leader records every write statement it executes (every `INSERT`, `UPDATE`, `DELETE`) and sends that log to followers, which parse and run each statement as though a client had sent it.

Here is an illustrative statement log (my own example):

```
-- statement 1: fine to replay anywhere
INSERT INTO orders (customer, item) VALUES ('ana', 'lamp');
-- statement 2: NOW() gives a different time on each replica when replayed
UPDATE orders SET shipped_at = NOW() WHERE customer = 'ana';
-- statement 3: result depends on current data and on execution order
DELETE FROM orders WHERE shipped_at < '2024-01-01';
```

- Line 1 is deterministic, so replaying it gives the same row everywhere (except that an autoincrement ID depends on ordering).
- Line 2 calls `NOW()`, a nondeterministic function, so each follower would store a different timestamp.
- Line 3 depends on existing data; if replicas run statements in different orders, they delete different rows.

Where it breaks:

- Nondeterministic functions such as `NOW()` or `RAND()` produce different values per replica.
- Statements using an autoincrement column, or that depend on existing data (`UPDATE ... WHERE <condition>`), must run in **exactly** the same order on every replica, which is restrictive when many transactions run concurrently.
- Side effects (triggers, stored procedures, user-defined functions) may differ per replica unless fully deterministic.

Workarounds exist, e.g. the leader replaces `NOW()` with a fixed value when logging, but there are so many edge cases that other methods are now preferred.

Who uses it: MySQL used it before version 5.1. Because it is compact it still appears today, but MySQL now switches to row-based replication by default whenever a statement is nondeterministic. **VoltDB** uses statement-based replication safely by requiring transactions to be deterministic.

Write-ahead log (WAL) shipping

Recall from Chapter 3 that storage engines append every write to a log:

- In a **log-structured** engine (SSTables/LSM-trees), the log *is* the main storage; background jobs compact old segments and garbage-collect them.
- In a **B-tree**, each modification goes to a **write-ahead log** before touching the tree, so a crash can be recovered from.

Either way, there is an append-only byte log of all writes. The leader can simply send that same log over the network; a follower replaying it builds identical data structures. **Used by PostgreSQL and Oracle**, among others.

Downside: the WAL is very low level (which bytes changed in which disk blocks), so replication is **tightly coupled to the storage engine**. If the on-disk format changes between versions, leader and followers usually cannot run different versions. That matters operationally: if followers *could* run a newer version, you could do a **zero-downtime upgrade** (upgrade followers first, then fail over to an upgraded one). With WAL shipping that version mismatch is often not allowed, so upgrades need downtime.

Logical (row-based) log replication

Use a **different log format for replication** than for storage, decoupling replication from engine internals. This is a **logical log** (as opposed to the engine's physical representation). For a relational database it is a sequence of row-level records:

- **Insert:** new values of all columns.
- **Delete:** enough to identify the row uniquely, normally the primary key; if the table has none, the old values of all columns.
- **Update:** enough to identify the row, plus the new column values (all of them, or at minimum those that changed).

A transaction touching several rows produces several records followed by a commit record. **MySQL's binlog** in row-based mode works like this.

A sketch of what such records might look like (illustrative JSON, my own data):

```
{"op": "insert", "table": "books", "row": {"id": 7, "title": "Dune", "stock": 3}}
{"op": "update", "table": "books", "key": {"id": 7}, "changed": {"stock": 2}}
{"op": "delete", "table": "books", "key": {"id": 4}}
{"op": "commit", "txid": 912}
```

- Line 1: an insert carries every column value.
- Line 2: an update names the row by primary key and gives the changed column.
- Line 3: a delete only needs the key.
- Line 4: marks the end of the transaction that produced the previous records.

Benefits:

- Easier to keep **backward compatible**, so leader and followers can run different software versions or even different storage engines.
- Easier for **external applications** to parse, e.g. to feed a data warehouse or build custom indexes and caches. This is **change data capture** (Chapter 11).

Trigger-based replication

The previous methods live entirely inside the database. Sometimes you need more flexibility: replicate only a subset of data, replicate between different kinds of database, or apply custom **conflict resolution**. Then replication moves up into the application layer.

Options:

- Tools like **Oracle GoldenGate** read the database log and expose changes to an application.

- Or use **triggers** and **stored procedures**. A trigger is custom code the database runs automatically on each write. It can log the change into a separate table; an external process reads that table, applies any application logic, and replicates the change elsewhere. **Databus for Oracle** and **Bucardo for Postgres** work this way.

A minimal illustration (PostgreSQL-style SQL, my own names):

```
CREATE TABLE change_queue (tbl text, row_id int, changed_at timestamptz);

CREATE FUNCTION log_change() RETURNS trigger AS $$
BEGIN
  INSERT INTO change_queue VALUES (TG_TABLE_NAME, NEW.id, now());
  RETURN NEW;
END $$ LANGUAGE plpgsql;

CREATE TRIGGER books_changes AFTER INSERT OR UPDATE ON books
  FOR EACH ROW EXECUTE FUNCTION log_change();
```

- The first statement creates a side table that will collect changes.
- The function inserts one record per changed row (table name, row id, time).
- The trigger runs that function after every insert/update on `books`; an outside process would poll `change_queue` and ship the rows elsewhere.

Trade-off: more overhead and more prone to bugs and limitations than built-in replication, but valued for its flexibility.

Method	What is sent	Pros	Cons	Examples
Statement-based	SQL statements	Compact	Nondeterminism, ordering, side effects	MySQL < 5.1, VoltDB
WAL shipping	Low-level disk-block changes	Exact copy, no extra log	Tied to storage format; version upgrades need downtime	PostgreSQL, Oracle
Logical (row-based)	Row-level changes	Version-independent; good for CDC	Book names no specific drawback; it is simply a second log format separate from storage	MySQL binlog (row mode)
Trigger-based	Whatever app code records	Very flexible	Overhead, bugs	Databus, Bucardo

Problems with Replication Lag

Replication is not only for surviving failures; it also helps **scalability** (more requests than one machine can take) and **latency** (replicas near users).

With a single leader, writes go through one node but reads can go anywhere. For read-heavy workloads (common on the web), you can create many followers and spread reads across them: a **read-scaling architecture**. Add followers to add read capacity. This really only works with **asynchronous** replication: synchronous replication to every follower would make any single node failure or network glitch stop all writes, and more nodes means more chances one is down.

But reading from an async follower can return **outdated data**. Run the same query on leader and follower at the same moment and you may get different answers. This is temporary: stop writing, wait, and followers catch up. Hence the name **eventual consistency**.

> **Origin of the term.** Douglas Terry et al. coined the phrase "eventual consistency"; it was popularised by Werner Vogels, and became a rallying cry for NoSQL projects. But async-replicated

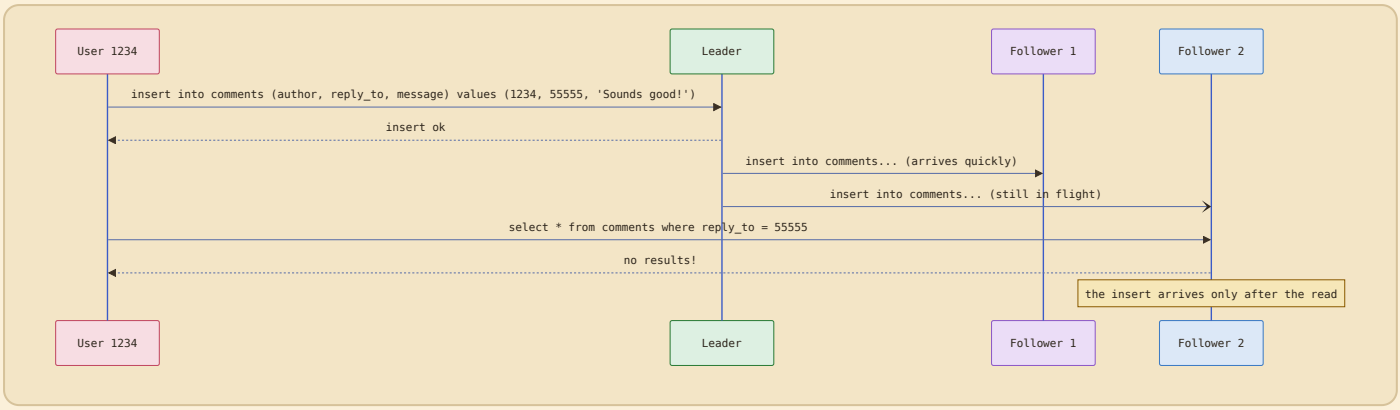
> *relational* databases behave exactly the same way.

"Eventually" is intentionally vague: there is no limit on how far a replica may fall behind. Normally the **replication lag** is a fraction of a second, but near capacity or with network trouble it can grow to seconds or minutes, and then the anomalies become real application bugs. Three such anomalies follow.

Reading Your Own Writes

Problem. A user submits something (a customer record, a comment) and then views it. The write goes to the leader, but the view may be served from a follower. If the follower has not received the write yet, the user thinks their data vanished.

Figure 5-3 (redrawn): user 1234 posts a comment via the leader, then immediately reads from follower 2, which has not received the insert yet, and gets no results.



Guarantee needed: read-after-write consistency (also called **read-your-writes**). If the user reloads, they always see updates *they* made. It says nothing about other users' updates.

Techniques with a leader-based system:

1. **Read possibly-modified data from the leader**, everything else from followers. You need a way to know what might have changed without querying. Example: on a social network only the owner edits their profile, so always read *your own* profile from the leader and other people's from followers.
2. If most things are user-editable, rule 1 sends almost everything to the leader and defeats read scaling. Use other criteria: e.g. for **one minute after the user's last update**, read from the leader; or monitor lag and avoid any follower more than one minute behind.
3. **Client remembers the timestamp of its latest write**; the system ensures the replica serving that user reflects at least that timestamp. If not, use another replica or wait until it catches up. The timestamp can be **logical** (e.g. the log sequence number) or the **system clock** (then clock synchronisation becomes critical; see Chapter 8).
4. With replicas in **several datacenters**, any request that must hit the leader has to be routed to the leader's datacenter.

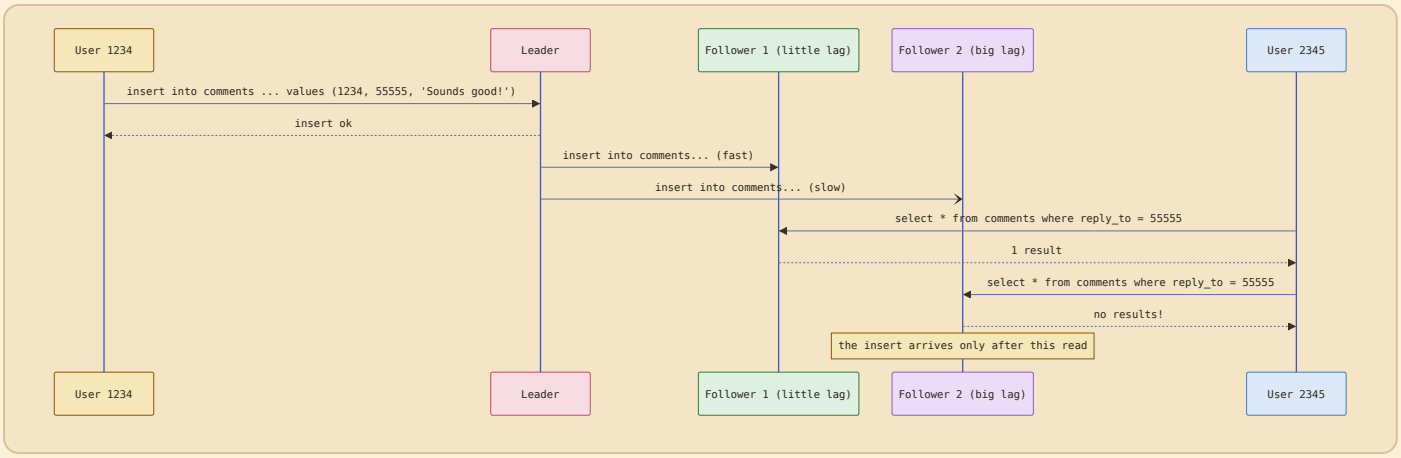
Cross-device read-after-write. A user enters data on their laptop and checks on their phone. Extra issues:

- Timestamp-remembering approaches get harder: one device does not know what the other wrote, so this metadata must be **centralised**.
- Different devices may reach **different datacenters** (home broadband vs mobile network take different routes). If your technique relies on leader reads, you may first have to send every device belonging to that user to one common datacenter.

Monotonic Reads

Problem: time appears to go backwards. A user queries twice; the first hits a fresh follower, the second a stale one (likely if each page refresh goes to a random server).

Figure 5-4 (redrawn): user 2345 first sees user 1234's new comment on a fresh follower, then refreshes, hits a stale follower and the comment is gone.



Had the first read also returned nothing, the user would not notice; seeing the comment appear and then disappear is what confuses.

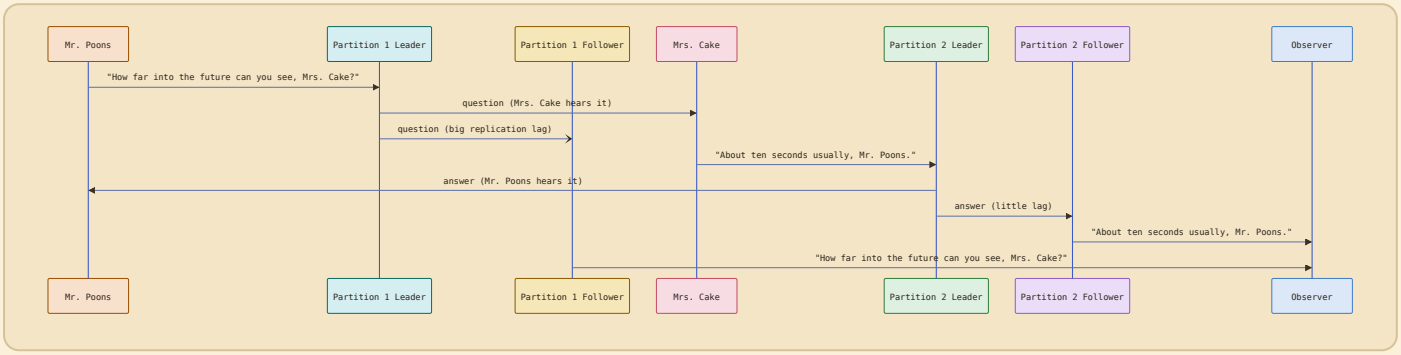
Guarantee: monotonic reads. Weaker than strong consistency, stronger than eventual consistency. You might still read old data, but successive reads by one user never go to an *older* state after seeing a newer one.

How: make each user always read from the **same replica** (different users can use different replicas), e.g. choose the replica by **hashing the user ID** rather than at random. If that replica fails, the user must be rerouted.

Consistent Prefix Reads

Problem: causality violated. The book uses a dialogue between Mr. Poons (asks how far ahead Mrs. Cake can see) and Mrs. Cake (answers about ten seconds). The answer depends on the question. If an observer reads the answer via a fast replica and the question via a slow one, they see the reply **before** the question, as though Mrs. Cake were psychic.

Figure 5-5 (redrawn): Mr. Poons's question goes to partition 1, whose follower lags a lot; Mrs. Cake's answer goes to partition 2, whose follower lags little. Each speaker hears the other via the leaders, but the observer, reading from the followers, receives the answer first.



Guarantee: consistent prefix reads. If writes happen in a certain order, anyone reading them sees them in that order.

This is mostly a problem in **partitioned (sharded)** databases (Chapter 6). If a database applied all writes in one global order, reads would always see a consistent prefix. But partitions usually run independently with **no global order**, so a reader can see some parts old and some new.

Fixes: put causally related writes in the **same partition** (not always efficient), or use algorithms that **track causal dependencies** explicitly (see "The happens-before relationship" below).

Solutions for Replication Lag

- Ask: *what if lag grew to minutes or hours?* If that is fine, great. If users would suffer, design for something stronger, for instance read-after-write. **Treating async replication as if it were synchronous is a recipe for future trouble.**
- Applications can offer stronger guarantees than the database (e.g. reading certain things from the leader), but handling it in the application is complicated and easy to get wrong.
- It would be better for the database to "do the right thing". That is the role of **transactions**: stronger guarantees from the database so applications stay simpler.
- Single-node transactions are old. Many distributed (replicated/partitioned) systems dropped them, saying they cost too much performance and availability and that eventual consistency is unavoidable at scale. The book calls this partly true but **too simplistic**; it develops a more nuanced view in Chapters 7 and 9 and Part III.

Anomaly	What the user sees	Guarantee	Typical fix
Own write missing	"My post disappeared"	Read-after-write	Read own data from leader; track last-write timestamp
Time goes backwards	Item appears, then vanishes	Monotonic reads	Pin each user to one replica (hash of user ID)
Effect before cause	Answer before question	Consistent prefix reads	Causally related writes in same partition; causal tracking

Multi-Leader Replication

Single-leader's big weakness: **all writes go through one leader**. If you cannot reach it (e.g. a network break between you and it), you cannot write.

> **Partitioned note.** In a partitioned database each partition has its own single leader; different partitions' leaders may live on different nodes, but each partition still has exactly one.

The natural extension: let **several nodes accept writes**. Each node that handles a write forwards it to all others. This is **multi-leader** (master–master, active/active) replication; each leader is simultaneously a follower of the other leaders.

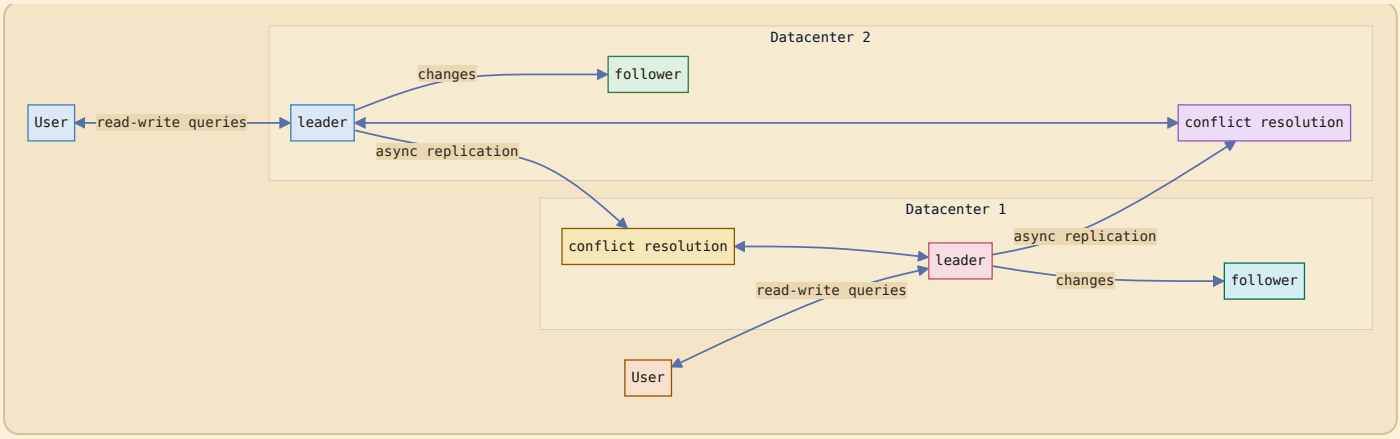
Use Cases for Multi-Leader Replication

Inside a single datacenter the extra complexity is rarely worth it. It does make sense in these cases:

Multi-datacenter operation

Replicas in several datacenters (to survive losing one, or to be near users). With a single leader, it sits in one datacenter and all writes must travel there. With multi-leader, **every datacenter gets its own leader**: normal leader–follower replication inside each datacenter, and leader-to-leader replication between them.

Figure 5-6 (redrawn): each datacenter has its own leader, which serves the local users' read-write queries and sends changes to a local follower; the two leaders replicate to each other, each passing incoming changes through a conflict-resolution step.



Aspect	Single-leader across datacenters	Multi-leader
Performance	Every write crosses the internet to the leader's datacenter, adding latency and undermining the point of multiple sites	Writes handled locally, replicated asynchronously; the inter-datacenter delay is hidden from users
Datacenter outage	Failover promotes a follower in another datacenter	Each datacenter keeps running on its own; replication catches up when the failed one returns
Network problems	Very sensitive to the inter-datacenter link (public internet, less reliable than a local network), since writes go over it synchronously	Async replication tolerates temporary interruptions; writes continue

Tools: some databases support it by default; often it is bolted on via external tools such as **Tungsten Replicator** (MySQL), **BDR** (PostgreSQL), **GoldenGate** (Oracle).

Big downside: the same data can be changed concurrently in two datacenters, producing **write conflicts** that need resolving. Also, because multi-leader is often retrofitted, it interacts badly with features such as **autoincrementing keys, triggers, and integrity constraints**. It is often regarded as dangerous territory to avoid if possible.

Clients with offline operation

Think of **stateful, offline-capable clients**: an app that must work without internet, e.g. a **calendar** on your phone, laptop and other devices. You must be able to read and add meetings anytime; changes made offline sync when you reconnect.

Here **each device has a local database acting as a leader**, and there is async multi-leader replication (sync) between devices. Lag can be **hours or days**. Architecturally it is the multi-datacenter case pushed to an extreme: treat each device as a "datacenter" joined to the others by a very unreliable link. The long history of broken calendar sync shows how hard this is. **CouchDB** was built with exactly this style of use in mind.

Collaborative editing

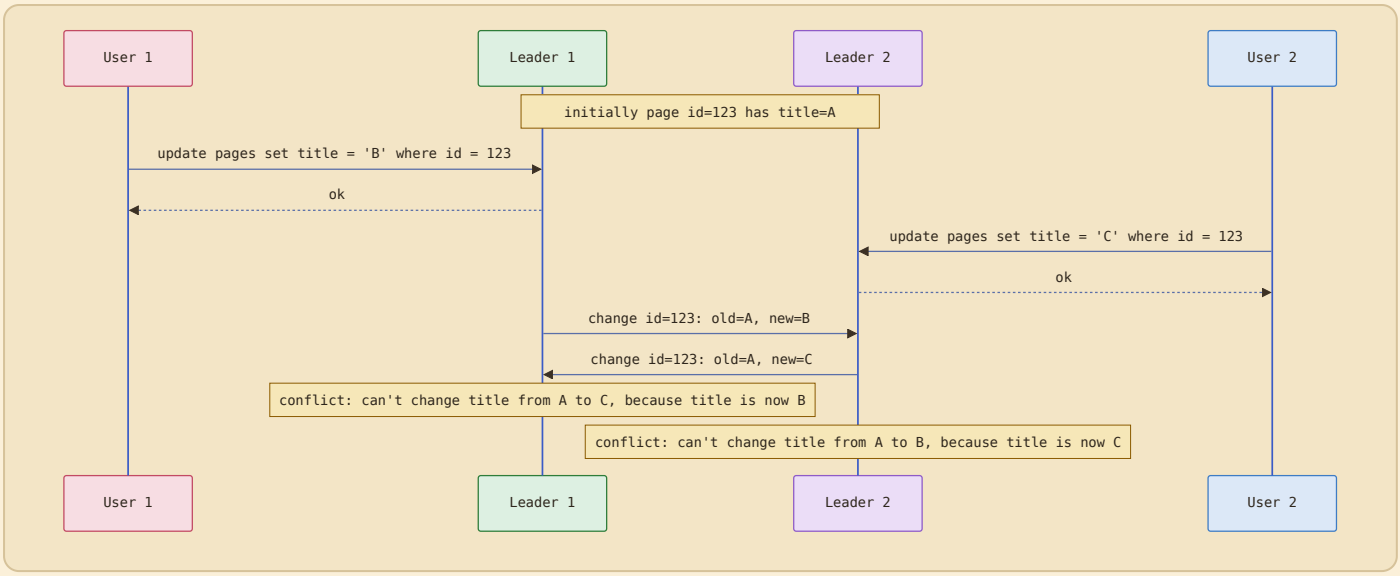
Real-time collaborative editors like **Etherpad** and **Google Docs** let several people edit at once. Each user's edits apply instantly to their local replica (the document state in their browser or app) and replicate asynchronously to the server and other editors, much like the offline case.

- To **guarantee no conflicts**, lock the document so only one user edits at a time; others wait until the lock is released. That model behaves like **single-leader replication where transactions run on the leader**.
- For faster collaboration you make the unit of change tiny (e.g. **one keystroke**) and avoid locking, which brings all the multi-leader challenges, including conflict resolution.

Handling Write Conflicts

Conflicts are the central headache of multi-leader replication.

Figure 5-7 (redrawn): page id=123 starts with title A. User 1 changes it to B on leader 1 while user 2 changes it to C on leader 2; both get "ok", and each leader only finds the conflict when the other's change arrives.



This cannot happen with a single leader.

Synchronous versus asynchronous conflict detection

- **Single leader:** the second writer either blocks until the first write completes, or its transaction is aborted and the user retries.
- **Multi-leader:** both writes succeed locally; the conflict is detected **later, asynchronously**, by which point asking the user may no longer be possible.
- You *could* detect conflicts synchronously by waiting for replication to all replicas before confirming, but that throws away multi-leader's main benefit (independent writes on each leader). If you want that, just use single-leader.

Conflict avoidance

Simplest strategy: make sure all writes for a given record go through **the same leader**, so conflicts cannot occur. Since many implementations handle conflicts poorly, this is frequently recommended.

Example: route each user to a **"home" datacenter** (perhaps the nearest) and use that datacenter's leader for their reads and writes. Different users have different homes, but for any one user it behaves like single-leader.

It breaks down when the designated leader must change, e.g. a datacenter fails and traffic moves, or the user relocates and is now nearer another datacenter. Then concurrent writes on different leaders become possible again.

Converging toward a consistent state

A single leader applies writes in sequence, so the last write to a field decides its value. With multiple leaders there is **no defined order**. In Figure 5-7, leader 1 sees B then C; leader 2 sees C then B. Neither order is "more correct". If each simply applied writes in arrival order, leader 1 would end with C and leader 2 with B: permanently inconsistent, which is unacceptable. Resolution must be **convergent**: all replicas reach the same final value once all changes have replicated.

Ways to converge:

1. **Give each write a unique ID** (timestamp, long random number, UUID, or hash of key and value); keep the write with the highest ID and drop the others. With timestamps this is **last write wins (LWW)**: popular but **dangerously prone to data loss**.
2. **Give each replica a unique ID**; writes from a higher-numbered replica always win. Also loses data.
3. **Merge the values**, e.g. sort alphabetically and concatenate (the title might become "B/C").
4. **Record the conflict** in a data structure that keeps all information, and let application code resolve it later (maybe by asking the user).

Custom conflict resolution logic

The right resolution is application-specific, so most multi-leader tools let you supply code, run either:

- **On write:** when the database finds a conflict in the incoming replicated changes, it calls your handler. **Bucardo** lets you write a Perl snippet for this. The handler runs in a background process, must be quick, and cannot prompt a user.
- **On read:** all conflicting versions are stored; the next read returns them all to the application, which may ask the user or resolve automatically and write back the result. **CouchDB** works this way.

Conflict resolution usually works **per row or document, not per transaction**. So if a transaction atomically makes several writes (Chapter 7), each write is resolved separately.

> **Automatic Conflict Resolution.** Custom handlers get complicated and buggy. A well-known example is **Amazon**'s shopping cart: for a while its resolution logic preserved *additions* to a cart but not *removals*, so deleted items sometimes came back. Research directions:
>
> - **CRDTs (conflict-free replicated datatypes)**: sets, maps, ordered lists, counters, etc. that multiple users can edit concurrently and that resolve conflicts sensibly on their own. Some are implemented in **Riak 2.0**.
> - **Mergeable persistent data structures**: track history explicitly like **Git** and use a **three-way merge** (CRDTs use two-way merges).
> - **Operational transformation**: the algorithm behind **Etherpad** and **Google Docs**, built specifically for concurrent edits to an ordered sequence, like the characters making up a document.
>
> Database implementations are still young, but these could make multi-leader sync much easier.

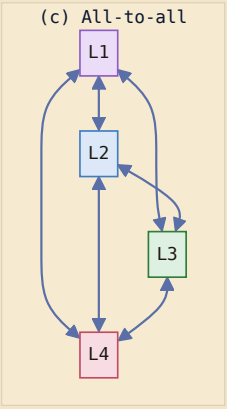
What is a conflict?

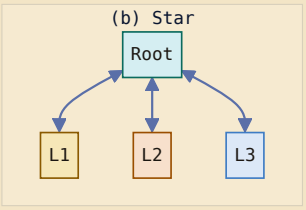
Some are obvious: two concurrent writes set the same field of the same record to different values (Figure 5-7). Others are subtle. Example: a **meeting-room booking system** must not allow two overlapping bookings of the same room. Even if the app checks availability first, two bookings made on **two different leaders** can still conflict. There is no quick answer; Chapter 7 shows more examples and Chapter 12 discusses scalable ways to detect and resolve conflicts.

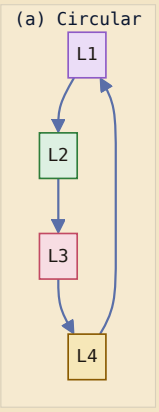
Multi-Leader Replication Topologies

A **replication topology** is the set of paths along which writes travel between nodes. With two leaders there is only one sensible choice (each sends to the other). With more, several are possible.

Figure 5-8 (redrawn): three example topologies with four leaders each: (a) circular, (b) star (one root in the middle), (c) all-to-all.







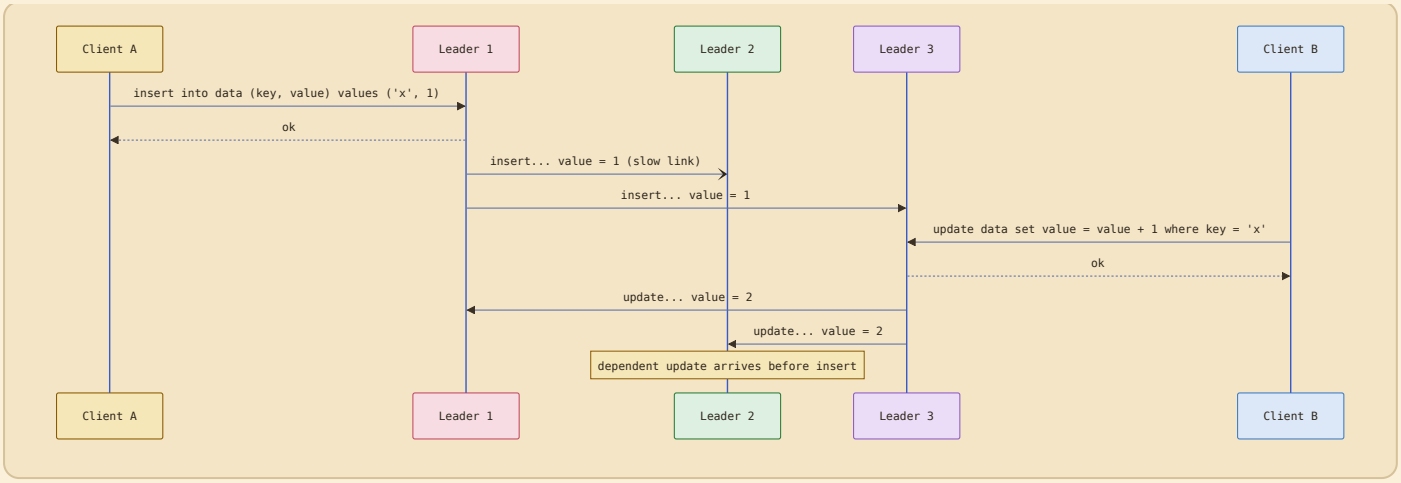
- **All-to-all:** every leader sends to every other; the most general.
- **Circular:** every node takes writes from one neighbour and passes them, together with its own writes, on to one other neighbour. **MySQL supports only this by default.**
- **Star:** one designated root forwards writes to all others; can be generalised to a **tree**. (Unrelated to the *star schema* of data warehousing, which is about data model structure, not network paths.)

Avoiding infinite loops. In circular and star layouts, writes pass through several nodes, so nodes forward what they receive. Each node has a unique ID, and each write in the log is tagged with the IDs of every node it has passed through. A node that receives a change tagged with its own ID ignores it, knowing it already processed it.

Fault tolerance. In circular and star layouts, **one failed node can block replication** between others until fixed; rerouting around it is usually manual. Denser topologies like all-to-all avoid this single point of failure because messages can take other paths.

All-to-all's own problem: messages overtaking each other. Some links are faster (e.g. due to congestion elsewhere), so replication messages can arrive out of order.

Figure 5-9 (redrawn): client A inserts row x on leader 1, client B increments it on leader 3, but the insert travels slowly to leader 2, so leader 2 receives the dependent update before the insert.



This is a **causality** issue like consistent prefix reads: the update depends on the insert, so everyone must apply the insert first. **Attaching timestamps is not enough**, because clocks are not reliably in sync (Chapter 8). The right tool is **version vectors** (later in this chapter). Yet many systems implement conflict detection poorly: at the time of writing, **PostgreSQL BDR** gives no causal ordering, and **Tungsten Replicator for MySQL** does not even try to detect conflicts.

Advice: if you use multi-leader replication, know these issues, read the docs carefully, and test thoroughly that you get the guarantees you think you do.

Leaderless Replication

Single- and multi-leader both have a leader decide the order of writes, with followers applying that order. **Leaderless** systems drop the leader: **any replica can accept writes directly from clients**.

History: some of the earliest replicated systems were leaderless, but the idea faded while relational databases dominated. It returned after **Amazon** built its in-house **Dynamo**. **Riak, Cassandra, and Voldemort** are open-source leaderless stores inspired by it, hence the name **Dynamo-style**.

> **DynamoDB is different.** Dynamo is internal to Amazon. AWS's hosted product **DynamoDB** uses a completely different design: single-leader replication.

In some implementations the client sends writes to several replicas itself; in others a **coordinator node** does so for it. Unlike a leader, a coordinator does **not impose an order** on writes, and this has big consequences.

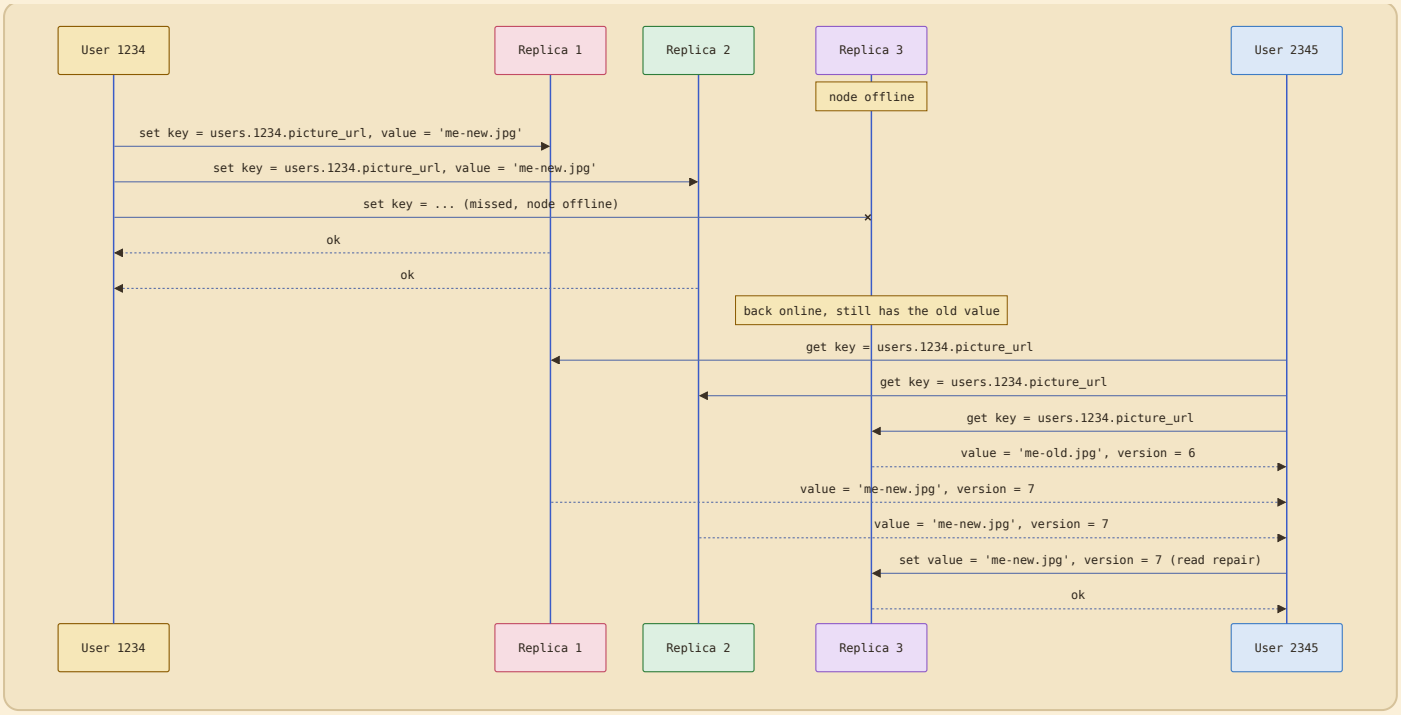
Writing to the Database When a Node Is Down

Three replicas, one down (maybe rebooting for an update). In a leader-based system you might need failover to keep writing. In a leaderless system **there is no failover**:

1. User 1234's client fires the write at **all three** replicas at once.
2. The two live replicas accept it; the down one misses it.
3. If two acknowledgements out of three are enough, the write counts as **successful**, and the client ignores the missed replica.

When the down node returns, it lacks the writes it missed, so reading from it gives **stale** values. So **reads also go to several nodes in parallel**. Replies can differ (fresh from one node, old from another); **version numbers** tell it which is newer.

Figure 5-10 (redrawn): user 1234 writes a new picture URL to all three replicas while replica 3 is offline; two oks are enough. Later user 2345 reads from all three, gets version 7 from replicas 1 and 2 but the old version 6 from replica 3, and repairs replica 3.



Read repair and anti-entropy

How does a returning node catch up? Dynamo-style stores use two mechanisms:

Mechanism	How it works	Notes
Read repair	During a parallel read, the client spots stale replies (e.g. version 6 from replica 3 vs version 7 from replicas 1 and 2) and writes the newer value back to the stale replica.	Works well for frequently read values.
Anti-entropy process	A background process keeps comparing replicas and copying missing data across.	Unlike a replication log, it copies in **no particular order**, and there can be a long delay.

Not all systems have both: **Voldemort currently has no anti-entropy process**. Without anti-entropy, rarely read values may be missing on some replicas, i.e. **less durable**, since read repair only fixes what gets read.

Quorums for reading and writing

If every successful write is on at least 2 of 3 replicas, at most one replica is stale, so reading from 2 guarantees at least one fresh answer, even if the third is down or slow.

General rule: with **n** replicas, a write must be confirmed by **w** nodes and a read must query at least **r** nodes. If **w + r > n**, the read set and write set overlap, so at least one node read is up to date. These are **quorum reads and writes**; think of w and r as the minimum votes needed. (Example above: n = 3, w = 2, r = 2.)

> **Strict vs sloppy.** This kind is sometimes called a **strict quorum**, as opposed to the sloppy quorums described below.

Tuning in Dynamo-style databases (all three are configurable):

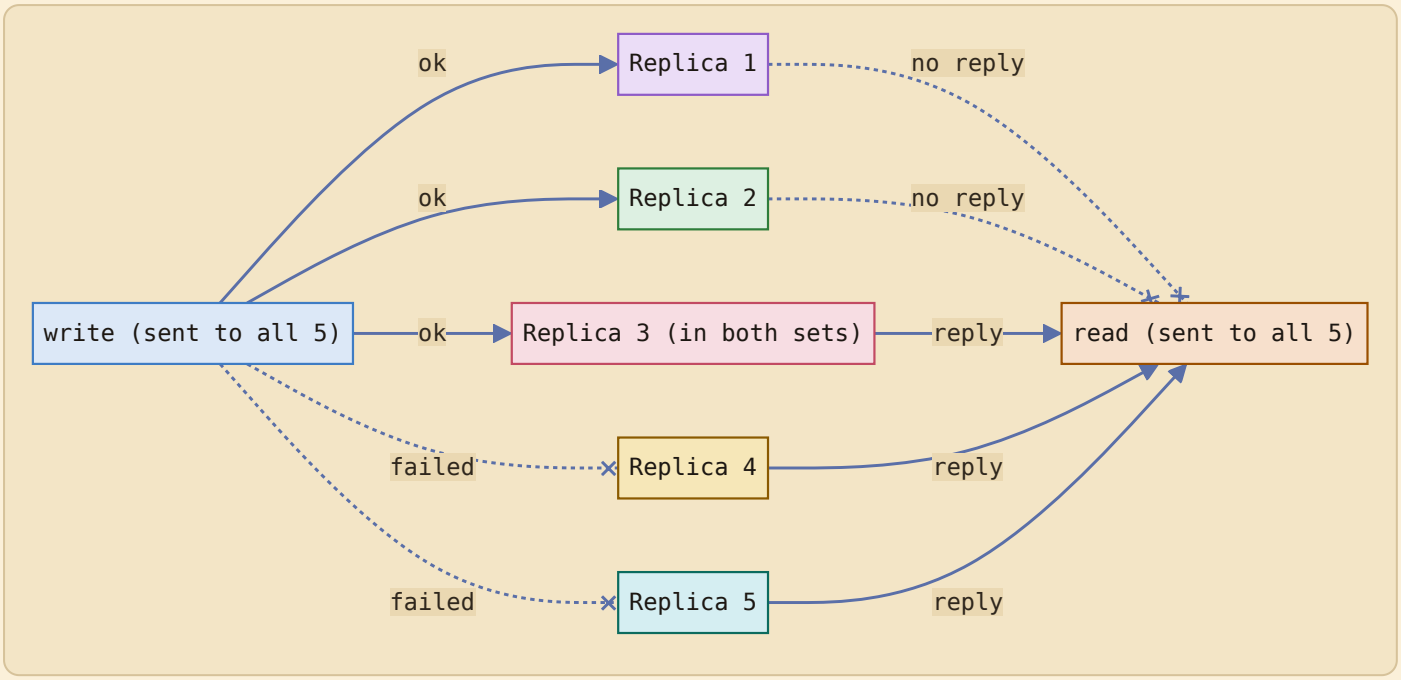
- A common choice: **n odd (typically 3 or 5)** and **w = r = (n + 1) / 2 rounded up**.
- Few writes and many reads? Set **w = n, r = 1**: faster reads, but a single failed node makes **all writes fail**.

> **n is per value, not per cluster.** A cluster can have more than n nodes; each value is just stored on n of them. That is how the dataset gets partitioned across a big cluster (Chapter 6).

How the quorum condition tolerates failures:

- **w < n**: writes still succeed with a node down.
- **r < n**: reads still succeed with a node down.
- **n = 3, w = 2, r = 2**: tolerates **one** unavailable node.
- **n = 5, w = 3, r = 3**: tolerates **two** unavailable nodes.
- Requests are normally sent to **all n** replicas in parallel; w and r only set how many replies to **wait for**.

Figure 5-11 (redrawn): with n = 5, w = 3, r = 3, the write is sent to all five replicas but only replicas 1-3 succeed; the read is sent to all five but only replicas 3-5 answer. Replica 3 is in both sets, so the read sees the latest write.



If fewer than w or r nodes respond successfully, the write or read **returns an error**. Why a node fails (crash, power loss, full disk, network break) does not matter; only whether it answered successfully.

Limitations of Quorum Consistency

Why w + r > n usually works: the written set and the read set must share at least one node, which has the latest value.

- w and r are often chosen as a **majority** (more than n/2), which satisfies w + r > n while tolerating up to n/2 failures. But quorums need not be majorities; they only need to **overlap**. Other assignments give flexibility in algorithm design.
- You can also set **w + r ≤ n** (quorum condition not met). Requests still go to n nodes but fewer successes are needed. Stale reads become **more likely**, but get **lower latency and higher availability**: when many replicas are unreachable you can still work until reachable replicas fall below w or r.

Even with w + r > n, stale reads can happen. Edge cases:

1. **Sloppy quorum** in use: the w writes and r reads may land on different nodes, so no guaranteed overlap.

2. **Concurrent writes**: unclear which came first. The only safe option is to merge; picking a winner by timestamp (LWW) can lose writes to **clock skew**.
3. **A write concurrent with a read**: the write may be on only some replicas, so the read may return old or new.
4. **Partial write failure**: succeeded on some replicas (others had full disks), fewer than w overall. The replicas that did store it do **not undo** it, so a write reported as failed may or may not show up in later reads.
5. **Restored from stale replica**: a node holding the new value fails and is restored from a replica with an old value, so the count holding the new value may drop below w.
6. **Unlucky timing** even when all is working (see "Linearizability and quorums" in Chapter 9).

So quorums *seem* to promise the latest value but in practice it is not that simple. Dynamo-style databases are generally built for workloads that **tolerate eventual consistency**. Treat w and r as knobs on the *probability* of stale reads, not absolute guarantees. In particular, the three lag guarantees from earlier (read-your-writes, monotonic reads, consistent prefix) are usually **absent**. Stronger guarantees need transactions or consensus (Chapters 7 and 9).

Monitoring staleness

Even if stale reads are acceptable, you must know whether replication is healthy and get alerted if it falls far behind (e.g. network trouble, overloaded node).

- **Leader-based:** easy. Writes are applied in the same order everywhere and every node's progress can be expressed as a log position (count of writes applied). **Lag = leader's position minus follower's position.** Databases usually expose this metric.
- **Leaderless:** no fixed write order, so it is harder. If read repair is the sole repair mechanism (anti-entropy absent), nothing **limits how old** a rarely read value can be.

Research exists on measuring staleness in leaderless systems and on forecasting what fraction of reads will be stale for given n, w and r, but it is not yet common practice. The book argues staleness should be a standard metric: eventual consistency is vague, but for operations you need to **quantify "eventual"**.

Sloppy Quorums and Hinted Handoff

Properly configured quorums tolerate individual node failures **without failover**, and tolerate slow nodes because they only wait for w or r replies. That makes leaderless databases attractive for **high availability, low latency, and tolerance of occasional stale reads**.

But quorums are less fault-tolerant than they could be. A network break can cut a client off from many nodes; those nodes are alive, but to that client they may as well be dead, and fewer than w or r may be reachable.

In a large cluster (many more than n nodes), the client can probably reach *some* nodes, just not the particular n "home" nodes for this value. Designers then choose:

- Fail every request that cannot gather w (or r) responses? **Or**
- Accept the writes anyway, storing them on reachable nodes that are **not** among the value's n home nodes?

The second is a **sloppy quorum**: reads and writes still need w and r successes, but these can include non-home nodes. The book's analogy: locked out of your house, you ask to sleep on a neighbour's couch.

When the network recovers, writes a node held on another's behalf are sent to the correct home nodes: **hinted handoff** (you found your keys, and the neighbour politely sends you home).

Trade-off: great for **write availability** (any w reachable nodes suffice), but w + r > n no longer **assures you of reading the newest value**, since it may sit on nodes outside the n. A sloppy quorum is

really just a **durability assurance** (the data is on w nodes *somewhere*), not a quorum in the traditional sense, until hinted handoff finishes.

Configuration: optional in all common Dynamo implementations. **Enabled by default in Riak; disabled by default in Cassandra and Voldemort.**

Multi-datacenter operation

Leaderless replication also suits multiple datacenters, since it is built to tolerate conflicting concurrent writes, network breaks, and latency spikes.

System	How it handles multiple datacenters
Cassandra, Voldemort	Stay within the normal leaderless model: n counts nodes in **all** datacenters, and you configure how many of the n live in each. Each write goes to all replicas, but the client usually waits only for a quorum **in its local datacenter**, so it is unaffected by cross-datacenter delays. Remote writes are often configured to be asynchronous (with some flexibility).
Riak	Keeps client-to-node traffic **inside one datacenter**; n is the number of replicas within that datacenter. Replication between datacenter clusters happens asynchronously in the background, similar to multi-leader.

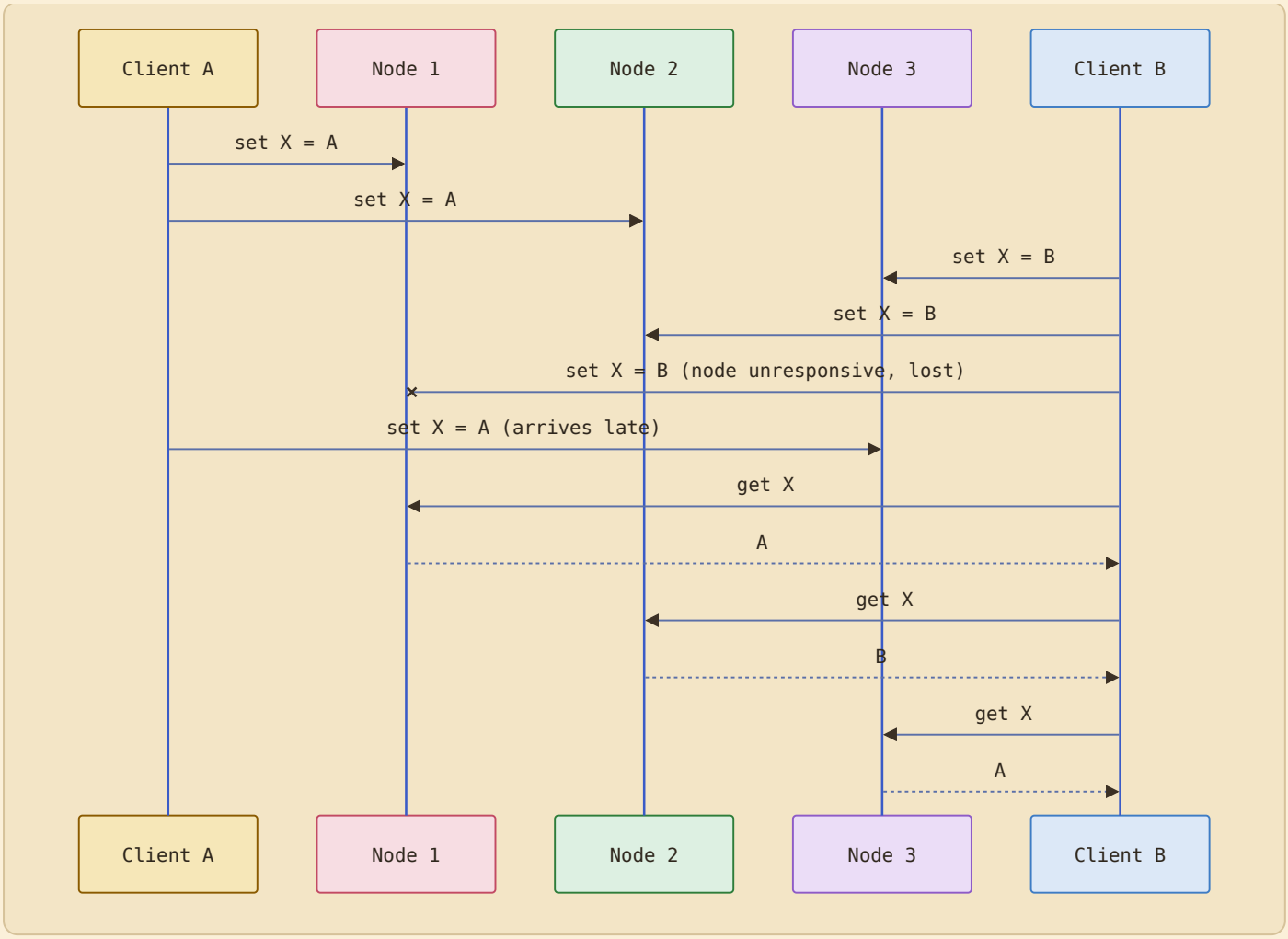
Detecting Concurrent Writes

Dynamo-style databases let several clients write the same key concurrently, so **conflicts occur even with strict quorums**. It resembles multi-leader conflicts, and in Dynamo-style stores **read repair** and **hinted handoff** can create conflicts as well.

Root cause: because of variable network delays and partial failures, events reach different nodes in different orders. Example with clients A and B writing key X on a three-node store:

- Node 1 gets A's write but never B's (transient outage).
- Node 2 gets A then B.
- Node 3 gets B then A.

Figure 5-12 (redrawn): clients A and B concurrently set key X; node 1 is unresponsive to B's write, and nodes 2 and 3 receive the two writes in opposite orders. If each node just overwrites on arrival, a later get X returns A, B and A from nodes 1, 2 and 3: they disagree forever.



To be eventually consistent, replicas must **converge**. Unfortunately most implementations handle this poorly, so **you, the developer, must understand your database's conflict handling** to avoid losing data.

Last write wins (discarding concurrent writes)

Idea: each replica keeps only the "most recent" value and lets "older" ones be overwritten. Provided we can always decide which of two writes is newer, and every write eventually reaches every replica, all copies converge.

Why the quotes around "recent": in Figure 5-12 the two clients were unaware of each other, so calling either one "first" is meaningless. The writes are **concurrent**, and their order is **undefined**. LWW **forces** an arbitrary order, e.g. by attaching a timestamp and keeping the largest.

- **Used by:** the only conflict-resolution method in **Cassandra**; optional in **Riak**.
- **Cost:** convergence at the expense of **durability**. Of several concurrent writes, all reported successful (each reached w replicas), only one survives; the rest are **silently discarded**. LWW can even drop **non-concurrent** writes (see "Timestamps for ordering events" in Chapter 8).
- Acceptable in some cases, like **caching**. Where data loss is unacceptable, avoid LWW.
- **Only safe usage:** write each key **once** and treat it as immutable, so there are no concurrent updates to the same key. Recommended Cassandra practice: use a **UUID as the key**, giving every write its own key.

The "happens-before" relationship and concurrency

How do we tell whether two operations are concurrent?

- In **Figure 5-9**, the writes are **not concurrent**: B's increment operates on the value A inserted, so B builds on A and must be later. B is **causally dependent** on A.

- In **Figure 5-12**, the writes **are concurrent**: neither client knew of the other, so there is no causal dependency.

Definition: operation A **happens before** B if B knows about A, depends on A, or builds on A. Two operations are **concurrent** when **neither** one happens before the other, i.e. each is unaware of the other.

So for any A and B there are exactly three cases: A before B, B before A, or concurrent. We need an algorithm to tell which. If one happened before the other, the later one should overwrite; if concurrent, there is a conflict to resolve.

> **Concurrency, Time, and Relativity.** "Concurrent" does not mean "at literally the same instant". Clocks in distributed systems make exact simultaneity hard to judge anyway (Chapter 8). What matters is only whether each operation was **unaware** of the other, regardless of physical time. Some compare this to special relativity: information cannot travel faster than light, so two distant events closer in time than light needs to travel between them cannot influence each other. In computers, two operations can be concurrent even when light would easily have had time, e.g. if a slow or broken network prevented one from learning of the other.

Capturing the happens-before relationship

Start with a **single replica**, then generalise. Two clients add items to the same shopping cart (the book notes you could picture two air-traffic controllers adding aircraft to a sector instead). The cart starts empty; five writes occur:

1. **Client 1 adds milk.** First write to the key; server stores it as **version 1** and echoes the value and version back.
2. **Client 2 adds eggs**, unaware of milk. Server assigns **version 2** and keeps `[milk]` and `[eggs]` as **two separate values**, returning both with version 2.
3. **Client 1 adds flour**, unaware of eggs, sending `[milk, flour]` with **version 1**. The server sees this supersedes `[milk]` (version 1) but is concurrent with `[eggs]`. It stores `[milk, flour]` as **version 3**, drops `[milk]`, keeps `[eggs]`, and returns both.
4. **Client 2 adds ham.** It last received `[milk]` and `[eggs]`, so it merges them, adds ham, and sends `[eggs, milk, ham]` with **version 2**. The server treats it as replacing `[eggs]`; `[milk, flour]` stays because it is concurrent, so the server now holds `[milk, flour]` (v3) and `[eggs, milk, ham]` (**v4**).
5. **Client 1 adds bacon.** It last received `[milk, flour]` and `[eggs]` at version 3, merges, adds bacon, sends `[milk, flour, eggs, bacon]` with **version 3**. This overwrites `[milk, flour]` (`[eggs]` was already gone) while `[eggs, milk, ham]` is concurrent with it and stays: two values remain.

Figure 5-13 (redrawn): two clients editing one cart; the server uses version numbers to decide what to overwrite and what to keep side by side.

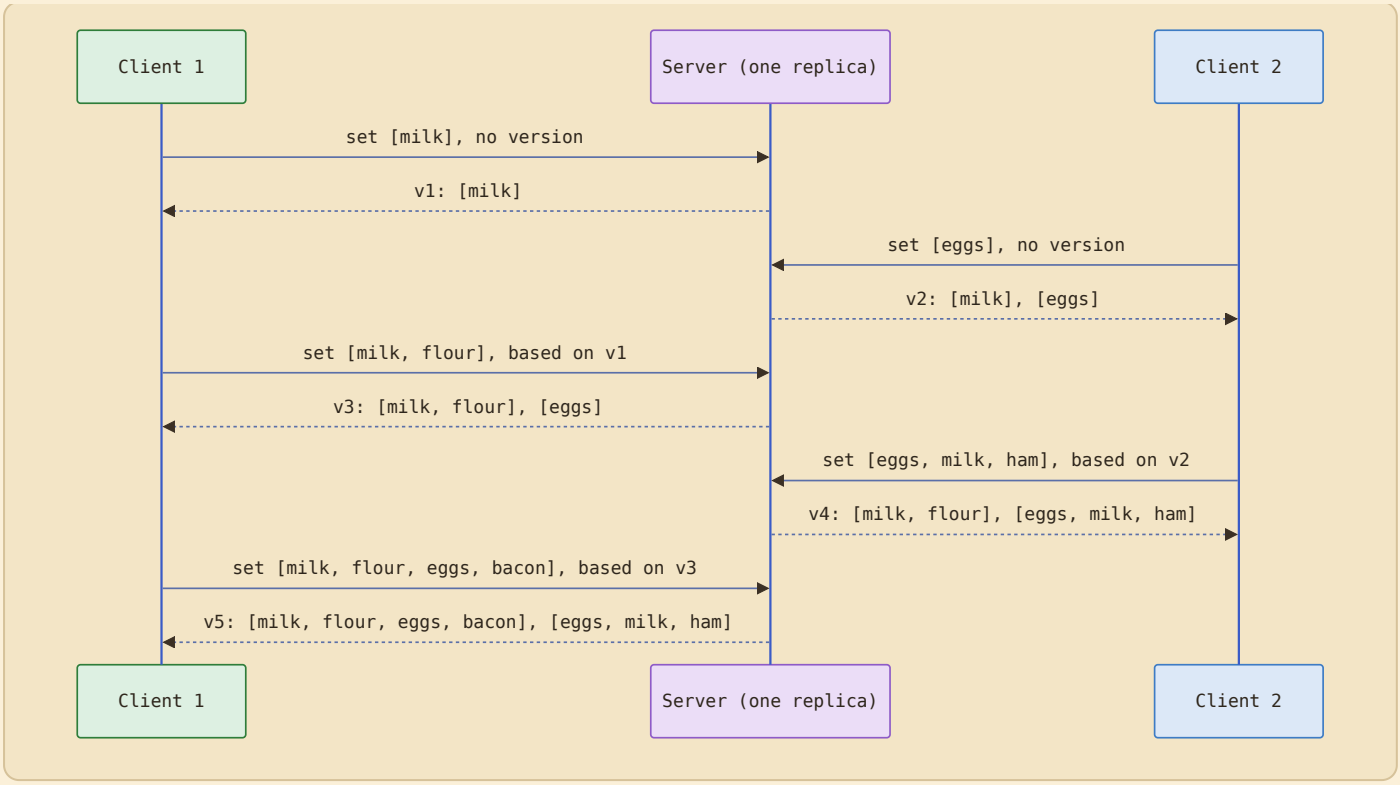
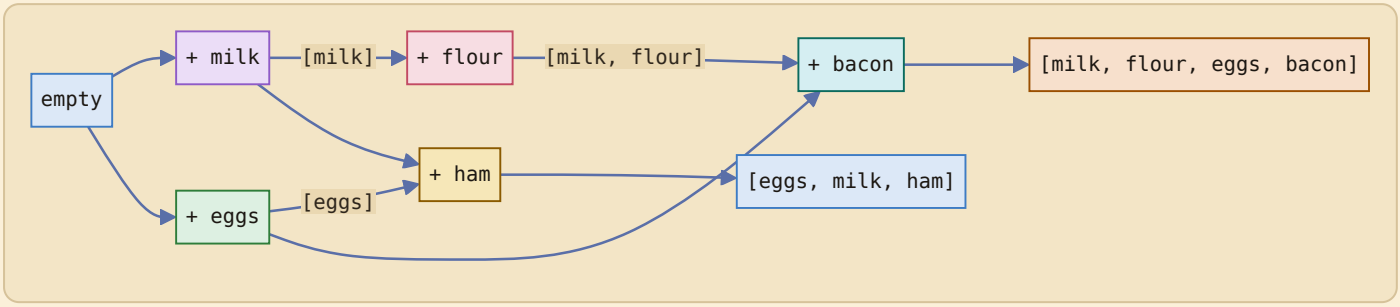


Figure 5-14 (redrawn): the causal-dependency graph of those writes. An arrow means the later operation knew about (built on) the earlier one; edge labels show the cart value passed along, and the two arrows leaving on the right are the final siblings.



The clients are never fully current (there is always a concurrent operation), yet old versions are eventually overwritten and **no write is lost**. The server does not need to understand the value; it only looks at version numbers, so the value can be any data structure.

The algorithm:

1. The server keeps a **version number per key**, increments it on every write, and stores it with the written value.
2. On a **read**, the server returns **all values not yet overwritten** along with the newest version number. Every write **must be preceded by a read**.
3. On a **write**, the client includes the **version from its prior read** and must **merge** all values it received in that read. (A write's response can look like a read, returning current values, which lets writes be chained as in the cart example.)
4. When the server gets a write with version v, it may **overwrite all values with version ≤ v** (they have been merged in), but must **keep values with higher versions** (they are concurrent).

A write with **no version number** is concurrent with everything: it overwrites nothing and simply appears as another value on later reads.

Merging concurrently written values

No data is silently lost, but clients must **clean up** by merging the concurrent values. **Riak calls these concurrent values siblings.**

Merging siblings is the same problem as multi-leader conflict resolution. Picking one by version or timestamp (LWW) loses data, so something smarter in application code may be needed.

- **Cart example:** take the **union**. The last two siblings, `[milk, flour, eggs, bacon]` plus `[eggs, milk, ham]`, merge into `[milk, flour, eggs, bacon, ham]` with duplicates removed (milk and eggs show up in both siblings despite each being added only once).
- **Removals break plain union:** if an item was removed in only one sibling, the union brings it back (the Amazon cart problem). Fix: on removal, leave a marker with a suitable version number saying the item was deleted. This deletion marker is a **tombstone** (seen before in log compaction for hash indexes, Chapter 3).

Merging by hand is complex and error-prone, so there are efforts at data structures that merge automatically. **Riak's datatype support** uses **CRDTs**, which merge siblings sensibly, **including preserving deletions**.

A tiny illustration of union-with-tombstones (Python, my own example):

```
a = {"pens": "added", "tape": "added"}
b = {"pens": "removed", "glue": "added"}   # "removed" acts as a tombstone
merged = {}
for k in a.keys() | b.keys():
    states = {a.get(k), b.get(k)} - {None}
    merged[k] = "removed" if "removed" in states else "added"
visible = sorted(k for k, s in merged.items() if s == "added")   # ['glue', 'tape']
```

- `a` and `b` are two sibling versions of a cart; `b` removed pens.
- The loop visits every item mentioned in either sibling.
- If any sibling holds a tombstone for an item, the merged state is "removed", so the deletion survives the merge.
- `visible` lists what the user should see: pens stays deleted. (This naive rule means a removal always beats a concurrent re-add; real CRDTs use version information to decide.)

Version vectors

With **multiple replicas and no leader**, one version number per key is not enough. Instead keep a **version number per replica, per key**. Each replica increments its own counter when it processes a write and also tracks the counters it has seen from every other replica. That tells it which values to overwrite and which to keep as siblings.

The set of counters from all replicas is a **version vector**. The most interesting variant is the **dotted version vector**, used in **Riak 2.0**, and it works much like the cart example.

- Like the single version number, the version vector is sent to clients on reads and must be **sent back on the next write**. Riak encodes it as a string called the **causal context**.
- It lets the database tell **overwrites from concurrent writes**.
- The application may still have to merge siblings.
- It makes it safe to **read from one replica and write back to another**: siblings may appear, but nothing is lost as long as they are merged correctly.

> **Version vectors vs vector clocks.** A version vector is sometimes called a vector clock, but they are not quite the same thing. The distinction is subtle; when what you are comparing is replica state, use version vectors.

Summary

Replication serves four purposes:

Purpose	Meaning
High availability	Keep running when a machine, several machines, or a whole datacenter fails
Disconnected operation	Keep working during a network interruption
Latency	Put data near users so interaction is faster
Scalability	Serve more reads than one machine could, by reading from replicas

A simple goal (same data on several machines) turns out to be very tricky: you must reason about concurrency and everything that can fail, at minimum unavailable nodes and network interruptions, not to mention nastier faults like silent corruption from software bugs.

Three approaches:

Approach	How writes flow	Pros	Cons
Single-leader	All writes to one leader, which streams changes to followers; reads anywhere (followers may be stale)	Easy to understand; no conflict resolution	Every write depends on reaching the one leader; failover is fraught
Multi-leader	Each write goes to one of several leaders; leaders stream changes to each other and to followers	More robust to faulty nodes, network problems, latency spikes	Conflicts; hard to reason about; weak consistency
Leaderless	Each write to several nodes; reads from several nodes in parallel to detect and fix stale data	Same robustness benefits; no failover	Very weak consistency guarantees; conflicts

Sync vs async strongly affects behaviour under faults. Async is fast when all is well, but think through what happens when lag grows and servers fail: promoting an async follower after a leader failure can **lose recently committed data**.

Consistency models for lag:

- **Read-after-write:** users always see what they themselves submitted.
- **Monotonic reads:** after seeing data at one point in time, users never later see an earlier state.
- **Consistent prefix reads:** data appears in causally sensible order (question before answer).

Concurrency: multi-leader and leaderless allow concurrent writes, hence conflicts. We saw an algorithm (version numbers / version vectors) to decide whether one operation happened before another or they were concurrent, and ways to resolve conflicts by merging.

Next chapter: the counterpart of replication, **partitioning** a large dataset.

Interview angle

1. Single-leader vs multi-leader vs leaderless? 30-second answer: Single-leader sends every write through one node that streams an ordered log to followers; simple, no conflicts, but writes depend on that one leader and failover is risky. Multi-leader lets several nodes (typically one per datacenter, or each offline device) accept writes and replicate asynchronously to each other; better write latency and outage tolerance, but you must resolve conflicts. Leaderless (Dynamo-style: Riak, Cassandra, Voldemort) sends writes and reads to several replicas and uses quorums; no failover needed, very available, but only weak consistency.

- *Follow-up: why not multi-leader in one datacenter?* The added complexity (conflicts, trouble with autoincrement keys, triggers, constraints) rarely pays off when all nodes share a fast, reliable local network.
- *Follow-up: is DynamoDB leaderless?* No. Despite the name, the book notes AWS DynamoDB uses single-leader replication; Dynamo was Amazon's internal system.

2. Synchronous vs asynchronous replication? 30-second answer: Sync means the leader waits for a follower's ack before confirming, so that follower is guaranteed current, but if it stalls all writes block. Async means the leader does not wait: fast and available, but confirmed writes can be lost if the leader dies. In practice you use semi-synchronous (one sync follower, the rest async) or fully async.

- *Follow-up: why not make all followers sync?* Any one node outage would halt all writes; with more nodes, the chance something is down rises.

3. What can go wrong in failover? 30-second answer: With async replication the new leader may lack the old leader's last writes, which are usually discarded (durability violation). Discarded writes can clash with outside systems (GitHub reused MySQL autoincrement keys that Redis also used, leaking private data). Split brain can leave two leaders accepting writes. Choosing the timeout is a trade-off between slow recovery and needless failovers that worsen overload.

- *Follow-up: how do systems deal with split brain?* Fencing / STONITH shuts one node down; if badly designed it can shut down both.

4. Explain quorums (n, w, r). 30-second answer: A value lives on n replicas; a write needs w acks and a read asks r replicas. If w + r > n, the sets overlap so at least one replica read has the latest write. Typical: n = 3 or 5, w = r = majority. n = 3, w = r = 2 tolerates one failure; n = 5, w = r = 3 tolerates two. Read repair and anti-entropy bring stale replicas up to date.

- *Follow-up: does w + r > n guarantee fresh reads?* No: sloppy quorums, concurrent writes resolved by LWW (clock skew), writes concurrent with reads, partially failed writes not rolled back, restores from stale replicas, and timing edge cases can all produce stale reads. Treat it as a probability knob.
- *Follow-up: what is a sloppy quorum?* When home nodes are unreachable, writes go to other reachable nodes and are later sent home by hinted handoff. Improves write availability but breaks the overlap guarantee. On by default in Riak, off in Cassandra and Voldemort.

5. Replication-lag anomalies and fixes. 30-second answer: Read-your-writes (user does not see own update): read own editable data from the leader, or track last-write timestamp/LSN. Monotonic reads (data goes backwards on refresh): pin each user to one replica via hash of user ID. Consistent prefix (answer before question, mostly in partitioned stores): keep causally related writes in one partition or track causality.

- *Follow-up: what extra issues come with multiple devices?* Last-write metadata must be centralised, and devices may hit different datacenters, so you may need to steer every device of that user to one datacenter.

6. How do you detect concurrent writes, and why not just LWW? 30-second answer: Two writes are concurrent if neither knew about the other. Servers track a version per key (per replica per key in a version vector); clients send back the version they read, the server overwrites values at or below it and keeps higher ones as siblings for the app to merge. LWW converges but silently drops all but one concurrent write, so it is only safe when keys are written once (e.g. UUID keys in Cassandra).

- *Follow-up: how do you handle deletions when merging siblings?* Leave tombstones; or use CRDTs (as in Riak) that merge automatically and preserve deletions.

Self-check

1. **Name the four ways a leader's changes can be shipped to followers and one weakness of each.** Statement-based (nondeterministic functions, ordering, side effects); WAL shipping (coupled to storage format, upgrades may need downtime); logical/row-based (the book lists no specific drawback; its point is that it is decoupled from the storage engine, so it is the version-tolerant built-in option); trigger-based (overhead and bug-prone).
2. **Why is copying the leader's data files not enough to set up a new follower, and what is done instead?** Writes keep happening, so a file copy captures inconsistent moments. Instead take a consistent snapshot tied to a log position (LSN / binlog coordinates), copy it, then request and apply all changes since that position until caught up.
3. **Scenario: users complain that after posting a comment and reloading, it sometimes disappears. Which guarantee is missing and give two fixes.** Read-after-write consistency. Fixes: read the user's own content from the leader (or for one minute after their last write), or have the client remember its last write timestamp/LSN and only read from replicas that have reached it.
4. **With n = 5, what w and r would you choose to tolerate two failed nodes while keeping the quorum condition?** w = 3, r = 3 (3 + 3 > 5; each operation can proceed with two nodes down).
5. **Trade-off: what do you gain and lose by setting w = n and r = 1?** Very fast reads (one reply suffices), but any single node failure makes all writes fail.
6. **Why can timestamps not correctly order the insert and update arriving at leader 2 in Figure 5-9? What does work?** Clocks across nodes are not reliably synchronised, so timestamps may misorder causally related events. Version vectors can capture the causal dependency.
7. **What is the difference between read repair and anti-entropy, and which system lacks anti-entropy?** Read repair fixes stale replicas when a client notices them during a read (good for hot data). Anti-entropy is a background process comparing replicas and copying missing data in no particular order. Voldemort has no anti-entropy, so rarely read values can be less durable.
8. **In the version-number algorithm, what does the server do when it receives a write carrying version 3 while it holds values at versions 3 and 4?** It overwrites values with version ≤ 3 (they were merged into the new write) and keeps the version 4 value as a concurrent sibling.
9. **Why does conflict avoidance stop working sometimes?** It depends on each record having a fixed home leader; if a datacenter fails or a user moves closer to another one, the leader changes and concurrent writes on different leaders become possible.
10. **Scenario: an offline-first notes app syncs across phone and laptop. Which replication style is it, and what must you plan for?** Multi-leader (each device is a leader). Plan for very long lag (hours or days) and conflict resolution, e.g. CRDTs or on-read resolution as in CouchDB.

Chapter 6 — Partitioning

Book: Chapter 6, pp. 199–220

Overview

Chapter 5 kept whole copies of the data on several nodes. That stops being enough when the dataset is **too big for one machine** or the **query rate is too high** for one machine. The answer is to break the data into **partitions** (also called **sharding**) and spread them over many nodes.

This chapter answers: *how do you decide which record lives on which node, how do you keep secondary indexes working, how do you move data around when nodes join or leave, and how does a request find the right node?*

> **Terminology zoo.** The same idea has many names: a **shard** in MongoDB, Elasticsearch and SolrCloud; HBase says **region**, Bigtable says **tablet**, Cassandra and Riak say **vnode**, and Couchbase says **vBucket**. The book sticks with **partition**, the most established term.

> **Not a network partition.** Partitioning here means deliberately splitting a large database into smaller ones. It is unrelated to *network partitions* (netsplits), which are a kind of network fault covered in Chapter 8.

Key ideas:

- Usually each record (row, document) belongs to **exactly one** partition. Each partition acts like a small database of its own, though some operations may span several partitions.
- The main goal is **scalability**. In a shared-nothing cluster (see the Part II introduction), partitions sit on different nodes, so the data is spread over many disks and the queries over many processors.
- A query that touches one partition can be answered by that partition's node alone, so adding nodes raises throughput. Large, complex queries can in principle be parallelised across nodes, but that is much harder.
- History: **Teradata** and **Tandem NonStop SQL** led the way with partitioned databases in the 1980s; the idea was later rediscovered by NoSQL stores and by data warehouses built on Hadoop. Some systems target transactions and others analytics; that changes tuning, but the partitioning fundamentals are the same.

Everyday analogy. A library too big for one room is split across several rooms. You need a rule for which book goes in which room (partitioning scheme), a way to search by topic across rooms (secondary indexes), a plan for moving shelves when a new room opens (rebalancing), and a directory at the entrance telling visitors which room to go to (request routing).

Key terms

Term	Plain-English meaning
Partition (shard, region, tablet, vnode, vBucket)	A subset of the data; each record belongs to exactly one.
Sharding	Another name for partitioning.
Skew	Unfair partitioning where some partitions get more data or queries than others.
Hot spot	A partition with disproportionately high load.
Key-range partitioning	Each partition owns a continuous range of keys, like encyclopedia volumes.
Partition boundaries	The key (or hash) values where one partition ends and the next begins.
Range scan / range query	Fetching all keys between a lower and upper bound.
Concatenated index	Treating a multi-part key as one sortable index, so related records can be fetched in one query.
Hash partitioning	Each partition owns a range of **hash values**; a key goes where its hash falls.
Consistent hashing	Karger et al.'s method of choosing partition boundaries randomly; the book advises calling database schemes "hash partitioning" instead.
Compound primary key	Cassandra key of several columns: first part hashed to choose the partition, the rest used for sorting within it.
Secondary index	An index for finding records by a non-key value (e.g. all red cars).
Document-partitioned (local) index	Each partition indexes only its own documents.
Term-partitioned (global) index	One index covering all partitions, itself partitioned by the indexed term.
Term	In full-text search, a word in a document; here, a value like `color:red`.
Scatter/gather	Sending a query to all partitions and combining the results.
Tail latency amplification	When a request waits on many sub-requests, the slowest one dictates the total time.
Rebalancing	Moving load (data and requests) from one node to another.
Hash mod N	Assigning a key to node `hash(key) % N`; a bad rebalancing approach.
Fixed number of partitions	Create many more partitions than nodes up front; move whole partitions.
Dynamic partitioning	Split partitions that grow too big, merge ones that shrink.
Pre-splitting	Configuring an initial set of partitions on an empty database.
Partitioning proportionally to nodes	A fixed number of partitions per node (Cassandra, Ketama).
Cascading failure	Overload spreading from node to node, e.g. triggered by automatic rebalancing.
Request routing	Getting each request to the node that owns the relevant partition.
Service discovery	The general problem of finding which network address serves something.

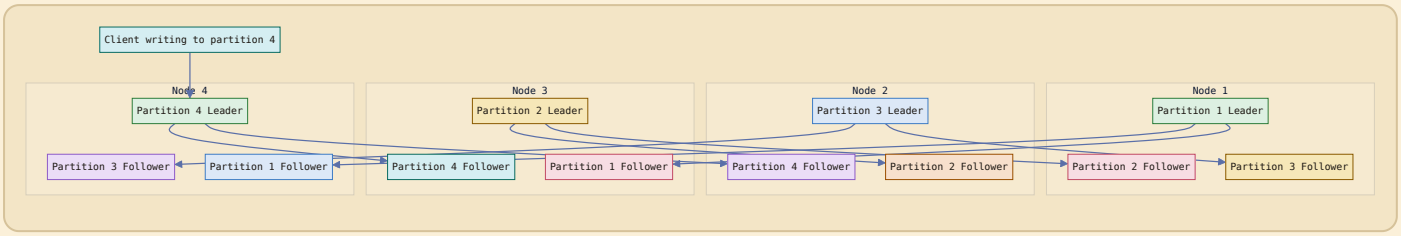
Term	Plain-English meaning
Routing tier	A partition-aware load balancer that forwards requests but handles none itself.
ZooKeeper	A coordination service that stores cluster metadata such as the partition-to-node map.
Gossip protocol	Nodes spreading cluster-state changes to each other directly.
MPP (massively parallel processing)	Analytic relational databases that split complex queries into stages run in parallel across nodes.

Partitioning and Replication

Partitioning is normally **combined with replication**: every record is in exactly one partition, but that partition is copied onto several nodes for fault tolerance. A node can hold many partitions.

With leader–follower replication, each partition has its **leader on one node and followers on others**. Any given node can therefore be leader for some partitions and follower for others.

Figure 6-1 (redrawn): four nodes, four partitions, each partition with one leader and two followers. Every node leads one partition and follows two others; a client writing to partition 4 goes to its leader on node 4. Arrows are per-partition replication streams.



Everything from Chapter 5 applies equally to replicating partitions. Since the choice of partitioning scheme is largely independent of the replication scheme, the rest of the chapter ignores replication for simplicity.

Partitioning of Key-Value Data

Goal: spread data and query load **evenly**. With every node carrying a fair share, 10 nodes could in theory handle 10 times the data and 10 times the read/write throughput of one node (ignoring replication).

If partitioning is unfair, it is **skewed**, and much less effective. In the extreme, all load hits one partition: **9 of 10 nodes sit idle** and the one busy node is your bottleneck. A partition with disproportionately high load is a **hot spot**.

Why not assign records to nodes at random? Data would be spread evenly, but to read a particular item you would not know where it is and would have to **query every node in parallel**.

Better: assume a simple **key-value** model where records are always accessed by primary key. The book compares this to an old paper **encyclopedia**, where entries are sorted by title so you can find one quickly.

Partitioning by Key Range

What it is. Give each partition a **continuous range of keys**, from a minimum to a maximum, like the volumes of a printed encyclopedia. If you know the boundaries you know which partition has a key; if

you also know which node holds that partition, you go straight there (or pull the right volume off the shelf).

Figure 6-2 (redrawn): a 12-volume print encyclopedia partitioned by key range; each volume covers an uneven span of titles.



Ranges need not be equal. Data is rarely spread evenly across the alphabet: in Figure 6-2 volume 1 holds A and B, but volume 12 holds T, U, V, X, Y and Z. Giving every two letters their own volume would make some volumes huge. Boundaries must **adapt to the data**, chosen either by an administrator or automatically by the database (see Rebalancing below).

Used by: Bigtable and HBase (Bigtable's open-source counterpart), RethinkDB, and MongoDB before version 2.4.

Benefit: sorted keys and range scans. Within each partition, keys can be kept sorted (as in SSTables/LSM-trees, Chapter 3). Range scans become easy, and the key can act as a **concatenated index** to fetch related records in one query. Example: a **sensor network** whose key is the measurement timestamp (year-month-day-hour-minute-second); fetching, say, all readings for one month is a simple range scan.

Downside: hot spots from access patterns. With a timestamp key, partitions correspond to time ranges, e.g. one per day. Since sensors write data as it is measured, **every write goes to today's partition**, which is overloaded while the others idle.

Fix: put something other than the timestamp first in the key. For instance, **put the sensor name in front of the timestamp**, so data is partitioned by sensor first and time second. With many sensors active at once, writes spread across partitions. The cost: fetching several sensors over a time range now needs **one range query per sensor**.

An illustrative key layout (my own example):

```
key = <device_id> + ":" + <timestamp>
therm-07:2024-03-01T10:00:00   -> partition holding therm-00 .. therm-09
therm-42:2024-03-01T10:00:00   -> partition holding therm-40 .. therm-49
```

- The first line gives the key format: device first, then time.
- Two writes at the same moment from different devices land in different partitions, spreading write load.
- A report on `therm-07` for March is still one range scan, but a report over all devices needs one scan per device.

Partitioning by Hash of Key

Since key ranges invite skew and hot spots, lots of distributed datastores pick the partition with a **hash function** applied to the key.

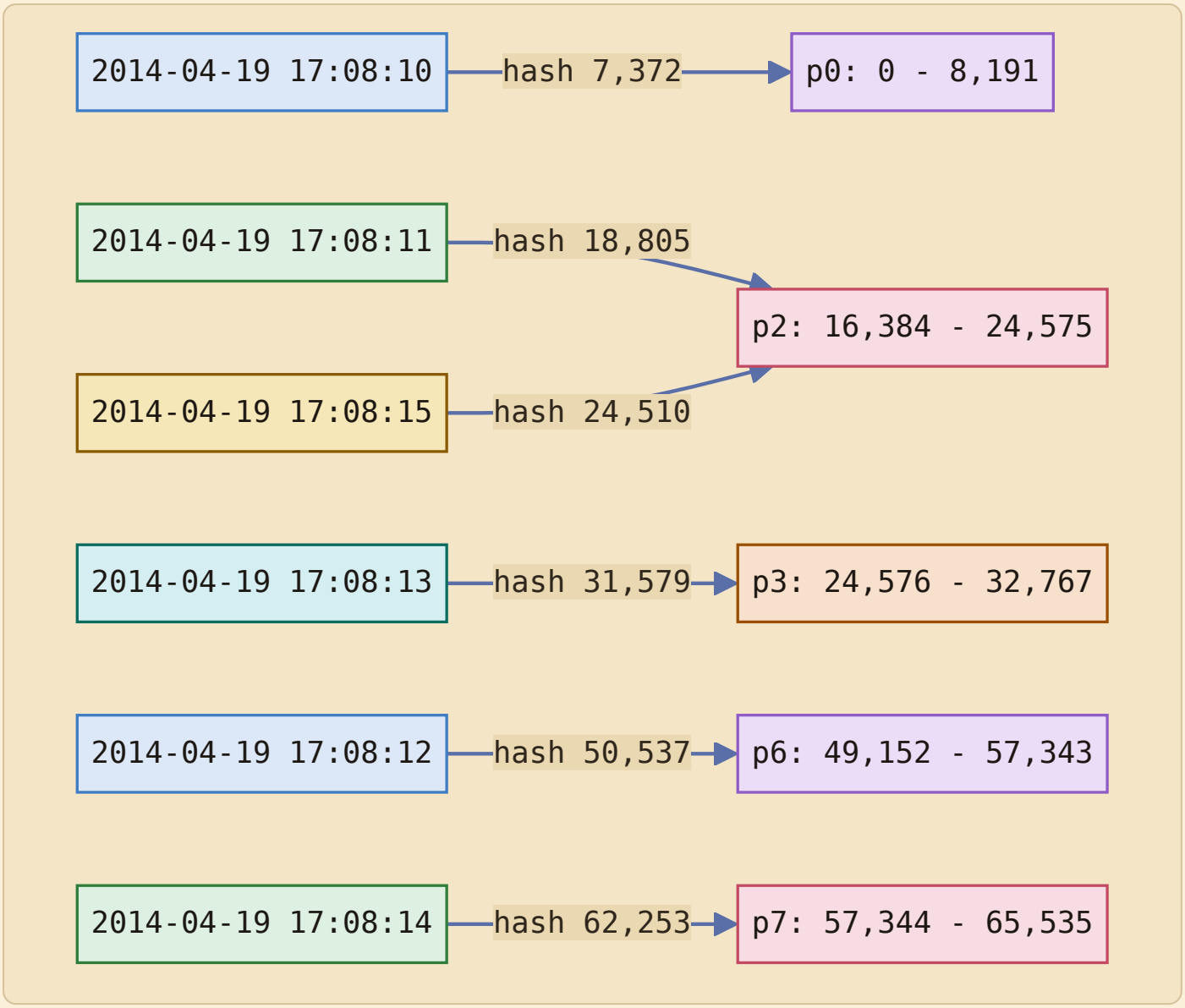
How a good hash helps. It turns skewed input into a uniform output. A 32-bit hash function given any string produces what looks like a random number in the range 0 to 2^32 − 1; even near-identical strings produce hashes spread evenly over that range.

Choosing the hash function. It need not be cryptographically strong: **Cassandra and MongoDB use MD5**, **Voldemort uses the Fowler–Noll–Vo** function. Built-in language hash functions (used for hash tables) may be **unsuitable**: with Java's `Object.hashCode()` or Ruby's `Object#hash`, one process may compute a different hash for a key than another process does.

How it works:

1. Pick a suitable hash function.
2. Give each partition a **range of hashes** instead of a range of keys.
3. Store each key in the partition whose range contains `hash(key)`.

Figure 6-3 (redrawn): six consecutive timestamp keys, one second apart, hash (first 2 bytes of MD5, so 0..65,535) to scattered values; the hash space is cut into eight equal ranges p0..p7, and each key lands in the partition whose range contains its hash.



This spreads keys fairly. Boundaries can be **evenly spaced** or **chosen pseudorandomly**; the latter is sometimes called **consistent hashing**.

> **Consistent Hashing.** Karger et al. defined consistent hashing as a technique for balancing load across caches spread over the internet, such as a **CDN** (content delivery network), using randomly chosen partition boundaries so no central control or distributed consensus is needed. The word "consistent" is unrelated to replica consistency (Chapter 5) or ACID consistency (Chapter 7); it describes a particular approach to rebalancing. As the rebalancing section shows, it works poorly for databases and is rarely used in practice (some databases' docs still say "consistent

> hashing", often inaccurately). To avoid confusion, the book recommends just saying **hash partitioning**.

The cost: no efficient range queries. Keys that were neighbours are now scattered over all partitions, so sort order is gone.

- In **MongoDB** with hash-based sharding, any range query goes to **all** partitions.
- **Riak, Couchbase and Voldemort** do not support range queries on the primary key.

Cassandra's compromise: compound primary keys. A table can have a primary key of several columns. **Only the first column is hashed** to choose the partition; the **remaining columns form a concatenated index** that sorts data within Cassandra's SSTables. So you cannot range-scan over the first column, but if you fix the first column's value you can efficiently range-scan the others.

This suits **one-to-many** data. Example from the book: on a social media site one user posts many updates. With primary key `(user_id, update_timestamp)`, you can fetch all of one user's updates in a time window, sorted by time. Different users live on different partitions, but each user's updates sit together, in time order, on one partition.

A Cassandra-style illustration (CQL, my own table):

```
CREATE TABLE ride_events (
  driver_id  uuid,
  event_time timestamp,
  status     text,
  PRIMARY KEY ((driver_id), event_time)
);

SELECT status FROM ride_events
WHERE driver_id = 5b1c0f2e-7a3d-4c1e-9f00-1a2b3c4d5e6f
  AND event_time >= '2024-05-01' AND event_time < '2024-05-02';
```

- `PRIMARY KEY ((driver_id), event_time)`: `driver_id` is the part that is hashed (decides the partition); `event_time` sorts rows inside that partition.
- The `SELECT` fixes `driver_id`, so only one partition is contacted, and the time condition is an efficient range scan within it.
- A query giving only a time range and no `driver_id` could not use this layout efficiently.

Skewed Workloads and Relieving Hot Spots

Hashing reduces hot spots but **cannot eliminate them**: if every read and write targets the **same key**, all requests still go to one partition.

This is unusual but real. Example: a **celebrity** on a social media site with millions of followers does something and triggers a storm of activity, i.e. a flood of writes to one key (maybe the celebrity's user ID, or the ID of the action people are commenting on). Hashing does not help, because identical IDs hash identically.

Today most data systems **cannot compensate for this automatically**, so the application must.

Technique: split the hot key.

1. For a key known to be very hot, **add a random number to the start or end** of it.
2. Just a **two-digit decimal** random number spreads writes evenly across **100 different keys**, which can live on different partitions.

Costs:

- **Reads** must now fetch from all 100 keys and combine the results.

- **Bookkeeping**: only do this for the few hot keys (for the vast majority of low-traffic keys it is pointless overhead), so you must track which keys are split.

The book expects that future systems may detect and handle skew automatically, but for now it is up to you to weigh the trade-offs for your particular application.

A sketch of the split (Python, my own example):

```python
import random
HOT = {"post:9001"}                       # keys known to be hot

def write_key(key):
    if key in HOT:
        return f"{key}#{random.randint(0, 99):02d}"   # one of 100 sub-keys
    return key

def read_keys(key):
    return [f"{key}#{i:02d}" for i in range(100)] if key in HOT else [key]
```

- `HOT` is the bookkeeping: the set of keys that have been split.
- `write_key` appends a random two-digit suffix to hot keys, so writes spread over 100 keys.
- `read_keys` shows the read penalty: a hot key must be read as 100 keys and combined; ordinary keys are untouched.

Partitioning and Secondary Indexes

So far records were found only by **primary key**, which directly identifies the partition. **Secondary indexes** complicate this. A secondary index usually does not identify a record uniquely; it finds all records with a given value: all actions by user 123, all articles containing the word *hogwash*, all cars whose color is red.

- Relational databases rely on them heavily (the book calls them the **bread and butter** there), and document databases use them a lot too.
- Many key-value stores (e.g. **HBase, Voldemort**) avoid them because of implementation complexity; others (e.g. **Riak**) have begun to add them since they help so much with data modelling.
- They are the **whole reason for existing** of search servers like **Solr and Elasticsearch**.

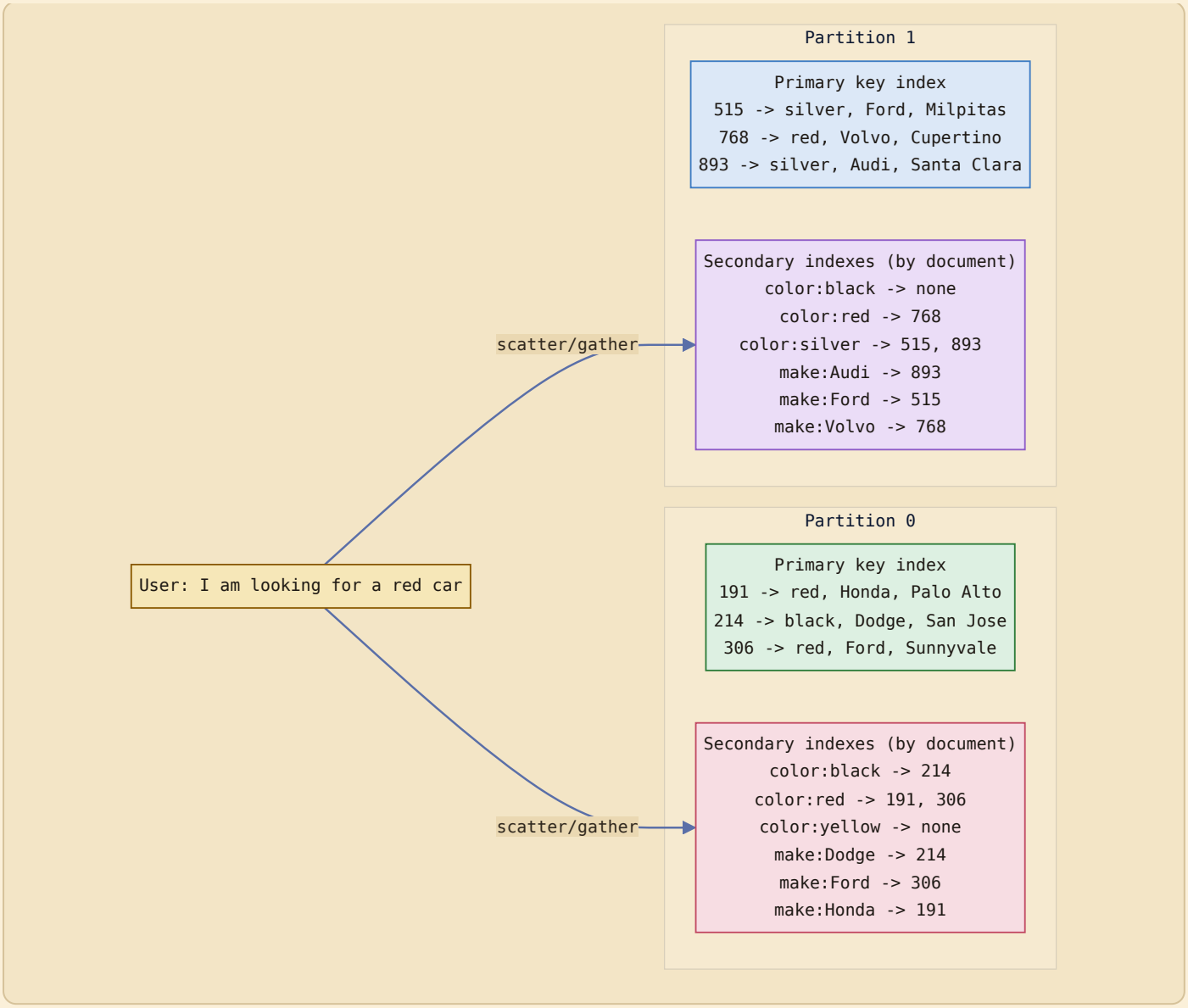
The problem: secondary indexes **do not map neatly onto partitions**. Two main approaches: **document-based** and **term-based** partitioning.

Partitioning Secondary Indexes by Document

Example from the book: a used-car sales website. Each listing has a unique **document ID**, and the database is partitioned by it (IDs 0–499 in partition 0, 500–999 in partition 1, etc.). Users filter by **color** and **make**, so you declare secondary indexes on those fields (columns in a relational database). The database then indexes automatically: when a red car is added, its partition adds its ID to the index entry `color:red`.

> **Rolling your own index.** In a pure key-value store you might build a secondary index yourself in application code by mapping values to document IDs. Be very careful: race conditions and partial write failures (some changes saved, others not) can easily leave the index out of sync with the data (see multi-object transactions, Chapter 7).

Figure 6-4 (redrawn): each partition keeps a primary-key index and its own secondary indexes covering only its own cars, so a search for red cars must scatter to both partitions and gather the results.



How it works. Each partition is completely independent and maintains secondary indexes **only for its own documents**, ignoring the rest. A write (add, remove, update) touches **only the partition holding that document ID**. Hence the name **local index** (vs. global index, next section).

Reading needs care. Unless you did something special with document IDs, cars of a given color or make are spread across partitions (red cars appear in partitions 0 and 1). So a query for red cars must go to **all partitions** and the results must be combined. This is **scatter/gather**.

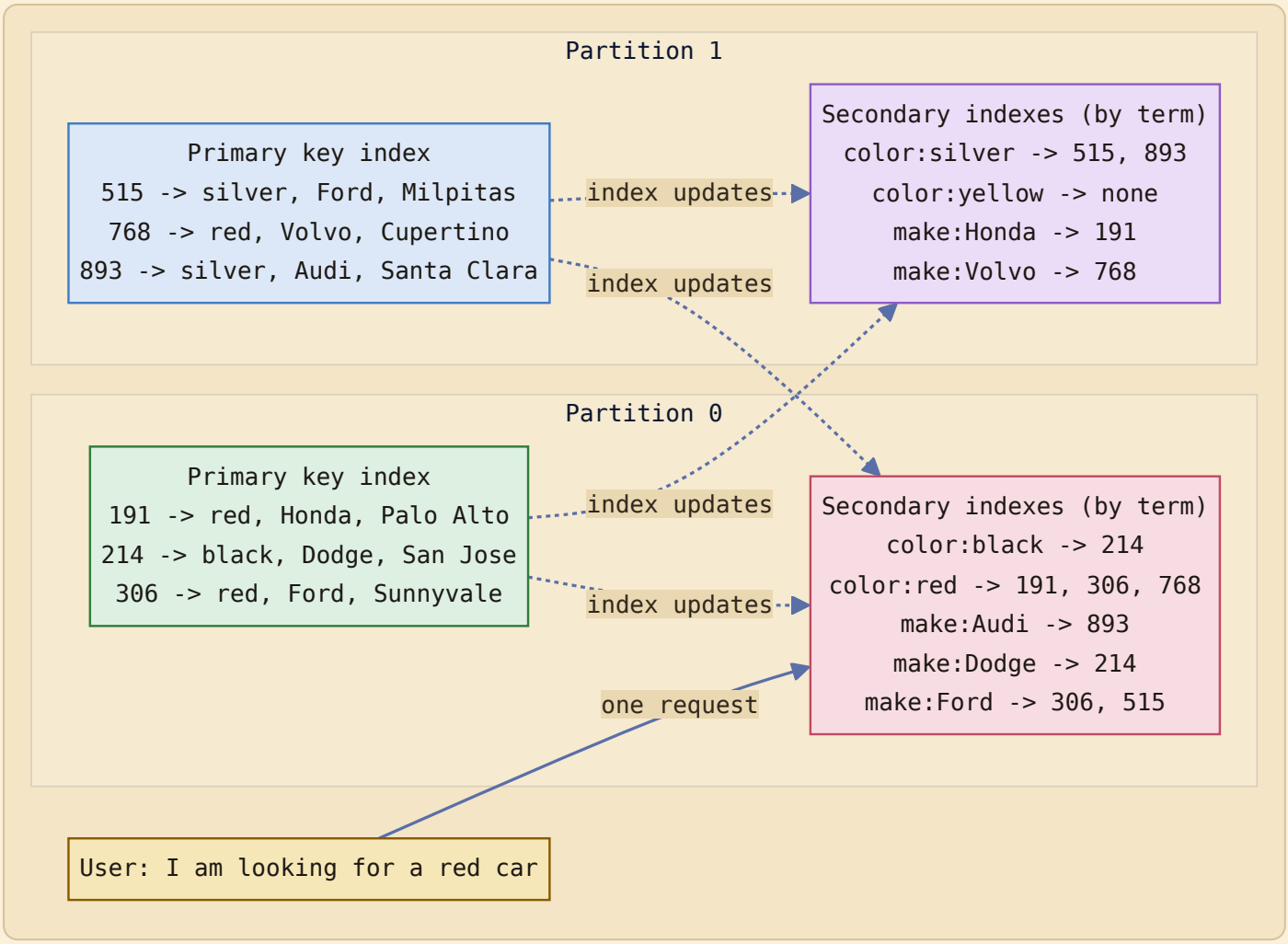
- It can make secondary-index reads **expensive**. Even in parallel, it suffers from **tail latency amplification** (the slowest partition decides the response time).
- Still **widely used**: MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, and VoltDB all use document-partitioned secondary indexes.
- Vendors usually advise designing partitioning so secondary-index queries hit **one partition**, but that is often impossible, especially with **several secondary indexes in one query** (e.g. filtering by color **and** make).

Partitioning Secondary Indexes by Term

Idea. Build a **global index** covering all partitions. It cannot live on a single node (it would become a bottleneck and defeat partitioning), so the global index is **itself partitioned**, but differently from the primary key.

Figure 6-5 (redrawn): the same cars, but the secondary indexes are global and partitioned by term: color:black, color:red and makes Audi to Ford live on partition 0; color:silver, color:yellow and

makes Honda to Volvo on partition 1. Writing one car may update index entries on both partitions (dashed), while a search for red cars reads one index partition.



In the book's figure, colors starting **a to r** are in index partition 0 and **s to z** in partition 1; the make index is split between **f and h**. This is a **term-partitioned** index: the term being searched for (e.g. `color:red`) decides which index partition holds it. The word *term* comes from **full-text indexes**, where terms are the words in documents.

By term or by hash of term?

- **By the term itself**: helps with **range scans**, for instance over a numeric property like a car's asking price.
- **By hash of the term**: spreads load **more evenly**.

	Document-partitioned (local)	**Term-partitioned (global)**
Where the index lives	With each data partition, covering only its documents	Separately, partitioned by indexed value
Writes	Touch one partition: fast and simple	One document may touch **several index partitions**, maybe on several nodes: slower, more complex
Reads	**Scatter/gather** to all partitions	Only the partition(s) holding the term
Freshness	Updated with the document	Often **asynchronous**; may lag
Examples	MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB	DynamoDB global secondary indexes, Riak search, Oracle data warehouse (choice of local or global)

Keeping a global index up to date. Ideally every write would appear in the index immediately. For a term-partitioned index that would need a **distributed transaction** across all affected index

partitions, which not every database supports (Chapters 7 and 9). In practice, global secondary index updates are often **asynchronous**: reading the index right after a write may not show it yet. **Amazon DynamoDB** says its global secondary indexes normally update within **a fraction of a second**, but propagation can take longer during infrastructure faults.

Other users of global term-partitioned indexes: **Riak's search feature**, and the **Oracle data warehouse**, where you can pick between local and global indexing. Chapter 12 returns to implementing term-partitioned secondary indexes.

Rebalancing Partitions

Things change over time:

- Query throughput rises, so you want **more CPUs**.
- The dataset grows, so you want **more disks and RAM**.
- A machine **fails** and others must take over its work.

Each requires moving data and requests between nodes. Moving load from one node to another is **rebalancing**. Whatever the partitioning scheme, rebalancing is expected to meet some minimum requirements:

1. Afterwards, load (storage, reads, writes) is **shared fairly** among nodes.
2. During it, the database **keeps accepting reads and writes**.
3. **No more data than necessary** moves, to keep rebalancing fast and limit network and disk I/O.

Strategies for Rebalancing

How not to do it: hash mod N

Earlier (Figure 6-3) the advice was to split hash values into **ranges** and assign ranges to partitions (partition 0 if 0 ≤ hash(key) < b0, partition 1 if b0 ≤ hash(key) < b1, and so on). Why not just use **mod** (the `%` operator)? `hash(key) mod 10` gives 0–9 (the last decimal digit of the hash), which seems a neat way to pick among 10 nodes numbered 0–9.

The problem: when the number of nodes **N** changes, **most keys must move**. The book's example with hash(key) = 123456:

Nodes	123456 mod N	Key lives on
10	6	node 6
11	3	node 3
12	0	node 0

Such constant moving makes rebalancing far too expensive. We need a method that moves only what is necessary.

A quick demonstration (Python, my own numbers):

```
keys = range(100_000)
moved = sum(1 for k in keys if hash(str(k)) % 4 != hash(str(k)) % 5)
print(moved / len(keys))   # roughly 0.8: about 80% of keys change node going from 4 to 5 nodes
```

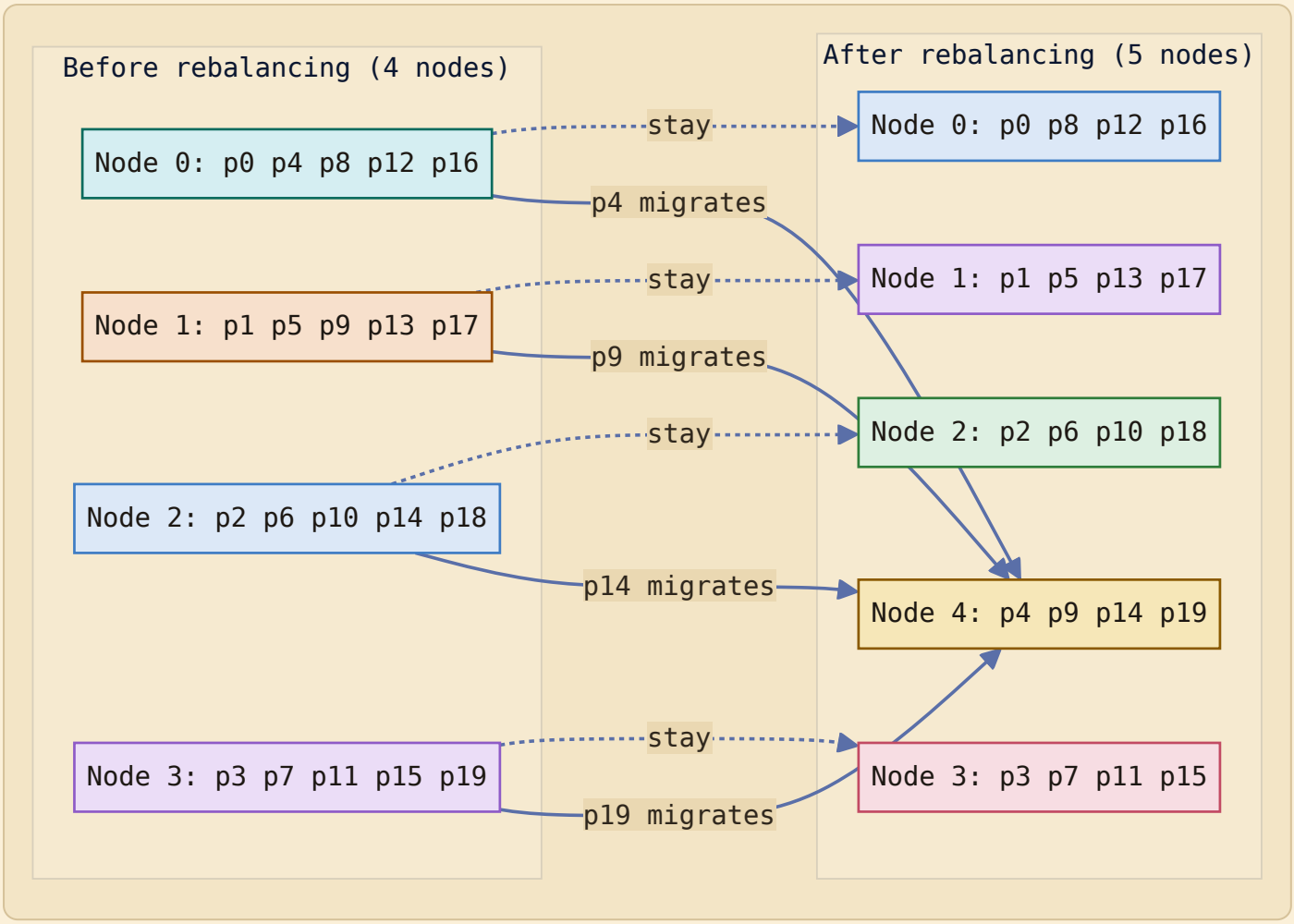
- Each key's node is `hash % N`; we compare N = 4 against N = 5.
- `moved` counts keys whose node changes; the result shows most keys would have to migrate for a single added node. (Python randomises string hashes per process, the same pitfall the book notes for Java and Ruby, but the proportion moved is what matters here.)

Fixed number of partitions

Idea: create **many more partitions than nodes** from the start and give each node several. E.g. a **10-node** cluster might start with **1,000 partitions**, about **100 per node**.

Adding a node: the newcomer **takes a few partitions away from each existing node** until the distribution is even again. Removing a node does the reverse.

Figure 6-6 (redrawn): 20 fixed partitions on a 4-node cluster; when node 4 joins, it takes one partition from each existing node (p4, p9, p14, p19). All other partitions stay where they were, and no key changes partition.



Key points:

- **Only whole partitions move.** The number of partitions and the key-to-partition mapping never change; only the **partition-to-node** assignment does.
- Reassignment is **not instant** (lots of data crosses the network), so the **old assignment keeps serving** reads and writes during the transfer.
- You can account for **uneven hardware** by giving more powerful nodes more partitions.
- **Used by:** Riak, Elasticsearch, Couchbase, Voldemort.

Choosing the number:

- It is usually **fixed at setup** and never changed. Splitting and merging are possible in principle, but fixed counts are operationally simpler, so many such databases do not implement splitting.
- So the count you choose at the start also caps the **number of nodes** the cluster can ever grow to: pick it high enough for future growth.
- But each partition has **management overhead**, so too many is counterproductive.
- If total data size varies a lot (small now, maybe huge later), it is hard to get right: each partition holds a fixed fraction of all data, so partition size grows with the dataset. **Very**

large partitions make rebalancing and recovery expensive; **very small** ones add overhead. Performance is best when partitions are "just right", which is hard with a fixed count and changing data volume.

Dynamic partitioning

For **key-range** partitioning, fixed partitions with fixed boundaries are awkward: wrong boundaries could put all data in one partition and leave the rest empty, and fixing boundaries by hand is tedious.

So key-range databases such as **HBase and RethinkDB** create partitions **dynamically**:

1. When a partition exceeds a configured size (**10 GB by default in HBase**), it **splits into two**, roughly half the data each.
2. If heavy deletion makes a partition fall under some threshold, it can **merge** with its neighbouring range.
3. The book compares this to a **B-tree** at its top level, where full pages are split in two.
4. Each partition is on one node, and a node can hold many partitions, as with fixed partitioning. After a split, one half can **move to another node** to balance load. In HBase, partition files move via **HDFS**, the underlying distributed filesystem.

Advantage: the number of partitions **adapts to data volume**: few partitions (low overhead) when data is small; with huge data, each partition's size is capped at a configurable maximum.

Caveat: an empty database starts with **one partition**, since there is no prior information about where boundaries should go. Until the first split, **one node handles all writes** while others idle. Mitigation: **HBase and MongoDB** allow **pre-splitting** (configuring an initial set of partitions on an empty database). With key-range partitioning, pre-splitting requires knowing the key distribution in advance.

Dynamic partitioning is not limited to key ranges; hashed data works too. **MongoDB since version 2.4** offers key-range and hash partitioning alike, and splits partitions dynamically under either.

Partitioning proportionally to nodes

Compare the approaches:

- **Dynamic partitioning:** number of partitions is proportional to **dataset size** (split/merge keeps each partition between a min and max size).
- **Fixed number of partitions:** size of each partition is proportional to **dataset size**.
- In both, how many nodes you run has **no bearing on how many partitions exist**.

Third option, used by Cassandra and Ketama: tie the partition count to the node count, i.e. keep a **fixed number of partitions per node**.

- With a fixed node count, partition size grows with the data; adding nodes makes partitions smaller again. Since more data usually means more nodes anyway, partition size stays **fairly stable**.
- **When a new node joins**, it picks a fixed number of existing partitions **at random**, **splits** each, and takes **one half** of each, leaving the other halves in place.
- Random choices can make **unfair splits**, but averaged over many partitions (**256 per node by default in Cassandra**) the new node gets a fair share. **Cassandra 3.0** introduced an alternative rebalancing algorithm that avoids unfair splits.
- Random boundaries require **hash-based partitioning** (boundaries are picked from the hash function's output range). This is the closest match to the **original definition of consistent hashing**. Newer hash functions can do something similar with lower metadata overhead.

Strategy	Number of partitions	Partition size	Who uses it	Watch out for
hash mod N	= nodes	varies	(avoid)	Most keys move when N changes
Fixed number of partitions	Fixed at setup	Grows with data	Riak, Elasticsearch, Couchbase, Voldemort	Choosing the count; caps node count
Dynamic partitioning	Grows with data	Bounded (e.g. HBase 10 GB split)	HBase, RethinkDB, MongoDB	Starts with one partition; pre-split
Proportional to nodes	Fixed per node	Stable-ish	Cassandra (256/node), Ketama	Random splits can be unfair

Operations: Automatic or Manual Rebalancing

Rebalancing ranges from **fully automatic** (the system decides when to move partitions, no admin involved) to **fully manual** (an admin explicitly sets the assignment and it changes only when they reconfigure). In between: **Couchbase, Riak, and Voldemort** generate a **suggested** assignment automatically, but an administrator must **commit** it before it takes effect.

	Fully automatic	Human in the loop
Convenience	Less routine operational work	Slower
Predictability	Can be unpredictable	Helps avoid operational surprises

Why automation can hurt. Rebalancing is **expensive**: requests are rerouted and lots of data moves. Done carelessly, it can overload the network or nodes and hurt other requests. Worse combined with **automatic failure detection**:

1. A node is overloaded and temporarily slow.
2. Other nodes decide it is dead.
3. They automatically rebalance load away from it.
4. That adds load to the overloaded node, the other nodes and the network, making things worse, possibly a **cascading failure**.

So having a **human in the loop** can be good: slower, but safer.

Request Routing

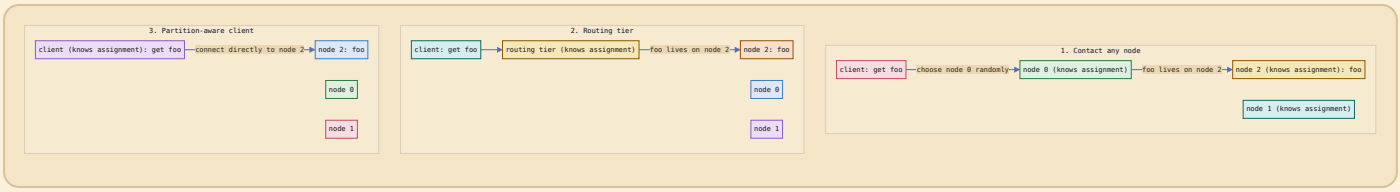
With data spread over nodes, how does a client know **which node to contact**? Assignments change as partitions are rebalanced, so something must track them to answer: *to read or write key "foo", which IP address and port do I connect to?*

This is an example of **service discovery**, a problem for any networked software, especially highly available software running redundantly on several machines. Many companies have built in-house service discovery tools, and many are open source.

Three high-level approaches:

1. **Contact any node**, for example through a **round-robin load balancer**. If it owns the partition, it handles the request; otherwise it **forwards** to the right node, gets the reply, and passes it back.
2. **Routing tier**: all requests go first to a routing tier that decides the right node and forwards the request. It handles no requests itself; it is a **partition-aware load balancer**.
3. **Partition-aware client**: clients know the partitioning and assignment and connect **directly** to the right node.

Figure 6-7 (redrawn): three ways to route a request for key "foo", which lives on node 2. The component that knows the partition-to-node assignment is, in turn, every node (1), a routing tier (2), or the client itself (3).

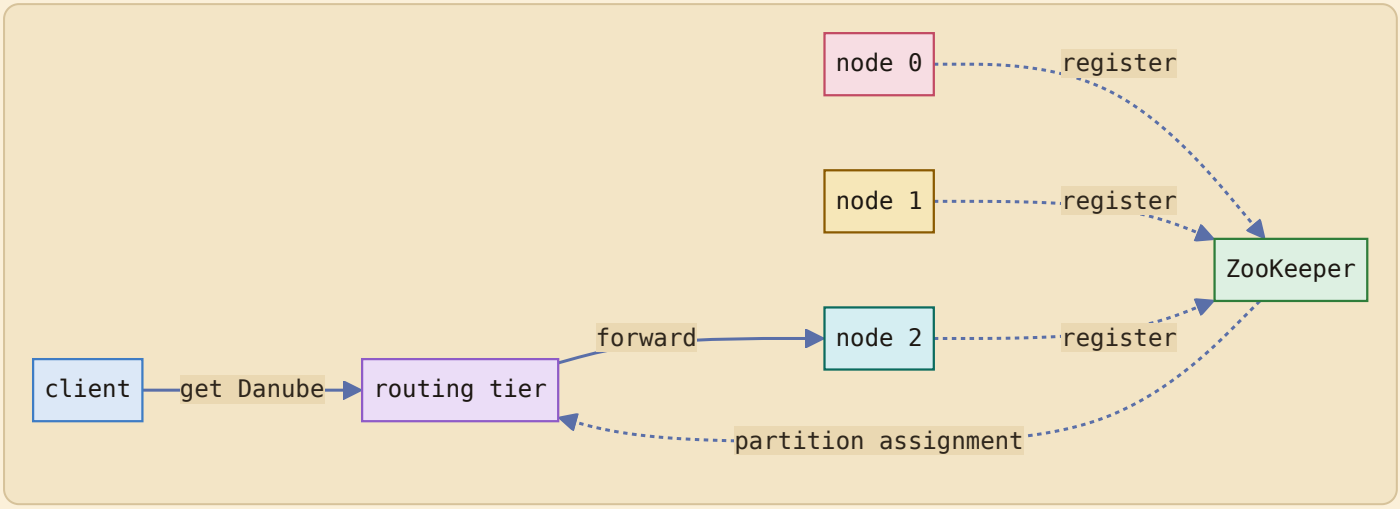


The core difficulty in all three: how does the routing decision-maker (a node, the routing tier, or the client) **find out** when partitions move to different nodes? All participants must agree, or requests go to the wrong nodes. Distributed consensus protocols exist but are hard to implement correctly (Chapter 9).

Using a coordination service (ZooKeeper). Many systems rely on **ZooKeeper** to track cluster metadata:

1. Each node **registers** itself in ZooKeeper.
2. ZooKeeper keeps the **authoritative mapping** of partitions to nodes.
3. Routing tiers or partition-aware clients **subscribe** to it.
4. Whenever a partition changes owner or a node is added/removed, ZooKeeper **notifies** the routing tier so it stays current.

Figure 6-8 (redrawn): the nodes register with ZooKeeper, which holds the authoritative table of key range -> partition -> node -> IP address; the routing tier keeps its copy in sync with ZooKeeper and forwards a client's get "Danube" to node 2.



Key range	Partition	Node	IP address
A-ak - Bayes	partition 0	node 0	10.20.30.100
Bayeu - Ceanothus	partition 1	node 1	10.20.30.101
Ceara - Deluc	partition 2	node 2	10.20.30.102
Delusion - Frenssen	partition 3	node 0	10.20.30.100
Freon - Holderlin	partition 4	node 1	10.20.30.101
Holderness - Krasnoje	partition 5	node 2	10.20.30.102
Krasnokamsk - Menadra	partition 6	node 0	10.20.30.100
Menage - Ottawa	partition 7	node 1	10.20.30.101
Otter - Rethimnon	partition 8	node 2	10.20.30.102
Reti - Solovets	partition 9	node 0	10.20.30.100
Solovyov - Truck	partition 10	node 1	10.20.30.101
Trudeau - Zywiec	partition 11	node 2	10.20.30.102

Who does what:

System	Routing approach
LinkedIn Espresso	Uses **Helix** for cluster management (Helix relies on ZooKeeper), with a routing tier as in Figure 6-8
HBase, SolrCloud, Kafka	Use ZooKeeper to track partition assignment
MongoDB	Similar architecture, but its own **config server** implementation and **mongos** daemons as the routing tier
Cassandra, Riak	**Gossip protocol** among nodes spreads cluster-state changes; requests can go to any node, which forwards them (approach 1). More complexity inside the nodes, but no dependency on an external service like ZooKeeper
Couchbase	Has no automatic rebalancing, keeping its design simpler; usually configured with a routing tier called **moxi** that learns routing changes from cluster nodes

Finding the IP addresses. Even with a routing tier or random-node approach, clients must find IPs to connect to. These change much less often than partition assignments, so **DNS** is usually good enough.

Parallel Query Execution

So far the focus has been simple queries that read or write **one key** (plus scatter/gather for document-partitioned indexes), which is roughly what most **NoSQL** distributed datastores support.

Massively parallel processing (MPP) relational databases, often used for **analytics**, support far richer queries. A typical data warehouse query has several **joins, filters, groupings and aggregations**. The MPP **query optimizer** splits it into **execution stages and partitions**, many of which run **in parallel on different nodes**. Queries that scan large parts of the dataset benefit most.

Fast parallel execution of warehouse queries is a specialised area with much commercial interest given how important analytics is to business. Chapter 10 discusses some techniques.

Summary

- **Why partition:** when there is too much data to store and process on one machine.
- **Goal:** spread data and query load evenly, avoiding **hot spots**. That needs a scheme suited to your data and **rebalancing** when nodes are added or removed.

Two main partitioning approaches:

	Key range partitioning	**Hash partitioning**
How	Keys sorted; a partition owns all keys from some minimum to some maximum	Hash each key; a partition owns a range of hashes
Range queries	Efficient	Inefficient (key order destroyed)
Hot-spot risk	High if the app often accesses keys close together in sort order	Lower; load more evenly spread
Typical rebalancing	Dynamic: split a range in two when a partition grows too big	Fixed number of partitions created in advance, several per node, whole partitions moved; dynamic also possible

Hybrid: a **compound key**, one part identifying the partition and another giving sort order (Cassandra).

Secondary indexes must also be partitioned:

- **Document-partitioned (local):** index stored alongside the primary key and value in the same partition. Writes update one partition; reads need **scatter/gather** across all partitions.
- **Term-partitioned (global):** index partitioned separately by indexed value; an entry may point to records in all partitions. Writes may update **several** index partitions; reads are served from **one**.

Routing: approaches span everything from a basic partition-aware load balancer up to elaborate **parallel query execution** engines.

By design, partitions operate mostly independently, which is what lets a partitioned database scale. But operations that write to **several partitions** are hard to reason about: what if the write to one partition succeeds and another fails? Later chapters address this.

Interview angle

1. Key-range vs hash partitioning? 30-second answer: Key-range assigns contiguous sorted key ranges to partitions (Bigtable, HBase, RethinkDB, MongoDB before 2.4). Range scans are cheap, but sequential keys like timestamps create hot spots. Hash partitioning assigns ranges of hash values (MD5 in Cassandra and MongoDB, Fowler–Noll–Vo in Voldemort), spreading load evenly but losing efficient range queries. Cassandra's compound key hashes the first column and sorts by the rest, getting some of both.

- *Follow-up: how would you avoid the timestamp hot spot for sensor data?* Put the sensor name before the timestamp so writes spread across partitions; the price is one range query per sensor when reading many sensors.
- *Follow-up: why not use Java's* `hashCode()`*?* The same key can hash differently in different processes, so nodes would disagree about where a key lives.

2. Why is hash mod N bad for rebalancing, and what do real systems do? 30-second answer: Changing N moves most keys (hash 123456 goes to node 6, 3, 0 for N = 10, 11, 12). Instead: a fixed large number of partitions with whole partitions moved (Riak, Elasticsearch, Couchbase, Voldemort); dynamic

splitting and merging (HBase at 10 GB by default, RethinkDB, MongoDB); or a fixed number of partitions per node (Cassandra with 256 per node, Ketama).

- *Follow-up: downside of a fixed partition count?* It caps the number of nodes and is hard to size when data volume varies: too big makes rebalancing and recovery costly, too small adds overhead.
- *Follow-up: downside of dynamic partitioning?* An empty database starts with one partition, so one node takes all writes until the first split; pre-splitting (HBase, MongoDB) helps if you know the key distribution.

3. Local vs global secondary indexes? 30-second answer: Local (document-partitioned) indexes live with each partition's data: writes touch one partition, reads must scatter/gather and suffer tail-latency amplification (MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB). Global (term-partitioned) indexes are partitioned by term: reads hit one partition, but writes touch several index partitions and are usually applied asynchronously (DynamoDB global secondary indexes update within a fraction of a second normally, longer during faults).

- *Follow-up: why not update global indexes synchronously?* It would need a distributed transaction across all affected index partitions, which many databases do not support.

4. How do you handle a celebrity hot key? 30-second answer: Hashing cannot help because the same key always hashes the same. Append a random two-digit number to split writes across 100 keys; reads then fetch all 100 and combine, and you must track which keys are split. Only do this for the few hot keys.

5. How does a request find the right partition? 30-second answer: Three options: any node forwards (Cassandra, Riak with gossip), a routing tier (Espresso via Helix/ZooKeeper, MongoDB mongos with config servers, Couchbase moxi), or partition-aware clients. The hard part is keeping everyone's view of the partition map in agreement; many systems store it in ZooKeeper, which notifies routers of changes. DNS is fine for finding node IPs.

- *Follow-up: should rebalancing be automatic?* Fully automatic rebalancing plus automatic failure detection can turn a slow node into a cascading failure; a human committing suggested plans (Couchbase, Riak, Voldemort) is slower but safer.

Self-check

1. **What is skew, and what is a hot spot?** Skew is unfair partitioning where some partitions get more data or queries than others; a hot spot is a partition with disproportionately high load. In the extreme, 9 of 10 nodes idle while one is the bottleneck.
2. **Why not just assign records to random nodes?** Load would be even, but you would not know where any record lives and would have to query all nodes in parallel for every read.
3. **Scenario: an IoT system keys readings by timestamp in a key-range-partitioned store and one node is overloaded. Why, and what change fixes it?** All current writes fall into the partition for the current time range (e.g. today). Prefix the key with the sensor name so writes spread across partitions; reading many sensors then needs one range query per sensor.
4. **In Cassandra with primary key (user_id, update_timestamp), which queries are efficient?** Queries fixing a user_id and scanning a range of timestamps: they touch one partition and use the sorted order. A range over user_id itself is not efficient because user_id is hashed.
5. **Trade-off: document-partitioned vs term-partitioned index for a read-heavy search page with many filters?** Term-partitioned makes each term lookup a single-partition read instead of scatter/gather, which suits read-heavy use; the costs are slower multi-partition writes and index updates that are often asynchronous (stale shortly after a write).
6. **With hash(key) = 123456, where does the key go with 10, 11, and 12 nodes under mod N?** Nodes 6, 3, and 0 respectively, which is why mod N moves too much data.

7. **In a fixed-partition scheme, what changes when a node is added, and what never changes?** Only the partition-to-node assignment changes (the new node takes a few partitions from each existing node). The number of partitions and the key-to-partition mapping never change. The old assignment serves requests while data transfers.
8. **What is pre-splitting and why is it needed?** Configuring an initial set of partitions on an empty database (HBase, MongoDB). Without it, dynamic partitioning starts with one partition, so a single node handles all writes until the first split. For key ranges you must know the key distribution in advance.
9. **How do Cassandra and Riak learn about partition assignment changes, compared with HBase?** Cassandra and Riak use a gossip protocol among nodes and let any node forward requests; HBase (like SolrCloud and Kafka) uses ZooKeeper to track assignments.
10. **Scenario: an overloaded but alive node triggers automatic rebalancing. What can happen?** Other nodes think it is dead and move its load, adding load to it, the other nodes and the network, which can cause a cascading failure. A human in the loop reduces this risk.

Chapter 7 — Transactions

Book: Chapter 7, pp. 221–272

Overview

The question this chapter answers: when many things can fail at once (crashes, network cuts, several users touching the same data), how can a database let an application act as if most of those problems simply do not happen? The answer the industry settled on decades ago is the **transaction**.

The chapter opens with a list of what goes wrong in real data systems:

- The database software or its hardware can die at any moment, even halfway through writing something.
- The application itself can crash in the middle of a sequence of steps.
- The network can drop, cutting the app off from the database, or one database node off from another.
- Several clients can write at the same moment and clobber each other's changes.
- A client can read data that is only half-updated and therefore makes no sense.
- Race conditions between clients can produce strange bugs.

Building fault-tolerance by hand for all of this is hard work that needs careful thought and lots of testing. A **transaction** packages a group of reads and writes into one logical unit. Either the whole unit **commits** (succeeds) or it **aborts** / **rolls back** (fails, and nothing it did survives). If it fails, the app can just try again. This removes the nightmare of **partial failure** — some steps done, some not.

Key framing points from the book:

- Nobody discovered transactions in nature; engineers *designed* them to make programming against a database simpler. The guarantees they give are called **safety guarantees**: the database handles certain faults and concurrency problems so the app does not have to.
- Not every app needs them. Sometimes you deliberately weaken or drop transactions to get more speed or availability, and some safety properties can be had without transactions at all.
- To decide whether you need them, you must know exactly what they guarantee and what they cost. The details are subtle.
- The chapter spends most of its time on **concurrency control**: the race conditions that can happen and the isolation levels (read committed, snapshot isolation, serializability) that guard against them.
- Everything here applies to single-node and distributed databases alike; Chapter 8 then covers problems unique to distributed systems.

The chapter's epigraph (from the Google Spanner paper, 2012) argues that it is better to let programmers deal with performance problems caused by *too many* transactions than to make them constantly work around *not having* transactions.

Everyday analogy: a transaction is like a bank teller processing a transfer on paper. The teller does not hand you a receipt until both the withdrawal from one account and the deposit into the other are written in the ledger. If the phone rings halfway through and the teller forgets, the half-finished entry is crossed out, and you simply start over. Meanwhile, other customers are not allowed to peek at the ledger while it shows a half-done transfer.

Key terms

Term	Plain-English meaning
Transaction	A group of reads and writes the database treats as one unit: all of it happens or none of it does.
Commit	The transaction finished successfully; its writes are now permanent and visible.
Abort / rollback	The transaction failed; the database throws away every write it made.
Partial failure	Some operations in a group succeeded while others did not — the mess transactions exist to prevent.
Safety guarantees	Promises the database makes (e.g., atomicity, isolation) so the application can ignore certain errors.
ACID	Atomicity, Consistency, Isolation, Durability — the classic label for transaction guarantees (coined 1983).
BASE	Basically Available, Soft state, Eventual consistency — a vague label that effectively means "not ACID".
Atomicity (ACID)	If a transaction cannot finish, all its writes are undone. Better thought of as "abortability".
Consistency (ACID)	Application-defined rules (invariants) about the data stay true. Really a property of the app, not the DB.
Invariant	A statement about the data that must always hold (e.g., debits and credits balance).
Isolation (ACID)	Transactions running at the same time do not interfere with each other.
Serializability	Usually regarded as the strongest isolation: the outcome is the same as if transactions ran one after another.
Durability (ACID)	Once committed, data is not lost even if the machine crashes.
Write-ahead log (WAL)	An append-only file of intended changes, used to recover after a crash.
Race condition	A bug whose appearance depends on the unlucky timing of concurrent operations.
Multi-object transaction	A transaction that changes several rows/documents/records together.
Single-object operation	An atomic change to just one object (e.g., one key-value pair).
Denormalization	Storing a copy or derived value (like a counter) to avoid expensive queries; must be kept in sync.
Compare-and-set (CAS)	Write only if the current value is still what you read earlier.
Isolation level	How strongly a database separates concurrent transactions (read committed, snapshot, serializable, ...).
Read uncommitted	A very weak level: prevents dirty writes but allows dirty reads.
Read committed	Prevents dirty reads and dirty writes.
Dirty read	Seeing another transaction's writes before it has committed.
Dirty write	Overwriting another transaction's value before it has committed.
Row-level lock	A lock on one row/document that a writer holds until it commits or aborts.

Term	Plain-English meaning
Read skew / nonrepeatable read	Reading different parts of the database at different moments, so the combined picture is inconsistent.
Snapshot isolation	Each transaction reads from a frozen, consistent picture of the database as of its start.
MVCC	Multi-version concurrency control: keeping several committed versions of each object so readers can see old snapshots.
Transaction ID (txid)	A unique, ever-increasing number the database gives each transaction.
Garbage collection (in MVCC)	A background process that deletes old row versions no one can see anymore.
Append-only / copy-on-write B-tree	A B-tree that never overwrites pages; each write creates new pages and a new root.
Repeatable read	An SQL-standard isolation name whose meaning differs by vendor; often actually snapshot isolation.
Lost update	Two read-modify-write cycles run concurrently and one silently overwrites the other's change.
Read-modify-write cycle	Read a value, compute a new one in app code, write it back.
Atomic write operation	A database-provided update (like `value = value + 1`) that is safe under concurrency.
Cursor stability	Implementing atomic updates by taking an exclusive lock on an object when it is read.
Explicit locking	The app asks the DB to lock specific rows (e.g., `SELECT ... FOR UPDATE`).
Siblings	Conflicting versions of a value kept side by side in a replicated DB, to be merged later.
Commutative operation	An operation whose result does not depend on order (e.g., increment, add to set).
Last write wins (LWW)	Conflict resolution that keeps only the "latest" write, discarding others — loses updates.
Write skew	Two transactions read the same data, then each updates *different* objects, together breaking a rule.
Phantom	One transaction's write alters which rows another transaction's search would match.
Materializing conflicts	Creating artificial rows purely so there is something concrete to lock.
Actual serial execution	Literally running one transaction at a time on a single thread.
Stored procedure	Transaction code submitted to and run entirely inside the database.
OLTP	Online transaction processing: many short reads/writes typical of user-facing apps.
Anti-caching	Abort a transaction needing on-disk data, load the data in the background, then restart it.
Cross-partition transaction	A transaction that touches data on more than one partition; needs coordination.
Two-phase locking (2PL)	Readers and writers block each other; locks are held until the transaction ends.

Term	Plain-English meaning
Shared lock / exclusive lock	Shared: many readers may hold it. Exclusive: only one holder, blocks everyone else.
Deadlock	Two transactions each wait for a lock the other holds; the DB aborts one.
Predicate lock	A lock on *all objects matching a condition*, including ones not yet inserted.
Index-range lock (next-key lock)	A cheaper approximation of a predicate lock, attached to a range in an index.
Pessimistic concurrency control	Wait whenever something might go wrong (2PL, serial execution).
Optimistic concurrency control	Proceed without waiting; check at commit time and abort if something went wrong.
Serializable snapshot isolation (SSI)	Optimistic algorithm on top of snapshot isolation that detects conflicts and aborts to guarantee serializability.
Premise	A fact a transaction read and based its decision on (e.g., "two doctors are on call").
Contention	Lots of transactions competing for the same objects simultaneously.

The Slippery Concept of a Transaction

Nearly every relational database today, and some non-relational ones, support transactions. Their style goes back to **IBM System R** in **1975**, the first SQL database. Although the internals have evolved, the transaction support you find in **MySQL, PostgreSQL, Oracle and SQL Server** looks remarkably like System R's, even 40 years later.

In the late 2000s, **NoSQL** (non-relational) databases became popular. They offered new data models (Chapter 2) and shipped with replication (Chapter 5) and partitioning (Chapter 6) built in. The biggest casualty was transactions: many of these systems dropped them entirely or used the word "transaction" for something much weaker than before.

Two exaggerated opinions grew out of this:

- "Transactions kill scalability; any big system must give them up for speed and availability."
- "Serious apps with valuable data *must* have transactions" (a line favoured by some database vendors).

The book calls both views hype. Like any design choice, transactions have benefits and limits, and you need to know the details to weigh them.

The Meaning of ACID

ACID stands for **Atomicity, Consistency, Isolation, Durability**. **Theo Härder and Andreas Reuter** coined it in **1983** to give precise names to database fault-tolerance mechanisms.

Common confusion: in practice, "ACID" means different things in different products. Isolation in particular is interpreted in many ways. So when a vendor says its system is "ACID compliant", you still do not know exactly what you get — the term has become largely marketing.

BASE. Systems that are not ACID are sometimes labelled BASE: *Basically Available, Soft state, Eventual consistency*. This is even fuzzier than ACID; the only reliable meaning of BASE is "not ACID".

Atomicity

What it is. In general, "atomic" means "cannot be split into smaller pieces". But the word means slightly different things in different fields:

- In **multi-threaded programming**, an atomic operation means no other thread can observe it half-done; you only ever see the state before or after.
- In **ACID**, atomicity is **not** about concurrency at all (that is the "I", isolation). ACID atomicity is about **what happens when a fault strikes after only some of a client's writes have been applied**.

How it works.

1. A client groups several writes into a transaction.
2. Partway through, something goes wrong — the process crashes, the network drops, the disk fills up, or an integrity constraint is violated.
3. Because the transaction cannot commit, the database aborts it and **discards or undoes every write it already made**.

Why it matters. Without atomicity, after an error you cannot tell which changes landed and which did not. Retrying could apply the same change twice (duplicates or wrong data). With atomicity, an aborted transaction is guaranteed to have changed nothing, so retrying is safe.

The book suggests **"abortability"** would have been a more accurate name: the defining feature is the ability to abort on error and have all writes thrown away.

Consistency

The word "consistency" is badly overloaded. The book lists at least four meanings:

Where	What "consistency" means there
Replication (Chapter 5)	Replica consistency, and *eventual consistency* in asynchronously replicated systems.
Partitioning (Chapter 6)	*Consistent hashing*, a technique some systems use for rebalancing.
CAP theorem (Chapter 9)	Linearizability.
ACID	The database being in an application-defined "good state".

ACID consistency explained. You have **invariants** — facts about your data that must always be true. Example from the book: in an accounting system, total credits and debits across all accounts must balance. If each transaction starts from a valid state and its writes keep things valid, the invariants always hold.

The catch. Only the **application** knows what the invariants are, so it is the app's job to write transactions that preserve them. The database cannot stop you writing data that breaks your own rules. A few rule types *can* be enforced by the database itself — **foreign key constraints** and **uniqueness constraints** — but in general the app decides what counts as valid; the database just stores it.

So A, I and D are **guarantees the database provides**, while consistency is a **property of the application** (which relies on A and I to achieve it). The "C" does not really belong in ACID.

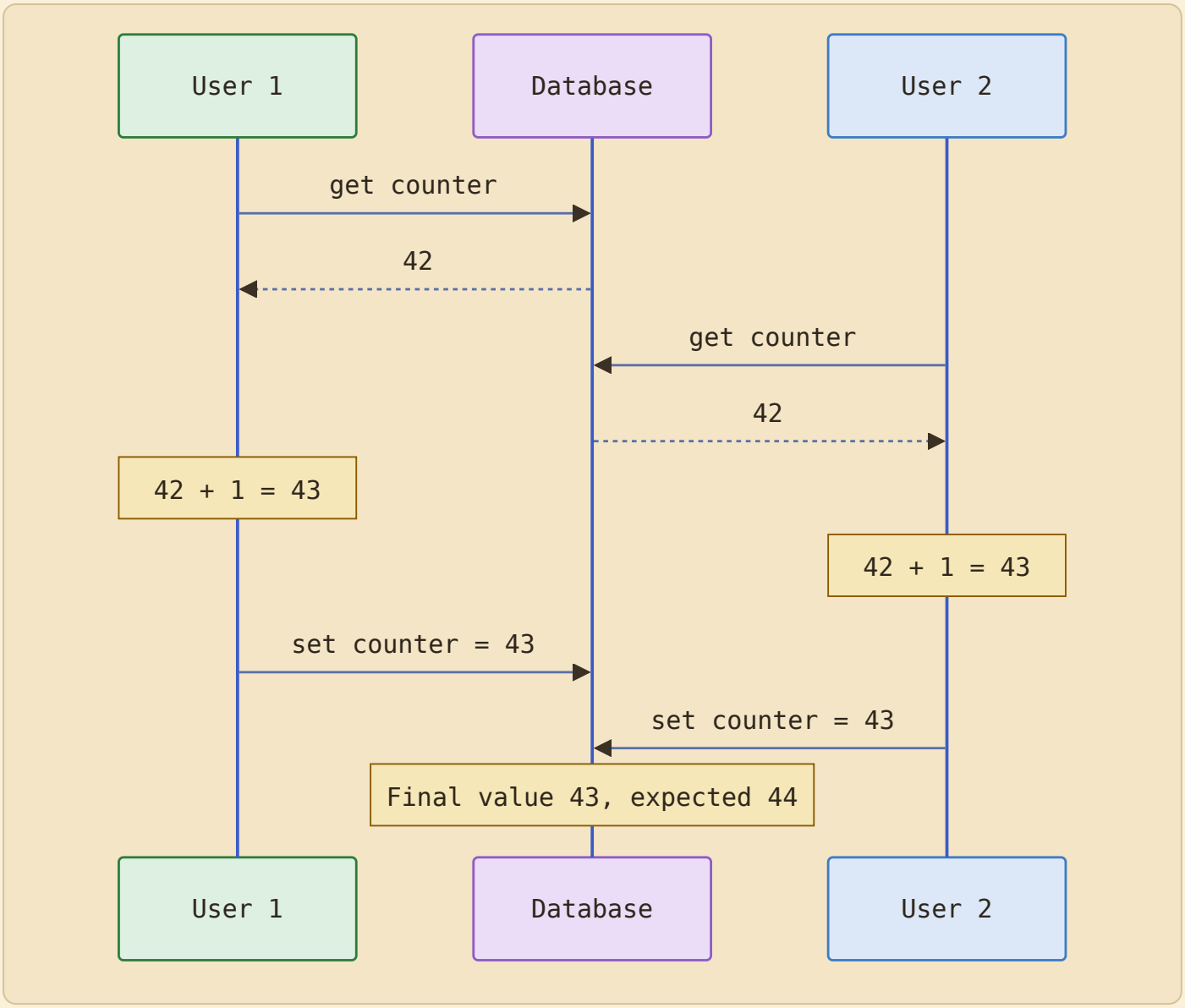
> **Aside:** Joe Hellerstein has commented that the C was added mainly so the acronym would work, and that it was not considered important in the original paper.

Isolation

The problem. Many clients use the database at once. That is fine when they touch different data, but when they touch the **same records** you get **race conditions**.

The book's example (Figure 7-1). Two clients each want to increment a counter. Each reads the current value, adds 1 in its own code, and writes the result back (assume there is no built-in increment). The counter starts at 42; after two increments it should be 44, but because both read 42 before either wrote, it ends at **43**. One increment vanished.

Figure 7-1 (redrawn): two clients increment the same counter concurrently and one increment is lost



What isolation means in ACID. Concurrent transactions must not interfere with each other. Classic database textbooks formalise this as **serializability**: every transaction may behave as if it is the only one running; the database makes sure that the final committed result equals *some* one-after-another (serial) order, even though the transactions actually overlapped.

Common confusion. In practice, full serializable isolation is **rarely used** because it costs performance. Some popular databases do not even implement it: **Oracle 11g** has a level named "serializable", but it actually provides **snapshot isolation**, which is weaker. (Snapshot isolation is covered under Weak Isolation Levels below.)

Durability

What it is. A database should be a safe place to keep data. **Durability** promises that once a transaction has committed, its data will not be forgotten, even if hardware fails or the database crashes.

How it is achieved.

- **Single-node database:** the data has been written to **non-volatile storage** (hard drive or SSD), usually together with a **write-ahead log** (see Chapter 3) so the on-disk structures can be recovered if they get corrupted.
- **Replicated database:** the data has been copied to **some number of nodes**.
- In both cases, the database must **wait** until those writes or copies finish before it tells the client "committed".

Limitation. **Perfect durability does not exist.** If every disk and every backup is destroyed simultaneously, nothing can save you.

> **Replication and Durability (sidebar).** Durability used to mean writing to archive tape, then to disk/SSD, and more recently to replicas. None is perfect:
>
> - Data on the disk of a dead machine is safe but *unreachable* until the machine is fixed or the disk moved; replicated systems can stay available.
> - A **correlated fault** — say, a power outage, or a bug that makes all nodes crash when fed one particular input — can take out all replicas at once and wipe anything held only in memory — so disk writes still matter even for in-memory databases.
> - With **asynchronous replication**, recent writes can be lost if the leader fails.
> - On sudden power loss, **SSDs** have been shown to sometimes break their promises; even `fsync` is not guaranteed to work correctly. Disk **firmware has bugs** too.
> - Tricky interplay between the storage engine and the **filesystem** can produce hard-to-find bugs and corrupted files after a crash.
> - Data on disk can **silently rot**. If it went unnoticed for a while, replicas and recent backups may be corrupt too, so you may need an older historical backup.
> - One study found that **30% to 80% of SSDs** develop at least one bad block in their **first four years**. Magnetic disks have fewer bad sectors but fail completely more often than SSDs.
> - An **unpowered** SSD may begin to lose data after **a few weeks**; how fast depends on temperature.
>
> Takeaway: there are no absolute guarantees, only risk-reduction techniques — disk writes, replication to remote machines, and backups — which should be combined. Treat theoretical "guarantees" sceptically.

Single-Object and Multi-Object Operations

A quick recap of the two ACID properties that describe what happens when a client makes **several writes in one transaction**:

- **Atomicity:** if something fails midway, abort and discard all writes so far — an **all-or-nothing** guarantee, so you never deal with partial failure.
- **Isolation:** concurrent transactions do not interfere. If one transaction makes several writes, another transaction sees **all of them or none**, never a subset.

These definitions assume you are changing **several objects** (rows, documents, records) at once — a **multi-object transaction**. That is needed whenever several pieces of data must stay in sync.

The book's email example. To show a user's unread count you could run a counting query. Here is an equivalent query of my own:

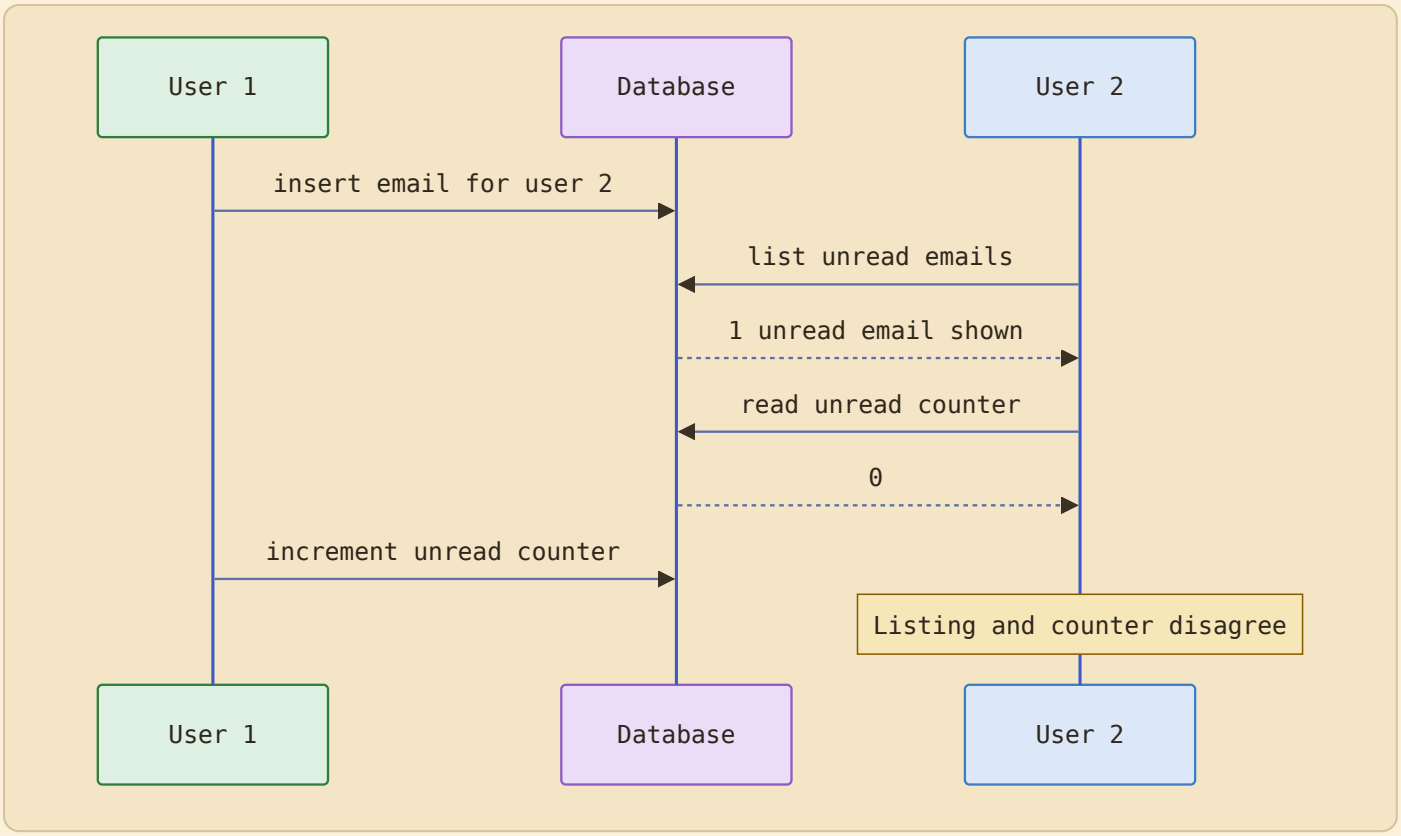
```sql
SELECT COUNT(*) FROM messages
WHERE owner_id = 7 AND is_read = false;
```

- `SELECT COUNT(*)` — count matching rows.
- `FROM messages` — look in the messages table.
- `WHERE owner_id = 7 AND is_read = false` — only this user's unread messages.

With lots of mail this can be too slow, so you might **denormalize**: keep an `unread_count` field that you increment when a message arrives and decrement when one is read. Now two things must change together.

Figure 7-2 — isolation violated. User 1 inserts a new email and then increments the counter. Between those two writes, User 2 reads: the mailbox shows the new unread email, but the counter still says 0. That is a **dirty read** (reading uncommitted data). Isolation would guarantee User 2 sees both changes or neither.

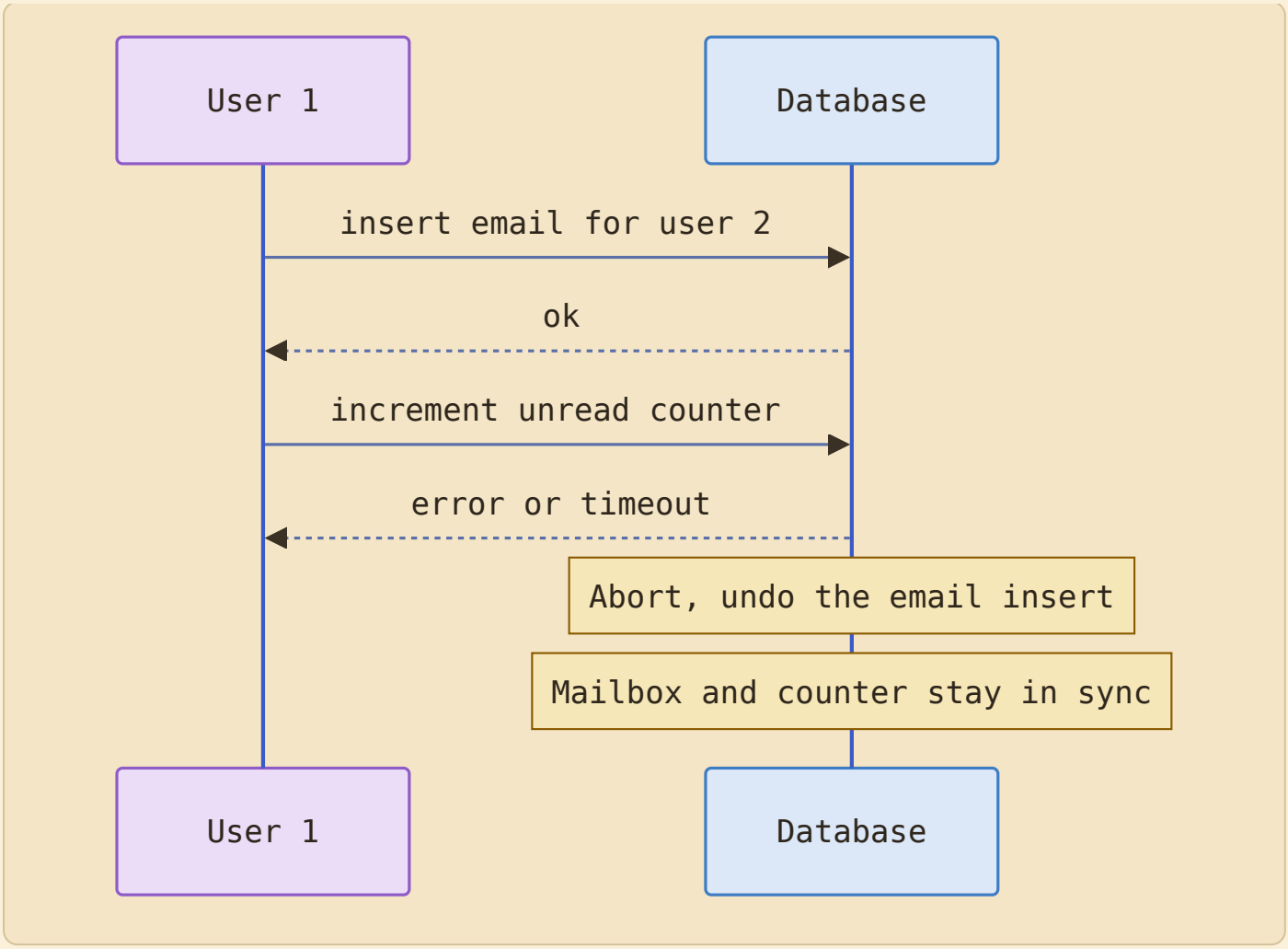
Figure 7-2 (redrawn): a reader sees the new email but not the updated counter (dirty read)



> **Footnote:** a wrong unread counter is not a big deal. But swap in a customer's account balance and a payment, and the same anomaly is serious.

Figure 7-3 — atomicity needed. The email is inserted, but the counter update fails (error). Without atomicity the mailbox and counter drift apart. With an atomic transaction, the failure aborts the whole thing and the inserted email is **rolled back**.

Figure 7-3 (redrawn): an error during the counter update causes the earlier email insert to be undone



How does the database know which operations belong together? In relational databases it is usually tied to the client's **TCP connection**: on a given connection, everything between `BEGIN TRANSACTION` and `COMMIT` is one transaction.

> **Footnote:** tying transactions to a TCP connection is not ideal. If the connection breaks, the transaction must abort. If it breaks *after* the client asked to commit but *before* the server confirmed, the client cannot tell whether it committed. A transaction manager can fix this with a unique **transaction ID** that is independent of the connection (revisited in Chapter 12, "The End-to-End Argument for Databases").

Many **non-relational** databases have no way to group operations like this. Even a **multi-put** (update several keys in one call) may lack transaction semantics: some keys may be written while others fail, so you end up half-updated.

Single-object writes

Atomicity and isolation matter even for **one** object. Imagine writing a **20 KB JSON document**:

- The network drops after **10 KB** — does the database store a broken half-document?
- Power fails while overwriting the old value on disk — do you get old and new bytes spliced together?
- Another client reads mid-write — does it see a half-updated value?

Because those outcomes would be baffling, storage engines almost always provide **atomicity and isolation for a single object** (such as one key-value pair) on one node:

- **Atomicity** via a **log for crash recovery** (see Chapter 3).

- **Isolation** via a **lock on each object**, so only one thread touches it at a time.

Some databases go further with **more complex single-object atomic operations**:

- An **increment** operation, which avoids the read-modify-write cycle of Figure 7-1.
- **Compare-and-set**, which writes only if nobody else changed the value in the meantime.

These help prevent lost updates, but calling them **transactions** stretches the usual meaning. Vendors have marketed compare-and-set and friends as "lightweight transactions" or even "ACID", which the book calls misleading. The normal understanding is that a transaction bundles **several operations across several objects** into a single unit of execution.

> **Footnote:** strictly, "atomic increment" uses "atomic" in the multi-threading sense. In ACID terms it would be an *isolated* or *serializable* increment — but that is nitpicking.

The need for multi-object transactions

Many distributed datastores dropped multi-object transactions because they are **hard across partitions** and can hurt when very high availability or performance is needed. But nothing makes distributed transactions *impossible* (Chapter 9 covers how).

Could every app get by with just key-value, single-object operations? Sometimes yes, but often several objects must change together:

1. **Foreign keys / graph edges.** In relational models, rows point to rows in other tables; in graph models, vertices have edges to other vertices. When inserting several records that reference each other, the references must be correct, or the data stops making sense.
2. **Denormalized documents.** In a document model, fields that change together are often in one document, so no multi-object transaction is needed for that. But document databases without joins push you toward denormalization, and updating denormalized copies (like Figure 7-2) means touching several documents at once. Transactions keep those copies in sync.
3. **Secondary indexes.** Almost every database except pure key-value stores has them, and they must be updated whenever a value changes. From a transaction's viewpoint an index is a separate object, so without isolation a record might appear in one index but not yet in another.

You *can* build these apps without transactions, but error handling gets much harder without atomicity, and missing isolation causes concurrency bugs (discussed next; Chapter 12 explores alternatives).

Handling errors and aborts

Philosophy. A key feature of transactions is that they **can be aborted and safely retried**. ACID databases would rather abandon a transaction completely than leave it half done and risk breaking atomicity, isolation or durability.

Not everyone follows it. Databases with **leaderless replication** (Chapter 5) work on a **"best effort"** basis: the database does what it can, and if it hits an error it will not undo what already happened. Recovery is the application's job.

Common mistake. Developers often think only about the happy path. Popular **ORM** (object-relational mapping) frameworks (the book names **ActiveRecord** from Rails, and **Django**) do **not retry** aborted transactions; the error bubbles up as an exception, user input is lost, and the user sees an error page. That wastes the whole point of aborts, which is to allow safe retries.

Retrying is useful but not perfect:

Problem with retry	Why / what to do
The commit actually succeeded, but the acknowledgement was lost on the network	Retrying runs it twice, unless the app has its own **deduplication**.
The error was caused by **overload**	Retrying makes it worse. Limit retries, use **exponential backoff**, and treat overload errors specially.
The error is **permanent** (e.g., constraint violation)	Retrying is pointless. Only retry **transient** errors: deadlock, isolation violation, brief network glitches, failover.
The transaction has **side effects outside the database** (e.g., sending an email)	Those may happen even if the transaction aborts; you do not want to resend the email on each retry. To make several systems commit or abort together, see **two-phase commit** (Chapter 9).
The **client process dies** while retrying	Whatever it was trying to write is lost.

Weak Isolation Levels

When do concurrency issues arise? Two transactions that touch different data can safely run in parallel. Trouble only starts when one transaction **reads data another is modifying**, or two transactions **modify the same data** at once.

Why they are nasty.

- They are **hard to find by testing**: they only appear when the timing is unlucky, which may be rare and hard to reproduce.
- They are **hard to reason about**, especially in big codebases where you do not know what other code touches the same data. With many concurrent users, any value might change under you at any time.

Databases have long tried to hide this with **transaction isolation**. **Serializable isolation** would let you pretend there is no concurrency at all. But it has a **performance cost**, so many systems instead use **weaker isolation levels** that stop some problems but not all. These are harder to understand and cause subtle bugs, yet they are widely used.

Real damage. Weak-isolation bugs have caused **large financial losses** (the book's references include bitcoin stolen from an exchange, e.g. Poloniex in 2014), triggered **investigations by financial auditors**, and **corrupted customer data**. The usual reaction — "use an ACID database for money!" — misses the point, because many popular relational ("ACID") databases also default to weak isolation and would not have prevented those bugs.

So instead of trusting labels, you need to **understand the kinds of race conditions** and how to prevent them. The book walks through weak (non-serializable) levels informally with examples; rigorous definitions live in the academic literature.

Read Committed

The most basic isolation level is **read committed**. It makes two promises:

1. **No dirty reads:** when you read, you only see committed data.
2. **No dirty writes:** when you write, you only overwrite committed data.

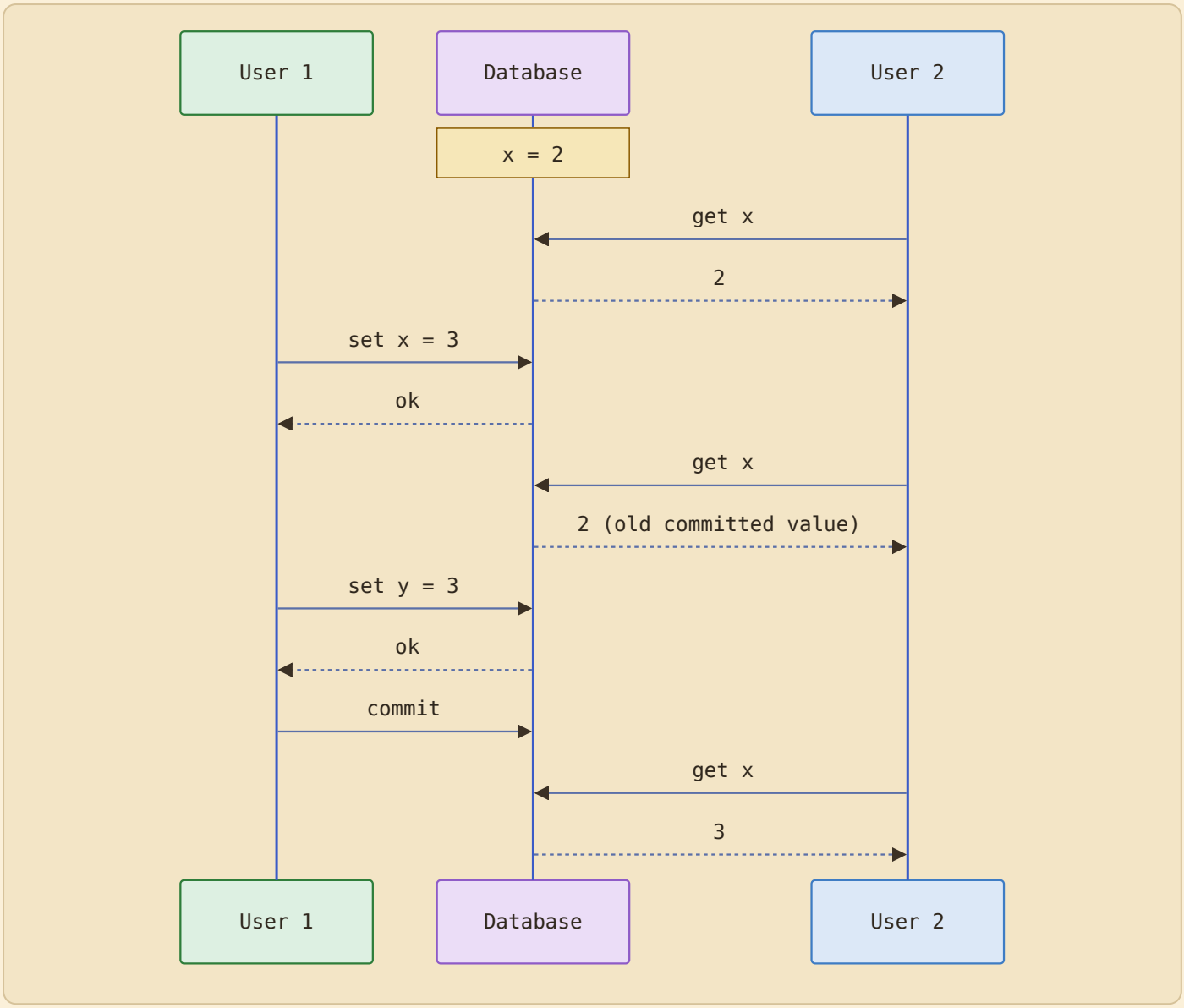
> **Footnote:** some databases offer an even weaker level, **read uncommitted**, which prevents dirty writes but *allows* dirty reads.

No dirty reads

What it is. A **dirty read** happens if a transaction can see data written by another transaction that has not yet committed or aborted. Under read committed, a transaction's writes become visible to others **only when it commits**, and then **all at once**.

Figure 7-4. User 1's transaction sets `x = 3` and then `y = 3` but has not committed. Each time User 2 reads `x` in the meantime it still gets the old value **2**. Only after User 1 commits does User 2 see 3.

Figure 7-4 (redrawn): no dirty reads, the new x becomes visible to user 2 only once user 1 commits



Why prevent dirty reads?

- If a transaction updates several objects, a dirty read can show **some updates but not others** — exactly the unread-email-vs-counter mismatch in Figure 7-2. Seeing a half-done state confuses users and can make other transactions decide wrongly.
- If a transaction **aborts**, its writes are rolled back (Figure 7-3). With dirty reads allowed, someone may have read data that **never actually got committed**, which gets very hard to reason about.

No dirty writes

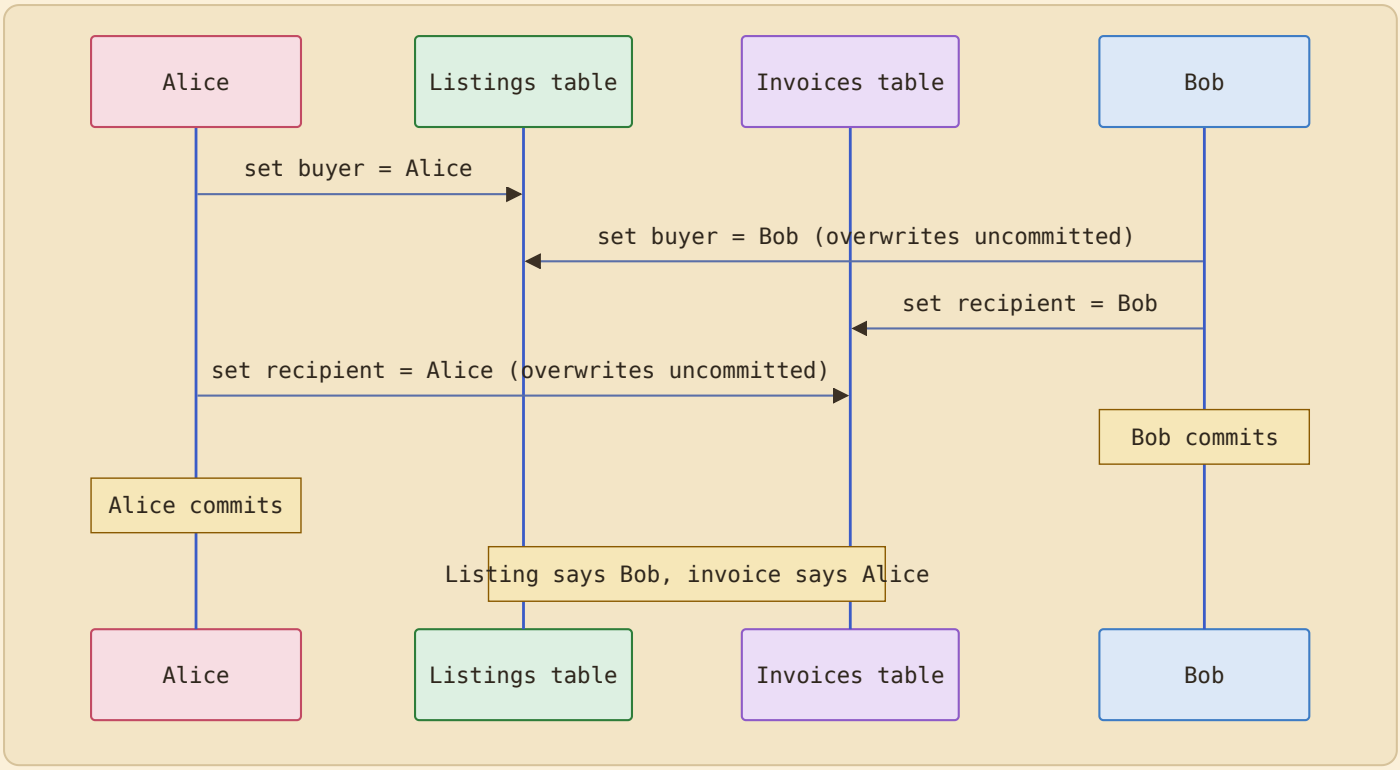
What it is. Normally, when two transactions write the same object, the later write overwrites the earlier one. A **dirty write** is when the earlier write belongs to a transaction that **has not committed**

yet, and the later write overwrites that uncommitted value. Read committed prevents this, typically by **making the second writer wait** until the first transaction commits or aborts.

What it protects against.

- **Mixing up multi-object writes.** The book's used-car website example (Figure 7-5): Alice and Bob try to buy the same car at the same time. A purchase needs two writes: update the **listing** to the buyer, and send the **invoice** to the buyer. With dirty writes, Bob wins the listings update but Alice wins the invoices update — the car goes to Bob while Alice gets the bill. Read committed stops that.
- **What it does *not* stop:** the counter race in Figure 7-1. In that case the second writer only writes once the first transaction has already committed, so no dirty write occurs. It is still wrong, but for a different reason (a **lost update**, covered below).

Figure 7-5 (redrawn): with dirty writes, Bob ends up with the listing while Alice gets the invoice



Implementing read committed

Read committed is **very popular**: it is the default level in plenty of databases, including **Oracle 11g**, **PostgreSQL**, **SQL Server 2012** and **MemSQL**.

Preventing dirty writes — row-level locks.

1. A transaction that wants to modify an object (row or document) first **acquires a lock** on it.
2. It **holds** that lock until it commits or aborts.
3. Only one transaction can hold the lock on a given object; another writer must **wait** until the first finishes.
4. The database does this **automatically** at read committed and stronger levels.

Preventing dirty reads — option 1: read locks (rarely used). Make readers briefly take the same lock and release it right after reading. A read then cannot happen while an uncommitted value exists. **Problem:** one long-running write transaction would force many read-only transactions to wait, hurting their response time. It also harms operability: a slowdown in one part of the app spreads to unrelated parts via lock waits.

Preventing dirty reads — option 2: remember old values (what most databases do). For every object being written, the database keeps **both the old committed value and the new uncommitted value** set by

the lock-holding transaction. While the writer is still running, other readers get the **old** value. Once it commits, readers switch to the **new** value. This is the approach shown in Figure 7-4.

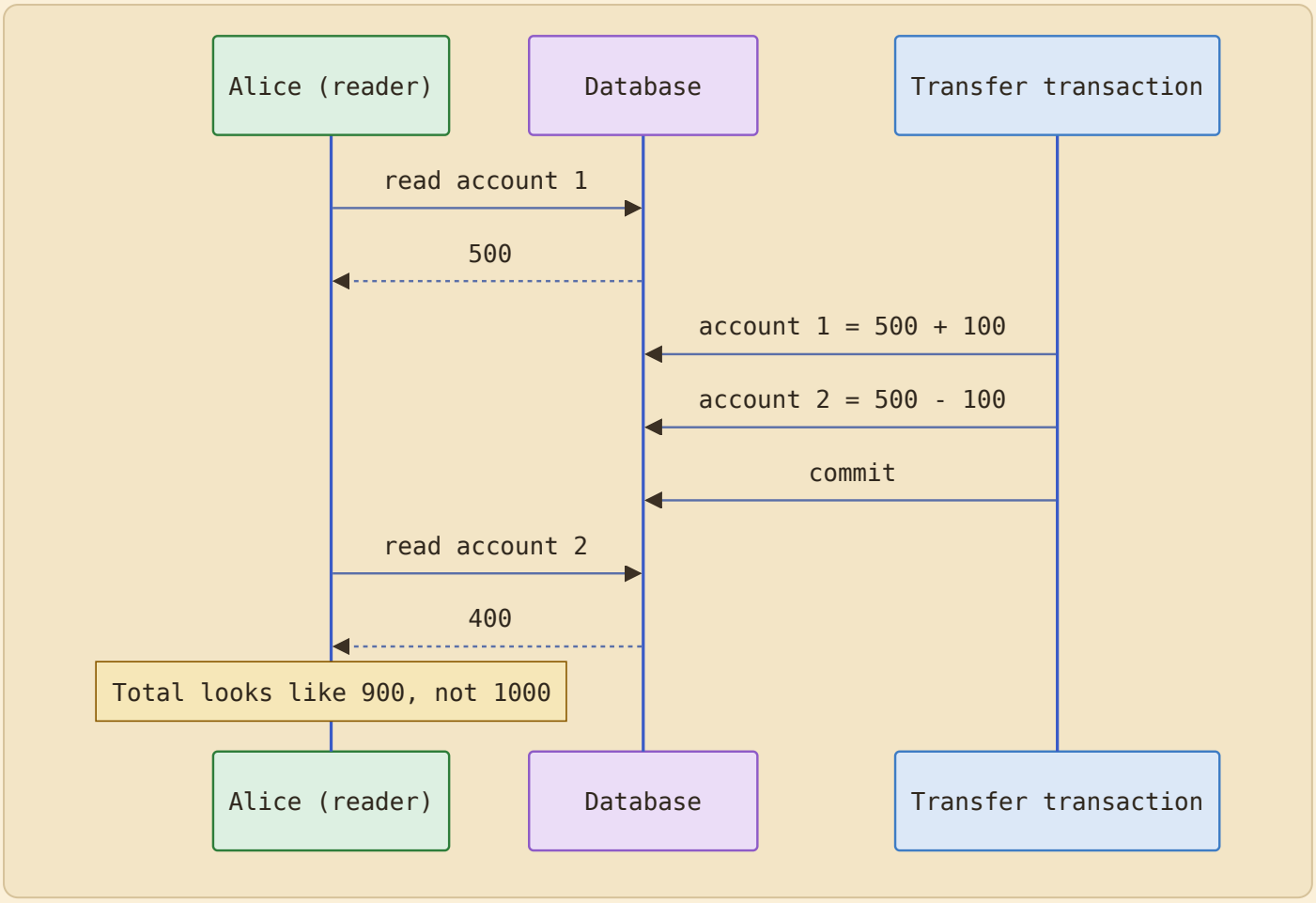
> **Footnote:** when the book was written, just two mainstream databases implemented read committed with locks: **IBM DB2** and **Microsoft SQL Server with** `read_committed_snapshot=off`.

Snapshot Isolation and Repeatable Read

Read committed looks like it covers everything: it supports aborts (for atomicity), hides half-finished transactions, and stops writes from interleaving. Those are real improvements over having no transactions. But plenty of concurrency bugs remain.

Figure 7-6 — read skew. Alice has **$1,000** split across two accounts, **$500** each. A transaction moves **$100** from account 2 to account 1. Alice checks her balances at an unlucky moment: she reads account 1 *before* the incoming $100 arrives (sees **$500**) and account 2 *after* the $100 left (sees **$400**). It looks like she has only **$900** — $100 has disappeared.

Figure 7-6 (redrawn): Alice reads the two accounts at different moments during a transfer and sees only 900 dollars total



The name for this is **read skew** (also **nonrepeatable read**): if Alice re-read account 1 at the end she would see **$600**, a different value. Read committed **allows** this, because each value she saw really was committed when she read it.

> **"Skew" is overloaded.** Earlier (Chapter 6) skew meant an unbalanced workload with hot spots. Here it means a **timing anomaly**.

For Alice this is temporary — reloading the page shortly after shows consistent balances. But some situations **cannot tolerate** even temporary inconsistency:

- **Backups.** Copying a large database can take **hours** while writes keep arriving. Parts of the backup may be older and parts newer. If you ever restore it, the inconsistencies (like vanished money) become **permanent**.
- **Analytic queries and integrity checks.** Big scans for analytics (Chapter 3) or periodic checks for corruption give **nonsense** if they see different parts of the database at different moments.

The fix — snapshot isolation. Each transaction reads from a **consistent snapshot**: it sees everything that was committed **when the transaction started**. Even if others change data later, the transaction keeps seeing the old data from that point in time.

Why it helps. It is ideal for long-running, read-only jobs like backups and analytics. Reasoning about a query while its data changes underneath it is very hard; a frozen snapshot is easy to understand.

Who supports it: **PostgreSQL, MySQL with InnoDB, Oracle, SQL Server**, and others.

Implementing snapshot isolation

Writes: like read committed, snapshot isolation typically uses **write locks** to stop dirty writes, so a writer can block another writer on the same object.

Reads: **no locks at all.** The key performance principle is:

> **A reader never has to wait for a writer, and a writer never has to wait for a reader.**

This lets the database serve long-running reads on a consistent snapshot while processing writes normally, with no lock contention between them.

MVCC. Snapshot isolation generalises the Figure 7-4 trick. Because different in-progress transactions may need to see the database at **different points in time**, the database may keep **several committed versions** of each object side by side. This is **multi-version concurrency control (MVCC)**.

Level	Versions needed	Snapshot scope (typical MVCC implementation)
Read committed	Two are enough: committed + not-yet-committed	A new snapshot for **each query**
Snapshot isolation	Many committed versions	One snapshot for the **whole transaction**

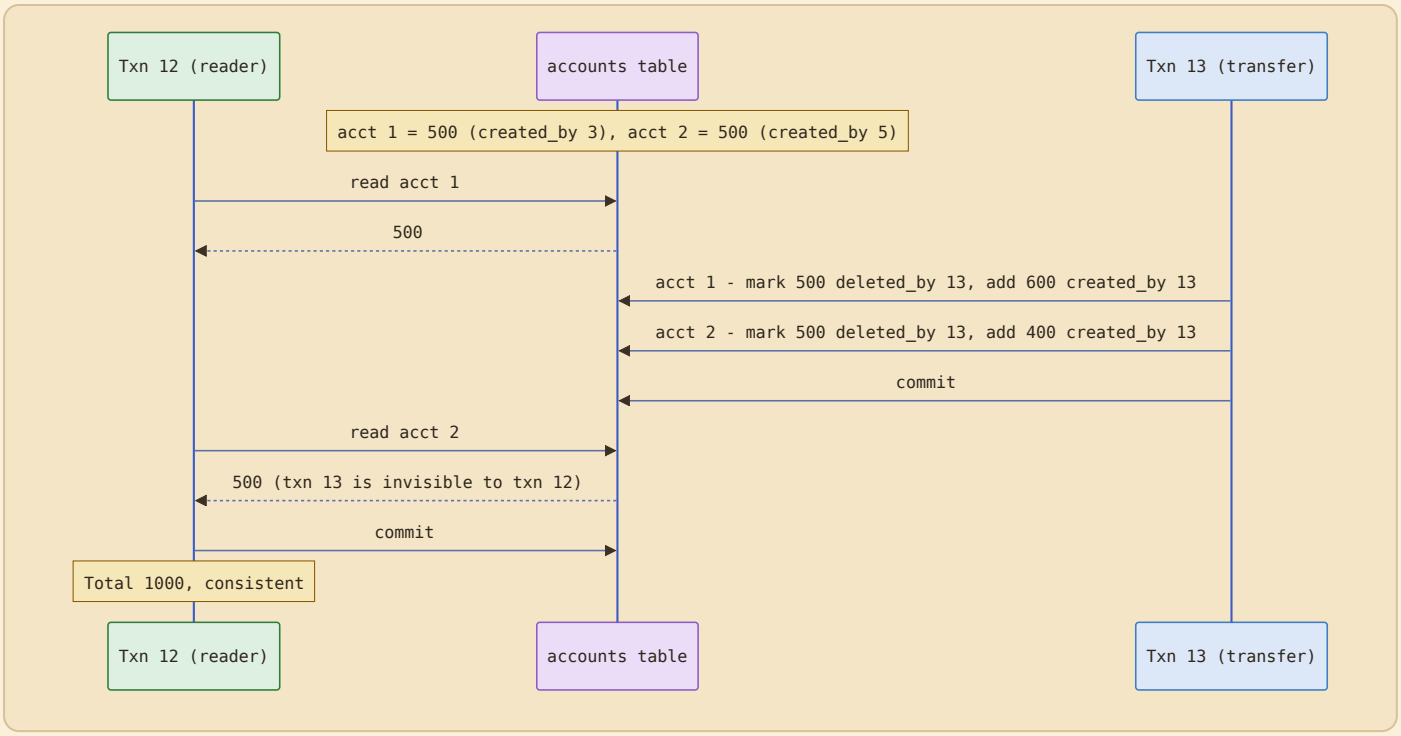
Storage engines that support snapshot isolation usually implement read committed with MVCC too.

How PostgreSQL does it (Figure 7-7; others are similar).

1. When a transaction starts it gets a unique, **always-increasing transaction ID (txid).**
2. Anything it writes is **tagged with its txid**.
3. Each row has a `created_by` field (the txid that inserted it) and a `deleted_by` field (initially empty).
4. **Delete:** the row is not removed; its `deleted_by` is set to the deleting txid.
5. **Update:** translated into a **delete plus a create**. In Figure 7-7, transaction 13 takes $100 from account 2, so the table now holds two rows for account 2: the **$500** row with `deleted_by = 13`, and a new **$400** row with `created_by = 13`.
6. Later, once no transaction can possibly see the deleted data, a **garbage collection** process removes those rows and frees space.

> **Footnote:** PostgreSQL txids are **32-bit**, so they wrap around after about **4 billion** transactions. PostgreSQL's **vacuum** process does cleanup so that the overflow does not affect data.

Figure 7-7 (redrawn): MVCC in PostgreSQL, an update creates a new row version and marks the old one deleted



Visibility rules for observing a consistent snapshot

When a transaction reads, txids decide what it may see. The rules:

1. At the **start** of a transaction, the database lists every other transaction still **in progress**. Their writes are **ignored**, even if they commit later.
2. Writes by **aborted** transactions are ignored.
3. Writes by transactions with a **later txid** (started after this one) are ignored, **whether or not** they committed.
4. **Everything else** is visible.

These apply to both creations and deletions. In Figure 7-7, transaction 12 reading account 2 sees **$500**: the deletion of the $500 row was done by transaction 13 (rule 3 hides it), and the $400 row created by 13 is also hidden by rule 3.

Equivalently, an object is **visible** if **both** hold:

- The transaction that **created** it had **already committed** when the reader started.
- The object is **not marked deleted**, or the deleter had **not yet committed** when the reader started.

A long-running transaction may keep reading versions that others have long since overwritten or deleted. Because values are never updated in place — a new version is created each time — the database can offer consistent snapshots at only a small cost.

Indexes and snapshot isolation

How do indexes work when there are many versions?

- **Approach 1:** the index points to **all versions** of an object, and an index lookup **filters out** versions invisible to the current transaction. When garbage collection removes an old version, its index entries go too. Many implementation details affect MVCC performance; for example, **PostgreSQL** avoids index updates when the versions of an object fit on the **same page**.

- **Approach 2 — append-only / copy-on-write B-trees:** used by **CouchDB, Datomic and LMDB**. Pages are never overwritten. Changing a page makes a **new copy**; its parents up to the **root** are copied and updated to point at the new children. Untouched pages are shared and stay immutable. Each write transaction (or batch) produces a **new root**, and each root *is* a consistent snapshot as of when it was made. No txid filtering is needed, because later writes cannot alter an existing tree — they only create new roots. The cost: you need background **compaction and garbage collection**.

Repeatable read and naming confusion

Snapshot isolation is useful, especially for read-only transactions, but vendors call it different things:

Database	Name used for snapshot isolation
Oracle	"serializable"
PostgreSQL, MySQL	"repeatable read"
IBM DB2	uses "repeatable read" to mean **serializability** (not snapshot isolation)

Why the mess?

- The **SQL standard** takes its isolation definitions from **System R (1975)**, before snapshot isolation was invented, so the standard has no concept of it.
- The standard defines **repeatable read**, which looks superficially like snapshot isolation. PostgreSQL and MySQL use that name because their snapshot isolation satisfies the standard's requirements, letting them claim compliance.
- The standard's definitions are themselves **flawed**: vague, imprecise, and more tied to particular implementations than a standard ought to be. So databases that all claim "repeatable read" give quite different guarantees.
- Research literature has a **formal** definition of repeatable read, but most implementations do not meet it.

The book's punchline: in effect, **no one really knows what "repeatable read" means.**

Preventing Lost Updates

Read committed and snapshot isolation mostly describe what a **read-only** transaction can see while writes are happening. Apart from dirty writes, we have not yet looked at **two transactions writing concurrently**. The most famous such conflict is the **lost update** (Figure 7-1).

What it is. It happens with a **read-modify-write cycle**: read a value, change it in app code, write it back. If two transactions do this concurrently, one change can vanish, since the second write was computed without seeing the first — the later write **clobbers** the earlier one.

Where it shows up:

- Bumping a counter, or changing an account balance.
- Editing part of a complex value, such as appending an item to a list inside a JSON document (parse, modify, write back).
- Two people editing a **wiki page** at once, each saving by sending the **whole page**, overwriting whatever is there.

Since it is so common, there are several solutions.

Atomic write operations

Many databases offer **atomic update operations** so you do not need a read-modify-write cycle in app code. When your logic can be expressed with them, they are usually the **best solution**. My own example:

```sql
UPDATE page_views SET hits = hits + 1 WHERE page = '/home';
```

- `UPDATE page_views` — the table holding counters.
- `SET hits = hits + 1` — the database reads and increments in one safe step.
- `WHERE page = '/home'` — only this page's counter.

This is concurrency-safe in most relational databases. Similarly, **MongoDB** has atomic operations for modifying part of a JSON document, and **Redis** has atomic operations on data structures like **priority queues**. Not everything fits — a wiki edit is arbitrary text editing — but when atomic operations fit, use them.

> **Footnote:** you *can* express text editing as a stream of atomic mutations, but it is fairly complicated (see "Automatic Conflict Resolution" in Chapter 5).

How they are implemented.

- Grab an **exclusive lock** on the object at read time, so other transactions cannot read it until the update has been applied. This is sometimes called **cursor stability**.
- Or run all atomic operations on a **single thread**.

Common pitfall. **ORM frameworks** make it easy to accidentally write unsafe read-modify-write code instead of using the database's atomic operations. Experts can avoid this, but it is a common source of subtle bugs that tests seldom reveal.

Explicit locking

If built-in atomic operations are not enough, the **application can lock** the objects it will update. It then does its read-modify-write, and any other transaction trying to read the same object waits until the first is done.

The book's example: a multiplayer game where several players can move the **same piece**. An atomic operation is not enough because the move must also be **checked against the game rules** — logic you cannot reasonably express in a query. So you lock the piece's row. My own version:

```sql
BEGIN TRANSACTION;

SELECT * FROM tokens
  WHERE label = 'knight' AND match_id = 51
  FOR UPDATE;

-- app checks the move is legal, then:
UPDATE tokens SET square = 'e5' WHERE id = 908;

COMMIT;
```

- `BEGIN TRANSACTION` — start the transaction.
- `SELECT ... FOR UPDATE` — read the knight's row **and lock it**; `FOR UPDATE` tells the DB to lock every row this query returns.
- The comment — app code validates the move while holding the lock.
- `UPDATE tokens SET square = 'e5'` — write the new position.
- `COMMIT` — make it permanent and release the lock.

It works, but you must think carefully about your logic; forgetting a lock somewhere reintroduces the race.

Automatically detecting lost updates

Atomic operations and locks prevent lost updates by making read-modify-write cycles happen **one at a time**. The alternative: allow them to **run concurrently**; when the transaction manager **notices** a lost update, it **aborts** the guilty transaction, which then retries.

Advantage: databases can do this check **efficiently alongside snapshot isolation**.

Database / level	Detects lost updates automatically?
PostgreSQL repeatable read	Yes
Oracle serializable	Yes
SQL Server snapshot isolation	Yes
MySQL/InnoDB repeatable read	**No**

Some authors argue a database must prevent lost updates to count as snapshot isolation — by that definition, **MySQL does not provide snapshot isolation**.

Why this feature is great: it needs **no special code** in the app. You might forget a lock or an atomic operation, but detection happens automatically, so it is **less error-prone**.

Compare-and-set

Databases **without** transactions sometimes offer an atomic **compare-and-set**: update only if the value is **unchanged since you read it**; otherwise the update does nothing and you must retry the cycle.

My own example, for a shared note:

```
-- Safe or not depends on the database implementation
UPDATE notes SET body = 'edited text'
  WHERE id = 77 AND body = 'text I originally read';
```

- `UPDATE notes SET body = 'edited text'` — the new value.
- `WHERE id = 77` — the target note.
- `AND body = 'text I originally read'` — the compare part: only apply if nobody changed it.
- After running it, check how many rows changed; if zero, someone else edited it — re-read and retry.

Warning: if the database lets the `WHERE` clause read from an **old snapshot**, the condition can still be true while another write is in flight, so lost updates are **not** prevented. Check that your database's compare-and-set is actually safe.

Conflict resolution and replication

Replication adds a new dimension: copies live on several nodes and can be changed concurrently on different nodes.

- **Locks and compare-and-set assume a single up-to-date copy.** Multi-leader and leaderless replication allow concurrent writes that replicate asynchronously, so there is no single up-to-date copy and these techniques **do not apply**. (Revisited under linearizability in Chapter 9.)
- **Common approach instead:** let concurrent writes create multiple conflicting versions (**siblings**) and merge them afterwards using app code or special data structures (Chapter 5).
- **Atomic operations can work well** if they are **commutative** — you can apply them in any order on different replicas and get the same result. Incrementing a counter and adding to a set are examples. **Riak 2.0 datatypes** build on this idea: concurrent updates from different clients are merged automatically so that nothing is lost across replicas.
- **Last write wins (LWW)** is **prone to lost updates** — and unfortunately it is the **default** in many replicated databases.

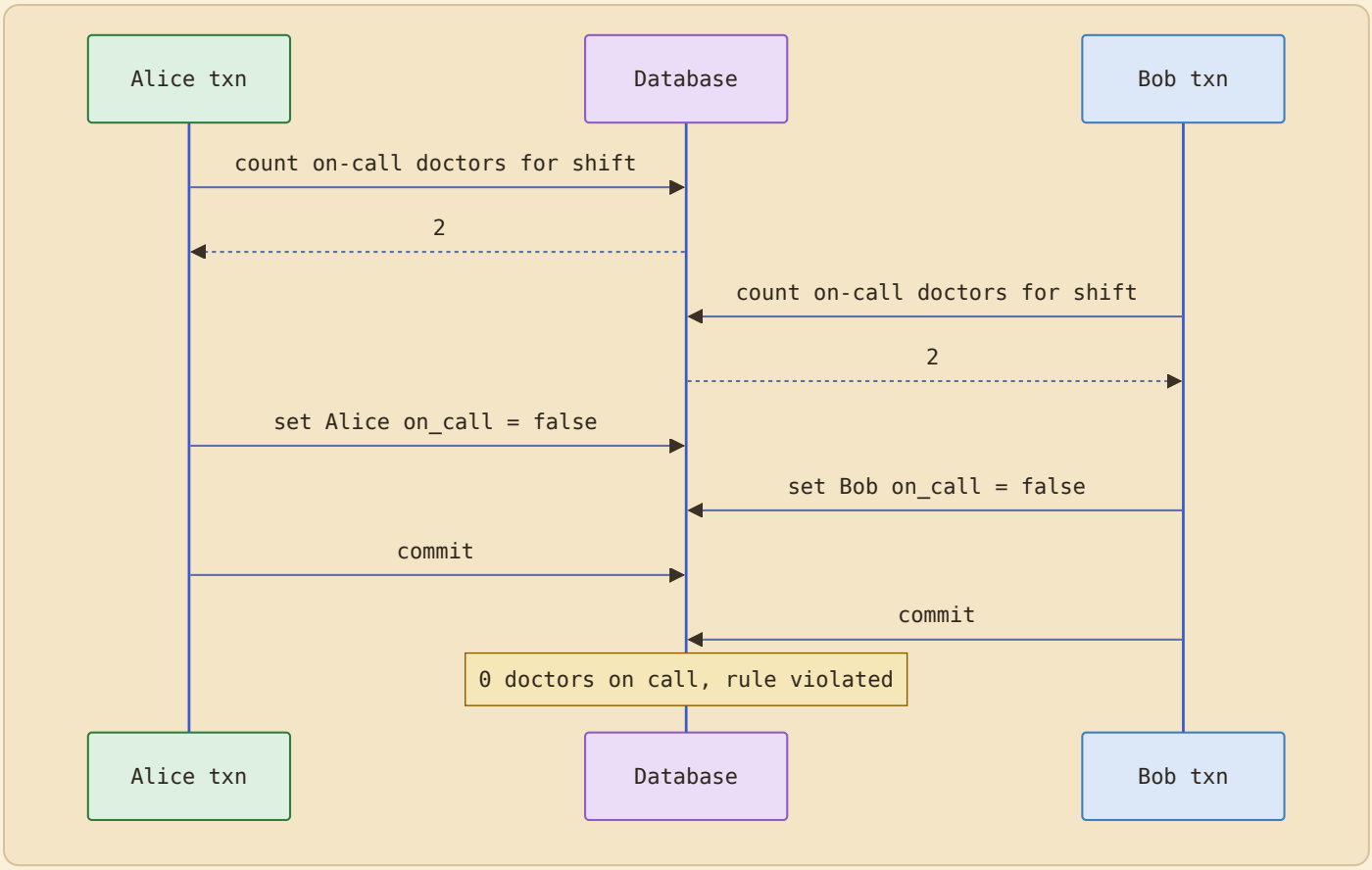
Write Skew and Phantoms

Dirty writes and lost updates are races where transactions write the **same** object. There are subtler conflicts.

The book's hospital example (Figure 7-8). An app lets doctors manage **on-call shifts**. The hospital wants several doctors on call but **must have at least one**. A doctor may give up a shift if at least one colleague remains on call. **Alice and Bob** are the only two on call for a shift; both feel ill and click "go off call" at almost the same moment.

1. Each transaction checks how many doctors are on call. Under **snapshot isolation**, **both see 2**.
2. Both conclude it is safe for one doctor to leave.
3. Alice's transaction updates **her** record; Bob's updates **his**.
4. Both commit. Now **zero** doctors are on call — the rule is broken.

Figure 7-8 (redrawn): Alice and Bob both see two doctors on call, both go off call, leaving nobody



Characterizing write skew

What it is. This anomaly is **write skew**. It is not a dirty write or a lost update, because the two transactions update **different** objects (Alice's row and Bob's row). The conflict is less obvious, but it is clearly a race: run one after the other, the second doctor would have been refused.

A useful mental model: write skew is a **generalisation of lost update**. It can arise whenever two transactions **read an overlapping set of objects** and each then **modifies some of those objects** (maybe not the same ones). In the special case where both modify an identical object, the result is instead a dirty write or a lost update, depending on timing.

Your options are narrower than for lost updates:

Technique	Helps with write skew?
Atomic single-object operations	**No** — several objects are involved.
Automatic lost-update detection in snapshot isolation	**No** — not detected in PostgreSQL repeatable read, MySQL/InnoDB repeatable read, Oracle serializable, or SQL Server snapshot isolation. Only **true serializable** isolation prevents it automatically.
Database constraints	Only if you can express a **multi-object** constraint ("at least one on call"). Most databases lack built-in support, but **triggers** or **materialized views** may work, depending on the database.
Explicitly locking the rows the transaction depends on	**Yes** — the second-best option if serializable is unavailable.

My own version of the locking fix for an on-call rota:

```
BEGIN TRANSACTION;

SELECT * FROM rota
  WHERE available = true
  AND slot_id = 42 FOR UPDATE;

UPDATE rota
  SET available = false
  WHERE engineer = 'Priya'
  AND slot_id = 42;

COMMIT;
```

- `SELECT ... WHERE available = true AND slot_id = 42 FOR UPDATE` — read *and lock* every engineer currently available in slot 42. A second transaction doing the same check must wait.
- The app checks the count is at least 2 before continuing.
- `UPDATE rota SET available = false WHERE engineer = 'Priya'` — take one person off.
- `COMMIT` — release the locks; the waiting transaction now sees the new count.

More examples of write skew

Once you know the pattern, you see it everywhere:

- **Meeting room booking.** You must never have two bookings for the same room at the same time. The app checks for overlapping bookings, and if none, inserts one. Snapshot isolation does **not** stop another user inserting a clashing booking concurrently; you need serializable isolation. My own illustration:

  ```
  BEGIN TRANSACTION;

  -- Any booking for desk 9 that overlaps 09:00-11:00?
  SELECT COUNT(*) FROM desk_bookings
    WHERE desk_id = 9 AND
      ends_at > '2024-03-04 09:00' AND starts_at < '2024-03-04 11:00';

  -- If the count was zero:
  INSERT INTO desk_bookings (desk_id, starts_at, ends_at, person_id)
    VALUES (9, '2024-03-04 09:00', '2024-03-04 11:00', 314);

  COMMIT;
  ```

    - The `SELECT COUNT(*)` finds overlaps: an existing booking overlaps if it ends after our start *and* starts before our end.
    - The `INSERT` adds the booking if nothing overlapped.
    - Two concurrent transactions can both count zero and both insert — a double booking.

    > **Footnote:** PostgreSQL's **range types** make this more elegant, but they are not widely supported elsewhere.

- **Multiplayer game.** The lock in the earlier example stops two players moving the **same** piece, but not moving **two different pieces to the same square**, or other rule-breaking combinations. A unique constraint might help for some rules; otherwise you are exposed to write skew.
- **Claiming a username.** Two people sign up with the same name at once; each transaction checks it is free and then creates the account. Not safe under snapshot isolation. **Fix:** a **unique constraint** — the second insert violates it and is aborted.
- **Preventing double-spending.** A service lets users spend money or points and must stop them overspending. One approach: insert a tentative spending item, list all items in the account, and check the total stays positive. With write skew, two spending items inserted concurrently can push the balance negative without either transaction noticing.

Phantoms causing write skew

All these examples share a shape:

1. A `SELECT` checks a **requirement** by searching for rows matching a condition (at least two doctors on call; no overlapping booking; the square is empty; the username is free; enough money remains).
2. Based on the result, app code **decides** whether to proceed or report an error and abort.
3. If proceeding, it **writes** (`INSERT`, `UPDATE` or `DELETE`) and commits. That write **changes the precondition** of step 2 — rerunning the step 1 query after commit would now give a different answer (one fewer doctor, room now booked, square occupied, name taken, less money).

The steps can be reordered — e.g., write first, then query, then decide to commit or abort.

Why locking does not always work. In the doctors case, the row changed in step 3 was **among the rows the step 1 query found**, so `SELECT FOR UPDATE` could lock it. The other four examples check for the **absence** of rows, and the write **adds** a matching row. If step 1 returns nothing, `SELECT FOR UPDATE` has **nothing to lock**.

Definition. A **phantom** is the effect where one transaction's write alters what another transaction's **search query** would return. Snapshot isolation already protects **read-only** queries from phantoms; the trouble is **read-write** transactions, where phantoms can produce particularly nasty write skew.

Materializing conflicts

If phantoms are a problem because there is **no object to lock**, why not **create one artificially**?

How it works (meeting rooms).

1. Create a table of **time slots × rooms** — each row is one room for one period (say **15 minutes**).
2. **Pre-populate** it for all combinations for some time ahead (e.g., the **next six months**).
3. A booking transaction first locks (`SELECT FOR UPDATE`) the slot rows for its room and time range.
4. With those locks held, it checks for overlaps and inserts the booking as before.

The slots table stores **no booking data** — it is purely a set of locks that stops concurrent changes to the same room and time.

This is **materializing conflicts**: turning a phantom into a lock conflict on concrete rows. **Downsides:** working out how to materialize conflicts is hard and error-prone, and letting concurrency control

leak into your data model is ugly. Treat it as a **last resort**; a serializable isolation level is usually much better.

Serializability

The chapter so far has shown race conditions that read committed and snapshot isolation prevent, and others (write skew, phantoms) they do not. The situation is grim:

- Isolation levels are **hard to understand** and **implemented inconsistently** across databases (e.g., "repeatable read" varies a lot).
- Looking at app code, it is **hard to tell** whether it is safe at a given level, especially in large apps where you do not know everything running concurrently.
- There are **no good tools** for detecting race conditions. Static analysis might help in principle, but research has not reached practice. Testing is hard because the problems are **nondeterministic**.

This has been true since weak isolation levels appeared in the **1970s**. Researchers' answer has always been: **use serializable isolation**.

What it guarantees. **Serializable isolation** is usually considered the **strongest** level. Even if transactions run in parallel, the outcome equals running them **one at a time** in some order. So if each transaction is correct on its own, it stays correct under concurrency — the database prevents **all** race conditions.

So why doesn't everyone use it? Because of how it has to be implemented. Most serializable databases today use one of three techniques:

1. **Actual serial execution** — literally one at a time.
2. **Two-phase locking (2PL)** — for decades the only viable option.
3. **Optimistic concurrency control**, such as **serializable snapshot isolation (SSI)**.

This chapter treats them on a single node; Chapter 9 generalises to multiple nodes.

Actual Serial Execution

The idea. Remove concurrency entirely: run **one transaction at a time, in order, on a single thread**. Conflicts cannot happen, so isolation is serializable **by definition**.

Why only recently? Designers decided a single-threaded transaction loop was feasible only around **2007**. For the previous **30 years**, multi-threading was considered essential for performance. Two things changed:

- **RAM got cheap** enough that for many uses the whole **active dataset fits in memory** (Chapter 3). With no waiting for disk, transactions run much faster.
- Designers realised **OLTP transactions are usually short**, with only a few reads and writes. Long analytic queries are typically read-only and can run on a **consistent snapshot** (snapshot isolation) **outside** the serial loop.

Who does it: **VoltDB/H-Store, Redis and Datomic**. A single-threaded system can sometimes **beat** a concurrent one because it skips locking overhead. But its throughput is capped at **one CPU core**, and transactions must be structured differently to use that thread well.

Encapsulating transactions in stored procedures

History. Early database designers hoped a transaction could cover a **whole user flow**. Booking a flight, for instance, means searching routes, fares and seats; choosing an itinerary; booking each leg; entering passenger details; paying. Making all that one atomic transaction seemed appealing.

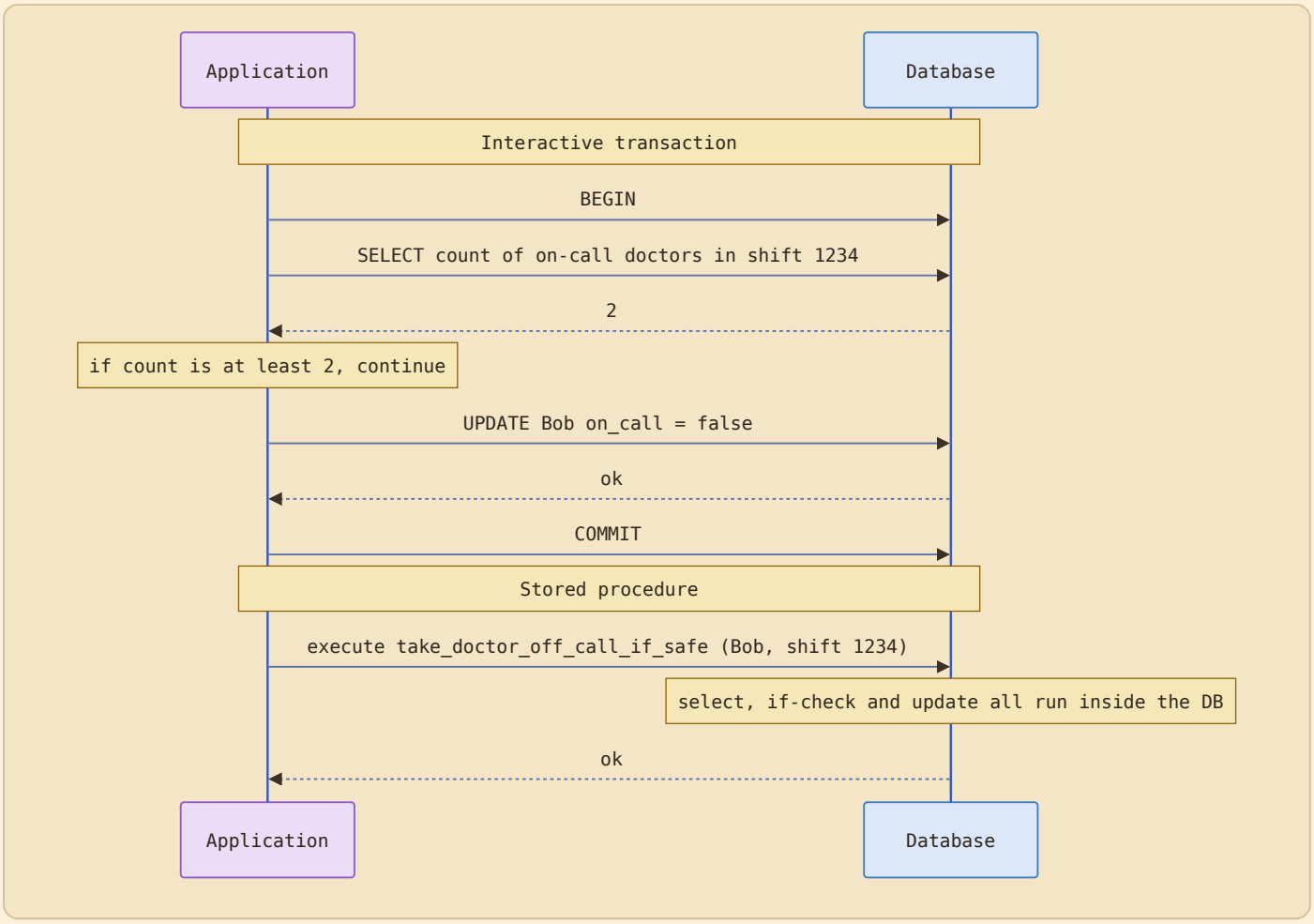
Problem: **humans are slow.** If a transaction waits on user input, the database must juggle huge numbers of mostly **idle** concurrent transactions, which most cannot do efficiently. So nearly all OLTP

apps keep transactions short and never wait for a user inside one. On the web, a transaction **commits within one HTTP request**; a new request starts a new transaction.

Still interactive, though. Even without the human, transactions are usually executed **one statement at a time** in an interactive client/server style: the app sends a query, reads the result, maybe sends another based on it, and so on — lots of **network round trips** between the app server and the database server. If the database allowed only one transaction at a time, it would sit idle much of the time, **waiting** on the app's next query, and throughput would be terrible. That is why such databases must run many transactions concurrently.

The fix. Single-threaded serial systems **forbid interactive multi-statement transactions**. The app must send the **entire transaction code up front** as a **stored procedure**. If all needed data is in memory, the procedure runs very fast with no network or disk waits.

Figure 7-9 (redrawn): interactive transaction with many round trips vs a stored procedure sent in one call (using the on-call example)



Pros and cons of stored procedures

Stored procedures have existed in relational databases for a long time and were added to the SQL standard as **SQL/PSM** in **1999**. They earned a bad reputation because:

- **Each vendor has its own language — Oracle: PL/SQL, SQL Server: T-SQL, PostgreSQL: PL/pgSQL**, etc. These have not kept pace with general-purpose languages, look dated, and lack good library ecosystems.
- **Code inside the database is hard to manage**: harder to debug, awkward for version control and deployment, trickier to test, and hard to hook into metrics/monitoring.
- **Databases are more performance-sensitive** than app servers, since one database often serves many app servers. A badly written procedure (heavy on memory or CPU) causes more damage than the same bad code in an app server.

These can be overcome. Modern systems use **general-purpose languages**:

System	Stored procedure language
VoltDB	Java or Groovy
Datomic	Java or Clojure
Redis	Lua

With stored procedures and **in-memory** data, single-threaded execution works: no I/O waits and no concurrency-control overhead, so throughput on one thread is quite good.

Replication with procedures. **VoltDB** replicates by running the **same stored procedure on each replica** instead of copying the resulting writes. So procedures must be **deterministic** (same result on every node). If a transaction needs the current date/time, it must use special **deterministic APIs**.

Partitioning

Problem. Serial execution simplifies concurrency but limits throughput to **one CPU core on one machine**. Read-only transactions can run elsewhere on snapshots, but **write-heavy** apps can bottleneck on the single thread.

Solution: partition the data (Chapter 6), which **VoltDB** supports.

1. Find a partitioning where **each transaction reads and writes within a single partition**.
2. Give each partition its **own single-threaded transaction processor**, independent of the others.
3. Assign each CPU core its own partition → throughput **scales linearly** with cores.

Catch — cross-partition transactions. A transaction touching several partitions must be **coordinated** across all of them, with the stored procedure running **in lock-step** on each to keep serializability. That overhead makes them **vastly slower**: VoltDB reports a rate of roughly **1,000 writes per second** for cross-partition transactions, which is several orders of magnitude lower than it achieves within one partition, and more machines **do not** increase it.

Whether transactions can stay within one partition depends on the **data's structure**. Simple key-value data partitions easily; data with **multiple secondary indexes** tends to need lots of cross-partition coordination (Chapter 6).

> **Summary of serial execution**

Serial execution gives serializable isolation **within certain limits**:

- **Every transaction must be small and fast** — one slow transaction stalls everything.
- **The active dataset must fit in memory.** Rarely used data could live on disk, but if a single-threaded transaction needed it, everything would slow to a crawl.
- **Write throughput must fit on one CPU core**, or transactions must be partitionable **without** cross-partition coordination.
- **Cross-partition transactions** are possible, but only up to a hard limit.

> **Anti-caching (footnote).** If a transaction needs data not in memory, a good approach may be to **abort** it, **load** the data into memory in the background while other transactions continue, then **restart** it once the data is there.

Two-Phase Locking (2PL)

For about **30 years**, **two-phase locking (2PL)** was the only widely used serializability algorithm.

> **2PL is not 2PC.** Two-phase locking and two-phase commit (2PC) sound alike but are completely different. 2PC is covered in Chapter 9.

> **Footnote:** the variant described here is also known as **strong strict two-phase locking (SS2PL)**, a name that separates it from other flavours of 2PL.

What it is. We already saw locks preventing dirty writes: a second writer waits until the first transaction finishes. 2PL makes the lock rules **much stricter**. Many transactions may **read** the same object together if nobody is writing it, but **writing** (modify or delete) needs **exclusive** access:

- If **A has read** an object and **B wants to write** it, B waits until A commits or aborts (so B cannot change it behind A's back).
- If **A has written** an object and **B wants to read** it, B waits until A commits or aborts (reading an old version, as in Figure 7-1, is **not** allowed under 2PL).

So in 2PL, **writers block writers *and* readers, and readers block writers.** Snapshot isolation promises the opposite (no reader/writer blocking in either direction), and that contrast is the essential difference between the two. In return, 2PL is **serializable**, so it prevents **all** the races above, including lost updates and write skew.

Implementation of two-phase locking

Who uses it: **MySQL (InnoDB)** and **SQL Server** use 2PL for their **serializable** level; **DB2** uses it for its level called **repeatable read**.

How it works. Each object has a lock that can be in **shared** or **exclusive** mode.

1. **To read**, acquire the lock in **shared** mode. Many transactions can share it, but if someone holds it **exclusively**, readers wait.
2. **To write**, acquire it in **exclusive** mode. No one else may hold it in any mode, so if any lock exists, the writer waits.
3. If a transaction **reads then writes** an object, it can **upgrade** its shared lock to exclusive (same as acquiring exclusive directly).
4. Once acquired, a lock is **held until the transaction ends** (commit or abort). This gives the name: **phase one** — acquiring locks while running; **phase two** — releasing them all at the end.

Deadlocks. With so many locks, it is easy for A to wait on B while B waits on A — a **deadlock**. The database **detects** deadlocks automatically and **aborts one** transaction so the rest can proceed. The aborted one must be **retried by the app**.

Performance of two-phase locking

The big downside — and why not everyone has used it since the 1970s — is **performance**: throughput and query response times are **significantly worse** than with weak isolation.

Reasons:

- Overhead of acquiring and releasing many locks.
- More importantly, **reduced concurrency**: whenever two transactions might race in any way, one must **wait** for the other.
- Traditional relational databases **do not cap transaction duration** (they were designed for interactive apps waiting on humans), so a wait has **no upper bound**. Even with short transactions, a **queue** can form on a popular object, so one transaction may wait for several others.
- So 2PL databases can have **unstable latencies** and be very slow at **high percentiles** when there is contention. One slow transaction, or one that touches lots of data and grabs many locks, can bring the rest of the system to a **halt** — bad when robust operation matters.

- **Deadlocks** can happen under lock-based read committed too, but are **much more frequent** under 2PL serializable (depending on access patterns). Each deadlock abort means redoing the work; frequent deadlocks waste a lot of effort.

Predicate locks

The gap. The lock description above skips **phantoms** — one transaction changing the results of another's **search query**. A serializable database **must** prevent them.

Meeting rooms again. If one transaction has searched for bookings of a room in some time window, another transaction must not concurrently **insert or update** a booking for that room and overlapping time. (Other rooms, or the same room at a non-overlapping time, are fine.)

The concept: a predicate lock. Like a shared/exclusive lock, but instead of belonging to one object, it belongs to **all objects matching a search condition** — for example (my own query):

```
SELECT * FROM desk_bookings
  WHERE desk_id = 9 AND
    ends_at > '2024-03-04 09:00' AND
    starts_at < '2024-03-04 11:00';
```

- The whole `WHERE` condition is the **predicate** that gets locked: desk 9, overlapping 09:00–11:00.

How it restricts access:

1. If **A wants to read** objects matching a condition, it takes a **shared-mode predicate lock** on that condition. If **B holds an exclusive lock** on any matching object, A waits.
2. Before A may **insert, update or delete** anything, it must check the object's **previous and new values** against all existing predicate locks. If one held by B matches, A waits for B to commit or abort.

Key idea: a predicate lock covers objects that **do not exist yet** but might be added (phantoms). **2PL plus predicate locks** prevents all write skew and other races, so it is truly **serializable**.

Index-range locks

Problem: predicate locks **perform poorly** — with many active locks, checking for matches is slow. So most 2PL databases use **index-range locking** (also called **next-key locking**), a **simplified approximation**.

Why approximation is safe: you can widen a predicate to match **more** objects. A lock on "room 123, noon–1pm" can be approximated by "room 123 at any time" or "all rooms, noon–1pm". Any write matching the original also matches the wider version.

How it works. The bookings table probably has an index on `room_id` and/or on `start_time`/`end_time` (otherwise the query would be very slow):

- If the database used the `room_id` **index** to find room 123's bookings, it attaches a **shared lock to that index entry**, meaning "someone searched room 123".
- If it used a **time index**, it puts a shared lock on a **span of that index's values**, meaning "someone searched bookings overlapping noon–1pm on **January 1, 2018**".

Either way, an approximation of the search is attached to an index. Any transaction that wants to insert/update/delete a booking for that room or an overlapping time must **update the same part of the index**, meets the shared lock, and **waits**.

Trade-off. This protects against phantoms and write skew. It is **less precise** than predicate locks (may lock more than strictly needed) but has **much lower overhead** — a good compromise.

Fallback. If there is **no suitable index**, the database can take a **shared lock on the whole table**. Safe, but bad for performance, since it blocks all other writers to that table.

Serializable Snapshot Isolation (SSI)

The picture so far is bleak: serializability implementations that **don't perform well** (2PL) or **don't scale well** (serial execution), versus weak levels that perform well but allow races (lost updates, write skew, phantoms). Must we choose?

Maybe not. **Serializable snapshot isolation (SSI)** is **very promising**: full serializability with only a **small performance penalty** over snapshot isolation. It is fairly new: first published in **2008**, it was the topic of **Michael Cahill's** PhD thesis.

Where it is used: single-node — the **serializable** level of **PostgreSQL since version 9.1**; distributed — **FoundationDB** uses a similar algorithm. Being young, it is still proving itself in practice, but it may become fast enough to be the **default** in the future.

Pessimistic versus optimistic concurrency control

	Pessimistic	**Optimistic**
Idea	If anything *might* go wrong (someone else holds a lock), **wait** until safe	**Carry on**, hoping all is well; at **commit**, check whether isolation was violated and **abort** if so
Analogy	**Mutual exclusion** in multi-threaded programming	"Ask forgiveness, not permission"
Examples	2PL; serial execution (extremely pessimistic)	SSI
Weakness	Waiting and reduced concurrency	Performs badly under **high contention** — many aborts; retries add load when near max throughput
When it does well	Serial execution: when every transaction is tiny and fast. (For 2PL the book mainly stresses the costs: blocking, unstable latency, deadlocks.)	With **spare capacity** and **low contention**, usually **faster** than pessimistic methods

Serial execution as extreme pessimism: it is like every transaction holding an exclusive lock on the whole database (or partition) for its duration. It compensates by making transactions **very fast**, so the "lock" is held only briefly.

Optimistic concurrency control is an old idea, debated for a long time.

Reducing contention: **commutative atomic operations** help. If many transactions increment a counter, the order does not matter (as long as the counter is not also read in the same transaction), so they can all apply without conflicting.

What is new in SSI: it is built on **snapshot isolation** — all reads come from a consistent snapshot. That is the main difference from earlier optimistic techniques. On top of it, SSI adds an algorithm to **detect serialization conflicts among writes** and decide **which transactions to abort**.

Decisions based on an outdated premise

Recall the write-skew pattern: a transaction reads data, looks at the result, and **acts** (writes) based on it. Under snapshot isolation, by commit time that result may be **out of date**.

The transaction is acting on a **premise** — a fact true at the start (e.g., "two doctors are on call"). By commit time the premise may be **false**.

The database cannot see how app logic uses a query result. To be safe, it must assume **any change to the query result (premise) may invalidate the transaction's writes** — i.e., there may be a **causal dependency** between reads and writes. For serializability, the database has to **spot** any transaction that might have relied on a premise that is no longer true, and **abort** it.

How can it tell a query result might have changed? Two cases:

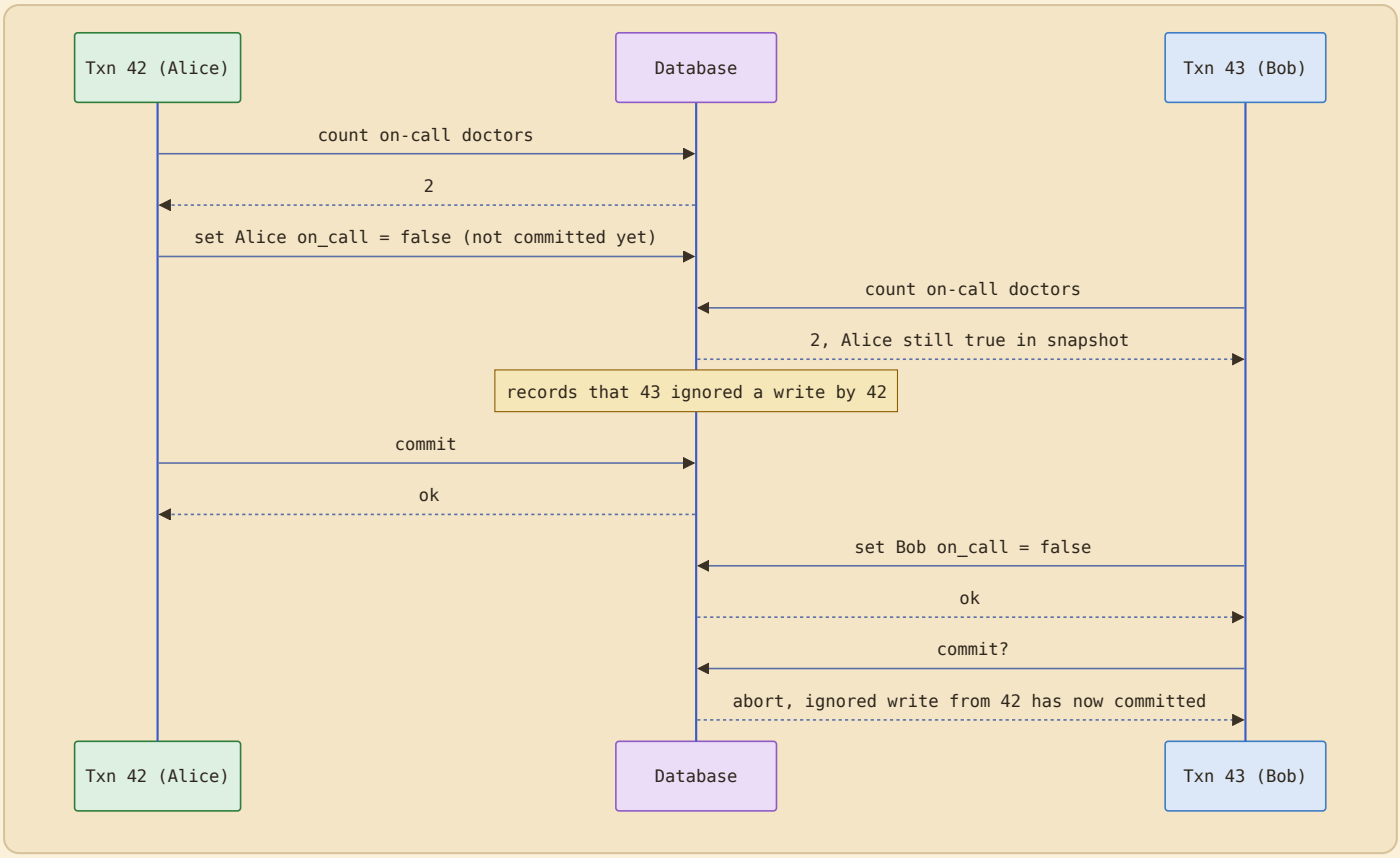
1. **Reads of a stale MVCC version** — an uncommitted write happened **before** the read.

2. **Writes that affect earlier reads** — the write happens **after** the read.

Detecting stale MVCC reads

The situation (Figure 7-10). Under MVCC snapshots, a transaction ignores writes by transactions that had not committed when its snapshot was taken. Transaction **43** sees Alice with `on_call = true` because transaction **42** (which changed Alice's status) has not committed. But by the time 43 wants to commit, **42 has committed** — the write 43 ignored is now real, and 43's premise is false.

Figure 7-10 (redrawn): txn 43 read a value that txn 42 had changed but not committed, so 43 aborts at commit. In the book's table, shift 1234 holds two versions of Alice (on_call true, created_by 1, deleted_by 42; and on_call false, created_by 42), plus Bob (true) and Carol (false); after the abort, 43 retries.



How SSI detects it:

1. Whenever MVCC visibility rules cause a transaction to **skip over** someone else's write, the database **records** that fact.
2. When the transaction **commits**, the database checks whether any of those ignored writes **have since committed**.
3. If so, it **aborts** the transaction.

Why wait until commit instead of aborting at the stale read?

- If 43 is **read-only**, it never needs aborting — no write, so no write skew risk — and at read time the database cannot know whether 43 will write later.
- Transaction 42 might still **abort**, or still be **uncommitted** when 43 commits, so the read may not be stale after all.
- By not aborting unnecessarily, SSI keeps snapshot isolation's big benefit: **long-running reads** on a consistent snapshot still work.

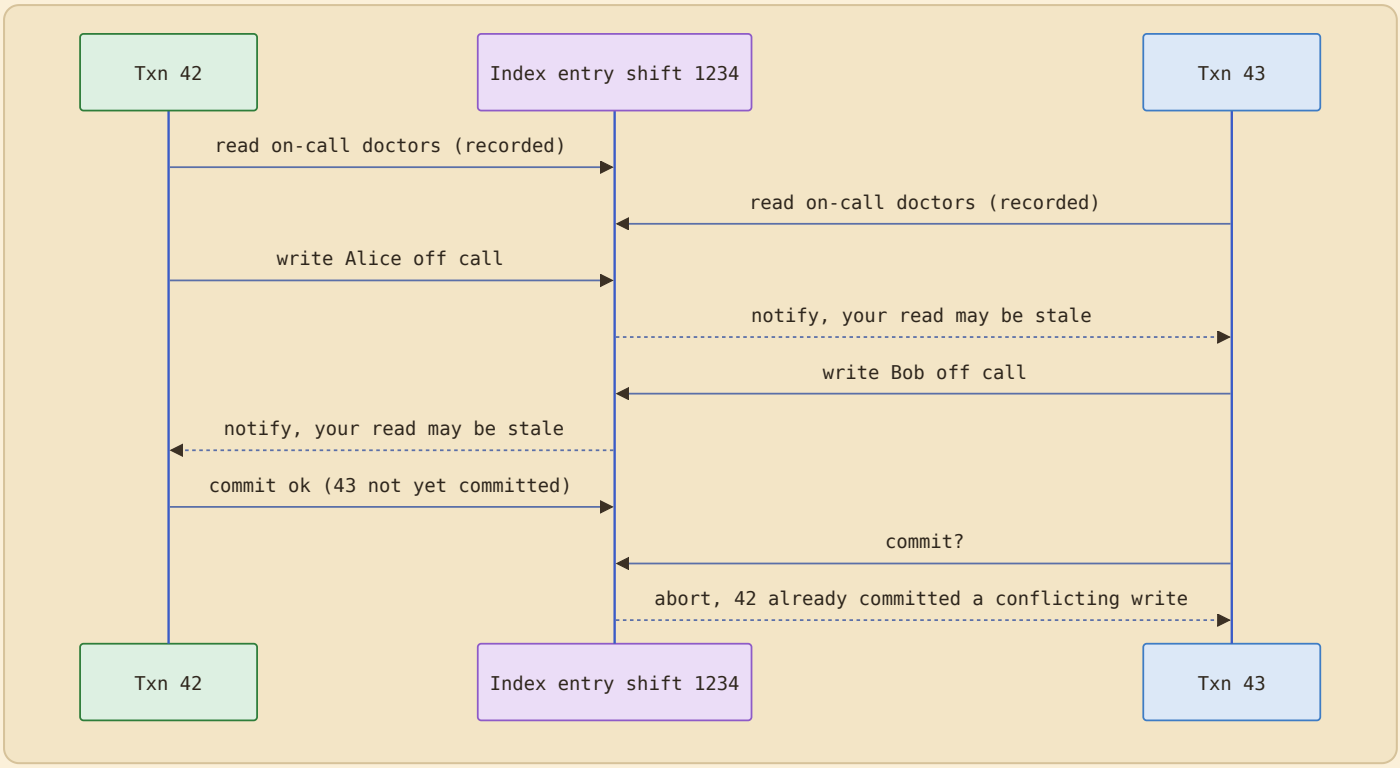
Detecting writes that affect prior reads

The situation (Figure 7-11): another transaction modifies data **after** it was read.

How SSI detects it: it borrows the idea of **index-range locks** from 2PL, but these locks **do not block**.

1. Transactions 42 and 43 each query the on-call doctors for **shift 1234**. If `shift_id` is indexed, the database can note on **index entry 1234** that 42 and 43 **read** it. (With no index, it can track this at the **table** level.)
2. These notes are temporary: after a transaction ends and **every transaction that ran concurrently with it has also ended**, its read records can be dropped.
3. When a transaction **writes**, it looks in the indexes for other transactions that **recently read** the affected data. This resembles taking a write lock on the key range, but instead of blocking, it acts as a **tripwire**: it **notifies** those readers that their data may be out of date.
4. In Figure 7-11, 43 notifies 42 that its read is outdated, and 42 notifies 43 likewise.
5. **42 commits first** and succeeds: 43's write affected 42, but 43 has not committed, so that write has not taken effect yet.
6. By the time **43 attempts to commit**, 42's conflicting write is already committed, so **43 is aborted**.

Figure 7-11 (redrawn): SSI records reads on index entry shift 1234 and aborts the second committer



Performance of serializable snapshot isolation

Tracking granularity trade-off. Tracking reads and writes **in detail** lets the database be precise about who must abort, but bookkeeping becomes expensive. **Coarser** tracking is faster but may abort **more transactions than necessary**.

Smarter analysis. Sometimes a transaction reading overwritten data is still fine — it can be proven the result is serializable anyway. **PostgreSQL** uses this theory to **reduce unnecessary aborts**.

Versus 2PL: the big win is that transactions **don't block waiting for locks**. Just like plain snapshot isolation, **readers and writers never wait for each other**, so latency is **more predictable and less variable**. **Read-only queries run on a consistent snapshot with no locks**, great for read-heavy workloads.

Versus serial execution: SSI is **not limited to one CPU core**. **FoundationDB** spreads conflict detection over **multiple machines**, reaching very high throughput. Even with data partitioned across machines, transactions can read and write **multiple partitions** while staying serializable.

Abort rate matters. The rate of aborts strongly affects SSI performance. A transaction that reads and writes over a **long time** is likely to hit conflicts and abort, so SSI needs **read-write**

transactions to be fairly short (long **read-only** ones are OK). Even so, slow transactions probably hurt SSI **less** than they hurt 2PL or serial execution.

Approach	How it serializes	Main strengths	Main weaknesses
Actual serial execution	One transaction at a time, single thread, stored procedures, in-memory	Simple; no lock overhead	Limited to one core per partition; transactions must be tiny; data must fit in RAM; cross-partition is slow (~1,000 writes/s in VoltDB)
Two-phase locking	Shared/exclusive locks held to end; predicate or index-range locks for phantoms	Decades of use; prevents all anomalies	Readers and writers block each other; unstable, high tail latency; frequent deadlocks
SSI	Snapshot isolation + detect conflicts, abort at commit	No blocking; predictable latency; scales across machines (FoundationDB)	Aborts under high contention; long read-write transactions likely to abort; relatively new

Chapter summary

(Book section: Summary)

- **Transactions are an abstraction**: they let an app pretend certain concurrency problems and certain hardware/software faults do not exist. Many kinds of error collapse into one simple outcome — **abort** — and the app just retries.
- Not every app needs them: very simple access patterns (read/write a single record) may be fine without. For complex access patterns, they **greatly shrink** the number of error cases to think about.
- Without transactions, crashes, network interruptions, power outages, full disks and unexpected concurrency can leave data inconsistent — e.g., **denormalized data drifting** from its source — and reasoning about complex interacting accesses gets very hard.
- The chapter focused on **concurrency control** and three widely used isolation levels: **read committed**, **snapshot isolation** (which some vendors label repeatable read), and **serializable**, characterised by these races:

Anomaly	What happens	What prevents it
Dirty reads	Reading another client's uncommitted writes	Read committed and stronger
Dirty writes	Overwriting another client's uncommitted write	Almost all transaction implementations
Read skew (nonrepeatable read)	Seeing different parts of the DB at different moments	Snapshot isolation, usually via MVCC
Lost updates	Two concurrent read-modify-write cycles; one overwrites the other	Some snapshot isolation implementations detect it automatically; others need a manual lock (`SELECT FOR UPDATE`)
Write skew	A decision based on a read whose premise is no longer true by write time	Only serializable isolation
Phantom reads	Another client's write changes the result of your search	Snapshot isolation handles simple phantoms; phantoms causing write skew need things like index-range locks

- Weak levels protect against some anomalies and leave the rest to you (e.g., explicit locking). **Only serializable isolation** covers all of them. Three ways to implement it:
  - **Literally serial execution** — simple and effective if each transaction is very fast and throughput fits on one core.
  - **Two-phase locking** — the standard for decades, but many apps avoid it for performance reasons.
  - **Serializable snapshot isolation (SSI)** — fairly new; optimistic, lets transactions run without blocking, checks at commit and aborts if not serializable.
- The examples used the relational model, but transactions are valuable **whatever the data model**.
- Everything here assumed a **single machine**; distributed transactions raise new challenges, covered in Chapters 8 and 9.

Interview angle

1. What does ACID mean, and which letter is misleading? *30-second answer:* Atomicity = abort on failure and undo everything (really "abortability"). Consistency = app-defined invariants hold; that is the app's responsibility, not the DB's, so the C barely belongs. Isolation = concurrent transactions don't interfere; the textbook ideal is serializability, but most DBs default to weaker levels. Durability = committed data survives crashes, via disk + WAL or replication — never perfect.

- *Follow-up: Is "ACID compliant" a meaningful claim?* Not really; implementations differ, especially on isolation. Oracle's "serializable" is actually snapshot isolation.
- *Follow-up: How is ACID atomicity different from atomic operations in threading?* Threading atomicity is about no one seeing a half-done state (concurrency). ACID atomicity is about undoing partial work after a fault.

2. Explain read committed vs snapshot isolation. *30-second answer:* Read committed stops dirty reads and dirty writes: row locks for writes, and readers get the last committed value. It still allows read skew — different parts read at different moments. Snapshot isolation gives each transaction a consistent snapshot from its start, implemented with MVCC (versions tagged with txids plus visibility rules). Readers never block writers and vice versa, which makes it great for backups and analytics.

- *Follow-up: Why not use read locks to prevent dirty reads?* A long write would stall many readers and spread slowdowns across the app.
- *Follow-up: What is "repeatable read"?* A standard name that vendors interpret differently — snapshot isolation in PostgreSQL/MySQL, serializability in DB2 (and Oracle calls its snapshot isolation "serializable").

3. What is a lost update and how do you prevent it? *30-second answer:* Two concurrent read-modify-write cycles; one write clobbers the other. Fixes: atomic updates (`SET x = x + 1`), explicit locks (`SELECT ... FOR UPDATE`), automatic detection (PostgreSQL repeatable read, Oracle serializable, SQL Server snapshot isolation — not MySQL/InnoDB repeatable read), compare-and-set, and in replicated systems commutative operations or sibling merging. LWW loses updates.

- *Follow-up: Is compare-and-set always safe?* No — if the WHERE clause reads from an old snapshot, it can pass despite a concurrent write.

4. What is write skew? Give an example and fix. *30-second answer:* Two transactions read the same data, each decides based on it, then write *different* objects; together they break an invariant. Example: two on-call doctors both go off call. Snapshot isolation doesn't catch it. Fixes: serializable isolation; or lock the rows you depend on with `FOR UPDATE`; or a multi-object constraint via triggers/materialized views; for "absence" checks (bookings, usernames), unique constraints, materializing conflicts, or serializable.

- *Follow-up: Why can't FOR UPDATE fix the booking case?* The check looks for rows that don't exist; there's nothing to lock. That's a phantom.

5. Compare the three ways to implement serializability. *30-second answer:* Serial execution — single thread, stored procedures, in-memory data; simple but capped at one core per partition and cross-partition is slow. 2PL — shared/exclusive locks held to commit, plus predicate or index-range locks; correct but slow, blocking, deadlock-prone, bad tail latency. SSI — snapshot isolation plus conflict detection, aborting at commit; no blocking, predictable latency, but aborts rise with contention and long read-write transactions.

- *Follow-up: 2PL vs 2PC?* Completely different: 2PL is for isolation on one DB; 2PC is atomic commit across nodes (Chapter 9).
- *Follow-up: When is optimistic worse than pessimistic?* Under high contention near max throughput — aborts and retries add load.

6. Why should apps retry aborted transactions, and when is retrying dangerous? *30-second answer:* Aborts exist to enable safe retries; many ORMs (ActiveRecord, Django) don't retry. Dangers: the commit may have succeeded but the ack was lost (double execution — dedupe needed); overload (use backoff and limits); permanent errors (pointless); external side effects like emails; the client dying mid-retry.

Self-check

1. **Why does the book suggest "abortability" as a better name for atomicity?** Because the defining feature is that a failed transaction can be aborted and all its writes discarded, making retry safe — not anything about concurrency.
2. **Which ACID property is the application's job rather than the database's, and why?** Consistency. Only the app knows its invariants; the DB can check some (foreign keys, uniqueness) but generally just stores what it is given.
3. **Under read committed, can a transaction see $900 instead of $1,000 during a transfer? What is this called and what fixes it?** Yes — read skew (nonrepeatable read). Snapshot isolation fixes it by reading everything from one consistent snapshot.
4. **In PostgreSQL's MVCC, what happens physically when a row is updated?** The old row gets `deleted_by = txid`, and a new row with `created_by = txid` is added. Garbage collection later removes invisible old versions.
5. **List the four visibility rules for a snapshot.** Ignore writes by transactions in progress at start; ignore aborted transactions' writes; ignore writes by transactions with a later txid; everything else is visible.
6. **Scenario:** your app adds a tag to a JSON array by reading the document, appending in code, and writing it back. Two users do it at once and one tag disappears. Name the anomaly and two fixes. Lost update. Use the database's atomic document operation (e.g., MongoDB's), or lock the document with `SELECT FOR UPDATE`; or rely on automatic lost-update detection if the DB has it.
7. **Why is materializing conflicts a last resort?** It is hard and error-prone to design, and it leaks concurrency control into the data model. Serializable isolation is usually better.
8. **Trade-off:** why does 2PL have unstable high-percentile latency? Readers and writers block each other, waits are unbounded, queues form on hot objects, one big or slow transaction can stall many, and deadlocks force redo work.
9. **Why does SSI wait until commit to abort a transaction that read a stale value?** The transaction may be read-only (no risk), and the other writer might still abort or stay uncommitted. Waiting avoids needless aborts and keeps long reads working.
10. **Scenario:** a workload has high write throughput, many secondary indexes and frequent cross-entity transactions. Would actual serial execution be a good fit? Probably not: throughput is limited to one core per partition, data with many secondary indexes tends to need cross-partition

coordination, and cross-partition transactions are slow (about 1,000 writes/s in VoltDB) and do not scale by adding machines. SSI or 2PL would fit better.

Chapter 8 — The Trouble with Distributed Systems

Book: Chapter 8, pp. 273–320

Overview

The question this chapter answers: once your software runs on many machines connected by a network, what exactly can go wrong, what can you rely on, and what can you *never* be sure of?

Earlier chapters already dealt with faults: replica failover and replication lag (Chapter 5) and concurrency control in transactions (Chapter 7). The book now says those chapters were still *too optimistic*. This chapter turns pessimism up to the maximum and adopts a Murphy's-law stance: **whatever can fail, will fail** — which, the book notes, experienced operators consider a reasonable assumption.

> **Footnote:** the one exception: the chapter assumes faults are **non-Byzantine** (nodes may fail but do not lie). Byzantine faults are discussed near the end.

Working across many computers is *fundamentally* different from writing software for one computer, mainly because there are so many new ways for things to fail. The chapter is deliberately gloomy; it surveys:

1. **Unreliable networks** — messages can be lost or delayed, and you cannot tell which.
2. **Unreliable clocks** — every machine's idea of the time is a bit (or a lot) wrong.
3. **Process pauses** — a program can freeze for seconds or minutes without noticing.
4. **Knowledge, truth and lies** — how a node can reason about the state of the system when it cannot trust its own view, and how we define "correct" algorithms using *system models*.

The goal of engineers is still to build systems that meet users' expectations *despite* all this. Chapter 9 presents algorithms that do so; this chapter first maps out the enemy.

The chapter's epigraph is a joke by Kyle Kingsbury (2013) that rewrites a pop lyric into a plea about laggy networks and data that may or may not be stored.

Everyday analogy: imagine coordinating a group project only by posting letters. Letters can get lost, arrive late, or arrive in any order. If a friend does not reply, you cannot tell whether your letter was lost, their reply was lost, or they are simply on holiday. Everyone's wristwatch shows a slightly different time. Sometimes a friend falls asleep for an hour mid-sentence and wakes up thinking nothing happened. That is distributed computing.

Key terms

Term	Plain-English meaning
Distributed system	Software running on several computers that talk only by sending messages over a network.
Deterministic	Same input, same operation → always the same result. Single computers aim for this.
Partial failure	Some parts of the system are broken while others work fine — unpredictably.
Nondeterministic	May sometimes work and sometimes fail, with no way to predict which.
High-performance computing (HPC)	Supercomputers with thousands of CPUs for heavy scientific jobs (weather, molecular dynamics).
Cloud computing	Multi-tenant datacenters of commodity machines on IP/Ethernet, elastic resources, metered billing.
Checkpoint	Saving the state of a long computation to durable storage so it can restart from there.
Commodity machines	Ordinary, cheap servers — cheaper per unit of performance but they fail more often.
RDMA	Remote direct memory access — one machine reading another's memory directly; used in supercomputers.
Clos topology	A datacenter network layout that gives high bisection bandwidth.
Rolling upgrade	Restarting nodes one at a time to upgrade software without downtime.
Error-correcting codes	Encoding that lets data survive a channel that occasionally flips bits.
IP / TCP	IP delivers packets unreliably; TCP builds a reliable, ordered stream on top of it.
Shared-nothing	Each machine has its own CPU, memory and disk; they cooperate only via the network.
Asynchronous packet network	A network that promises nothing about when (or whether) a packet arrives.
Timeout	Giving up waiting for a reply after a chosen time and assuming it will not come.
Network partition / netsplit	Part of the network cut off from the rest. The book mostly says "network fault" instead.
Chaos Monkey	Netflix's tool for deliberately causing failures to test resilience.
RST / FIN packet	TCP packets that reset or close a connection; an OS sends them if no process listens on a port.
ICMP Destination Unreachable	A router's reply saying an IP address cannot be reached.
Cascading failure	Overloaded nodes wrongly declared dead push load onto others, which then also fail.
Unbounded delay	No upper limit on how long a packet or request can take.

Term	Plain-English meaning
Network congestion	Packets waiting because a link is busy.
Queueing delay	Time spent waiting in a switch, OS or VM buffer before being handled.
Flow control (congestion avoidance, backpressure)	A sender slowing itself down to avoid overloading the network or receiver.
UDP	A transport protocol without flow control or retransmission; used when late data is useless.
VoIP	Voice over IP — phone calls over the internet.
Noisy neighbor	Another customer on shared hardware who hogs resources and makes your latency vary.
Jitter	Variability of response times / delays.
Phi Accrual failure detector	A failure detector that adapts timeouts to the measured distribution of response times.
Synchronous network	A network with reserved bandwidth per connection and therefore bounded delay (e.g., telephone circuits).
Circuit switching	Reserving a fixed amount of bandwidth along a whole route for the duration of a call.
Packet switching	Sending packets that share bandwidth opportunistically (Ethernet, IP).
Bounded delay	A guaranteed maximum end-to-end latency.
Bursty traffic	Data transfers with no fixed bandwidth need, which want to finish as fast as possible.
ATM (Asynchronous Transfer Mode)	A 1980s hybrid circuit/packet network technology that competed with Ethernet.
InfiniBand	A network with link-layer end-to-end flow control, reducing queueing.
Quality of service (QoS)	Prioritising and scheduling packets.
Admission control	Rate-limiting senders.
Duration	A time interval (e.g., how long a request took).
Point in time	A specific date and time (e.g., when an article was published).
Quartz crystal oscillator	The hardware that drives a computer's clock; not perfectly accurate.
NTP	Network Time Protocol — adjusts a computer's clock using time from a group of servers.
Time-of-day clock (wall-clock time)	Returns current date and time; can jump backward or forward.
Epoch	Midnight UTC, January 1, 1970 — the reference point for Unix timestamps.
Leap second	An extra (or missing) second making a minute 61 (or 59) seconds long.
Monotonic clock	Always moves forward; good for measuring durations but its absolute value is meaningless.

Term	Plain-English meaning
Slewing	NTP speeding up or slowing down a clock's rate slightly instead of jumping it.
Clock drift	A clock running faster or slower than it should.
ppm (parts per million)	Unit for drift; 200 ppm is the figure Google assumes.
Smearing	Spreading a leap-second adjustment gradually over a day.
PTP (Precision Time Protocol)	A more precise clock-synchronisation protocol than NTP.
Clock skew	The difference between two machines' clocks.
Last write wins (LWW)	Keep the write with the highest timestamp, discard the rest.
Causality	"B depended on A, so B happened after A".
Version vectors	A way of tracking causality between writes (Chapter 5).
Logical clock	A counter-based clock that orders events without measuring real time.
Physical clock	A clock measuring real elapsed time (time-of-day or monotonic).
Confidence interval (clocks)	A range of times the real time probably lies in, e.g., 95% sure it is between 10.3 and 10.5 s.
TrueTime	Google Spanner's clock API that returns `[earliest, latest]` for the current time.
Lease	A lock with a timeout; the holder must renew it before it expires.
Stop-the-world GC pause	The garbage collector halting all application threads.
Live migration	Moving a running VM to another host; causes a pause.
Steal time	CPU time a VM loses to other VMs on the same host.
Page fault / swapping / paging	Accessing memory that was moved to disk, forcing slow disk I/O.
Thrashing	The OS spending nearly all its time swapping pages rather than doing work.
SIGSTOP / SIGCONT	Unix signals that freeze and resume a process.
Preempt	Interrupt a running thread so something else can run.
Hard real-time system	A system that *must* respond before a deadline or the whole system fails.
RTOS	Real-time operating system — guarantees CPU time allocations in set intervals.
System model	A formal statement of what an algorithm may assume about timing and failures.
Asymmetric fault	A node can receive messages but its outgoing messages are lost (or vice versa).
Quorum	A vote among nodes; decisions need a minimum number of votes (often a majority).
Split brain	Two nodes both believing they are the leader.
Fencing token	An increasing number issued with each lock/lease, which the resource checks to reject stale holders.

Term	Plain-English meaning
zxid / cversion	ZooKeeper's transaction ID and node version, usable as fencing tokens.
Byzantine fault	A node sending arbitrary, faulty or deliberately misleading messages ("lying").
Byzantine Generals Problem	Reaching agreement when some participants may be traitors.
Two Generals Problem	Two parties agreeing via messengers who may be delayed or lost.
Byzantine fault-tolerant	Keeps working correctly even when some nodes misbehave or attackers interfere.
Supermajority	More than two-thirds of nodes.
Synchronous / partially synchronous / asynchronous model	Timing assumptions: always bounded / usually bounded / no timing assumptions at all.
Crash-stop / crash-recovery / Byzantine (arbitrary) faults	Node failure assumptions: crash forever / crash and maybe come back with disk intact / do anything.
Stable storage	Non-volatile disk storage assumed to survive crashes.
Safety property	"Nothing bad happens" — once violated, you can point to the moment it broke, and it cannot be undone.
Liveness property	"Something good eventually happens" — may be unmet now but can still be satisfied later.
Limping node	A node that is degraded (e.g., very slow) but not dead.

Faults and Partial Failures

On a single computer. A program on one machine usually behaves predictably: it works or it does not. When a computer seems to be "having a bad day" (often fixed by a reboot), that is mostly bad software. There is no fundamental reason for flakiness: with working hardware, the same operation always gives the same result — it is **deterministic**. If the hardware fails (memory corruption, a loose connector), the typical result is a **total** failure — kernel panic, "blue screen of death", failure to boot. A good single computer is either fully working or fully broken, not in between.

This is intentional. Computers are designed to **crash rather than return a wrong answer**, since wrong answers are confusing and hard to handle. They hide messy physics behind an **idealised system model** that looks mathematically perfect: every CPU instruction behaves identically each time, and data written to memory or disk stays intact. This goal of always-correct computation dates back to the **very first digital computer**.

With many computers connected by a network, this breaks down. You can no longer hide in the idealised model; you face the physical world, where a surprising variety of things fail. The book quotes an anecdote by **Coda Hale** listing things he had dealt with: long network partitions inside one datacenter, **PDU** (power distribution unit) failures, switch failures, whole racks accidentally power-cycled, whole-datacenter backbone and power failures, and a driver with low blood sugar crashing a **Ford pickup truck** into a datacenter's **HVAC** (heating, ventilation and air conditioning) system — and he notes he is not even an operations person.

Definition — partial failure. In a distributed system, some parts may be broken in unpredictable ways while others are fine. Partial failures are **nondeterministic**: an operation involving several nodes and the network may work sometimes and fail unpredictably at other times. It may even be impossible to **tell whether an operation succeeded**, because the time a message spends in transit is itself nondeterministic. This combination is what makes distributed systems hard.

Cloud Computing and Supercomputing

There is a spectrum of philosophies for building large-scale computing:

	High-performance computing (HPC)	**Cloud computing**
Typical use	Compute-heavy science: **weather forecasting**, **molecular dynamics** (simulating atoms and molecules)	Online internet services
Hardware	**Specialised**, each node quite reliable; communicate via **shared memory** and **RDMA**	**Commodity** machines: same performance for less money (economies of scale), but **higher failure rates**
Network	Special layouts like **multi-dimensional meshes** and **toruses**, which suit known HPC communication patterns	**IP and Ethernet**, arranged in **Clos topologies** for high bisection bandwidth
Other traits	Nodes assumed close together	Loosely defined; **multi-tenant datacenters**, **elastic/on-demand** resources, **metered billing**

Traditional enterprise datacenters sit somewhere in between.

How supercomputers handle faults.

1. A job periodically **checkpoints** its state to durable storage.
2. If a node fails, the usual approach is to **stop the whole cluster workload**.
3. Once the faulty node is repaired, the computation **restarts from the last checkpoint**.

So a supercomputer behaves more like a **single computer** than a distributed system: it turns a partial failure into a **total failure** (like a kernel panic).

Why internet services must be different. The book focuses on systems for internet services, which differ from supercomputers in several ways:

- **Online, low latency, always.** Taking the service down to fix a node is unacceptable. By contrast, an offline (batch) job such as a weather simulation can be halted and rerun with little harm.
- **Commodity hardware** fails more often than specialised supercomputer hardware.
- **Different networks** (IP/Ethernet in Clos topologies vs. meshes/toruses).
- **Something is always broken.** The bigger the system, the likelier some component is faulty. Once you have **thousands of nodes**, you should simply expect that some part is broken at any given moment. If the error handling is just "give up", a large system may spend most of its time recovering instead of working.
- **Fault tolerance helps operations.** If the system tolerates failed nodes, you can perform **rolling upgrades** (Chapter 4): nodes restart in turn while the service keeps running. In the cloud, if a VM performs badly you can kill it and request another, hoping it is faster.
- **Geographic distribution.** To keep data near users for lower latency, communication often crosses the **internet**, which is slower and less reliable than local networks. Supercomputers assume all nodes are close together.

Consequence. To make distributed systems work, you must **accept partial failure** and build **fault tolerance into the software** — building a **reliable system from unreliable components**. Perfect reliability is impossible (Chapter 1), so you also need to understand the limits of what you can promise.

Even small systems. In a system of only a few nodes, most parts work most of the time, but eventually something breaks, and the software must handle it. Fault handling must be **part of the design**, and operators need to know what the software will do in a fault.

It is unwise to assume faults are rare and hope for the best. Consider a **wide range** of faults, even unlikely ones, and **deliberately create them in testing**. In this field, being **suspicious, pessimistic**

and even paranoid is rewarded.

> **Building a Reliable System from Unreliable Components (sidebar).** It seems a system can be no more reliable than its weakest part — but that is false; building something more reliable on a less reliable base is an old idea in computing. Examples:
>
> - **Error-correcting codes** let digital data cross a channel that sometimes flips bits (e.g., radio interference on wireless networks).
> - **IP** is unreliable: packets can vanish, arrive late, arrive twice, or arrive out of order. **TCP**, layered above it, retransmits missing packets, removes duplicates, and puts packets back in sending order.
>
> There is always a **limit**: error-correcting codes handle a few single-bit errors, but if interference swamps the signal there is a fundamental cap on how much data gets through. TCP hides loss, duplication and reordering, but **cannot remove network delay**. Still, the higher layer handles tricky low-level faults so the remaining ones are easier to reason about (revisited in Chapter 12's end-to-end argument).

Unreliable Networks

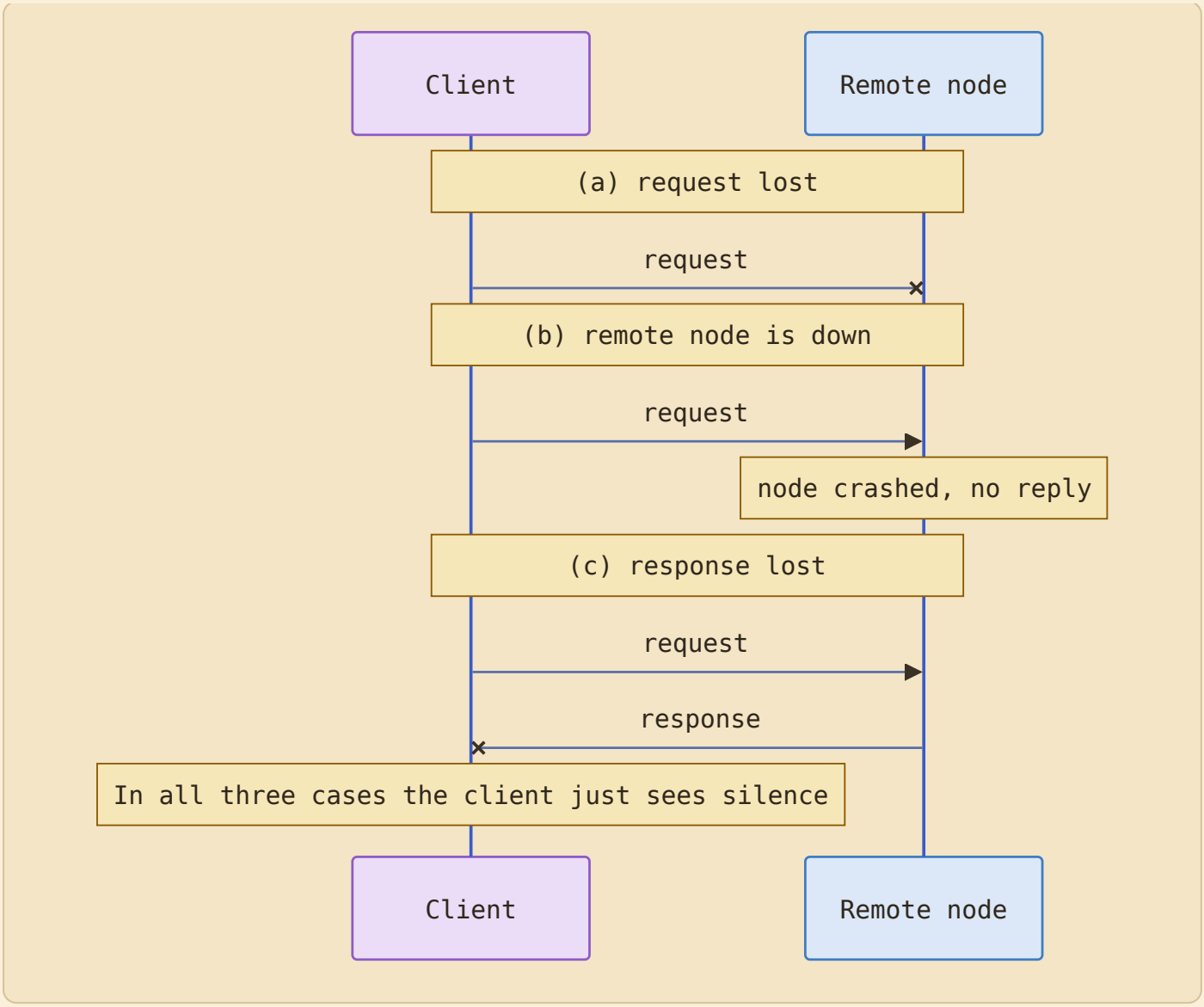
Setting. The book focuses on **shared-nothing** systems (Part II introduction): a set of machines connected by a network, where each has its **own memory and disk** and cannot touch another's except by sending requests over the network.

Why shared-nothing dominates internet services: it is **comparatively cheap** (no special hardware), it can use **commoditised cloud services**, and it can reach high reliability through **redundancy across geographically distributed datacenters**.

The network is asynchronous. The internet and most datacenter networks (often **Ethernet**) are **asynchronous packet networks**: a node can send a packet to another node, but there is **no guarantee when — or whether — it will arrive**. When you send a request and wait for an answer, lots can go wrong (Figure 8-1 shows some cases):

1. The **request was lost** (maybe someone unplugged a cable).
2. The request is **stuck in a queue** and will arrive later (network or recipient overloaded).
3. The **remote node failed** (crashed or powered down).
4. The remote node **temporarily stopped responding** (e.g., a long **garbage collection pause**) but will resume later.
5. The remote node **processed the request, but the response was lost** (e.g., a misconfigured switch).
6. The remote node processed it, but the **response is delayed** (network or your own machine overloaded).

Figure 8-1 (redrawn): after sending a request with no reply, the sender cannot tell which of these happened



The sender cannot even tell if its packet arrived; only a reply could say so, and the reply can be lost or delayed too. In an asynchronous network these cases are **indistinguishable**: all you know is that you have **not received a response yet**, and you cannot tell why.

*The usual approach: a **timeout**.* After a while you stop waiting and assume no response is coming. But when the timeout fires, you **still do not know** whether the remote node got the request — and if it is still queued somewhere, it may be delivered later even though you gave up.

Network Faults in Practice

After decades of building networks, you might expect them to be reliable by now. They are not.

Evidence (systematic studies and many anecdotes): network problems are surprisingly common, even in a datacenter run by a single company.

- A study of one **medium-sized datacenter** counted roughly **12 network faults a month**: **half** disconnected a **single machine**, **half** disconnected an **entire rack**.
- A different study looked at how often **load balancers**, **aggregation switches** and **top-of-rack switches** fail. It found that adding **redundant networking gear** reduces faults less than you would hope, because it does not protect against **human error** (e.g., misconfigured switches), a **major cause** of outages.
- **Public clouds such as EC2** are notorious for frequent **transient network glitches**; well-run private datacenter networks can be more stable.

- Nobody is immune: a problem during a **switch software upgrade** triggered a network topology reconfiguration during which packets were delayed for **more than a minute**. (The body text does not name the incident; the book's references include a GitHub downtime post-mortem for it, and the index lists it as split brain caused by a 1-minute packet delay.)
- **Sharks** may bite **undersea cables** and damage them.
- A network interface was seen **dropping all inbound packets while sending outbound ones fine** — a link working in one direction says nothing about the other.

> **Network partitions.** A fault that isolates one section of the network from the others is often called a **network partition** or **netsplit**. The book prefers the more general phrase **network fault**, so readers don't mix it up with the partitions (shards) of a database from Chapter 6.

Your software must handle them. Even if faults are rare where you run, they *can* happen, so any network communication may fail — no way around it.

Without **defined and tested** handling for network faults, anything at all can go wrong: a cluster may **deadlock** and stay unable to serve requests even after the network recovers, or it may even **delete all your data**. Software in unanticipated situations may do arbitrary unexpected things.

Handling ≠ tolerating. If your network is usually reliable, it can be valid to just **show users an error** during network trouble. But you must **know how your software reacts** and make sure it **recovers**. Intentionally **causing** network failures to see how the system copes can be worthwhile — the idea behind **Chaos Monkey** (Chapter 1).

Detecting Faults

Many systems must detect faulty nodes automatically:

- A **load balancer** must stop sending requests to a dead node (take it **out of rotation**).
- In **single-leader replication**, if the leader fails, a follower must be **promoted** (Chapter 5).

Network uncertainty makes it hard to know whether a node works. Sometimes you **do** get explicit feedback:

Situation	Feedback you might get	Limitation
Machine reachable, but **no process listening** on the port (e.g., it crashed)	The OS closes/refuses the TCP connection with a **RST** or **FIN** packet	If the node crashed *mid-request*, you cannot know how much it processed
Node **process crashed** (or killed by an admin) but the **OS still runs**	A script can **notify other nodes** so one takes over quickly without waiting for a timeout — **HBase** does this	Needs the OS to be alive
You can access the **switch management interface**	Query switches for **hardware-level link failures** (e.g., machine powered off)	Not possible over the internet, in a shared datacenter without switch access, or if a network problem blocks the management interface
A **router** is sure the IP is unreachable	It may send an **ICMP Destination Unreachable** packet	Routers have no magic failure detection either

Key lessons.

- Fast feedback about a dead node is useful but **cannot be relied on**.
- Even if **TCP acknowledges** delivery, the **application** may crash before handling the packet. Only an explicit **success reply from the application** proves a request worked.

- If something goes wrong you *might* get an error somewhere in the stack, but generally you should plan for **silence**. You may retry a few times (TCP already retries invisibly, and you can add application-level retries), wait for a timeout, and finally **declare the node dead** if you hear nothing.

Timeouts and Unbounded Delays

If a timeout is the only reliable fault detector, **how long should it be?** There is no simple answer.

Timeout	Benefit	Cost
Long	Fewer false alarms	Long wait before a node is declared dead; users wait or see errors
Short	Faster detection	Higher risk of wrongly declaring a node dead when it was only briefly slow (load spike on node or network)

Dangers of declaring a node dead too early:

- If the node is in fact alive and partway through something (the book's example: **sending an email**), a second node taking over may cause it to be **done twice** (discussed later in this chapter and in Chapters 9 and 11).
- The "dead" node's work is handed to other nodes, loading them and the network further; in a system already under strain this makes matters **worse**. If the "dead" node was only **slow due to overload**, moving its load elsewhere may trigger a **cascading failure**; at worst every node pronounces every other node dead and the whole system halts.

An imaginary ideal. Suppose a network guaranteed that every packet is either delivered within time **d** or lost (never later), and every non-failed node handled a request within time **r**. Then every successful request would get a reply within **2d + r**, and no reply by then would prove the network or node was broken. **2d + r** would be a sensible timeout.

Reality. Most systems have **neither** guarantee:

- Asynchronous networks have **unbounded delays**: they try to deliver quickly, but there is **no upper limit**.
- Most servers **cannot guarantee** handling requests within a maximum time (see "Response time guarantees" below).

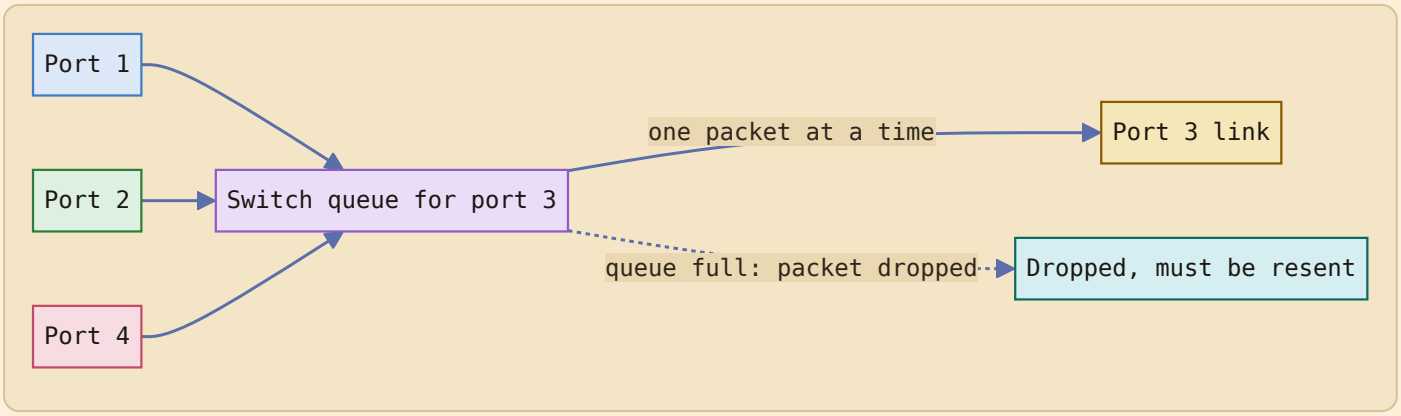
For failure detection, being fast **most** of the time is not enough: with a low timeout, one **transient spike** in round-trip times can throw the system off balance.

Network congestion and queueing

Like road travel times, which vary mostly because of **traffic jams**, packet delay variability is mostly due to **queueing**:

1. **Switch queues.** If several nodes send to the **same destination** at once, the switch must queue the packets and feed them onto the destination link **one at a time** (Figure 8-2). On a busy link a packet may wait for a slot — **network congestion**. If the queue **fills up**, the packet is **dropped** and must be resent, even though the network is technically working.
2. **OS queues.** At the destination, if all **CPU cores are busy**, the OS queues the incoming request until the app can take it — for an arbitrary time depending on load.
3. **VM pauses.** Under virtualisation, a guest OS is frequently **frozen for tens of milliseconds** so another VM can use the CPU core. Meanwhile the VM cannot read network data, so the **virtual machine monitor buffers** it, adding more variability.
4. **TCP flow control** (also called **congestion avoidance** or **backpressure**): a node limits its own sending rate to avoid overloading a link or the receiver, which means **extra queueing at the sender** before data even enters the network.

Figure 8-2 (redrawn): ports 1, 2 and 4 all send to port 3, so the switch queue for port 3 fills up



TCP retransmission adds delay too. TCP treats a packet as **lost** if it is not acknowledged within a timeout (computed from observed round-trip times) and **retransmits** it automatically. The app does not see the loss, but it **does see the delay** (waiting for the timeout, then for the retransmission to be acknowledged).

> **TCP Versus UDP.** Latency-sensitive apps like **videoconferencing** and **VoIP** use **UDP**. It trades reliability for less delay variability: UDP has **no flow control** and **no retransmission**, avoiding some causes of variable delay (though switch queues and scheduling delays still apply). UDP fits when **late data is worthless**. In a VoIP call, there is no time to resend a lost packet before its audio must play, so the app plays **silence** for that slot (a brief glitch) and moves on. The retry happens at the **human layer** — someone asks the other person to repeat what they said.

Utilisation matters. All these factors make delays variable. Queueing delays vary **most** when a system is **near maximum capacity**: with spare capacity, queues drain quickly; in a heavily used system, long queues form fast.

Shared environments. In **public clouds and multi-tenant datacenters**, customers share network links, switches, and even each machine's **network interface and CPUs** (on VMs). **Batch jobs like MapReduce** (Chapter 10) can easily **saturate** links. Since you cannot control or see other customers' usage, delays can vary wildly if a **noisy neighbor** uses lots of resources.

Choosing timeouts in practice.

1. Choose them **experimentally**: measure the **distribution of round-trip times** over a long period and across many machines to learn the expected variability.
2. Considering your app's needs, pick a trade-off between **detection delay** and **risk of premature timeouts**.
3. **Better still:** rather than hard-coding a timeout, the system can **keep measuring response times and how much they vary (jitter)** and **tune the timeout on the fly**. The **Phi Accrual failure detector** does this and is used in **Akka** and **Cassandra**. **TCP retransmission timeouts** work in a similar way.

Synchronous Versus Asynchronous Networks

Why not fix it in hardware? Distributed systems would be far simpler if networks delivered packets with a **fixed maximum delay** and **never dropped** them. Could we build that?

Compare with the telephone network. The traditional **fixed-line telephone network** (not cellular, not VoIP) is **extremely reliable**: delayed audio and dropped calls hardly ever happen. A phone call requires **consistently low latency end to end**, plus enough bandwidth to carry the voice samples.

How it works — a circuit.

1. Dialling sets up a **circuit**: a **set, guaranteed slice of bandwidth** is reserved along the **entire route** between the callers.

2. The circuit stays until the call ends.
3. Example: **ISDN** operates at a constant **4,000 frames per second**. A call is given **16 bits** in each frame, in each direction. So for the whole call, each side can send exactly **16 bits of audio every 250 microseconds**.

This network is **synchronous**: even across several routers, data does not queue, because the call's 16 bits are **already reserved** in the next hop. Since nothing queues, the **worst-case end-to-end latency is a known constant** — a **bounded delay**.

Can we not simply make network delays predictable?

Circuit vs. TCP connection.

Circuit (telephone)	TCP connection (packet-switched)
Fixed reserved bandwidth that **nobody else can use** while the circuit exists	Packets **opportunistically use whatever bandwidth is available**
Suits constant-rate data (audio/video calls)	Suits variable-sized blocks (an email, a web page), sent as fast as possible
Uses bandwidth for the whole call	Uses **no bandwidth when idle** (except perhaps an occasional **TCP keepalive** packet)

Were datacenter networks and the internet **circuit-switched**, a maximum round-trip time could be promised whenever a circuit was created. But they aren't: **Ethernet and IP use packet switching**, suffer from queueing and hence **unbounded delays**, and have **no concept of a circuit**.

Why packet switching? Because it is optimised for **bursty traffic**. A circuit suits audio/video calls needing a steady bit rate. But fetching a web page, sending email or transferring a file has **no particular bandwidth requirement** — you just want it done **as fast as possible**.

Transferring a file over a circuit would require **guessing a bandwidth**:

- **Too low:** the transfer is needlessly slow and capacity sits unused.
- **Too high:** the network **refuses to create the circuit**, because it only builds circuits whose bandwidth it can promise.

So circuits for bursty transfers **waste capacity** and **slow transfers down**. TCP instead **adapts its rate** to the capacity available.

Hybrids. There have been attempts at networks supporting both circuit and packet switching, such as **ATM**. **InfiniBand** has similarities: it does **flow control end to end at the link layer**, which cuts down queueing in the network, though link congestion can still cause delays. With careful **quality of service** (QoS: prioritising and scheduling packets) and **admission control** (rate-limiting senders), you can **emulate circuit switching** on packet networks or give **statistically bounded delay**.

> **Footnote — ATM.** Asynchronous Transfer Mode competed with Ethernet in the **1980s** but was rarely used except in the **core switches of telephone networks**. Despite the acronym, it is unrelated to cash machines. The book muses that in a parallel universe where the internet ran on something like ATM, video calls would probably be far more reliable.

> **Latency and Resource Utilization (sidebar).** Variable delays come from **dynamic resource partitioning**.
>
> - Imagine a cable linking two telephone exchanges with capacity for **10,000 calls at once**. Every circuit switched over it occupies one slot. The wire is split **statically**: even if you are the only caller and **9,999 slots are empty**, your circuit gets the same fixed bandwidth as when the wire is full.

> - The **internet shares bandwidth dynamically**: senders jostle to get packets onto the wire fast, and switches decide moment by moment which packet goes next. The downside is **queueing**; the upside is **maximum utilisation**. Since the wire has a fixed cost, better utilisation makes each byte **cheaper**.
> - **CPUs** are similar: sharing a core dynamically among threads means a thread sometimes waits in the OS **run queue** and is paused for varying times, but hardware is used better than with a static CPU-cycle allocation per thread. Better utilisation is also a big reason for **virtual machines**.
> - **Latency guarantees** are possible with **static partitioning** (dedicated hardware, exclusive bandwidth) — at the cost of lower utilisation, i.e., **more expensive**. Multi-tenancy with dynamic partitioning is **cheaper** but has **variable delays**.
> - So variable network delay is **not a law of nature**, just the outcome of a **cost/benefit trade-off**.

Current reality. Such QoS is **not enabled** in multi-tenant datacenters, public clouds, or across the internet. Deployed technology gives **no guarantees** on network delay or reliability; you must **assume congestion, queueing and unbounded delays**. Therefore there is **no "correct" timeout value** — determine it **experimentally**.

> **Footnote — BGP.** Peering agreements between ISPs and routes set up through the **Border Gateway Protocol (BGP)** resemble circuit switching more than IP does; at that level you can buy **dedicated bandwidth**. But internet routing works at the level of **networks**, not individual host connections, and on a **much longer timescale**.

Unreliable Clocks

Applications use clocks to answer questions like these (the book's list):

1. Has this request timed out yet?
2. What is the service's **99th percentile** response time?
3. How many queries per second did it handle on average over the **last five minutes**?
4. How much time did a visitor spend on the website?
5. When was this article published?
6. When should the reminder email go out?
7. When does this cache entry expire?
8. What is the timestamp on this log error?

Items **1–4 measure durations** (intervals, e.g., from request to response). Items **5–8 describe points in time** (events at a particular date and time).

Why time is tricky in distributed systems.

- **Communication is not instantaneous.** A message is always received *after* it was sent, but because network delay varies, you do not know **how much** after. That makes it hard to determine the **order** of events across machines.
- **Each machine has its own clock**, a physical device — usually a **quartz crystal oscillator**. These are **not perfectly accurate**, so each machine's time may be a bit fast or slow compared with others.
- Clocks can be **synchronised to a degree**. The usual tool is the **Network Time Protocol (NTP)**; it corrects a computer's clock using time reported by a group of servers, which in turn get time from a more accurate source such as a **GPS receiver**.

Monotonic Versus Time-of-Day Clocks

Modern computers have at least **two** kinds of clock. Both measure time, but they serve **different purposes**.

Time-of-day clocks

What it is. A **time-of-day clock** behaves the way you intuitively expect a clock to: it tells you today's date and the current time according to a calendar (**wall-clock time**).

Examples: `clock_gettime(CLOCK_REALTIME)` on Linux and `System.currentTimeMillis()` in Java. Both give the number of seconds (or milliseconds) elapsed since the **epoch** — **00:00 UTC on 1 January 1970** in the Gregorian calendar — **ignoring leap seconds**. Some systems use other reference dates.

> **Footnote:** the "real-time" in `CLOCK_REALTIME` is unrelated to real-time operating systems (discussed under "Response time guarantees").

Properties and pitfalls.

- Usually **synchronised with NTP**, so ideally a timestamp on one machine means the same as on another.
- A local clock that has run **well ahead** of the NTP server can be **reset by force**, so it seems to **leap back in time**.
- Such jumps, together with the usual **disregard of leap seconds**, mean time-of-day clocks are **a poor choice for timing how long something took**.
- Historically they had **coarse resolution** — e.g., stepping in **10 ms** increments on older Windows systems. Less of a problem on recent systems.

Monotonic clocks

What it is. A **monotonic clock** is for measuring a **duration**, such as a timeout or response time. On Linux that is `clock_gettime(CLOCK_MONOTONIC)`; in Java, `System.nanoTime()`. "Monotonic" refers to the promise that it **never goes backward**, unlike a time-of-day clock.

How to use it.

1. Read the monotonic clock.
2. Do some work.
3. Read it again. The **difference** is the elapsed time.

Caveats.

- The **absolute value is meaningless** — e.g., nanoseconds since boot. **Never compare** monotonic values **across two computers**.
- On servers with **multiple CPU sockets**, each CPU may have its own **unsynchronised timer**. The OS compensates to present a monotonic view to threads moving between CPUs, but the book advises treating that guarantee with **some scepticism**.
- **NTP slewing:** if NTP sees the local quartz running fast or slow, it may **adjust the rate** of the monotonic clock. By default NTP allows speeding up or slowing down by **up to 0.05%**, but it **cannot make the monotonic clock jump**.
- **Resolution** is usually good: **microseconds or finer** on most systems.

In distributed systems, measuring elapsed time (such as timeouts) with a monotonic clock **generally works well**: it assumes no synchronisation across nodes and tolerates small measurement inaccuracies.

	Time-of-day clock	Monotonic clock
Answers	"What date/time is it?" (point in time)	"How much time passed?" (duration)
Linux / Java	`CLOCK_REALTIME` / `System.currentTimeMillis()`	`CLOCK_MONOTONIC` / `System.nanoTime()`
Can jump backward?	Yes (NTP reset)	No (only slewed up to 0.05%)
Comparable across machines?	Ideally yes, if synced	No — absolute value is arbitrary
Needs NTP?	Yes, to be useful	No

Clock Synchronization and Accuracy

Monotonic clocks need no synchronisation, but time-of-day clocks must be set from NTP or another external source to be useful. Unfortunately, getting the right time is **far less reliable** than you would hope. The book's examples:

- **Quartz drift.** A computer's quartz clock **drifts** (runs fast or slow), varying with **temperature**. **Google assumes 200 ppm** drift for its servers — equal to **6 ms** of drift if resynced every **30 seconds**, or **17 seconds** if resynced **once a day**. Drift caps the best possible accuracy even when everything works.
- **Forced resets.** If a clock differs too much from the NTP server, the client may **refuse to sync** or the clock is **forcibly reset**. Apps watching the time may see it **go backward or jump forward**.
- **Firewalled from NTP.** A node accidentally cut off from NTP servers may go **unnoticed** for a while; anecdotes suggest this really happens.
- **Network delay limits NTP.** NTP can be no better than the network delay. One experiment achieved a **minimum error of 35 ms** syncing **over the internet**, but occasional delay spikes caused errors of **about a second**. With large delays, the NTP client may **give up** entirely, depending on configuration.
- **Wrong NTP servers.** Some servers are wrong or misconfigured, off by **hours**. NTP clients are fairly robust — they query **several servers and ignore outliers** — but it is unsettling to stake correctness on time told to you by **a stranger on the internet**.
- **Leap seconds** make a minute **59 or 61 seconds** long, breaking timing assumptions in systems not designed for them. Leap seconds have **crashed many large systems** (the body text names no specific incident; the book's references and index include a 2012 report of a leap second — via a Linux bug — crashing "half the internet", and Cloudflare DNS being affected in 2017), showing how easily wrong clock assumptions creep in. The best approach may be to have NTP servers **"lie"** by applying the leap second **gradually over a day** — called **smearing** — though real NTP server behaviour varies.
- **Virtual machines.** The hardware clock is **virtualised**, creating extra problems for accurate timekeeping. When VMs share a CPU core, each VM is **paused for tens of milliseconds** while another runs; to the app this looks like the clock **suddenly jumping forward**.
- **Devices you don't control** (mobile, embedded): you probably **cannot trust the hardware clock at all**. Certain users **set incorrect dates on purpose**, for instance to dodge time limits in **games**, so the clock may be wildly in the past or future.

High accuracy is possible, at a price. The **MiFID II** draft European regulation requires **high-frequency trading funds** to sync clocks **within 100 microseconds of UTC**, so that odd market events like **"flash crashes"** can be investigated and **market manipulation** spotted. That accuracy is reachable with **GPS receivers**, the **Precision Time Protocol (PTP)**, and careful deployment and monitoring — but it takes significant effort and expertise, and many things can go wrong. A misconfigured NTP daemon or a firewall blocking NTP traffic can let drift errors grow large quickly.

Relying on Synchronized Clocks

Clocks look simple but hide pitfalls: a day is **not always exactly 86,400 seconds** long, a time-of-day clock can **run backward**, and one node's time may differ a lot from another's.

Same attitude as for networks. Networks behave well most of the time, but software must assume occasional faults and handle them gracefully. Likewise, clocks work well most of the time, but **robust software must be prepared for wrong clocks**.

Why clock faults are sneaky. A defective CPU or misconfigured network usually stops a machine working entirely, so it is noticed and fixed fast. A defective quartz clock or misconfigured NTP client lets most things **appear fine** while the clock **drifts further and further** from reality. Software relying on accurate clocks then tends to suffer **silent, subtle data loss** rather than a dramatic crash.

What to do. If your software needs synchronised clocks, **monitor clock offsets between all machines**. A node whose clock wanders too far from the rest should be **treated as dead and kicked out of the cluster**, so broken clocks are caught before they do much damage.

Timestamps for ordering events

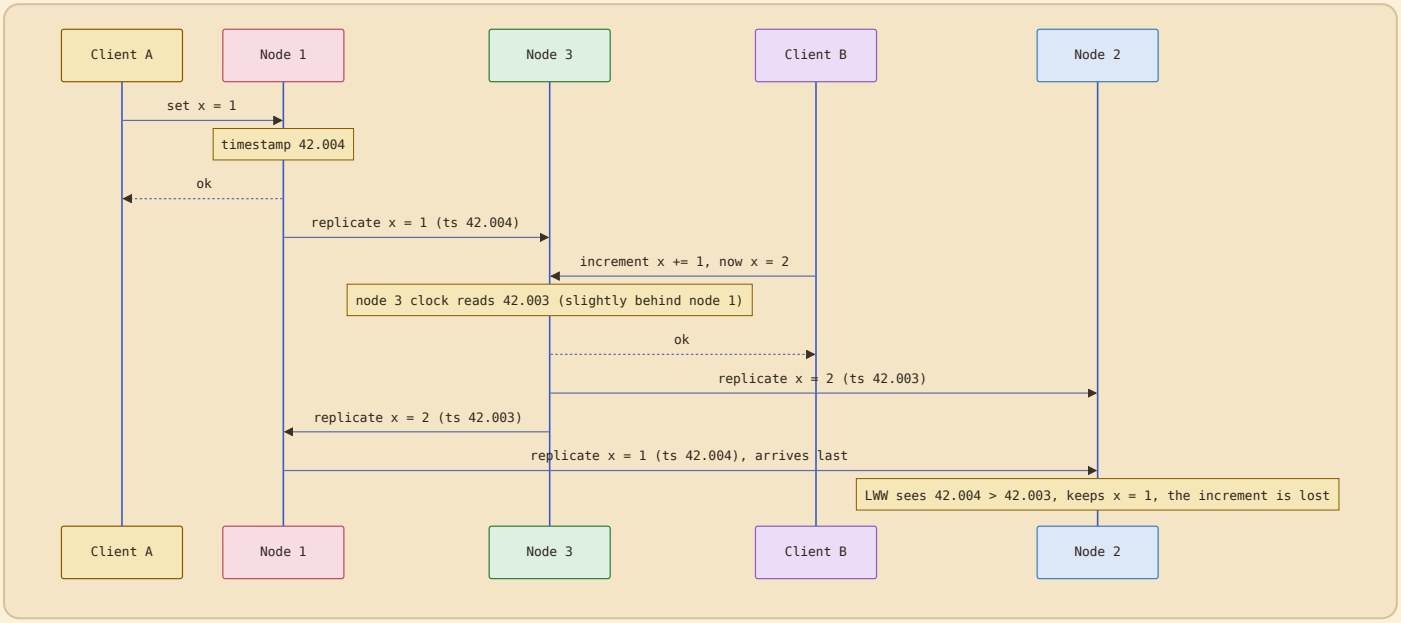
The tempting mistake. Relying on clocks to decide the order of events on different nodes — e.g., when two clients write to a distributed database, whose write came first, and which is the latest?

Figure 8-3 illustrates a risky reliance on time-of-day clocks in a **multi-leader** replicated database (similar to Figure 5-9 in Chapter 5):

1. On **node 1**, **client A** sets `x = 1`.
2. That write is replicated to **node 3**.
3. On **node 3**, **client B** increments x, making it `x = 2`.
4. Both writes are replicated to **node 2**.

Each write is tagged with a timestamp from the **time-of-day clock of the node where it originated**. Synchronisation in this example is **excellent**: nodes 1 and 3 differ by **less than 3 ms** — probably better than you'd get in practice. Yet the timestamps order events **wrongly**: `x = 1` has timestamp **42.004 s**, while `x = 2` has **42.003 s**, even though `x = 2` clearly happened later. Node 2 concludes `x = 1` is newer and **drops** `x = 2` — client B's increment is **lost**.

Figure 8-3 (redrawn): client B's write is causally later but gets an earlier timestamp, so node 2 discards it under LWW



This is last write wins (LWW), common in multi-leader setups and in leaderless databases like **Cassandra** and **Riak** (Chapter 5). Some implementations generate timestamps on the **client** instead of the server, but that does not fix LWW's basic problems:

- **Writes can mysteriously vanish.** If one node's clock **runs behind**, its writes cannot replace values written earlier by a node whose clock **runs ahead** until enough time to cover the skew has passed. Arbitrary amounts of data can be **silently dropped** with no error reported.
- **LWW cannot tell sequential from concurrent writes.** In Figure 8-3, B's increment definitely came after A's write; truly concurrent writes are ones where neither writer knew of the other. Telling them apart needs extra **causality tracking** such as **version vectors** (Chapter 5).
- **Identical timestamps.** Two nodes can generate writes with the **same timestamp**, especially with **millisecond** resolution. A **tiebreaker** (even just a large random number) is needed, but that can also **violate causality**.

So "keep the most recent value" sounds good, but what counts as "recent" is decided by the **time-of-day clock on one machine, and it may be off**. Even with closely NTP-synced clocks, a packet stamped **100 ms** by its sender could be received at **99 ms** by the receiver's clock — apparently arriving before departure, which cannot happen.

Could NTP be made accurate enough? Probably **not**: NTP's accuracy is limited by **network round-trip time** plus quartz drift and other errors. Correct ordering would need a clock source **much more accurate** than the thing being measured (network delay).

The safer alternative: logical clocks. **Logical clocks** are based on **incrementing counters** rather than a quartz oscillator. They measure neither time of day nor elapsed seconds — only **relative order** (did one event happen before another?). Time-of-day and monotonic clocks, which do track real elapsed time, are grouped together as **physical clocks**. (More on ordering in Chapter 9.)

Clock readings have a confidence interval

Precision is not accuracy. You may read a clock with **microsecond or nanosecond resolution**, but that does not make the value that accurate — it almost certainly is not. Quartz drift can easily be **several milliseconds** even when syncing with a **local-network NTP server every minute**. With a **public-internet** NTP server, the best is probably **tens of milliseconds**, and errors can spike to **over 100 ms** under network congestion.

So think of a reading as a range. A reading is better viewed as an interval than a point, with a **confidence level** attached — e.g., a system might be **95% sure** the current time lies between **10.3 s and 10.5 s** after the minute, and have no finer knowledge than that. When your uncertainty is **±100 ms**, the microsecond part of a timestamp tells you **nothing useful**.

Calculating the uncertainty:

- With a **GPS receiver** or **atomic (caesium) clock** attached directly, the manufacturer reports the expected error.
- When getting time from a **server**, uncertainty ≈ **expected quartz drift since the last sync** + **the NTP server's own uncertainty** + **network round-trip time** to the server (roughly speaking, and only if the server is trustworthy).

Problem: most systems **don't expose this uncertainty**. The value returned by `clock_gettime()` gives no hint whether its error is **five milliseconds or five years**.

Exception — Google's TrueTime API in Spanner. It **explicitly reports** the confidence interval. Asking for the current time returns **two values,** `[earliest, latest]` — the earliest and latest possible timestamps. The clock knows the real time lies somewhere in that range. The interval's width depends partly on **how long since the local quartz clock last synced** with a more accurate source.

Synchronized clocks for global snapshots

Recap. **Snapshot isolation** (Chapter 7) is valuable when a database has to handle **quick, small read-write transactions** alongside **big, slow read-only ones** (backups, analytics). Read-only transactions see a consistent state at one point in time without locking or interfering with writers.

How it's usually done: a **monotonically increasing transaction ID**. A write with a greater ID than the snapshot is **invisible** to the snapshot. On a single node, a **simple counter** suffices.

The distributed problem. Across many machines and possibly multiple datacenters, producing one **global, ever-increasing transaction ID** that spans all partitions is difficult, since it requires **coordination**. The IDs must respect **causality**: whenever B reads something A wrote, B's ID has to be **larger** than A's, otherwise the snapshot is inconsistent. With many small, fast transactions, generating IDs becomes an **untenable bottleneck**.

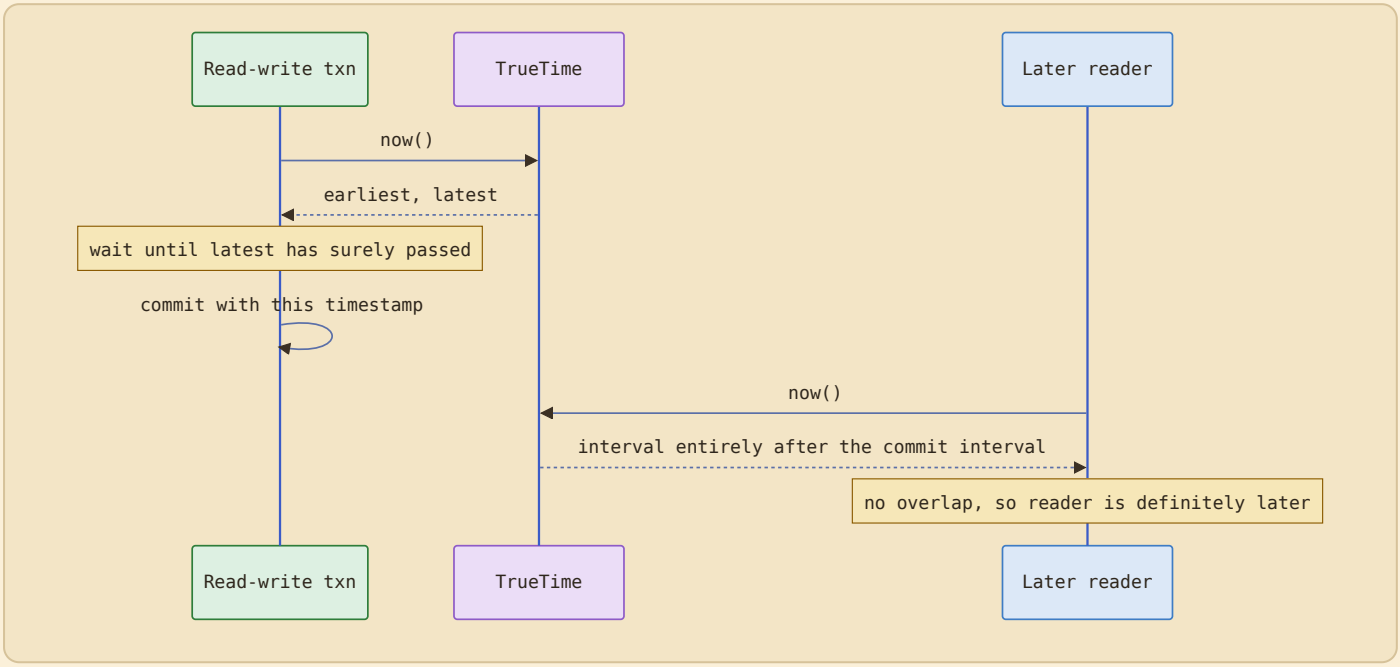
> **Footnote — Snowflake.** Distributed sequence generators like **Twitter's Snowflake** produce **approximately** increasing unique IDs scalably (for example, by handing out **ranges of IDs** to different nodes). But they usually **cannot guarantee causally consistent ordering**, because blocks are handed out on a longer timescale than database reads and writes happen.

Idea: use synchronised clock timestamps as transaction IDs. If sync were good enough, later transactions would have higher timestamps — exactly what we need. The obstacle is **clock uncertainty**.

How Spanner does it (snapshot isolation **across datacenters**), using TrueTime confidence intervals:

1. Each timestamp is an interval: A = [A_earliest, A_latest], B = [B_earliest, B_latest].
2. If the intervals **do not overlap** (A_earliest < A_latest < B_earliest < B_latest), then **B definitely happened after A**.
3. Only if they **overlap** is the order uncertain.
4. So that timestamps respect causality, Spanner **pauses for the width of the confidence interval before it commits any read-write transaction**. Any later transaction that could read its data is then far enough in the future that their intervals don't overlap.
5. To keep that wait short, Spanner keeps clock uncertainty small: Google installs a **GPS receiver or an atomic clock in every datacenter**, keeping clocks synced to roughly **7 ms**.

Spanner commit-wait (illustrative diagram of the steps above, not a book figure)



Using clock sync for distributed transaction semantics is an **active research area**; the ideas have **not yet appeared in mainstream databases outside Google**.

Process Pauses

Another dangerous use of clocks. Consider a database with **one leader per partition**; only the leader accepts writes. How does a node know it is **still the leader** (not declared dead by others) and may safely accept writes?

Leases. The leader can get a **lease** from the other nodes — like a **lock with a timeout**. Only one node holds the lease at a time, so the holder knows it is leader **until the lease expires**. To stay leader, it must **renew** the lease before expiry. If it fails, it stops renewing and another node takes over when the lease expires.

The book shows a request-handling loop for this. Here is my own version of the same idea, in the same Java-like style:

```
while (running) {
    Job job = queue.nextJob();

    // Renew early: keep at least 5 seconds of lease left
    long remaining = myLease.expiresAtMillis() - System.currentTimeMillis();
    if (remaining < 5000) {
        myLease = myLease.extend();
    }

    if (myLease.stillValid()) {
        handle(job);
    }
}
```

- `while (running)` — loop forever handling work.
- `queue.nextJob()` — take the next incoming request.
- `remaining = expiresAtMillis() - System.currentTimeMillis()` — compares an expiry time **set by another machine** with the **local time-of-day clock**.
- `if (remaining < 5000) extend()` — renew when little time is left.
- `if (stillValid()) handle(job)` — only process while we think we hold the lease.

(The book's version uses a **10-second** buffer, and imagines the expiry being computed on another machine as, for example, **current time plus 30 seconds**.)

What is wrong with it?

1. **It relies on synchronised clocks.** The expiry is set by a **different machine** but compared against the **local** clock. If clocks differ by **more than a few seconds**, the code misbehaves.
2. **Even with only a local monotonic clock**, it assumes very little time passes between **checking the time** and **processing the request**. Normally that is true, so the 10-second buffer is plenty. But imagine the thread **freezes for 15 seconds** near the `lease.isValid()` check (the book's code; `stillValid()` in mine). By the time it handles the request, the lease has most likely **lapsed** and a **different node may have become leader**. Nothing tells the thread it was paused, so it will not notice until the **next loop iteration** — after it may already have done something **unsafe**.

Is a 15-second pause realistic? Unfortunately yes. Causes:

- **Garbage collection.** Many runtimes (e.g., the **JVM**) have a GC that sometimes **stops all threads**. These **"stop-the-world" pauses** have been known to last **several minutes**. Even "concurrent" collectors like **HotSpot JVM's CMS** must stop the world occasionally. Tuning allocation patterns or GC settings can shorten pauses, but for robust guarantees you must **assume the worst**.

- **VM suspend/resume.** A VM can be **suspended** (all processes paused, memory saved to disk) and **resumed** at any point, for any length of time. This is used for **live migration** between hosts without reboot; the pause length then depends on how fast processes are writing to memory.
- **End-user devices** such as laptops can be suspended arbitrarily, e.g., when the **lid is closed**.
- **Context switches.** When the OS switches to another thread, or the **hypervisor** switches to another VM, the current thread can pause at **any point** in the code. For VMs, CPU time spent in other VMs is **steal time**. Under heavy load (long run queue), the paused thread may wait a while to run again.
- **Synchronous disk I/O.** A thread may wait on a slow disk. Disk access can happen **unexpectedly** — e.g., the **Java classloader lazily loads class files** on first use, at any time. **I/O pauses and GC pauses can combine**. When what looks like a local disk is really a **network filesystem** or a **network block device** (for example **Amazon's EBS**), I/O latency also inherits **network delay variability**.
- **Swapping (paging).** If the OS may swap to disk, a simple memory access can cause a **page fault** requiring a page to be loaded from disk, pausing the thread. Under high memory pressure, another page may need to be swapped out. In extreme cases the OS spends most of its time swapping — **thrashing**. So paging is often **disabled on servers** (preferring to kill a process to free memory over risking thrashing).
- **SIGSTOP.** A Unix process can be paused with **SIGSTOP** (e.g., **Ctrl-Z** in a shell) until resumed with **SIGCONT**. Even if your environment never uses it, an operations engineer might send it **by accident**.

The core problem. All of these can **preempt** a thread at any point and resume it later **without it noticing**. It resembles making multi-threaded code thread-safe on one machine: you can assume **nothing about timing**. On one machine you have good tools — **mutexes, semaphores, atomic counters, lock-free data structures, blocking queues** — but these **don't translate directly** to distributed systems, which have **no shared memory**, only messages over an unreliable network.

Every node has to assume it might be **frozen for a long stretch at any instant**, even halfway through a function. While it is frozen, other nodes carry on and may **decide it has died**. On waking, it may have no idea anything happened until it next looks at its clock.

Response time guarantees

Threads and processes may pause for **unbounded** time in many languages and OSes — but those causes **can be eliminated** with enough effort.

Hard real-time systems. Some software must respond within a specified time or cause serious damage: computers that steer **aircraft, rockets, robots, cars** and other physical machinery have to react to sensor input fast and predictably. There is a **deadline**; missing it may cause **failure of the whole system**. The book's example: if a car's sensors detect a crash, you would not want the **airbag release delayed by a GC pause**.

> **Is real-time really real?** For **embedded systems**, "real-time" describes a system engineered and tested so that it **always meets its timing guarantees**. On the **web**, "real-time" is used loosely for servers pushing data to clients and for stream processing, with **no hard response-time constraints** (Chapter 11).

What real-time guarantees require (every layer of the stack):

1. A **real-time operating system (RTOS)** that schedules processes with **guaranteed CPU time in specified intervals**.
2. **Library functions** that document their **worst-case execution times**.
3. **Dynamic memory allocation** restricted or **forbidden** (real-time garbage collectors exist, but the app must not give the GC too much work).
4. An **enormous amount of testing and measurement** to confirm the guarantees hold.

Costs. Lots of extra work, and a sharply limited choice of languages, libraries and tools (most offer no real-time guarantees). So real-time systems are **very expensive**, used mainly in **safety-critical embedded devices**. Also, **"real-time" ≠ "high-performance"**: real-time systems can deliver **less throughput**, because meeting deadlines outranks everything else (compare the Latency and Resource Utilization sidebar).

For most **server-side data systems**, real-time guarantees are **not economical or appropriate**, so these systems must **live with** pauses and clock instability.

Limiting the impact of garbage collection

You can reduce the harm of pauses **without** real-time scheduling. Language runtimes have some freedom in **when** they schedule GC, since they can track allocation rate and remaining free memory.

Idea 1 — treat GC as a planned outage.

1. The runtime **warns** the application that a node will soon need a GC pause.
2. The app **stops sending new requests** to that node.
3. It **waits** for outstanding requests to finish.
4. The node runs GC with **no requests in progress**.

This hides GC pauses from clients and reduces **high-percentile response times**. Some **latency-sensitive financial trading systems** use this approach.

Idea 2 — only collect short-lived objects, and restart. Use the GC only for **short-lived objects** (fast to collect) and **restart processes periodically**, before enough long-lived objects pile up to need a **full GC**. Restart **one node at a time**, shifting traffic away first, like a **rolling upgrade** (Chapter 4).

These measures **cannot fully prevent** GC pauses but usefully **reduce their impact**.

Knowledge, Truth, and Lies

Recap of what's different. Distributed systems have **no shared memory**, only **message passing over an unreliable network with variable delays**, and suffer **partial failures, unreliable clocks and processing pauses**.

The consequence is disorienting. No node can be **certain** of anything; all it can do is **infer** from which messages arrive (or fail to). It learns another node's state (what data it holds, whether it works) only by **exchanging messages**. If a remote node does not respond, there is **no way to know its state**, since network problems cannot reliably be told apart from node problems.

The discussion drifts toward philosophy: what counts as true in the system, how confident can we be when our means of observing and measuring are flawed, should software obey physical laws such as **cause and effect**?

The practical answer: system models. We don't need to solve philosophy. We **state our assumptions** about behaviour — the **system model** — and design the system to meet them. **Algorithms can be proved correct within a given system model**, so reliable behaviour is achievable even when the model guarantees very little. It is **not straightforward**, though. The rest of this chapter explores knowledge and truth; Chapter 9 gives algorithms that provide specific guarantees under specific assumptions.

The Truth Is Defined by the Majority

The book tells three scenarios:

1. **Asymmetric fault.** A node can **receive** all messages, but its **outgoing** messages are dropped or delayed. It works perfectly and gets requests, but others cannot hear its replies. After a

timeout they **declare it dead**. The book's image: the node is carried to the graveyard shouting that it is alive, but nobody hears, so the funeral goes ahead.

2. **Aware but powerless.** A slightly better case: the semi-disconnected node notices its messages are not acknowledged and realises there is a network fault. It is still wrongly declared dead and **can do nothing** about it.
3. **Long GC pause.** A node hits a stop-the-world GC; **all threads pause for one minute**, so no requests are processed and no responses sent. Others wait, retry, lose patience and declare it dead. Then the GC ends and the node carries on as though nothing happened — to everyone's surprise, like a corpse sitting up in its coffin and chatting. At first the node doesn't even realise a whole minute passed or that it was declared dead; from its viewpoint, hardly any time has elapsed.

Moral: a node **cannot necessarily trust its own judgement**. A distributed system cannot rely exclusively on a **single node**, which may fail at any time and leave the system stuck.

Solution: quorums. Many distributed algorithms rely on a **quorum** — **voting** among nodes (Chapter 5). Decisions need a **minimum number of votes** from several nodes, reducing dependence on any one.

- This includes **declaring nodes dead**: if a quorum says a node is dead, it **must be considered dead**, even if it feels very much alive. The individual node must **accept the decision and step down**.
- Usually the quorum is a strict **majority** — over half the nodes — although other quorum types exist.
- With a majority quorum the system survives individual node failures: **3 nodes survive 1 failure**, and **5 nodes survive 2**.
- It is still **safe**, because there can be **only one majority** at a time — never two majorities with conflicting decisions. (More on quorums with consensus in Chapter 9.)

The leader and the lock

Often a system needs **exactly one** of something:

- Only **one leader** per database partition, to avoid **split brain** (Chapter 5).
- Only **one transaction or client** may hold the **lock** for a resource/object, so concurrent writes don't corrupt it.
- Only **one user** may register a given **username**, since it must uniquely identify a user.

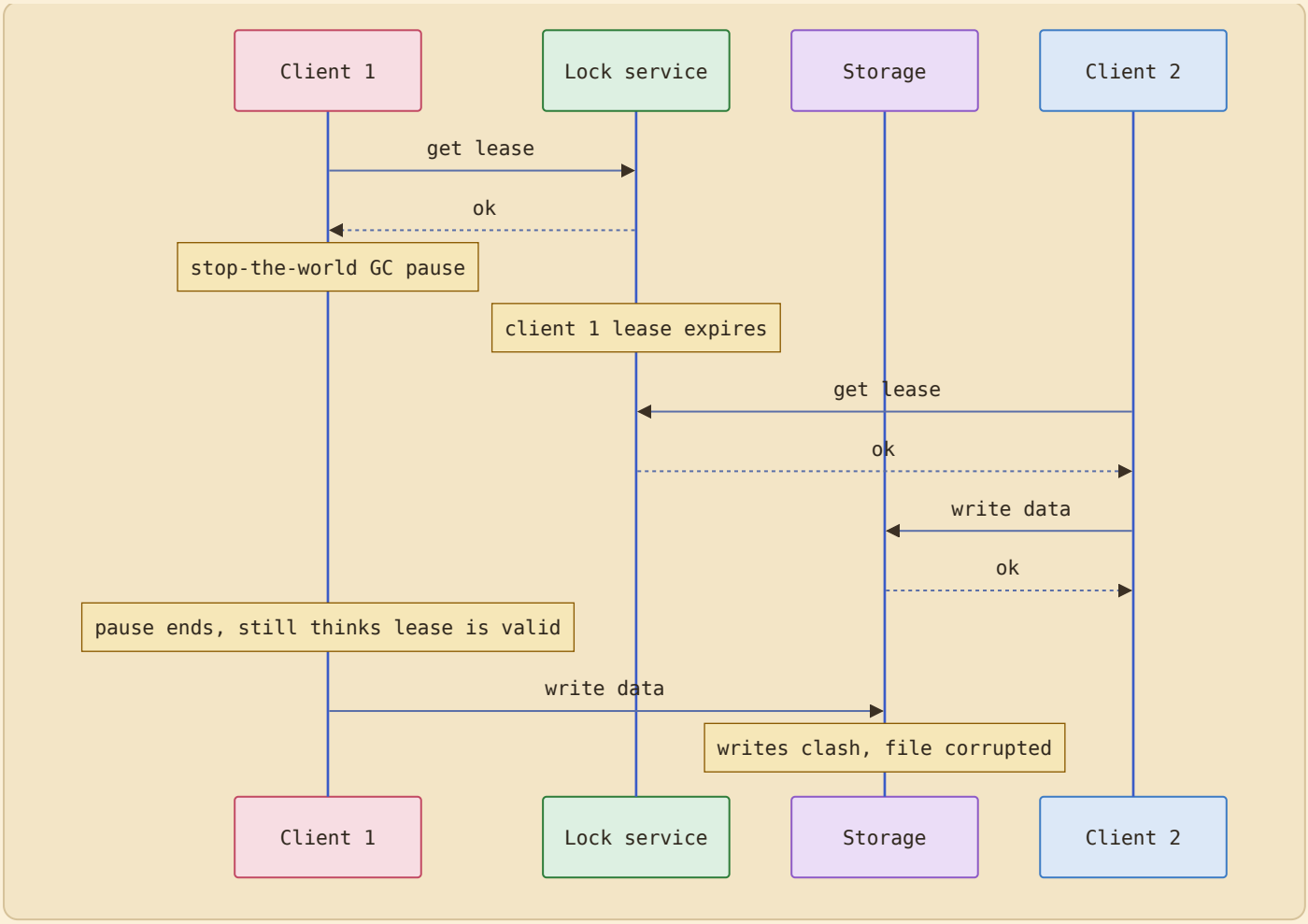
The trap. A node may **believe it is "the chosen one"** (partition leader, lock holder, the handler that grabbed the username) while a **quorum disagrees**. It may once have been leader, but if the others pronounced it dead in the meantime (because of a **network interruption** or a **GC pause**, say), it may have lost the role and a replacement may already have been **elected**. If it keeps acting as the chosen one, it can send messages in its self-appointed role; if others believe them, the system may do something wrong.

Figure 8-4 — a real bug. An incorrect distributed lock caused data corruption; **HBase used to have this problem**. Goal: a file in a storage service may be accessed by only **one client at a time**, since concurrent writes would corrupt it. The attempted fix: before touching the file, a client first acquires a **lease** from a **lock service**.

What goes wrong (the "Process Pauses" problem):

1. Client 1 gets the lease.
2. Client 1 **pauses** (e.g., GC) for too long; its **lease expires**.
3. Client 2 gets a lease for the same file and **starts writing**.
4. Client 1 resumes, **wrongly believes** its lease is still valid, and **also writes**.
5. The writes clash and the file is **corrupted**.

Figure 8-4 (redrawn): client 1's lease expires during a GC pause, but it still writes, corrupting the file



Fencing tokens

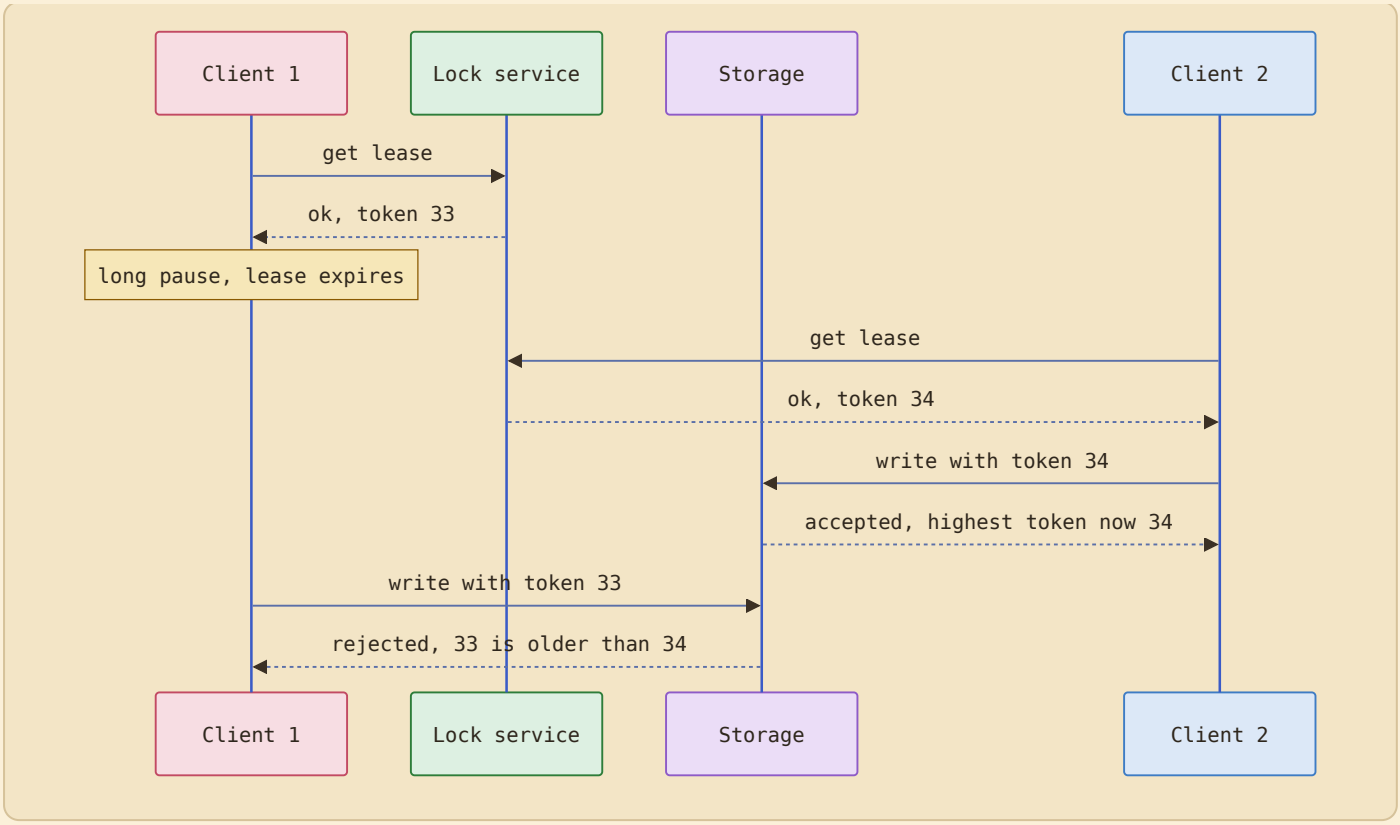
Goal. When a lock or lease protects a resource (like the storage in Figure 8-4), a node that **falsely believes** it is the chosen one must be **unable to disrupt** the system. A fairly simple technique is **fencing** (Figure 8-5).

How it works.

1. Whenever the lock service hands out a lock or lease, it includes a **fencing token** — a number that **increases on every grant** (e.g., incremented by the lock service).
2. Every write request a client sends to the storage service must **include its current token**.
3. The storage service **remembers the highest token it has processed** and **rejects** any write with an **older (lower) token**.

In Figure 8-5: client 1 gets the lease with token **33**, then pauses long and the lease expires. Client 2 gets the lease with token **34** and writes with 34. Client 1 wakes and writes with **33**. The storage server has already seen **34**, so it **rejects 33**.

Figure 8-5 (redrawn): storage accepts writes only in increasing token order, so the stale token 33 is rejected



With ZooKeeper as the lock service, the transaction ID `zxid` or node version `cversion` can serve as the fencing token, since both are guaranteed to be **monotonically increasing**.

Key requirement: the **resource itself** must **actively check tokens** and reject older ones. It is **not enough** to rely on clients checking their own lock status. For resources that don't support fencing tokens, you may work around it (e.g., for a file storage service, put the **token in the filename**). But **some check** is needed so requests outside the lock's protection are not processed.

Is server-side checking a burden? Arguably it's **good**: a service shouldn't assume clients always behave, since clients are often run by people with **very different priorities** from the service operators. Every service should **protect itself from accidentally abusive clients**.

Byzantine Faults

Limit of fencing. Fencing tokens catch a node acting wrongly **by accident** (e.g., it hasn't learned its lease expired). But a node that **deliberately** wanted to break the guarantees could simply send a **fake token**.

The book's assumption: nodes can be **unreliable, yet they are honest**. A node might be slow or never answer (a fault), and might hold outdated state (after a GC pause or network delay), but **whatever it does say is "true"** — as far as it knows, it is obeying the protocol.

Byzantine faults. Problems become far harder when nodes might **"lie"** — return arbitrary, broken, or corrupted answers, e.g., claim to have received a message they never got. That is a **Byzantine fault**, and reaching consensus in such an untrusting setting is the **Byzantine Generals Problem**.

> **The Byzantine Generals Problem (sidebar).** It generalises the **Two Generals Problem**: two army generals camped at different sites must agree on a battle plan but can only communicate by **messengers** who may be delayed or lost (like network packets); consensus is covered in Chapter 9. In the Byzantine version there are **n generals**, some of them **traitors**. Loyal generals send truthful messages; traitors try to deceive and confuse with fake messages while staying hidden, and **nobody knows in advance** who the traitors are.

> *Name origin:* **Byzantium**, an ancient Greek city, was later renamed **Constantinople**; today it is **Istanbul**, Turkey. There's no historical evidence its generals were more scheming than others; the name comes from "Byzantine" meaning **excessively complicated, bureaucratic, devious**, used in politics long before computers. **Lamport** was looking for a nationality nobody would be offended by, and someone warned him off the alternative name "**The Albanian Generals Problem**".

Byzantine fault tolerance. We call a system **Byzantine fault-tolerant** when it still behaves correctly even if some nodes **malfunction and disobey the protocol**, or **malicious attackers** interfere with the network. It matters in specific settings:

- **Aerospace.** **Radiation** can corrupt data in memory or CPU registers, making a computer respond unpredictably. Failure would be catastrophic (an aircraft crashing and killing everyone aboard, or a **rocket crashing into the International Space Station**), so **flight control systems must tolerate Byzantine faults**.
- **Multiple organisations.** Some participants may **cheat or defraud** others, so a node cannot simply trust another's messages. **Bitcoin and other blockchain peer-to-peer networks** can be viewed as a mechanism letting parties who distrust each other agree on whether a transaction took place, **without a central authority**.

Why this book usually ignores them.

- In **your own datacenter**, all nodes are controlled by your organisation (so hopefully trustworthy), and **radiation** is low enough that memory corruption isn't a major issue.
- Byzantine fault-tolerant protocols are **quite complicated**, and fault-tolerant embedded systems need **hardware-level support**. For most server-side data systems, the **cost makes them impractical**.
- **Web applications** certainly must anticipate arbitrary, even **malicious, behaviour from clients** (e.g., from browsers under users' control). That's why **input validation, sanitisation and output escaping** matter — which guard against attacks like **SQL injection** and **cross-site scripting**. But we don't use Byzantine protocols for that; the **server is the authority** deciding what clients may do. In **peer-to-peer networks** with no central authority, BFT is more relevant.
- **Software bugs** could count as Byzantine faults, but if **every node runs the same software**, a BFT algorithm can't help. Most BFT algorithms need a **supermajority — more than two-thirds** — of nodes working correctly (with **four nodes, at most one** may malfunction). Using this against bugs would require **four independent implementations** of the same software, hoping the bug is only in one.
- It would be nice if a protocol protected against **vulnerabilities, security compromises and attacks**, but that's unrealistic too: an attacker who compromises one node can probably compromise **all**, since they run the same software. **Traditional mechanisms** — **authentication, access control, encryption, firewalls**, etc. — remain the main defence.

Weak forms of lying

Even assuming honest nodes, it can pay to guard against **weak forms of "lying"** — invalid messages from **hardware issues, software bugs, misconfiguration**. These are not full BFT and would not stop a determined adversary, but they're **simple, pragmatic** steps:

- **Corrupted packets.** Packets occasionally get corrupted by faulty hardware or by bugs in OSes, drivers, routers and so on. **TCP and UDP checksums** usually catch this, but sometimes miss it. Simple fixes suffice, like **checksums in the application-level protocol**.
- **Input sanitising.** A **public** application must carefully check user input — e.g., values within a **reasonable range**, and **string length limits** to prevent **denial of service** through huge memory allocations. A service that is only reachable **internally**, behind a firewall, may relax these checks, but **basic sanity checks** (e.g., in protocol parsing) are still a good idea.
- **Multiple NTP servers.** NTP clients can be configured with **several server addresses**. It queries every one, **estimates how wrong each is**, and verifies that **most of them agree** on a time window. As long as most are fine, a misconfigured server reporting wrong time is detected as an **outlier** and excluded. Multiple servers make NTP **more robust** than a single one.

System Model and Reality

Many algorithms exist for distributed problems (e.g., **consensus**, Chapter 9). To be useful they must tolerate the faults described in this chapter, and they must be written **without depending too much on specific hardware and software details**. That requires **formalising** the faults we expect, via a **system model** — an abstraction describing **what an algorithm may assume**.

Timing assumptions — three common models:

Model	Assumption	Realistic?
Synchronous	**Bounded** network delay, **bounded** process pauses, **bounded** clock error. (Not perfectly synced clocks or zero delay — just a known fixed upper bound that is never exceeded.)	**No** for most practical systems — unbounded delays and pauses do happen
Partially synchronous	Behaves synchronously **most of the time**, but **sometimes exceeds** the bounds for delay, pauses and drift; then they can become **arbitrarily large**	**Yes** — a realistic model of many systems
Asynchronous	**No timing assumptions** at all — the algorithm **has no clock**, hence **no timeouts**	Possible to design for, but **extremely limiting**

Node failure assumptions — three common models:

Model	Assumption
Crash-stop faults	A node fails only by **crashing**: it stops responding at some moment and is **gone forever**, never returning.
Crash-recovery faults	Nodes may crash at any moment and **maybe resume** after an unknown time. They have **stable storage** (non-volatile disk) that **survives crashes**, but **in-memory state is lost**.
Byzantine (arbitrary) faults	Nodes may behave in **any way whatsoever**, even attempting to mislead other nodes.

When modelling actual systems, the combination that usually serves best is **partial synchrony plus crash-recovery faults**. But how do algorithms cope with it?

Correctness of an algorithm

To say what "correct" means, describe the algorithm's **properties**. For a **sorting algorithm**, the output has the property that for any two distinct elements, the one further **left is smaller** than the one further right — a formal definition of "sorted".

Likewise for a distributed algorithm. For **generating fencing tokens** for a lock, we might require:

- **Uniqueness:** no two token requests return the **same value**.
- **Monotonic sequence:** if request x returned token t_x and request y returned t_y, and **x completed before y began**, then **t_x < t_y**.
- **Availability:** a node that requests a token and **doesn't crash eventually** gets a response.

We call an algorithm **correct within a system model** when its properties hold in **every** scenario that the model permits. Yet if every node crashes, or every network delay becomes infinite, no algorithm can do anything — so how can this make sense?

Safety and liveness

Distinguish two kinds of properties. In the example, **uniqueness** and **monotonic sequence** are **safety** properties; **availability** is a **liveness** property.

A giveaway: liveness properties often contain the word **"eventually"**. (Yes — **eventual consistency is a liveness property**.)

Informal definitions: safety = **nothing bad happens**; liveness = **something good eventually happens**. Don't lean on these too hard, since "good" and "bad" are subjective. The **precise, mathematical** definitions are:

	Safety	Liveness
When violated	You can **point to a specific moment** it broke (e.g., the operation that returned a duplicate token)	It may be unmet at some moment (a request sent, no reply yet)
Can it recover?	**No** — once violated, the damage is done	**Yes** — there's always hope it will be satisfied later (the reply arrives)
Typical requirement	Must **always** hold in **all** situations of the system model — even if all nodes crash or the whole network fails, the algorithm must **never return a wrong result**	May come with **caveats** — e.g., a request must get a reply only if a **majority of nodes haven't crashed** and the network **eventually recovers**

The **partially synchronous model's definition** requires the system to **eventually return to a synchronous state** — any network interruption lasts only a **finite** time and then is repaired.

Mapping system models to the real world

System models and safety/liveness properties help reason about correctness, but when implementing, **messy reality bites again**: the model is a **simplification**.

- Crash-recovery algorithms usually assume data in **stable storage survives crashes**. But what if the **disk data is corrupted**, or **wiped** by hardware error or misconfiguration? Or suppose a **firmware bug** makes a server **not detect its own hard drives after rebooting**, although they are properly attached? (The body text does not name this incident; the book's references include a GitHub incident report for it.)
- **Quorum algorithms** (Chapter 5) depend on every node **retaining the data it said it stored**. If a node can suffer **amnesia** and forget data, the quorum condition breaks, and so does correctness. A new model might assume stable storage **mostly** survives but **sometimes** is lost — but that model is **harder to reason about**.

Theory vs. implementation. A theoretical description may simply **assume** certain things don't happen — and in non-Byzantine systems we must make some assumptions. But a real implementation may still need code for the "impossible" case, even if it just prints a rude message and exits with an odd error code — i.e., lets a **human operator** clean up. (The book suggests this may be exactly what separates **computer science** from **software engineering**.)

A tiny illustration of that "handle the impossible" idea (my own, in C):

```c
if (stored_version > proposed_version && is_leader) {
    fprintf(stderr, "invariant broken: stale leader wrote\n");
    exit(99);   /* stop and let an operator investigate */
}
```

- The condition describes a situation the algorithm's model says **cannot happen**.
- Instead of silently continuing, the program logs it and exits so a **human** can fix things.

Models are still valuable. Abstract system models are **incredibly helpful** for distilling real-world complexity into a **manageable set of faults** to reason about and solve systematically. Showing that an algorithm's properties always hold in a given model is how we **prove it correct**.

Proving correctness doesn't guarantee a real implementation always behaves correctly, but it is an **excellent starting point**: analysis can expose flaws that might stay hidden in a real system for a long time and bite only when assumptions (e.g., about timing) fail in unusual circumstances. **Theory and real-world testing matter just as much as each other.**

Chapter summary

(Book section: Summary)

Problems covered:

- **Networks:** any packet may be **lost or arbitrarily delayed**, and so may the reply. With no reply, you **can't tell** whether your message got through.
- **Clocks:** a node's clock can be **far out of step** with others even with NTP configured, may **jump forward or backward**, and relying on it is dangerous since you usually **don't know its error interval**.
- **Process pauses:** a process may **pause for a long time** at any point (e.g., stop-the-world GC), be **declared dead**, then come back **unaware** it was paused.

Partial failure is the **defining characteristic** of distributed systems: anything involving other nodes may occasionally fail, randomly slow down, or not respond (and eventually time out). We build **tolerance of partial failures** into software so the whole keeps working even when parts are broken.

- **Detecting faults is hard.** Most systems lack accurate failure detection, so most algorithms rely on **timeouts** — which **can't distinguish network from node failures**, and variable delays sometimes cause **false suspicion** of a crash.
- **Limping nodes.** A node can be **degraded** rather than dead — e.g., because of a **driver bug**, a **Gigabit network interface** abruptly falling to **1 Kb/s** of throughput. Such a "limping" node can be harder to handle than a cleanly failed one.
- **Tolerating faults is hard too.** There is **no global variable, shared memory, common knowledge or shared state** between machines. Nodes can't even agree on the **time**. Information flows only over the unreliable network. **Major decisions can't be made safely by one node**, so we need protocols that get a **quorum** to agree.
- **The shock.** Coming from a single computer's idealised, deterministic world, distributed systems' messy reality is jarring. Conversely, distributed-systems engineers often consider a problem **trivial if it can be solved on one computer** — and one computer can do a lot today. **If staying on one machine lets you keep Pandora's box shut, that is usually the better choice.**
- **But scalability isn't the only reason to distribute** (Part II introduction): **fault tolerance** and **low latency** (data near users) are equally important and **can't be achieved with one node**.
- **Unreliability isn't a law of nature.** Hard real-time response guarantees and bounded network delays are **possible**, but **very expensive** and lower hardware utilisation. Outside safety-critical domains, most systems pick **cheap but unreliable** rather than **reliable but costly**.
- **Supercomputers** assume reliable components and must be **stopped and restarted entirely** on a component failure. **Distributed systems can run forever** without service-level interruption, since faults and maintenance are handled **per node** — at least in theory (in practice, rolling a **bad configuration change to all nodes** still brings a distributed system to its knees).
- This chapter was all problems; **Chapter 9 moves to solutions** — algorithms designed to cope with them.

Interview angle

1. Why is a timeout the only practical way to detect failures, and how do you choose one? *30-second answer:* In an asynchronous network, a lost request, a dead node, a paused node and a lost reply all look identical: silence. Explicit signals (TCP RST/FIN, HBase-style crash scripts, switch queries, ICMP unreachable) help sometimes but can't be relied on. Too short a timeout causes false positives, duplicate actions and cascading failures; too long delays recovery. Measure round-trip distributions and choose experimentally, or adapt automatically with jitter-aware detectors like Phi Accrual (Akka, Cassandra).

- *Follow-up: Why can't we get bounded delays like the phone network?* Ethernet/IP are packet-switched for bursty traffic; circuits reserve fixed bandwidth (e.g., ISDN, 16 bits every 250 µs) and waste capacity. Bounded delay is possible with static partitioning or QoS, but it's a cost/utilisation trade-off not enabled in clouds.
- *Follow-up: If TCP acked my packet, did the server process it?* Not necessarily — the app may crash first. You need an application-level positive response.

2. Time-of-day vs monotonic clocks — which for what? *30-second answer:* Time-of-day (wall clock, `System.currentTimeMillis()`) gives a point in time since the 1970 epoch; synced by NTP, can jump backward, ignores leap seconds — bad for durations. Monotonic (`System.nanoTime()`) only moves forward, may be slewed up to 0.05%, meaningless absolute value, can't compare across machines — use it for timeouts and durations.

- *Follow-up: How accurate is NTP?* Limited by network delay: ~35 ms minimum over the internet with spikes around a second; drift at Google's assumed 200 ppm is 6 ms per 30 s or 17 s per day.

3. Why is last-write-wins with timestamps dangerous? *30-second answer:* Clock skew can make a causally later write carry an earlier timestamp (Figure 8-3: 42.003 vs 42.004 with under 3 ms skew), so it's silently dropped. LWW can't tell sequential from concurrent writes, and ties need tiebreakers that can also break causality. Cassandra and Riak use LWW. Use logical clocks/version vectors for ordering.

- *Follow-up: How does Spanner use clocks safely?* TrueTime returns [earliest, latest]; Spanner waits out the interval before committing so non-overlapping intervals give a definite order; GPS/atomic clocks per datacenter keep uncertainty around 7 ms.

4. How can a leader or lock holder be wrong about still holding its lease? How do you fix it? *30-second answer:* Process pauses — GC (sometimes minutes), VM suspend/live migration, context switches/steal time, disk I/O (even lazy classloading, EBS), swapping/thrashing, SIGSTOP. The node resumes believing its lease is valid and writes, as in the HBase bug. Fix: fencing tokens — monotonically increasing numbers issued with each lease (e.g., ZooKeeper zxid/cversion) that the storage checks, rejecting stale ones (33 after 34).

- *Follow-up: Do fencing tokens stop malicious nodes?* No — a lying node can fake a token. That's Byzantine fault tolerance, which needs >2/3 honest nodes and is usually not worth it inside one datacenter.

5. What system model do real systems use, and what are safety vs liveness? *30-second answer:* Usually partially synchronous timing with crash-recovery faults. Safety ("nothing bad happens") must always hold, and a violation is permanent and pinpointable (e.g., duplicate token). Liveness ("something good eventually happens") can have caveats like "if a majority is up and the network recovers". Eventual consistency is a liveness property.

- *Follow-up: Why use a majority quorum?* Only one majority can exist at a time, so decisions can't conflict; 3 nodes tolerate 1 failure, 5 tolerate 2.

Self-check

1. **List the six things that can happen when you send a request and get no reply.** Request lost; request queued for later; remote node crashed; remote node temporarily paused (e.g., GC); request processed but reply lost; request processed but reply delayed.
2. **How do supercomputers handle a node failure, and why doesn't that work for internet services?** They checkpoint, stop the whole job, fix the node and restart from the checkpoint — turning partial failure into total failure. Internet services must stay online with low latency, run on failure-prone commodity hardware, and at thousands of nodes something is always broken.
3. **What did the medium-sized datacenter study find?** About 12 network faults per month; half disconnected a single machine and half an entire rack.

4. **Scenario:** you set a very short timeout on an overloaded cluster. What might happen? Slow-but-alive nodes are declared dead, their load shifts to others, which then get slower and are also declared dead — a cascading failure; actions like sending emails may also be performed twice.
5. **Why is the ISDN phone network able to guarantee delay while Ethernet can't?** ISDN reserves 16 bits per frame (4,000 frames/s) along the whole route for each call, so there's no queueing and delay is bounded. Ethernet/IP are packet-switched, sharing bandwidth dynamically, so queues form and delays are unbounded.
6. **Trade-off:** why would a VoIP app choose UDP over TCP? Late audio is worthless; UDP avoids retransmission and flow control delays, and the app fills gaps with silence instead.
7. **Why does Google's 200 ppm drift assumption matter, and what are the numbers?** It bounds achievable accuracy: 6 ms drift with resync every 30 s, 17 s with resync once a day.
8. **What is a clock confidence interval and which API exposes it?** A range in which the true time lies (e.g., 95% sure it's 10.3–10.5 s past the minute). Google Spanner's TrueTime returns [earliest, latest]; `clock_gettime()` doesn't expose it.
9. **Scenario:** a storage service receives a write with fencing token 41 after already processing one with token 42. What should it do and why must the storage do it? Reject it — token 41 comes from a holder whose lease has expired. The resource itself must check, because a paused client can't know its lease has lapsed.
10. **Classify: "no two nodes are ever leader for the same term" and "every request eventually gets a response." Which can have caveats?** The first is safety (must always hold); the second is liveness, which may be conditioned on a majority being up and the network eventually recovering.

Chapter 9 — Consistency and Consensus

Book: Chapter 9, pp. 321–383

Overview

Chapter 8 was a catalogue of everything that breaks in a distributed system: packets get dropped, duplicated, reordered or delayed for an unknown time; clocks drift; processes freeze for garbage collection or crash outright. Chapter 9 asks the constructive follow-up question: **given all of that, how do we build systems that keep working correctly anyway?**

The cheapest answer is "just let the whole service go down and show an error". When that is not acceptable, we need *fault tolerance* — the service keeps behaving correctly even though some parts inside it have failed.

The book's strategy mirrors what it did with transactions in Chapter 7. A transaction lets an application *pretend* that crashes don't happen (atomicity), that nobody else is touching the data at the same time (isolation), and that disks never lose data (durability). The faults still happen; the abstraction just hides them. Chapter 9 hunts for similar **general-purpose abstractions for distributed systems**: guarantees that are implemented once, carefully, and then relied on by many applications.

The single most important such abstraction is **consensus**: getting several nodes to agree on something. A classic use: a single-leader database loses its leader, and the remaining nodes must agree on *one* new leader. If two nodes both think they are leader (**split brain**), both accept writes and data is usually lost. Consensus done correctly prevents that — but doing it correctly despite network faults and crashes turns out to be surprisingly hard.

The chapter builds up to consensus in three steps:

1. **Linearizability** — among the strongest consistency models that real systems offer: replicated data behaves as if there were only one copy. What it buys you and what it costs.
2. **Ordering guarantees** — causality, total orders, Lamport timestamps, and **total order broadcast**, which turns out to be deeply linked to both linearizability and consensus.
3. **Distributed transactions and consensus** — how two-phase commit (2PC) makes a transaction atomic across several nodes, why 2PC is a weak form of consensus, and how real fault-tolerant consensus algorithms (Paxos, Raft, Zab, Viewstamped Replication) and services built on them (ZooKeeper, etcd) work.

Along the way the chapter also explains the *limits* of what is possible — results such as the CAP theorem and the FLP impossibility result — at an intuitive level rather than with formal proofs.

> **Opening epigraph.** The chapter opens with a quote from Jay Kreps (2013) asking, roughly, whether it is better to be available but wrong or correct but down. That tension — correctness vs. availability — runs through the entire chapter.

Everyday analogy. Imagine a group of friends deciding where to eat over a flaky group chat. Messages arrive late or out of order, and some friends' phones die mid-conversation. *Linearizability* is like everyone reading from one shared whiteboard: whatever is written there is the single truth. *Causality* is the rule that nobody should see "Great, see you there!" before seeing the message naming the restaurant. *Consensus* is the group actually settling on one restaurant — everyone ends up with the same answer, nobody changes their mind later, and the group can still decide even if a couple of phones die.

Key terms

Term	Plain-English meaning
Fault tolerance	Keeping the service working correctly even though some components (nodes, network links) have failed.
Consensus	Getting several nodes to agree on one value (e.g. who the leader is), irrevocably, even when some nodes fail.
Split brain	Two nodes both believe they are the leader; both accept writes and the data diverges.
Eventual consistency (convergence)	If writes stop, all replicas will *eventually* return the same value — but with no bound on when.
Consistency model	A precise promise about what values reads can return when data is replicated and accessed concurrently.
Linearizability	Also called atomic consistency, strong consistency, immediate consistency or external consistency. The system behaves as if there were one copy of the data and every operation happens atomically at a single instant; a *recency* guarantee.
Register	In the theory literature, a single object that can be read and written — one key, one row, or one document.
Regular register	A register whose reads, if they overlap a write, can give back the previous value or the new one (and may flip-flop). Weaker than linearizable.
Compare-and-set (CAS)	Atomic operation: "set x to *new* only if x currently equals *old*; otherwise fail".
Serializability	Transaction isolation property: transactions behave as if run one after another in *some* order (Chapter 7).
Strict serializability / strong one-copy serializability (strong-1SR)	Serializability and linearizability together.
Quorum	A subset of nodes (usually a majority) whose agreement is required; in Dynamo-style systems, w + r > n.
CAP theorem	Informal rule, named by Eric Brewer in 2000: when the network is partitioned you must choose between linearizability and total availability.
Network partition	A network fault where nodes are alive but can't talk to each other. (Not the same as data partitioning/sharding.)
CP / AP	Labels for "consistent but not available when partitioned" vs. "available but not consistent when partitioned". The book advises avoiding them.
Memory barrier / fence	A CPU instruction that forces caches and memory to be synchronised so another core sees the write.
Causality	Cause-before-effect ordering: a question before its answer, a row's creation before its update.
Happens-before	A happened before B if B could have known about, depended on, or built upon A.
Concurrent operations	Neither happened before the other; there is no causal link.
Causally consistent	A system that respects the order imposed by causality.
Total order	Any two elements can be compared (like numbers).

Term	Plain-English meaning
Partial order	Some pairs are comparable, some are not (like sets; like causality).
Version vector	Per-node counters used to tell whether two operations are concurrent or causally related (Chapter 5).
Sequence number / logical clock	A counter-based number that orders operations without relying on a physical time-of-day clock.
Lamport timestamp	A pair (counter, node ID) where every node bumps its counter to the largest value it has seen; gives a total order consistent with causality.
Total order broadcast (atomic broadcast, total order multicast)	A messaging protocol guaranteeing every node receives the same messages in the same order, with none lost.
State machine replication	Replicas stay identical by applying the same deterministic writes in the same order.
Fencing token	A number that increases each time a lock is granted, used to reject requests from stale lock holders.
zxid	ZooKeeper's monotonically increasing transaction ID, usable as a fencing token.
Sequential consistency (timeline consistency)	Everyone sees operations in the same order, but reads may be stale; slightly weaker than linearizability.
Chain replication	A replication scheme where reads can go to a replica that is updated synchronously on writes.
Atomic commit	Getting every node in a distributed transaction to agree to all commit or all abort.
Two-phase commit (2PC)	The classic atomic commit algorithm: a *prepare* phase (vote) then a *commit/abort* phase.
Coordinator (transaction manager)	The component that runs 2PC, collects votes and records the decision.
Participant	A database node taking part in a distributed transaction.
Commit point	The moment the coordinator durably writes its commit/abort decision to its log.
In doubt (uncertain)	A participant that voted "yes" but hasn't heard the coordinator's decision; it can neither commit nor abort on its own.
Blocking atomic commit	A protocol (like 2PC) that can get stuck waiting for a crashed node to recover.
Three-phase commit (3PC)	A nonblocking alternative to 2PC that only works if network delays and response times are bounded.
Perfect failure detector	A mechanism that always correctly tells whether a node has crashed; impossible with unbounded network delay.
XA (X/Open eXtended Architecture)	A 1991 standard C API for running 2PC across different technologies (databases, message brokers).
JTA / JDBC / JMS	Java Transaction API (Java's XA binding), and the Java database and messaging driver APIs that support it.
Heterogeneous distributed transaction	A distributed transaction spanning different systems (e.g. a database and a message broker).

Term	Plain-English meaning
Database-internal distributed transaction	A distributed transaction among nodes of the same database software.
Exactly-once processing	Ensuring a message's effects happen once in total, even if processing is retried.
Heuristic decision	An XA escape hatch letting a participant commit or abort an in-doubt transaction on its own — a polite word for "probably breaking atomicity".
Orphaned transaction	An in-doubt transaction whose outcome the coordinator can no longer determine; needs manual resolution.
FLP result	Fischer, Lynch and Paterson's proof that no deterministic algorithm can always reach consensus if a node may crash, in the asynchronous model (no clocks/timeouts).
Uniform agreement / Integrity / Validity / Termination	The four formal properties of consensus (explained below).
Safety vs. liveness	Safety = "nothing bad ever happens"; liveness = "something good eventually happens". Termination is liveness; the rest are safety.
Byzantine fault	A node that behaves arbitrarily or maliciously (e.g. sends contradictory messages).
Epoch number (ballot / view / term number)	A monotonically increasing number identifying a leadership period; the leader is unique within an epoch.
Multi-Paxos	The optimisation of Paxos that decides a sequence of values instead of one.
Dynamic membership	Consensus extensions that let the set of voting nodes change.
Lease	A lock with an expiry time, so it is released automatically if the holder dies.
Ephemeral node	A ZooKeeper entry tied to a client session; it disappears when the session times out.
Watch / change notification	A subscription that tells a client when a ZooKeeper value changes.
Service discovery	Finding which network address to connect to for a named service.
Membership service	A service that decides which nodes are currently alive members of a cluster.

Consistency Guarantees

The problem. In Chapter 5 ("Problems with Replication Lag") we saw that if you look at two replicas at the same instant you will often see different data, because each write reaches each replica at a slightly different time. This happens with *every* replication style — single-leader, multi-leader and leaderless.

Eventual consistency. Most replicated databases promise at least *eventual consistency*: stop writing, wait some unspecified amount of time, and eventually every read returns the same value. The inconsistency is temporary and heals itself (once any network faults are repaired too). The book suggests **convergence** would be a better name, since what is promised is that replicas converge to one value.

Why it is a weak guarantee. It says nothing about *when* convergence happens. Until then a read could return anything — or nothing. For example, you write a value and immediately read it back, but the

read goes to another replica and you don't see your own write (the "read your own writes" problem from Chapter 5).

Why it is hard for developers. A database looks like a variable you can assign and read, but it doesn't behave like one. In an ordinary single-threaded program, if you set `x = 5` and then read `x`, you never get the old value, and the read never fails. With eventual consistency you might. Bugs caused by this are subtle, rarely show up in testing (things usually work), and appear only under faults such as a network interruption or under heavy concurrency.

Stronger models cost something. The chapter explores *stronger* consistency models. They are easier to program against correctly, but they may perform worse or be less fault-tolerant. Seeing several models lets you pick the one that fits your needs.

> **Consistency models vs. isolation levels.** There is some resemblance to the hierarchy of transaction isolation levels in Chapter 7, and some overlap, but they are largely separate concerns. *Isolation* is about avoiding race conditions between concurrently running transactions. *Distributed consistency* is about coordinating the state of replicas despite delays and faults.

Roadmap of the chapter (these topics look separate but turn out to be tightly connected):

- Linearizability — one of the strongest common models — and its pros and cons.
- Ordering of events, especially causality and total ordering.
- Atomically committing a distributed transaction, which leads to solutions for consensus.

Linearizability

The idea. With eventual consistency, asking two replicas the same question at the same time can give two different answers. What if the database could fake having *just a single replica* — a single copy of everything? Every client would then see identical data and nobody has to think about replication lag.

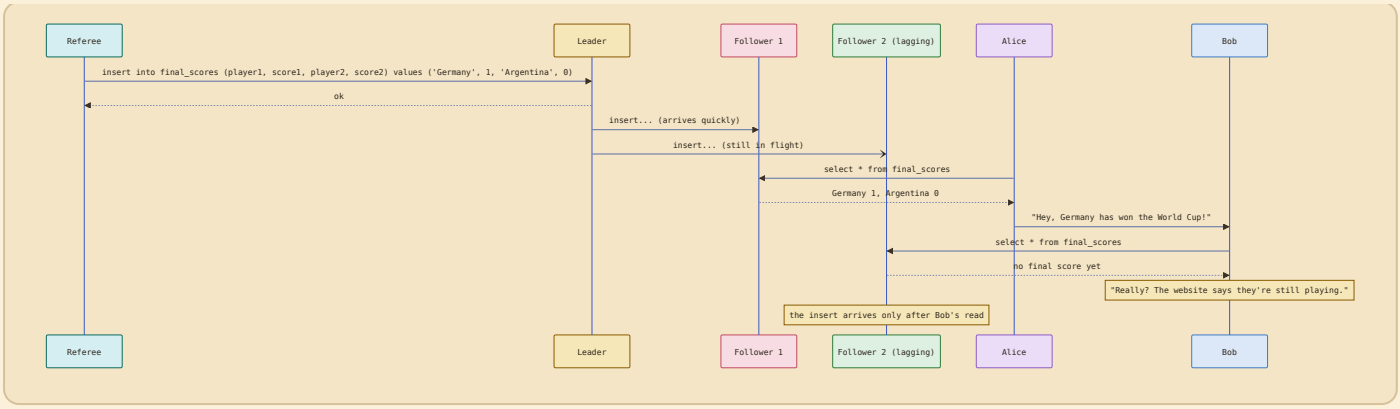
That illusion is **linearizability** (also called *atomic consistency*, *strong consistency*, *immediate consistency*, or *external consistency*). Informally:

- the system appears to hold a **single copy** of the data, and
- every operation on it appears to happen **atomically**, at one instant.

In reality there may be many replicas, but the application can ignore them.

Recency guarantee. In a linearizable system, the moment one client's write *completes successfully*, every client that reads afterwards must see that value (or a newer one). Reads must never come from a stale cache or lagging replica. So linearizability is best understood as a **recency guarantee**.

Figure 9-1 (redrawn): a non-linearizable sports website. The referee's final score reaches follower 1 quickly but follower 2 late; Alice reads follower 1 and announces the result, then Bob reads the lagging follower 2 and is told the game is still on.



The example (Figure 9-1). Alice and Bob sit in the same room, both watching for the result of the 2014 FIFA World Cup final on their phones. Just after the result is published, Alice refreshes, sees the winner, and tells Bob. Bob refreshes too, but his request lands on a replica that is lagging, so his phone says the match is still being played.

If they had refreshed at *exactly* the same time, getting different answers would be less surprising — neither knows precisely when the server handled their request. But Bob *knows* he refreshed *after* hearing Alice, so he reasonably expects a result at least as fresh as hers. Getting an older answer is a **violation of linearizability**.

What Makes a System Linearizable?

"Behave as if there is one copy" sounds simple, but pinning down exactly what it means takes care. The book works through timing diagrams.

Setup. Three clients read and write the same key `x`. Theory papers call `x` a **register**. In a real system it might be a single key (key-value store), a single row (relational table) or a single document (document database).

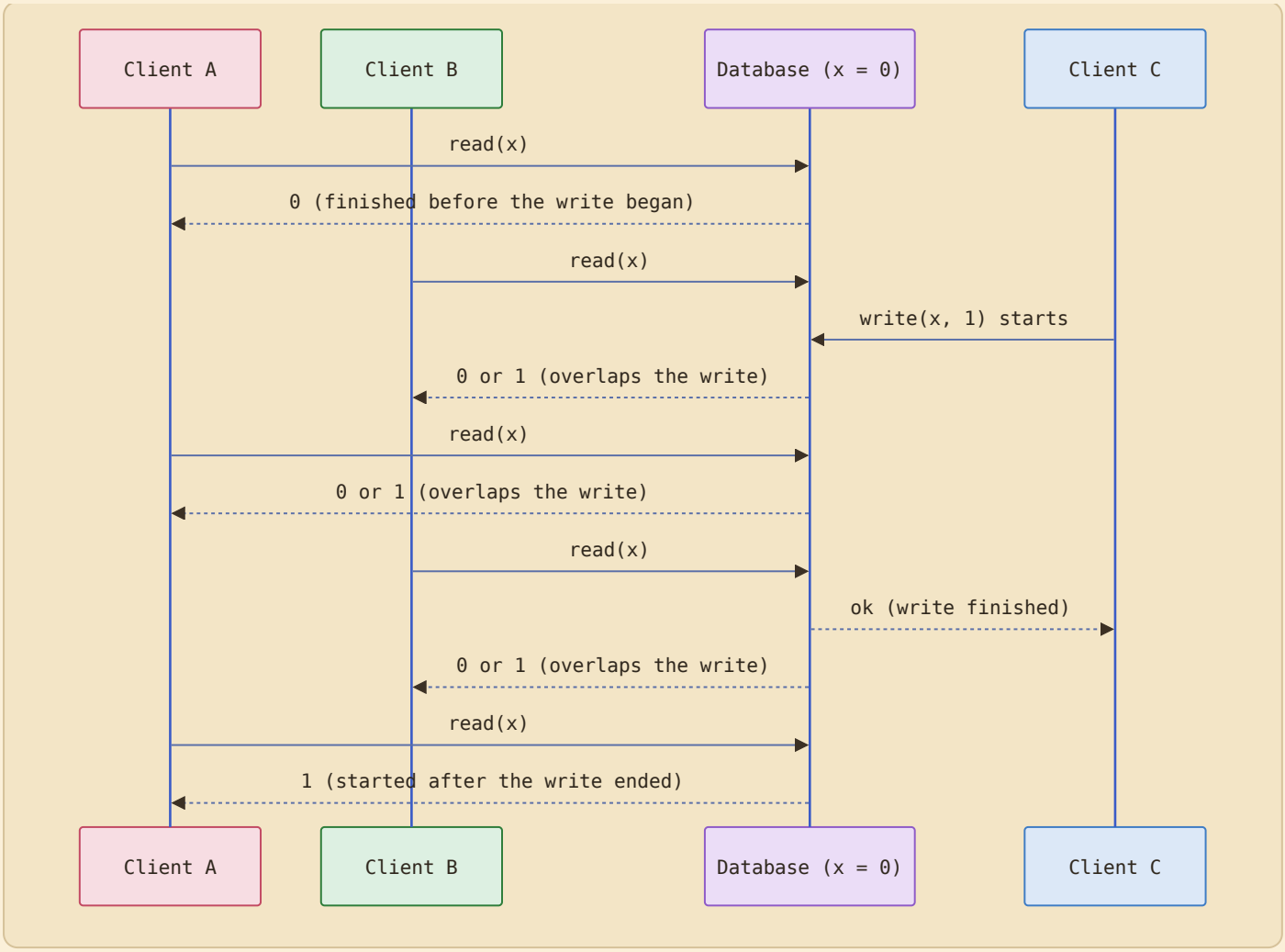
In the diagrams, each operation is a **bar**: the left end is when the client *sent* the request and the right end is when it *received* the response. Because network delays vary, the client doesn't know exactly when the database actually executed the request — only that it happened *somewhere between* sending and receiving.

> **Aside: the imaginary global clock.** These diagrams draw a horizontal time axis as if a perfect global clock existed. Real systems don't have one (Chapter 8), but for *analysing* an algorithm it is fine to pretend such a clock exists, as long as the algorithm itself cannot use it — the algorithm only sees the imperfect local clocks (quartz oscillator plus NTP).

Operation notation:

- `read(x) ⇒ v` — the client read register `x` and the database returned `v`.
- `write(x, v) ⇒ r` — the client asked to set `x` to `v`, and the database replied `r` (`ok` or `error`).

Figure 9-2 (redrawn): client C writes x = 1 while A and B keep reading. A's first read ends before the write starts (0); A's last read starts after the write ended (1); the three reads that overlap the write may return either 0 or 1.



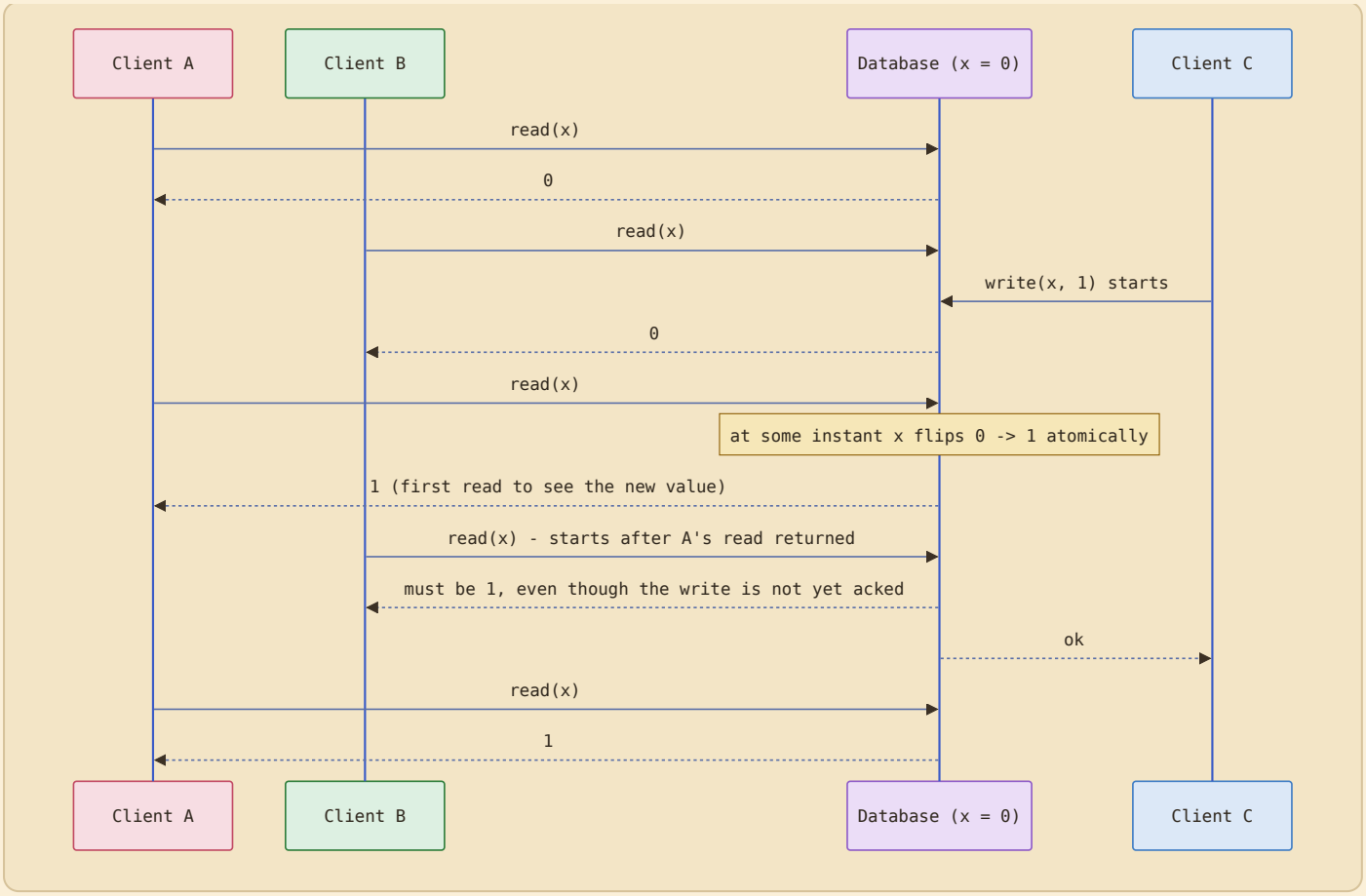
Walking through Figure 9-2. Initially `x = 0`. Client C writes `x = 1`, while clients A and B keep polling `x`:

1. A read that **finishes before the write starts** must return the old value, **0**.
2. A read that **starts after the write has finished** must give back **1**, assuming the database is linearizable. Reasoning: the write took effect somewhere inside its bar; the read took effect somewhere inside its bar; if the read's bar begins after the write's bar ends, the read necessarily happened after the write.
3. A read whose bar **overlaps** the write's bar may return **either 0 or 1**, because we can't tell whether the write had taken effect when the read was processed. Such reads are *concurrent* with the write.

That's not enough yet. If overlapping reads can return old *or* new, a reader could watch `x` flip 0 → 1 → 0 → 1 while the write is in progress. A true "single copy" would never do that.

> **Regular register.** If reads that overlap a write are allowed to give back either the previous or the new value, the register is called a *regular register*. It is weaker than a linearizable one.

Figure 9-3 (redrawn): same clients as Figure 9-2, with the extra linearizability constraint. A's second read is the first to return 1; B's second read starts after that and so must also return 1, although C's write has not been acknowledged yet; all later reads return 1.



The extra rule (Figure 9-3). In a linearizable system we imagine an instant — somewhere inside the write's bar — at which `x` **flips atomically** from 0 to 1. Therefore:

> If any client's read has returned the new value, every read beginning *after that read completed* has to see the new value as well — even while the write itself is still unacknowledged.

In the figure, A is first to read 1. B's read starts strictly after A's read returned, so B must also see 1, though C's write is still in flight. This is exactly the Alice-and-Bob situation.

Adding compare-and-set (Figure 9-4). A third operation type:

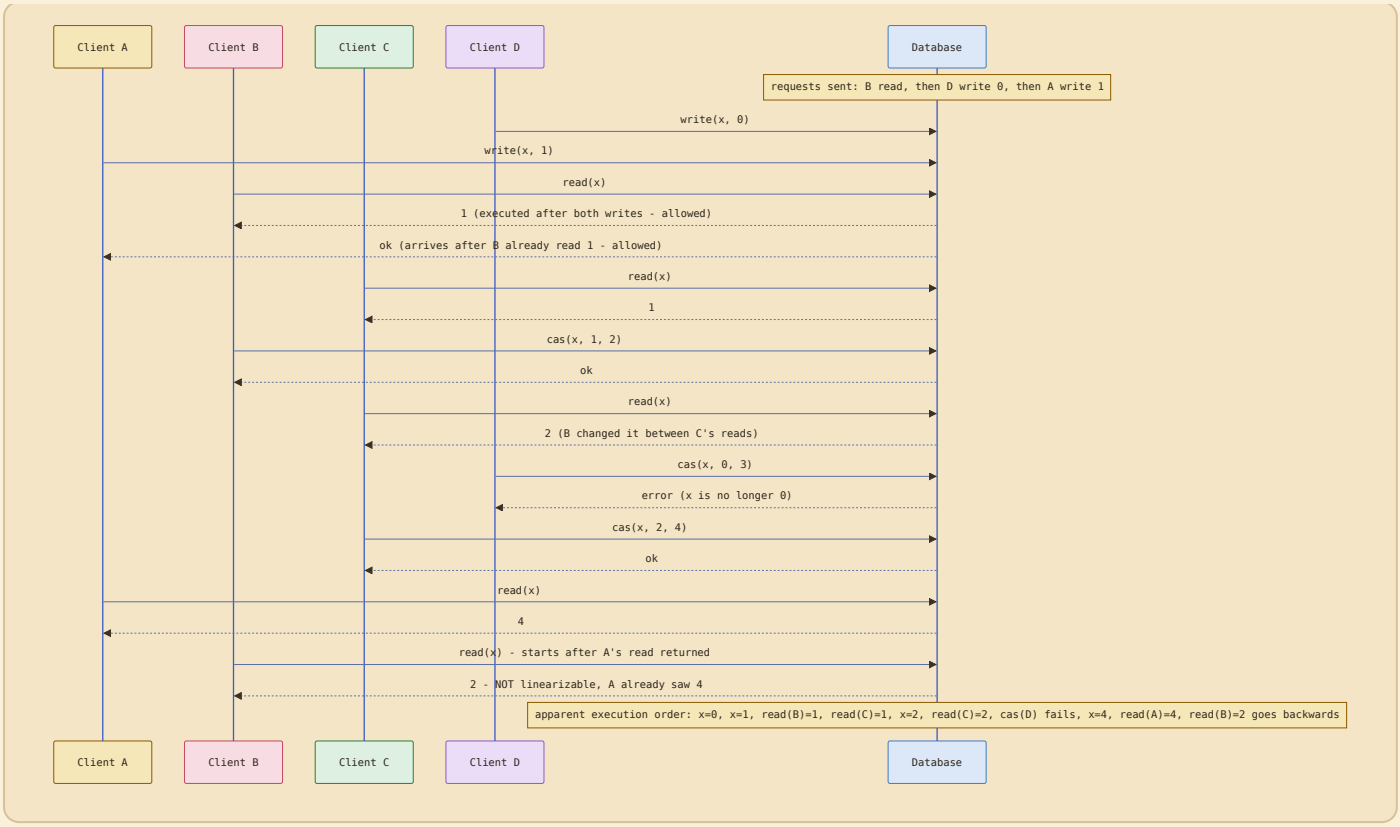
- `cas(x, v_old, v_new) ⇒ r` — atomic compare-and-set: if `x` currently equals `v_old`, set it to `v_new`; if not, leave `x` unchanged and return an error. `r` is `ok` or `error`.

The book then marks, inside each bar, a vertical line at the instant the operation *appears* to have executed, and joins these markers in order. Two requirements:

1. The joined-up sequence must be a **valid sequential history** for a register: each read returns the value of the most recent write before it.
2. The line joining the markers must **always move forward in time** (left to right), never backwards.

Requirement 2 is what enforces recency: once a value has been written *or read*, all later reads see it until it is overwritten.

Figure 9-4 (redrawn): the order in which the database appears to execute operations; B's last read violates linearizability



Interesting details in Figure 9-4:

- **Order of processing ≠ order of sending.** B sent a read first, then D sent `write(x, 0)`, then A sent `write(x, 1)`. Yet B's read returned 1. That's fine: the database handled D's write first, A's write second, and B's read last. Because the three requests were concurrent, any order is acceptable — perhaps B's request was delayed in the network.
- **A read can see a write before the writer hears "ok".** B read 1 *before* A got its `ok`. That's not reading a value before it exists — A's acknowledgment was merely delayed on its way back.
- **No transaction isolation is assumed.** Other clients may change the value at any time: C read 1 and later 2, because B changed it in between. A `cas` lets you check nobody else changed the value: B's and C's `cas` succeed, D's fails because `x` was no longer 0 when D's request was processed.
- **B's last read breaks linearizability.** It overlaps C's `cas` from 2 to 4, so on its own returning 2 would be fine. But A had already read 4 *before B's read began*, so B must not see anything older than 4. Same pattern as Alice and Bob.

Testing for linearizability. The formal definition is more precise than this intuition (the paper the book cites for it, in its reference list, is Herlihy and Wing's 1990 "Linearizability" paper). You *can* check whether a system behaved linearizably by recording the timing of every request and response and searching for a valid sequential order that fits — possible, but computationally expensive.

(Not from the book text — context: the book's reference for this is Kyle Kingsbury's write-up on his Knossos checker. In general such checkers have to search through the possible orderings of the recorded operations, which is why the check gets expensive as histories grow.)

> **Linearizability versus serializability** (a sidebar the book stresses). Both sound like "can be put in a sequential order", but they are very different guarantees:

	Serializability	Linearizability
Kind of guarantee	*Isolation* property of **transactions**	*Recency* guarantee on reads/writes of a **single register**
Scope	Transactions that read/write **many** objects	One object at a time; no grouping into transactions
Promise	Result equals *some* serial order of transactions — which may differ from the real order they ran in	Every operation appears to take effect at one instant between its start and end, respecting real time
Prevents write skew?	Yes	No, not by itself (unless you add something like materialising conflicts)

A database offering **both** is said to provide *strict serializability* (also called *strong one-copy serializability*, strong-1SR). Serializability implemented via **two-phase locking** or **actual serial execution** is typically linearizable too. **Serializable snapshot isolation (SSI) is not**: it reads from a consistent snapshot on purpose (to avoid reader/writer lock contention), and a snapshot by definition excludes writes newer than itself.

Relying on Linearizability

Seeing a football score a few seconds late causes no real harm. So when does linearizability actually *matter*? The book gives three areas.

Locking and leader election

A single-leader system must make sure there is really just **one** leader (no split brain). One way to elect a leader is with a **lock**: every node tries to acquire it at startup, and whoever succeeds becomes leader. However the lock is built, it **must be linearizable** — every node has to agree who holds it, or the lock is useless.

- **Coordination services** like **Apache ZooKeeper** and **etcd** are often used for distributed locks and leader election. Internally they use consensus algorithms to provide linearizable operations in a fault-tolerant way.
- Getting locks and leader election right still has subtle traps — e.g. the *fencing* problem from Chapter 8 ("The leader and the lock"). Libraries such as **Apache Curator** add higher-level recipes on top of ZooKeeper.
- Still, a linearizable storage service is the foundation all of this is built on.

> **Footnote: ZooKeeper/etcd reads can be stale.** Strictly speaking these systems make *writes* linearizable, but by default a read may be served by any replica and therefore be stale. You can ask for a linearizable read: etcd calls it a **quorum read**; in ZooKeeper you call `sync()` before reading.

Locks can also be very fine-grained. **Oracle Real Application Clusters (RAC)** takes out a separate lock for **each disk page**; several database nodes share access to one disk storage system. Because these linearizable locks sit on the critical path of every transaction, RAC deployments usually have a **dedicated cluster interconnect network** just for traffic between database nodes.

Constraints and uniqueness guarantees

Typical uniqueness constraints: a username or email address identifies exactly one user; a file storage service cannot hold two files with the same path and name. If you want to enforce this **at write time** — so that when two people try to create the same name simultaneously, one gets an error — you need linearizability.

- This is really a lock: registering a username is like taking a "lock" on that name.
- It is also like a **compare-and-set**: set `username → user_id` only if the username is currently unclaimed.

Similar constraints: a bank balance must never go negative; you must not sell more items than are in stock; a flight seat or theatre seat must not be sold to two people at once. All of these need **one up-to-date value that every node agrees on** (the balance, the stock level, the seat's occupant).

In practice some such constraints can be treated *loosely* — an overbooked flight can move passengers and compensate them — and then linearizability may not be needed (Chapter 12, "Timeliness and Integrity"). But a **hard uniqueness constraint**, like those in relational databases, does require linearizability. Other constraints, such as **foreign keys** or **attribute constraints**, can be enforced *without* linearizability.

A small SQL illustration (own example):

```sql
CREATE TABLE accounts (
  handle  TEXT PRIMARY KEY,    -- hard uniqueness constraint
  user_id BIGINT NOT NULL
);

-- Two sign-ups race for the same handle:
INSERT INTO accounts VALUES ('river_otter', 101);  -- session 1
INSERT INTO accounts VALUES ('river_otter', 202);  -- session 2
-- Exactly one succeeds; the other gets a unique-violation error.
```

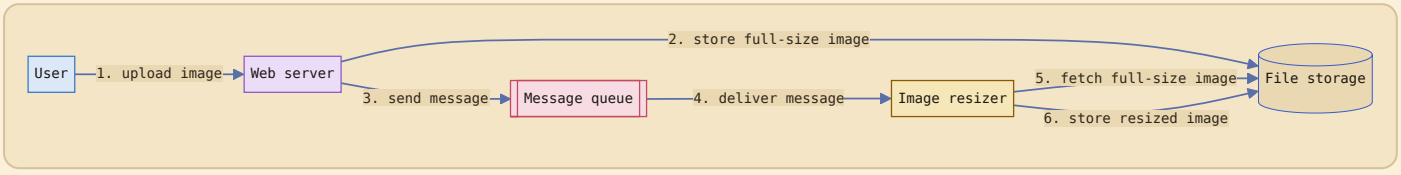
- `PRIMARY KEY` declares that `handle` must be unique.
- On a single node the database decides the winner locally. On a replicated system, deciding "who was first" *immediately* for every node is the job that needs linearizability (effectively a compare-and-set on the key).

Cross-channel timing dependencies

Notice in Figure 9-1 that Bob only *noticed* the stale result because Alice told him out loud. Without that, he'd have refreshed again later and seen the score. The violation was visible only because there was a **second communication channel** (Alice's voice) running alongside the database.

The same thing happens in software. Example: a website where users upload photos, and a background job makes smaller thumbnail versions.

Figure 9-5 (redrawn): web server and image resizer communicate through two channels, file storage and a message queue, so the message can outrun the storage's replication.



How it works:

1. The web server writes the full photo to the **file storage** service. (The photo isn't put on the queue itself, since brokers are designed around small messages, whereas a photo can be several megabytes.)
2. Once that write is done, the web server puts a small "please resize photo X" message on the **message queue** (Chapter 11).
3. The resizer receives the message and fetches the photo from file storage.

The race. If the file storage is linearizable, all is well. If it isn't, the queue (steps 3–4) might be *faster than the storage service's internal replication*. At step 5 the resizer may then get a

previous version of the image, or find nothing at all. If it resizes an old version, the full-size and thumbnail images become **permanently inconsistent**.

The root cause: two channels (storage and queue) between the same two components, and without linearizability's recency guarantee they can race. It's the Alice-and-Bob situation again (database replication vs. a spoken voice).

Linearizability is the *simplest* fix but not the only one. If you control the second channel (you control the queue; you don't control Alice's voice), you can use techniques similar to those for "read your own writes" in Chapter 5, at the price of extra complexity. (**Own illustration, not from the book:** the web server could put the stored photo's version number in the queue message; the resizer then checks that the copy it fetched is at least that version, and waits/retries if it is older. The book only says "techniques similar to read-your-own-writes" and gives no concrete mechanism.)

Implementing Linearizable Systems

Since linearizability means "act as if there is one copy and operations are atomic", the most obvious implementation is to **literally keep one copy**. But then a single node failure loses the data, or at least makes it unavailable until the node returns. So we need replication — and the question becomes which replication methods from Chapter 5 *can* be linearizable:

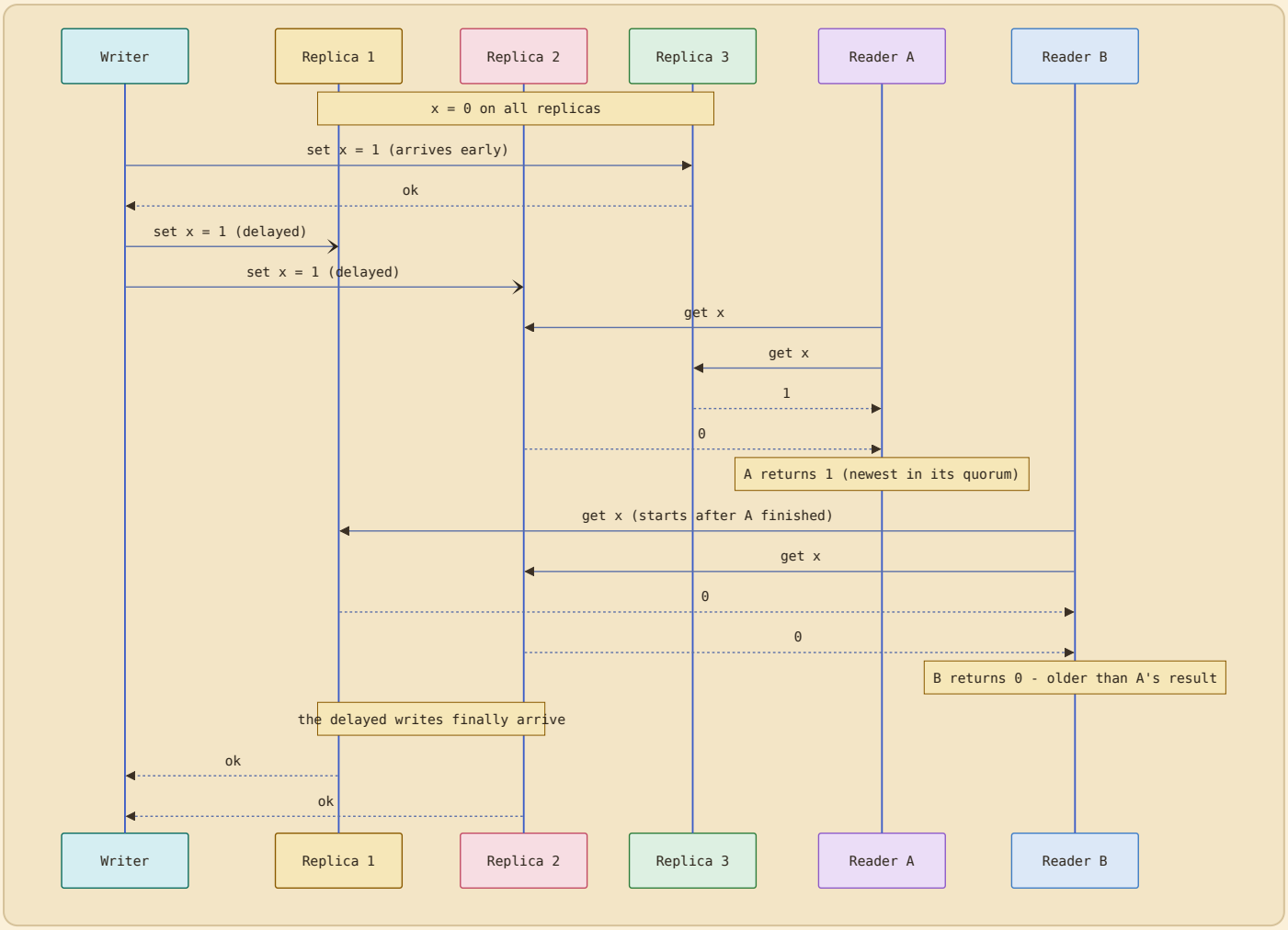
Replication method	Linearizable?	Why
Single-leader	*Potentially*	Reading from the leader (or from followers updated synchronously) can be linearizable. But not every single-leader database is (e.g. it uses snapshot isolation by design, or has concurrency bugs). It also relies on knowing for sure who the leader is: a node may wrongly believe it is still leader ("The Truth Is Defined by the Majority", Chapter 8), and a delusional leader serving requests will likely break linearizability. With asynchronous replication, failover can even lose committed writes, violating both durability and linearizability.
Consensus algorithms	*Yes*	Some resemble single-leader replication, but include safeguards against split brain and stale replicas, so they can implement linearizable storage safely. **ZooKeeper** and **etcd** work this way.
Multi-leader	*No*	Writes are processed concurrently on several nodes and replicated asynchronously, producing conflicting writes that need resolution — conflicts that exist precisely because there's no single copy.
Leaderless (Dynamo-style)	*Probably not*	People sometimes claim quorum reads/writes (w + r > n) give "strong consistency", but this isn't quite true. **Last write wins** using time-of-day clock timestamps (as in **Cassandra**) is almost certainly non-linearizable, because clock skew means timestamps may not match the real order. **Sloppy quorums** destroy any chance of linearizability. Even *strict* quorums can behave non-linearizably (next section).

> **Footnote: sharding doesn't break it.** Splitting a single-leader database into partitions, each with its own leader, doesn't affect linearizability, because linearizability is a *single-object* guarantee. Transactions that span partitions are a different matter (covered later in the chapter).

Linearizability and quorums

It feels as if strict quorums in a Dynamo-style system should be linearizable. With variable network delays, they aren't.

Figure 9-6 (redrawn): a non-linearizable execution despite a strict quorum (n = 3, w = 3, r = 2). The write reaches replica 3 early but replicas 1 and 2 late; reader A sees 1 via replica 3, then reader B, starting after A finished, reads replicas 1 and 2 and gets 0 from both.



Step by step:

1. `x = 0` everywhere. A writer updates `x` to 1, sending the write to each of the three replicas (n = 3, w = 3).
2. While the write is in progress, reader A reads from two replicas (r = 2). One of them already has 1, so A sees the new value.
3. Also during the write, but *after A's read finished*, reader B reads from a different pair of replicas, both still holding 0, so B sees the old value.
4. The quorum condition w + r > n holds (3 + 2 > 3) — yet B started after A completed and got an *older* value. Not linearizable. Alice and Bob again.

Can it be fixed? Yes, at a performance cost:

- A **reader** must perform **read repair synchronously** — push the newest value to stale replicas — *before* returning its result.
- A **writer** must **read the latest state from a quorum** before sending its write.

What real systems do:

- **Riak** does *not* do synchronous read repair, because of the performance penalty.
- **Cassandra** *does* wait for read repair on quorum reads, but still loses linearizability when several clients write the same key concurrently, because it uses last-write-wins conflict resolution.
- Even with these fixes, only linearizable **reads and writes** can be built this way. A linearizable **compare-and-set** cannot, because CAS requires a consensus algorithm.

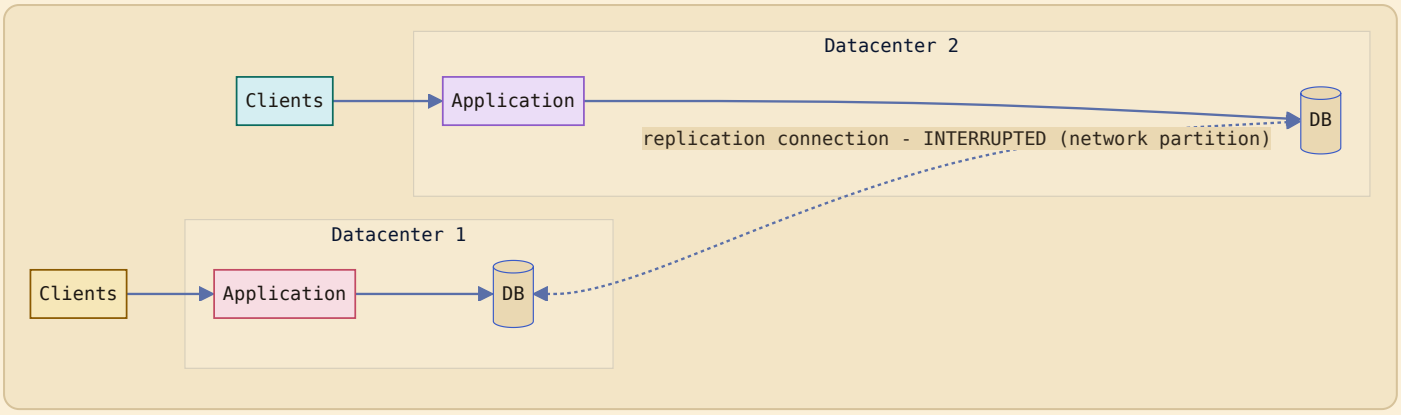
Bottom line: it is safest to assume a leaderless Dynamo-style system is **not** linearizable.

The Cost of Linearizability

Some replication methods can be linearizable and some can't. What do you give up by insisting on it?

Recall from Chapter 5 that **multi-leader replication** suits deployments spanning **multiple datacenters**.

Figure 9-7 (redrawn): two datacenters, each with an application server and a database replica; the replication connection between the databases is cut by a network partition, while clients can still reach their local datacenter.



Suppose the link *between* the two datacenters breaks, but inside each datacenter the network is fine and clients can still reach their local datacenter.

- **Multi-leader:** each datacenter keeps working normally. Writes are replicated asynchronously anyway, so they just queue up and are exchanged when the link returns.
- **Single-leader:** the leader lives in one datacenter. All writes and all linearizable reads must go to it, so clients attached to the *follower* datacenter must send those requests over the inter-datacenter link. When the link breaks, those clients **cannot write and cannot do linearizable reads**. They can still read from the local follower, but those reads may be stale. If the application needs linearizable reads and writes, it is **unavailable** in any datacenter cut off from the leader. Clients that can reach the leader's datacenter directly are fine; the others suffer an outage until the link is repaired.

The CAP theorem

This isn't a quirk of single- vs. multi-leader: **any linearizable database** has this problem, whatever its implementation. And it isn't specific to multiple datacenters — it can happen on any unreliable network, even inside one datacenter. The trade-off:

- **If you require linearizability** and some replicas are cut off from the others by a network problem, then those replicas **can't process requests** while disconnected — they must wait for the network to heal or return an error. Either way they are *unavailable*.
- **If you don't require linearizability**, you can design the system so each replica handles requests on its own even when disconnected (e.g. multi-leader). It stays *available* during the network problem, but its behaviour isn't linearizable.

> **Footnote: CP and AP.** People sometimes call the first choice **CP** (stays consistent, gives up availability during a network partition) and the second **AP** (stays available, gives up consistency during a partition). The book says this classification has several flaws and is best avoided.

So applications that don't need linearizability tolerate network problems better. This is popularly known as the **CAP theorem**, named by **Eric Brewer in 2000** — although distributed database designers had known the trade-off **since the 1970s**.

History. CAP was first offered as a *rule of thumb*, deliberately without precise definitions, to start a conversation about database trade-offs. At the time many distributed databases aimed at linearizability on clusters with **shared storage** (the reference the book cites here is an Oracle RAC book). CAP nudged engineers to explore **shared-nothing** designs better suited to large web services, and deserves credit for that culture shift — witness the wave of new databases from the mid-2000s known as **NoSQL**.

> **The unhelpful CAP theorem** (sidebar). CAP is often stated as "Consistency, Availability, Partition tolerance — pick 2 of 3". That phrasing misleads, because a network partition is a *fault*, not an option you choose: you'll get partitions regardless of your preferences. When the network works, a system can be both linearizable and fully available. Only *when a fault occurs* must you pick one. A better phrasing is **"either Consistent or Available when Partitioned"**. A more reliable network just forces this choice less often. On top of that, discussions of CAP use several contradictory definitions of "availability", and the formal theorem's definition doesn't match everyday usage; many systems called "highly available" don't meet CAP's peculiar definition. Given all this confusion, the book recommends avoiding CAP altogether.

Why CAP is of limited practical value. As formally defined it is very narrow (the formal proof the book cites, from its reference list, is Gilbert and Lynch's 2002 paper):

- it considers **one consistency model** — linearizability;
- and **one kind of fault** — network partitions (nodes alive but unable to talk to each other).

It is silent about network delays, crashed nodes and every other trade-off. So despite its historical influence it offers little help in designing real systems. There are many other impossibility results in distributed computing, and CAP has been **superseded by more precise results**; today it is mostly of historical interest.

> **Footnote: two meanings of "partition".** This book uses *partitioning* to mean deliberately splitting a big dataset into shards (Chapter 6). A *network partition* is a type of network fault. The book normally doesn't single it out from other faults, but since it's the "P" in CAP, the clash of terms can't be avoided here.

Linearizability and network delays

Surprisingly few systems are linearizable in practice. Even the **RAM in a modern multi-core CPU isn't linearizable**: if one core writes to a memory address and another core reads that address shortly afterward, the reader is **not guaranteed** to see the new value — unless a **memory barrier (fence)** is used.

Why? Every CPU core keeps a private memory **cache** plus a **store buffer** (general background, not the book's wording: keeping these copies in sync is the job of *cache coherence*). Memory accesses hit the cache first, and changes reach main memory asynchronously. Cache access is much faster than main memory, so this is essential for performance — but it creates several copies of the data (main memory plus caches) updated asynchronously, so linearizability is lost.

Crucially, it makes no sense to justify this with CAP: inside one computer we assume communication is reliable, and we don't expect a core to keep running normally if disconnected from the rest of the machine. **The reason for dropping linearizability here is performance, not fault tolerance.**

Likewise, many distributed databases that forgo linearizability do so chiefly for **speed**, not so much for fault tolerance. **Linearizability is slow — all the time, not just during a network fault.**

Could a cleverer algorithm make it fast? Apparently not. **Attiya and Welch** proved that for linearizability, the **latency of reads and writes can be no smaller than something proportional to how uncertain network delays are**. On networks with highly variable delays (most real networks,

Chapter 8) linearizable operations will inevitably be slow. No quicker linearizable algorithm exists, whereas weaker models can run far faster — which is why this trade-off matters for latency-sensitive systems. Chapter 12 discusses ways to avoid linearizability without sacrificing correctness.

Ordering Guarantees

A linearizable register acts like a single copy where each operation takes effect at one instant. That automatically means operations happen in **some well-defined order** — the order we drew by joining the markers in Figure 9-4.

Ordering keeps coming up throughout the book, which hints that it is a fundamental idea. Recap of where it has appeared:

- **Chapter 5 (replication):** in single-leader replication, the leader's main job is fixing the **sequence of writes** in the replication log, i.e. the sequence followers apply them in. Without a single leader, concurrent operations can conflict.
- **Chapter 7 (serializability):** transactions must behave *as if* executed in some sequential order — either literally running them one at a time, or running them concurrently while preventing serialization conflicts (with locks or aborts).
- **Chapter 8 (clocks):** timestamps and clocks are another attempt to impose order on a messy world, e.g. deciding which of two writes came later.

Ordering, linearizability and consensus turn out to be **deeply connected**. This part is more abstract than most of the book, but it clarifies what systems can and cannot do.

Ordering and Causality

One big reason ordering matters is that it **preserves causality** — the idea that causes come before effects. The book lists earlier examples where causality mattered:

- **Consistent prefix reads (Chapter 5, Figure 5-5):** an observer of a conversation saw the *answer* before the *question*. That breaks our sense of cause and effect: whoever answered must have seen the question first (unless psychic). There is a **causal dependency** between question and answer.
- **Multi-leader replication among three leaders (Figure 5-9):** some writes "overtook" others because of network delays, so a replica could see an *update* to a row that didn't yet exist. Causality: a row must be created before it can be updated.
- **Detecting concurrent writes (Chapter 5):** given operations A and B, exactly one of three things holds — A happened before B, B happened before A, or they are concurrent. This **happens-before** relation *is* causality: if A happened before B, B might have known about A, built on A, or depended on A. If they are concurrent, neither knew about the other.
- **Snapshot isolation (Chapter 7):** a transaction reads from a "consistent snapshot". *Consistent* here means **respecting causality**: a snapshot that includes an answer has to include the question it answers. Viewing the whole database at a single point in time guarantees this — everything that causally came before that point is visible, nothing that came after is. **Read skew** (non-repeatable reads, Figure 7-6) is precisely reading a state that violates causality.
- **Write skew (Chapter 7, Figure 7-8, the doctors-on-call example):** Alice left the on-call rota because her transaction thought Bob was covering, and Bob did the same thinking Alice was. Going off call was *causally dependent* on observing who was on call. **Serializable snapshot isolation (SSI)** detects write skew by tracking causal dependencies between transactions.
- **Alice and Bob's football score (Figure 9-1):** Bob getting a stale result *after hearing Alice* is a **causality violation** — Alice's exclamation depended on the score being announced, so after hearing her, Bob should be able to see the score too. The image-resizing example (cross-channel timing dependencies) is the same pattern in disguise.

Causal order. Causality orders events: cause before effect; a message is sent before it is received; a question precedes its answer. And chains form: one node reads data and writes something as a result; another node reads *that* and writes something else; and so on. These chains of dependent operations define the **causal order** of the system — what happened before what.

A system that obeys this ordering is **causally consistent**. Example: **snapshot isolation provides causal consistency** — if you read some data, you can also see all data that causally precedes it (unless it has since been deleted).

The causal order is not a total order

Two kinds of order:

- A **total order** lets you compare *any* two elements. Natural numbers are totally ordered: given 5 and 13, you can always say 13 is bigger.
- A **partial order** lets you compare *some* pairs but not others. Mathematical sets are the example: is {a, b} bigger than {b, c}? Neither contains the other, so they are **incomparable**. One set is "greater" only when it contains all the elements of the other.

This difference maps onto consistency models:

	Linearizability	**Causality**
Kind of order	**Total** order of operations	**Partial** order
Why	Single copy + atomic operations ⇒ for any two operations you can say which came first (the timeline in Figure 9-4)	Two operations are ordered if one happened before the other; if they are **concurrent** they are **incomparable**
Concurrency?	None — everything lies on one timeline	Yes — the timeline branches and merges

So by this definition, **a linearizable datastore has no concurrent operations**: there is one timeline along which every operation is totally ordered. Many requests may be *waiting*, but the datastore handles each one atomically at a single instant, on a single copy, on a single timeline.

Concurrency means the timeline **branches and merges**, and operations on different branches can't be compared. Chapter 5's Figure 5-14 showed this: not a straight line but a tangle of concurrent operations, with arrows showing causal dependencies — the partial order.

> **Git analogy.** Version history in a distributed version control system like **Git** looks just like a causal dependency graph. Commits often follow each other in a straight line, but when several people work at once the history **branches**, and **merge** commits join those concurrent lines back together.

Linearizability is stronger than causal consistency

Linearizability implies causality: any linearizable system preserves causality correctly. In particular, when a system has several communication channels (like the message queue and file storage of Figure 9-5), linearizability makes causality hold automatically, with no extra machinery such as passing timestamps between components.

That is what makes linearizable systems easy to reason about and attractive. But, as shown in "The Cost of Linearizability", it hurts performance and availability, especially with large network delays (e.g. geographically distributed systems). That's why some distributed systems have dropped linearizability — gaining performance but becoming harder to work with.

The middle ground. Linearizability isn't the only way to preserve causality. A system can be **causally consistent without paying linearizability's performance price** — and **CAP does not apply** to

it. In fact, **no consistency model stronger than causal consistency can avoid being slowed by network delays while staying available through network failures.**

In many cases, systems that *seem* to need linearizability really only need causal consistency, which is cheaper to implement. Researchers are designing new causality-preserving databases whose performance and availability resemble those of eventually consistent systems. This research is recent, not much has reached production yet, and challenges remain — but it is a promising direction.

Capturing causal dependencies

The book only sketches the key ideas of how a non-linearizable system keeps causal consistency:

1. **You must know what happened before what.** This is a partial order: concurrent operations can be processed in any order, but if one happened before another, every replica must process them in that order.
2. **Wait for predecessors.** Before a replica applies an operation, every operation that causally precedes it must already have been applied. If one is missing, the later operation waits.
3. **Track each node's "knowledge".** If a node had seen value X at the time it issued write Y, then Y might depend causally on X. The book likens this to questions in a fraud trial: *did the CEO know about X when making decision Y?*
4. **Use generalised version vectors.** The techniques resemble "Detecting Concurrent Writes" in Chapter 5, which found concurrent writes to a *single key* in a leaderless store to avoid lost updates. Causal consistency needs more: tracking dependencies **across the whole database**. **Version vectors** can be generalised to do this.
5. **Know which version the application read.** That's why, in Chapter 5's Figure 5-13, the version number from the previous read is passed back to the database with each write. SSI uses a similar idea: at commit time the database checks whether the data the transaction read is still up to date, which requires tracking which data each transaction read.

Sequence Number Ordering

The problem with tracking causality exactly. Causality is a great concept, but tracking *every* causal dependency is often impractical. Applications typically read lots of data before writing, and it's unclear whether the write depends on all of those reads or just some. Recording every read would be a large overhead.

A better way: sequence numbers or timestamps. Give each operation a number that orders it. This number need not come from a physical **time-of-day clock** (which has many problems, Chapter 8). It can come from a **logical clock** — an algorithm that generates a sequence of numbers identifying operations, usually counters incremented on every operation.

Properties of such numbers:

- **Compact** — just a few bytes.
- **Totally ordered** — every operation gets a unique number, and any two can be compared to see which is later.

We specifically want a total order **consistent with causality**: if A causally happened before B, A gets a *lower* number than B. Concurrent operations can be ordered arbitrarily. Such an order captures all causal information but also imposes *more* ordering than causality strictly needs.

> **Footnote: a total order that ignores causality is easy but useless.** For instance, give each operation a random UUID and sort UUIDs alphabetically. It's a valid total order, but tells you nothing about what actually happened first or what was concurrent.

Single-leader case. The replication log of a single-leader database already defines a total order of writes consistent with causality. The leader just increments a counter per operation, giving each a **monotonically increasing** sequence number. A follower applying writes in log order is always **causally consistent** — even while lagging behind.

Noncausal sequence number generators

Without a single leader (multi-leader or leaderless replication, or a partitioned database), how do you generate sequence numbers? Three methods seen in practice:

1. **Per-node independent sequences.** E.g. with two nodes, one hands out **odd** numbers only and the other **even** numbers only. More generally, reserve some bits of the number for a **unique node ID** so two nodes can never produce the same number.
2. **Physical-clock timestamps.** Attach a time-of-day timestamp to each operation. Not sequential, but with high enough resolution they may totally order operations. The **last write wins** conflict resolution method relies on this.
3. **Pre-allocated blocks.** Node A claims numbers **1 to 1,000**, node B claims **1,001 to 2,000**. Each hands out numbers from its own block independently and grabs a new block when running low.

All three **perform and scale better** than routing every operation through one leader's counter, and they give each operation a unique, roughly increasing number. But they share a flaw: **the numbers are not consistent with causality**, because they don't capture ordering *across* nodes:

Method	Why causality breaks
Odd/even (per-node)	Nodes process different numbers of operations per second, so the even counter may lag the odd one (or vice versa). Given an odd- and an even-numbered operation, you can't tell which really came first.
Physical timestamps	**Clock skew** can give a causally later operation a *lower* timestamp (Chapter 8, Figure 8-3).
Block allocator	A causally later operation might get a number from 1–1,000 while the earlier one got a number from 1,001–2,000.

> **Footnote: Spanner's exception.** Physical timestamps *can* be made causally consistent: **Google's Spanner** estimates how far clocks may be skewed and **deliberately waits for that uncertainty interval to pass** before a write commits, ensuring a causally later transaction gets a larger timestamp (Chapter 8, "Synchronized clocks for global snapshots"). But most clocks can't provide the uncertainty metric this needs.

Lamport timestamps

There *is* a simple way to generate sequence numbers consistent with causality: the **Lamport timestamp**, proposed by **Leslie Lamport in 1978**, in a paper that has become among the most frequently cited in the distributed systems field.

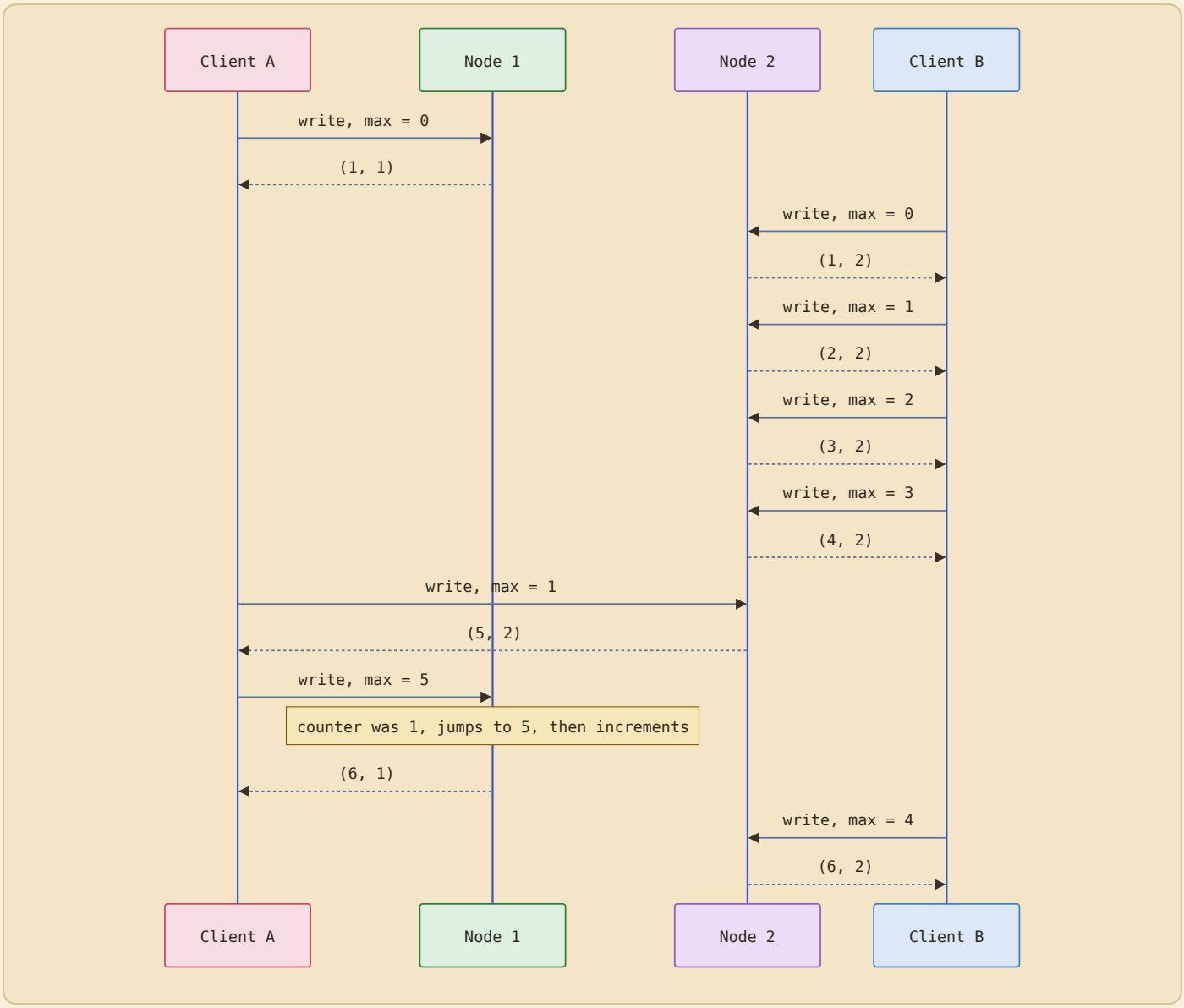
How it works:

1. Every node has a **unique ID**, and keeps a **counter** of operations it has processed.
2. A Lamport timestamp is the pair **(counter, node ID)**. Two nodes may share a counter value, but adding the node ID makes every timestamp unique.
3. **Comparing** two timestamps: the larger counter wins; if counters tie, the larger node ID wins. This gives a **total order**. It has nothing to do with physical time.

4. **The key trick:** every node *and every client* tracks the **largest counter value it has come across** and attaches it to each request. Whenever a node gets a request or response whose attached maximum exceeds its own counter, it **immediately raises its counter to that maximum**.

Steps 1–3 alone would be no better than the odd/even scheme. Step 4 is what makes Lamport timestamps causally consistent: as long as the maximum travels with every operation, **every causal dependency leads to a larger timestamp**.

Figure 9-8 (redrawn): Lamport timestamps (counter, node ID). Each client sends the maximum counter it has seen; a node first raises its counter to that maximum, then increments it. The result is a total order consistent with causality.



In the book's figure, node 2 hands client A a counter of **5**; A then passes 5 along to node 1. Node 1 had only reached **1**, yet it leaps to 5 at once, so the operation it stamps next gets **6**.

Own-code illustration of the rule:

```python
class LamportNode:
    def __init__(self, node_id):
        self.node_id = node_id
        self.counter = 0

    def handle(self, request_max_seen):
        # 1. catch up to anything the caller has already seen
        self.counter = max(self.counter, request_max_seen)
        # 2. this operation gets the next value
        self.counter += 1
        # 3. timestamp = (counter, node_id); caller remembers counter as its new max
        return (self.counter, self.node_id)

n1, n2 = LamportNode(1), LamportNode(2)
print(n1.handle(0))   # (1, 1)
n2.counter = 4        # pretend node 2 has already handled 4 operations (as in the diagram)
print(n2.handle(1))   # (5, 2)  client A only knew max 1; node 2's own counter wins
print(n1.handle(5))   # (6, 1)  A passes max 5; node 1 jumps from 1 to 5, then +1
```

- `max(...)` is the catch-up step: a node never issues a number lower than something its caller already knows about.
- `+= 1` gives the new operation a fresh value.
- Tuples compare counter first, then node ID — exactly the Lamport ordering rule.

Lamport timestamps vs. version vectors (commonly confused):

	Version vectors	**Lamport timestamps**
Purpose	Tell whether two operations are **concurrent** or one **depends** on the other	Always impose a **total order**
Can detect concurrency?	Yes	**No** — from the total order you can't tell concurrent from causally dependent
Size	One counter per node	**More compact** — a single pair

Timestamp ordering is not sufficient

Lamport timestamps give a total order consistent with causality — but that still isn't enough for many common problems.

Example: unique usernames. If two users try to create accounts with the same username at the same time, exactly one should succeed.

Tempting idea: totally order all operations with Lamport timestamps, and if two accounts claim the same name, the one with the lower timestamp wins and the other fails. Since timestamps always compare, this seems fine.

Why it fails: it works only **after the fact**, once you've collected every username-creation operation in the system. But a node that just received a sign-up request must decide **right now** whether to accept it. At that moment it doesn't know whether another node is concurrently creating the same name, nor what timestamp that node will assign.

To be sure no other node is creating the same name with a lower timestamp, the node would have to **ask every other node**. If any node has failed or is unreachable, the whole system **grinds to a halt** — not fault-tolerant at all.

The underlying issue: a total order of operations only *emerges* once you've gathered all operations. Operations you don't yet know about might need to be slotted in at various places in the order.

Conclusion: to implement something like a uniqueness constraint, a total order isn't enough — you also need to know **when the order is finalised**. Once you are certain nobody can slip a claim to the

same username in *before* yours, declaring success is safe. This idea — knowing when the total order is final — is what **total order broadcast** provides.

Total Order Broadcast

On a single CPU core, the total order is just the order the CPU ran the operations. Across many machines, making **every node agree on one total order** is difficult. Timestamps/sequence numbers turned out weaker than single-leader replication (using timestamp ordering for uniqueness can't tolerate any faults).

Single-leader replication gets a total order by picking one node as leader and sequencing everything on **one CPU core** of that node. The hard questions are:

- how to **scale** when throughput exceeds what one leader can handle, and
- how to **fail over** when the leader dies.

In the literature this problem is called **total order broadcast** or **atomic broadcast**.

> **Footnote on naming.** "Atomic broadcast" is the traditional name but confusing: it is unrelated to the A in ACID, and only loosely connected to atomic operations in multithreaded code or to atomic (linearizable) registers. **Total order multicast** is another synonym.

> **Scope of ordering guarantee** (sidebar). Partitioned databases with one leader per partition often keep order **only within each partition**, so they can't provide consistency guarantees (such as consistent snapshots or foreign-key references) **across** partitions. A total order spanning every partition can be achieved, but only with additional coordination.

Definition. Total order broadcast is normally framed as a protocol for passing messages between nodes, with two **safety** properties that must always hold:

1. **Reliable delivery** — no messages are lost: if any node gets a message, every node gets it.
2. **Totally ordered delivery** — every node receives messages in the **same order**.

A correct algorithm must keep both properties even when nodes or the network are faulty. Of course nothing is delivered while the network is down, but the algorithm keeps retrying so that messages get through once it is repaired — still in the correct order.

Using total order broadcast

- **ZooKeeper and etcd implement total order broadcast** — a clue that it is closely tied to consensus.
- **Database replication:** if every message is a write, and all replicas apply an identical sequence of writes, the replicas stay in step (apart from temporary lag). This is **state machine replication** (revisited in Chapter 11).
- **Serializable transactions:** as in "Actual Serial Execution" (Chapter 7), if each message is a **deterministic transaction run as a stored procedure**, and every node processes the messages in the same order, all partitions and replicas stay consistent.
- **Order is fixed at delivery time.** A node may **not retroactively insert** a message earlier in the order once later messages have been delivered. *This is exactly why it is stronger than ordering by timestamps.*
- **It's a log.** Think of total order broadcast as building a **log** — the same idea as a replication log, a transaction log, or a write-ahead log — where delivering a message means appending it. Since every node gets identical messages in identical order, every node reads an identical sequence.
- **Lock service with fencing tokens.** Every lock-acquire request is appended to the log; messages are numbered sequentially in log order; that sequence number, being monotonically increasing, serves as a **fencing token** (Chapter 8). In ZooKeeper this number is the **zxid**.

Implementing linearizable storage using total order broadcast

Linearizability has a total order of operations (Figure 9-4). Is it the same thing as total order broadcast? **Not quite, but they are closely linked.**

- Total order broadcast is **asynchronous**: messages are delivered reliably in a fixed order, but there's **no promise about *when***, so one node may lag behind the others.
- Linearizability promises **recency**: any read sees the most recent write.

> **Footnote: formal relationship.** In formal terms, a plain linearizable read-write register is the *easier* problem. Total order broadcast is equivalent to consensus, and consensus **can't be solved deterministically in the asynchronous crash-stop model** — yet a linearizable read-write register *is* achievable in that same model. But once the register supports atomic operations like **compare-and-set** or **increment-and-get**, it becomes equivalent to consensus. So consensus and linearizable registers are closely related.

Building linearizable CAS on top of total order broadcast — the username example. Imagine each possible username has a linearizable register with an atomic CAS, initially `null` (name not taken). To claim a name, run CAS on its register: set it to the user's account ID **only if it is currently `null`**. If several users race, only one CAS succeeds; the rest see a non-null value.

Implementing that CAS with total order broadcast used as an **append-only log**:

1. **Append** a message to the log that provisionally names the username you are after.
2. **Read** the log and **wait until your own message is delivered back** to you.
3. **Look at all messages claiming that username.** If the **first** such message is yours, you've won: commit the claim (maybe by appending another message) and acknowledge the client. If the first one is someone else's, **abort**.

Because every node sees log entries in the same order, when there are concurrent writes all nodes agree which came first. Choosing the first conflicting write as winner and aborting the rest means all nodes agree whether each write committed or aborted. A log can support **serializable multi-object transactions** in a similar way.

> **Footnote: why you must wait in step 2.** If you acknowledge the write as soon as it's queued, without waiting for it to come back, you get something like the memory model of **multi-core x86 processors** — which is **neither linearizable nor sequentially consistent**.

Own pseudocode:

```
def claim_username(log, name, account_id):
    my_msg = log.append({"claim": name, "by": account_id})   # step 1
    log.wait_until_delivered(my_msg)                          # step 2
    first = next(m for m in log.read_all()                    # step 3
                 if m.get("claim") == name)
    if first is my_msg:
        log.append({"commit": name, "by": account_id})
        return "ok"
    return "error: name taken"
```

- `append` puts the request into the single agreed order.
- `wait_until_delivered` ensures we've seen everything that was ordered before our own message.
- Picking the *first* claim is deterministic, so every node reaches the same verdict.

Reads are still a problem. This procedure makes **writes** linearizable but not reads: reading from a store that is updated asynchronously from the log may return stale data. Precisely, the procedure

gives **sequential consistency** (also called **timeline consistency**), slightly weaker than linearizability. Three ways to make reads linearizable:

Option	How	Real system example
Sequence reads through the log	Append a message, read the log, do the actual read when your message comes back. The message's log position defines when the read happened.	**etcd quorum reads** work somewhat like this
Fetch the latest log position	If the log lets you fetch the latest position linearizably, get it, wait until everything up to it has been delivered to you, then read.	Idea behind **ZooKeeper's** `sync()`
Read from a synchronously updated replica	Read from a replica that is updated synchronously on writes, so it must be current.	**Chain replication**

Implementing total order broadcast using linearizable storage

Now the reverse: given linearizable storage, build total order broadcast.

Algorithm. Suppose you have a linearizable integer register supporting atomic **increment-and-get** (an atomic compare-and-set would also work).

1. For every message to broadcast, **increment-and-get** the register.
2. Attach the returned value to the message as its **sequence number**.
3. **Send** it to every node, retransmitting whatever gets lost.
4. Recipients **deliver messages consecutively by sequence number**.

Key difference from Lamport timestamps: no gaps. Numbers from a linearizable counter form a sequence **without gaps**. So if a node has delivered message 4 and receives message 6, it knows it must **wait for message 5** before delivering 6. With Lamport timestamps you can't know whether a gap is real — and *that gap-awareness is precisely what separates total order broadcast from timestamp ordering.*

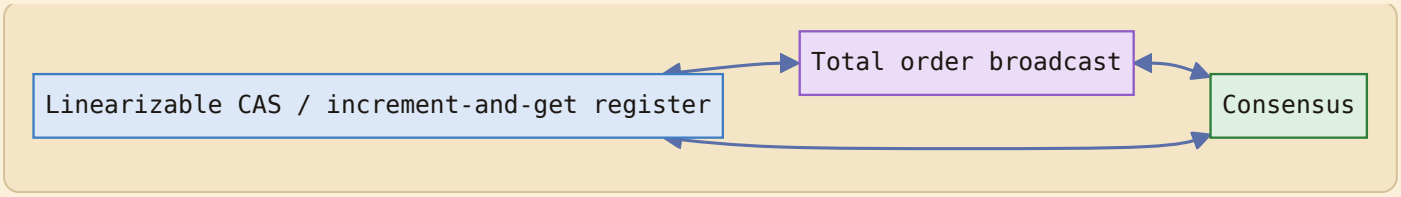
```
def broadcast(counter, nodes, payload):
    seq = counter.increment_and_get()          # linearizable, gap-free
    for n in nodes:
        n.send_with_retry(seq, payload)

class Receiver:
    def __init__(self): self.next_seq, self.buffer = 1, {}
    def on_message(self, seq, payload):
        self.buffer[seq] = payload
        while self.next_seq in self.buffer:     # deliver strictly in order
            deliver(self.buffer.pop(self.next_seq))
            self.next_seq += 1                  # a gap means "wait"
```

- The receiver holds message 6 in `buffer` until 5 arrives, because it *knows* 5 must exist.

How hard is a linearizable increment-and-get? Trivial if nothing ever fails — just hold the number in a variable on a single machine. The difficulty is coping when the network to that node breaks and **recovering the value when the node fails**. Think the problem of linearizable sequence-number generation through carefully and you **always arrive at a consensus algorithm**.

That's no accident: there is a proof that **consensus, total order broadcast, and a linearizable register offering compare-and-set (or increment-and-get) are all equivalent**. A solution to any one of them can be converted into a solution to the others — a profound and surprising result. Time to tackle consensus directly.



(Diagram, not a book figure: the three problems proven equivalent.)

Distributed Transactions and Consensus

What consensus is. Informally: get several nodes to **agree on something**. It sounds easy, and many broken systems were built by people who assumed it was. It ranks among the most fundamental and important problems in the field.

The book deliberately puts consensus late, because its subtleties only make sense after replication (Ch. 5), transactions (Ch. 7), system models (Ch. 8), plus this chapter's linearizability and total order broadcast. Even researchers took **decades** to fully understand it, with many misunderstandings along the way.

Where nodes need to agree:

- **Leader election.** In single-leader replication, all nodes must agree who the leader is. A network fault can make leadership contested (some nodes can't talk to others). Consensus prevents a bad failover leading to **split brain**, where two leaders both accept writes, their data diverges, and you get inconsistency and data loss.
- **Atomic commit.** In a database with transactions spanning several nodes or partitions, a transaction might succeed on some nodes and fail on others. To keep ACID atomicity, all nodes must agree on the outcome: **all abort/roll back** if anything went wrong, or **all commit** if nothing did. This is the **atomic commit problem**.

> **Footnote: atomic commit vs. consensus.** They're formalised a little differently. Atomic commit may commit **only when all participants vote to commit**, and is obliged to abort if even one participant needs to. Consensus may decide **any value proposed by one of the participants**. Still, the two are **reducible to each other**. *Nonblocking* atomic commit is a harder problem than consensus (see three-phase commit below).

> **The Impossibility of Consensus** (sidebar). The **FLP result** — after **Fischer, Lynch and Paterson** — proves that **when any node might crash, no algorithm is guaranteed to always reach consensus**. Since nodes can always crash, is consensus impossible? The catch: FLP is proved in the **asynchronous system model** (Chapter 8), a very restrictive model where the algorithm is **deterministic and can't use clocks or timeouts**. Allow the algorithm **timeouts**, or any other means of *suspecting* that a node has crashed (even a means that is occasionally mistaken), and consensus becomes solvable. Even just letting the algorithm use **random numbers** is enough to escape the impossibility. So FLP matters a lot in theory, but in practice distributed systems can usually reach consensus.

Plan for this section: first the atomic commit problem and the **two-phase commit (2PC)** algorithm — the most common solution, implemented in many databases, messaging systems and application servers. 2PC turns out to be *a consensus algorithm of sorts — just a rather poor one*. Learning from its weaknesses leads to better algorithms such as those in **ZooKeeper (Zab)** and **etcd (Raft)**.

Atomic Commit and Two-Phase Commit (2PC)

Recap from Chapter 7: atomicity gives simple semantics when something fails halfway through a set of writes. A transaction either **commits** (all its writes become durable) or **aborts** (all its writes are rolled back — undone or discarded).

Atomicity stops failed transactions leaving half-finished results and half-updated state lying around. It is particularly important for **multi-object transactions** and for databases with **secondary indexes**: every secondary index is its own data structure, separate from the main data, so a data change must be mirrored in the index. Atomicity keeps the index **consistent with the primary data** (an inconsistent index would be of little use).

From single-node to distributed atomic commit

On one node, atomicity is usually handled by the storage engine:

1. The client asks the node to commit.
2. The node first makes the transaction's writes durable (usually via a **write-ahead log**).
3. Next it adds a **commit record** to that on-disk log.
4. If it crashes midway, recovery on restart reads the log: if the commit record reached disk before the crash, the transaction **counts as committed**; otherwise its writes are **rolled back**.

So single-node commit depends on the **order of durable writes: data first, then the commit record**. The decisive instant is when the **commit record has been fully written to disk**: up to then, a crash means abort; after that, the transaction is committed even if the database crashes. Atomicity is ultimately provided by **one device** — the controller of one disk drive on one node.

On several nodes — e.g. a transaction touching several objects in a sharded database, or a secondary index that is **term-partitioned** whose entry sits on another node than the primary data — things are harder. Most "NoSQL" distributed datastores **don't support** such distributed transactions, but various clustered relational systems do.

Just sending "commit" to every node and letting each commit independently is **not enough**; commit could succeed on some nodes and fail on others:

- some nodes detect a **constraint violation or conflict** and must abort while others commit fine;
- some commit requests are **lost in the network** and time out (abort) while others arrive;
- some nodes **crash** before writing the commit record and roll back on recovery, while others commit.

Then the nodes become inconsistent with each other (like Figure 7-3). And a commit, once made on one node, **can't be retracted** if another node later aborts. Therefore **a node may commit only once it is sure all other nodes in the transaction will also commit.**

Why commits must be irrevocable. Once committed, data is **visible to other transactions**, and other clients may start relying on it — the foundation of **read committed** isolation. If a committed transaction could later be aborted, every transaction that read its data would be built on data that "never existed" and would need reverting too.

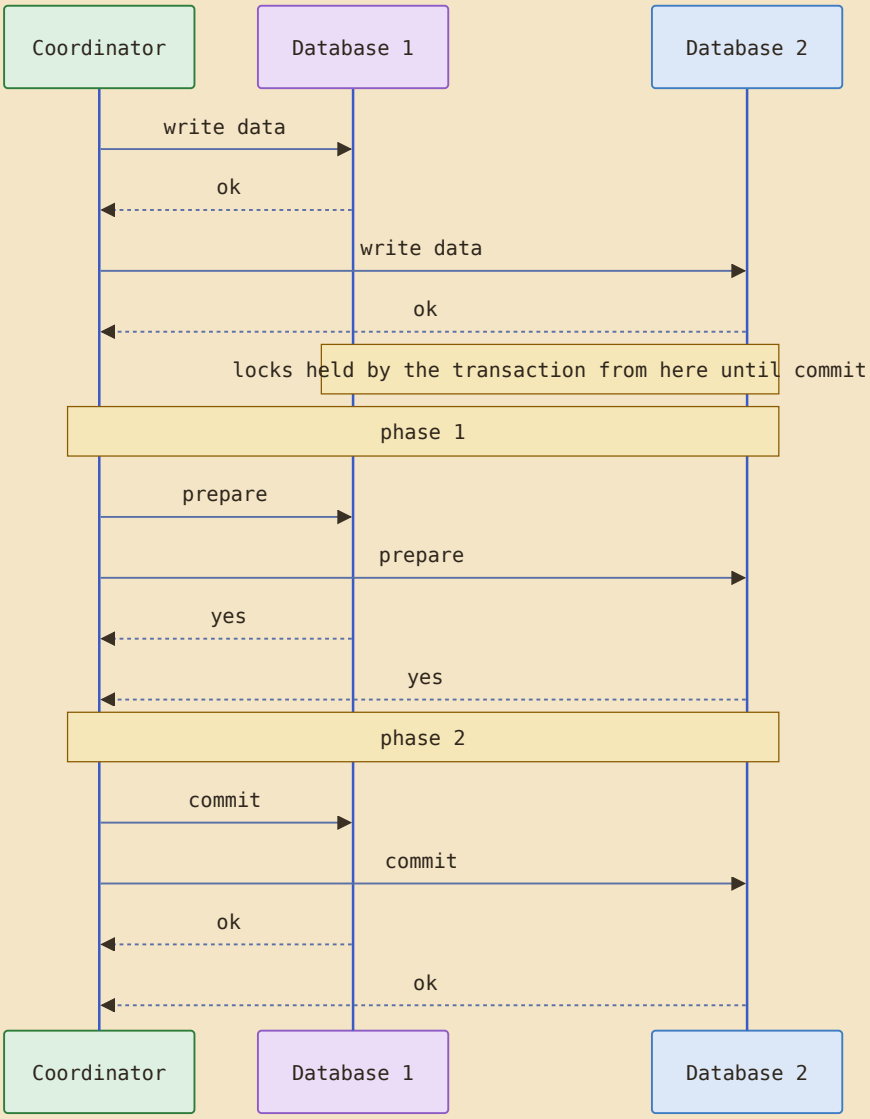
> **Compensating transactions.** The *effects* of a committed transaction can later be undone by another, **compensating transaction**. But to the database that's a separate transaction, so any correctness requirements spanning the two are the **application's** problem.

Introduction to two-phase commit

Two-phase commit (2PC) achieves atomic commit across multiple nodes: either all commit or all abort. It's a classic algorithm in distributed databases. It's used **internally** by some databases and also exposed to applications as **XA transactions** (supported, for example, by the **Java Transaction API**) or through **WS-AtomicTransaction** in the world of **SOAP** web services.

Rather than one commit request, commit/abort happens in **two phases**, which is where the name comes from.

Figure 9-9 (redrawn): a successful two-phase commit. Data is written on both databases, then phase 1 asks each to prepare (both vote yes) and phase 2 tells each to commit. Each database holds the transaction's locks from its write until its commit (the book's shaded bars).



> **Don't confuse 2PC and 2PL** (sidebar). Two-phase **commit** gives **atomic commit** in a distributed database. Two-phase **locking** (Chapter 7) gives **serializable isolation**. They are completely different; best to treat them as unrelated and ignore the unlucky name similarity.

New component: the coordinator (also called **transaction manager**). It doesn't exist in single-node transactions. It frequently lives as a **library inside the same application process** that asked for the transaction (for instance inside a **Java EE container**); it may also run as its own process or service. Examples: **Narayana, JOTM, BTM, MSDTC**.

Basic flow:

1. The application reads and writes data on several database nodes as usual. These nodes are the **participants**.
2. At commit time, the coordinator begins **phase 1** by asking every participant, via a **prepare** request, whether it is able to commit.

3. The coordinator collects responses:
    - if **all** say **"yes"**, it sends **commit** in **phase 2**, and the commit actually happens;
    - if **any** says **"no"**, it sends **abort** to all nodes in phase 2.

> **The wedding analogy.** Like a traditional Western wedding: the minister asks the bride and the groom, one at a time, whether they wish to marry, and usually both answer "I do". Having both acknowledgments, the minister declares them married — the transaction commits and the happy news is broadcast to all attendees. If either says no, the ceremony is aborted.

A system of promises

Why does 2PC guarantee atomicity when a one-phase commit across nodes doesn't? Prepare and commit messages can get lost too. The answer is in the detail:

1. **Global transaction ID.** When the application starts a distributed transaction, it asks the coordinator for a **globally unique transaction ID**.
2. **Local transactions tagged with it.** The application starts a single-node transaction on each participant, tagged with that global ID. All reads and writes happen in these local transactions. Should something fail at this stage (a crashed node, a timed-out request), either the coordinator **or any participant** is free to abort.
3. **Prepare.** When the application is ready, the coordinator sends **prepare**, tagged with the global ID, to every participant. If any prepare request **fails or times out**, the coordinator sends **abort** for that ID to all participants.
4. **Participant's promise.** On receiving prepare, a participant makes sure it can **definitely commit under all circumstances**: it writes all the transaction's data to disk (a crash, power failure or full disk are *not* acceptable excuses to refuse later) and checks for conflicts or constraint violations. By replying **"yes"**, it **promises** to commit without error if asked. It **gives up the right to abort** — but hasn't committed yet.
5. **Coordinator's decision = the commit point.** Having received all prepare responses, the coordinator decides definitively: **commit only if every participant voted yes**. It has to **record this decision durably in its on-disk transaction log**, so it survives a coordinator crash. This write is the **commit point**.
6. **Enforce the decision forever.** Only after the decision is on disk does the coordinator send commit/abort to all participants. If a request fails or times out, it **must retry forever** until it succeeds. There's no turning back: a commit decision is enforced no matter how many attempts are needed. Should a participant crash in the meantime, the transaction is committed **when it recovers** — having voted yes, it can't refuse.

Two points of no return:

- When a **participant votes "yes"**, it promises it will be able to commit later (though the coordinator may still choose to abort).
- Once the **coordinator decides**, that decision is **irrevocable**.

These promises are what make 2PC atomic. (A single-node commit merges both into one event: writing the commit record to the log.)

Wedding analogy, continued. Before "I do", either of you can abort ("No way!"). After "I do", you can't take it back. If you faint after saying it and miss the minister's pronouncement, you're still married. On waking, you can ask the minister what happened to your global transaction ID, or simply wait for the minister to repeat the commit — those retries kept going while you were out cold.

Pseudocode of the coordinator (own example):

```python
def two_phase_commit(coord_log, participants, txid):
    # Phase 1: collect votes
    votes = []
    for p in participants:
        try:
            votes.append(p.prepare(txid))       # participant fsyncs, then votes
        except Timeout:
            votes.append("no")
    decision = "commit" if all(v == "yes" for v in votes) else "abort"

    coord_log.write_and_fsync(txid, decision)   # COMMIT POINT

    # Phase 2: enforce the decision, forever if necessary
    for p in participants:
        while True:
            try:
                p.finish(txid, decision); break
            except Timeout:
                continue                        # retry until success
```

- A timeout during phase 1 counts as "no" — safe, because nobody has committed yet.
- The `write_and_fsync` line is the commit point; a crash before it means abort, after it means the decision stands.
- The endless retry loop in phase 2 is the coordinator keeping its promise.

Own example, not from the book — in PostgreSQL the participant side of 2PC is exposed directly as SQL (this is what PostgreSQL's XA support in drivers such as JDBC is built on). It only works if the server setting `max_prepared_transactions` is above 0 (the default is 0):

```sql
BEGIN;
UPDATE stock SET qty = qty - 1 WHERE sku = 'lamp-42';
PREPARE TRANSACTION 'order-7781';   -- phase 1: durable, votes "yes"
-- ... later, when the coordinator has decided:
COMMIT PREPARED 'order-7781';       -- phase 2 (or ROLLBACK PREPARED)
```

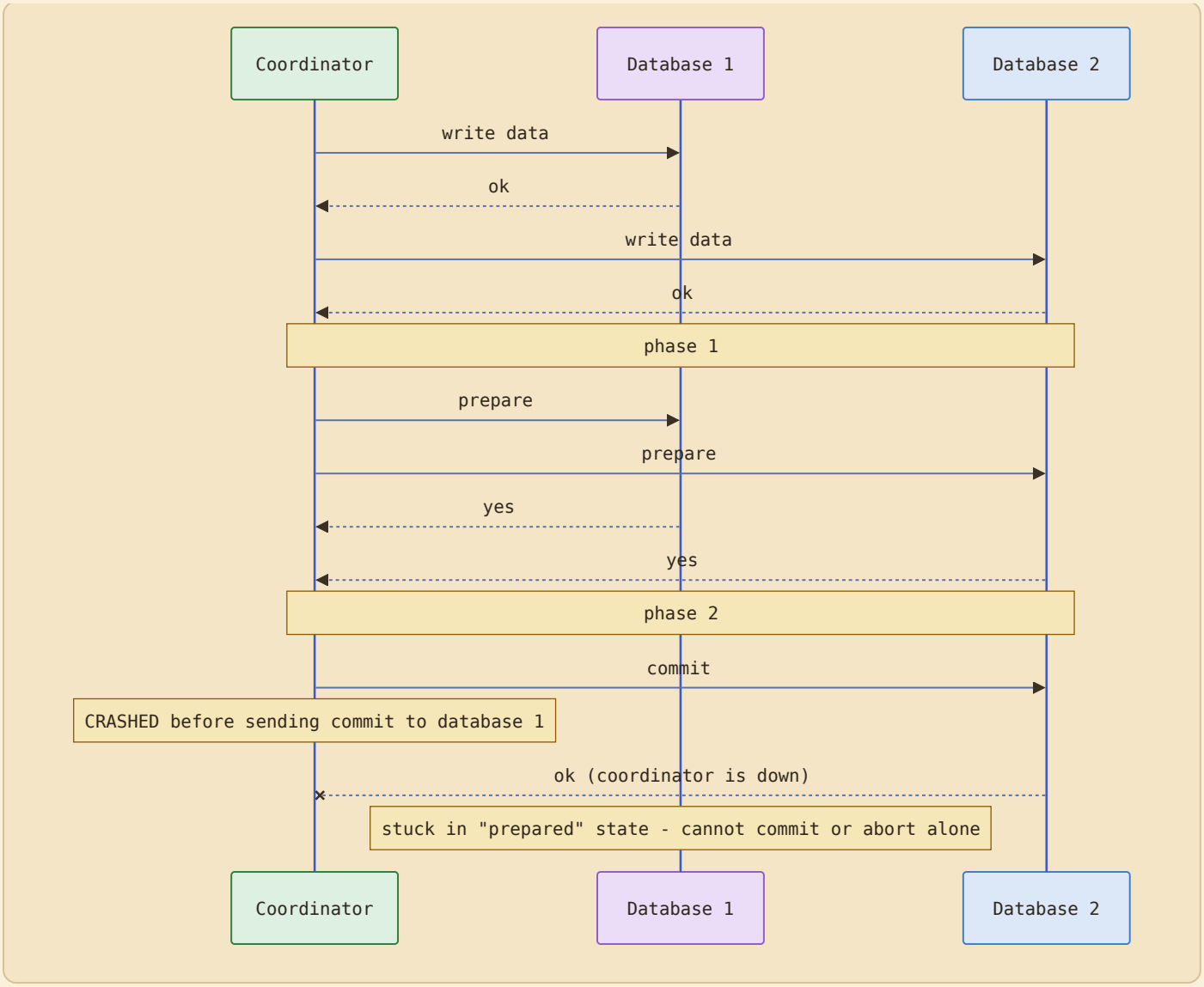
- `PREPARE TRANSACTION` writes the transaction's state durably and keeps its locks, but doesn't commit — the participant's promise. If it succeeds, that is a "yes" vote; if it fails, the transaction is rolled back (a "no" vote). The prepared transaction is detached from the session and survives a server restart.
- `COMMIT PREPARED` / `ROLLBACK PREPARED` (which any session may issue) carries out the coordinator's decision. Until one of them runs, the prepared transaction is *in doubt* and holds its locks — exactly the problem described below. Prepared-but-unresolved transactions are listed in the `pg_prepared_xacts` view.

Coordinator failure

Failures of participants or the network are handled: if any **prepare** fails/times out → abort; if any **commit/abort** request fails → retry indefinitely. But what if the **coordinator** crashes?

- **Before sending prepare:** a participant can safely abort.
- **After a participant voted "yes":** it can **no longer abort on its own** — it must wait to hear from the coordinator. Should the coordinator crash, or the network break, at this point, all the participant can do is **wait**. Its transaction is **in doubt** (or **uncertain**).

Figure 9-10 (redrawn): the coordinator crashes after both participants voted yes. Database 2 gets the commit and commits; the commit for database 1 is never sent, so database 1 is stuck in the "prepared" state, unable to decide on its own.



In the book's example, the coordinator decided **commit**, and **database 2 received the commit**. The coordinator crashed **before sending commit to database 1**, so database 1 doesn't know what to do. A **timeout doesn't help**: aborting unilaterally would make it inconsistent with database 2 (which committed); committing unilaterally is unsafe too, because another participant might have aborted.

Without the coordinator, a participant **can't know** the outcome. In principle participants could talk among themselves to learn how each voted, but **that's not part of 2PC**.

2PC can only finish once the coordinator has recovered. That's why the coordinator must write its decision to its log **before** sending commit/abort: on recovery, it reads the log to resolve all in-doubt transactions. **If the coordinator's log holds no commit record for a transaction, that transaction is aborted.** So 2PC's commit point boils down to an ordinary **single-node atomic commit on the coordinator**.

Three-phase commit

2PC is a **blocking** atomic commit protocol, since it may hang until the coordinator comes back. Theoretically, an atomic commit protocol can be **nonblocking** (doesn't get stuck when a node fails), but making that work in practice isn't easy.

Three-phase commit (3PC) was proposed as an alternative. However, 3PC relies on **network delays being bounded** and **node response times being bounded**. In most real systems — unbounded network delays and process pauses (Chapter 8) — it **can't guarantee atomicity**.

In general, nonblocking atomic commit needs a **perfect failure detector**: a reliable way to tell whether a node has crashed. With unbounded delays, **timeouts can't reliably detect failures** — a

request may time out because of a network problem even though no node crashed. That's why **2PC is still used**, despite the known coordinator-failure problem.

Distributed Transactions in Practice

Distributed transactions — especially 2PC — have a **mixed reputation**. They provide an important safety guarantee that is hard to get otherwise; yet critics say they create operational headaches, ruin performance and over-promise. **Many cloud services deliberately don't offer distributed transactions** because of the operational trouble they bring.

Performance. Some implementations are very slow: **MySQL**'s distributed transactions have been reported as **more than 10 times slower** than its single-node transactions. Much of 2PC's inherent cost comes from the **extra disk forcing (`fsync`)** needed for crash recovery, plus **extra network round-trips**.

Rather than dismiss them, the book examines them, because there are lessons to learn. First, two quite different things get called "distributed transactions":

	Database-internal distributed transactions	**Heterogeneous distributed transactions**
Participants	Nodes of **one** distributed database (which replicates/partitions by default), all running the **same software**	**Two or more different technologies** — e.g. databases from different vendors, or non-databases such as **message brokers**
Examples	**VoltDB**, **MySQL Cluster's NDB** storage engine	A database + a message queue in one transaction
Protocol freedom	Can use any protocol and technology-specific optimisations; no compatibility needed	Must ensure atomic commit across systems that may be totally different internally
How well it works	Often **quite well**	**Much more challenging**

Exactly-once message processing

Heterogeneous transactions let very different systems be integrated powerfully. Example: a **message queue** message is marked **processed exactly when** the database transaction that handled it **commits**. This works by committing the **queue acknowledgment together with the database writes** atomically, as one transaction — possible with distributed transaction support even though the broker and database are unrelated technologies on different machines.

If either the delivery or the database transaction fails, **both abort**, so the broker can safely redeliver later. By atomically committing the message and its processing side effects, the message ends up **processed exactly once in effect**, however many retries were needed. The abort discards the side effects of any partial attempt.

Catch: this only works if **every system affected** supports the **same atomic commit protocol**. Suppose processing a message also **sends an email**, and the email server doesn't support 2PC: if processing fails and is retried, the email may be sent **twice or more**. Only when all side effects are rolled back on abort can processing be retried as if nothing happened. (Exactly-once processing returns in Chapter 11.)

XA transactions

X/Open XA (**eXtended Architecture**) is a standard for 2PC across heterogeneous technologies, introduced in **1991** and widely implemented:

- **Databases:** PostgreSQL, MySQL, DB2, SQL Server, Oracle.
- **Message brokers:** ActiveMQ, HornetQ, MSMQ, IBM MQ.

Key facts:

- **XA is not a network protocol** — it's just a **C API** for talking to a transaction coordinator. Other languages have bindings: in **Java EE**, XA is implemented through the **Java Transaction API (JTA)**,

supported by many **JDBC** (Java Database Connectivity) database drivers and **JMS** (Java Message Service) message broker drivers.

- XA assumes the application talks to participant databases/brokers through a **network driver or client library**. A driver with XA support uses the XA API to check whether a given operation belongs to a distributed transaction; if it does, the driver forwards the necessary details to the database server. The driver additionally offers **callbacks** that let the coordinator tell the participant to **prepare, commit or abort**.
- The **coordinator implements the XA API**. The standard doesn't say how; in practice it's usually a **library loaded into the application's process** (not a separate service). It tracks the participants, collects their responses after asking them to prepare (via driver callbacks), and keeps a **log on the local disk** of each transaction's commit/abort decision.

When the application crashes: if the application process or its machine dies, **the coordinator dies with it**. Participants with prepared-but-uncommitted transactions are stuck **in doubt**. As the coordinator's log lives on that application server's own disk, **the same server has to come back up**, and the coordinator library then reads its log to recover each transaction's outcome. Only then can it use the drivers' XA callbacks to tell participants to commit or abort. **The database server can't contact the coordinator directly** — all communication goes through the client library.

Holding locks while in doubt

Why fuss about in-doubt transactions — couldn't everything else simply continue? **The problem is locks.**

- Transactions usually take **row-level exclusive locks** on rows they modify (to prevent dirty writes, Chapter 7).
- With serializable isolation via **two-phase locking**, they also take **shared locks** on rows they read.
- The database **can't release these locks until the transaction commits or aborts** (the shaded area in Figure 9-9).

So with 2PC, a transaction holds its locks **for as long as it is in doubt**. A coordinator that crashes and needs **20 minutes** to restart means 20 minutes of held locks. A coordinator whose log is **completely lost** means the locks are held **forever** — or until an administrator fixes it manually.

As long as the locks are in place, **those rows can't be modified by any other transaction**, and depending on the database, some may not even be able to **read** them. Other transactions touching that data **block**, which can leave **large parts of the application unavailable** until someone resolves the in-doubt transaction.

Recovering from coordinator failure

In theory, a restarted coordinator recovers cleanly from its log and resolves every in-doubt transaction. In practice, **orphaned in-doubt transactions** happen: the coordinator can't determine the outcome for some reason (e.g. the log was **lost or corrupted by a software bug**). They can't be resolved automatically, so they **sit in the database forever**, holding locks and blocking others.

Rebooting the database servers won't help: a correct 2PC implementation must **keep an in-doubt transaction's locks across restarts**, or it would risk breaking atomicity.

The only way out: an **administrator manually decides** to commit or roll back. They must examine each in-doubt transaction's participants, find out whether any has already committed or aborted, and apply the same outcome to the rest. This can take a lot of manual work, most likely **under intense stress and time pressure** in the middle of a major production outage (why else would the coordinator be in such a state?).

Heuristic decisions. Many XA implementations offer an emergency way out: a participant may **unilaterally decide** to commit or abort an in-doubt transaction without the coordinator's decision. **"Heuristic" is a euphemism for "probably breaking atomicity"**, because it violates 2PC's system of promises. It's meant only for escaping **catastrophes**, not regular use.

Limitations of distributed transactions

XA solves a real, important problem — keeping several data systems consistent — but brings major operational problems. The crucial realisation: **the coordinator is really a database in its own right** (it stores transaction outcomes), so it deserves the same care as any important database:

1. **Single point of failure.** If the coordinator isn't replicated and runs on one machine, when it fails, other application servers get stuck behind the locks of in-doubt transactions. Surprisingly, **lots of coordinator implementations don't offer high availability out of the box**, or support replication only in a basic way.
2. **Application servers are no longer stateless.** Many server-side apps follow a **stateless** model (as HTTP favours), with all persistent state in a database, so servers can be added or removed at will. But a coordinator inside the application server changes this: its **logs become crucial durable state** — as important as the databases — since they're needed to recover in-doubt transactions.
3. **Lowest common denominator.** XA must work with many kinds of systems, so it can't do much. It **can't detect deadlocks across systems** (that would need a standard protocol for exchanging lock-wait information), and it **doesn't work with SSI** (that would need a protocol for identifying conflicts across systems).
4. **Failure amplification.** For **database-internal** transactions (not XA) the limits are smaller — e.g. **distributed SSI is possible**. But 2PC still needs **all participants to respond** to commit. A single broken component makes the whole transaction fail. Distributed transactions therefore tend to **amplify failures**, the opposite of fault tolerance.

Does this mean giving up on keeping systems consistent with each other? No — alternative approaches achieve the same without heterogeneous distributed transactions (Chapters 11 and 12). But first, back to consensus.

Fault-Tolerant Consensus

Informal definition. Several nodes agree on something. Examples: several people simultaneously try to grab the **final seat on a flight**, the **same theatre seat**, or register the **same username**; consensus decides which of the mutually incompatible operations wins.

Formal definition. One or more nodes **propose** values, and the algorithm **decides** on one of them. In the seat example, each node serving a customer proposes that customer's ID, and the decision says who gets the seat.

A consensus algorithm must satisfy:

Property	Meaning	Type
Uniform agreement	No two nodes decide differently.	Safety
Integrity	No node decides twice.	Safety
Validity	Any value a node decides must have been proposed by one of the nodes.	Safety
Termination	Every node that doesn't crash eventually decides some value.	**Liveness**

> **Footnote: uniform consensus.** This variant is called **uniform consensus**; it is equivalent to ordinary consensus in asynchronous systems whose failure detectors are unreliable. Academic papers usually say *processes* rather than *nodes*.

Why each property:

- **Agreement + integrity** are the core: everyone decides the same, and nobody changes their mind.
- **Validity** rules out silly solutions — e.g. an algorithm that always decides `null` satisfies agreement and integrity but not validity.

- **Termination** captures **fault tolerance**. Without it, the first three are easy: hard-code one node as **"dictator"** that makes every decision. But if it fails, no more decisions can be made — exactly what happens to **2PC** when the coordinator fails. Termination says the algorithm can't just sit idle forever; it must **make progress**, and the surviving nodes must still reach a decision even when some fail.

The crash model. Consensus assumes a crashed node **disappears and never comes back**. Picture not a software crash but an **earthquake**: a landslide wipes out your datacenter and the node lies beneath **30 feet of mud**, never coming back. Under this model, an algorithm that must **wait for a recovery** can never satisfy termination. In particular, **2PC does not satisfy termination**.

How many failures can be tolerated? If *all* nodes crash, nothing can be decided. It's proven that any consensus algorithm needs **at least a majority of nodes functioning correctly** to guarantee termination; that majority can safely form a **quorum**. So termination assumes **fewer than half** the nodes are crashed or unreachable.

But most implementations keep the **safety properties** (agreement, integrity, validity) **always** — even if a majority fails or there's a severe network problem. A large outage can stop the system from processing requests, but **can't corrupt it into making invalid decisions**.

Byzantine faults. Most consensus algorithms assume **no Byzantine faults** (Chapter 8): a node that doesn't follow the protocol (e.g. sends contradictory messages to different nodes) may break safety. Consensus can be made **Byzantine-robust provided that under one-third of the nodes suffer Byzantine faults**, but the book doesn't cover those algorithms in detail.

Consensus algorithms and total order broadcast

The most famous fault-tolerant consensus algorithms: **Viewstamped Replication (VSR)**, **Paxos**, **Raft** and **Zab**. They share a lot but aren't identical. The book doesn't go into full detail — knowing the common high-level ideas is enough unless you are building one yourself — which the book **doesn't recommend**, as it's very hard.

Most of them don't actually use the formal single-value model above. They decide a whole **sequence of values**, so they are really **total order broadcast algorithms**.

Why that's natural: total order broadcast has every node receive every message **exactly once and in one agreed order**. That amounts to **consensus run over and over**: per round, nodes propose which message should go next, and the round's decision fixes the next message in the order. Each consensus property maps to a broadcast property:

Consensus property	Total order broadcast consequence
Agreement	Every node delivers **identical messages in an identical order**
Integrity	Messages are **not duplicated**
Validity	Messages are **neither corrupted nor invented**
Termination	Messages are **not lost**

Viewstamped Replication, Raft and Zab run total order broadcast **natively**, which is more efficient than repeated one-value consensus. For **Paxos**, this optimisation is called **Multi-Paxos**.

Single-leader replication and consensus

Single-leader replication (Chapter 5) funnels every write through the leader, which passes them to followers in one fixed order. Isn't that total order broadcast? Why didn't Chapter 5 need consensus?

It depends on how the leader is chosen:

- **Manually chosen by operators:** effectively a "consensus algorithm" of the **dictatorial** kind. Only one node accepts writes (decides the order of writes in the log); if it goes down, writes are

unavailable until humans configure a new leader. Works fine in practice, but **doesn't satisfy termination**, since progress needs human intervention.

- **Automatic leader election and failover:** a follower is promoted if the leader fails. That moves us nearer to fault-tolerant total order broadcast — and therefore to consensus.

The chicken-and-egg problem. To avoid split brain, all nodes must agree who the leader is — so electing a leader **needs consensus**. Yet consensus algorithms are really total order broadcast, which resembles single-leader replication, which requires a leader… So **electing a leader seems to require a leader, and solving consensus seems to require solving consensus first.** How to escape?

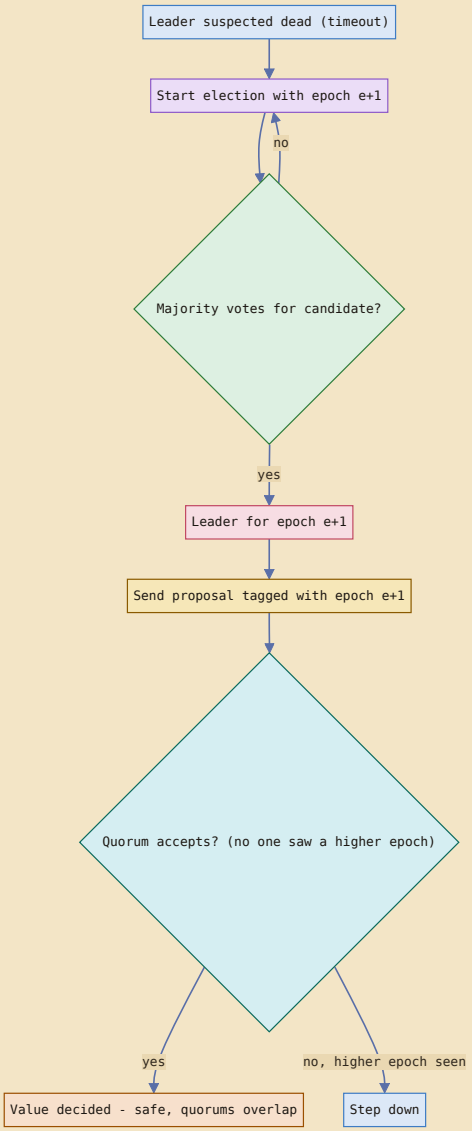
Epoch numbering and quorums

All these protocols use a leader internally, but **don't guarantee the leader is unique**. They make a weaker promise: they define an **epoch number** and promise **at most one leader per epoch**.

Algorithm	Name for the epoch number
Paxos	**ballot number**
Viewstamped Replication	**view number**
Raft	**term number**

How it works, step by step:

1. **New election ⇒ new epoch.** Each time the nodes *suspect* the leader has died, they hold a vote for a new one. The election gets an **incremented epoch number**, so epochs are **totally ordered and monotonically increasing**.
2. **Higher epoch wins.** If two leaders from different epochs conflict (maybe the old one wasn't dead after all), the leader with the **higher epoch** prevails.
3. **A leader can't trust itself.** Before deciding anything, a leader must check there's no other leader with a higher epoch that could make a conflicting decision. As Chapter 8 put it, the truth is decided by the majority: a node believing it leads doesn't mean the others accept it.
4. **Quorum vote on every proposal.** For each decision, the leader distributes its proposal and **waits until a quorum replies in favour**. The quorum is typically — **but not always** — a **majority**. A node votes for a proposal **only if it knows of no other leader with a higher epoch**.
5. **Two rounds of voting, overlapping quorums.** There are two votes: one to **elect a leader**, one to **approve a leader's proposal**. The key insight: **the two quorums must overlap**. If a proposal vote succeeds, at least one node that voted for it also took part in the **most recent leader election**. So if the proposal vote reveals no higher epoch, the leader can conclude **no newer election has happened**, knows it still leads, and can safely **decide** the value.



(Diagram, not a book figure: the epoch-and-quorum idea in flowchart form.)

Comparison with 2PC. This looks superficially like two-phase commit, but differs in crucial ways:

	Two-phase commit	Fault-tolerant consensus
Who leads	Coordinator is **not elected**	Leader is **elected** (per epoch)
Votes needed	**"Yes" from every** participant	Only a **majority** (quorum)
After a failure	Wait for the coordinator to recover	A defined **recovery procedure** brings nodes back to a consistent state once a new leader takes over, preserving safety

These differences are what make consensus algorithms correct and fault-tolerant.

Limitations of consensus

Consensus algorithms are a **huge breakthrough**: they bring concrete safety (agreement, integrity, validity) to an environment where everything else is uncertain, while staying fault-tolerant (they progress whenever a majority is up and reachable). Because they give you total order broadcast, they can also offer **linearizable atomic operations** in a fault-tolerant way. Yet they aren't used everywhere, because the benefits cost something:

1. **Synchronous replication cost.** Voting on proposals before deciding is a kind of **synchronous replication**. Many databases are configured for **asynchronous** replication, accepting that some committed data may be lost on failover, for better performance.
2. **Strict majority needed.** Tolerating **1 failure needs at least 3 nodes** (the remaining 2 of 3 are a majority); tolerating **2 failures needs at least 5** (3 of 5). If a network failure splits the nodes, **only the majority side can make progress**; the rest is blocked.
3. **Fixed membership.** Most algorithms assume a **fixed set of voting nodes**, so the cluster's membership can't simply be changed. **Dynamic membership** extensions exist but are much less well understood.
4. **Timeouts and false elections.** Failure detection relies on **timeouts**. With highly variable delays — especially in **geographically distributed** systems — a node may wrongly think the leader failed due to a transient glitch. Safety isn't harmed, but **frequent leader elections cause terrible performance**: the system may spend more time electing leaders than doing useful work.
5. **Sensitivity to network problems. Raft** has been shown to have nasty edge cases: if the whole network works except **one consistently unreliable link**, leadership can **bounce between two nodes** forever, or the leader is repeatedly forced to resign, so the system effectively **never makes progress**. Other algorithms have similar issues; algorithms that cope well with unreliable networks remain **an open research question**.

Membership and Coordination Services

ZooKeeper and **etcd** get described as "distributed key-value stores" or as "coordination and configuration services". Their API looks like a database: read and write a key's value, iterate over keys. So why do they bother with consensus, and what makes them different from other databases?

You rarely use ZooKeeper directly — it's not a good general-purpose database. You more likely rely on it **indirectly**: **HBase, Hadoop YARN, OpenStack Nova and Kafka** each depend on a ZooKeeper cluster behind the scenes.

Design: ZooKeeper and etcd hold **small amounts of data that fit entirely in memory** (still written to disk for durability) — you wouldn't store all your application data there. That data is replicated to all nodes with a **fault-tolerant total order broadcast algorithm** — exactly what database replication needs (same writes, same order, consistent replicas).

ZooKeeper is modelled on **Google's Chubby lock service**. Besides total order broadcast (and thus consensus), it offers features that are especially handy for distributed systems:

Feature	What it gives you
Linearizable atomic operations	With atomic compare-and-set you can build a **lock**: if several nodes try the same operation concurrently, only one succeeds. Consensus guarantees atomicity and linearizability even if nodes fail or the network breaks. A distributed lock is usually a **lease** with an expiry time, so it's released eventually if the client fails.
Total ordering of operations	When a lock or lease guards a resource, a **fencing token** (a number that grows every time the lock is taken) prevents clients from clashing after a process pause. ZooKeeper totally orders every operation, assigning each one an ever-increasing **zxid** (transaction ID) and **cversion** (version number).
Failure detection	Clients keep a **long-lived session** with ZooKeeper servers and exchange **heartbeats**. Brief connection drops or a ZooKeeper node failure don't end the session; but if heartbeats stop for longer than the **session timeout**, the session is declared dead. A session's locks can optionally be freed automatically on timeout — ZooKeeper calls these **ephemeral nodes**.
Change notifications	A client can **watch** locks and values created by others and be told when they change — e.g. learn when a client joins the cluster (from the value it writes) or fails (its session expires, so its ephemeral nodes disappear). This avoids frequent polling.

Of these, **only the linearizable atomic operations truly need consensus** — but it's the **combination** that makes ZooKeeper so useful for coordination.

Allocating work to nodes

1. **Leader/primary selection.** You run several instances of a process or service and need one to be **leader or primary**; if it fails, another takes over. Useful for single-leader databases, **job schedulers** and similar stateful systems.
2. **Partition assignment.** You have some partitioned resource — a database, message streams, file storage, a distributed actor system, and so on — and must decide **which partition goes on which node**. When nodes **join**, some partitions move to them to rebalance load (Chapter 6); when nodes **leave or fail**, others take over their work.

These tasks can be done with careful use of **atomic operations, ephemeral nodes and notifications** in ZooKeeper. Done right, the application **recovers from faults automatically**, without humans. It's not easy, even with libraries like **Apache Curator** that provide higher-level tools over the ZooKeeper client API — but far better than implementing consensus from scratch, which has a **poor track record**.

A sketch of leader election with ZooKeeper-style primitives (own example, pseudocode):

```
# Every candidate creates an EPHEMERAL + SEQUENTIAL node under /election
me = zk.create("/election/candidate-", data=my_address,
               ephemeral=True, sequential=True)    # e.g. candidate-0000000017

def check_leadership():
    children = sorted(zk.get_children("/election"))
    if children[0] == me.name:
        become_leader()                            # lowest number wins
    else:
        # watch the candidate just ahead of me; if its session dies,
        # its ephemeral node disappears and I get notified
        prev = children[children.index(me.name) - 1]
        zk.exists("/election/" + prev, watch=lambda _: check_leadership())

check_leadership()
```

- **Ephemeral** ties the node to the session: if this process dies, its entry vanishes (failure detection).
- **Sequential** numbering comes from ZooKeeper's total order, so all candidates agree who is "lowest".
- The **watch** is a change notification, so no polling is needed.

Scale. An application may start on one node and grow to **thousands**. Majority voting across thousands of machines would be hopelessly inefficient. So ZooKeeper runs on a **small fixed set of nodes (typically three or five)**, does its majority votes among them, and serves a potentially huge number of clients. It lets you **"outsource"** coordination work — **consensus, operation ordering, failure detection** — to an external service.

Slow-changing data. ZooKeeper data typically changes slowly: facts such as "**node 10.1.1.23 currently leads partition 7**", changing on a scale of **minutes or hours**. It's **not** meant for runtime application state that could change **thousands or even millions of times each second**. To replicate that kind of state between nodes, use other tools such as **Apache BookKeeper**.

Service discovery

ZooKeeper, etcd and **Consul** are also used for **service discovery**: finding which **IP address** to connect to for a given service. In cloud datacenters, **virtual machines come and go** constantly, so you often don't know service IPs in advance. Instead, services **register their network endpoints in a service registry** at startup, where other services can find them.

Does service discovery need consensus? Less clear. DNS is the traditional way to look up a service's IP. It uses many **layers of caching** for performance and availability. **DNS reads are absolutely not**

linearizable, and slightly stale results usually aren't a problem. What matters more is that DNS is **reliably available and robust to network interruptions**.

However, **leader election does need consensus**. So if your consensus system already knows the leader, it can sensibly also help other services **find out who the leader is**. To support that, some consensus systems offer **read-only caching replicas**: they asynchronously receive the log of all consensus decisions but **don't vote**, so they can serve reads that **don't need to be linearizable**.

Membership services

ZooKeeper and similar systems belong to a long line of research on **membership services**, going back to the **1980s**, which matters for highly reliable systems such as **air traffic control**.

A membership service decides **which nodes are currently active, live members of a cluster**. As Chapter 8 showed, unbounded network delays mean you can never be certain whether another node has failed. But by **coupling failure detection with consensus**, nodes can **agree** on which nodes are considered alive.

A node might still be wrongly declared dead by consensus even though it's alive. Even so, **agreement on the current membership** is very valuable. Example: choosing a leader could simply mean picking the **lowest-numbered current member** — but that only works if all nodes agree on who the current members are.

Summary

The book's own summary ties the chapter together:

- **Linearizability** — makes replicated data look like a **single copy** with **atomic** operations. Easy to understand (the database acts just like a variable in single-threaded code), but **slow**, especially with large network delays.
- **Causality** — orders events by cause and effect. Weaker than linearizability: some things can be **concurrent**, so history is a timeline that **branches and merges**. Causal consistency skips linearizability's coordination overhead and **copes far better with network problems**.
- **Limits of causal ordering** — even with causal ordering captured (e.g. with **Lamport timestamps**), some things can't be done that way. The **unique username** example: to accept a registration, a node must know that no other node is concurrently registering the same name. That led to **consensus**.
- **Consensus** — deciding something so that **all nodes agree** and the decision is **irrevocable**. Many problems turn out to be **equivalent** to consensus (a solution to one can be transformed into a solution to another):

Equivalent problem	What must be decided
Linearizable compare-and-set registers	Whether to set the value, atomically, based on whether the current value equals the given parameter
Atomic transaction commit	Whether a distributed transaction commits or aborts
Total order broadcast	The order in which to deliver messages
Locks and leases	Which of several racing clients acquired the lock
Membership/coordination service	Using some failure detector (such as timeouts), which nodes are alive and which are dead because their sessions timed out
Uniqueness constraint	Which of several concurrent transactions creating conflicting records with the same key succeeds, and which fails with a constraint violation

All of these are easy **on a single node**, or if you give all decision-making power to **one node** — which is what a **single-leader database** does, and why it can offer linearizable operations, uniqueness

constraints, a totally ordered replication log and more.

But if that leader **fails** or becomes **unreachable**, the system can't progress. Three ways to handle that:

1. **Wait for the leader to recover**, accepting that the system is blocked meanwhile. **Many XA/JTA transaction coordinators** do this. It doesn't fully solve consensus — it violates **termination**: if the leader never recovers, the system blocks forever.
2. **Manually fail over**: humans pick a new leader and reconfigure. **Many relational databases** work this way. It's consensus by **"act of God"** — a human operator outside the system decides. Failover speed is limited by human speed, usually slower than computers.
3. **Automatically choose a new leader** with an algorithm. This needs a **consensus algorithm**, and you should use a **proven** one that correctly handles adverse network conditions.

A single-leader database can be linearizable **without a consensus round per write**, but it **still depends on consensus to keep and hand over leadership**. So a leader merely **"kicks the can down the road"**: consensus remains necessary, only in another place and less frequently. The good news: fault-tolerant consensus algorithms and systems exist.

Tools like **ZooKeeper** provide an **"outsourced" consensus, failure detection and membership service**. Not easy to use, but much better than building your own algorithms to survive all the problems in Chapter 8. If you need something **reducible to consensus** and want it **fault-tolerant**, use something like ZooKeeper.

Still, **not every system needs consensus**: **leaderless and multi-leader** systems usually don't use global consensus. Their conflicts come from *not* having consensus across leaders — but maybe that's fine; maybe we can live without linearizability and learn to work with **branching and merging version histories**.

The chapter drew on a lot of theoretical research. The papers can be hard and sometimes make unrealistic assumptions, but they're **very valuable** for practical work: they show what can and can't be done and reveal counterintuitive ways distributed systems fail.

This ends **Part II** (replication, partitioning, transactions, failure models, consistency and consensus). **Part III** returns to practical systems: building powerful applications from heterogeneous building blocks.

Chapter summary

- **Eventual consistency** only promises convergence "some time"; stronger models are easier to use but cost performance/availability.
- **Linearizability** = one copy + atomic operations + real-time recency. Once any read sees a new value, all later reads must too. It's needed for **locks/leader election**, **hard uniqueness constraints**, and avoiding **cross-channel races**.
- It's **not** serializability (transactions, some serial order) — both together = **strict serializability**. SSI isn't linearizable.
- **Which replication is linearizable:** single-leader (potentially), consensus (yes), multi-leader (no), leaderless (probably not — even strict quorums fail under variable delay).
- **CAP**: during a network partition, choose linearizability *or* availability. The book considers CAP too narrow and confusing to be useful. The deeper cost is **latency**: linearizable response time is at least proportional to network delay uncertainty (Attiya and Welch). Even multi-core RAM gives up linearizability — for **speed**.
- **Causality** is a **partial** order; linearizability is a **total** order. Linearizable ⇒ causal. **Causal consistency** is the strongest model that stays fast and available under network faults.
- **Lamport timestamps** (counter, node ID) + "carry the max" give a total order consistent with causality, but can't tell when the order is **final** — so they can't enforce uniqueness on the spot.

- **Total order broadcast** (reliable + same-order delivery, order fixed at delivery) = a replicated log; it powers state machine replication, fencing tokens (zxid), and linearizable CAS. Linearizable CAS/increment-and-get ⇔ total order broadcast ⇔ consensus.
- **2PC**: prepare (participants promise) → coordinator writes decision (commit point) → commit/abort retried forever. Coordinator crash leaves participants **in doubt**, holding locks. 3PC needs bounded delays. XA is a C API for heterogeneous 2PC; coordinators become single points of failure; heuristic decisions break atomicity.
- **Consensus** properties: agreement, integrity, validity (safety) + termination (liveness); needs a majority alive. VSR, Paxos (Multi-Paxos), Raft, Zab implement total order broadcast using **epochs** and **overlapping quorums**.
- **ZooKeeper/etcd**: small in-memory data, linearizable ops, zxid ordering, sessions/ephemeral nodes, watches; used for leader election, partition assignment, service discovery, membership.

Interview angle

1. "What is linearizability?" *30-second answer:* It's the promise that a replicated system acts like it holds one copy of the data, with each operation taking effect atomically at some instant between its request and response. Practically, once a write completes — or once *anyone* has read the new value — every later read everywhere returns that value or a newer one. It's a recency guarantee on single objects.

- *Follow-up: how is it different from serializability?* Serializability is about multi-object transactions behaving as if run in some serial order (not necessarily real-time order). Linearizability is about single-register recency in real time. Both together is strict serializability; SSI is serializable but not linearizable because it reads from snapshots.
- *Follow-up: do quorums ($w + r > n$) give linearizability?* Not necessarily. With variable delays, one reader can see the new value from one replica while a later reader hits two stale replicas. You'd need synchronous read repair and read-before-write; Cassandra still breaks with LWW, and CAS can't be done this way at all.

2. "Explain the CAP theorem." *30-second answer:* When a network partition happens, a system must either refuse some requests to stay linearizable (consistent) or keep serving requests and give up linearizability (available). The popular "pick 2 of 3" framing is misleading: partitions aren't optional, so it's really "consistent or available *when partitioned*".

- *Follow-up: why do many databases avoid linearizability, then?* Mostly for performance, not partition tolerance. Linearizable operations are slow all the time — response time is at least proportional to network delay uncertainty. Multi-core CPUs drop it for the same reason.
- *Follow-up: what's the strongest model that stays available?* Causal consistency.

3. "How does two-phase commit work, and what's its main weakness?" *30-second answer:* The coordinator assigns a global transaction ID; participants do their work; then the coordinator sends *prepare*. Each participant durably writes its data, checks constraints and votes yes — promising it can commit. If all vote yes, the coordinator writes "commit" to its log (the commit point) and sends commit, retrying forever. The weakness: if the coordinator crashes after participants voted yes, they're stuck *in doubt*, holding locks until it recovers.

- *Follow-up: why not just time out?* Aborting might contradict a participant that already committed; committing might contradict one that aborted. Only the coordinator knows.
- *Follow-up: does 3PC fix it?* Only if network delays and response times are bounded, which real networks don't guarantee. Nonblocking commit needs a perfect failure detector.
- *Follow-up: what are XA's practical problems?* Coordinator is often an unreplicated library in the app server (single point of failure, app no longer stateless), lowest-common-denominator (no cross-system deadlock detection, no SSI), failure amplification, orphaned transactions needing manual or heuristic resolution. MySQL distributed transactions reported over 10x slower.

4. "What is consensus and what properties must it have?" *30-second answer:* Nodes propose values and all decide on one. Properties: uniform agreement (no two decide differently), integrity (decide at most once), validity (decided value was proposed), termination (every non-crashed node eventually decides). The first three are safety, termination is liveness and needs a majority of nodes up.

- *Follow-up: isn't consensus impossible (FLP)?* Only in the asynchronous model without clocks, timeouts or randomness. With timeouts or random numbers it's solvable in practice.
- *Follow-up: how do Raft/Paxos avoid split brain?* Epoch (term/ballot/view) numbers make the leader unique per epoch; every decision needs a quorum that overlaps the election quorum, so a stale leader learns of a higher epoch and can't decide.
- *Follow-up: how many nodes to tolerate 2 failures?* Five.

5. "What's the relationship between total order broadcast, linearizability and consensus?" *30-second answer:* Total order broadcast delivers every message to every node in the same order, with order fixed at delivery — it's a replicated log. You can build linearizable compare-and-set from it (append a claim, wait for it to come back, first claim wins), and build it from a linearizable increment-and-get counter (gap-free sequence numbers). Linearizable CAS, total order broadcast and consensus are all equivalent; consensus algorithms like Raft and Zab are really total order broadcast algorithms.

- *Follow-up: why aren't Lamport timestamps enough?* They give a total order consistent with causality, but you never know whether some other node will later produce a lower timestamp — the order isn't final, and there can be gaps. Total order broadcast lets you know when it is final.

6. "What is ZooKeeper used for?" *30-second answer:* It's an outsourced coordination service running consensus (Zab) on 3 or 5 nodes: linearizable atomic operations for locks/leases, zxid ordering for fencing tokens, session heartbeats with ephemeral nodes for failure detection, and watches for change notifications. Systems like HBase, Hadoop YARN, OpenStack Nova and Kafka use it for leader election, partition assignment and membership.

- *Follow-up: would you store application data in it?* No — it's for small, slow-changing data (minutes/hours); use something like Apache BookKeeper for high-rate replicated state.

Self-check

1. **Why is eventual consistency hard to program against?** Because it gives no bound on when replicas converge; you might not even read your own write. Unlike a variable, the database can return old values, and the resulting bugs only appear under faults or high concurrency.
2. **In Figure 9-4, why is B's final read not linearizable even though it overlaps C's `cas(x, 2, 4)`?** Because A had already read 4 before B's read began. After any read returns a new value, all later reads must return it too, so B may not return 2.
3. **A website stores uploaded photos, then enqueues a resize job. Occasionally thumbnails are of an old photo. What's happening, and name two fixes.** A cross-channel race: the queue delivered the job faster than the file storage replicated the photo, so the resizer read a stale version. Fix with linearizable file storage, or — since you control the queue — a read-your-own-writes-style technique (the book names no specific mechanism; one own illustration is putting the photo's version number in the queue message and having the resizer wait until it can read at least that version).
4. **Does a Dynamo-style store with n = 3, w = 3, r = 2 give linearizability?** No. During a write, reader A can see the new value on one replica, and a later reader B can read two replicas that haven't received it yet and return the old value. Synchronous read repair plus read-before-write fixes reads/writes (at a cost), but not CAS.
5. **Why can't odd/even per-node counters give a causally consistent order, and how do Lamport timestamps fix it?** Nodes run at different rates, so one counter lags the other and numbers don't

reflect cross-node causality. Lamport timestamps have every node and client carry the maximum counter seen and bump local counters to it, so every causal successor gets a larger timestamp.

6. **Why aren't Lamport timestamps enough to enforce unique usernames immediately?** The node can't know whether another node is concurrently assigning a lower timestamp to the same name; the total order is only known after all operations are collected. Checking with every node would halt on any failure.
7. **Name the two points of no return in 2PC.** A participant voting "yes" (it gives up the right to abort), and the coordinator writing its decision to its log (the decision becomes irrevocable).
8. **The coordinator's log is lost after a crash. What happens to the participants?** Transactions in doubt become orphaned: they hold their locks indefinitely, even across database restarts, blocking other transactions. An administrator must resolve them manually, or use a heuristic decision (which probably breaks atomicity).
9. **How does a Raft leader know it's still the leader when it makes a decision?** It sends each proposal to the nodes and needs a quorum to accept; nodes only accept if they know of no higher term. Because the proposal quorum overlaps the most recent election quorum, if no higher term appears the leader knows no newer election happened.
10. **Your team wants a 4-node consensus cluster to "tolerate more failures than 3 nodes". Is that right?** No. A majority of 4 is 3, so it still tolerates only 1 failure — the same as 3 nodes. You need 5 nodes to tolerate 2 failures.

Chapter 10 — Batch Processing

Book: Chapter 10, pp. 389–438 (with the Part III introduction just before it)

Overview

Part III opener: Derived Data (summary of the Part III introduction)

Parts I and II of the book built up everything that goes into *one* distributed database: how bytes sit on disk, how data is replicated and partitioned, how transactions and consensus work. Real applications are messier. A large application usually needs to read and process its data in many different ways, and no single database is good at all of them. So in practice an application glues together several stores: an OLTP database, a cache, a full-text search index, an analytics warehouse, and so on, plus the plumbing that moves data between them. Part III is about that integration problem. The book points out that vendors who promise "our product does everything" tend to ignore it, yet wiring different systems together correctly is one of the most important jobs in any non-trivial application.

The Part intro splits data systems into two broad kinds:

Kind	What it means	Examples
System of record (also called *source of truth*)	Holds the authoritative copy of the data. New data (e.g., user input) is written here first. Each fact is stored exactly once, usually in normalized form. If another system disagrees with it, the system of record wins by definition.	The primary OLTP database of an app
Derived data system	Holds data produced by taking data from some other system and transforming it. If you lose it, you can rebuild it from the source.	Caches (fall back to the DB on a miss), indexes, denormalized values, materialized views, recommendation models built from usage logs

Key points from the Part intro:

- Derived data is technically **redundant** (it duplicates information that exists elsewhere), but it is often essential for fast reads. It is typically denormalized, and one source can feed several derived datasets, each giving a different "view" of the same facts.
- Making the record-vs-derived distinction explicit clarifies the **dataflow** of your system: which component takes which inputs, produces which outputs, and depends on what.
- A given database product is *not* inherently one or the other. It is just a tool; whether it acts as a system of record or a derived store depends on how *your application* uses it.
- Chapter plan for Part III: Chapter 10 looks at batch dataflow systems like MapReduce; Chapter 11 applies the same ideas to streams (same kind of work, lower delay); Chapter 12 ties it all together into ideas for building reliable, scalable, maintainable applications in the future.

What this chapter answers

Question: How do you process a huge, fixed pile of data (terabytes of logs, a whole database snapshot) to produce some useful output, reliably, across many machines?

The chapter contrasts three styles of system:

Style	How it is triggered	Main performance metric	Example
Services (online systems)	A client sends a request and waits for a reply	Response time; availability also matters a lot	Web server, database, cache
Batch processing (offline systems)	A job is started (often on a schedule, e.g., once a day) over a large, fixed input; a run can last minutes or even days, so nobody sits waiting for them	**Throughput**: how long it takes to chew through an input of a given size	MapReduce, Spark jobs
Stream processing (near-real-time / nearline)	Consumes events shortly after they occur; has inputs and outputs rather than request/response	Latency lower than batch	Covered in Chapter 11

Why it matters: batch jobs build search indexes, recommendation models, analytics tables, and derived databases. The design ideas behind batch processing (immutable inputs, derived outputs, rerunnable jobs) are also the foundation for stream processing and for the "derived data" architecture of Part III.

A little history the book mentions:

- MapReduce was published by Google in **2004**, and was once described (somewhat over-enthusiastically) as the algorithm behind Google's massive scalability. It was later implemented in open source systems including **Hadoop, CouchDB, and MongoDB**.
- Compared with the parallel query engines built for data warehouses years earlier, MapReduce is quite *low-level*, but it let people process data at unprecedented scale on cheap commodity machines. Its importance is now declining, yet it is still the clearest way to understand why and how batch processing works.
- Batch processing is ancient: long before programmable digital computers, **punch card tabulating machines** such as the **Hollerith machines** used for the **1890 US Census** computed aggregate statistics over large inputs in a semi-mechanized batch style. MapReduce also bears an eerie likeness to the electromechanical **IBM card sorters** that companies relied on for **business data processing** during the 1940s and 1950s. History repeats itself.

Everyday analogy: A bank's end-of-day reconciliation. During the day, tellers serve customers one at a time (online system). At night, a clerk takes the whole day's stack of slips (a fixed, bounded input), sorts them by account number, and totals each account (batch job). Nobody is waiting at the counter for the result; what matters is getting through the whole stack by morning.

Key terms

Term	Plain-English meaning
System of record / source of truth	The authoritative copy of data; written first; wins any disagreement
Derived data	Data computed from other data; can be rebuilt if lost (caches, indexes, views)
Online system (service)	Handles requests while a user waits; judged by response time
Batch processing (offline)	Runs a job over a large, bounded input and writes output; judged by throughput
Stream processing (nearline)	Processes events soon after they happen; between online and batch
Throughput	Amount of data processed per unit of time
Bounded input	Input of a known, fixed size (e.g., yesterday's logs), so the job can finish
Working set	The data a job needs random access to in memory
Unix pipe	Connects one program's stdout to the next program's stdin via a small memory buffer
Uniform interface	Every program reads/writes the same kind of thing (in Unix: a file descriptor = sequence of bytes)
stdin / stdout	A program's default input and output streams
Loose coupling / late binding / inversion of control	The program does not decide where its input comes from or where its output goes; the caller wires it up
HDFS	Hadoop Distributed File System; open source version of Google File System (GFS)
NameNode	Central HDFS server that tracks which machine stores which file block
Shared-nothing	Architecture where each machine has its own CPU, RAM and disks, connected only by a normal network
Erasure coding (e.g., Reed–Solomon)	Storing redundancy codes so lost data can be rebuilt using less space than full copies
MapReduce	Programming model: a *mapper* emits key-value pairs, framework sorts by key, a *reducer* processes each key's values
Mapper	Function called once per input record; outputs zero or more key-value pairs; keeps no state between records
Reducer	Function called once per key with an iterator over all values for that key
Shuffle	Partitioning mapper output by reducer, sorting it, and copying it to reducers (no randomness involved)
Putting computation near the data (data locality)	Running a task on a machine that already stores a copy of its input
Workflow	A chain of jobs where one job's output directory is the next job's input
Workflow scheduler	Tool that starts jobs when their inputs are ready (Oozie, Azkaban, Luigi, Airflow, Pinball)

Term	Plain-English meaning
Join	Combining records that refer to each other (e.g., event with user ID + user profile)
Equi-join	A join where records match on equal values of a field
Full table scan	Reading every record of the input
Activity events / clickstream	Log of actions logged-in users took on a website
Sort-merge join	Reduce-side join: both inputs are keyed by join key, sorted, and merged in the reducer
Secondary sort	Arranging values *within* a key in a chosen order (e.g., user record first, then events by time)
Sessionization	Grouping all events of one user session to see the sequence of actions
Skew / hot spot	One partition gets far more work than others
Hot key / linchpin object	A key with a disproportionate amount of data (e.g., a celebrity)
Reduce-side join	Join logic runs in the reducer after sorting and shuffling
Map-side join	Join done in mappers only, no reducers and no sorting, by exploiting known input properties
Broadcast hash join	Small input loaded fully into a hash table in every mapper of the large input
Partitioned hash join (bucketed map join)	Both inputs partitioned the same way; each mapper joins one matching pair of partitions
Map-side merge join	Both inputs partitioned *and sorted* the same way; mapper merges them like a reducer would
Immutable	Never modified once written
Human fault tolerance	Ability to recover from buggy code (roll back code, rerun job)
MPP database	Massively parallel processing database: runs analytic SQL in parallel over a cluster
Data lake / enterprise data hub	Dumping raw data from across an organization into one place and deciding on schema later
Schema-on-read	The reader interprets the structure of the data, instead of enforcing it at write time
Sushi principle	"Raw data is better": keep data unprocessed so many views can be derived
Preemption	Killing a low-priority task to free resources for a higher-priority one
Materialization	Eagerly computing an intermediate result and writing it out (e.g., to files)
Straggler	A task that takes much longer than its peers
Dataflow engine	Engine (Spark, Tez, Flink) that runs a whole multi-stage workflow as one job
Operator	A generalized processing function in a dataflow engine (not restricted to map/reduce roles)

Term	Plain-English meaning
RDD	Resilient Distributed Dataset: Spark's abstraction that records how each piece of data was computed
Checkpoint	Saving state to durable storage so computation can resume from there
Deterministic operator	Always produces the same output given the same input
Pipelined execution	Passing output to the next operator incrementally, without waiting for all input
DAG	Directed acyclic graph; how dataflow engines arrange operators (not the same as graph data!)
Transitive closure	Repeatedly following edges until no new vertices are reached
Bulk synchronous parallel (BSP) / Pregel model	Graph processing in rounds; vertices send messages to each other between rounds
"Thinking like a vertex"	Writing graph code from the viewpoint of a single vertex
Declarative query language	You state *what* you want; an optimizer decides *how*
Cost-based query optimizer	Picks join algorithms and join order based on estimated cost
Vectorized execution	Processing data in tight CPU-cache-friendly loops without per-record function calls
k-nearest neighbors	Finding items closest to a given item in a multi-dimensional space

Batch Processing with Unix Tools

The chapter starts small: one machine, standard Unix commands. The point is that the *design ideas* of Unix carry straight over to huge distributed systems.

The running example is a web server log. Each request adds one line. In nginx's default format, a line records: client IP, a literal hyphen placeholder, remote user (a hyphen if not logged in), local time, the request line (method, path, protocol), HTTP status, response size in bytes, referrer URL, and user-agent string. The book's sample line decodes as: on 27 Feb 2015 at 17:55:11 UTC, IP 216.58.210.78 requested `/css/typography.css`, got status 200 with 3,377 bytes, using Chrome 40, because the page `http://martin.kleppmann.com/` referenced it.

A made-up line in the same shape:

```
10.1.2.3 - - [03/Mar/2021:08:00:01 +0000] "GET /blog/intro.html HTTP/1.1" 200 5120
"https://example.org/" "Mozilla/5.0 (X11; Linux x86_64) Firefox/86.0"
```

- `10.1.2.3` = `$remote_addr`, the client's IP address.
- `-` `-` = a fixed hyphen in the format string, then `$remote_user` (a hyphen because the user is not logged in).
- `[03/Mar/2021:08:00:01 +0000]` = `$time_local`.
- `"GET /blog/intro.html HTTP/1.1"` = `$request`; splitting the whole line on spaces, the URL is the **7th field**.
- `200` = `$status`, `5120` = `$body_bytes_sent`.
- The last two quoted strings = `$http_referer` and `$http_user_agent`.

Simple Log Analysis

Goal: find the 5 most requested pages. A Unix pipeline does it (own example, same idea):

```
cat /srv/logs/web-access.log |   # 1. stream the log file
  awk '{print $7}'           |   # 2. keep only field 7: the requested URL
  sort                       |   # 3. sort URLs so identical ones sit next to each other
  uniq -c                    |   # 4. collapse runs of identical lines, prefixing a count
  sort -r -n                 |   # 5. sort numerically (-n) by that count, biggest first (-r)
  head -n 5                      # 6. keep the first 5 lines only
```

How it works, step by step:

1. `cat` reads the file. (A footnote notes people like to say `cat` is unnecessary since `awk` could take the file name directly; the book uses it because it makes the linear pipeline obvious.)
2. `awk` splits each line on whitespace and prints the 7th field (the URL).
3. `sort` orders URLs alphabetically, so a URL requested *n* times appears as *n* adjacent lines.
4. `uniq -c` removes duplicate *adjacent* lines and, thanks to `-c`, prints how many there were.
5. The second `sort` orders by that leading number, largest first.
6. `head` keeps the top 5.

The output is a list like `4189 /favicon.ico`, `3631 /2013/05/24/...`, and so on down to the 5th URL. The book's point: this cryptic-looking one-liner can process **gigabytes of logs in seconds**, and is trivially tweakable:

- Skip CSS files: change the awk program to `'$7 !~ /\.css$/ {print $7}'`.
- Count top client IPs instead: `'{print $1}'`.

The book recommends learning `awk`, `sed`, `grep`, `sort`, `uniq`, and `xargs`: many analyses take minutes to write and run surprisingly fast. (One of the book's references is a blog post claiming command-line tools can beat a Hadoop cluster by 235x.)

Chain of commands versus custom program

You could write the same thing as a small program. Own Python example (the book uses Ruby):

```
from collections import Counter

hits = Counter()                                # dict of URL -> count, default 0
with open('/srv/logs/web-access.log') as f:
    for line in f:
        url = line.split()[6]                   # 7th whitespace field (index 6, zero-based)
        hits[url] += 1                          # bump that URL's counter
for url, n in hits.most_common(5):              # sort by count descending, take top 5
    print(n, url)                               # print "count url"
```

- `Counter()` is an in-memory hash table from URL to count.
- `line.split()[6]` extracts the URL (index 6 because lists start at 0).
- `hits[url] += 1` increments the count.
- `most_common(5)` sorts by count and takes the top 5.

It is less terse than the pipeline but readable; preferring one or the other is partly taste. The *real* difference is in how they execute on large inputs.

Sorting versus in-memory aggregation

Approach	How it counts	Memory needed	When it wins
Hash table (the script)	Keeps one counter per distinct URL in memory	Proportional to the number of **distinct** URLs (a million hits to one URL still costs one entry)	When the working set fits in RAM; for most small/mid websites, all distinct URLs plus counters fit in about 1 GB, even on a laptop
Sorting (the pipeline)	Sorts the list so duplicates are adjacent, then counts runs	Can spill to disk	When the working set is bigger than memory

Why sorting scales: it is the same trick as SSTables and LSM-trees (Chapter 3). Sort chunks in memory, write each chunk to disk as a sorted segment, then merge the segments into one sorted file. Mergesort reads and writes **sequentially**, which disks handle well (optimizing for sequential I/O keeps recurring throughout the book).

GNU Coreutils `sort` (Linux) automatically spills to disk when data exceeds memory and parallelizes across CPU cores. So the simple pipeline scales to big inputs without running out of memory; the likely bottleneck becomes how fast the input can be read from disk.

The Unix Philosophy

The ease of the log example is not luck: composing small programs was a core design idea of Unix and still matters today.

> **Historical note.** Doug McIlroy, who invented Unix pipes, wrote in **1964** that programs should be connectable like segments of a garden hose, screwing in another piece whenever data needs to be processed differently. The plumbing metaphor stuck.

The Unix philosophy as described in **1978**, paraphrased:

1. Each program should do one thing well; for a new job, write a new program rather than piling features on an old one.
2. Assume a program's output will feed some other, not-yet-written program. Keep output free of clutter; avoid rigid columnar or binary input formats; do not require interactive input.
3. Get software (even an OS) into people's hands for trial quickly, ideally in weeks; don't be afraid to scrap the clumsy bits and rebuild them.
4. Lighten programming work with tools rather than unskilled helpers, even when making those tools is a detour and some get thrown away afterwards.

The book observes this (automation, rapid prototyping, incremental iteration, experimentation, splitting big projects into chunks) sounds a lot like today's **Agile and DevOps** movements; little has changed in four decades.

`sort` is the model "do one thing well" tool. It is arguably a better sort than most language standard libraries (those typically neither spill to disk nor use several threads), yet on its own it is barely useful; it becomes powerful combined with tools like `uniq`. A shell like `bash` lets you compose programs written by different people. What makes that composability possible?

A uniform interface

- If any program's output must be usable as any program's input, **all programs must share one I/O interface**.
- In Unix, that interface is a **file** (more precisely, a *file descriptor*): just an ordered sequence of bytes.

- Because it is so simple, lots of different things fit behind it: a real file, a pipe to another process (stdin, stdout, Unix sockets), a device driver (e.g., `/dev/audio`, `/dev/lp0`), a TCP connection socket.
- By convention many (not all) Unix tools treat the bytes as **ASCII text**, with records separated by newline `\n` (ASCII 0x0A). `awk`, `sort`, `uniq`, `head` all assume this. The choice is arbitrary: the ASCII **record separator 0x1E** was designed for exactly this and arguably would have been better. What matters is that everyone agreed.
- Parsing *within* a record is less standardized: whitespace, tabs, CSV, pipe-separated... even `xargs` has about half a dozen options for how to parse input.
- It is not pretty: `{print $7}` is cryptic; ideally you would write something like `{print $request_url}`. The book comes back to this idea later (structured formats like Avro/Parquet on Hadoop).

> **Aside: the web as a uniform interface.** URLs and HTTP play the same role: any site can link to any resource on any other site, even when run by unrelated organizations. Before that, bulletin board systems (BBSs) each had their own phone number and modem settings; to follow a reference you had to hang up, dial another BBS, and search manually. You could not link directly into another BBS's content.

Even decades later, few pieces of software compose as well as Unix tools. You cannot easily pipe your email account and your shopping history through a custom tool into a spreadsheet and then post the result somewhere. Even databases that share a data model often make it hard to move data from one to another. The book calls this lack of integration the **Balkanization of data**.

Separation of logic and wiring

- Unix tools read **stdin** and write **stdout**. By default stdin is the keyboard and stdout the screen, but you can redirect either to a file, or use a pipe to connect one process's stdout to another's stdin. A pipe uses a small in-memory buffer; it does *not* write the whole intermediate stream to disk.
- A program *can* open files itself, but the Unix approach works best when the program ignores file paths and just uses stdin/stdout. Then the shell user decides the wiring; the program neither knows nor cares where data comes from or goes. You can call this **loose coupling**, **late binding**, or **inversion of control**.
- So you can write your own tool (e.g., user-agent string to browser name, or IP address to country code) and drop it into the pipeline; `sort` does not care whether its neighbor is an OS tool or your code.

Limits of stdin/stdout:

- Programs needing **several inputs or outputs** are possible but awkward.
- You cannot pipe output straight into a **network connection**. (Footnote: except via separate tools like `netcat` or `curl`. BSD sockets broke the "everything is a file" convention; research OSes **Plan 9** and **Inferno** are more consistent, where a TCP connection shows up as a file under `/net/tcp`.)
- If a program opens files, spawns subprocesses, or opens network connections itself, then *it* owns that wiring. It may be configurable via command-line options, but you lose the shell's flexible rewiring.

Transparency and experimentation

Why Unix tools are so pleasant for exploration:

- **Inputs are treated as immutable.** Rerun commands as often as you like with different options; the input files are never damaged.

- **You can stop the pipeline anywhere**, pipe into `less`, and eyeball whether the output looks right. Great for debugging.
- **You can save one stage's output to a file** and feed it to the next stage, so you can restart later stages without redoing the whole pipeline.

Unix tools are blunt compared with a relational query optimizer, but superb for experimenting. Their **biggest limitation**: they run on a single machine. That is where Hadoop comes in.

MapReduce and Distributed Filesystems

What it is: MapReduce is roughly "Unix tools spread over up to thousands of machines". It is also blunt and brute-force, yet surprisingly effective.

Parallels with Unix:

- One MapReduce **job** ≈ one Unix **process**: takes one or more inputs, produces one or more outputs.
- A job normally **does not modify its input** and has **no side effects** except producing output.
- Output files are written **once, sequentially**; no part of a file is modified after it is written.
- Where Unix uses stdin/stdout, MapReduce reads and writes files on a **distributed filesystem**. In Hadoop that is **HDFS**, an open source re-implementation of Google's **GFS**.

Other distributed filesystems: **GlusterFS**, **Quantcast File System (QFS)**. Object stores like **Amazon S3, Azure Blob Storage, OpenStack Swift** are similar in many ways. The chapter uses HDFS as the example, but the ideas apply generally.

> **Footnote: HDFS vs object stores.** HDFS lets the scheduler run a task on a machine that holds a copy of the input file, while object stores typically separate compute from storage. Local reads help when network bandwidth is the bottleneck. But if erasure coding is used, that locality benefit disappears, because pieces from several machines must be combined to rebuild the file.

How HDFS works:

1. HDFS follows the **shared-nothing** principle (from Part II), unlike **shared-disk** storage such as **NAS** (Network Attached Storage) and **SAN** (Storage Area Network), which rely on a central storage appliance, often custom hardware and special networking like **Fibre Channel**. Shared-nothing needs only ordinary computers on an ordinary datacenter network.
2. A **daemon** process runs on every machine, exposing a network service so other nodes can read files stored on that machine's disks.
3. A central server, the **NameNode**, records which file blocks live on which machine.
4. Conceptually, the result is one huge filesystem that can use disk space on every machine where the daemon runs.
5. To survive machine/disk failure, blocks are **replicated**: either plain copies on several machines (as in Chapter 5) or **erasure coding** (an error-correcting-code technique) like **Reed–Solomon codes**, which recovers lost data with less storage overhead than full copies. This is like **RAID** (redundancy across disks in one machine), except done over a normal network without special hardware.

Scale: when the book was written, the largest HDFS clusters spanned **tens of thousands of machines** with **hundreds of petabytes** in total. This is viable because storing and accessing data on HDFS with commodity hardware and open source software costs far less than equivalent capacity on a dedicated storage appliance.

MapReduce Job Execution

MapReduce is a programming framework for processing big datasets stored in a filesystem like HDFS. The log example maps onto it almost exactly:

1. **Read input files and split into records.** (In the log example, one record = one line; `\n` is the separator.) Handled by the *input format parser*.
2. **Call the mapper** on each record to extract a key and a value. (Log example: `awk '{print $7}'`; key = URL, value empty.) *You write this.*
3. **Sort all key-value pairs by key.** (Log example: the first `sort`.) *Implicit*: MapReduce always sorts mapper output before the reducer sees it.
4. **Call the reducer** to walk over the sorted pairs. Equal keys are adjacent, so values can be combined without holding much state. (Log example: `uniq -c`.) *You write this.*

The two callbacks:

- **Mapper:** called once per input record; extracts key(s) and value(s). It may emit any number of pairs (including zero). It carries **no state** between records; every record is handled on its own.
- **Reducer:** the framework collects every value sharing a key and invokes the reducer once for that key, handing it an **iterator** over the values. The reducer can emit output records (e.g., the count for that URL).

The log pipeline had a *second* sort (step 5, ranking URLs by count). In MapReduce you would write a **second job** whose input is the first job's output. Big picture: the mapper's role is to put data into a form suitable for sorting; the reducer's role is to process data that has been sorted.

Own illustrative MapReduce-style code (pseudocode in Python) counting requests per URL:

```
def mapper(line):                # called once per input record (one log line)
    fields = line.split()
    yield (fields[6], 1)         # key = URL, value = 1 (one hit)

def reducer(url, counts):        # called once per distinct key, after sort
    yield (url, sum(counts))     # counts is an iterator of all the 1s for this URL
```

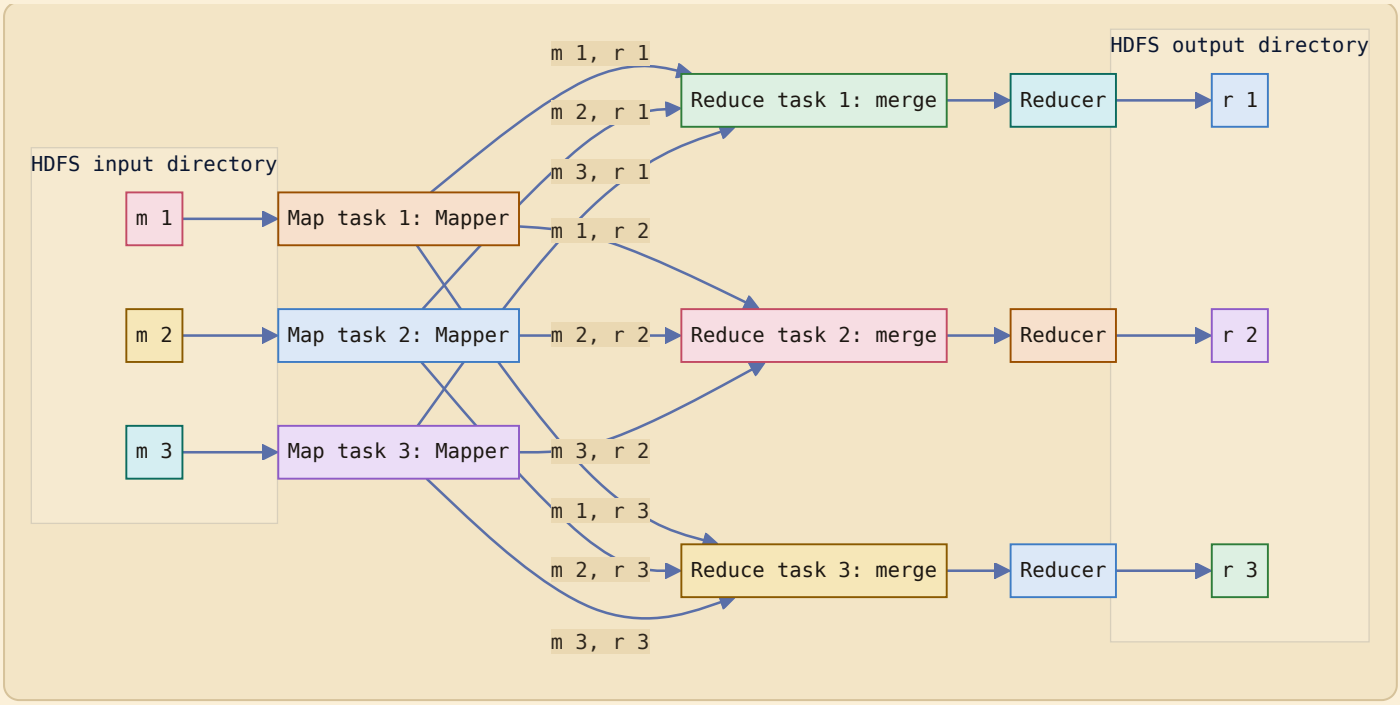
- The mapper holds no state; it just emits `(url, 1)`.
- The framework partitions, sorts, and groups by URL.
- The reducer sums the values for one URL and emits the total.

Distributed execution of MapReduce

The big difference from Unix pipelines: MapReduce **parallelizes across many machines without you writing parallel code**. Mapper and reducer only see one record (or one key) at a time and do not know where data comes from or goes, so the framework can move data between machines for you.

- Unix tools *can* be used as mappers/reducers in a distributed job, but usually they are functions in a normal language. In **Hadoop MapReduce** they are Java classes implementing an interface; in **MongoDB and CouchDB** they are JavaScript functions.
- Parallelism is based on **partitioning** (Chapter 6). A job's input is usually an HDFS directory, and **each file or file block is a separate partition** handled by its own **map task**.

Figure 10-1 (redrawn): a job with three map tasks and three reduce tasks. Map task i reads input block m i from HDFS, and its mapper writes three sorted files "m i, r 1", "m i, r 2", "m i, r 3" on local disk, one per reducer. Reduce task j fetches file "m i, r j" from every map task, merges them, runs the reducer, and writes output r j to HDFS.



How a job runs, step by step:

1. **Scheduling map tasks near the data.** Input files are typically hundreds of MB each. The scheduler tries to run each mapper on a machine that holds a replica of its input block, if that machine has spare RAM and CPU. This is **putting the computation near the data**: less copying over the network, better locality.
2. **Shipping code.** The mapper code usually is not on that machine yet, so the framework copies it there (e.g., JAR files for Java), then launches the task, which reads its input block and feeds the mapper record by record. The mapper emits key-value pairs.
3. **Deciding reducers.** The number of map tasks = number of input blocks. The number of reduce tasks is **chosen by the job author** (can differ from the map count). To guarantee that pairs sharing a key meet at one reducer, the framework picks the reducer from a **hash of the key** (as in "Partitioning by Hash of Key", Chapter 6).
4. **Sorting in stages.** The data is too big to sort on one machine. Every map task **splits its output into one partition per reducer** (via key hash) and stores each partition as a **sorted file on its own local disk**, much like SSTables/LSM-trees do.
5. **Fetching.** When a mapper finishes, the scheduler tells the reducers they can fetch from it. Each reducer connects to every mapper and downloads the sorted file for its own partition.
6. **The shuffle.** Steps 4–5 (partition by reducer, sort, copy to reducers) are called the **shuffle**. Common confusion: unlike shuffling cards, there is **no randomness**.
7. **Merge.** The reducer merges the files from all mappers, keeping sort order, so equal keys from different mappers end up adjacent.
8. **Reduce.** For each key, the reducer gets an iterator that streams through that key's records bit by bit (they might not all fit in memory). It can run arbitrary logic and emit any number of records.
9. **Write output.** Output goes to the distributed filesystem: usually one copy on the reducer's local disk plus replicas on other machines.

MapReduce workflows

- One job can do only so much. In the log example, one job can count views per URL but cannot also rank them, because ranking needs a second sort.
- So jobs are chained into **workflows**: output of one job = input of the next.
- Hadoop MapReduce has **no built-in workflow support**. The chaining is implicit, via **directory names**: job 1 writes to a chosen HDFS directory, job 2 is configured to read that directory. The

framework sees two unrelated jobs.

- Chained MapReduce jobs therefore behave less like a Unix pipe (direct, small buffer) and more like commands that each write a **temporary file** read by the next command. (Pros/cons are discussed under "Materialization of Intermediate State".)
- A job's output counts as valid **only if the job fully succeeds**; MapReduce throws away partial output of failed jobs. So a job may start only after all jobs producing its inputs have succeeded.
- **Workflow schedulers** for Hadoop handle these dependencies: **Oozie, Azkaban, Luigi, Airflow, Pinball**. They also help manage large collections of jobs.
- Scale: recommendation systems commonly use workflows of **50 to 100 MapReduce jobs**, and in big organizations many teams run jobs that read each other's outputs, so tooling matters.
- Higher-level tools on Hadoop (**Pig, Hive, Cascading, Crunch, FlumeJava**) also build multi-stage MapReduce workflows and wire the stages together automatically.

Reduce-Side Joins and Grouping

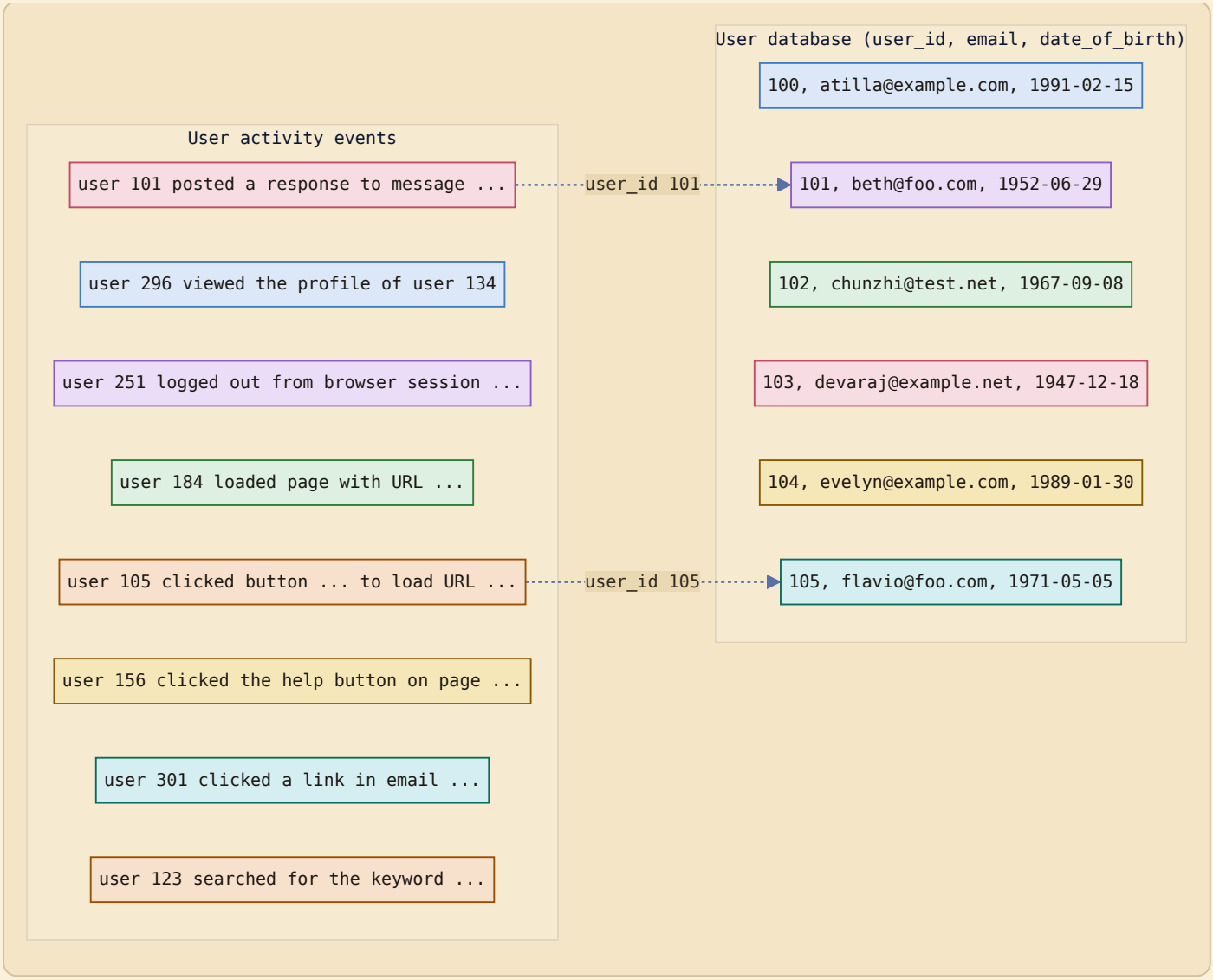
Chapter 2 discussed joins at the data-model level; now we see how they are actually executed.

- Records often reference others: a **foreign key** (relational), a **document reference** (document model), or an **edge** (graph model). You need a **join** whenever code must see both sides of such a link. Denormalization reduces joins but usually cannot eliminate them.
- In a database, a query touching few records uses an **index** to find them (maybe several index lookups for a join). MapReduce has **no indexes** in the usual sense.
- A MapReduce job reads **all** of its input files, which a database would call a **full table scan**. That is absurdly expensive if you only want a few records, but for analytics (aggregating over lots of records) scanning everything is reasonable, especially when parallelized across machines.
- So in batch processing, a "join" means resolving **every** occurrence of an association across the whole dataset (e.g., for all users at once), not looking up one user (which an index does far better).

> **Footnote: equi-joins.** The joins in the book are generally **equi-joins**: records match when a field has an identical value (like an ID). Some databases support more general joins (e.g., using less-than instead of equality), but the book does not cover them.

Example: analysis of user activity events

Figure 10-2 (redrawn): on the left, a log of user activity events (each mentions only a user ID); on the right, a user database table with user_id, email and date_of_birth. The arrows show two events being matched to their user rows (values as in the book's figure).



- Left: a log of what logged-in users did on a website (**activity events** / **clickstream data**). Right: the users database.
- In star-schema terms (Chapter 3), the event log is the **fact table** and the user DB is one of the **dimensions**.
- Goal: correlate activity with profile data, e.g., using date of birth to find which pages are popular with which age groups.
- Events carry only the user ID; copying the full profile into every event would be wasteful. So you must join.

Naive approach and why it fails: loop over events and query a remote user DB for each user ID.

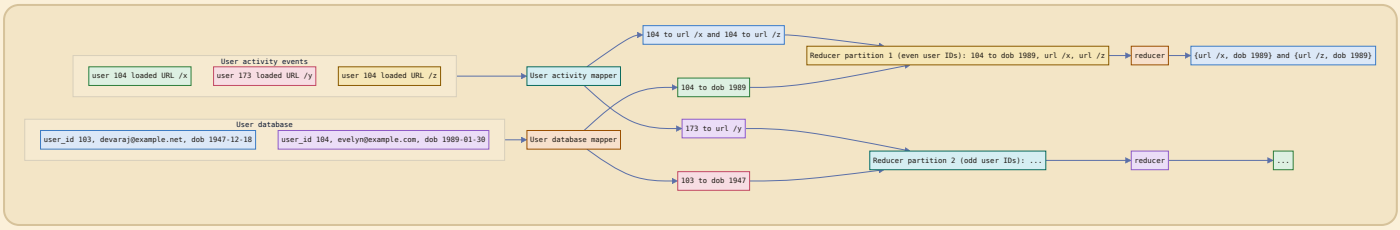
- Throughput is capped by the **round-trip time** to the DB server.
- A local cache helps only if the data distribution is favorable.
- Running many queries in parallel could easily **overwhelm the database**.
- Also, the job becomes **nondeterministic**, because the remote DB can change while the job runs.

Better approach: keep the computation local to one machine as much as possible. Copy the user database into the distributed filesystem (e.g., extract it from a backup via **ETL**, see "Data Warehousing"), alongside the event logs. Now both datasets are files in HDFS and MapReduce can bring matching records together efficiently.

Sort-merge joins

Figure 10-3 (redrawn): joining on user ID in the reducers via sort-merge, using the book's example values. The activity mapper turns each event into (user ID, URL); the user-database mapper turns

each profile into (user ID, date of birth). Partitioning sends even-numbered user IDs to reducer partition 1 and odd-numbered ones to partition 2. Partition 1's reducer sees user 104's date of birth first, then both URLs, and outputs one joined record per URL. (If an input is split across several files, several mappers can process it in parallel.)



How it works:

1. One set of mappers reads the **activity events**, emitting key = user ID, value = the event.
2. Another set reads the **user database**, emitting key = user ID, value = date of birth. (If inputs are split across many files, each file can have its own mapper, running in parallel.)
3. The framework partitions by key and sorts, so the user record and all events for the same user ID become **adjacent** in the reducer's input.
4. With a **secondary sort**, the job can arrange that the reducer always sees the **user record first**, then the events in **timestamp order**.
5. The reducer is called once per user ID. It keeps the date of birth in a variable, then loops over the events, emitting pairs like (viewed URL, viewer age in years).
6. Later jobs could compute the age distribution per URL and cluster by age group.

Why it is efficient: the reducer holds **one user record in memory** at a time and makes **no network requests**. The name **sort-merge join** comes from sorting mapper output by key and then merging the sorted lists from both join inputs in the reducer.

Own illustrative code (MapReduce pseudocode):

```
def event_mapper(event):                    # input 1: activity events
    yield (event['uid'], ('E', event['ts'], event['url']))

def user_mapper(user):                      # input 2: user profiles
    yield (user['uid'], ('U', user['birth_year']))

def join_reducer(uid, values):              # values arrive user-record-first (secondary sort)
    birth_year = None
    for v in values:
        if v[0] == 'U':
            birth_year = v[1]               # remember the one profile field we need
        elif birth_year is not None:
            _, ts, url = v
            yield (url, 2021 - birth_year)  # emit (page, approximate viewer age)
```

- Both mappers use the user ID as key, so matching records meet in one reducer call.
- The tag `'U'` / `'E'` tells the reducer which input a value came from.
- Only one profile is held in memory at a time.

Bringing related data together in the same place

- The mappers plus the sort guarantee that everything needed to join one user ID lands in **one reducer call**. The reducer can then be simple, single-threaded code with high throughput and low memory use.
- Mental model: mappers **"send messages"** to reducers. The emitted key acts like a **destination address**: not an IP and port, just a string, but all pairs with the same key get delivered to the same place.

- MapReduce thereby **separates network communication** (getting data to the right machine) **from application logic** (processing it). Contrast with typical database use, where a fetch is often buried deep inside application code.
- Because the framework handles all network traffic, it also shields your code from **partial failures** (e.g., another node crashing): failed tasks are retried transparently without affecting your logic.

GROUP BY

Grouping (like SQL `GROUP BY`) uses the same "bring related data together" pattern. After grouping, you typically aggregate per group, e.g.:

- Count records per group (like the page-view count; `COUNT(*)` in SQL).
- Sum a field (`SUM(fieldname)`).
- Pick the top *k* records by some ranking.

Implementation: have mappers use the **grouping key** as the output key; partitioning and sorting bring each group to one reducer. So on MapReduce, **grouping and joining look very similar**.

Own SQL example of the same idea:

```
SELECT url, COUNT(*) AS views      -- one output row per distinct URL
FROM page_views                    -- the input "table" (log records)
GROUP BY url;                      -- in MapReduce: mapper emits url as the key
```

Sessionization: grouping all events of one user session to recover the sequence of actions. Uses: deciding whether users who saw a new site version buy more than those who saw the old one (**A/B testing**), or whether a marketing campaign is worthwhile. With many web servers, one user's events are scattered across many servers' logs; use a session cookie, user ID, or similar as the grouping key, so one user's events meet in one place while different users are spread over partitions.

Handling skew

- "All records with the same key go to one place" breaks down when **one key has a huge amount of data**. Example: most social network users have a few hundred connections, but a few **celebrities** have millions of followers. Such records are called **linchpin objects** or **hot keys**.
- Sending all activity for a celebrity (e.g., replies to their posts) to one reducer causes **skew** (a **hot spot**): one reducer does far more work than the rest.
- A MapReduce job finishes only when **all** mappers and reducers finish, so downstream jobs wait for the slowest reducer.

Remedies the book lists:

Technique	How it works	Cost / requirement
Pig skewed join	First runs a **sampling job** to find hot keys. During the join, mappers route each hot-key record to **a randomly picked reducer out of several** (instead of deterministically by hash). Records from the *other* join input with that key are **replicated to all** those reducers.	Spreads work over several reducers; cost is replicating the other input
Crunch sharded join	Same idea as Pig's	Hot keys must be **specified explicitly**, no sampling job
Hive skewed join optimization	Hot keys are declared in **table metadata**; their records are stored in **separate files**. At join time, a **map-side join** is used for the hot keys	Requires explicit declaration
Two-stage aggregation (for grouping)	Stage 1 sends records to a **random reducer**; each reducer groups its subset of the hot key and outputs a compact partial aggregate. Stage 2 job combines the partial values into one value per key	Extra job

The random-reducer trick is the same as the randomization used to relieve hot spots in partitioned databases (Chapter 6, "Skewed Workloads and Relieving Hot Spots").

Map-Side Joins

The joins above do the join logic in the reducers, so they are **reduce-side joins**. Mappers just prepare data: extract key and value, assign a reducer partition, sort.

	Reduce-side join	Map-side join
Assumptions about input	**None**: works whatever the input's structure	Requires known properties (size, partitioning, sorting)
Cost	Sorting, copying to reducers, and merging can be **expensive**; depending on buffer sizes, data may hit disk multiple times along the way	Faster: no reducers and no sorting
Structure	Mappers + shuffle + reducers	Cut-down job: each mapper reads one input block and writes one output file

Broadcast hash joins

When: a **large** dataset joined with a **small** one that fits entirely **in memory** in each mapper.

How it works:

1. Suppose the user database in Figure 10-2 is small.
2. When each mapper starts, it loads the **entire** user DB out of HDFS and builds an **in-memory hash table** from it.
3. It then scans its block of activity events, looking up each event's user ID in the hash table.
4. There is one map task per block of the **large** input; every one of them loads the full small input.

Name: **broadcast** because the small input is effectively sent to every partition of the large input; **hash** because of the hash table. Supported by **Pig** ("replicated join"), **Hive** ("MapJoin"), **Cascading**, **Crunch**, and data warehouse engines like **Impala**.

> **Footnote.** The example assumes one hash table entry per key (true for users: an ID identifies one user). In general the hash table may hold several entries per key, and the join outputs every match.

Variant: instead of an in-memory hash table, store the small input as a **read-only index on local disk**. The hot parts stay in the OS **page cache**, so lookups are almost as fast as in memory, without needing the data to fit in RAM.

Partitioned hash joins

When: both inputs are **partitioned the same way**.

Example: partition both the activity events and the users by the **final digit (base 10) of each user's ID**, giving 10 partitions on each side. **Mapper 3** loads all users whose ID ends in 3 into a hash table, then scans all events for users whose IDs end in 3.

- If partitioning is correct, every pair of records that could join sits in the same-numbered partition, so each mapper reads just one partition of each input, and its hash table is smaller.
- Requirement: both inputs have the **same number of partitions**, assigned by the **same key** and **same hash function**. This is reasonable if the inputs came from earlier MapReduce jobs that already grouped them that way.
- In **Hive** these are called **bucketed map joins**.

Map-side merge joins

When: inputs are partitioned the same way **and sorted by the same key**.

- Now it does not matter whether either input fits in memory: the mapper can do the merge a reducer would do, reading both files incrementally in ascending key order and matching equal keys.
- If this is possible, earlier jobs probably produced this partitioned, sorted layout. In principle the join could have happened in that earlier job's reduce stage. But a separate map-only join can still make sense, e.g., when other jobs also want those partitioned, sorted datasets.

MapReduce workflows with map-side joins

- The join choice affects the **shape of the output** for downstream jobs:
  - A reduce-side join produces output **partitioned and sorted on the join key**.
  - Map-side join output is **partitioned and sorted like the large input** (one map task per block of the large input, whether broadcast or partitioned join).
- Map-side joins assume more about input size, sorting, and partitioning. So when optimizing joins, you must know the **physical layout** of datasets: not just encoding format and directory name, but also **number of partitions** and the **keys** used for partitioning and sorting.
- In Hadoop this partitioning metadata is often kept in **HCatalog** and the **Hive metastore**.

The Output of Batch Workflows

What is all this processing *for*? Recall (Chapter 3):

- **OLTP** queries look up a few records by key via indexes to show to a user.
- **Analytic** queries scan many records, group and aggregate, and produce **reports** (a metric over time, top 10 items, breakdowns) for analysts or managers making business decisions.

Batch processing is **neither** exactly. It is closer to analytics (it scans big chunks of input), but a MapReduce workflow is not the same as an analytic SQL query, and what it produces is frequently **some other kind of structure rather than a report**.

Building search indexes

- Google's **original** use of MapReduce was building its search engine's indexes, implemented as **5 to 10 chained MapReduce jobs**. Google later moved away from MapReduce for this, but the example still illuminates MapReduce. **Hadoop MapReduce remains a good way to build indexes for Lucene/Solr.**
- Recap of a full-text index like **Lucene** (Chapter 3): a file (the **term dictionary**) where you look up a keyword and get the list of document IDs containing it (the **postings list**). Real indexes

also need extra data for relevance ranking, spelling correction, synonyms, etc., but the principle holds.

- Building it in batch for a fixed document set: mappers **partition the documents**, every reducer **builds an index over its own partition**, and the resulting index files land in HDFS. Such **document-partitioned** indexes (Chapter 6) parallelize very well.
- Since searching is read-only, index files are **immutable** once built.
- When documents change, two options:

Option	How	Pros	Cons
Periodic full rebuild	Rerun the whole indexing workflow on all docs; swap the old index files for the new ones wholesale	Simple mental model: feed in documents, get out indexes	Expensive if only a few documents changed
Incremental	Lucene writes new segment files and merges/compacts them in the background (Chapter 3)	Efficient for small changes	More complex; more on incremental processing in Chapter 11

Key-value stores as batch process output

- Another common output: **machine learning** systems like **classifiers** (e.g., spam filtering, anomaly detection, image recognition) and **recommendation systems** (people you may know, products you may like, related searches).
- These jobs usually output a **database**, e.g., queried by user ID to get suggested friends, or by product ID to get related products.
- The web app that serves users usually runs **separately** from the Hadoop cluster. How does batch output reach a database the app can query?

Bad idea: write directly from mappers/reducers to the production database, one record at a time. It works (if firewalls allow it), but:

1. A network request **per record** is orders of magnitude slower than normal batch throughput; client-side batching still performs poorly.
2. Many parallel tasks writing at batch rates can **overwhelm the database**, hurting its query performance and causing operational problems elsewhere.
3. It breaks MapReduce's **all-or-nothing** guarantee. Normally, a job that succeeds produces output identical to each task having run once, retries notwithstanding, and a failed job produces no output. Writes to an outside system are **side effects others can see**, and the framework cannot hide them, so you must worry about partial results being visible, plus Hadoop **task attempts** and **speculative execution** (duplicate task runs).

Good idea: have the batch job construct a fresh database itself, emitting it as files into the job's HDFS output directory (just like the search index files). The files are immutable and can be **bulk-loaded** into servers that answer read-only queries.

- Key-value stores that support building their files in MapReduce: **Voldemort**, **Terrapin**, **ElephantDB**, and **HBase bulk loading**.
- This suits MapReduce well: extracting a key and sorting by it is already much of the work of building an index.
- Since these stores are read-only (written once by a batch job), their data structures are simple; e.g., they do **not need a WAL** (write-ahead log).
- **Voldemort loading, step by step:**
  1. Server keeps serving requests from the **old** data files.
  2. Meanwhile, the new files are downloaded from HDFS onto the server's own disk.
  3. When copying finishes, the server **atomically switches** to the new files.
  4. If anything goes wrong, it can switch back, since the old files are still there and immutable.

Philosophy of batch process outputs

Unix makes dataflow explicit: read input, write output, leave input untouched, fully replace previous output, no other side effects. So you can rerun freely while debugging. MapReduce output handling follows the same philosophy. Treating inputs as immutable and avoiding side effects (like writing to external DBs) gives good performance **and** maintainability:

- **Recover from bugs easily.** If buggy code produces bad output, roll back the code and rerun; or even simpler, keep the old output in another directory and switch back. Contrast: with read-write transactional DBs, rolling back buggy code does **not** undo the bad data it wrote. This ability to recover from buggy code is called **human fault tolerance**.
- **Faster feature development**, because mistakes are not irreversible. **Minimizing irreversibility** helps Agile development.
- **Safe automatic retries.** If a task fails, the framework reruns it on the same input. A bug makes it crash repeatedly and eventually fail the job after a few tries; a transient fault is tolerated. Retrying is safe only because inputs are immutable and failed tasks' outputs are discarded.
- **Reuse of inputs.** The same files can feed many jobs, including **monitoring jobs** that compute metrics and check whether output looks as expected (e.g., compare with the previous run's output).
- **Separation of logic and wiring** (input/output directories are configuration): one team builds a job that does one thing well; other teams decide where and when to run it.

Where Hadoop differs from Unix: Unix tools assume untyped text and spend effort on parsing (`{print $7}`). Hadoop often avoids that low-value parsing by using structured formats: **Avro** (Chapter 4; e.g., Avro object container files) and **Parquet** (column-oriented storage, Chapter 3), which give efficient schema-based encoding and support schema evolution.

Comparing Hadoop to Distributed Databases

Hadoop is like a distributed Unix: **HDFS** plays the filesystem, and **MapReduce** is an oddly specific process that always runs `sort` between its map and reduce phases. We have built joins and grouping on these primitives.

When the MapReduce paper came out, it was in a sense **not new**: all these parallel join and processing algorithms already existed in **massively parallel processing (MPP) databases** over ten years before; pioneers included **Teradata**, **Tandem NonStop SQL**, and the **Gamma database machine**.

Biggest difference: MPP databases focus on running **analytic SQL in parallel**, whereas MapReduce + a distributed filesystem behaves more like a **general-purpose operating system** able to execute **arbitrary programs**.

Diversity of storage

- Databases make you structure data by a model (relational, document). Distributed filesystem files are **just bytes**, in any model/encoding: DB records, but equally text, images, videos, genome sequences, sensor readings, feature vectors, sparse matrices, or anything else.
- Hadoop made it possible to **dump data into HDFS first and figure out processing later**. MPP databases usually need careful up-front modeling of data and query patterns before import into their proprietary storage format.
- A purist would prefer careful modeling (better quality data), but in practice **making data available quickly**, even in a quirky raw format, is often worth more than agreeing on the ideal model first.
- Similar to a **data warehouse**: just gathering data from across an organization is valuable because it enables joins between previously separate datasets. Careful MPP schema design slows collection; collecting raw data and designing schema later speeds it up. This is called a **data lake** or **enterprise data hub**.

- Raw dumping shifts interpretation to the **consumer** (**schema-on-read**, Chapter 2) rather than forcing the producer to standardize. Useful when the producing and consuming teams have different priorities. There may be no single ideal model, only different views for different purposes; raw data allows several transformations. This is the **sushi principle: "raw data is better."**
- Hence Hadoop is often used for **ETL**: dump raw data from OLTP systems into HDFS, write MapReduce jobs to clean and transform it into relational form, then load the result into an MPP data warehouse where analysts query it. Modeling still happens, but as a **separate step decoupled from collection**, which is possible because the filesystem accepts any format.

Diversity of processing models

	MPP database	Hadoop (MapReduce and beyond)
Architecture	**Monolithic**, tightly integrated: storage layout, query planning, scheduling, execution tuned together	Open platform: many processing models over shared files
Strength	Excellent performance on the queries it was designed for; SQL is expressive and elegant, usable from BI tools like **Tableau** without code	Run **arbitrary code** over large datasets
Weakness	Not everything fits SQL	Raw MapReduce is limited/slow for some tasks, so other models were built on Hadoop

- Tasks that do not fit SQL well: machine learning and recommendation systems, full-text search with relevance ranking, image analysis. These are application-specific (feature engineering for ML, natural language models for translation, risk estimation for fraud prediction) and require **code, not just queries**.
- With HDFS + MapReduce you can build a SQL engine on top (**Hive** did exactly that), and also write many batch jobs that SQL cannot express.
- Later, MapReduce proved too limiting and too slow for some workloads, so more models were developed on Hadoop (see "Beyond MapReduce"). SQL plus MapReduce was not enough; Hadoop's openness made more models feasible, which a monolithic MPP DB would not allow.
- Crucially, all these models can run on **one shared cluster** reading the **same files**. No need to import data into several specialized systems; not moving data around makes it easier to get value and to experiment.
- The Hadoop ecosystem includes random-access OLTP stores like **HBase** (LSM-based) and MPP-style analytic databases like **Impala**. **Neither uses MapReduce, but both store data in HDFS**; very different approaches coexist in one system.

Designing for frequent faults

Two more differences: fault handling and memory/disk use. Batch jobs are less fault-sensitive than online systems: failure does not immediately hurt users, and jobs can be rerun.

	MPP databases	MapReduce
Node crash mid-query	**Abort the whole query**; user resubmits or it reruns automatically. Fine because queries run seconds to minutes	**Retry just the failed map or reduce task**; job as a whole is unaffected
Memory vs disk	Keep as much as possible **in memory** (e.g., hash joins) to avoid disk reads	Very **eager to write to disk**, partly for fault tolerance, partly assuming data will not fit in memory anyway
Best for	Short queries	Huge, long jobs likely to hit at least one task failure, where rerunning everything would be wasteful

Task-level recovery adds overhead that slows the fault-free case, but can be worth it if task failures are frequent enough. But machine failures are actually fairly rare in most clusters, so most jobs will not see one. Is the overhead worth it?

Why MapReduce was designed this way (Google's environment):

1. Google runs **mixed-use datacenters**: online production services and offline batch jobs share machines.
2. Every task is granted a fixed **share of resources** (cores, memory, disk and so on), enforced using **containers**.
3. Each task has a **priority**. If a higher-priority task needs resources, lower-priority tasks on that machine can be **terminated (preempted)**.
4. Priority also sets **price**: teams pay for resources, and higher priority costs more.
5. This lets low-priority resources be **overcommitted** (the system can reclaim them), which improves machine utilization and efficiency compared with segregating production and non-production work.
6. MapReduce runs at low priority, so it can be preempted at any time. In the book's words, batch jobs "pick up the scraps under the table": they use whatever resources the high-priority processes leave.

The numbers: at Google, a MapReduce task running for **one hour** has about a **5% risk of being terminated** to make room for higher-priority work. That rate exceeds failures from hardware, reboots, and other causes by **more than 10x (an order of magnitude)**. At that rate, a job with **100 tasks of 10 minutes each** has a **greater than 50% risk** that at least one task gets killed before finishing.

So: MapReduce tolerates frequent unexpected task termination **not because hardware is unreliable**, but because the freedom to kill processes enables better cluster resource utilization.

Open source schedulers use preemption less. **YARN's CapacityScheduler** supports preemption for balancing resources between queues, but **general priority preemption** was missing from **YARN, Mesos, and Kubernetes** when the book was written. Where tasks are rarely killed, MapReduce's design choices make less sense, which motivates the alternatives in the next section.

Beyond MapReduce

- MapReduce got huge hype in the late 2000s, but it is only one programming model. Depending on data volume, structure, and processing type, other tools may fit better.
- The book spent so long on MapReduce because it is a **clear, simple abstraction** over a distributed filesystem: simple to *understand*, not simple to *use*. Writing complex jobs with raw MapReduce APIs is hard; e.g., you must implement join algorithms yourself.
- Higher-level models (**Pig, Hive, Cascading, Crunch**) were built on top to make common tasks easier; they are easy to learn if you understand MapReduce.
- But some problems are in the **execution model itself** and cannot be fixed by adding abstraction layers; they show up as poor performance. MapReduce is **very robust** (it will finish almost arbitrarily large jobs on unreliable, multi-tenant clusters with frequent task kills, just slowly), while other tools can be **orders of magnitude faster** for some workloads.
- Chapter 11 will treat stream processing as another way to speed up batch processing.

Materialization of Intermediate State

- Each MapReduce job is **independent**; it touches the world only via its input and output directories. To chain jobs, you point job 2's input at job 1's output, and an external scheduler starts job 2 after job 1 finishes.
- That is sensible if job 1's output is a dataset you want to **publish widely** in the organization: others can refer to it by name and reuse it. Publishing to a well-known location gives **loose coupling** (jobs need not know who produces or consumes their data).

- Frequently, though, a job's output feeds exactly **one other job**, owned by the **same team**. Then the files are just **intermediate state**, a way to pass data along. The 50–100-job recommendation workflows have lots of it.
- Writing intermediate state out to files is **materialization**: eagerly computing a result and writing it out, rather than computing on demand. (Same term as in materialized views, Chapter 3.)
- Unix pipes do **not** fully materialize; they stream output to input incrementally through a small buffer.

Downsides of MapReduce fully materializing intermediate state (vs pipes):

1. **Waiting for stragglers.** A job can start only when **all** tasks of the jobs producing its input are done; piped processes start together and consume output as soon as it appears. Because of skew or uneven machine load, jobs often have a few **straggler** tasks, and waiting for them slows the whole workflow.
2. **Redundant mappers.** Mappers often just re-read the file a reducer just wrote and prepare it for the next partition-and-sort. If the reducer output were already partitioned and sorted like mapper output, reducers could be chained directly without mapper stages in between.
3. **Over-replication.** Intermediate files in a distributed filesystem are **replicated across several nodes**, which is overkill for temporary data.

Dataflow engines

To fix these problems, new engines for distributed batch computation appeared; the best known are **Spark**, **Tez**, and **Flink**. Design details differ, but they share one idea: **handle the entire workflow as a single job** instead of independent subjobs.

- Because they explicitly model data flowing through several stages, they are called **dataflow engines**.
- Like MapReduce, they invoke a user-defined function over and over, **one record per call, single-threaded**, parallelize by **partitioning inputs**, and copy one function's output over the network as another's input.
- Unlike MapReduce, functions need not alternate strictly as map then reduce. They are general **operators**, and there are several ways to connect one operator's output to the next:
  1. **Repartition and sort by key**, like MapReduce's shuffle; enables sort-merge joins and grouping.
  2. **Partition several inputs the same way but skip sorting**; saves work for partitioned hash joins, where partitioning matters but order does not (building a hash table scrambles order anyway).
  3. **Broadcast**: send the same output to all partitions of the join operator, for broadcast hash joins.
- Based on research systems **Dryad** and **Nephele**.

Advantages over MapReduce:

- **Sort only where needed**, not always between every map and reduce.
- **No unnecessary map tasks**: a mapper's work can usually be folded into the previous reduce operator (a mapper does not change partitioning).
- **Locality optimizations**: all joins and data dependencies are declared, so the scheduler knows what data is needed where. It can place a consumer task on the same machine as the producer, exchanging data via a **shared memory buffer** instead of the network.
- **Intermediate state in memory or on local disk** is usually enough, much less I/O than HDFS (replicated and written to disk on each replica). MapReduce already did this for mapper output; dataflow engines generalize it to all intermediate state.
- **Operators start as soon as their input is ready**, without waiting for the whole previous stage.
- **Reuse JVM processes** for new operators, reducing startup cost versus MapReduce, which launches a **new JVM per task**.

Result: the same computations as MapReduce workflows, usually **significantly faster**. Since operators generalize map and reduce, the same code runs on either engine: **Pig, Hive, or Cascading workflows can move from MapReduce onto Tez or Spark just by changing configuration**, without touching code.

- **Tez** is a fairly **thin library** relying on the **YARN shuffle service** to copy data between nodes.
- **Spark and Flink** are **big frameworks** with their own network layer, scheduler, and user-facing APIs.

Fault tolerance

- Fully materializing to a distributed filesystem makes MapReduce fault tolerance easy: the data is durable, so a failed task just restarts elsewhere and re-reads the same input.
- Spark, Flink, and Tez **keep intermediate state out of HDFS**. If a machine dies and its intermediate state is lost, they **recompute** it from data still available: a prior intermediate stage if possible, else the original input (usually on HDFS).
- To recompute, the framework has to remember **the recipe for each piece of data**: which input partitions it came from and which operators were applied.
  - **Spark:** the **resilient distributed dataset (RDD)** abstraction tracks data **ancestry**.
  - **Flink: checkpoints operator state**, so it can resume an operator that failed mid-run.

Determinism matters. If lost data was already sent downstream, and the recomputed version differs, downstream operators face contradictions between old and new data. For nondeterministic operators the usual fix is to **kill the downstream operators too** and rerun them on the new data. To avoid such cascading faults, make operators **deterministic**. Nondeterminism creeps in easily:

- Many languages do **not guarantee iteration order** over hash tables.
- Many probabilistic/statistical algorithms use **random numbers** (fix: pseudorandom numbers with a **fixed seed**).
- Reading the **system clock** or consulting **external data sources**.

Recomputation is not always best: when the intermediate result is **far smaller** than its source, or producing it **burns a lot of CPU**, writing it to files is likely cheaper than redoing it.

Discussion of materialization

- Unix analogy: MapReduce = writing each command's output to a temp file; dataflow engines ≈ **Unix pipes**. **Flink** especially is built on **pipelined execution**: passing output incrementally to the next operators without waiting for complete input.
- **Sorting is an exception**: it must consume its entire input before emitting anything, because the very last input could have the smallest key and need to come out first. Any operator needing a sort must accumulate state at least temporarily. Most other stages, though, can be pipelined.
- When the job finishes, output must go somewhere **durable** so users can find it, usually the distributed filesystem again. So with dataflow engines, materialized HDFS datasets are still usually the **inputs and final outputs**; inputs are immutable and output is completely replaced, as with MapReduce. The win is that the **intermediate** results no longer have to be written to the filesystem.

	MapReduce workflow	Dataflow engine (Spark/Tez/Flink)
Unit of scheduling	Independent jobs chained by directories	Whole workflow as one job (DAG of operators)
Intermediate state	Written to HDFS, replicated	In memory or local disk
Sorting	Always between map and reduce	Only where needed
Start of next stage	After all preceding tasks finish	As soon as input is ready (pipelined)
Fault recovery	Re-run failed task from durable input	Recompute lost data from lineage (Spark RDD) or checkpoints (Flink)
Process startup	New JVM per task	Reuse JVMs

Graphs and Iterative Processing

- Chapter 2 discussed graphs for **OLTP**-style queries (quickly find a few matching vertices). Here: **batch** analysis over an **entire graph**, common in ML (recommendation engines) and ranking.
- Famous example: **PageRank**, which estimates a web page's popularity from which other pages link to it; used as part of how web search engines order results.

> **Naming confusion warning.** Dataflow engines like Spark, Flink, and Tez arrange their operators as a **directed acyclic graph (DAG)**. That is **not** graph processing: in a dataflow engine the *flow between operators* is a graph while the records are typically relational-style tuples; in graph processing, by contrast, the *data* is the graph.

- Many graph algorithms move **one edge at a time**, joining a vertex with a neighbor to pass information along, and **repeat until done** (no more edges to follow, or some metric **converges**). Chapter 2's Figure 2-6 example (listing all locations within North America by repeatedly following "within" edges) is a **transitive closure**.
- You can store a graph in a distributed filesystem (files of vertices and edges), but plain MapReduce does a **single pass** and cannot express "repeat until done". So it is done **iteratively**:
  1. An **external scheduler** launches a batch job that computes one step of the algorithm.
  2. When it finishes, the scheduler checks the **completion condition** (no more edges to follow, or change since last iteration below a threshold).
  3. If not finished, go back to step 1 for another round.
- This works but is **very inefficient in MapReduce**: each iteration reads the **whole input** and writes a **completely new output**, even when very little of the graph changed since the last round.

The Pregel processing model

- An optimization for batch graph processing: the **bulk synchronous parallel (BSP)** model. Implementations include **Apache Giraph**, the **GraphX** API in Spark, and the **Gelly** API in Flink. Also called the **Pregel model** after Google's Pregel paper that popularized it.
- Analogy with MapReduce: mappers "send messages" to reducer calls via keys. In Pregel, **one vertex sends a message to another vertex**, usually along the graph's edges.

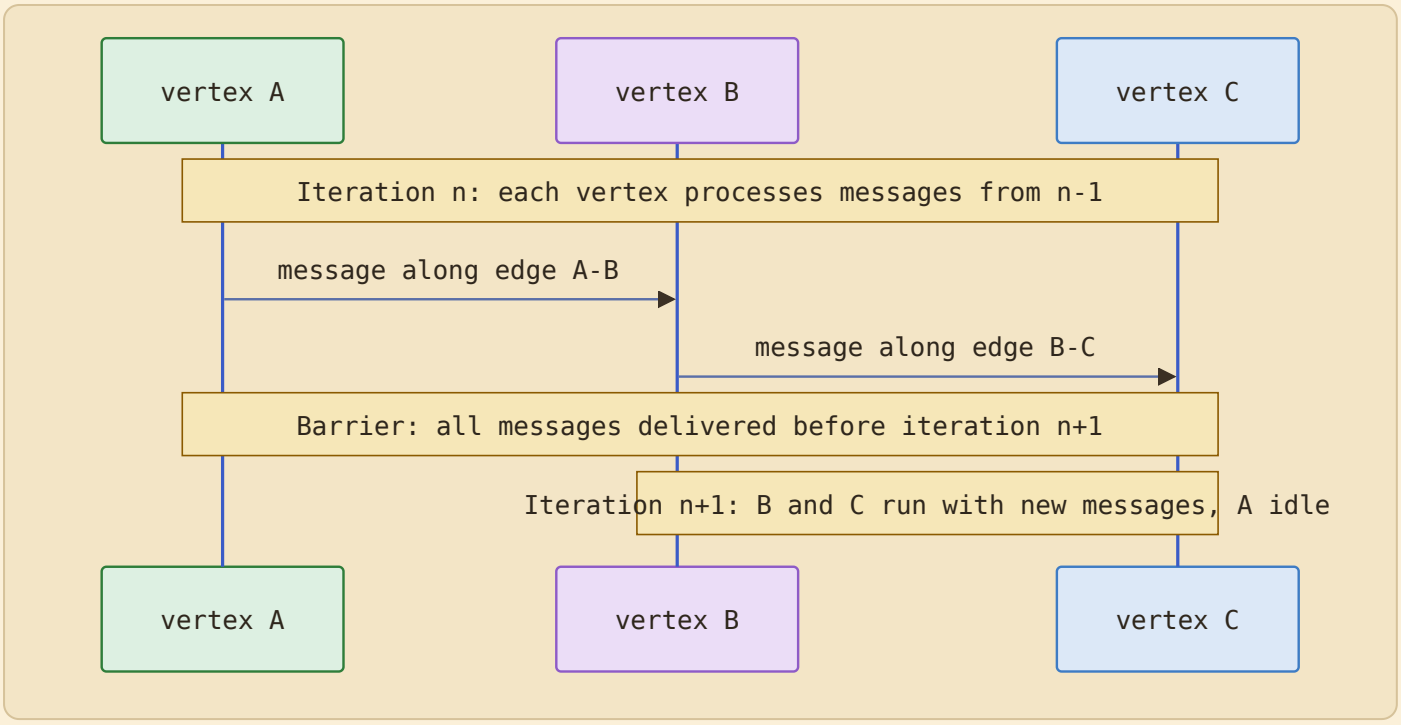
How it works:

1. The computation proceeds in **iterations** (rounds).
2. Every iteration invokes a function **once per vertex**, passing in the messages addressed to that vertex (similar to a reducer call).

3. Unlike MapReduce, a vertex **remembers its state in memory** between iterations, so the function processes only **new** messages.
4. Regions of the graph where no messages are flowing need **no work** at all.
5. Between rounds, the framework hands over every message produced in the round before.

Comparison with the **actor model** (Chapter 4): picture each vertex as an actor, with the difference that vertex state and inter-vertex messages are **durable and fault-tolerant**, and communication happens in **fixed rounds**. Actors normally have no such timing guarantee.

Illustrative sketch (no numbered figure in the book): one Pregel superstep.



Own illustrative code (vertex-centric pseudocode, shortest distance from a source):

```
def compute(vertex, messages):                # called once per vertex per iteration
    best = min(messages, default=vertex.value)
    if best < vertex.value or vertex.first_round:
        vertex.value = best                   # state is kept in memory across rounds
        for edge in vertex.out_edges:
            send(edge.target, best + edge.weight)   # message delivered next round
    vertex.vote_to_halt()                     # inactive until a new message arrives
```

- Each call sees only this vertex and its incoming messages ("thinking like a vertex").
- State (`vertex.value`) persists between rounds, so only new messages cause work.
- `send` addresses a vertex ID; the framework routes it.

Fault tolerance (Pregel)

- Vertices communicate **only by messages**, never by querying each other directly. That boosts performance: messages can be sent in **batches** and less time is spent waiting.
- The only waiting is **between iterations**: since all messages from one iteration must be delivered in the next, the previous iteration must fully finish and its messages be copied over the network first.
- Even though the network may drop, duplicate, or delay messages arbitrarily (Chapter 8), Pregel implementations guarantee each message is processed **exactly once** by its target vertex in the next iteration. As with MapReduce, fault recovery is invisible to the algorithm.

- Mechanism: **periodically checkpoint** all vertex state at the **end of an iteration** (write it fully to durable storage). If a node dies and loses its in-memory state, the easy fix is to **roll the entire graph computation back** to the most recent checkpoint and continue from there. If the algorithm is **deterministic** and **messages are logged**, you can recover **only the lost partition** (as with dataflow engines).

Parallel execution

- A vertex does not know which machine it runs on; it just sends messages to a **vertex ID**. The framework partitions the graph (which vertex runs where) and routes messages.
- Because code deals with one vertex at a time (**"thinking like a vertex"**), the framework is free to split the graph any way it likes. Ideally, vertices that talk a lot would be colocated, but finding such a partitioning is hard, so graphs are often partitioned simply by an **arbitrarily assigned vertex ID**, with no attempt to group related vertices.
- Consequence: lots of **cross-machine communication**, and the intermediate state (messages) is often **bigger than the original graph**. Network overhead can significantly slow distributed graph algorithms.

Practical advice from the book:

Graph size	Best approach
Fits in memory on one machine	A **single-machine** algorithm, possibly even a single-threaded one, will likely outperform a distributed batch job
Bigger than memory but fits on one machine's disks	Single-machine frameworks like **GraphChi** are viable
Too big for one machine	Distributed approach like **Pregel** is unavoidable; efficient parallel graph algorithms are an area of ongoing research

High-Level APIs and Languages

- Since MapReduce first got popular, execution engines have matured: infrastructure can now store and process **many petabytes** on clusters of **over 10,000 machines**. With operating at that scale considered more or less solved, attention has moved to: a better **programming model**, more **efficient processing**, and a **wider range of problems**.
- **Hive, Pig, Cascading, Crunch** became popular because hand-writing MapReduce is laborious. When **Tez** appeared, these languages could move to the new engine **without rewriting job code**. **Spark and Flink** have their own high-level dataflow APIs, often inspired by **FlumeJava**.
- These APIs use **relational-style building blocks**: join datasets on a field, group tuples by key, filter by condition, aggregate by counting/summing/etc. Internally they use the join and grouping algorithms from this chapter.
- Benefits:
  - **Less code**.
  - **Interactive use**: write analysis code bit by bit in a shell and run it often to see what it does. Great for exploring data, and reminiscent of the Unix philosophy.
  - Not only human productivity: they also improve **machine-level execution efficiency** (next subsections).

Own illustrative high-level dataflow example (PySpark-style):

```
views = spark.read.parquet("hdfs:///logs/views")           # large input
users = spark.read.parquet("hdfs:///snapshots/users")      # small input
by_age = (views.join(users, "uid")                         # declarative join: engine picks algorithm
               .groupBy("url", "age_band")                 # grouping by key
               .count())                                   # aggregation
by_age.write.parquet("hdfs:///derived/views_by_age")       # final output, materialized
```

- Only *what* to join and group is stated; the optimizer may choose a broadcast hash join if `users` is small.
- Parquet is columnar, so only the needed columns are read.

The move toward declarative query languages

- Expressing joins as **relational operators** (rather than code that performs the join) lets the framework inspect join inputs and **choose the best join algorithm** automatically. **Hive, Spark, and Flink** ship **cost-based query optimizers** capable of this and can even **reorder joins** to minimize intermediate state.
- Join choice hugely affects performance, and it is convenient not having to memorize every algorithm. That is possible when joins are **declarative**: the app says which joins it needs, the optimizer decides how (idea from "Query Languages for Data", Chapter 2).
- But MapReduce and its successors differ from fully declarative SQL: they are built on **function callbacks**. For each record or group, a user-defined function is called and may run **arbitrary code**. Advantage: you can use the huge ecosystem of libraries for parsing, natural language analysis, image analysis, numerical/statistical algorithms.
- This freedom to run arbitrary code has long distinguished MapReduce-heritage systems from **MPP databases**. Databases do have **user-defined functions**, but they are often cumbersome and poorly integrated with language package managers and dependency systems (**Maven** for Java, **npm** for JavaScript, **Rubygems** for Ruby).
- Still, dataflow engines found value in **more declarative features beyond joins**:
  - If a callback just filters or selects fields, calling a function per record costs significant **CPU overhead**. Expressed declaratively, the optimizer can exploit **column-oriented storage** and read only needed columns from disk.
  - **Hive, Spark DataFrames, and Impala** use **vectorized execution**: tight inner loops friendly to CPU caches, avoiding function calls. **Spark generates JVM bytecode**; **Impala uses LLVM** to emit native machine code for them.
- Net effect: with declarative high-level APIs plus optimizers, batch frameworks **look more like MPP databases** (with comparable performance), while keeping their **flexibility**: run arbitrary code, read arbitrary formats.

Specialization for different domains

- Arbitrary code is useful, but common patterns recur, so reusable building blocks are worthwhile. MPP databases traditionally serve **business intelligence** and reporting, which is just one domain.
- **Statistical and numerical algorithms** for ML (classification, recommendation) are increasingly important. Reusable implementations: **Mahout** (ML on MapReduce, Spark, Flink) and **MADlib** (comparable features running within **Apache HAWQ**, a relational MPP database).
- **Spatial algorithms** like **k-nearest neighbors**: find items near a given item in a multi-dimensional space (similarity search).
- **Approximate search** for **genome analysis**: find strings that closely resemble each other without being exact matches.
- Batch engines are used for an ever-wider range of domains. As batch systems gain built-in functionality and declarative operators, while MPP databases grow more flexible and programmable, the two **converge**: in the end, both are just systems for storing and processing data.

Chapter summary

- **Unix roots.** Tools like `awk`, `grep`, `sort` show principles that carried into MapReduce and dataflow engines: **immutable inputs**, outputs meant to feed an **unknown next program**, and solving problems by **composing small tools** that each do one thing well.
- **Uniform interface.** Unix: files and pipes. MapReduce: the **distributed filesystem**. Dataflow engines add pipe-like transport to avoid materializing intermediate state, but a job's initial input and final output are still usually on HDFS.
- **Two core problems** of distributed batch frameworks:
  - **Partitioning.** MapReduce: mappers per input block; mapper output is **repartitioned, sorted, and merged** into however many reducer partitions the author configured, to bring related data (same key) together. Dataflow engines avoid sorting unless needed but partition similarly.
  - **Fault tolerance.** MapReduce writes to disk often, so a single failed task is easy to recover but the fault-free case is slower. Dataflow engines materialize less and keep more in memory, so they recompute more after a failure; **deterministic operators** reduce recomputation.
- **Join algorithms** (also used inside MPP databases and dataflow engines):
  - **Sort-merge join:** each input's mapper extracts the join key; partition, sort, merge bring matching records to one reducer call, which outputs joined records.
  - **Broadcast hash join:** small input loaded into a hash table in each mapper of the large input; scan large input and probe.
  - **Partitioned hash join:** both inputs partitioned identically (same key, hash function, partition count); hash join per partition.
- **Restricted programming model = hidden distributed-systems problems.** Callbacks are assumed **stateless** with **no external side effects** beyond their output. So tasks can be **retried safely**, failed tasks' output discarded, and if several attempts of a partition succeed, only one output becomes visible. The final output equals a fault-free run, which is far stronger than what typical online services offer when they update databases as a by-product of serving requests.
- **Defining feature of batch:** it reads input and produces **derived** output without modifying the input, and the input is **bounded** (known, fixed size, e.g., log files at a point in time or a DB snapshot). So the job knows when it has read everything and eventually **finishes**.
- **Next:** stream processing, where input is **unbounded**, so a job is never complete; similar in some ways, but that assumption changes a lot.

Interview angle

1. Explain MapReduce in 30 seconds. A batch framework over a distributed filesystem. You write a *mapper* that turns each input record into key-value pairs and a *reducer* that processes all values for one key. The framework runs mappers near the data (one per input block), partitions their output by hash of key, sorts it, ships each partition to its reducer (the shuffle), merges, and calls the reducer per key. Output is written to HDFS. Inputs are never modified, so failed tasks can be retried safely.

- *Follow-up: Why is there always a sort?* Sorting makes equal keys adjacent so the reducer can process a key's values with little memory; it also scales past RAM using external merge sort with sequential I/O.
- *Follow-up: What is the shuffle?* Partitioning mapper output by reducer, sorting it on the mapper's local disk, and having reducers fetch and merge their partition from every mapper. No randomness.

2. How do you join two large datasets in a batch job? Default: a **reduce-side sort-merge join**: map both inputs to (join key, record), let the shuffle bring matching keys together, optionally secondary-sort so one side arrives first, and join in the reducer. If one side is small, use a

broadcast hash join (load it into every mapper's hash table). If both are already partitioned identically, a **partitioned hash join**; if also sorted, a **map-side merge join**. Never query a remote database per record: too slow, overloads the DB, and makes the job nondeterministic.

- *Follow-up: How do you handle a hot key (celebrity)?* Detect hot keys (Pig samples; Crunch/Hive need them declared), spread hot-key records across several random reducers and replicate the other side to each; or for aggregations, do a two-stage aggregation (random partial aggregates, then combine).
- *Follow-up: Does join choice affect downstream jobs?* Yes: reduce-side output is partitioned/sorted by join key; map-side output follows the large input's layout.

3. How should a batch job publish results to a serving database? Do not write record-by-record into the production DB (slow, overloads it, breaks all-or-nothing semantics with visible partial results and duplicate task attempts). Instead build immutable database files inside the job, write them to HDFS, and bulk-load them into a read-only store (Voldemort, Terrapin, ElephantDB, HBase bulk load). Voldemort serves old files while copying new ones, then atomically switches, and can switch back.

- *Follow-up: What's the operational benefit?* Human fault tolerance: buggy output is fixed by rolling back code and rerunning, or pointing back at the previous output.

4. MapReduce vs Spark/Flink (dataflow engines)? MapReduce materializes every intermediate result to replicated HDFS, always sorts, has redundant map stages, waits for all tasks of a previous job, and starts a new JVM per task. Dataflow engines treat the whole workflow as a DAG of operators, sort only when needed, keep intermediate data in memory/local disk, pipeline operators, reuse JVMs, and optimize locality. They recover by recomputing lost partitions from lineage (Spark RDD) or checkpoints (Flink).

- *Follow-up: Why does determinism matter for recovery?* If recomputed data differs from what downstream already consumed, you must also kill and rerun downstream operators. Avoid unordered hash iteration, unseeded randomness, clocks, external lookups.
- *Follow-up: When is recomputation worse than materializing?* When the intermediate result is tiny compared with its source, or very CPU-expensive to produce.

5. Why was MapReduce so eager to write to disk and retry at task granularity? Because at Google, batch tasks run at low priority on shared machines and get preempted: an hour-long task has about a 5% chance of being killed, over 10x the hardware failure rate. With 100 tasks of 10 minutes, the chance at least one is killed exceeds 50%. Task-level retry avoids rerunning whole jobs. Open source schedulers (YARN, Mesos, Kubernetes) mostly lacked general priority preemption at the time, so these trade-offs make less sense there.

6. Hadoop vs MPP databases? MPP databases are monolithic, need upfront schema modeling, and are excellent for analytic SQL. Hadoop is like a general-purpose OS: any data format (dump raw data, schema-on-read, data lake, sushi principle) and any processing model (MapReduce, SQL via Hive, Spark, HBase, Impala) on one shared cluster. MPP aborts the whole query on failure and keeps data in memory; MapReduce retries tasks and spills to disk. They are converging as batch frameworks get declarative optimizers and MPP DBs get more programmable.

Self-check

1. **What distinguishes a system of record from a derived data system, and can the same database product be either?** A system of record holds the authoritative, normalized copy written first; derived data is computed from other data and can be rebuilt if lost. Yes: the distinction depends on how the application uses the tool, not on the tool.
2. **Why can the `sort | uniq -c` pipeline handle more distinct URLs than an in-memory hash table approach?** `sort` spills sorted chunks to disk and merges them (external mergesort, sequential I/O), so memory is not a hard limit; a hash table needs every distinct key in RAM.

3. **List the three things that make Unix tools composable and easy to experiment with.** A uniform interface (files as byte sequences, newline-separated text), separation of logic and wiring (stdin/stdout, shell decides the plumbing), and transparency (immutable inputs, inspect any stage, save stage output to files).
4. **In a MapReduce job, what decides the number of map tasks and the number of reduce tasks?** Map tasks: the number of input file blocks. Reduce tasks: configured by the job author. The reducer for a key is chosen by hashing the key.
5. **Scenario: you must join 10 TB of click events with a 200 MB table of product metadata. Which join do you pick and why?** Broadcast hash join: the small table fits in memory, so each mapper over a block of clicks loads it into a hash table and probes it; no shuffle or sort needed.
6. **Why is writing directly from reducers into a production database a bad idea? Give three reasons.** Per-record network calls are orders of magnitude slower; parallel tasks can overwhelm the DB and hurt its serving; and it creates externally visible side effects that break MapReduce's all-or-nothing output guarantee (partial results, retried and speculative task attempts).
7. **What is "human fault tolerance" and how do immutable batch outputs provide it?** The ability to recover from deploying buggy code. Since inputs are unchanged and outputs are fully replaced, you roll back the code and rerun, or switch back to the previous output directory. A read-write DB would keep the bad writes.
8. **Name three drawbacks of MapReduce materializing intermediate state, and how dataflow engines address them.** Waiting for all tasks of the previous job (stragglers), redundant mappers that re-read reducer output, and replication of temporary files in HDFS. Dataflow engines run the whole workflow as one job, fold map work into preceding operators, keep intermediate data in memory/local disk, and start operators as soon as input is ready.
9. **Why is iterative graph processing inefficient in plain MapReduce, and how does Pregel improve it?** Each iteration rereads the whole graph and writes a whole new output even if little changed. Pregel keeps vertex state in memory between rounds and only processes vertices that received new messages, with checkpoints at iteration boundaries for fault tolerance.
10. **Trade-off: when might a single machine beat a distributed graph framework?** When the graph fits in one machine's memory (or even on its disks, using something like GraphChi): distributed graph processing suffers heavy cross-machine messaging because partitioning by arbitrary vertex ID ignores which vertices communicate, so messages can outgrow the graph itself.
11. **What does "bounded input" mean, and why is it the defining property of batch processing?** The input has a known, fixed size (e.g., logs up to a point in time or a DB snapshot), so the job knows when it has read everything and can finish. Stream processing drops this assumption.

Chapter 11 — Stream Processing

Book: Chapter 11, pp. 439–487

Overview

Question this chapter answers: What if the input never ends? How do you move, store, and process data that keeps arriving (clicks, sensor readings, database writes) with low delay, while keeping the good properties of batch processing (derived outputs, rerunnability, fault tolerance)?

Where it fits: Chapter 10's batch jobs read a set of files and produce new files, a form of **derived data** that can be recreated by rerunning the job (search indexes, recommendations, analytics). But batch assumed the input is **bounded**: known, finite size, so the job knows when it has read everything. Sorting, the heart of MapReduce, *needs* this: it cannot emit its first output until it has seen all input, because the very last record might have the smallest key.

Real data is usually **unbounded**: your users produced data yesterday, today, and will tomorrow; unless the business shuts down, the dataset is never "complete". Batch systems fake completeness by chopping data into fixed time slices (a day's data at the end of each day, or an hour's at the end of each hour). With a daily job, a change to the input only appears in the output a day afterwards, and many impatient users will not wait that long. You can shrink the slice (a second's worth every second), or drop slices entirely and **process each event as it happens**. That is **stream processing**.

A "stream" in general means data made available **incrementally over time**. The idea shows up in Unix stdin/stdout, lazy lists in programming languages, filesystem APIs like Java's `FileInputStream`, TCP connections, and audio/video delivery over the internet. This chapter treats **event streams as a way of managing data**: the never-ending, processed-bit-by-bit sibling of Chapter 10's batch data. Plan:

1. How streams are represented, stored, and sent over a network ("Transmitting Event Streams").
2. How streams relate to databases ("Databases and Streams").
3. How to process streams continuously and build applications with them ("Processing Streams").

Why it matters: fraud detection, monitoring, live dashboards, keeping caches/search indexes in sync with a database, and real-time analytics all depend on it. The log-based messaging and change-data-capture ideas here underpin modern data integration (Kafka and friends).

Everyday analogy: Batch processing is reading yesterday's newspaper: a finished, bounded bundle. Stream processing is a live news ticker: items keep scrolling past forever, you react to each as it appears, and there is no "last page".

Key terms

Term	Plain-English meaning
Bounded / unbounded data	Finite, known-size input vs. input that keeps arriving forever
Stream	Data made available bit by bit over time
Event	A small, self-contained, immutable record of something that happened at some time, usually with a timestamp
Producer (publisher, sender)	Creates events
Consumer (subscriber, recipient)	Processes events
Topic / stream	A named group of related events (like a filename groups related records)
Polling	Consumer repeatedly asking "anything new?"
Trigger	Database feature that reacts to a change (e.g., a row insert); limited
Messaging system	Pushes messages containing events from producers to consumers
Publish/subscribe	Many producers can publish to a topic; many consumers can subscribe
Backpressure (flow control)	Blocking the producer when the consumer cannot keep up
UDP multicast	Unreliable one-to-many network sending; used for low-latency feeds
Brokerless messaging	Libraries (ZeroMQ, nanomsg) sending pub/sub directly between nodes
Webhook	Registering a callback URL with a service, which calls it when an event occurs
Message broker (message queue)	A server, essentially a database specialized for message streams, through which producers and consumers communicate
JMS / AMQP	Standards for traditional message brokers
Load balancing (messaging)	Each message goes to one of the consumers, sharing work
Fan-out	Each message goes to all consumers
Acknowledgment	Consumer telling the broker it finished a message, so it can be removed
Redelivery	Broker resending an unacknowledged message, possibly to another consumer
Log	Append-only sequence of records on disk
Partitioned log	A topic split into partitions, each an independent log on possibly different machines
Offset	Monotonically increasing sequence number of a message within a partition
Log-based message broker	Broker storing messages in a durable partitioned log (Kafka, Kinesis Streams, DistributedLog)
Consumer group	Set of consumer nodes sharing a topic's partitions
Head-of-line blocking	One slow message delays all later messages behind it
Consumer offset	The position up to which a consumer has processed a partition
Log segment	Chunk of a log file; old segments are deleted or archived
Circular buffer / ring buffer	Fixed-size buffer that discards the oldest data when full
Replay	Re-reading old messages from an earlier offset

Term	Plain-English meaning
Dual writes	Application writing the same change separately to each system (DB, index, cache)
Race condition	Outcome depends on unlucky timing of concurrent operations
Change data capture (CDC)	Observing every change written to a database and extracting it as a stream
Initial snapshot	Consistent full copy of a database matched to a known log offset
Log compaction	Keeping only the latest value for each key in a log
Tombstone	Special null value marking a key as deleted
Change stream / change feed	A database's first-class API for subscribing to its changes
Event sourcing	Storing all application changes as immutable, application-level events
Domain-driven design (DDD)	Software design community where event sourcing originated
Command	A user request that may still be rejected by validation
Changelog	Log of all changes; describes the evolution of state over time
Ledger	Accountants' append-only log of transactions
Compensating transaction	A new entry that corrects an earlier mistaken one instead of erasing it
CQRS	Command query responsibility segregation: separating the write form of data from its read forms
Excision / shunning	Datomic's / Fossil's features for truly deleting data from immutable history
Operator / job	A piece of code that processes streams
Complex event processing (CEP)	Searching streams for patterns of events using stored queries
Stream analytics	Aggregations and statistics over streams (rates, averages)
Window	The time interval over which an aggregation is computed
Probabilistic algorithm	Gives approximate answers with far less memory (Bloom filter, HyperLogLog)
Materialized view	A derived, query-optimized view kept up to date as underlying data changes
Percolator	Elasticsearch feature for matching documents against stored queries
Distributed RPC	Storm feature interleaving user queries with stream events
Event time	When the event actually happened (timestamp in the event)
Processing time	When the stream processor handles the event (processor's clock)
Straggler event	Event arriving after its window was declared complete
Tumbling / hopping / sliding / session window	Types of time windows (defined below)
Stream-stream join (window join)	Joining two event streams on related events within a time window
Stream-table join (enrichment)	Adding database info to each event using a local, CDC-updated copy
Table-table join	Joining two changelog streams to maintain a materialized join view
Slowly changing dimension (SCD)	Data-warehouse problem of joining with a value that changes over time

Term	Plain-English meaning
Exactly-once (effectively-once) semantics	Output looks as though each input was processed exactly once, despite retries
Microbatching	Splitting a stream into tiny batches (~1 second) processed like batch jobs
Checkpointing	Periodically saving operator state to durable storage
Idempotent	Doing it many times has the same effect as doing it once
Fencing	Preventing a node wrongly believed dead from interfering

Transmitting Event Streams

In batch processing, a job's inputs and outputs are **files**. What is the streaming equivalent?

- **Events instead of records.** A file is first parsed into records. In streaming, a record is usually called an **event**: a small, self-contained, **immutable** object describing something that occurred at a particular moment. It usually carries a **timestamp** from a time-of-day clock (Chapter 8).
- **Examples:** a user action (viewing a page, making a purchase), or a machine source (periodic temperature sensor reading, CPU utilization metric). Each line of Chapter 10's web server log is an event.
- **Encoding:** text, JSON, or a binary form (Chapter 4). Encoding lets you store the event (append to a file, insert into a relational table, write to a document DB) or send it over the network to another node.
- **Producers and consumers.** In batch, a file is written once and may be read by several jobs. In streaming, a **producer** creates each event once (**publisher**, **sender**) and may be processed by several **consumers** (**subscribers**, **recipients**). Where a filesystem groups related records by **filename**, a streaming system groups related events into a **topic** or **stream**.

Why not just use a database or file? In principle that works: producers write every event to the datastore; each consumer periodically **polls** for events newer than its last run. That is effectively what a daily batch job does. But for continual, low-delay processing, polling gets expensive if the store is not built for it: the more often you poll, the smaller the share of requests that find anything new, so overhead climbs. It is better if consumers are **notified** when new events appear.

Databases have not traditionally been good at notifications. Relational **triggers** can respond to a change such as an inserted row, but they can do very little and have been something of an afterthought in database design. So specialized tools were built for delivering event notifications.

Messaging Systems

What: A **messaging system** tells consumers about new events: the producer emits a message carrying the event, and the system **pushes** it out to consumers. (Chapter 4's "Message-Passing Dataflow" touched on this.)

The simplest possible messaging system would be a direct link between producer and consumer, such as a Unix pipe or a TCP connection. But pipes and TCP connect exactly **one sender to one receiver**; messaging systems let **many producers** send to the same topic and **many consumers** receive from it. Within this **publish/subscribe** model, systems vary widely and no approach is right for everything. Two questions help tell them apart:

Question 1: What if producers send faster than consumers can process? Three options:

Option	What happens	Example / caveat
Drop messages	Excess is discarded	Fine only if loss is tolerable
Buffer in a queue	Messages wait	Must ask: as the queue grows, does the system crash when it no longer fits in memory, or spill to disk? And how does disk access affect performance?
Backpressure (flow control)	Producer is blocked from sending more	Unix pipes and TCP do this: a small fixed-size buffer; when full, the sender blocks until the receiver drains it

Question 2: What if nodes crash or go offline temporarily; are messages lost? As with databases, durability needs some mix of writing to disk and/or replication, which costs something. If losing the odd message is tolerable, the same hardware can usually deliver more throughput with less latency.

Whether loss is OK depends on the application:

- **Sensor readings and metrics** sent periodically: an occasional missing point may not matter, since a fresh value comes shortly. **Beware:** if many messages are dropped, it may not be obvious that the metrics are wrong.
- **Counting events:** reliable delivery matters more, since every lost message makes the counters wrong.

Recall that batch systems (Chapter 10) give a strong guarantee: failed tasks are retried and their partial output is discarded, so output equals a failure-free run. Later the chapter looks at achieving similar guarantees for streams.

Direct messaging from producers to consumers

Some messaging systems skip intermediary nodes and talk directly over the network:

- **UDP multicast** is popular in the **financial industry** for low-latency streams such as **stock market feeds**. UDP itself does not guarantee delivery, but protocols layered on top at the application level can recover missing packets, provided the producer keeps a record of what it sent and retransmits on request.
- **Brokerless messaging libraries** such as **ZeroMQ** and **nanomsg** do something similar, implementing pub/sub over TCP or IP multicast.
- **StatsD** and **Brubeck** use unreliable **UDP** to collect metrics from all machines on a network for monitoring. In the StatsD protocol, **counter** metrics are correct only if every message arrives, so UDP makes them approximate at best (see Chapter 8, "TCP Versus UDP").
- If a consumer exposes a network service, producers can push via a direct **HTTP or RPC** request (Chapter 4). This is the idea behind **webhooks**: one service registers a callback URL with another, which calls that URL whenever an event occurs.

Limits: these work well in their niches, but application code must generally cope with **message loss**. They tolerate few faults: even when protocols detect and retransmit lost packets, they usually assume producers and consumers are **always online**. An offline consumer may miss messages sent while it was unreachable. Some protocols let the producer retry failed deliveries, but that fails if the **producer crashes** and loses its buffer of messages to retry.

Message brokers

What: A widely used alternative is a **message broker** (also called a **message queue**), essentially a **database optimized for message streams**. It runs as a server that both producers and consumers connect to as clients: producers hand messages to it, consumers fetch messages from it.

Why it helps:

- **Centralizing data** in the broker makes it easy to cope with clients that connect, disconnect, or crash at will. **Durability becomes the broker's job.**

- Some brokers keep messages only **in memory**; others can be configured to persist them **to disk**, so a broker crash loses nothing.
- With slow consumers, brokers generally allow **unbounded queueing** (rather than dropping or backpressure), though this can depend on configuration.
- Consequence: consumers are **asynchronous**. A producer usually waits only for the broker to confirm it has **buffered** the message, not for consumers to process it. Consumers get the message at some unspecified later moment: usually well under a second, but possibly much later when a backlog has built up.

Message brokers compared to databases

Some brokers can even take part in **two-phase commit** using **XA** or **JTA** (Chapter 9), which makes them resemble databases. But there are important practical differences:

Aspect	Database	Traditional message broker
Retention	Keeps data until explicitly deleted	Most **delete a message once it is successfully delivered**; not suitable for long-term storage
Working set	Can be large	Assumes a **small working set** (short queues). If lots of messages must be buffered for slow consumers (maybe spilling to disk), each message takes longer and **overall throughput may degrade**
Selecting data	**Secondary indexes** and various search methods	**Subscribing to a subset of topics** matching a pattern. Different mechanisms, same purpose: letting a client pick the part of the data it cares about
Change notification	Query results are a **point-in-time snapshot**; if someone later changes the data, you do not find out unless you re-query or poll	**No arbitrary queries**, but clients **are notified** when data changes (new messages arrive)

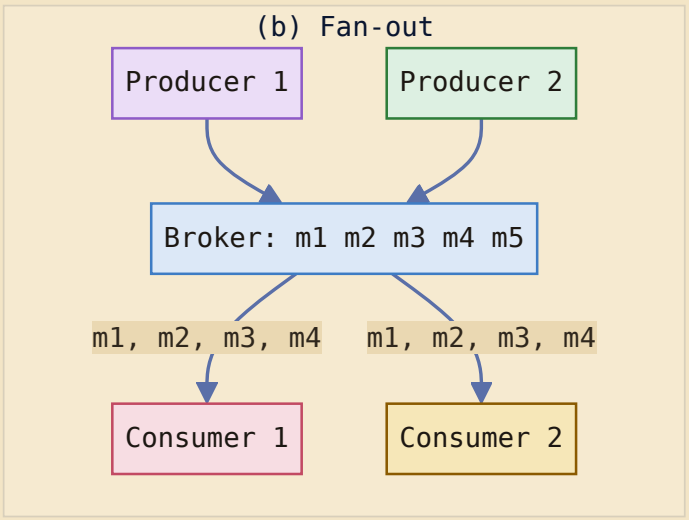
This classic picture of a broker is written down in the **JMS** and **AMQP** standards, and products implementing it include **RabbitMQ**, **ActiveMQ**, **HornetQ**, **Qpid**, **TIBCO Enterprise Message Service**, **IBM MQ**, **Azure Service Bus**, and **Google Cloud Pub/Sub**.

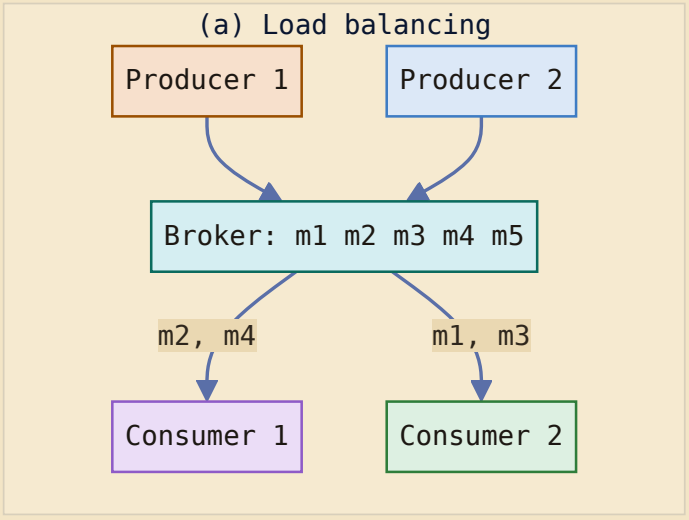
Multiple consumers

When several consumers read the same topic, there are two main patterns:

- **Load balancing:** each message goes to **one** of the consumers, so they share the work. The broker may assign messages arbitrarily. Useful when messages are **expensive to process** and you want to add consumers to parallelize. (In **AMQP**: several clients consuming from the same queue. In **JMS**: a **shared subscription**.)
- **Fan-out:** each message goes to **all** consumers. Independent consumers can each "tune in" to the same broadcast with no interference between them; this mirrors several batch jobs reading the same input file. (**JMS**: **topic subscriptions**. **AMQP**: **exchange bindings**.)

Figure 11-1 (redrawn): producers 1 and 2 publish m1 to m5 into the broker. (a) Load balancing: consumer 1 processes m2 and m4, consumer 2 processes m1 and m3, so each message goes to one consumer. (b) Fan-out: consumer 1 and consumer 2 each process m1, m2, m3, m4 (m5 has not been delivered yet when the figure ends).





The two can be **combined**: e.g., two separate consumer groups each subscribe to a topic, so each group as a whole gets every message, but inside each group only one node gets any given message.

Acknowledgments and redelivery

Problem: consumers can crash anytime, so a broker might deliver a message that the consumer never processes, or only partly processes.

Mechanism:

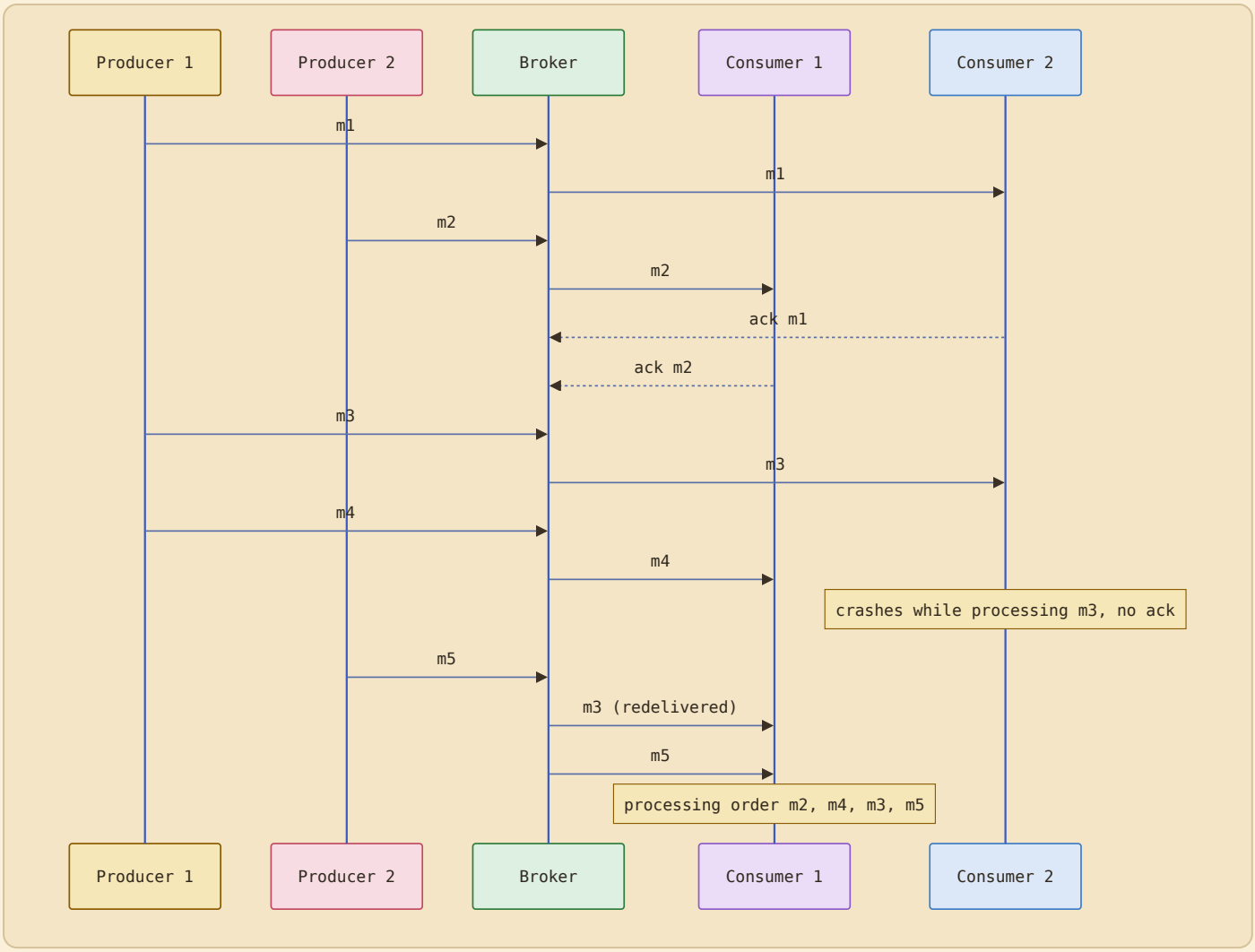
1. Broker delivers a message to a consumer.
2. The consumer must **explicitly acknowledge** when it has finished, so the broker can remove the message from the queue.
3. If the connection closes or times out with no acknowledgment, the broker assumes the message was **not processed** and **redelivers** it to another consumer.

Caveat: the message might actually have been fully processed with only the ack lost in the network. Handling that needs an **atomic commit protocol** (Chapter 9, "Distributed Transactions in Practice").

Effect on ordering (Figure 11-2): combined with load balancing, redelivery **reorders** messages. Consumers generally process messages in the order producers sent them. But suppose consumer 2 crashes while handling **m3**, at the same moment consumer 1 is handling **m4**. The unacknowledged m3 is

later redelivered to consumer 1, so consumer 1 processes **m4, m3, m5**. Now m3 and m4 are processed in a different order than producer 1 sent them.

Figure 11-2 (redrawn): producer 1 sends m1, m3, m4 and producer 2 sends m2, m5. Consumer 2 handles m1, consumer 1 handles m2, then consumer 2 dies midway through m3 while consumer 1 is busy with m4. The broker later redelivers m3 to consumer 1, which therefore processes m4, m3, m5.



Even though brokers otherwise try to preserve order (both **JMS and AMQP require it**), load balancing plus redelivery **inevitably reorders** messages. To avoid it, use a **separate queue per consumer** (no load balancing). Reordering does not matter if messages are fully independent, but it can matter a lot if there are **causal dependencies** between them (seen later in the chapter).

Partitioned Logs

Mindset problem: sending a packet or calling a network service is normally **transient**; it leaves no permanent trace (you *could* record it via packet capture and logging, but normally don't). Even brokers that persist messages on disk remove them shortly after delivering them, since their whole design assumes messages are fleeting. Databases and filesystems are the opposite: whatever you write is expected to stay until someone explicitly deletes it.

This difference hugely affects how **derived data** is built:

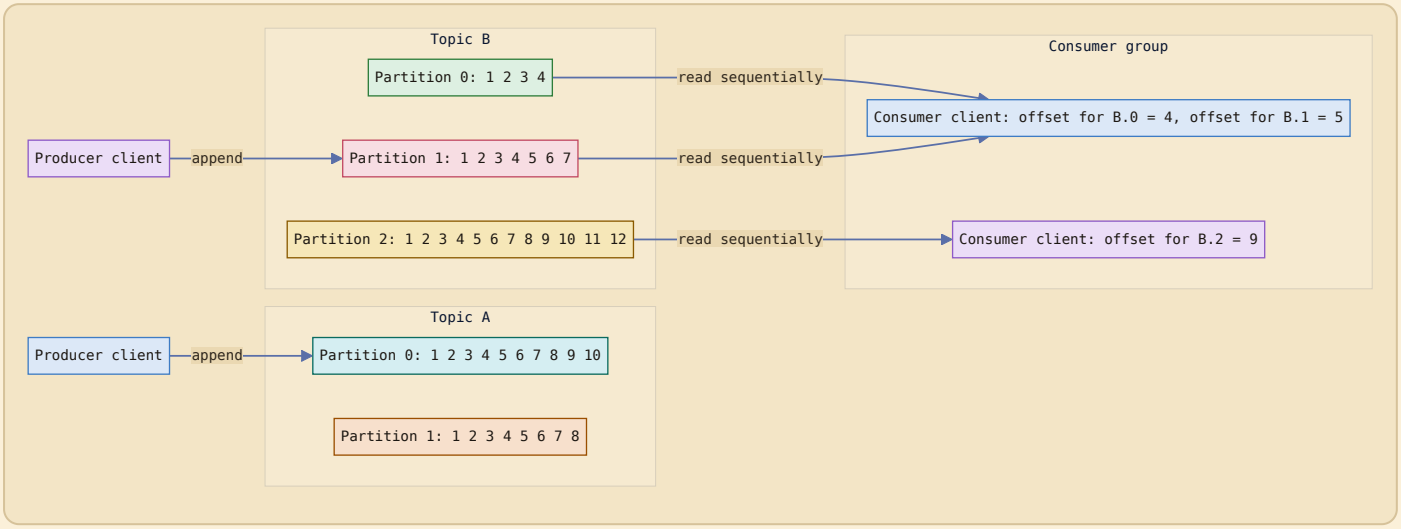
- Batch jobs can be rerun as often as you like on read-only input without harming it.
- With **AMQP/JMS**-style messaging, receiving is **destructive** if the ack deletes the message from the broker, so rerunning a consumer will not give the same result.
- A **new consumer** added to such a system usually only gets messages sent **after it registered**; earlier messages are gone forever. With files and databases, a new client can read data written arbitrarily long ago (unless the application overwrote or deleted it).

Idea: combine the **durable storage** of databases with the **low-latency notifications** of messaging. That is the **log-based message broker**.

Using logs for message storage

- A **log** is simply records on disk that are only ever **appended** to (seen in Chapter 3 for log-structured storage and write-ahead logs, and Chapter 5 for replication).
- Used as a broker: to send, a **producer appends** the message at the end of the log; to receive, a **consumer** reads through the log **in order**. Having caught up with the end, the consumer waits for a notification that something new was appended. The Unix command `tail -f`, which follows a file as data is added to it, behaves much the same way.
- **Scaling:** one disk has limited throughput, so the log is **partitioned** (Chapter 6). Partitions can live on different machines, each an independent log readable and writable separately. A **topic** is then a **group of partitions** carrying messages of the same type.
- **Offsets:** within each partition, the broker gives every message a **monotonically increasing sequence number**, the **offset**. Such numbering works because a partition only grows at its end, which puts its messages in a **total order**. Between different partitions, however, **no order is guaranteed**.

Figure 11-3 (redrawn): Topic A has partitions 0 (messages 1 to 10) and 1 (1 to 8); topic B has partitions 0 (1 to 4), 1 (1 to 7) and 2 (1 to 12). The numbers in the boxes are offsets. Producer clients append new messages to the end of a partition (one to topic A partition 0, one to topic B partition 1). A consumer group of two clients reads topic B sequentially: one client owns B.0 and B.1 (recorded offsets 4 and 5), the other owns B.2 (offset 9).



Systems that work like this: **Apache Kafka**, **Amazon Kinesis Streams**, and **Twitter's DistributedLog**. **Google Cloud Pub/Sub** has a similar architecture but presents a **JMS-style API** to clients instead of exposing a log. Although every message goes to disk, these brokers handle **millions of messages per second** thanks to partitioning over many machines, and get fault tolerance by **replicating** messages.

Logs compared to traditional messaging

- **Fan-out is trivial:** many consumers can read the log independently; reading does not delete anything.
- **Load balancing** works by assigning **whole partitions** (not individual messages) to nodes in a **consumer group**. Each client consumes all messages in its assigned partitions, typically **sequentially and single-threaded**.

Downsides of this coarse-grained load balancing:

1. The number of consumer nodes sharing a topic is **at most the number of partitions**, since one partition's messages go to one node.

2. A single **slow message holds up** all later messages in that partition (**head-of-line blocking**, Chapter 1).

> **Footnote: sharing one partition.** You could have two consumers both read a whole partition, one handling only even-numbered offsets and the other odd. Or spread processing over a thread pool, though that complicates offset management. Generally, single-threaded processing per partition is preferable; add parallelism by adding partitions.

Situation	Better choice
Messages **expensive** to process, want **per-message parallelism**, ordering **not important**	**JMS/AMQP**-style broker
High throughput, each message **fast** to process, ordering **important**	**Log-based** broker

Consumer offsets

- Reading a partition sequentially makes progress easy to track: every message with an offset **below** the consumer's current offset is done; everything **above** is unseen.
- So the broker need not track acks per message, only **periodically record consumer offsets**. Less bookkeeping plus chances for **batching and pipelining** boost throughput.
- This offset is very much like the **log sequence number** in single-leader replication (Chapter 5, "Setting Up New Followers"): it lets a disconnected follower reconnect and resume without skipping writes. Same principle here: **the broker acts like a leader database, the consumer like a follower**.
- **Consumer failure:** another node in the group takes over the failed consumer's partitions and starts from the **last recorded offset**. Any messages the failed node handled after its last recorded offset will be **processed a second time**. (Handled later in the chapter.)

Disk space usage

- Only appending would eventually fill the disk. So the log is split into **segments**, and old segments are periodically **deleted or moved to archive storage**. (A smarter method, log compaction, comes later.)
- So a **slow consumer** that falls so far behind that its offset points at a **deleted segment** will **miss messages**. The log is effectively a **bounded-size buffer that discards old messages when full**, a **circular buffer** or **ring buffer**, but on disk, so it can be **very large**.

Back-of-the-envelope calculation (book's numbers):

1. When the book was written, a typical big hard drive stored **6 TB** with a sequential write speed of **150 MB/s**.
2. Writing at full speed, filling it takes about **11 hours** (6,000,000 MB ÷ 150 MB/s ≈ 40,000 s ≈ 11 h).
3. So one disk buffers **about 11 hours** of messages before overwriting old ones. The ratio stays the same with many disks and machines.
4. Real deployments rarely use full disk write bandwidth, so a log can usually retain **several days or even weeks** of messages.

Throughput stays **roughly constant regardless of retention**, because each message hits the disk no matter what. Contrast with systems that keep messages in memory and spill to disk only when queues grow: those are fast with short queues and **much slower** once they start writing to disk, so their throughput depends on how much history is retained.

When consumers cannot keep up with producers

- In the drop / buffer / backpressure taxonomy, a log counts as **buffering**, using a **big buffer of fixed size** bounded by available disk space.

- A consumer that falls behind further than the retained data **cannot read** them anymore, which means the broker has in effect **dropped messages older than the buffer can hold**.
- You can **monitor how far a consumer lags** behind the log head and **alert** if it is far behind. With such a big buffer, a human has time to fix the slow consumer before it misses anything.
- Only the lagging consumer is affected; **others are not disrupted**. Big operational win: you can read a **production log** while developing, testing, or debugging, with little risk of harming production services.
- A consumer that is shut down or crashes **stops using resources**; only its offset remains. Contrast: with traditional brokers you must remember to **delete queues** whose consumers are gone, or they keep piling up messages and stealing memory from active consumers.

Replaying old messages

- With AMQP/JMS brokers, processing and acking is **destructive** (messages get deleted). In a log-based broker, consuming is like **reading a file**: **read-only**, the log is unchanged.
- The only side effect (apart from the consumer's output) is that the **consumer offset moves forward**, and the consumer itself controls that offset. E.g., start a **copy of a consumer with yesterday's offsets**, write output elsewhere, and **reprocess the last day's messages**. Repeat as often as you like with different code.
- This makes log-based messaging like **batch processing**: derived data is cleanly separated from input via a repeatable transformation. That encourages experimentation and makes recovering from mistakes and bugs easier, which is why it suits **integrating dataflows across an organization**.

Own illustrative example (Kafka-style consumer pseudocode):

```
consumer = Consumer(group="search-indexer", topic="page-edits")
consumer.seek(partition=0, offset=41_200)   # rewind to replay from an older position
for msg in consumer.poll_forever():         # sequential read; nothing is deleted
    index.update(msg.key, msg.value)        # derived output goes somewhere else
    consumer.commit(msg.offset + 1)         # record progress periodically, not per-message ack
```

- `seek` works because the log is retained: you can replay.
- `commit` records the offset; after a crash, processing resumes from here (anything processed after the last commit is redone).

Databases and Streams

Log-based brokers took ideas **from databases** and applied them to messaging. The reverse also works: apply ideas from messaging and streams **to databases**.

- An event records something that happened: a user action (typing a search query), a sensor reading, **or a database write**. That a write occurred is itself an event you can capture, store, and process. So the link between databases and streams is **fundamental**, not just about storing logs on disk.
- A **replication log** (Chapter 5) *is* a stream of write events that the leader emits while executing transactions; followers apply it to their copy and end up with the same data.
- The **state machine replication** principle (Chapter 9, "Total Order Broadcast"): if each event is a database write and all replicas apply identical events in identical order, every replica ends in the same state (assuming processing is **deterministic**). Another case of event streams.

Keeping Systems in Sync

- No single system satisfies every storage, query, and processing need. Nontrivial applications combine several: an **OLTP database** for user requests, a **cache** for common requests, a **full-text**

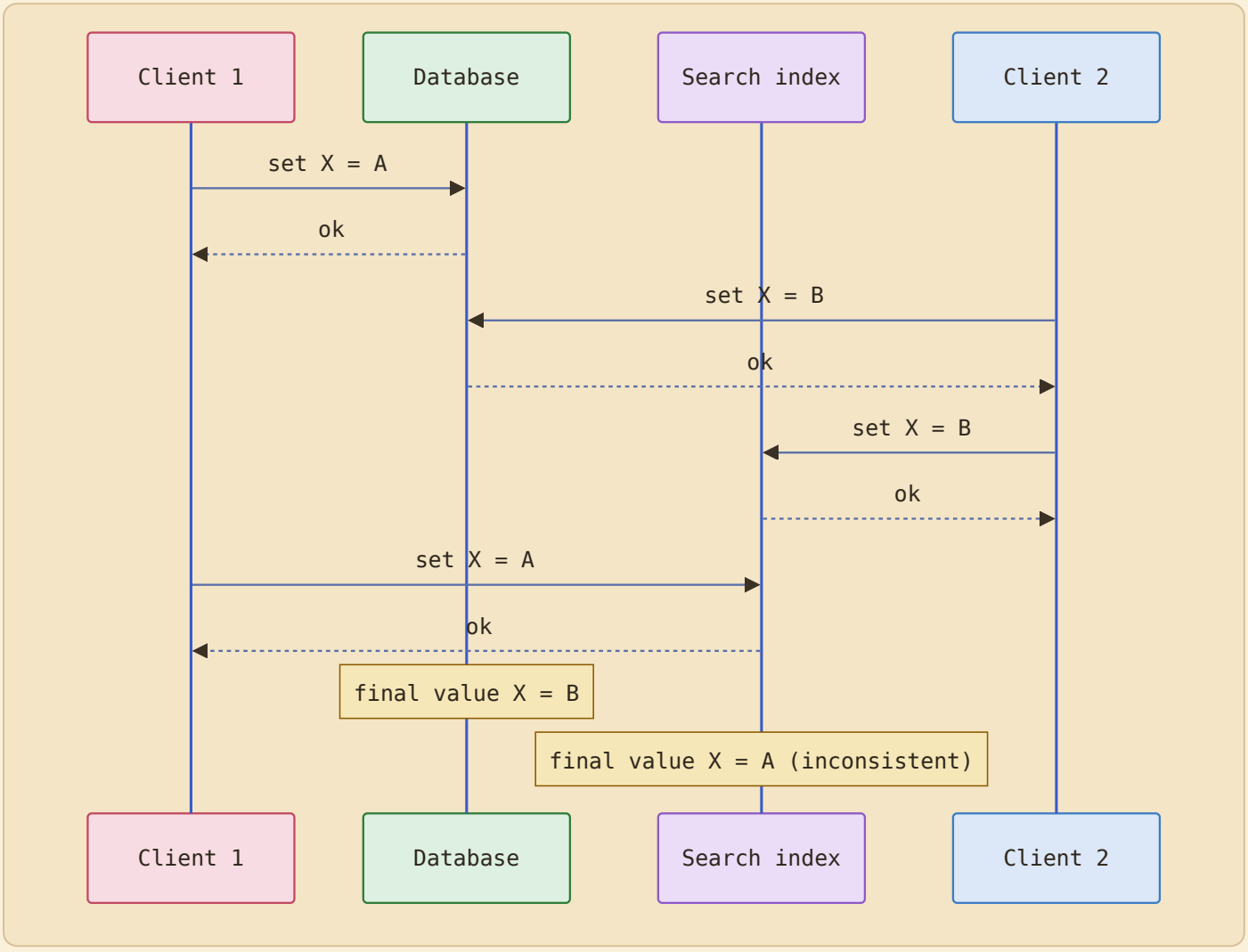
index for search, a **data warehouse** for analytics. Each keeps its own copy of the data in its own optimized representation.

- They must be **kept in sync**: an item updated in the database must also be updated in the cache, search indexes, and warehouse.
- For warehouses this is usually **ETL**: take a full copy of a database, transform it, bulk-load it into the warehouse, i.e., a **batch process**. Search indexes and recommendation systems can likewise be built in batch (Chapter 10).
- If periodic full dumps are too slow, a sometimes-used alternative is **dual writes**: the application code itself writes to each system on every change (e.g., write the DB, then update the search index, then invalidate cache entries, or even do these concurrently).

Problem 1 with dual writes: race condition (Figure 11-4).

1. Two clients concurrently update item **X**: client 1 sets it to **A**, client 2 to **B**.
2. Each client writes the database first, then the search index.
3. Unlucky interleaving: the **database** receives client 1's write (A) then client 2's (B), so it ends at **B**.
4. The **search index** receives client 2's write first, then client 1's, so it ends at **A**.
5. The two systems are now **permanently inconsistent**, even though no error occurred.

Figure 11-4 (redrawn): each write is acknowledged with "ok". The database gets A then B (final X = B), but the search index gets B then A (final X = A).



Without an extra concurrency-detection mechanism, such as **version vectors** (Chapter 5, "Detecting Concurrent Writes"), you would not even notice concurrent writes happened; one value silently overwrites the other.

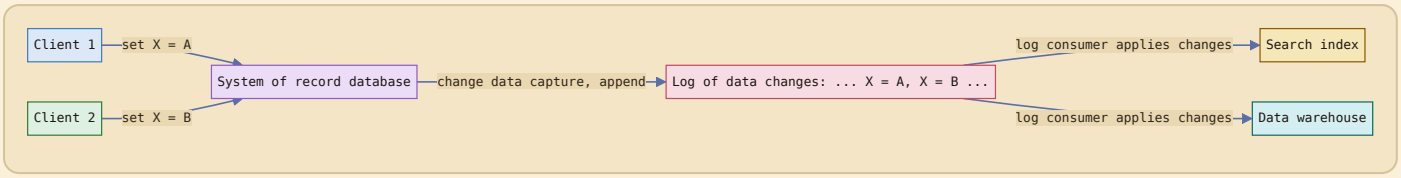
Problem 2: partial failure. One write might fail while the other succeeds. This is about **fault tolerance**, not concurrency, yet it also leaves the systems inconsistent. Guaranteeing all-or-nothing across both is the **atomic commit** problem, and solving it is **costly** (Chapter 9, 2PC).

Root cause: with a single-leader replicated database, the leader fixes the write order, so state machine replication works among its replicas. In Figure 11-4 there is **no single leader**: the database and the search index might each have their own leader, but neither is a follower of the other, so conflicts arise (as with multi-leader replication, Chapter 5). It would be better to have **one leader** (say the database) and make the search index its **follower**. Is that possible in practice? Enter CDC.

Change Data Capture

- Most databases' replication logs have long been treated as an **internal implementation detail**, not a public API. Clients are meant to use the data model and query language, not parse replication logs.
- For **decades**, many databases had **no documented way** to get the log of changes written to them. So replicating all changes to another storage technology (search index, cache, warehouse) was hard.
- **Change data capture (CDC)**: **watching every change written to a database** and pulling it out in a form that **other systems can replicate**. It is most useful when changes are exposed **as a stream the moment they are written**.
- Example: capture changes in a database and continually apply them to a search index. If changes are applied **in the same order**, the search index should match the database. The search index, like any other derived system, is simply **one more consumer of the change stream**.

Figure 11-5 (redrawn): client 1 sets X = A and client 2 sets X = B in the system-of-record database. Change data capture appends the changes, in the order the database applied them, to a log of data changes (... X = A, X = B ...). A log consumer applies them in that same order to the derived data systems: a search index and a data warehouse.



Implementing change data capture

- Log consumers are **derived data systems** (Part III intro): the search index and warehouse data are just other **views** of the **system of record**. CDC makes sure every change to the system of record is reflected in the derived systems, so they hold accurate copies.
- **CDC makes one database the leader** (the one changes are captured from) and **turns the others into followers**.
- A **log-based message broker** is a good transport for change events because it **preserves message order** (avoiding Figure 11-2's reordering).

Implementation options:

Approach	How	Trade-off
Database triggers (Chapter 5, "Trigger-based replication")	Register triggers observing all changes to data tables and add entries to a **changelog table**	Tend to be **fragile** with **significant performance overhead**
Parsing the replication log	Read the DB's own log	**More robust**, but has challenges such as **handling schema changes**

Named systems:

- **LinkedIn's Databus**, **Facebook's Wormhole**, **Yahoo!'s Sherpa** use this idea at large scale.
- **Bottled Water**: CDC for **PostgreSQL**, built on an API for decoding PostgreSQL's **write-ahead log**.

- **Maxwell** and **Debezium**: similar for **MySQL** by parsing the **binlog**.
- **Mongoriver**: reads the **MongoDB oplog**.
- **GoldenGate**: similar facilities for **Oracle**.

CDC is usually **asynchronous** like brokers: the system of record does **not wait** for consumers to apply a change before committing. Upside: adding a **slow consumer** does not hurt the system of record much. Downside: all the **replication lag** problems apply (Chapter 5).

Initial snapshot

- With the log of **all** changes ever made, you could rebuild the full database state by replaying it. But keeping every change forever usually needs **too much disk**, and replaying takes **too long**, so the log gets **truncated**.
- Building a new full-text index, for instance, needs a **full copy** of the database; recent changes alone would miss items not updated recently. So without the complete history, you start from a **consistent snapshot** (Chapter 5, "Setting Up New Followers").
- The snapshot must correspond to a **known position/offset in the change log**, so you know where to start applying changes after loading the snapshot. Some CDC tools have this built in; others leave it as a **manual** step.

Log compaction

If you can keep only limited history, you need the snapshot process every time you add a new derived system. **Log compaction** is a good alternative.

- From Chapter 3 ("Hash Indexes", Figure 3-2): every so often the storage engine looks for log records sharing a **key**, discards the duplicates, and retains **just the newest update for each key**. It runs in the **background**.
- A **tombstone** (a special **null** value) marks a key as **deleted**, and causes its removal during compaction. Otherwise, a key that is never overwritten or deleted stays in the log **forever**.
- Hence a compacted log's size is governed by what the database **currently contains**, **not by how many writes** have ever happened. Frequently overwritten keys eventually have old values **garbage-collected**, leaving just the latest.
- Same idea for **log-based brokers and CDC**: if every change has a **primary key** and each update **replaces** the previous value for that key, keeping just the latest write per key is enough.
- To rebuild a derived system (e.g., a search index), start a **new consumer at offset 0** of the compacted topic and scan all messages. The log is guaranteed to have the **latest value for every key** (maybe some older ones too), i.e., a **full copy** of the database, **without taking another snapshot** of the source.
- **Apache Kafka supports log compaction**. It lets the broker serve as **durable storage**, not just transient messaging.

Own illustrative example of compaction on a CDC topic:

```
before compaction (offset: key=value)      after compaction
0: user:7 = {name: "Ana"}                  3: user:9 = {name: "Raj"}
1: user:9 = {name: "Raj"}                  4: user:7 = {name: "Ana M."}
2: user:5 = {name: "Li"}                   5: user:5 = null (tombstone, removed later)
3: user:9 = {name: "Raj"}
4: user:7 = {name: "Ana M."}
5: user:5 = null
```

- Older values for `user:7` and `user:9` are dropped; only the newest per key survives.
- The tombstone for `user:5` tells compaction to eventually remove that key entirely.

API support for change streams

Databases increasingly offer change streams as a **first-class interface** instead of retrofitted, reverse-engineered CDC:

- **RethinkDB**: queries can **subscribe to notifications** when their results change.
- **Firebase** and **CouchDB**: data synchronization based on a **change feed** also exposed to applications.
- **Meteor**: subscribes to data changes through the **MongoDB oplog** and refreshes the user interface accordingly.
- **VoltDB**: transactions can continuously **export data as a stream**. In the relational model, an output stream is a **table that transactions can insert tuples into but that cannot be queried**. The stream is the log of tuples committed transactions wrote to that special table, **in commit order**. External consumers read it asynchronously to update derived systems.
- **Kafka Connect**: an effort to integrate CDC tools for many database systems with **Kafka**. Once change events are in Kafka, they can update derived systems like search indexes and feed stream processors.

Event Sourcing

Event sourcing is a technique from the **domain-driven design (DDD)** community with parallels to CDC. As with CDC, **every change to application state is kept as an event in a log**. The big difference is the **level of abstraction**:

	Change data capture	**Event sourcing**
How the app uses the DB	**Mutably**: updates and deletes at will	App logic is **explicitly built on immutable events** written to an **event log**
Where the log comes from	Extracted **at a low level** (e.g., parsing the replication log), guaranteeing the extracted order matches the real write order (avoiding Figure 11-4's race)	The **event store is append-only**; updates/deletes are **discouraged or prohibited**
App awareness	App **need not know** CDC is happening	App is designed around events
What events mean	**Low-level state changes**	Things that happened **at the application level**

Why it is powerful for data modeling: recording the **user's actions** as immutable events is more meaningful than recording their **effect** on a mutable database. It makes applications easier to **evolve**, aids **debugging** (you can later work out why something happened), and protects against **application bugs**.

The book's example: the event "**student cancelled their course enrollment**" states the intent of one action neutrally. Describing it instead by its side effects (a row removed from the **enrollments** table plus a cancellation reason inserted into the **student feedback** table) bakes in lots of assumptions about how the data will later be used. If a new feature is later added, say **offering the freed place to whoever is next on the waiting list**, event sourcing lets you simply chain that new side effect off the existing event.

Related ideas: event sourcing resembles the **chronicle data model**, and an event log resembles the **fact table** of a star schema (Chapter 3). Specialized databases like **Event Store** support event-sourced apps, but the technique is tool-agnostic: you can build this style of application on a **conventional database** or a **log-based broker** too.

Own illustrative example of event-sourced events (JSON):

```
{"type": "SeatHoldRequested", "show": "S12", "seat": "C7", "user": "u42", "ts": "2021-03-
03T19:00:05Z"}
{"type": "SeatHoldConfirmed", "show": "S12", "seat": "C7", "user": "u42", "ts": "2021-03-
03T19:00:06Z"}
{"type": "SeatReleased",      "show": "S12", "seat": "C7", "user": "u42", "ts": "2021-03-
04T08:12:40Z"}
```

- Each line is an immutable fact about what a user did, not a row update.
- Releasing the seat does not delete the earlier events; it is a new event.

Deriving current state from the event log

- An event log alone is not very useful to users, who want **current state**, not history. On a **shopping website**, users expect to see their cart's **current contents**, not an append-only list of every change they ever made to it.
- So event-sourced apps **transform the event log** (how data is **written**) into application state suitable for display (how data is **read**). The transformation can use arbitrary logic, but should be **deterministic**, so rerunning it gives the same state.
- Replaying the log reconstructs current state, as with CDC, but **log compaction differs**:

	CDC	Event sourcing
Event content	Update event usually holds the **entire new version** of the record	Event expresses **user intent**, not the mechanics of the resulting state update
Can later events override earlier ones?	Yes: current value for a key is fully determined by its **latest** event	Usually **no**: later events do not replace earlier ones
Log compaction	Can discard earlier events for the same key	**Not possible in the same way**; you need the **full history**

- Event-sourced apps usually store **snapshots** of the derived current state so they need not reprocess the whole log every time. But this is **only a performance optimization** for reads and crash recovery; the goal remains that **every raw event is kept forever** and the **entire log can be reprocessed** on demand. (Questioned in "Limitations of immutability".)

Commands and events

Event sourcing carefully separates **commands** and **events**:

1. A user request first arrives as a **command**. It **may still fail**, e.g., if an integrity condition is violated.
2. The application must **validate** that it can execute the command.
3. Once validation passes and the command is accepted, it turns into an **event**: **durable** and **immutable** from then on.

Example: a user tries to **register a username**, or **book a seat on a plane or in a theater** (the same example as Chapter 9, "Fault-Tolerant Consensus"). The app checks the username/seat is not taken; if OK, it emits an event such as "username registered by user ID X" or "seat reserved for customer Y".

- Once generated, an event is a **fact**. If the customer later changes or cancels, it remains true that they **once held** that reservation; the change/cancellation is a **separate, later event**.
- A consumer of the event stream **may not reject an event**: when the event reaches it, the event is already fixed in the log and may have been read by other consumers. So **validation of a command must happen synchronously, before it becomes an event**, e.g., with a **serializable transaction** that checks the command and emits the event as one atomic step.
- **Alternative:** split the seat reservation into **two events**: a **tentative reservation**, then a separate **confirmation** once validated (Chapter 9, "Implementing linearizable storage using total order broadcast"). This allows validation to happen **asynchronously**.

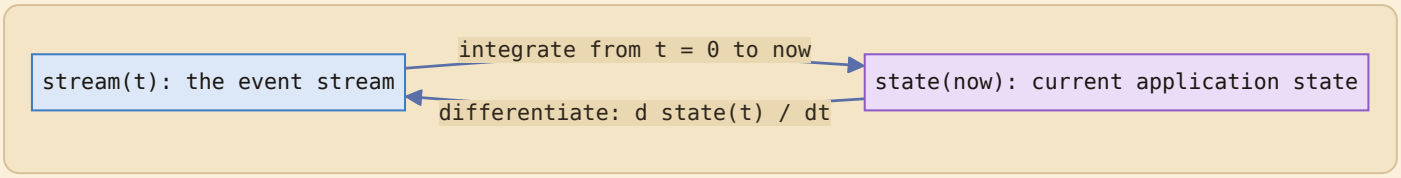
State, Streams, and Immutability

Batch processing benefits from **immutable input files** (run experiments without fear of damage). The same **immutability** principle is what makes event sourcing and CDC so powerful.

- Databases usually store the **current state**, optimized for reads and convenient for queries. State **changes**, so databases support updates and deletes as well as inserts. How does that square with immutability?
- State that changes is always **produced by the events that modified it** over time. Book examples: the list of **available seats** results from the **reservations** processed; the **account balance** results from **credits and debits**; the **response-time graph** of a web server aggregates the individual response times of all requests.
- However state changes, a **sequence of events** caused it. Even when things are done and undone, those events still **happened**. Key idea: **mutable state and an immutable, append-only event log do not conflict; each is one side of the same coin**. The **changelog** (the log of every change) shows how the state evolved over time.

The calculus view (Figure 11-6): for the mathematically minded, **integrating the event stream over time yields the application state**, and **differentiating the state with respect to time yields the change stream**. The analogy has limits (e.g., a second derivative of state does not seem meaningful), but it is a useful starting point.

Figure 11-6 (redrawn): the book shows two formulas. state(now) = integral from t = 0 to now of stream(t) dt, and stream(t) = d state(t) / dt.



- Storing the changelog **durably** simply makes state **reproducible**. If you treat the **event log as the system of record** and all mutable state as **derived** from it, reasoning about dataflow gets easier.

> **Pat Helland's view (paraphrased).** A transaction log records every change to the database, and fast appends are the only way to change it. Seen that way, the database's contents are just a **cache of the latest values** from the log. **The log is the truth**; the database merely caches part of it, specifically the most recent value of every record and index entry.

- **Log compaction** narrows the gap between a log and database state: it retains the newest version of every record and throws away versions that were overwritten.

Advantages of immutable events

- **Old idea: accounting.** Accountants have used immutability for **centuries**. Each transaction goes into an **append-only ledger**, essentially a log of events recording money, goods, or services moving between parties. Reports like the **profit and loss** statement or the **balance sheet** are **derived** by summing the ledger's transactions.
- **Mistakes are not erased.** Accountants add a **compensating transaction** (e.g., refunding an incorrect charge). The wrong transaction stays in the ledger forever, for **auditing**. If wrong figures derived from the ledger were already published, the next accounting period's figures include a **correction**. This is entirely normal in accounting.
- **Auditability** matters most in finance but helps many less-regulated systems too. As with batch outputs (Chapter 10), if buggy code writes bad data, recovery is much harder when the code can **destructively overwrite** data. An append-only log of immutable events makes it much easier to **diagnose and recover**.

- **More information than current state.** On a shopping site, a customer adds an item to the cart, then removes it. For order fulfillment, the second event cancels the first. But for **analytics**, it is useful to know the customer considered the item and then decided against it; maybe they buy it later, or found a substitute. The event log keeps this; a database that deletes removed cart items loses it.

Deriving several views from the same event log

- Separating mutable state from the immutable log lets you derive **several read-oriented representations** from the same events, just like multiple stream consumers (Figure 11-5):
  - **Druid**, an analytic database, takes its input straight from **Kafka** in this manner.
  - **Pistachio**, a distributed key-value store, treats **Kafka as its commit log**. (The text does not say who built it; the book's index files it under Yahoo!.)
  - **Kafka Connect sinks** export data from Kafka to many different databases and indexes.
  - Many other systems, such as **search servers**, could sensibly take input from a distributed log too.
- **Easier evolution:** with an explicit translation step from log to database, a new feature that presents existing data differently can build a **separate read-optimized view** from the log and run **alongside** existing systems without modifying them. Running old and new side by side is often easier than a complicated **schema migration** in place. When the old system is no longer needed, shut it down and reclaim its resources.
- **Flexibility:** storing data is easy if you do not care how it will be queried; much of the complexity of schema design, indexing, and storage engines comes from supporting particular query and access patterns (Chapter 3). So separating **the form data is written in** from **the form it is read in**, with **several read views**, gives lots of flexibility. This is called **command query responsibility segregation (CQRS)**.
- **Normalization debate dissolves:** traditional schema design rests on the **fallacy** that data has to be stored in exactly the shape in which it will be queried. Once you can translate a **write-optimized event log** into **read-optimized state**, normalization vs denormalization (Chapter 2) matters much less: it is perfectly reasonable to **denormalize** read views, because the translation process keeps them **consistent** with the log.
- **Book example: Twitter home timelines** (Chapter 1, "Describing Load"): a cache of recent tweets by the people a user follows, like a mailbox. Highly **denormalized** (your tweet is copied into every follower's timeline), yet the **fan-out service** updates all those copies whenever tweets are posted or follow relationships change, keeping the duplication manageable.

Concurrency control

Biggest downside of event sourcing and CDC: log consumers are usually **asynchronous**, so a user could append to the log, immediately read a derived view, and **find their write missing** (Chapter 5, "Reading Your Own Writes").

Fixes:

1. Update the read view **synchronously** with appending to the log. This needs a **transaction** combining the writes atomically: either store both the log and the read view in **one storage system**, or run a **distributed transaction** spanning the systems.
2. Use the approach from Chapter 9, "Implementing linearizable storage using total order broadcast".

On the other hand, deriving state from an event log **simplifies** some concurrency control:

- **Multi-object transactions** (Chapter 7) are often needed only because **a single user action touches data in several places**. Event sourcing lets you design each event to **describe one user action completely on its own**. The action then needs **one write in one place** (appending the event), which is easy to make **atomic**.

- If the event log and application state are **partitioned the same way** (e.g., a customer's event in log **partition 3** only ever changes **partition 3** of the state), then a plain **single-threaded consumer** of the log requires **no write concurrency control**, since it only ever handles one event at a time (compare "Actual Serial Execution", Chapter 7). Within a partition, the log fixes a **serial order** of events and so **eliminates concurrency's nondeterminism**.
- If an event touches **multiple state partitions**, more work is needed (Chapter 12).

Limitations of immutability

- Many non-event-sourced systems still use immutability: databases rely internally on **immutable structures or multi-version data** to offer point-in-time snapshots (Chapter 7, "Indexes and snapshot isolation"); version control systems **Git, Mercurial, and Fossil** use immutable data to keep file history.
- **Can you keep all history forever?** Depends on **churn**:
  - Workloads that mostly **add** data and rarely update/delete are easy to make immutable.
  - Workloads that **update and delete heavily on a relatively small dataset**: immutable history may grow **prohibitively large**, **fragmentation** can become a problem, and **compaction and garbage collection** performance becomes crucial to operational robustness.
- **Administrative deletion**, despite immutability:
  - **Privacy regulations** might oblige you to erase someone's personal data once they close their account.
  - **Data protection legislation** may require removing erroneous information.
  - Sensitive information that **leaked by accident** may have to be contained.
  - Appending a "please treat as deleted" event is **not enough**; you want to **rewrite history** as if the data had never been written. **Datomic** calls this **excision**; the **Fossil** version control system has a similar concept called **shunning**.
- **Truly deleting is surprisingly hard**, because copies live in many places: storage engines, filesystems, and SSDs frequently put new data in a fresh location rather than overwriting the old bytes, and **backups** are often deliberately immutable to prevent accidental deletion or corruption. Deletion is more "making data harder to retrieve" than "making it impossible to retrieve". Still, you sometimes must try (Chapter 12, "Legislation and self-regulation").

Processing Streams

So far the chapter covered the sources of streams (user activity events, sensors, writes to databases) and how they are transported (direct messaging, brokers, event logs). What can you **do** with a stream? Three broad options:

1. **Write the events to a storage system** (database, cache, search index) for other clients to query. As in Figure 11-5, a good way to keep a database in sync with changes elsewhere, especially if the **stream consumer is the only writer** to that database. This is the streaming equivalent of Chapter 10's batch outputs.
2. **Push events to users**, e.g., **email alerts**, **push notifications**, or a **real-time dashboard**. A **human** is the final consumer.
3. **Process input stream(s) to produce output stream(s)**, possibly through a **pipeline** of several stages before reaching output (option 1 or 2).

The rest of the chapter covers option 3. Code that does this is an **operator** or **job**. It is closely related to Unix processes and MapReduce jobs: a stream processor **reads input streams read-only** and **writes output to a different location append-only**.

- **Partitioning and parallelization** patterns are very similar to MapReduce and dataflow engines (Chapter 10), so the book does not repeat them. Basic **mapping** operations (transforming, filtering

records) work the same.

- **The crucial difference: a stream never ends.** Consequences:
  - **Sorting makes no sense** on unbounded data, so **sort-merge joins cannot be used**.
  - **Fault tolerance must change:** a batch job running a few minutes can restart a failed task from the beginning, but a stream job running for **several years** may not be able to restart from the beginning after a crash.

Uses of Stream Processing

Long used for **monitoring**, where an organization wants alerts when certain things happen:

- **Fraud detection**: notice unexpected changes in a credit card's usage pattern and block it if likely stolen.
- **Trading systems**: watch for market price movements and place trades according to predefined rules.
- **Manufacturing**: monitor factory machines and quickly identify the problem on malfunction.
- **Military and intelligence**: track a potential aggressor's activities and raise the alarm on signs of an attack.

These need fairly sophisticated **pattern matching and correlations**. Other uses have emerged too:

Complex event processing

- **CEP**, developed in the **1990s**, analyzes event streams, geared toward **searching for patterns of events**. Like a **regular expression** finds character patterns in a string, CEP lets you specify rules to find event patterns in a stream.
- Patterns are described either in a **high-level declarative, SQL-like query language** or through a **graphical user interface**. These queries are handed to an engine that reads the input streams and runs an internal **state machine** to do the matching. Whenever a pattern is found, it outputs a **complex event** (the source of the name) describing what was detected.
- **Reversed relationship between queries and data:** an ordinary database keeps data **permanently** while queries are **short-lived** (it looks up matching data, answers, and forgets the query). CEP stores **queries long-term**, and events continuously **flow past** them looking for a matching pattern.
- Implementations: **Esper**, **IBM InfoSphere Streams**, **Apama**, **TIBCO StreamBase**, **SQLstream**. **Samza** and similar distributed stream processors are also adding **SQL support** so streams can be queried declaratively.

Stream analytics

The boundary with CEP is blurry, but analytics is generally less about specific event sequences and more about **aggregations and statistical metrics** over many events, e.g.:

- Measuring the **rate** of some event type (how often per time interval).
- Computing a **rolling average** of a value over a time period.
- **Comparing current statistics to previous intervals** (detect trends, or alert on metrics unusually high/low versus the same time last week).
- Usually computed over **fixed time intervals**, e.g., a service's mean **queries per second** across the **past 5 minutes**, plus the **99th percentile** response time for that span. Averaging over a few minutes smooths second-to-second noise while still showing traffic changes promptly. The interval is a **window** (details in "Reasoning About Time").
- **Probabilistic algorithms** are sometimes used: **Bloom filters** for set membership (Chapter 3), **HyperLogLog** for estimating cardinality (number of distinct items), and assorted algorithms for **estimating percentiles** (Chapter 1). They give approximate results but need **much less memory** than exact ones.

- > **Common confusion:** this leads some to believe stream processing is **inherently lossy and inexact**. **Wrong**: nothing about stream processing is inherently approximate; probabilistic algorithms are **merely an optimization**.

- Open source distributed frameworks designed with analytics in mind: **Apache Storm, Spark Streaming, Flink, Concord, Samza, Kafka Streams**. Hosted: **Google Cloud Dataflow, Azure Stream Analytics**.

Maintaining materialized views

- Keeping derived systems (caches, search indexes, warehouses) up to date from a database's change stream (earlier section) is a special case of maintaining **materialized views** (Chapter 3): deriving an alternative, efficiently queryable view of a dataset and updating it whenever the underlying data changes.
- In **event sourcing**, application state, maintained by applying the event log, is **also a materialized view**.
- Unlike analytics, considering only events in a limited time window is usually **not enough**: building the view may need **all events over an arbitrary period**, except obsolete ones discarded by log compaction. Effectively a **window stretching back to the beginning of time**.
- Any stream processor could in principle do this, but retaining events indefinitely **clashes with what some analytics-focused frameworks assume**, since they mostly work with short, bounded windows. **Samza and Kafka Streams** support this usage, building on **Kafka's log compaction**.

Search on streams

- Besides CEP (patterns of **multiple** events), sometimes you need to search **individual events** by complex criteria like **full-text search** queries.
- **Media monitoring** services subscribe to news article feeds and media broadcasts, searching for mentions of companies, products, or topics of interest: formulate the query in advance, then continually match the news stream against it. **Real estate websites** can alert users as soon as a newly listed property fits their search. **Elasticsearch's percolator** feature is one way to implement this.
- **Inverted processing:** conventional search engines **index documents first, then run queries** over the index. Stream search **stores the queries** and runs documents past them, as in CEP. The simplest way tests every document against every query, which gets slow with many queries. Optimization: **build an index over the queries** as well as over the documents, so only candidate queries need to be checked.

Message passing and RPC

Message-passing systems (Chapter 4), e.g., the **actor model**, are also based on messages and events, but are normally not thought of as stream processors:

Actor frameworks	Stream processing
Mainly a way to manage **concurrency** and **distributed execution** of communicating modules	Mainly a **data management** technique
Communication often **ephemeral and one-to-one**	Event logs are **durable and multi-subscriber**
Can communicate **arbitrarily**, including **cyclic request/response**	Usually **acyclic pipelines**: every stream is produced by exactly one job, computed from a clearly defined set of inputs

Crossover exists:

- **Apache Storm's distributed RPC** farms user queries out to nodes that also process event streams; queries are **interleaved** with input-stream events, and results are aggregated and returned to the

user (see also Chapter 12, "Multi-partition data processing").
- Streams **can** be processed with actor frameworks, but many do **not guarantee message delivery on crashes**, so processing is not fault-tolerant without extra retry logic.

Reasoning About Time

Stream processors often handle time, especially for analytics with windows like "**the average over the last five minutes**". That sounds unambiguous, but is surprisingly tricky.

- **Batch:** tasks crunch quickly through a big collection of **historical** events. Any breakdown by time must use the **timestamp embedded in each event**. Consulting the clock of whatever machine runs the job is pointless, since the job's run time is unrelated to when the events occurred. A batch job may read **a year's** events in **a few minutes**; the timeline of interest is the year, not the minutes. Using event timestamps also makes processing **deterministic**: rerunning on the same input gives the same result (Chapter 10, "Fault tolerance").
- **Many stream frameworks** instead use the **local system clock** of the processing machine (**processing time**) for windowing. Simple, and fine if the gap between event creation and processing is negligible. It **breaks down** with significant **processing lag**.

Event time versus processing time

Reasons processing gets delayed: **queueing**, **network faults** (Chapter 8), **performance problems** that create contention in the broker or processor, a **restart** of the consumer, or **replaying historical events** while recovering from a fault or after a bug fix.

Message delays also cause **unpredictable ordering**. Example: a user makes one web request (handled by **server A**), then a second (handled by **server B**). A and B each emit an event, but **B's event reaches the broker before A's**. Stream processors see B then A, although they happened in the opposite order.

> **Star Wars analogy (credited to Kostas Kloudas of the Flink community).** Episode IV came out in 1977, Episode V in 1980, Episode VI in 1983, then Episodes I, II, III in 1999, 2002, 2005, and Episode VII in 2015. Watching in release order, the order you process the movies differs from the order of the story. **Episode number = event timestamp; the date you watched = processing time.** Humans cope; stream algorithms must be written explicitly to handle such timing and ordering issues.

Confusing the two gives bad data. A stream processor measures the **request rate** (requests per second). You **redeploy** it; it is down for **a minute**, then processes the backlog on restart. Measured by processing time, it looks like a sudden anomalous **spike** in requests while the backlog drains, although the real rate was **steady**.

Figure 11-7 (described): the top of the figure shows a web server emitting request events at a steady pace, and a stream processor that consumes them but has a "restarting" gap, after which it works through the backlog at a faster pace. Below are two charts. "Rate as measured by processing time" starts at 10, dips to 0 while the processor restarts, then climbs above 10 (to about 15) while the backlog drains, before settling back down. "Actual request rate" (by event time) is a flat line at 10 the whole time. The dip and the spike are artifacts purely of variations in processing rate.

Knowing when you're ready

With event-time windows, you can **never be sure** you have received **all** events for a window, or whether some are still coming.

Example: grouping into **one-minute windows** to count requests per minute. You have counted events with timestamps in the **37th minute** of the hour; time moves on and most incoming events are now in the **38th and 39th minutes**. When do you declare the 37th-minute window finished and output its count?

You can **time out** after seeing no new events for that window for a while, but some events might still be **buffered on another machine**, delayed by a **network interruption**. You must handle such **straggler events** arriving after the window was declared complete. Two broad options:

1. **Ignore stragglers**; normally they are a small percentage. **Count dropped events** as a metric, and **alert** when the number becomes significant.
2. **Publish a correction**: re-emit the window's value with the stragglers counted, possibly **retracting** the earlier result.

Sometimes producers can send a special marker meaning "**nothing with a timestamp before t will arrive after this**", letting consumers close windows. But with several producers on different machines, each with its own minimum-timestamp threshold, consumers must **track each producer individually**, and **adding/removing producers** becomes trickier.

Whose clock are you using, anyway?

Timestamping is even harder when events are **buffered at several points**. Example: a **mobile app** reports usage-metric events to a server. Used **offline**, it buffers events locally and sends them when a connection is next available, possibly **hours or even days later**. To consumers these look like **extremely delayed stragglers**.

- The event timestamp should really be **when the user interaction happened**, per the **device's local clock**. But a user-controlled device's clock **often cannot be trusted**: it may be accidentally or deliberately wrong (Chapter 8).
- The **server's receive time** (server clock) is more likely **accurate** (you control the server) but **less meaningful** for describing the user interaction.

Fix: log three timestamps.

1. When the event **occurred**, by the **device clock**.
2. When the event was **sent** to the server, by the **device clock**.
3. When the event was **received** by the server, by the **server clock**.

Then:

- **Offset estimate** = (3) − (2): the difference between the device clock and server clock, assuming **network delay is negligible** relative to the needed accuracy.
- **True event time estimate** = (1) + offset, assuming the device clock's offset **stayed the same** from the moment of the event until it was sent.

Own illustrative calculation:

```
t1 (occurred, device clock)  = 09:00:00
t2 (sent,     device clock)  = 12:00:00   # phone came back online 3 h later
t3 (received, server clock)  = 12:05:00
offset       = t3 - t2 = +5 min          # device clock runs 5 min slow
true time est = t1 + offset = 09:05:00
```

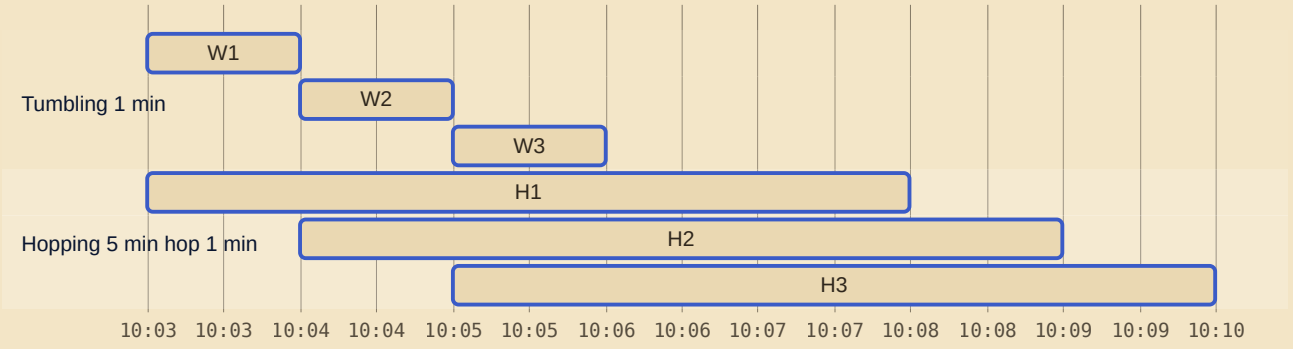
This problem is **not unique to streaming**: batch processing has exactly the same time-reasoning issues; streaming just makes the passage of time more noticeable.

Types of windows

Once you know how to timestamp events, you decide how to define windows, then aggregate within them (count events, average values, etc.). Common types:

Window	Definition	Book's example	How to implement
Tumbling	**Fixed length**; every event belongs to **exactly one** window	1-minute window: events from **10:03:00–10:03:59** in one window, **10:04:00–10:04:59** in the next	**Round each timestamp down** to the nearest minute to find its window
Hopping	**Fixed length**, but windows **overlap** for smoothing	**5-minute** window with **1-minute hop**: **10:03:00–10:07:59**, then **10:04:00–10:08:59**, and so on	Compute **1-minute tumbling** windows, then aggregate over several adjacent ones
Sliding	Contains all events occurring **within some interval of each other**	A 5-minute sliding window includes both **10:03:39** and **10:08:12**, since they are under 5 minutes apart; tumbling or hopping 5-minute windows would separate them because their **boundaries are fixed**	Keep a **buffer of events sorted by time**, removing old ones as they expire from the window
Session	**No fixed duration**; collects a **single user's** events that happen close together in time, closing after the user has been **inactive** for a while	E.g., the session closes once **30 minutes** pass with no events	Sessionization, common in **website analytics** (Chapter 10, "GROUP BY")

Illustrative diagram (not a numbered book figure): tumbling vs hopping boundaries for the same minute-by-minute timeline.



Own illustrative code for a tumbling-window count by event time:

```
from collections import defaultdict
counts = defaultdict(int)
def on_event(ev):
    window_start = ev.ts - (ev.ts % 60)   # round event time down to the minute
    counts[window_start] += 1             # tumbling: each event lands in exactly one window
```

Stream Joins

Chapter 10 showed batch jobs joining datasets by key, a key ingredient of data pipelines. Stream processing extends pipelines to incremental processing of **unbounded** data, so it needs joins just as much. But new events can appear **anytime**, making stream joins harder. Three types: **stream-stream**, **stream-table**, and **table-table** joins.

Stream-stream join (window join)

Example (book's): a website has a **search feature**, and you want to spot recent trends in which URLs people search for.

- Every search logs an event with the **query and results returned**.
- Every click on a result logs another event recording the **click**.
- Computing the **click-through rate** of every URL shown in results requires combining the search and click events, which share the same **session ID**. Advertising systems need similar analyses.

Complications:

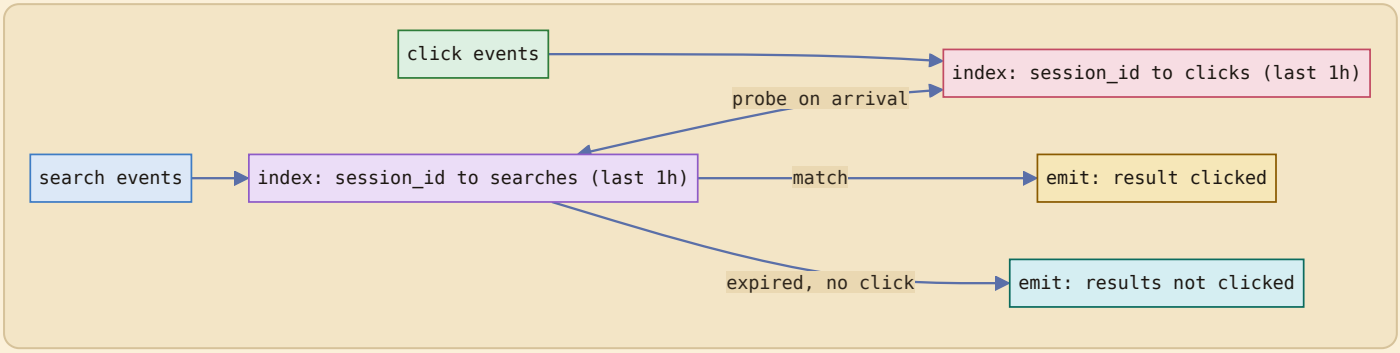
- If the user gives up on the search, **no click ever arrives**.
- Even when it comes, the delay varies wildly: often a few seconds, but possibly **days or weeks** (user runs a search, forgets the browser tab, returns and clicks later).
- Due to network delays, the **click may arrive before the search** event.
- So choose a **join window**, e.g., pair a click with a search only when **no more than one hour** separates them.

> **Common confusion:** embedding the search details into the click event is **not** equivalent to a join. It only tells you about searches that got clicks, not the ones where **no result was clicked**. Judging search quality needs correct click-through rates, which requires **both** kinds of events.

How it works:

1. The processor keeps **state**, e.g., all events from the **last hour**, **indexed by session ID**.
2. When a search or click event arrives, add it to the appropriate index.
3. Also check the **other** index for an event with the same session ID that already arrived.
4. When a pair matches, output an event recording **which result got clicked**.
5. If a search event **expires** with no matching click, emit an event saying **which results were not clicked**.

Illustrative diagram (not a numbered book figure): state kept for a stream-stream window join.



Stream-table join (stream enrichment)

Example (book's): Chapter 10's Figure 10-2 joined **user activity events** with a **database of user profiles**. Treat the activity events as a **stream** and do the join continuously: input is a stream of events containing a user ID; output is a stream of events where the user ID is **augmented with profile information**. This is called **enriching** the activity events with database information.

How it works:

1. Take the activity events one by one.
2. Find each event's user ID in the profile database.
3. Add the profile info to the event.

Where to look up:

- **Querying a remote database**: likely **slow** and risks **overloading** the database (same concern as Chapter 10).
- **Keep a copy of the database inside the stream processor** so lookups are local and need no network round trip. Very like Chapter 10's **map-side hash joins**: the local copy could be an **in-memory hash table** if small, or an **index on local disk**.

Difference from batch: a batch job reads a **snapshot** of the database taken at one moment, whereas a stream processor **runs for a long time** while the database keeps **changing**, so the local copy must stay **up to date**. Solution: **CDC**. Besides the activity stream, the processor also subscribes to the **profile database's changelog**; when a profile is created or modified, it updates its local copy. So we get a **join between two streams**: activity events and profile updates.

A stream-table join is thus very like a stream-stream join. The biggest difference: the **table changelog** side is joined using a window that extends back to the **"beginning of time"** (conceptually infinite), with newer record versions **overwriting** older ones. For the **event** stream, the join may keep **no window at all**.

Own illustrative code:

```
profiles = {}                                  # local copy of the users table
def on_profile_change(chg):                    # from the CDC changelog stream
    profiles[chg.user_id] = chg.new_row        # newer version overwrites older (infinite window)
def on_activity(ev):                           # from the activity event stream
    emit({**ev, "profile": profiles.get(ev.user_id)})   # enriched event, no window kept
```

Table-table join (materialized view maintenance)

Example (book's): Twitter timelines (Chapter 1). Reading a user's home timeline by iterating over everyone they follow, finding recent tweets, and merging them is too expensive. Instead keep a **timeline cache**: a per-user **"inbox"** that tweets are written into as they are sent, so reading a timeline is a **single lookup**.

Maintaining that cache requires these event handlers:

- When user **u** sends a new tweet, add it to the timeline of **every user following u**.
- When a user **deletes** a tweet, remove it from **all** users' timelines.
- When **u1 starts following u2**, add u2's **recent tweets** to u1's timeline.
- When **u1 unfollows u2**, remove u2's tweets from u1's timeline.

So the stream processor needs event streams for **tweets** (send, delete) and **follow relationships** (follow, unfollow), and must maintain a database of each user's **followers**, to know which timelines to update for a new tweet.

Equivalently, it maintains a **materialized view** of a query joining two tables (tweets and follows). Own SQL in the same spirit (different names than the book's):

```
SELECT f.reader_id AS timeline_owner,                        -- one timeline per follower
       array_agg(p.* ORDER BY p.posted_at DESC) AS timeline  -- that follower's posts, newest first
FROM   posts p
JOIN   subscriptions f ON f.author_id = p.author_id          -- follower sees posts by authors they
follow
GROUP  BY f.reader_id;
```

- `JOIN ... ON f.author_id = p.author_id` is exactly the stream join of post events and subscription events.
- `GROUP BY f.reader_id` produces one timeline row per reader.

- The timelines are a **cache of this query's result**, updated whenever either underlying table changes.

> **Footnote: product rule.** If a stream is the derivative of a table (Figure 11-6), and a join is a product of two tables **u·v**, then changes to the joined result obey the **product rule (u·v)' = u'v + uv'**. Put simply: new tweet changes get joined with the follower lists as they are **now**, and follower changes get joined with the tweets as they are **now**.

Time-dependence of joins

- All three join types share a structure: the processor builds **state** from **one** input (recent search/click events, user profiles, or follower lists) and **looks it up** for each message on the **other** input.
- **Order of state-maintaining events matters** (following then unfollowing differs from the reverse). A partitioned log keeps order **inside** each partition, but usually guarantees **nothing about order between different streams or partitions**.
- **Question:** if events on different streams happen at about the same time, which order are they processed in? In the stream-table case, when someone edits their profile, which of their activity events get the **old** profile (processed before the update) and which with the **new** one? In short: when state changes over time, **what point in time do you join against?**
- **Book example: tax rates.** Selling things requires applying the right **tax rate** to invoices, depending on **country or state**, **product type**, and **date of sale** (rates change). Joining sales with a tax-rate table, you probably want the rate **at the time of sale**, which may differ from the current rate when **reprocessing historical data**.
- If ordering across streams is undetermined, the join is **nondeterministic**: rerunning the same job on the same input may not give the same result, because input events may interleave differently.
- In data warehouses this is the **slowly changing dimension (SCD)** problem, often solved by giving each **version** of the joined record a **unique identifier**: every time the tax rate changes it gets a new ID, and the invoice records the ID of the rate at sale time. This makes the join **deterministic**, but **log compaction becomes impossible**, since all versions of the table's records must be kept.

Join type	Inputs	State kept	Window	Book example
Stream-stream	Two activity-event streams (or one stream self-joined)	Recent events of both sides, indexed by key	Limited (e.g., 1 hour)	Search events + click events by session ID
Stream-table	Activity stream + database changelog	Local copy of the table	Infinite for the table; none for the stream	Activity events enriched with user profiles
Table-table	Two database changelogs	Latest state of both tables	Infinite for both	Twitter timelines from tweets + follows

Fault Tolerance

Batch recap: if a MapReduce task fails, just restart it on another machine and discard the failed task's output. Retrying invisibly is possible because **inputs never change**, each task's output goes to **its own file in HDFS**, and that output becomes visible **only after the task succeeds**.

- The result: output identical to a run with no failures at all, even though some tasks did fail. It looks as if every input record was processed **exactly once**: none skipped, none processed twice. Restarts may actually process records multiple times, but the **visible effect** is as if once. This is **exactly-once semantics**, though **effectively-once** would be more accurate.

- **Streaming problem:** you cannot wait until a task finishes before revealing its output, because a stream is infinite and processing **never finishes**.

Microbatching and checkpointing

Microbatching (used in **Spark Streaming**):

1. Break the stream into **small blocks**.
2. Treat each block like a **miniature batch process**.

- Batch size is typically **around one second**, a performance compromise: **smaller batches** cost more in **scheduling and coordination**, while **bigger batches** make you **wait longer** to see results.
- As a side effect, microbatching creates a **tumbling window the size of one batch**, based on **processing time** rather than on the events' own timestamps. Jobs needing larger windows must explicitly **carry state over** from one microbatch to the next.

Checkpointing (used in **Apache Flink**):

1. Every so often, take **rolling checkpoints of operator state** and save them to **durable storage**.
2. If an operator crashes, it **restarts from its latest checkpoint** and **discards any output** produced between that checkpoint and the crash.

- Checkpoints are triggered by **barriers** in the message stream, similar to microbatch boundaries but **without forcing a particular window size**.

Limit: within the stream processing framework, both give the same exactly-once semantics as batch. But once output **leaves the processor** (database writes, messages to an outside broker, emails), the framework **cannot discard** a failed batch's output. Restarting a failed task then causes the external **side effect twice**; microbatching or checkpointing alone cannot prevent that.

Atomic commit revisited

To appear exactly-once despite faults, **every output and side effect** of handling an event must happen **if and only if** the handling succeeds. These include:

- Messages to **downstream operators** or to **outside messaging systems**, emails and **push notifications** included.
- **Database writes**.
- Changes to **operator state**.
- **Acknowledging input messages**, which in a log-based broker means **advancing the consumer offset**.

They must **all happen atomically, or none happen**, and not drift out of sync. This is the idea from Chapter 9's "Exactly-once message processing" (distributed transactions and 2PC).

Chapter 9 covered problems with traditional distributed transactions like **XA**. But in more **restricted environments** an efficient atomic commit facility is possible. Used in **Google Cloud Dataflow** and **VoltDB**, and **Apache Kafka** had comparable features planned. Unlike XA, these do **not** try to span **heterogeneous technologies**; they keep transactions **internal** by having the stream framework itself manage both state changes and messaging. Grouping several input messages into one transaction **amortizes** the protocol's cost.

Idempotence

Goal: throw away failed tasks' partial output so retries are **safe and never apply twice**. Distributed transactions achieve this; another is **idempotence**.

- **Idempotent operation:** doing it multiple times has the **same effect as once**.
  - **Assigning a fixed value to a key** in a key-value store is idempotent (rewriting just overwrites with an identical value).
  - **Incrementing a counter** is **not** (doing it again increments twice).

- **Making an operation idempotent with metadata:** messages read from **Kafka** each carry a durable, **monotonically increasing offset**. Whenever you write to an outside database, **save the offset of the message responsible for the latest write** together with the value. Then you can tell whether an update was already applied and skip repeating it.
- **Storm's Trident** state handling is based on a similar idea.

Own illustrative code:

```
def apply(msg):                                  # msg.offset is monotonically increasing
    row = db.get(msg.key) or {"total": 0, "last_offset": -1}
    if msg.offset <= row["last_offset"]:
        return                                   # already applied before the crash: skip
    row["total"] += msg.amount                   # non-idempotent increment...
    row["last_offset"] = msg.offset              # ...made safe by storing the offset with it
    db.put(msg.key, row)                         # single write stores both atomically
```

Assumptions when relying on idempotence:

1. A restarted task must see **identical messages in identical order** on replay (log-based brokers provide this).
2. Processing must be **deterministic**.
3. **No other node may update** that value at the same time.

When failing over between processing nodes, **fencing** may be needed (Chapter 8, "The leader and the lock") to stop interference from a node thought dead but actually alive. Despite the caveats, idempotent operations can achieve exactly-once semantics with only **small overhead**.

Rebuilding state after a failure

Any stateful stream process, e.g., **windowed aggregations** (counters, averages, histograms) and **tables and indexes used for joins**, must be able to **recover that state** after a failure.

Option	How	Trade-off / example
Remote datastore, replicated	Keep state in a remote DB	Querying a remote DB **per message can be slow** (as in stream-table joins)
Local state, replicated periodically	Keep state local to the processor; on recovery, the new task reads the replicated state and resumes **without data loss**	**Flink**: periodically snapshots operator state to durable storage such as **HDFS**. **Samza and Kafka Streams**: write state changes to a **dedicated, log-compacted Kafka topic**, much like CDC. **VoltDB**: **redundantly processes each input message on several nodes** (a form of statement-based replication; Chapter 7, "Actual Serial Execution")
Rebuild from input streams, no replication	Recompute state from inputs	If state aggregates a **fairly short window**, just **replay the input events** for that window. If state is a **local DB replica maintained by CDC**, rebuild it from the **log-compacted change stream**

All these trade-offs depend on the **underlying infrastructure's performance**: on some systems the **network can beat disk on latency**, and **network throughput can rival disk throughput**. No choice is **best everywhere**, and the merits of local vs remote state may **shift as storage and networking technologies evolve**.

Chapter summary

- **Stream processing = batch processing done continuously on unbounded input.** Message brokers and event logs are the streaming equivalent of a **filesystem**.

- **Two broker types:**

	AMQP/JMS-style broker	Log-based broker
Delivery	Assigns **individual messages** to consumers	Hands a **whole partition** to **one consumer node**, delivering its messages in a **fixed order**
Progress tracking	Consumer **acks each message**; message **deleted** once acked	Consumer **checkpoints the offset** of the last processed message
Parallelism	Per message	Through **partitioning**
Rereading	Not possible	Messages **retained on disk**; you can jump back and reread
Good for	**Asynchronous RPC**, e.g., a **task queue**, where exact order does not matter and old messages need not be reread	Stream processing that consumes inputs and produces **derived state or streams**

- The log approach resembles **database replication logs** (Chapter 5) and **log-structured storage engines** (Chapter 3).
- **Sources of streams:** user activity events, sensors emitting periodic readings, data feeds such as **financial market data**, and **writes to a database**. The changelog can be captured **implicitly via CDC** or **explicitly via event sourcing**. **Log compaction** lets a stream retain a **full copy** of a database.
- **Databases as streams** enable powerful integration: keep search indexes, caches, and analytics systems continuously updated by consuming the change log; build **fresh views** by consuming the log from the beginning to now.
- **Maintaining state as streams and replaying messages** underpin stream **joins** and **fault tolerance**.
- **Purposes of stream processing: CEP** (event patterns), **stream analytics** (windowed aggregations), **materialized views** (keeping derived data current).
- **Time is hard: processing time vs event time**, and **stragglers** arriving after a window seemed complete.
- **Three join types:**
  - **Stream-stream:** both inputs are activity events; find related events within a time window (e.g., two actions by the **same user within 30 minutes**). The two inputs can be the **same stream** (a **self-join**).
  - **Stream-table:** activity events + a **database changelog** keeping a local DB copy current; each event is **enriched** by querying it.
  - **Table-table:** both inputs are **changelogs**; each change on either side is combined with the **current state** of the opposite side, producing a stream of updates to the **materialized view of the join**.
- **Fault tolerance / exactly-once:** as in batch, discard partial output of failed tasks. But a stream job runs forever and emits output continuously, so you cannot discard everything; use finer-grained recovery: **microbatching, checkpointing, transactions, or idempotent writes**.

Interview angle

1. Kafka-style log vs RabbitMQ-style queue: when to use which? Traditional (AMQP/JMS) brokers deliver individual messages to consumers, delete them on ack, and load-balance per message; good for task queues where messages are expensive and order does not matter. Log-based brokers append to partitioned, durable logs; each partition is consumed in order by one consumer in a group; progress is an offset; messages are retained and can be replayed. Good for high-throughput, order-sensitive streams and for building derived data.

- *Follow-up: Why does a queue reorder messages?* Load balancing plus redelivery: if a consumer crashes mid-message, the broker redelivers it to another consumer that has already moved on (m4, m3, m5). Fix: one queue per consumer.
- *Follow-up: What limits parallelism in a log?* Consumers in a group ≤ number of partitions; a slow message causes head-of-line blocking in its partition.

2. How do you keep a search index / cache in sync with the database? Avoid dual writes: they race (DB ends at B, index at A) and can partially fail. Use change data capture: make the database the single leader, capture its replication log (e.g., Debezium/Maxwell for MySQL binlog, Bottled Water for PostgreSQL WAL, Mongoriver for the oplog), publish changes in order through a log-based broker, and let the index and cache consume them as followers. Bootstrap new consumers with a consistent snapshot tied to a log offset, or from a log-compacted topic starting at offset 0.

- *Follow-up: Downside?* CDC is asynchronous, so replication-lag anomalies (e.g., not reading your own writes) apply.

3. What is event sourcing, and how is it different from CDC? Event sourcing stores immutable application-level events ("student cancelled enrollment") as the source of truth and derives current state from them; CDC extracts low-level changes from a mutably used database. Event sourcing separates commands (validated, may fail) from events (facts, cannot be rejected by consumers), enables multiple read views (CQRS), auditability, and easier evolution. But you cannot compact the log per key, you need snapshots for speed, and deleting data (privacy law) requires excision-like features.

- *Follow-up: How do you validate a seat booking?* Synchronously before emitting the event (serializable transaction), or split into tentative-reservation and confirmation events.

4. Event time vs processing time; how do you handle late data? Event time is when it happened; processing time is when the processor sees it. Windowing by processing time creates artifacts (restart backlog looks like a spike). With event time you never know a window is complete: ignore stragglers (and monitor drops), or publish corrections/retractions; optionally use "no earlier than t" markers per producer. For untrusted device clocks, log occurred/sent (device) and received (server) timestamps and correct by the offset.

- *Follow-up: Name the window types.* Tumbling, hopping, sliding, session.

5. Stream joins? Stream-stream (window join, e.g., searches and clicks within 1 hour by session ID), stream-table (enrichment with a local CDC-maintained copy of the table), table-table (materialized view such as Twitter timelines from tweets and follows). All keep state from one input and query it with the other; cross-stream ordering is not guaranteed, so joins can be nondeterministic. The SCD fix is to version joined records (e.g., tax-rate IDs), at the cost of losing log compaction.

6. How do stream processors achieve exactly-once? Microbatching (Spark Streaming, ~1s batches) or checkpointing via barriers (Flink) give exactly-once inside the framework. For external side effects, use atomic commit kept internal to the framework (Google Cloud Dataflow, VoltDB, planned for Kafka) or idempotent writes (store the Kafka offset with the value; requires deterministic replay in the same order, no concurrent writers, fencing on failover). State is recovered via snapshots (Flink to HDFS), compacted changelog topics (Samza, Kafka Streams), redundant processing (VoltDB), or replaying the input.

- *Follow-up: Is "exactly-once" literally true?* No; records may be processed more than once, but the visible effect is as if once, so "effectively-once" is more accurate.

Self-check

1. **Why does polling a database for new events become expensive at low latency, and what do messaging systems do instead?** The more often you poll, the smaller the fraction of polls that find anything new, so overhead grows. Messaging systems push/notify consumers when new events arrive.

2. **List the three options when producers outpace consumers, and say which one Unix pipes/TCP use and which one a log-based broker uses.** Drop, buffer, backpressure. Pipes and TCP use backpressure (small fixed buffer, sender blocks). A log-based broker buffers with a large but fixed-size on-disk buffer.
3. **Using the book's numbers, how long can one disk buffer messages, and why is this usually longer in practice?** A 6 TB drive written sequentially at 150 MB/s fills in about 11 hours. Deployments rarely use full write bandwidth, so logs often hold days or weeks of messages.
4. **Scenario: two clients concurrently set item X to A and B using dual writes to a database and a search index. What can go wrong and how does CDC fix it?** Writes can interleave differently at each system (DB ends at B, index at A), leaving permanent silent inconsistency; also one write may fail. CDC makes the DB the single leader and applies its changes to the index in the DB's commit order.
5. **Why can a CDC topic be log-compacted but an event-sourced log generally cannot?** A CDC update carries the full new record, so the latest event per key determines state. Event-sourced events express intent and do not override earlier ones, so full history is needed.
6. **What is the difference between a command and an event, and where must validation happen?** A command is a request that may still fail validation; once accepted it becomes an immutable event (a fact). Consumers cannot reject events, so validation must happen synchronously before the event is written (or split into tentative + confirmation events).
7. **Give two reasons, other than performance, that immutability can be a problem.** Legal/administrative deletion needs (privacy regulation after account closure, data protection requiring removal of wrong data, containing leaks), which require rewriting history (Datomic excision, Fossil shunning); and true deletion is hard since copies persist in SSDs, filesystems, and immutable backups.
8. **A dashboard shows a traffic spike right after you redeployed the stream processor. What is likely happening?** It windows by processing time; the backlog accumulated during the one-minute downtime was processed all at once, creating an artificial spike although the real (event-time) rate was steady.
9. **Which window type would put events at 10:03:39 and 10:08:12 in the same 5-minute window, and why wouldn't the others?** A sliding window, since the events are under 5 minutes apart. Tumbling and hopping windows use fixed boundaries, so these events fall in different windows.
10. **What assumptions does idempotent-write-based exactly-once rely on?** Replay of the same messages in the same order after restart, deterministic processing, no other node concurrently updating the same value, plus fencing during failover to block zombie nodes.
11. **Trade-off: when would you rebuild operator state from the input stream instead of replicating it?** When the state is an aggregation over a fairly short window (replaying is fast), or when it is a CDC-maintained local database copy that can be rebuilt from a log-compacted change stream.

Chapter 12 — The Future of Data Systems

Book: Chapter 12, pp. 489–552

Overview

Every earlier chapter described how data systems work **today**. This last chapter changes tone: Kleppmann looks ahead and argues for how data systems **ought** to be built. Because this is opinion rather than settled fact, he writes in the first person and openly invites the reader to disagree.

The chapter opens with a quotation from Thomas Aquinas that is usually paraphrased as: *a captain whose only goal was to keep the ship safe would never leave port*. The point: a system exists to serve a purpose beyond merely surviving. Reliability, scalability and maintainability (the themes of Chapter 1) are means, not ends. The final goal is applications that are **robust, correct, evolvable, and ultimately good for people**.

The chapter tries to answer four big questions:

1. **Data integration** – No single tool does everything well, so real applications glue several tools together (a database, a search index, a cache, a warehouse...). How do you keep all those copies of the data consistent?
2. **Unbundling databases** – A database internally already keeps indexes, views and replicas in sync. Can we take those internal mechanisms "out" of the database and use them to wire an entire organisation's systems together?
3. **Aiming for correctness** – If we build systems out of asynchronous logs and stream processors instead of big distributed transactions, how can we still be sure data is not lost, duplicated or corrupted?
4. **Doing the right thing** – Data is often about people. What ethical responsibilities do engineers have around prediction, bias, surveillance and privacy?

Why it matters in real systems: almost every company of meaningful size ends up with a "zoo" of data stores fed from one another. The ideas here — a single ordered log as the source of truth, everything else derived from it, idempotent consumers, end-to-end request IDs, auditing — are exactly how many modern event-driven architectures (Kafka + stream processors + many downstream stores) are designed.

Everyday analogy. Think of a spreadsheet. You type a number into one cell and every formula that depends on it updates by itself. You never write "now please go refresh cell D17". This chapter's dream is a whole company's data infrastructure that behaves like that spreadsheet: change the source record, and every search index, cache, report and recommendation that depends on it updates automatically, correctly, and even if machines crash along the way.

Key terms

Term	Plain-English meaning
Data integration	Making sure the same data ends up, in the right shape, in every system that needs it (database, search index, cache, warehouse, ML model...).
System of record	The authoritative "source of truth" copy of the data; new writes land here first.
Derived data / derived dataset	Any dataset computed from other data by a repeatable process (index, cache, materialized view, ML model). You can throw it away and rebuild it.
Dataflow	The path data takes as it moves and gets transformed from system to system.
Change data capture (CDC)	Reading a database's stream of changes so other systems can apply the same changes.
Event sourcing	Storing every user action as an immutable event in a log and deriving current state from that log.
Total order / total order broadcast	Every consumer sees all events in exactly the same sequence; formally equivalent to consensus.
Causal dependency	Event B depends on event A (B happened because of, or after seeing, A), so B must not be processed as if A had not happened.
Logical timestamp	A counter-based timestamp (e.g., Lamport clock) that orders events without relying on physical clocks.
Idempotent	An operation that has the same effect whether you apply it once or many times.
Deterministic	Same input always gives the same output (no randomness, no clock reading, no network calls).
Reprocessing	Re-running a (new) transformation over all the old data to build a new derived view.
Lambda architecture	Running a batch system and a stream system side by side over the same events, then merging their outputs.
Unified batch and stream processing	Doing both historical reprocessing and live processing in one engine.
Exactly-once (effectively-once) semantics	The end result is as if every message were processed once, even though retries happened.
Event time vs processing time	When an event actually happened vs when the system gets round to processing it.
Unbundling databases	Treating the features inside a database (indexes, views, replication) as separate components you plug together with logs.
Federated database / polystore	One query interface that reads from many different underlying stores ("unifying reads").
Database inside-out	Another name for the unbundling / dataflow approach, after a 2014 talk by the author.
Derivation function	The code that turns base data into a derived dataset (e.g., "extract field and sort" for an index).
Stateless service	An application server that forgets everything after answering a request; state lives in a database.

Term	Plain-English meaning
Observer pattern	Code subscribes to a value and is notified when it changes, instead of repeatedly checking.
Stream-table join (enrichment)	Joining each incoming event with the latest state of some table kept locally.
Write path	Work done eagerly when data arrives (updating indexes, caches, views).
Read path	Work done lazily only when someone asks a question.
Offline-first application	App that works mainly against a local on-device database and syncs with servers in the background.
Server-sent events / WebSockets	Protocols letting a server push messages to a browser over a long-lived connection.
End-to-end argument	Some guarantees (dedup, integrity, security) can only be fully achieved by the endpoints of a communication, not by lower layers alone.
Operation identifier / request ID	A unique ID generated by the client for one logical action, used to detect and drop duplicates everywhere downstream.
Uniqueness constraint	Rule that no two records may share a value (username, seat, request ID).
Compensating transaction	A later action that corrects a mistake (refund, apology, rebooking) instead of preventing it up front.
Timeliness	Users see an up-to-date state. Violations are temporary ("eventual consistency").
Integrity	No lost, duplicated, contradictory or false data. Violations are permanent ("perpetual inconsistency").
Coordination-avoiding data system	A system that keeps integrity without synchronous cross-node coordination.
System model	The set of assumptions about what can and cannot go wrong (crashes, delays, disk loss...).
Auditing	Actively checking that data is still correct and complete.
Rowhammer	Hardware effect where repeatedly accessing memory rows flips bits in neighbouring rows.
Merkle tree	A tree of hashes that lets you cheaply prove a record is part of a dataset.
Certificate transparency	A public, Merkle-tree-based log used to check TLS/SSL certificates.
Predictive analytics	Using data to forecast things, including about individual people (credit risk, reoffending).
Algorithmic prison	Being systematically shut out of jobs, loans, travel, etc. because an algorithm labelled you risky.
Feedback loop	A prediction changes the world in a way that reinforces the prediction.
Systems thinking	Considering the whole system, including the humans in it, to foresee consequences.
Surveillance	Tracking people mainly to serve someone else's interests (e.g., advertisers).
Data exhaust	Label for behavioural data produced as a by-product of using a service.
Data broker	A company that buys, combines, infers and resells personal data.

Data Integration

Throughout the book, every problem had several solutions with different trade-offs: log-structured storage vs B-trees vs column storage (Chapter 3), single-leader vs multi-leader vs leaderless replication (Chapter 5), and so on. "Store data and find it again" has no single right answer.

Any one software product has to commit to one approach, because making even one code path fast and robust is hard; trying to cover every use case in one product nearly guarantees a mediocre implementation. So even a "general-purpose" database is really tuned for some particular usage pattern.

This creates two challenges:

1. **Matching tools to situations.** Vendors rarely advertise what their product is bad at. The earlier chapters are meant to give you the questions to ask so you can see the trade-offs yourself.
2. **Using data in many ways at once.** Even if you know exactly which tool suits which job, a complex application uses the same data in several different ways, and no one tool fits all of them. You inevitably combine several pieces of software.

Combining Specialized Tools by Deriving Data

The classic example: an OLTP database plus a full-text search index. PostgreSQL has built-in full-text search that is good enough for simple apps, but serious search needs a dedicated information-retrieval tool (e.g., Elasticsearch). Search indexes, on the other hand, make poor durable systems of record. So you need both.

And it rarely stops at two. You might also need:

- copies in analytics systems (data warehouses, batch and stream processors),
- caches or denormalized versions of objects,
- machine learning, classification, ranking or recommendation systems fed from the data,
- notifications triggered by changes.

> **Beware "99% of people only need X".** The author notes engineers often claim most people do or don't need some feature. He argues this reveals more about the speaker's own experience than about the technology. What looks useless to one team is central to another. The need for data integration usually becomes obvious only when you look at data flows across the **whole organisation**.

Reasoning about dataflows

When copies of the same data live in several stores (each optimised for a different access pattern), you must be crystal clear about:

- where data is **written first**, and
- which representation is **derived from which** source,
- how each copy gets populated, and in which format.

A clean design:

1. The application writes **only** to the system-of-record database.
2. Change data capture (CDC, Chapter 11) reads that database's change stream.
3. The index consumes those changes and applies them **in exactly the order they happened**.

If CDC is the *only* way the index is updated, the index is purely derived from the database and will agree with it (apart from software bugs). The database write is the single entry point for new information.

What goes wrong otherwise (dual writes). If the application writes to both the database and the search index itself, two clients can send conflicting writes concurrently and the two stores may apply them in opposite orders (the book's Figure 11-4 in Chapter 11). Neither store is "in charge" of the order, so they can make contradictory choices and stay inconsistent **forever**.

The fix is a single ordering authority. Push all user input through one system that decides the order of all writes; then every other representation can be derived by processing writes in that same order. This is state machine replication again (see "Total Order Broadcast", Chapter 9). Whether that log is a CDC stream or an event-sourcing log matters less than the principle: **decide on a total order once**.

Updating a derived system from an event log can usually be made **deterministic and idempotent**, which makes recovering from faults fairly easy (replay the log).

Derived data versus distributed transactions

The traditional way to keep different systems in agreement is a distributed transaction (atomic commit / two-phase commit, Chapter 9). How do derived-data systems compare?

Aspect	Distributed transactions	Log-based derived data (CDC / event sourcing)
How write order is decided	Locks for mutual exclusion (e.g., two-phase locking)	Position in a log
How "exactly once" is achieved	Atomic commit	Deterministic retry + idempotence
Timing guarantee	Usually linearizable, so you can read your own writes	Usually updated asynchronously, so no such timing guarantee by default
Fault behaviour	One failing participant aborts everything, spreading failure	Faults stay local; a slow consumer just falls behind
Practicality across vendors	XA has poor fault tolerance and performance	Much simpler abstraction, works across heterogeneous systems

The author's view:

- Distributed transactions have worked in constrained environments willing to pay their price.
- XA in particular performs and tolerates faults poorly (Chapter 9), which limits it severely.
- A better distributed transaction protocol might be possible, but getting everyone to adopt and integrate it would be hard and slow.
- So, for integrating different systems today, he considers **derived data driven by a log the best bet**.
- But he rejects telling people "eventual consistency is inevitable, deal with it" without guidance. Guarantees like reading your own writes are valuable. The "Aiming for Correctness" section later looks for a middle ground.

The limits of total ordering

For small systems, building one totally ordered log is easy — any single-leader database already does it. At larger scale, limits appear:

1. **Throughput.** A total order usually means every event goes through one leader node. If one machine can't keep up, you partition the log (Chapter 11, "Partitioned Logs"), and then the relative order of events in **different partitions** is undefined.
2. **Geography.** With several datacenters (e.g., to survive losing a whole datacenter), you typically have a leader per datacenter because synchronous cross-datacenter coordination is too slow (multi-leader replication, Chapter 5). Events that start in different datacenters have no defined order.

3. **Microservices.** A common design gives each service its own durable state with nothing shared. Events from different services have no defined order between them.
4. **Client-side state / offline clients.** Apps that update local state instantly (without waiting for the server) or work offline mean clients and servers very likely see events in different orders.

Formally, agreeing on a total order of events is called **total order broadcast**, and it is **equivalent to consensus** (Chapter 9). Most consensus algorithms assume one node can handle the whole event stream and have no way to spread the ordering work over many nodes. Designing consensus that scales past one node's throughput and works well across geographies is **still an open research problem**.

Ordering events to capture causality

When events are unrelated, lacking a total order is harmless — concurrent events can be ordered any way. Some cases are easy: all updates to the same object can be ordered by routing everything for that object ID to the same log partition. But causality can hide in subtler places.

The book's example (social network break-up):

1. User A and user B were a couple and have just broken up.
2. A **unfriends** B.
3. A then sends a message to remaining friends complaining about B. A expects B not to see it, because the message came *after* the unfriending.
4. But friendships are stored in one system and messages in another. The "unfriend happened before message" dependency can get lost.
5. The notification service might process the message event **before** the unfriend event, and notify B. Wrong.

The notification is effectively a **join** between messages and the friend list, so this is the same time-dependence-of-joins issue from Chapter 11. There's no simple answer, but starting points include:

Technique	Helps because	Limitation
Logical timestamps (Chapter 9, "Sequence Number Ordering")	Give a total order without coordination	Recipients must still handle out-of-order delivery; extra metadata must travel with events
Log an event recording **what the user saw** before deciding, give it a unique ID, and have later events reference that ID	Explicitly records the causal dependency	Not discussed here; revisited in "Reads are events too", which notes logging reads adds storage and I/O cost
Conflict resolution algorithms (Chapter 5)	Handle events that arrive in an unexpected order when maintaining state	Useless when the action has external side effects, like sending a notification

The author hopes application patterns will emerge that capture causality efficiently without forcing everything through the total-order-broadcast bottleneck.

Batch and Stream Processing

The author's summary of data integration's goal: **every place that needs the data gets it, in the shape it needs**. Getting there means reading inputs, then transforming, joining, filtering and aggregating them, training and evaluating models, and finally writing the results out — exactly what batch and stream processors are for. Their outputs are derived datasets: search indexes, materialized views, recommendations, aggregate metrics, etc.

From Chapters 10 and 11: batch and stream processing share most principles. The fundamental difference is that streams are **unbounded** (never end) while batch inputs are **finite and known in size**. Implementation differences exist but are blurring:

- **Spark** does streaming on top of a batch engine by chopping the stream into **microbatches**.
- **Apache Flink** does batch on top of a streaming engine.
- In principle either can emulate the other, though performance differs — e.g., microbatching can be slow for **hopping or sliding windows**.

Maintaining derived state

Batch processing has a strongly **functional** flavour, even if the code isn't in a functional language:

- deterministic, pure functions (the result is determined solely by what goes in),
- no side effects apart from explicit outputs,
- inputs treated as immutable, outputs as append-only.

Stream processing is similar but adds **managed, fault-tolerant state** to operators (Chapter 11).

Deterministic functions with clear inputs and outputs help fault tolerance (idempotence) **and** make an organisation's dataflows easier to reason about. Be it a cache, a statistical model or a search index, think of it as a **pipeline** deriving one thing from another: state changes in one system flow through functional code and get applied to derived systems.

Why asynchronous? You *could* maintain derived systems synchronously, the way a relational database updates secondary indexes in the same transaction as the table write. But asynchrony is what makes log-based systems robust:

- A fault in one part stays contained locally.
- Distributed transactions abort if **any** participant fails, so they amplify failures across the system.

Secondary indexes on partitioned data often cross partition boundaries (Chapter 6): a **term-partitioned** index needs writes to go to several partitions; a **document-partitioned** index needs reads sent to all partitions. That communication between partitions also scales best and is most dependable if the index is updated **asynchronously** (see also "Multi-partition data processing" below).

Reprocessing data for application evolution

Both styles are useful for derived data:

Stream processing	Batch processing
Reflects input changes in derived views with low delay	Reprocesses large amounts of accumulated history to create **new** views of existing data

Reprocessing is a powerful way to **evolve** a system (Chapter 4). Without it, schema evolution is limited to small steps like adding an optional field or a new record type — true for both schema-on-write and schema-on-read. With reprocessing you can restructure a dataset into an **entirely different model** to fit new requirements.

> **Schema migrations on railways (book sidebar).** In 19th-century England, railways used competing track **gauges** (distance between rails), and trains for one gauge couldn't run on another. After a standard gauge was chosen in **1846**, other tracks had to be converted without shutting lines for years. The trick: add a **third rail** to make "dual/mixed gauge" track, so trains of both gauges can run. Once all trains use the standard gauge, remove the extra rail. This lets old and new coexist during a slow migration — but it's expensive, which is why nonstandard gauges survive, e.g., **BART** in the San Francisco Bay Area differs from most of the US.

Derived views enable gradual migration, just like the third rail:

1. Keep the old schema and the new schema as **two independently derived views** over the same underlying data.
2. Route a small share of users to the new view to test performance and find bugs; most users stay on the old one.
3. Gradually raise the share on the new view.
4. Eventually drop the old view.

Every step is **reversible** — you always have a working system to fall back on. Lower risk of irreversible damage means more confidence, which means you can move faster.

The lambda architecture

If batch handles history and streaming handles recent updates, how do you combine them? The **lambda architecture** is a well-known proposal:

1. Record incoming data as immutable events appended to an ever-growing dataset (like event sourcing).
2. Derive read-optimised views from those events.
3. Run **two systems in parallel**: a batch system (e.g., **Hadoop MapReduce**) and a stream system (e.g., **Storm**).
4. The stream processor quickly produces an **approximate** update to the view.
5. Later the batch processor handles the same events and produces a **corrected** view.

Rationale: batch is simpler and less bug-prone; stream processors were considered flakier and tougher to make fault-tolerant; and the streaming half could get away with quick approximate algorithms while the batch half ran slower exact ones.

It was influential and positive — it popularised deriving views from immutable event streams and reprocessing when needed. But the author lists practical problems:

- **Double implementation.** The same logic must run in both a batch and a stream framework. Libraries like **Summingbird** abstract over both, but you still debug, tune and operate two systems.
- **Merging outputs.** Stream and batch produce separate outputs that must be merged per user request. Easy for simple aggregations over a tumbling window; much harder for joins, sessionization, or non-time-series output.
- **Full reprocessing is expensive.** So batch often runs incrementally (e.g., each hour's data at the end of the hour). That brings back stream-like problems — stragglers, windows spanning batch boundaries — and makes the batch layer more complex, defeating its purpose of being simple.

Unifying batch and stream processing

Newer work gets lambda's benefits without its downsides by running **both** historical reprocessing and live event processing **in the same system**. This needs:

Required feature	Why	Example
Replay historical events through the same engine that handles live events	Reprocessing uses the same code path	Log-based brokers can replay messages; some stream processors read input from HDFS
Exactly-once semantics in stream processors	Output equals a fault-free run even if faults happened; like batch, requires discarding partial output of failed tasks	Chapter 11 fault tolerance
Windowing by **event time**, not processing time	Processing time is meaningless when replaying old events	**Apache Beam** API, runnable on **Apache Flink** or **Google Cloud Dataflow**

Unbundling Databases

Viewed abstractly enough, a database, Hadoop and an operating system all do one job: **store data and let you process and query it**. A database stores records in some data model (rows, documents, graph vertices); an OS filesystem stores files. Both are "information management" systems. Chapter 10 already showed Hadoop is a bit like a distributed Unix.

There are practical differences — e.g., many filesystems struggle with a directory holding **10 million small files**, while a database with 10 million small records is perfectly normal. Still, comparing the two is instructive.

Two philosophies (both born in the early 1970s):

	Unix	**Relational databases**
Goal	Give programmers a logical but fairly **low-level** abstraction of the hardware	Give programmers a **high-level** abstraction hiding disk data structures, concurrency, crash recovery
Building blocks	Pipes, and files that are plain byte sequences	SQL and transactions
Why it's "simple"	A thin wrapper over hardware	A short declarative query taps huge machinery (optimizer, indexes, join algorithms, concurrency control, replication) without you knowing the details

The tension between them is decades old and unresolved. The author reads the **NoSQL movement** as applying the Unix-style low-level approach to distributed OLTP storage. This section tries to combine the best of both.

Composing Data Storage Technologies

Features we've seen databases provide:

- **Secondary indexes** — search records efficiently by a field's value (Chapter 3).
- **Materialized views** — a precomputed cache of query results (Chapter 3).
- **Replication logs** — keep copies on other nodes up to date (Chapter 5).
- **Full-text search indexes** — keyword search in text; built into some relational databases (Chapter 3).

Chapters 10 and 11 built the **same kinds of things** outside the database: full-text indexes built by batch jobs, materialized view maintenance in stream processors, and replicating database changes to derived systems with CDC. There's a clear parallel between database-internal features and the derived systems people build with batch and stream processors.

Creating an index

What does `CREATE INDEX` actually do in a relational database?

1. Take a **consistent snapshot** of the table and scan it.
2. Extract the values of the indexed field(s), **sort** them, and write out the index.
3. Process the **backlog** of writes made since the snapshot (assuming the table wasn't locked, so writes kept arriving).
4. From then on, keep the index updated whenever a transaction writes to the table.

This is almost identical to **setting up a new follower replica** (Chapter 5) and to **bootstrapping CDC** with an initial snapshot (Chapter 11). In effect, `CREATE INDEX` **reprocesses the existing dataset** and derives a new view of it. The input may be a snapshot of state rather than a full change log, but those are closely related (state = result of applying the stream of changes).

The meta-database of everything

Seen this way, **all the dataflow in an organisation looks like one giant database**. Every batch, stream or ETL job that moves data from one place and format to another behaves like the database subsystem that keeps indexes and materialized views current.

- Batch and stream processors ≈ elaborate **triggers, stored procedures and view-maintenance routines**.
- The derived systems they maintain ≈ different **index types** (like a relational DB offering B-tree, hash, spatial indexes...).
- The difference: instead of one integrated product, these "features" are separate software, on different machines, run by different teams.

Starting from "no single data model or storage format suits every access pattern", the author sees two ways to compose many tools into a cohesive whole:

	Federated databases: unifying reads	**Unbundled databases: unifying writes**
Idea	One query interface over many storage engines and processing methods (also called a **polystore**)	Reliably plug stores together so every change reaches every place, even with faults
Example	PostgreSQL's **foreign data wrapper**	Change data capture + event logs feeding many stores
Specialised access	Apps can still go straight to underlying engines; users combining data use the federated interface	Each store keeps its own API
Tradition it follows	Relational: one integrated system, high-level language, elegant semantics, complicated implementation	Unix: small tools doing one thing well, a uniform low-level API (pipes), composed by a higher-level language (the shell)
Weakness	Doesn't solve synchronising **writes**	Still missing the "shell" (see below)

"Unbundling" means taking a database's index-maintenance features apart so they can sync writes across **different technologies**.

Making unbundling work

Federation and unbundling are really one goal seen from two angles: building a reliable, scalable, maintainable system from diverse parts. Federated reads need mapping between data models — thoughtful but manageable. **Keeping writes to several stores in sync is the harder problem**, so the author focuses there.

- The traditional answer is distributed transactions across heterogeneous stores — which the author thinks is the **wrong** solution.
- Transactions **inside** one store or one stream processor are fine. For example, some stream processors use distributed transactions internally to get exactly-once semantics, and that can work well.
- But when a transaction must span systems built by **different groups of people** (say, a stream processor writing to a key-value store or search index), there's no standard protocol, so integration is hard.
- An **ordered event log with idempotent consumers** is a much simpler abstraction and far more feasible across heterogeneous systems.

The big win is **loose coupling**, in two senses:

1. **System level.** Asynchronous event streams make the whole robust to outages or slowness of single components. If a consumer is slow or down, the log **buffers** messages; producers and other consumers carry on. The broken consumer catches up after it's fixed and misses nothing.

Synchronous distributed transactions do the opposite: they turn local faults into large-scale failures.

2. **Human level.** Different teams can build, improve and run different components independently, each focusing on one thing, with clean interfaces between teams. Event logs are an interface strong enough to carry fairly strong consistency properties (because events are durable and ordered) yet general enough for almost any data.

Unbundled versus integrated systems

Unbundling won't **replace** databases:

- Stream processors still need databases to keep state.
- Batch and stream outputs still need databases to serve queries.
- Specialised engines still matter — e.g., **MPP data warehouse** query engines are excellent at exploratory analytic queries.

Costs of running many pieces:

- Every piece of software has its own learning curve, configuration issues and operational quirks, so **deploy as few moving parts as possible**.
- A single integrated product may give **better, more predictable performance** on the workloads it targets than a hand-assembled combination.
- Building for scale you don't need wastes effort and can lock you into an inflexible design — a form of premature optimisation (as the Preface says).

The aim of unbundling is **breadth, not depth**: not beating a specialist database on its own workload, but covering a much wider range of workloads than any one product can. **If one product already does everything you need, just use it.** Unbundling only pays off when no single tool meets all your requirements.

What's missing?

Tools for composing systems are improving, but a big piece is absent: the unbundled equivalent of the **Unix shell** — a high-level, declarative language for wiring storage and processing systems together.

The author's wish: to write something like `mysql | elasticsearch`, the unbundled version of `CREATE INDEX`. It would:

1. Take every document in a MySQL database and index it in an Elasticsearch cluster.
2. Continuously capture every later change to MySQL and apply it to the index automatically — no custom glue code.

Here is an illustrative sketch of what such a declaration might look like (this is imaginary syntax, not a real tool):

```
# hypothetical "data shell" - not a real product
postgres://shop/products  |  opensearch://catalog-index
#  left side : source of truth; take a snapshot, then follow its change stream
#  "|"       : keep the right side continuously in sync with the left
#  right side: derived view that can be dropped and rebuilt at any time
```

Similarly, it would be great to **declare caches** as materialized views over complex queries — even recursive graph queries and application logic — and have them precomputed and updated automatically. Early research such as **differential dataflow** works in this direction; the author hopes these ideas reach production.

Designing Applications Around Dataflow

Composing specialised stores and processors with application code has come to be called the **"database inside-out"** approach, after a 2014 conference talk by the author. He stresses it's a design pattern and a starting point for discussion rather than a grand "new architecture". The ideas come from many others:

- dataflow languages such as **Oz** and **Juttle**,
- **Elm**, a functional reactive programming (FRP) language,
- logic programming languages such as **Bloom**,
- the term *unbundling* in this sense comes from **Jay Kreps**.

Spreadsheets are far ahead of most mainstream programming languages at dataflow: put a formula in a cell (say, the sum of a column), and whenever any input changes, the result recalculates automatically. That is exactly what we want from data systems: change a record, and every index, cached view and aggregation that depends on it updates, without you worrying about how. Data systems still have lessons to learn from **VisiCalc (1979)**. The differences: data systems must also be **fault-tolerant, scalable and durable**, and must integrate technologies written by different people over time, reusing existing libraries and services (you can't force everything into one language or framework).

Application code as a derivation function

Every derived dataset comes from some **transformation function**:

Derived dataset	Derivation function
Secondary index	For each row/document, pick the indexed column values and sort by them (B-tree or SSTable, Chapter 3)
Full-text search index	Natural-language processing — language detection, word segmentation, stemming or lemmatization, spelling correction, synonym detection — then build a lookup structure such as an **inverted index**
Machine learning model	Feature extraction and statistical analysis applied to training data; predictions on new input are derived from input + model (and thus indirectly from training data)
Cache	Aggregates data into the shape the UI displays; so the cache logic depends on which fields the UI uses, and UI changes may require redefining and rebuilding the cache

- The secondary-index function is so common that databases build it in: just say `CREATE INDEX`.
- Basic full-text features for common languages may be built in, but advanced ones need domain-specific tuning.
- ML **feature engineering** is notoriously application-specific and needs detailed knowledge of user interaction and deployment.

When the derivation isn't a standard "cookie-cutter" function, you need **custom code**, and that's where databases struggle. Relational DBs offer triggers, stored procedures and user-defined functions to run app code inside the database, but these have been something of an afterthought in database design.

Separation of application code and state

In theory a database could host arbitrary application code, like an OS. In practice databases are a poor fit for modern development needs: managing dependencies and packages, version control, rolling upgrades, evolvability, monitoring and metrics, calls to network services, integration with external systems.

Tools like **Mesos, YARN, Docker and Kubernetes** are built specifically to run application code, and by focusing on that one job they do it far better than a database offering UDF execution as a side

feature.

So it makes sense for some parts of a system to specialise in **durable storage** and others in **running code**, interacting but independent.

Most web apps today run as **stateless services**: any request can go to any server, and the server forgets the request after replying. That's convenient (add/remove servers freely), but the state must live somewhere — usually a database. The trend is to keep application logic out of the database and persistent state out of the application.

> **The joke.** A functional-programming quip is that they support keeping "Church and state" apart. The author spells it out: *Church* means Alonzo Church, the mathematician behind the **lambda calculus**, the foundation of most functional languages. Lambda calculus has no variables that can be overwritten, so (mutable) state really does live apart from Church's work.

In this model, the database is like a **mutable shared variable** you access synchronously over the network; the app reads and updates it, and the database makes it durable and handles some concurrency control and fault tolerance.

But in most languages you **can't subscribe** to a variable's changes; you can only read it periodically. Unlike a spreadsheet, readers aren't told when it changes. (You can hand-code the **observer pattern**, but few languages have it built in.) Databases inherited this passive attitude: to find out if something changed you usually have to **poll** (re-run your query). Change subscriptions are only beginning to appear (Chapter 11, "API support for change streams").

Dataflow: Interplay between state changes and application code

Dataflow thinking renegotiates the relationship between code and state. Instead of a passive database manipulated by the app, think about the **collaboration between state, state changes, and the code that processes them**: code reacts to state changes in one place by producing state changes somewhere else.

Precedents:

- Treating a database's change log as a subscribable event stream (Chapter 11).
- **Actors** and other message-passing systems that respond to events (Chapter 4).
- The **tuple spaces** model of the 1980s: distributed computation as processes that watch state changes and react.
- Inside a database: a trigger firing on a change, or a secondary index updating after a table write.

Unbundling applies this to derived datasets **outside** the primary database — caches, full-text indexes, ML, analytics — using stream processing and messaging.

Important: keeping derived data up to date is a different task from running background jobs asynchronously (what messaging systems were traditionally designed for):

Requirement for derived data	Why	What breaks it
Order of state changes matters	Views derived from the same log must process events in the same order to stay consistent with each other	Many message brokers reorder when redelivering unacknowledged messages; dual writes are ruled out too
Fault tolerance is essential	Losing one message leaves the derived dataset permanently out of sync; both delivery and state updates must be reliable	Many actor systems keep actor state and messages in memory by default, lost on a crash

Stable ordering and fault-tolerant processing are demanding, but **much cheaper and more operationally robust than distributed transactions**. Modern stream processors provide them at scale and let you run

application code as stream operators — code that can do arbitrary processing that built-in database derivation functions can't. Just as Unix tools can be joined with pipes, **stream operators compose**: each takes streams of state changes in and emits streams of state changes out.

Stream processors and services

Today's trendy style splits functionality into **services** talking over **synchronous** network requests (e.g., REST APIs). The main advantage over a monolith is **organisational scalability via loose coupling**: different teams own different services and coordinate less (as long as services deploy independently).

Composing stream operators resembles microservices, but the communication is fundamentally different: **one-way, asynchronous message streams** instead of synchronous request/response.

Besides the fault-tolerance benefits of message passing (Chapter 4), dataflow can also be **faster**. The book's example: a customer buys an item priced in one currency and pays in another, so you need the current exchange rate.

1. **Microservices approach:** the purchase code **queries** an exchange-rate service or database at purchase time.
2. **Dataflow approach:** the purchase code **subscribes in advance** to a stream of exchange-rate updates and writes each new rate into a **local database** as it arrives. At purchase time it only reads the local DB.

The second replaces a synchronous network call with a local query (maybe same machine or even same process). It's **faster** and **more robust** to the other service failing. As the book puts it in spirit: the quickest, most dependable network request is the one you never make. Instead of an RPC we now have a **stream-table join** between purchase events and rate-update events.

> **Footnote.** A microservice could cache the rate locally to avoid the call, but keeping that cache fresh needs periodic polling or subscribing to a change stream — which is just the dataflow approach.

This join is **time-dependent**: if you reprocess purchase events later, the rate will have changed, so to reproduce the original output you need the **historical** rate at purchase time. Whether you query or subscribe, you must handle this (Chapter 11, "Time-dependence of joins").

Subscribing to changes rather than querying current state moves us towards the spreadsheet model: when data changes, everything depending on it updates quickly. Open questions remain (e.g., time-dependent joins), but the author finds dataflow-based applications very promising.

Observing Derived State

The dataflow systems above give you a process for creating and maintaining derived datasets (search indexes, materialized views, predictive models). Call it the **write path**: when information is written, it may pass through several batch/stream stages until every derived dataset reflects it.

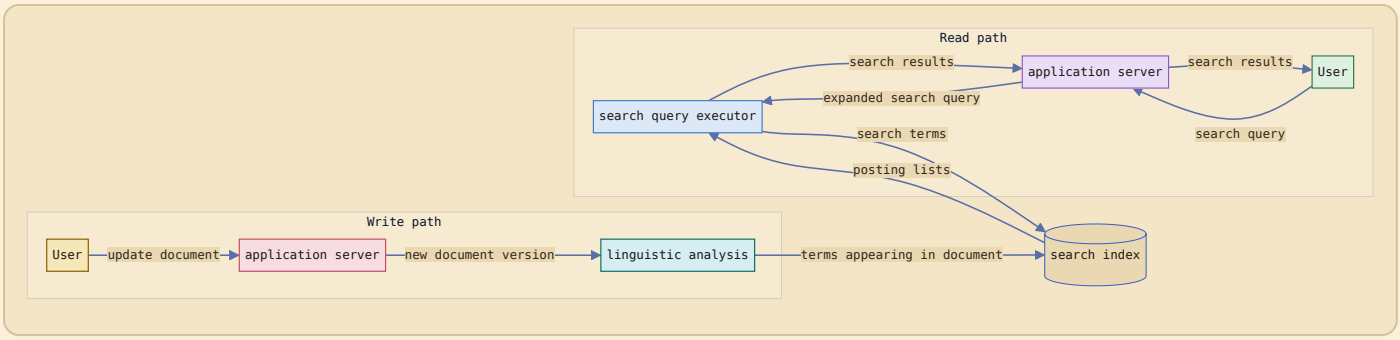
Why build a derived dataset at all? To **query it later**. That's the **read path**: to serve a request you read the derived dataset, maybe process the result further, and build the response.

Together they cover the full journey of data, from collection to consumption (probably by a human):

Write path	Read path
Precomputed; done **eagerly** as soon as data arrives, whether or not anyone asked	Done only when someone asks
Like **eager evaluation** in functional programming	Like **lazy evaluation**

The derived dataset is where the two paths meet, and it embodies a trade-off between work at write time and work at read time.

Figure 12-1 (redrawn): in a search index, writes (document updates) and reads (queries) meet at the index. On the write path a new document version goes through linguistic analysis and its terms go into the index; on the read path a query is expanded, its terms are looked up in the index, and the posting lists come back as results.



Materialized views and caching

Take a full-text search index. Both sides do work:

- **Writes** update index entries for every term in the document.
- **Reads** look up every word in the query and apply Boolean logic: AND (documents containing all words) or OR (any synonym of each word).

Now move the boundary around:

Option	Write-path work	Read-path work	Verdict
No index — scan every document like `grep`	None	Huge for many documents	Fine only for small collections
Normal index	Update entries per term	Look up terms + Boolean logic	The usual balance
Precompute results for every possible query	Infinite — the set of possible queries is infinite	Trivial lookup	Impossible
Precompute results only for the most popular queries, use the index for the rest	Must update these cached results when new matching documents arrive	Fast for common queries	A "cache of common queries", which is really a **materialized view**

> **Footnote nuance.** For a finite corpus, the number of distinct queries with **non-empty** results is finite, but it grows exponentially with the number of terms — still hopeless.

So an index is just one possible boundary. **Caches, indexes and materialized views all shift the boundary**: they do more work on the write path (precomputing) to save work on the read path.

This is exactly the **Twitter** home-timeline example from Chapter 1 ("Describing Load"), where the write/read boundary was drawn differently for celebrities than for ordinary users. The author notes that after some 500 pages the book has come full circle.

Stateful, offline-capable clients

Two decades of web apps have made us assume the **client/server model** — mostly stateless clients, servers owning the data — so strongly that we forget alternatives. It's worth questioning.

- Traditionally a browser was a stateless client, useless offline (you could basically just scroll a page loaded while online).
- Modern **single-page JavaScript apps** have lots of state: client-side UI interaction and persistent local storage in the browser.

- **Mobile apps** keep much state on the device and don't need a server round-trip for most interactions.

This revives **offline-first** apps: do as much as possible against a **local database on the device**, work without internet, and **sync in the background** when connected. Since mobile networks are often slow and unreliable, not blocking the UI on network calls and working offline is a big user benefit (Chapter 5, "Clients with offline operation").

Once state lives on devices, a useful mental model appears: **on-device state is a cache of server state**. The pixels on screen are a materialized view of the client's model objects; those model objects in turn replicate, locally, state held in a far-away datacenter.

Pushing state changes to clients

On a typical web page, if server data changes after you load the page, your browser doesn't know until you reload. It reads once and assumes the data is static — no subscription. So device state is a **stale cache** unless you poll. (RSS and similar HTTP feed subscriptions are basically polling.)

Newer protocols go beyond HTTP request/response:

- **Server-sent events** (the **EventSource API**) and
- **WebSockets**

both keep a TCP connection open so the server can **push** messages to the browser while it stays connected. The server can tell the client about changes to state the client has cached, reducing staleness.

In write-path/read-path terms, pushing changes to devices **stretches the write path right up to the end user's device**:

1. When a client starts, it uses the **read path** once to fetch initial state.
2. After that, it relies on a **stream of state changes** from the server.

Stream processing and messaging ideas therefore needn't stay in the datacenter; they can reach end-user devices.

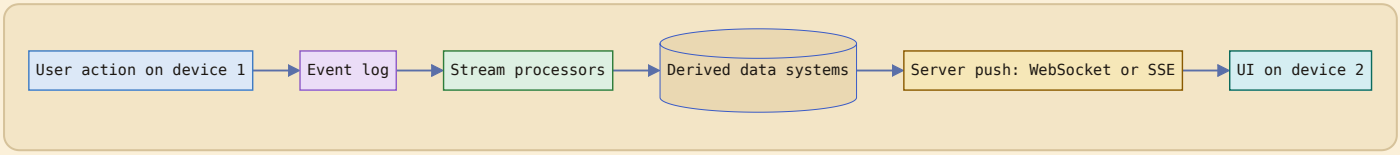
What about offline devices? Already solved: in Chapter 11 ("Consumer offsets"), a consumer of a log-based broker reconnects and resumes from its offset, missing nothing. Each device is just a small subscriber to a small event stream.

End-to-end event streams

Tools for stateful clients and UIs — the **Elm** language and Facebook's **React, Flux and Redux** — already handle client state by consuming an event stream (user input, server responses), much like event sourcing.

A natural extension is for the **server to feed its own state-change events into that client pipeline** as well. Then state changes flow along an **end-to-end write path**:

Illustration (not a book figure): the end-to-end write path from one user's device to another's screen.



Changes could travel quickly — e.g., in **less than one second** from end to end. Instant messaging and online games already work like this ("real-time" meaning low delay, not hard response-time guarantees as in Chapter 8).

Why not every app? Because stateless clients and request/response are deeply baked into databases, libraries, frameworks and protocols. Many datastores offer "one request, one response"; far fewer

offer "one request, a **stream** of responses over time" (subscriptions). Extending the write path to users means moving from **request/response** to **publish/subscribe** dataflow. The author believes the payoff — more responsive UIs, better offline support — is worth it, and urges designers to keep **subscribing to changes** in mind, not just querying current state.

Reads are events too

When a stream processor writes derived data into a store and user requests query that store, the store is **where the write path hands over to the read path**; it permits random-access lookups that would otherwise mean reading the entire event log.

Often the store is separate from the streaming system. But stream processors keep state for aggregations and joins anyway, normally hidden; some frameworks let outside clients **query that state**, making the stream processor a simple database.

The author goes further. Usually writes flow via the event log, whereas reads are short-lived network requests sent directly to the storage nodes. Alternatively:

1. Represent **read requests as events** too.
2. Send both read events and write events through the **same stream processor**.
3. The processor answers a read event by emitting the result on an **output stream**.

When both reads and writes are events routed to the same operator, you're really performing a **stream-table join**: query stream on one side, database on the other. The read event must go to the partition holding the data (Chapter 6, "Request Routing"), just as batch and stream joins must co-partition inputs by key (Chapter 10, reduce-side joins).

This link between serving requests and joins is fundamental:

- A **one-off read** passes through the join operator and is immediately forgotten.
- A **subscription** is a **long-lived join** against both earlier and later events.

Benefit: causality and provenance. Logging reads lets you reconstruct **what the user saw before deciding**. E.g., in an online shop, the predicted shipping date and stock status shown probably influence whether the customer buys; to analyse that, you must record the result of the customer's shipping/inventory query. This helps capture causal dependencies (see "Ordering events to capture causality" above) but costs extra storage and I/O; optimising that is an open research problem. If you already log reads for operations, making that log the **source** of the requests is a small step.

Multi-partition data processing

For single-partition queries, routing through streams is probably overkill. But the idea enables **distributed execution of complex queries** that combine data from many partitions, reusing the stream processor's routing, partitioning and joining infrastructure.

- **Storm's distributed RPC** supports this. It's been used to compute **how many people have seen a URL on Twitter** — the union of the follower sets of everyone who tweeted that URL. Since Twitter's users are partitioned, this combines results from many partitions.
- **Fraud prevention:** to score a purchase's risk, look up reputation scores for the user's IP address, email, billing address, shipping address, etc. Each reputation database is partitioned differently, so answering takes a **chain of joins** against datasets partitioned in different ways.

MPP databases' internal query execution graphs look similar. If you need multi-partition joins, a database with that built-in feature is probably the **simpler choice**; but treating queries as streams is an option for large-scale apps that outgrow off-the-shelf solutions.

Aiming for Correctness

Stateless vs stateful. If a stateless, read-only service misbehaves, you fix the bug, restart it, and all is well. Databases are different: they're built to remember things (more or less) forever, so a mistake's effects may **also last forever**. Stateful systems deserve more careful thought.

We want applications that are **reliable and correct** — programs whose behaviour is well defined and understood even when faults occur. For roughly **four decades**, the go-to tools here have been the transaction guarantees of **atomicity, isolation and durability** (Chapter 7). But:

- Those foundations are **weaker than they look** — consider the confusion around weak isolation levels (Chapter 7).
- Some areas are **abandoning transactions** for models with better performance and scalability but messier semantics (e.g., leaderless replication, Chapter 5).
- **Consistency** is talked about a lot but poorly defined (Chapter 7, Chapter 9). Some people say "embrace weak consistency" for availability without a clear idea of what that means in practice.

Our understanding and engineering methods are surprisingly **flaky**. It's very hard to tell whether an app is safe at a given isolation level or replication configuration. Simple solutions often seem fine at low concurrency without faults, then reveal subtle bugs under stress.

- **Kyle Kingsbury's Jepsen** experiments exposed stark gaps between products' **claimed** safety guarantees and their **actual** behaviour under network problems and crashes.
- Even if the infrastructure were flawless, application code must still **use it correctly**, which is error-prone when configuration is hard to understand (weak isolation levels, quorum settings, etc.).

If your app...	Then...
Can tolerate occasionally losing or corrupting data unpredictably	Life is easy; you might get away with hoping for the best
Needs stronger correctness	**Serializability** and **atomic commit** are the established tools, but they usually work only **within a single datacenter** (ruling out geo-distribution) and **limit scale and fault tolerance**

The author doesn't think traditional transactions are going away, but he doesn't think they're the **final word** either. This section explores correctness in **dataflow architectures**.

The End-to-End Argument for Databases

Using a data system with strong safety properties (e.g., serializable transactions) does **not** guarantee that your application is free of data loss or corruption. If the app has a bug that writes wrong data or deletes data, serializability won't help.

That might sound trivial, but application bugs and human mistakes happen. This was the argument (Chapter 11, "State, Streams, and Immutability") for **immutable, append-only data**: if faulty code can't destroy good data, recovery is easier. Immutability helps but isn't a cure-all; here's a subtler corruption case.

Exactly-once execution of an operation

Recall **exactly-once** (or **effectively-once**) semantics (Chapter 11). When processing a message fails, you can either:

- **give up** (drop the message → data loss), or
- **retry** — but maybe it actually succeeded the first time and you just never heard back, so it gets processed **twice**.

Processing twice is also corruption: charging a customer twice (overbilling), or incrementing a counter twice (inflating a metric). Exactly-once means arranging the computation so the **final effect equals a fault-free run**, even if retries happened.

The most effective approach: make the operation **idempotent** — same effect whether run once or many times. Turning a non-idempotent operation into an idempotent one takes care: you may need extra metadata (e.g., the set of operation IDs that already updated a value), and **fencing** when failing over between nodes (Chapter 8, "The leader and the lock").

Duplicate suppression

Duplicate suppression appears in many places beyond stream processing.

TCP numbers packets with sequence numbers so the receiver can put them in order and detect lost or duplicated packets. Lost ones are retransmitted and duplicates dropped by the TCP stack before the app sees the data.

But that only works within one TCP connection. Many databases tie a transaction to a client connection (queries sent on the same connection belong to the same transaction). Suppose the client sends `COMMIT`, then the network drops and the connection times out before the reply arrives. The client **doesn't know** whether the transaction committed or aborted (Chapter 8, Figure 8-1).

Our own version of the book's Example 12-1 (a non-idempotent money transfer; the book moves $11.00 between accounts 4321 and 1234 — here we use different values to illustrate):

```
BEGIN TRANSACTION;                                          -- start one atomic unit of work
UPDATE wallets SET funds = funds + 25.00 WHERE wallet_id = 77;  -- credit the receiving wallet
UPDATE wallets SET funds = funds - 25.00 WHERE wallet_id = 88;  -- debit the paying wallet
COMMIT;                                                     -- make both changes permanent together
```

- Line 1 starts a transaction so both updates succeed or fail together.
- Lines 2–3 add to one balance and subtract from another. "Add 25" is **not idempotent**: running it twice adds 50.
- Line 4 commits. If the reply to `COMMIT` is lost and the client reconnects and retries, TCP can't recognise the retry (it's a new connection). In the book's example this means **$22 could move instead of the intended $11**.

So even though this is the textbook example of **atomicity**, it's actually **not correct**, and **real banks don't work this way**.

Does two-phase commit help? 2PC breaks the one-connection-one-transaction link, because the coordinator must be able to reconnect after a network fault and instruct the database whether an in-doubt transaction should commit or abort. Is that enough to guarantee once-only execution? **No.**

Even if duplicates are suppressed between the database client and server, there's still the network between the **end user's device** and the **application server**:

1. A user on a weak mobile connection submits a form via HTTP POST.
2. The POST arrives, but the signal fades before the response comes back.
3. The user sees an error and retries. The browser warns that the form will be resubmitted and asks for confirmation; the user agrees, because they do want it done.
4. (The **Post/Redirect/Get** pattern avoids that warning in normal use, but doesn't help when the POST times out.)
5. To the web server the retry is a **new request**; to the database it's a **new transaction**. The usual deduplication mechanisms don't catch it.

Operation identifiers

To make an operation idempotent across **several network hops**, relying on the database's transactions isn't enough; you have to think about the request's **whole journey, end to end**.

1. Generate a **unique operation ID** (e.g., a **UUID**) in the client — say as a hidden form field — or compute a **hash of all relevant form fields**.
2. If the browser submits the POST twice, both carry the **same ID**.
3. Pass that ID all the way to the database and make sure only **one** operation per ID ever executes.

Our own version of the book's Example 12-2:

```
ALTER TABLE transfer_log ADD UNIQUE (op_id);   -- the database will reject a second row with the same op_id

BEGIN TRANSACTION;
INSERT INTO transfer_log (op_id, payer, payee, amount)
  VALUES ('9b1f2c4e-5a6d-4e7f-8a90-b1c2d3e4f501', 88, 77, 25.00);  -- record the request; fails if op_id already exists
UPDATE wallets SET funds = funds + 25.00 WHERE wallet_id = 77;     -- credit
UPDATE wallets SET funds = funds - 25.00 WHERE wallet_id = 88;     -- debit
COMMIT;
```

- The `UNIQUE` constraint on the ID column is the key: if a retry attempts to add an ID that is already present, the `INSERT` is rejected and the entire transaction **aborts**, so it can't take effect twice.
- Relational databases usually uphold a uniqueness constraint properly **even under weak isolation**, whereas doing "look first, then insert" in application code can break under non-serializable isolation (Chapter 7, write skew and phantoms).
- Bonus: the requests table is effectively an **event log**, pointing towards event sourcing. The balance updates don't even need to be in the same transaction — they are redundant, so a downstream consumer could **derive** them from the request event, provided the event is processed exactly once (again enforceable with the request ID).

The end-to-end argument

Duplicate suppression is one instance of a general principle, the **end-to-end argument**, set out by **Saltzer, Reed and Clark in 1984**. In paraphrase: *a function can only be implemented completely and correctly if the applications at the two ends of the communication take part; so the communication system itself cannot provide it completely. A partial version inside the communication system can still be useful to improve performance.*

Applied to our example:

- TCP suppresses duplicate **packets** within a connection.
- Some stream processors offer exactly-once at the **message-processing** level.
- Neither stops a **user** resubmitting after a timeout.
- TCP, database transactions and stream processors **by themselves** can't rule these duplicates out. You need an **end-to-end** solution: a transaction ID passed from the end-user client all the way to the database.

The same argument covers:

Concern	Low-level mechanism	What it misses	End-to-end fix
Data integrity	Checksums in **Ethernet, TCP and TLS** catch corrupted packets	Bugs in sending/receiving software; corruption on disk	**End-to-end checksums**
Encryption	Home **WiFi password** stops snooping on your WiFi	Attackers elsewhere on the internet	**End-to-end encryption and authentication**
Encryption	**TLS/SSL** between client and server stops network attackers	A compromised server	End-to-end encryption and authentication

The lower-level mechanisms (WiFi encryption, Ethernet checksums, TCP's dedup) remain **useful** — they make higher-level problems less likely. HTTP would often get garbled without TCP reordering packets. They're just **not sufficient** for end-to-end correctness.

Applying end-to-end thinking in data systems

Back to the thesis: strong safety properties in the database don't make the application immune to loss or corruption. **The application must take end-to-end measures** such as duplicate suppression.

That's a pity, since fault-tolerance machinery is tricky to build correctly. The low-level ones (like TCP's) work well, so the remaining high-level faults are fairly rare. It would be great to wrap the remaining high-level machinery in an **abstraction** so app code needn't worry — but the author fears we **haven't found that abstraction yet**.

- **Transactions** have long been the favourite abstraction and are useful. They squash many possible problems — crashes, disk failures, concurrent writes, broken constraints, network interruptions — into just **two outcomes: commit or abort**. A huge simplification, but not enough.
- Transactions are **expensive**, especially across **heterogeneous** storage technologies (Chapter 9).
- When we refuse distributed transactions as too costly, we end up **reimplementing fault tolerance in application code**. Since concurrency and partial failure are hard and counterintuitive, the author suspects most such app-level mechanisms **don't work correctly**, leading to lost or corrupted data.

So it's worth exploring fault-tolerance abstractions that make **application-specific end-to-end correctness** easy while also performing and operating well in large distributed environments.

Enforcing Constraints

Now think about correctness within the **unbundled database** idea. We've seen that a request ID travelling from the client to the database that stores the write gives end-to-end dedup. Can **other constraints** be handled too?

Focus on **uniqueness constraints** (like the one Example 12-2 relied on). Examples from Chapter 9:

- a username or email address must identify exactly one user,
- a file storage service can't have two files with the same name,
- a theatre or flight seat can't be sold to two people.

Similar constraints: an account balance must never go negative; don't sell more items than are in the warehouse; a meeting room can't have overlapping bookings. Techniques for uniqueness often work for these too.

Uniqueness constraints require consensus

From Chapter 9: in a distributed setting, enforcing uniqueness **requires consensus**. If several concurrent requests want the same value, the system must pick which one wins and reject the others.

- **Most common method:** make one node the **leader** and let it decide everything. That's OK provided you accept routing every request via one node (even for distant clients) and that node never dies. To tolerate leader failure, you're back to the consensus problem.
- **Scaling out:** partition by the value that must be unique. For request-ID uniqueness (Example 12-2), route all requests with the same ID to the same partition. For unique usernames, partition by **hash of username**.
- **Ruled out: asynchronous multi-master replication** — different masters could concurrently accept conflicting writes, breaking uniqueness.
- If you want to **immediately reject** violating writes, **synchronous coordination is unavoidable**.

Uniqueness in log-based messaging

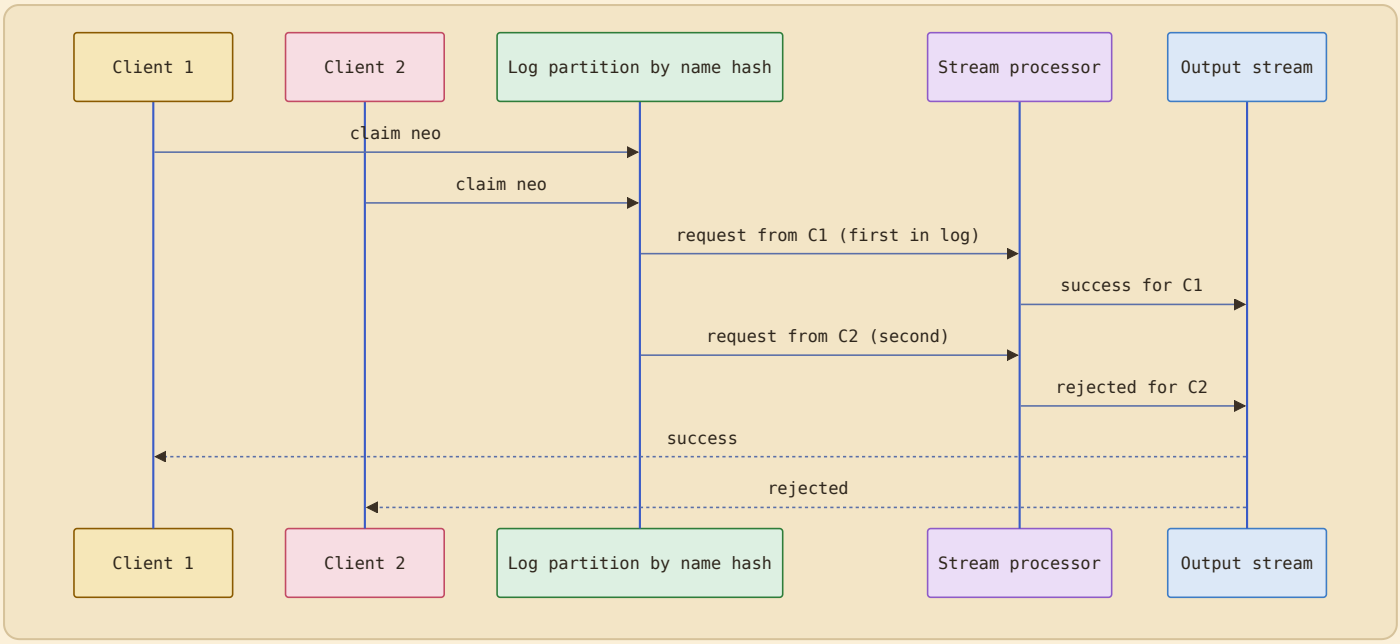
A log ensures every consumer receives messages in an **identical order** — formally **total order broadcast**, equivalent to consensus (Chapter 9). With log-based messaging in the unbundled approach,

we can enforce uniqueness similarly.

A stream processor consumes a log partition **sequentially on a single thread**. So if the log is partitioned by the value that must be unique, the processor can decide deterministically and unambiguously which conflicting operation came **first**. The book's example — several users claiming the **same username**:

1. Every username request becomes a **message** appended to a partition chosen by the **hash of the username**.
2. A stream processor reads requests in order, using a **local database** to track taken names. If the name is free, it records it as claimed and writes a **success** message to an output stream; if taken, it emits a **rejection** message.
3. The requesting client **watches the output stream** for the success/rejection matching its request.

Illustration (not a book figure): two clients race for the same username; the log order decides the winner.



This is essentially the "implementing linearizable storage using total order broadcast" algorithm from Chapter 9. It **scales** to high throughput by adding partitions, since each partition is processed independently.

It works for many constraints, not just uniqueness. The core principle: **route any writes that might conflict to the same partition and process them sequentially**. What counts as a conflict depends on the application (Chapter 5 "What is a conflict?", Chapter 7 write skew), but the stream processor can run **arbitrary validation logic**. **Bayou** pioneered a similar approach back in the **1990s**.

Multi-partition request processing

It gets trickier with several partitions. Example 12-2 may touch **three** of them: one holding the **request ID**, the one holding the **payee** account, and the one holding the **payer** account — independent things with no reason to be colocated.

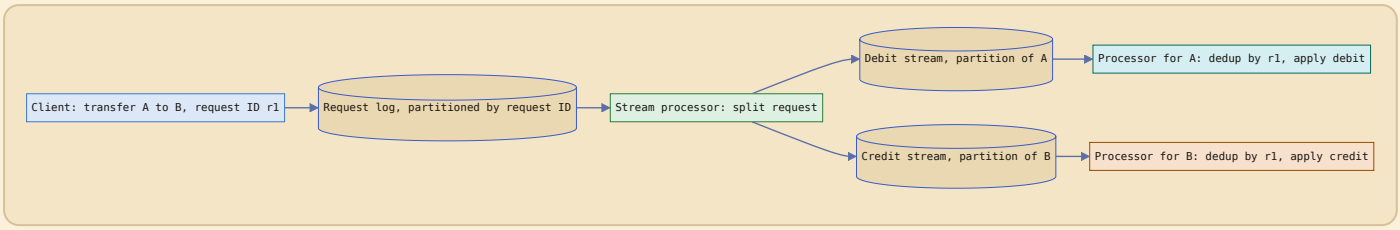
Traditional approach: an **atomic commit across all three**, which forces the transaction into a total order with everything else on those partitions. With cross-partition coordination, partitions can't be processed independently, so **throughput suffers**.

Log-based approach, same correctness, no atomic commit:

1. The client gives the transfer (account A → account B) a **unique request ID** and appends it to a log partition chosen by **request ID**.

2. A stream processor reads the request log. For each request it emits two messages: a **debit instruction** for payer A (partitioned by A) and a **credit instruction** for payee B (partitioned by B). Both carry the original request ID.
3. Further processors consume the credit and debit streams, **drop duplicates using the request ID**, and update the balances.

Illustration (not a book figure): a money transfer split into two differently partitioned stages, no atomic commit needed.



Why steps 1 and 2? If the client sent debit and credit **directly**, making sure both-or-neither happen would need an atomic commit across two partitions. Instead we **durably log the request as one message** and derive both instructions from it. **Single-object writes are atomic** in almost every data system, so the request is either in the log or not — no multi-partition atomic commit needed.

Crash handling: if the step-2 processor crashes, it resumes from its last **checkpoint**. It won't skip requests, but may reprocess some and emit duplicate credit/debit instructions. Being **deterministic**, it emits the **same** instructions, and the step-3 processors discard them thanks to the end-to-end request ID.

Preventing overdraft: add another stream processor, partitioned by the **payer's account number**, that tracks balances and validates transfers; only valid transactions are placed in the step-1 request log.

Result: breaking the multi-partition transaction into **two stages, each partitioned differently**, plus the end-to-end request ID achieves the same correctness (each request applied **exactly once** to both payer and payee), even with faults, **without an atomic commit protocol**. This parallels "Multi-partition data processing" above (and concurrency control in event sourcing, Chapter 11).

Timeliness and Integrity

Transactions are usually **linearizable**: the writer waits for commit, and from then on **every reader sees** its writes.

Not so when an operation is unbundled across several stream-processing stages: log consumers are **asynchronous by design**, so the sender doesn't wait for processing. But a client **can** choose to wait for a message on an output stream — as with the username check above.

In that example, the **correctness of the check doesn't depend on whether the sender waits**. Waiting just tells the sender synchronously whether it worked; that notification is **separable** from what processing the message actually does.

The author argues the word **consistency** mixes two different requirements:

	Timeliness	Integrity
Meaning	What users see reflects the **latest** state	**Absence of corruption**: no data loss, no contradictory or false data; derived datasets correctly reflect their source
Example of violation	Reading a stale replica shows an inconsistent state (Chapter 5, replication lag)	An index missing some records (not very useful)
Nature of violation	**Temporary** — waiting and retrying fixes it	**Permanent** — waiting won't fix corruption; needs explicit checking and repair
Related concepts	CAP's "consistency" = **linearizability**, a strong form of timeliness; weaker forms like **read-after-write** consistency can also be useful	ACID "consistency" usually means an app-specific idea of integrity; **atomicity and durability** are the main tools that protect it
Slogan	Violations = **"eventual consistency"**	Violations = **"perpetual inconsistency"**

Integrity usually matters much more than timeliness. Timeliness violations are annoying and confusing; integrity violations can be **catastrophic**.

Credit card example:

- A purchase from the **last 24 hours** not yet showing on your statement is normal — these systems lag; banks reconcile and settle **asynchronously**; timeliness isn't critical.
- It would be really bad, though, if the new balance **didn't match the previous balance plus the listed transactions** (a sum error), or if you were **charged but the merchant never got paid** (money vanishing). Those are **integrity** violations.

Correctness of dataflow systems

ACID transactions normally provide **both**: timeliness (through linearizability) and integrity (through atomic commit), so from an ACID viewpoint the distinction hardly matters.

Event-based dataflow systems decouple them:

- **No timeliness guarantee** when processing streams asynchronously — unless you deliberately build consumers that wait for a message before returning.
- **Integrity is central.** Exactly-once / effectively-once semantics exist to preserve integrity: a lost event or an event applied twice could violate it. So **reliable, fault-tolerant delivery** plus **dropping duplicates** (e.g., via idempotent operations) are essential.

Reliable stream processing can preserve integrity **without distributed transactions or atomic commit**, potentially achieving comparable correctness with much better performance and operational robustness. The recipe:

1. Represent the **write operation as a single message**, easily written atomically (fits event sourcing).
2. **Derive all other state updates** from that message using **deterministic derivation functions**, similar to stored procedures (Chapter 7, "Actual Serial Execution").
3. Pass a **client-generated request ID** through every level of processing for end-to-end duplicate suppression and idempotence.
4. Make messages **immutable** and allow derived data to be **recomputed** periodically, so bugs are easier to recover from.

The author sees this combination as a very promising direction for fault-tolerant applications.

Loosely interpreted constraints

A **traditional** uniqueness constraint requires consensus, typically by funnelling a partition's events through a single node. That's unavoidable if you want the traditional form, and stream processing

can't dodge it.

But many real applications can live with **weaker uniqueness**:

Situation	How it's handled after the fact
Two people grab the same username, or the same seat, at the same moment	Send one an apology and ask them to choose another. Such a corrective change is a **compensating transaction**
Customers order more items than are in the warehouse	Order more stock, apologise for the delay, offer a discount. You'd need the very same process if a **forklift truck destroyed** part of the stock — so the apology process must exist anyway, and a linearizable stock constraint may be unnecessary
Airlines **overbook** flights expecting no-shows; hotels overbook rooms expecting cancellations	"One person per seat" is **deliberately** broken for business reasons; compensation (refunds, upgrades, a free room at a neighbouring hotel) handles excess demand. Even without overbooking, apologies are needed for cancellations due to **bad weather or strikes**
Someone withdraws more money than they have	The bank charges an **overdraft fee** and asks for repayment; a **daily withdrawal limit** bounds the bank's risk

Many businesses find it fine to **break a constraint for a while and repair it afterwards with an apology**. The apology's cost (money, reputation) varies but is often low:

- A sent email can't be recalled, but a correction email can follow it.
- If you charge a card twice, refund one charge; the cost is processing fees and maybe a complaint.
- Cash paid out of an **ATM** can't be directly recovered, though if the account went overdrawn and the customer refuses to repay, debt collectors could in principle be sent.

Deciding whether that apology cost is tolerable is a **business call**. If it is, checking every constraint **before** writing is unnecessarily strict, and a linearizable constraint isn't needed. It can be reasonable to **write optimistically and check afterwards**. You can still validate **before** doing things that are expensive to undo — just not necessarily before writing the data.

These apps **do need integrity** (don't lose a reservation; don't let money vanish through mismatched credits and debits) but **don't need timeliness** in enforcing the constraint. This resembles conflict resolution in Chapter 5 ("Handling Write Conflicts").

Coordination-avoiding data systems

Two observations:

1. Dataflow systems can guarantee integrity of derived data **with no atomic commit, no linearizability and no synchronous coordination across partitions**.
2. Strict uniqueness constraints need timeliness and coordination, but many applications are fine with **loose constraints** that may be temporarily violated and fixed later, **as long as integrity holds throughout**.

Together: dataflow systems can manage data for many applications **without coordination** while still giving **strong integrity**. Such **coordination-avoiding** systems can outperform and be more fault-tolerant than systems needing synchronous coordination.

- Example: a system spread over **multiple datacenters** in a **multi-leader** setup, replicating **asynchronously** between regions. Each datacenter runs independently with no synchronous cross-region coordination. Timeliness is weak (it can't be linearizable without coordination), but integrity can be strong.
- **Serializable transactions** are still useful for maintaining derived state, but only **within a limited scope**, where they perform well. Heterogeneous distributed transactions like **XA** are **not required**.

- Synchronous coordination can still be added **where needed** (e.g., strict constraints before an irreversible operation), but not everything has to pay for it if only a small part of the app needs it.

The apology trade-off: coordination and constraints **reduce apologies due to inconsistency**, but may also reduce performance and availability, **increasing apologies due to outages**. You can't get apologies to zero; aim for the balance point where both inconsistencies and outages are kept acceptably rare.

Trust, but Verify

All our reasoning assumes some things can go wrong and others can't. Those assumptions are our **system model** (Chapter 8). We assume processes can crash, machines can lose power, and networks can delay or drop messages arbitrarily. We might also assume data isn't lost after **fsync**, memory isn't corrupted, and the CPU's **multiplication instruction** always gives the right answer.

These are reasonable (true most of the time) — it would be paralysing to constantly doubt the computer. Traditional system models are **binary** ("can happen" / "never happens"), but reality is **probabilistic**. The real question: do violations happen **often enough to meet in practice**?

- Data can get corrupted while sitting untouched on disk (Chapter 7).
- Network corruption can occasionally **slip past TCP checksums** (Chapter 8).

Anecdote: an application the author worked on collected crash reports from clients, and a few reports made sense only as **random bit-flips in the devices' memory**. Unlikely per device, but with enough devices, unlikely things happen.

- Apart from random corruption caused by **radiation** or faulty hardware, some pathological patterns of memory access can flip bits **even when the memory itself is healthy** — the **rowhammer** effect — which can be exploited to break OS security.
- Hardware is not quite the perfect abstraction it appears.
- Important caveat: on modern hardware such random bit-flips remain **very rare**; they're just not impossible, so they deserve attention.

Maintaining integrity in the face of software bugs

Checksums at the network, memory or filesystem level can't catch software bugs. Even popular databases contain bugs; the author has personally seen:

- **MySQL** not upholding a **uniqueness constraint** correctly, and
- **PostgreSQL's serializable isolation** level exhibiting **write skew** anomalies,

even though both are robust, respected, battle-tested databases. Less mature software is likely much worse. Despite careful design, testing and review, bugs creep in; they're rare and eventually fixed, but can corrupt data in the meantime.

Application code likely has far more bugs, since it gets much less review and testing than database code. Many apps don't even correctly use the database's integrity features, such as **foreign key** or **uniqueness constraints**.

ACID consistency assumes the database starts consistent and each transaction moves it from one consistent state to another. That only holds if transactions are **bug-free**. If the app misuses the database (e.g., uses a weak isolation level unsafely), integrity can't be guaranteed.

Don't just blindly trust what they promise

Since hardware and software fall short of the ideal, corruption seems **inevitable sooner or later**. So we need a way to **detect** it, fix it, and trace its cause. Checking data integrity is called **auditing**.

- Auditing isn't only for finance (Chapter 11, "Advantages of immutable events"), but it's crucial in finance precisely because everyone knows mistakes happen and must be detectable and fixable.

- Mature systems consider unlikely failures and manage the risk. Large-scale storage like **HDFS** and **Amazon S3 don't fully trust disks**: background processes continually **read back files, compare with other replicas, and move files between disks** to reduce the risk of silent corruption.
- To be sure your data is still there, **actually read it and check**. Usually it's fine; if not, find out early.
- Likewise, **test restoring from backups** periodically, or you may discover a broken backup only after data is lost.

A culture of verification

HDFS and S3 still assume disks **usually** work — reasonable, but different from assuming they **always** work. Few systems yet have this **"trust, but verify"** attitude of continual self-auditing; many treat correctness guarantees as absolute and make no provision for rare corruption. The author hopes for more **self-validating / self-auditing** systems.

He fears the **ACID culture** encouraged building apps on **blind trust** in technology (like transactions), neglecting auditability; since the trusted tech worked well enough, auditing wasn't seen as worth it.

Then the landscape changed: under the **NoSQL** banner, **weaker consistency** became normal and **less mature storage** became widespread. But audit mechanisms hadn't been developed, so apps kept being built on blind trust — now more dangerous.

Designing for auditability

If a transaction mutates several objects, it's hard to tell afterwards **what it meant**. Even with captured transaction logs (CDC), inserts/updates/deletes across tables don't show **why** they happened. The application logic invocation that decided them is **transient** and can't be reproduced.

Event-based systems are more auditable:

- In event sourcing, user input is a **single immutable event**; all state updates are **derived** from it.
- The derivation can be **deterministic and repeatable**: the same events through the same version of the code give the same state.
- Explicit dataflow (Chapter 10, "Philosophy of batch process outputs") makes data **provenance** clear, so integrity checks become feasible:
  - For the event log, use **hashes** to check storage isn't corrupted.
  - For derived state, **rerun** the batch/stream processors and compare the outputs, or even run a **second, redundant derivation side by side**.
- A deterministic, clearly defined dataflow also helps you **debug and trace** the reasons behind the system's behaviour — you can recreate precisely the conditions that led to a surprise, which amounts to **time-travel debugging**.

The end-to-end argument again

If we can't fully trust every component — every piece of hardware fault-free, every piece of software bug-free — we must **periodically check integrity**. Otherwise we only notice corruption after it's caused downstream damage, when it's harder and costlier to trace.

Integrity checks work best **end to end**: the more systems included in a check, the fewer places corruption can slip by unnoticed. If an entire derived data pipeline is verified end to end, all the disks, networks, services and algorithms along the way are **implicitly checked**.

Continuous end-to-end integrity checks give **confidence**, which lets you **move faster**. Like automated tests, auditing catches bugs quickly and lowers the risk that a change or a new storage technology causes damage. If you're not afraid of change, you can evolve the application much better.

Tools for auditable data systems

Right now, auditability is a **first-class concern** in only a few data systems.

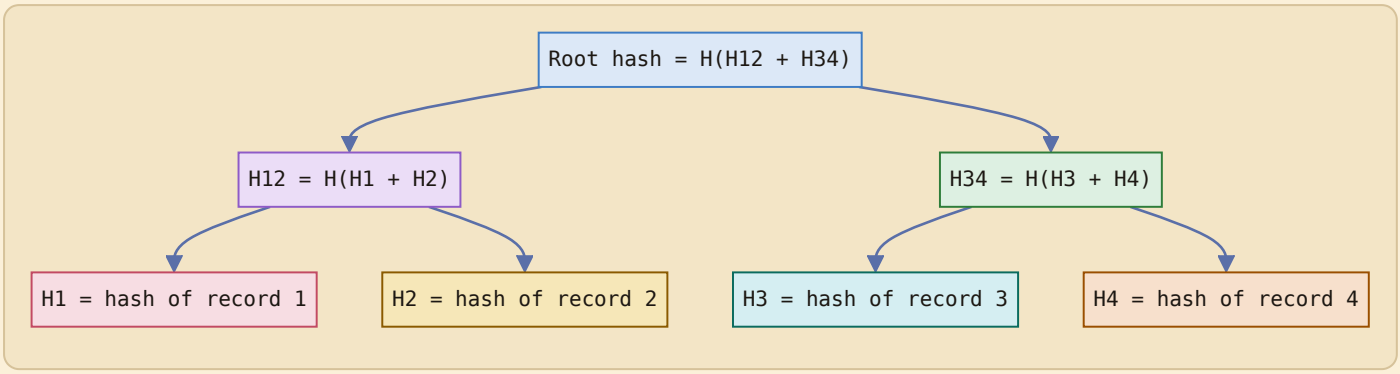
- Some apps build their own audit mechanisms, e.g., writing every change into a separate **audit table** — yet it remains hard to be sure both that table and the database itself are intact.
- Periodically signing a transaction log with a **hardware security module** makes it **tamper-proof**, yet that says nothing about whether the **correct** transactions were written into it.

An intriguing idea is using **cryptographic tools** to demonstrate system integrity robustly against many hardware and software issues, even **malicious** actions. **Cryptocurrencies, blockchains and distributed ledger technologies** — **Bitcoin, Ethereum, Ripple, Stellar** and others — explore this.

- The author says he isn't qualified to judge them as currencies or contract mechanisms.
- Seen as data systems, they're **distributed databases** (each with its own data model and transaction mechanism) whose replicas are hosted by **organisations that don't trust each other**; the replicas constantly verify one another and run a **consensus protocol** to settle which transactions to execute.
- He's **somewhat skeptical** of their **Byzantine fault tolerance** aspects (Chapter 8) and regards **proof of work** (as in Bitcoin mining) as **hugely wasteful**. Bitcoin's transaction throughput is **rather low**, though more for political and economic reasons than technical ones.
- The **integrity-checking** aspects are the interesting part.

Merkle trees underpin much cryptographic auditing: **trees of hashes** that can efficiently prove a record is in a dataset (and a few other things). Outside the crypto hype, **certificate transparency** uses Merkle trees to check the validity of **TLS/SSL certificates**.

Illustration (not a book figure): a Merkle tree over four records. To prove record 3 is included you only need H4 and H12 to recompute the root.



The author imagines such integrity-checking and auditing algorithms (as in certificate transparency and distributed ledgers) spreading to data systems generally. Effort is still needed so they **scale** as well as non-cryptographic systems and so their **performance cost** stays small — an area to watch.

Doing the Right Thing

In the book's final section the author steps back. The whole book compared architectures and techniques for reliable, scalable, maintainable systems, but left out something fundamental, which he now addresses: **ethics**.

- Every system has a **purpose**, and every action has **intended and unintended consequences**. The purpose may just be making money, but effects on the world can go far beyond it. Engineers are **responsible** for thinking those consequences through and consciously choosing what kind of world we want.
- We talk about data abstractly, but many datasets are **about people** — their behaviour, interests and identity. Such data must be handled with **humanity and respect**. Users are human, and **human dignity comes first**.

- Software development increasingly involves **important ethical choices**. Guidelines exist — e.g., the ACM's **Software Engineering Code of Ethics and Professional Practice** — but are rarely discussed, applied or enforced. So engineers and product managers are sometimes **very cavalier** about privacy and harmful consequences.
- **A technology is neither good nor evil by itself**; its use and its effect on people are what count. That's as true of a search engine as of a weapon such as a gun. It's not enough for engineers to care only about the tech while ignoring its effects; we **share** the ethical responsibility. Ethics is hard to reason about, but too important to skip.

Predictive Analytics

Predictive analytics is a big part of the "Big Data" hype. Forecasting the **weather** or how **diseases spread** is fairly harmless. Predicting whether a **convict will reoffend**, whether a **loan applicant will default**, or whether an **insurance customer will make expensive claims** is different: these **directly affect individual lives**.

Organisations naturally want to avoid risk: payment networks try to stop **fraudulent transactions**, banks try to dodge **bad loans**, airlines want no **hijackings**, companies want to avoid hiring **ineffective or untrustworthy** people. To them, a missed opportunity costs little, while a bad loan or bad hire costs a lot, so **when in doubt, they say no**.

But as algorithmic decisions spread, a person labelled risky by some algorithm (**rightly or wrongly**) may receive **many** of those "no"s. Being systematically excluded from **jobs, air travel, insurance, property rental, financial services** and other key parts of society is such a severe restriction of freedom that it's been called **"algorithmic prison"**. Where human rights are respected, criminal justice **presumes innocence** until guilt is proven; automated systems can exclude someone **systematically and arbitrarily**, with no proof of guilt and hardly any **chance to appeal**.

Bias and discrimination

Algorithmic decisions aren't inherently better or worse than human ones. People have biases even when trying not to, and discrimination can become **culturally institutionalised**. There's hope that data-based decisions could be **fairer** than subjective, instinctive human judgments and help people often **overlooked** by traditional systems.

But predictive analytics doesn't just automate a human's decision by coding explicit yes/no rules — it lets the **rules themselves be inferred from data**. Those learned patterns are **opaque**: we may see a correlation without knowing why. If the input has a **systematic bias**, the system will most likely **learn and amplify** it.

- Many countries' **anti-discrimination laws** forbid treating people differently based on **protected traits** — **ethnicity, age, gender, sexuality, disability, beliefs** and the like.
- Other data about a person may be analysed — but what if it **correlates** with protected traits? In **racially segregated neighbourhoods**, race can be predicted quite reliably from someone's **postal code**, or even from their **IP address**.
- So believing an algorithm can take **biased data** and produce **fair, impartial output** is absurd, yet proponents of data-driven decisions often seem to imply it. One satirical description of this attitude: machine learning as **"money laundering for bias"**.
- Predictive systems only **extrapolate from the past**; if the past was discriminatory, they **codify** it. A better future needs **moral imagination**, which only humans can supply. **Data and models ought to serve us, not rule us.**

Responsibility and accountability

Automated decisions raise **responsibility and accountability** questions. A human who errs can be held accountable, and the affected person can appeal. Algorithms err too — but who's accountable?

- Who is to blame when a **self-driving car** crashes?

- If an automated **credit scoring** system consistently disadvantages people of some **race or religion**, can they get redress?
- If your ML system's decision faces **judicial review**, can you explain to a judge how it decided?

Credit rating agencies are a long-standing case of deciding things about people from data. A bad score makes life hard, but at least it's normally based on **relevant facts about actual borrowing history**, and errors can be corrected (though agencies don't make that easy). **ML-based scoring** usually uses a **much wider range of inputs** and is **far more opaque**, so it's harder to see how a decision arose or whether someone is treated unfairly.

Traditional credit score	Predictive analytics
Asks about **your own** past behaviour	Asks **who resembles you**, and how such people behaved
Based on your own history	Implies **stereotyping**, e.g., by where you live (a close proxy for race and socioeconomic class)

What about people put in the **wrong bucket**? And if a decision is wrong because the **data is wrong**, recourse is almost impossible.

Much data is **statistical**: even if the overall probability distribution is right, **individual cases may be wrong**. Suppose your country's average life expectancy is **80 years**; you're not expected to die on your **80th birthday**; the average and distribution say little about one person's lifespan. Likewise, predictions are **probabilistic** and may be wrong for individuals.

Blind faith in data's supremacy for decisions is **not just delusional but dangerous**. As data-driven decisions spread, we'll need ways to make algorithms **transparent and accountable**, to **stop them entrenching existing biases**, and to **correct** the errors they will inevitably make.

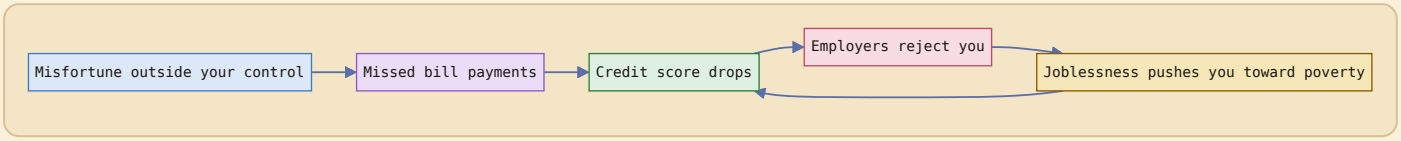
We also need to stop data from being turned against people, and instead unlock the **good** it can do. Analytics can reveal people's financial and social circumstances. That power could **target aid** at those most in need — or be used by **predatory businesses** to find vulnerable people and sell them risky products like **high-cost loans** and **worthless college degrees**.

Feedback loops

Even less far-reaching predictive systems like **recommendation systems** raise hard issues. When services get good at predicting what users want to see, they may show people **only opinions they already agree with**, creating **echo chambers** where **stereotypes, misinformation and polarisation** grow. Social media echo chambers are already affecting **election campaigns**.

When predictions affect lives, **self-reinforcing feedback loops** are especially harmful. The book's example — **employers using credit scores** to evaluate candidates:

Illustration (not a book figure): the credit-score feedback loop described in the book.



Imagine you're a capable employee with a healthy score, hit financial trouble through no fault of your own, miss payments, see your score fall, find it harder to get work, become poorer, and see your score fall further — a **downward spiral** driven by toxic assumptions **disguised as mathematical rigour and data**.

We can't always foresee such loops, but many consequences can be predicted by considering the **whole system** — the computer parts **and** the **humans** who interact with them — which is known as **systems thinking**. Ask how the analysis system responds to different behaviours, structures or

characteristics: does it **reinforce and amplify** existing differences (**rich get richer, poor get poorer**) or **combat injustice**? Even with good intentions, beware of **unintended consequences**.

Privacy and Tracking

Apart from automated decision-making, the very act of **collecting** data raises ethical issues. What's the relationship between the **collector** and the **people whose data is collected**?

- If a system stores only what a user **explicitly entered**, because they want it stored and processed a certain way, the system is **serving the user**: the user is the **customer**.
- If the user's behaviour is **recorded incidentally** while they do something else, things get murkier. The service no longer just follows the user's instructions; it develops **its own interests**, and these may **clash** with the user's.

Tracking behavioural data powers many useful user-facing features: which search results get clicked (improves ranking), "people who liked X also liked Y" recommendations, **A/B tests** and **user flow analysis** to improve the UI. These require some tracking, and **users benefit**.

But depending on the **business model**, tracking often goes further. If the service is funded by **advertising**, the **advertisers are the real customers**, and users' interests come second. Tracking gets more detailed, analyses reach further, and data is kept longer to build **detailed per-person profiles** for marketing.

The relationship now looks different: the user gets a **free service** and is **nudged to engage as much as possible**; the tracking primarily serves the **advertisers** who pay. The author says a more sinister word fits: **surveillance**.

Surveillance

Thought experiment: replace the word *data* with *surveillance* in common phrases and see if they still sound good. The book's example, paraphrased: "Our surveillance-driven company gathers real-time surveillance streams into a surveillance warehouse, where surveillance scientists use advanced analytics and surveillance processing to derive new insights."

The author admits this is unusually polemical for a book he jokingly retitles **"Designing Surveillance-Intensive Applications"**, but thinks strong words are needed:

- In trying to make software **"eat the world"**, we've built the **largest mass surveillance infrastructure** in history.
- Heading into an **Internet of Things**, we're approaching a world where every inhabited space has at least one **internet-connected microphone**: smartphones, smart TVs, voice-controlled assistants, baby monitors, even **children's toys** using cloud speech recognition. Many have **terrible security records**.
- Even the most **totalitarian, repressive regimes** could only dream of a microphone in every room and forcing everyone to carry a location-tracking device. Yet we seem to embrace this **voluntarily, even enthusiastically**. The only difference is that **corporations**, not government agencies, collect the data.

Not all data collection is surveillance, but viewing it that way clarifies our relationship with the collector. **Why do we accept corporate surveillance?**

- Perhaps you feel you have **"nothing to hide"** — meaning you fit existing power structures, aren't a marginalised minority, and needn't fear persecution. Not everyone is so lucky.
- Perhaps the purpose seems **benign**: not overt coercion, just better recommendations and more personalised marketing. But combined with predictive analytics, that line blurs.

Examples of it becoming less benign:

- **Car insurance premiums** tied to **tracking devices in cars**.
- **Health insurance coverage** depending on wearing a **fitness tracker**.

- When surveillance decides important life matters like **insurance or employment**, it looks less harmless.
- Analysis can reveal surprisingly **intrusive** things: a **smartwatch or fitness tracker's movement sensor** can infer what you're **typing (e.g., passwords)** with fairly good accuracy. And analysis algorithms will only improve.

Consent and freedom of choice

One could argue users **choose** the service, accept the **privacy policy and terms of service**, and therefore **consent**; and that they get valuable service in exchange, with tracking being necessary to provide it. Social networks, search engines and other free services are undoubtedly valuable — but the argument has problems:

1. **Users don't know** what data they feed in or how it's kept and processed; most privacy policies **obscure more than they explain**. Without understanding, there's **no meaningful consent**.
2. One user's data often reveals things about **other people** who aren't users and never agreed to anything.
3. The **derived datasets** discussed in this part of the book — combining the whole user base with behavioural tracking and external data — are precisely what users **can't meaningfully understand**.
4. Data is extracted through a **one-way process**, not a reciprocal relationship or fair exchange. There's **no dialogue** or negotiation over how much data is given for what service; the relationship is **asymmetric**, and the **service sets the terms**.

For someone who doesn't consent, the only alternative is **not using the service**. But that's not a free choice either: if a service is so widespread that most people consider it **necessary just to take part in social life**, opting out isn't reasonable — using it is **de facto mandatory**. In most Western social communities it's normal to carry a **smartphone**, use **Facebook** to socialise and **Google** to find information. With **network effects**, not using a service carries a **social cost**.

Declining a service over tracking is only possible for the **privileged few** who have the time and expertise to understand a privacy policy and can afford to lose the social or professional opportunities it offers. For the **less privileged**, there's no real choice: **surveillance becomes inescapable**.

Privacy and use of data

Some claim **"privacy is dead"** because some users post everything about their lives on social media, from mundane to deeply personal. The author says that's **false** and misunderstands privacy.

- **Privacy isn't keeping everything secret.** It's the **freedom to decide** which things you disclose, to whom, which you publish, and which you keep private.
- Privacy is a **decision right**: in every situation, each person picks their own point between full **secrecy** and full **transparency**. It's central to **freedom and autonomy**.

When data is extracted via surveillance, privacy rights aren't necessarily destroyed but **transferred to the data collector**. Companies are in effect asking people to **trust them** with the data, which shifts the decision about what to reveal from the individual to the **company**.

Companies then keep much of what surveillance reveals **secret**, because revealing it would seem **creepy** and hurt their business model (which depends on knowing more about people than competitors do). Intimate information surfaces only **indirectly**, e.g., as ad-targeting tools for specific groups (such as **people with a particular illness**).

Even if individuals can't be **re-identified** from the group an ad targets, they've **lost agency** over disclosing intimate facts like whether they have an illness. It's not the user deciding what to reveal to whom based on their preferences — it's the **company exercising the privacy right to maximise profit**.

- Many companies aim **not to be perceived as creepy** — managing **perceptions** rather than asking how intrusive their collection really is. Even perceptions are handled badly: something may be

factually correct yet **trigger painful memories** the user doesn't want brought up.

- With any data, expect that it may be **wrong, undesirable or inappropriate**, and build **mechanisms to handle those failures**. "Undesirable" and "inappropriate" need **human judgment**; algorithms are blind to them unless explicitly programmed to respect human needs. Engineers must be **humble** and plan for such failures.
- **Privacy settings** controlling what **other users** can see are a start at returning control. But the **service itself** still has **unrestricted access** and can use data however the privacy policy allows. Even if it promises not to **sell to third parties**, it typically reserves **unlimited rights to process and analyse internally**, often far beyond what users can see.

Moving privacy rights from individuals to companies **on this scale has never happened before**. Surveillance is old, but it used to be **costly and done by hand**, not **automated and scalable**. Relationships of trust (a patient and a doctor, a defendant and a lawyer) are old too, yet in those the use of data was tightly limited by **ethics, law and regulation**. Internet services make it easy to amass **huge amounts of sensitive data without meaningful consent** and use it at scale without users understanding.

Data as assets and power

Behavioural data, as a by-product of using a service, is sometimes called **"data exhaust"** — implying worthless waste, so analytics is a sort of recycling.

The author says look at it the **other way round**: if **targeted advertising** pays for the service, **behavioural data is the core asset**, and the app the user interacts with is merely a way to **lure users** into feeding ever more personal data into the surveillance machine. The human creativity and relationships expressed online get **cynically exploited** by the data extraction machine.

Evidence that personal data is a valuable asset:

- **Data brokers** — a **shady, secretive industry** buying, aggregating, analysing, inferring and reselling intrusive personal data, mostly for **marketing**.
- **Startups get valued by how many users** ("**eyeballs**") they have — which is to say, by how much they can surveil.

Because data is valuable, **many people want it**:

- **Companies** — that's why they collect it.
- **Governments** — via secret deals, coercion, legal compulsion, or plain theft.
- When a company goes **bankrupt**, its collected personal data is **sold as an asset**.
- Data is hard to secure, so **breaches** happen disconcertingly often.

Hence critics describe data as a **"toxic asset"**, or at minimum **"hazardous material"**. Even if we're confident we won't misuse it, every collection decision must weigh benefits against the danger of the data **reaching the wrong people**:

- **break-ins** by criminals or by hostile foreign spy agencies,
- **leaks by insiders**,
- the company falling under **unscrupulous management** that doesn't share our values,
- the country being **taken over by a regime** that forces us to hand over the data.

Consider not just today's politics but **all possible future governments**. Not every future government will respect **human rights and civil liberties**; as one quoted source puts it, deploying technology that might one day enable a **police state** is bad civic hygiene.

"Knowledge is power", and watching others while escaping being watched yourself is among the most significant kinds of power. That's why **totalitarian governments want surveillance**: it lets them control the population. Today's tech companies aren't openly seeking political power, but their accumulated data and knowledge hand them **considerable power over our lives**, much of it **hidden and outside public oversight**.

Remembering the Industrial Revolution

Data defines the **information age**. The internet, data storage, processing and software automation are reshaping the **global economy and society**, and will probably keep changing it radically for decades. The **Industrial Revolution** is a natural comparison.

Industrial Revolution	
Brought	Major technological and agricultural advances, sustained economic growth, much better long-run **living standards**
Also brought	Dreadful **air pollution** (smoke, chemical processes) and **water pollution** (industrial and human waste); factory owners in splendour while city workers endured bad housing and long, harsh working days; common **child labour**, including dangerous, poorly paid work in **mines**
Eventually fixed by (after a long time)	**Environmental protection** regulations, **workplace safety** protocols, **outlawing child labour**, **food health inspections**
Cost/benefit	Business costs rose when factories couldn't dump waste in rivers, sell tainted food or exploit workers — but society benefited hugely, and few would go back

Likewise the information age has a dark side to confront, and the author believes **data collection and use** is one such problem. He quotes **Bruce Schneier**, whose point in brief: *data is the information age's pollution problem, and protecting privacy is its environmental challenge*. Computers constantly generate information that lingers; how we contain and dispose of it is central to a healthy information economy. Just as we wonder how our ancestors ignored pollution, our grandchildren will judge how we handled data collection and misuse — and we should try to make them proud.

Legislation and self-regulation

Data protection laws might help preserve individual rights. The **1995 European Data Protection Directive** requires personal data to be gathered for **specified, explicit and legitimate purposes**, with no later processing **incompatible** with those purposes, and must be **adequate, relevant and not excessive** relative to those purposes.

But its effectiveness on today's internet is doubtful. These rules clash head-on with the **Big Data philosophy**: maximise collection, combine with other datasets, experiment and explore for new insights. Exploration means using data for **unforeseen purposes** — the reverse of the narrowly "specified and explicit" purposes the user agreed to (if consent is even meaningful). **Updated regulations are being developed.**

Companies collecting lots of personal data **oppose regulation** as burdensome and harmful to innovation. That's **partly justified**: sharing **medical data** has clear privacy risks but also opportunities — how many deaths might be prevented by better diagnostics or treatments from data analysis? **Over-regulation** could block such breakthroughs. Balancing opportunity and risk is hard.

The author's conclusion: the tech industry needs a **culture shift** about personal data.

- Stop treating users as **metrics to optimise**; remember they're **humans deserving respect, dignity and agency**.
- **Self-regulate** data collection and processing to build and keep the **trust** of people who depend on our software.
- **Teach end users** what happens to their data rather than hiding it from them.
- Let each person keep their **privacy** (control over their own data) instead of stealing it via surveillance.

Each person's right to control their own data resembles the wilderness of a **national park**: without explicit protection and care it will be destroyed — a **tragedy of the commons** leaving everyone worse off. **Ubiquitous surveillance is not inevitable**; we can still stop it.

How exactly is an open question. Starting points:

1. **Don't keep data forever**; **purge** it once it's no longer needed. This conflicts with **immutability** (Chapter 11, "Limitations of immutability"), but that can be solved.
2. A promising approach: enforce **access control through cryptographic protocols**, not merely by policy.
3. Overall, **culture and attitude changes** are needed.

Chapter summary

- **No single tool serves every use case**, so applications must **compose** several pieces of software. This is the **data integration** problem, solved by letting data changes flow between systems via **batch processing and event streams**.
- Designate some systems as **systems of record** and **derive** everything else (indexes, materialized views, ML models, statistical summaries) through transformations. Making derivations **asynchronous and loosely coupled** stops a problem in one area spreading to others, improving robustness and fault tolerance.
- Expressing dataflows as **transformations** helps **evolution**: to change a processing step (e.g., an index or cache structure), rerun the new code over the whole input to rederive the output; if something breaks, **fix the code and reprocess** to recover. Derived views allow **gradual, reversible migrations**. Unified batch/stream engines avoid the duplicated effort of the **lambda architecture**.
- Since databases already do this internally, the chapter reframes dataflow applications as **taking a database's components apart (unbundling)** and composing those loosely coupled parts (federation unifies reads; unbundling unifies writes). What's still missing is a high-level "shell" for composing systems.
- **Derived state can be observed** by subscribing to changes, and derived state can itself be observed downstream — all the way to **end-user devices**, giving UIs that update dynamically and work **offline**. Caches, indexes and views **move the line where write-path work stops and read-path work starts**; reads can even be modelled as events.
- For **correctness** under faults: strong **integrity** can be achieved scalably with asynchronous event processing, **end-to-end operation IDs** for idempotence, and **asynchronously checked constraints**. Clients can **wait** for the check or **proceed and risk apologising**. This is more scalable and robust than distributed transactions and matches how many businesses actually work.
- Separate **timeliness** (temporary violations) from **integrity** (permanent violations); integrity usually matters more. Dataflow + asynchronous constraint checks let us **avoid most coordination** while keeping integrity and performing well, even geo-distributed and under faults.
- **Audits** verify integrity and detect corruption: trust, but verify, end to end.
- **Ethics:** data can do good but also **significant harm** — justifying decisions that seriously affect lives and are hard to appeal, enabling **discrimination and exploitation**, normalising **surveillance**, exposing **intimate information**, risking **breaches**, and producing **unintended consequences**. Engineers are responsible for working towards a world where people are treated with **humanity and respect**.

Interview angle

1. How do you keep a database, a search index and a cache consistent? *30-second answer:* Don't let the application dual-write to all of them. Make one store the system of record, capture its changes in order (CDC or an event-sourcing log), and have each derived system consume that log and apply changes in the same order with idempotent, deterministic consumers. A single total order prevents stores from permanently diverging; replaying the log rebuilds any derived store.

- *Follow-up: Why not a distributed transaction (XA/2PC)?* — XA has poor fault tolerance and performance; one failing participant aborts everything and amplifies failures; and there's no standard protocol across heterogeneous systems built by different teams. A log with idempotent consumers is simpler and contains faults.
- *Follow-up: What do you give up?* — Timeliness: derived stores update asynchronously, so no read-your-writes by default. You can have clients wait for the change to appear if needed.

2. Lambda architecture vs a unified stream/batch engine? *30-second answer:* Lambda runs a batch layer (e.g., Hadoop MapReduce) and a speed layer (e.g., Storm) over the same immutable events; streaming gives fast approximate results, batch later corrects them. Problems: the same logic is maintained in two frameworks, outputs must be merged (hard for joins/sessionization), and full reprocessing is costly so batch becomes incremental and complex. Unified systems (e.g., Flink, Apache Beam on Flink/Cloud Dataflow) handle both with replayable logs, exactly-once semantics, and event-time windowing.

- *Follow-up: Why must windows use event time?* — When replaying history, processing time is just "now" and tells you nothing about when events happened.

3. How do you make a payment request exactly-once end to end? *30-second answer:* Generate a unique operation ID in the client (UUID or hash of form fields), pass it through every hop, and enforce a uniqueness constraint on it where the effect is recorded; a retry with the same ID is rejected. TCP dedup, DB transactions and stream-processor exactly-once each only cover their own hop — that's the end-to-end argument (Saltzer, Reed, Clark, 1984).

- *Follow-up: Does 2PC solve it?* — No; it handles coordinator–database reconnection, but not a user resubmitting a timed-out POST, which looks like a brand-new request.

4. How can you enforce uniqueness (e.g., usernames) in a log-based system? *30-second answer:* Partition the request log by hash of the username. A single-threaded stream processor per partition reads requests in order, checks a local table of taken names, and emits success or rejection to an output stream; the client waits on that stream. This is total order broadcast per partition — consensus is still needed, but it scales by adding partitions.

- *Follow-up: Multi-partition transfer without atomic commit?* — Log the whole request as one message (atomic single-object write), have a deterministic processor split it into debit and credit messages carrying the request ID, and have per-account processors dedupe by ID.

5. Timeliness vs integrity — which matters more? *30-second answer:* Timeliness is seeing up-to-date data; violations are temporary ("eventual consistency"). Integrity is no loss, duplication or contradiction; violations are permanent ("perpetual inconsistency") and need repair. Integrity usually matters far more — a card statement lagging a day is fine, money disappearing is not. Dataflow systems give up guaranteed timeliness but keep integrity via exactly-once processing and idempotence.

- *Follow-up: What are compensating transactions?* — Fix-ups after a loosely enforced constraint is violated (apologise and rebook, refund, overdraft fee). Whether the apology cost is acceptable is a business decision.

6. Why audit data if the database is ACID? *30-second answer:* Hardware corrupts data (bit-flips, rowhammer, disk rot, errors slipping past TCP checksums) and software has bugs — even MySQL (uniqueness) and PostgreSQL (write skew under serializable). ACID consistency assumes bug-free transactions. HDFS and S3 continually re-read and compare replicas. Event-sourced, deterministic dataflow makes end-to-end integrity checks possible (hash the log, rerun derivations).

Self-check

1. **Why is it dangerous for an application to write to both a database and a search index directly?** Concurrent conflicting writes can be applied in different orders by the two stores; neither is in

charge of ordering, so they may become permanently inconsistent. Fix: write once to the system of record and derive the index from its ordered change log.

2. **Name four situations where building a single total order of events becomes hard.** Throughput beyond one leader (partitioned logs), multiple geographically distributed datacenters with separate leaders, microservices each with their own state, and clients with local/offline state.
3. **In the unfriend-then-message example, what went wrong and what are possible remedies?** The notification service processed the message before the unfriend event because they lived in different systems, so the ex-partner was notified. Remedies: logical timestamps, logging an event of what the user saw (with an ID later events reference), or conflict resolution — though the last doesn't help with external side effects like notifications.
4. **What does `CREATE INDEX` have in common with adding a follower replica?** Both take a consistent snapshot, process it, then catch up on the backlog of changes since the snapshot, and keep up with ongoing changes — i.e., derive a new view by reprocessing existing data.
5. **Trade-off: when should you NOT unbundle?** When one product already meets all your needs. Many moving parts mean learning curves, configuration and operational quirks, and an integrated product may perform more predictably. Unbundling is about breadth, not beating a specialist on depth.
6. **Explain the write path and read path, and how a cache moves the boundary.** Write path = eager work when data arrives; read path = lazy work at query time. The derived dataset is where they meet. A cache/materialized view does more on the write path (precomputing) to save work on the read path; no index does the opposite.
7. **Scenario: a user on a flaky mobile network submits a payment and retries after an error. How do you prevent a double charge?** Include a client-generated operation ID (e.g., UUID in a hidden form field) with the request, pass it to the database, and enforce a unique constraint on it so the second insert fails and the transaction aborts.
8. **Why might an airline deliberately allow a "one person per seat" constraint to be violated?** Overbooking expects no-shows; excess demand is handled with compensation (refunds, upgrades). Apology processes are needed anyway (weather, strikes), so strict linearizable enforcement isn't worth the cost.
9. **What is "algorithmic prison", and why is predictive analytics different from a traditional credit score?** Systematic exclusion from jobs, travel, insurance, rentals and finance because an algorithm labelled you risky, often without proof or appeal. A credit score reflects your own borrowing history; predictive analytics judges you by similar people, which stereotypes and is opaque.
10. **According to the author, what does privacy actually mean, and why is "privacy is dead" wrong?** Privacy is the freedom to decide what to reveal to whom — a decision right — not total secrecy. People sharing things publicly are exercising that right; surveillance doesn't abolish privacy but transfers the right to the data collector.

Glossary

Book: Glossary, pp. 553–558

The book ends with a short glossary. Its author notes that the entries are deliberately brief: they capture the core idea of each term, not every subtlety, and point back to the main text for the details. The table below covers **every** glossary entry, rewritten in plain English, with links to the chapter notes where the term is explained properly. (The book's Index is not reproduced here.)

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12

> "Part II intro" is summarised at the start of the Chapter 5 notes, and "Part III intro" at the start of the Chapter 10 notes.

Term	Plain-English meaning	Where it's explained (chapter)
asynchronous	Carrying on without waiting for something to finish (for example, a message being sent to another node over the network), and making no assumption about how long that thing will take.	Ch 5 (synchronous vs asynchronous replication), Ch 8 (synchronous vs asynchronous networks; system models)
atomic	Has two meanings. (1) For concurrent operations: the operation seems to happen at one instant, so no other concurrent process can ever catch it half-done (compare *isolation*). (2) For transactions: a group of writes that is either committed as a whole or rolled back as a whole, even when faults happen.	Ch 7 (Atomicity), Ch 9 (atomic commit and 2PC)
backpressure	Making a data producer slow down because the receiver can't keep up. Another name for it is *flow control*.	Ch 11 (Messaging Systems)
batch process	A job that reads a fixed, usually large, input dataset and writes some other dataset as its output, leaving the input unchanged.	Ch 10
bounded	Having a known maximum limit or size. Used, for instance, about network delays and about datasets.	Ch 8 (timeouts and unbounded delays), Ch 11 (introduction)
Byzantine fault	A node misbehaving in an arbitrary way — for example, sending other nodes messages that contradict each other or are deliberately malicious.	Ch 8 (Byzantine Faults)
cache	A component that keeps recently used data so that later reads of the same data are quicker. It normally holds only part of the data, so anything missing must be fetched from a slower underlying store that has the full copy.	General term (the book cites no single section); see e.g. Ch 1, Ch 12
CAP theorem	A theoretical result that is widely misunderstood and, in the book's view, not useful in practice.	Ch 9 (The CAP theorem)
causality	The dependency that exists when one event "happens before" another: a later event replies to, builds on, or can only be understood in light of an earlier one.	Ch 5 (happens-before and concurrency), Ch 9 (Ordering and Causality)
consensus	The core distributed-computing problem of getting several nodes to agree on something, such as which node leads a database cluster. It is much harder than it first appears.	Ch 9 (Fault-Tolerant Consensus)
data warehouse	A database that combines data from several OLTP systems and prepares it for analytics.	Ch 3 (Data Warehousing)
declarative	Stating which properties the result should have, rather than the exact steps for producing it. For queries, a query optimizer receives the declarative query and chooses the best way to run it.	Ch 2 (Query Languages for Data)
denormalize	Deliberately adding some redundancy or duplication to a normalized dataset — usually as a cache or index — so reads are faster. A denormalized value is effectively a precomputed query result, much like a materialized view.	Ch 7 (single- and multi-object operations), Ch 11 (deriving several views from one event log)

Term	Plain-English meaning	Where it's explained (chapter)
derived data	A dataset produced from other data by a repeatable process that you could rerun if needed. It usually exists to make some specific kind of read fast. Indexes, caches and materialized views are examples.	Part III intro — Ch 10 overview; also Ch 12
deterministic	Describes a function that, given the same input, always returns the same output. It therefore can't rely on random numbers, the current time, network calls, or anything else unpredictable.	General term (no section cited); used in Ch 10, Ch 11, Ch 12
distributed	Running on several nodes that talk over a network. The hallmark is partial failure: one part may be broken while others work, and the software often can't tell exactly what has broken.	Ch 8 (Faults and Partial Failures)
durable	Stored in a way that you trust will survive various faults without being lost.	Ch 7 (Durability)
ETL	Extract–Transform–Load: pulling data out of a source database, reshaping it to suit analytic queries, and loading it into a data warehouse or batch processing system.	Ch 3 (Data Warehousing)
failover	In a single-leader system, handing the leader role over from one node to another.	Ch 5 (Handling Node Outages)
fault-tolerant	Able to recover by itself when something breaks, such as a machine crashing or a network link going down.	Ch 1 (Reliability)
flow control	Same thing as *backpressure*.	Ch 11
follower	A replica that never takes writes directly from clients; it only applies the data changes it receives from a leader. Also called a secondary, slave, read replica or hot standby.	Ch 5 (Leaders and Followers)
full-text search	Searching text for arbitrary keywords, often with extras like matching words that are spelled similarly, or synonyms. A full-text index is a type of secondary index that supports such searches.	Ch 3 (full-text search and fuzzy indexes)
graph	A data structure made of vertices (things you can refer to; also called nodes or entities) and edges (links from one vertex to another; also called relationships or arcs).	Ch 2 (Graph-Like Data Models)
hash	A function that converts an input into a number that looks random. The same input always yields the same number. Different inputs almost always yield different numbers, but occasionally two inputs give the same output — a *collision*.	Ch 6 (Partitioning by Hash of Key)
idempotent	An operation that is safe to retry: running it several times has the same effect as running it once.	Ch 11 (Idempotence)
index	A data structure that lets you quickly find all records having a given value in a given field.	Ch 3 (Data Structures That Power Your Database)

Term	Plain-English meaning	Where it's explained (chapter)
isolation	For transactions: how much transactions running at the same time can interfere with one another. Serializable isolation is the strongest guarantee, but weaker levels are also used.	Ch 7 (Isolation)
join	Combining records that share something. Most often, one record refers to another (via a foreign key, a document reference, or a graph edge) and the query must fetch the record being referred to.	Ch 2 (many-to-one and many-to-many relationships), Ch 10 (reduce-side joins and grouping)
leader	When data or a service is replicated over several nodes, the one replica designated as allowed to make changes. It may be chosen by an election protocol or appointed by an administrator. Also called primary or master.	Ch 5 (Leaders and Followers)
linearizable	Acting as though the system held just one copy of the data, changed only by atomic operations.	Ch 9 (Linearizability)
locality	A performance technique: storing pieces of data together when they are often needed together.	Ch 2 (data locality for queries)
lock	A mechanism guaranteeing that only one thread, node or transaction can use something at a time; anyone else wanting it has to wait until it is released.	Ch 7 (Two-Phase Locking), Ch 8 (the leader and the lock)
log	A file you can only append to, used to store data. Examples: a write-ahead log makes a storage engine crash-safe; a log-structured storage engine uses logs as its main storage format; a replication log copies writes from leader to followers; an event log can represent a data stream.	Ch 3 (B-tree reliability; SSTables and LSM-Trees), Ch 5 (Leaders and Followers), Ch 11 (Partitioned Logs)
materialize	To compute something eagerly and store the result, instead of calculating it only when someone asks.	Ch 3 (data cubes and materialized views), Ch 10 (materialization of intermediate state)
node	One running instance of some software on a computer, which communicates with other nodes over a network to get a job done.	General term (no section cited); used throughout Part II (Ch 5–Ch 9)
normalized	Organised so that nothing is stored redundantly. When a fact changes in a normalized database, you update it in exactly one place instead of in many copies.	Ch 2 (many-to-one and many-to-many relationships)
OLAP	Online analytic processing: a workload dominated by aggregations (count, sum, average, etc.) across very many records.	Ch 3 (Transaction Processing or Analytics?)
OLTP	Online transaction processing: a workload of quick queries that each read or write a few records, typically looked up by key.	Ch 3 (Transaction Processing or Analytics?)
partitioning	Breaking a dataset or computation that is too big for one machine into smaller pieces spread across many machines. Also called sharding.	Ch 6
percentile	A way to describe how values are distributed by counting how many fall above or below a threshold. E.g., the 95th percentile response time over some period is the time *t* such that 95% of requests in	Ch 1 (Describing Performance)

Term	Plain-English meaning	Where it's explained (chapter)
	that period finished faster than *t* and 5% took longer.	
primary key	A value (usually a number or string) that uniquely identifies one record. In many applications the system generates it when the record is created (e.g., sequentially or at random), rather than a user choosing it. Contrast with *secondary index*.	General term (no section cited); see Ch 3
quorum	The minimum number of nodes that must vote for an operation before it counts as successful.	Ch 5 (quorums for reading and writing)
rebalance	Moving data or services between nodes so the load is spread evenly.	Ch 6 (Rebalancing Partitions)
replication	Keeping copies of the same data on several nodes (replicas) so it is still reachable when a node can't be reached.	Ch 5
schema	A description of how some data is structured, including its fields and their types. Data can be checked against a schema at different points in its life, and a schema may change over time.	Ch 2 (schema flexibility in the document model), Ch 4
secondary index	An extra data structure maintained next to the main data store that lets you efficiently find records matching some condition.	Ch 3 (Other Indexing Structures), Ch 6 (Partitioning and Secondary Indexes)
serializable	A guarantee that transactions running concurrently produce the same outcome as if they had run one after another in some order.	Ch 7 (Serializability)
shared-nothing	An architecture of independent nodes, each with its own CPU, memory and disks, linked by an ordinary network — as opposed to shared-memory or shared-disk designs.	Part II intro — Ch 5 overview
skew	Two meanings. (1) Uneven load across partitions: some get lots of data or requests while others get little; also called hot spots. (2) A timing anomaly that makes events appear in an unexpected, out-of-sequence order — see read skew, write skew and clock skew.	(1) Ch 6 (skewed workloads and hot spots), Ch 10 (handling skew); (2) Ch 7 (read skew in snapshot isolation; write skew and phantoms), Ch 8 (clock skew in timestamps for ordering events)
split brain	Two nodes both believing, at the same moment, that they are the leader — which can break the system's guarantees.	Ch 5 (Handling Node Outages), Ch 8 (the truth is defined by the majority)
stored procedure	Packaging a transaction's logic so it runs entirely on the database server, with no back-and-forth to the client during the transaction.	Ch 7 (Actual Serial Execution)
stream process	A computation that keeps running forever, reading an endless stream of events and producing some output from them.	Ch 11
synchronous	The opposite of *asynchronous*: waiting for the thing to complete.	Ch 5, Ch 8

Term	Plain-English meaning	Where it's explained (chapter)
system of record	The system that holds the main, authoritative version of some data — the "source of truth". Changes go here first, and other datasets can be derived from it.	Part III intro — Ch 10 overview; also Ch 12
timeout	One of the simplest ways to detect a fault: noticing that no response arrived within some time. You can't tell, though, whether the remote node failed or the network had a problem.	Ch 8 (Timeouts and Unbounded Delays)
total order	An ordering of items (e.g., timestamps) in which every pair can be compared, so you can always tell which of the two comes first. If some pairs can't be compared, it is called a *partial order*.	Ch 9 (the causal order is not a total order)
transaction	Bundling several reads and writes into one logical unit so that error handling and concurrency become simpler to deal with.	Ch 7
two-phase commit (2PC)	An algorithm that makes sure several database nodes either all commit a transaction or all abort it.	Ch 9 (Atomic Commit and Two-Phase Commit)
two-phase locking (2PL)	An algorithm for serializable isolation in which a transaction takes a lock on every piece of data it reads or writes and keeps those locks until it finishes.	Ch 7 (Two-Phase Locking)
unbounded	Having no known maximum limit or size; the opposite of *bounded*.	Ch 8, Ch 11